



CuentaVoz

«Tu voz cuenta»

Manual de construcción del agente conversacional

Herramientas e instalación paso a paso · vistas de usuario · arquitectura y conexiones · código completo del prototipo (cerebro: Gemini / Google AI Studio) · diseño adaptable móvil/tableta/escritorio · ciberseguridad · panel gerencial nativo · plan de construcción por fases

Documento complementario a la Propuesta de Proyecto

Reto de Hotelería · Hackathon Colsubsidio x 30X

Equipo StockXperts: Luis Guillermo Nieto Patiño · Diana Carolina Argüello Casallas · Valentina Burbano Salazar · Luz Stephanie Puentes Morantes

Tabla de contenido

1. El ciclo del agente: paso a paso de cada turno	6
2. Casos de uso: todas las funcionalidades en un mapa	6
3. Paso a paso de una captura: las decisiones del agente	7
4. Herramientas tecnológicas e instalación desde cero: todo gratis, paso a paso	8
4.1 Las siete herramientas y para qué sirve cada una	9
4.2 Instalar las herramientas (Windows, unos 25 minutos)	9
4.3 Conseguir la llave de Gemini en Google AI Studio (gratis, 3 minutos)	10
4.4 Crear la carpeta del proyecto	10
4.5 Llenar los archivos y configurar las llaves	12
4.6 Levantar el proyecto por primera vez	12
4.7 Probar que el agente responde	13
4.8 Publicar en GitHub.....	13
4.9 Problemas frecuentes y cómo resolverlos	13
5. Vistas de la aplicación: cómo la ve el usuario	14
5.0a Cómo se conecta una vista con la siguiente	14
5.00 Identidad visual: marca Colsubsidio aplicada	16
5.0 Las vistas organizadas por el menú	17
5.1 Ingreso por perfil.....	18
5.2 Inicio: el punto de partida del día	19
5.3 Pedido por voz: del plato a los insumos	21
5.4 El pedido calculado: solo lo que de verdad falta	24
5.5 La receta que sustenta el cálculo	28
5.6 Conteo por voz (pantalla principal del auxiliar)	29
5.7 Desambiguación: elegir entre opciones.....	32
5.8 Alerta de anomalía: preguntar antes de guardar.....	34
5.9 Crear producto: pendiente de aprobación.....	36
5.10 Modo formulario: contingencia sin voz ni internet.....	37
5.11 Legalización del servicio: lo pedido contra lo usado	39
5.12 Cierre de la legalización y actualización del histórico.....	42
5.13 Tablero de bodegas en vivo.....	44
5.14 Detalle de una bodega	47
5.15 Consulta de artículo: dónde está y cuánto hay.....	50
5.16 Auditoría: comparativo y doble firma.....	54

5.17 Cierre con doble firma digital	57
5.18 Aprobaciones pendientes: cerrar el ciclo de lo creado.....	59
5.19 Bandeja de alertas.....	62
5.20 Reportes y consolidado.....	65
5.21 Análisis de consumo: lo pedido contra lo usado.....	69
5.22 Panel: resumen ejecutivo.....	71
5.23 Panel: bodegas y alertas.....	73
5.24 Ajustes: la aplicación se adapta a la operación.....	75
5.25 Gestión de usuarios y permisos	79
5.26 Registro de trazabilidad	83
5.27 Centro de ayuda y estado del sistema.....	85
5.27a Mensajes: bandeja de soporte en vivo	88
5.28 Mi perfil: datos personales y foto	91
5.29 Cerrar sesión con trabajo sin guardar.....	93
5b. El código del diseño adaptable.....	94
5b.0 Diseño adaptable: la misma aplicación en cualquier dispositivo.....	94
5b.1 Variables de diseño y tipografía.....	95
5b.2 Cuadrícula principal y puntos de quiebre	95
5b.3 Botones responsivos.....	96
5b.4 El micrófono: diferente en cada formato.....	96
5b.5 Hook de detección de dispositivo	97
5b.6 Cómo lo usa la pantalla de conteo	97
6. Cómo piensa el agente: intención y salida estructurada	99
6.1 Instrucciones del sistema (prompt del agente)	99
6.2 Esquema de salida (el JSON que el backend procesa).....	99
7. Las manos del agente: herramientas del backend	99
8. Arquitectura: cómo queda construido y cómo se conecta.....	101
8.1 Las cinco capas.....	101
8.2 Componentes y conexiones: quién habla con quién.....	101
8.3 Secuencia de un turno completo	102
9. Diagrama de clases y modelo de datos	103
10. Estados de la bodega y trabajo en paralelo	105
11. Conversaciones de ejemplo (guiones implementables).....	105
11.1 Conteo con conciliación de nombre y anomalía	105
11.2 «No entiendo esta tabla»: el agente explica el reporte	106

11.3 Consulta de datos con salida impresa	106
12. Código completo del prototipo	106
12.1 Dependencias y contenedores.....	107
12.2 Variables de entorno	108
12.3 Orquestación con Docker (uso local).....	108
12.4 Despliegue en producción: Render	109
12.5 Conexión a la base de datos	110
12.6 Las tablas (el diagrama de clases hecho código).....	111
12.7 Hora de Bogotá para toda la aplicación.....	115
12.8 Carga del extracto real de Colsubsidio.....	115
12.9 Servicio de conciliación (la sugerencia oficial de Colsubsidio)	118
12.10 Servicio de validación (incluye el mini reto de los negativos)	121
12.11 Servicio de recetas (pedido y legalización).....	122
12.12 Servicio de analítica (panel gerencial y reportes)	125
12.13 Huella del dispositivo (WebAuthn).....	127
12.14 Intérprete local (respaldo sin Gemini)	129
12.15 Reportes y almacenamiento de archivos	131
12.16 El cerebro: conexión con Gemini (Google AI Studio).....	133
12.17 El orquestador: donde el pensar se vuelve acción	138
12.18 Identidad y seguridad	149
12.19 La API completa: endpoints y tablero en vivo	150
12.20 Pruebas automáticas.....	223
12.21 Frontend: el puente con la API.....	224
12.22 Frontend: voz, huella e intérprete local.....	227
12.23 Frontend: piezas de layout compartidas.....	235
12.24 Frontend: estilo y diseño responsivo (PWA).....	244
12.25 Frontend: Ingreso e Inicio (vistas 5.1 y 5.2).....	251
12.26 Frontend: Pedido (vistas 5.3 a 5.5).....	256
12.27 Frontend: Legalización (vistas 5.11 y 5.12).....	268
12.28 Frontend: Conteo — la vista principal del auxiliar (vistas 5.6 a 5.10)	273
12.29 Frontend: Bodegas — tablero, detalle y consulta (vistas 5.13 a 5.15)	290
12.30 Frontend: Auditoría — comparativo, doble firma, aprobaciones y alertas (vistas 5.16 a 5.19) .	304
12.31 Frontend: Reportes y Panel (vistas 5.20 a 5.23).....	318
12.32 Frontend: Ajustes — catálogo, recetas, usuarios y trazabilidad (vistas 5.24 a 5.26)	332
12.33 Frontend: Ayuda y Mensajes (vistas 5.27 y 5.27a)	349

12.34 Frontend: Mi perfil y Cerrar sesión (vistas 5.28 y 5.29)	359
12.35 Frontend: App.jsx, el enrutador que une las treinta vistas	365
13. Panel gerencial: el dashboard nativo (sin Power BI).....	367
13.1 Las métricas: backend/servicios/analitica.py	368
13.2 Los dos endpoints del panel	368
13.3 Gráficas sin librerías: donas y barras con CSS puro	368
14. Ciberseguridad: el blindaje de CuentaVoz.....	369
14.1 Defensa en profundidad.....	369
14.2 Identidad: contraseñas cifradas, token firmado y perfiles.....	369
14.3 Blindaje de la API: límites, CORS cerrado y cabeceras.....	369
14.4 Configuración segura: secretos, contenedores, HTTPS y respaldos.....	370
14.5 Mapa de amenazas: cada ataque con su escudo	370
14.6 Datos personales y pruebas de seguridad.....	371
15. Plan de construcción paso a paso (para que todo funcione)	371
16. Matriz de cumplimiento: ninguna funcionalidad se queda en el papel	373

1. El ciclo del agente: paso a paso de cada turno

Cada vez que la persona habla ocurre un «turno», y todos los turnos —contar, corregir, preguntar, pedir ayuda— recorren el mismo ciclo de cinco pasos:

EL CICLO DEL AGENTE: QUÉ PASA EN CADA TURNO (objetivo: menos de 2 segundos)

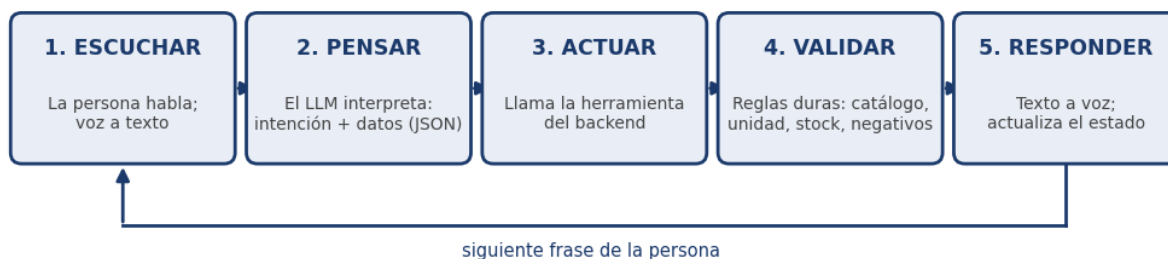


Figura 1. El ciclo del agente en cada turno de conversación.

1. **Escuchar.** La tableta captura el audio y el motor de voz a texto (Web Speech API en español de Colombia) lo convierte en una frase escrita. Este paso ocurre en el navegador de la tableta: no consume servidor ni requiere instalar nada.
2. **Pensar.** La frase viaja al backend y de allí al modelo de lenguaje —Gemini, con la llave gratuita de Google AI Studio— junto con el contexto de la sesión (bodega activa, último artículo, si hay algo pendiente de confirmar) y las reglas del sistema. El modelo devuelve la intención y los datos en un JSON fijo (sección 6).
3. **Actuar.** El orquestador ejecuta la herramienta que corresponde a esa intención: buscar el artículo, validar el conteo, consultar el stock, generar un reporte (sección 7). El modelo nunca toca los datos directamente.
4. **Validar.** La herramienta aplica las reglas duras en código determinista: ¿el artículo existe?, ¿la unidad coincide con el catálogo?, ¿la cantidad es posible (no negativa)?, ¿se desvía del stock esperado? Devuelve ok, o una alerta con el motivo exacto.
5. **Responder.** El agente arma la respuesta (confirmación, pregunta, explicación o alerta), la tableta la reproduce en voz y la muestra en pantalla, y el estado de la sesión queda actualizado para la siguiente frase.

La regla que sostiene todo el diseño: el agente propone, el backend dispone. El modelo de lenguaje interpreta y conversa, pero solo las herramientas del backend tocan los datos, y todas las validaciones críticas son código determinista que se comporta igual siempre. Por eso el sistema es inteligente y a la vez confiable — y por eso este manual puede darle a cada integrante una pieza clara que construir.

2. Casos de uso: todas las funcionalidades en un mapa

Antes de construir hay que ver completo lo que se promete. Este diagrama reúne, en una sola vista, todas las funcionalidades de la propuesta separadas por actor: el auxiliar de inventarios hace el conteo inicial;

el administrador de bodega audita, aprueba y cierra; y el extracto de My Inventory es la fuente de datos que alimenta las validaciones y recibe el archivo de carga:

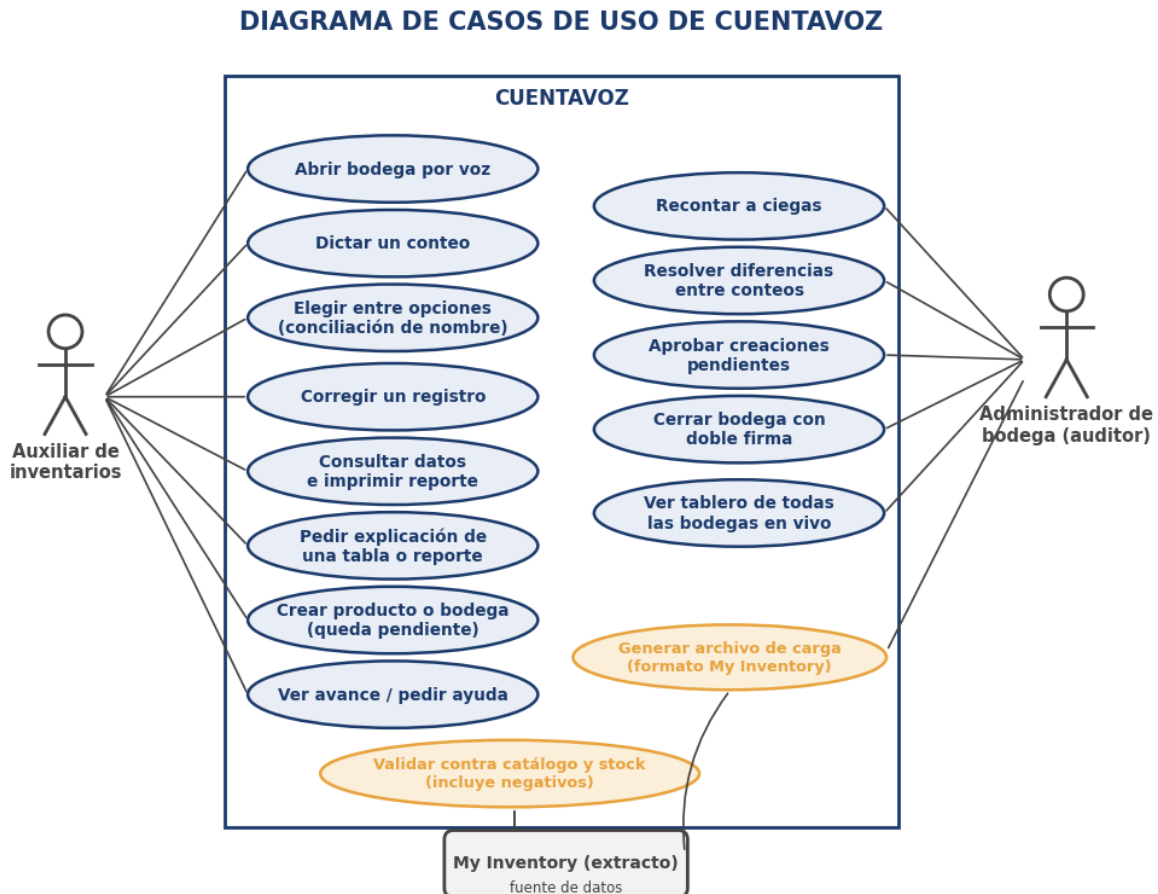


Figura 2. Casos de uso de CuentaVoz por actor; los óvalos amarillos son compartidos con la fuente de datos.

Cada óvalo reaparece más adelante tres veces: como pantalla en la sección 5, como código en la sección 12 y como prueba de cumplimiento en la matriz de la sección 16. Así ninguna funcionalidad se queda en el papel.

3. Paso a paso de una captura: las decisiones del agente

El momento más delicado es capturar un conteo. El diagrama muestra las decisiones del agente desde la frase dictada hasta el registro guardado, con los tres desvíos posibles: confianza media (elegir entre opciones), confianza baja (repetir o crear) y anomalía (preguntar antes de guardar):

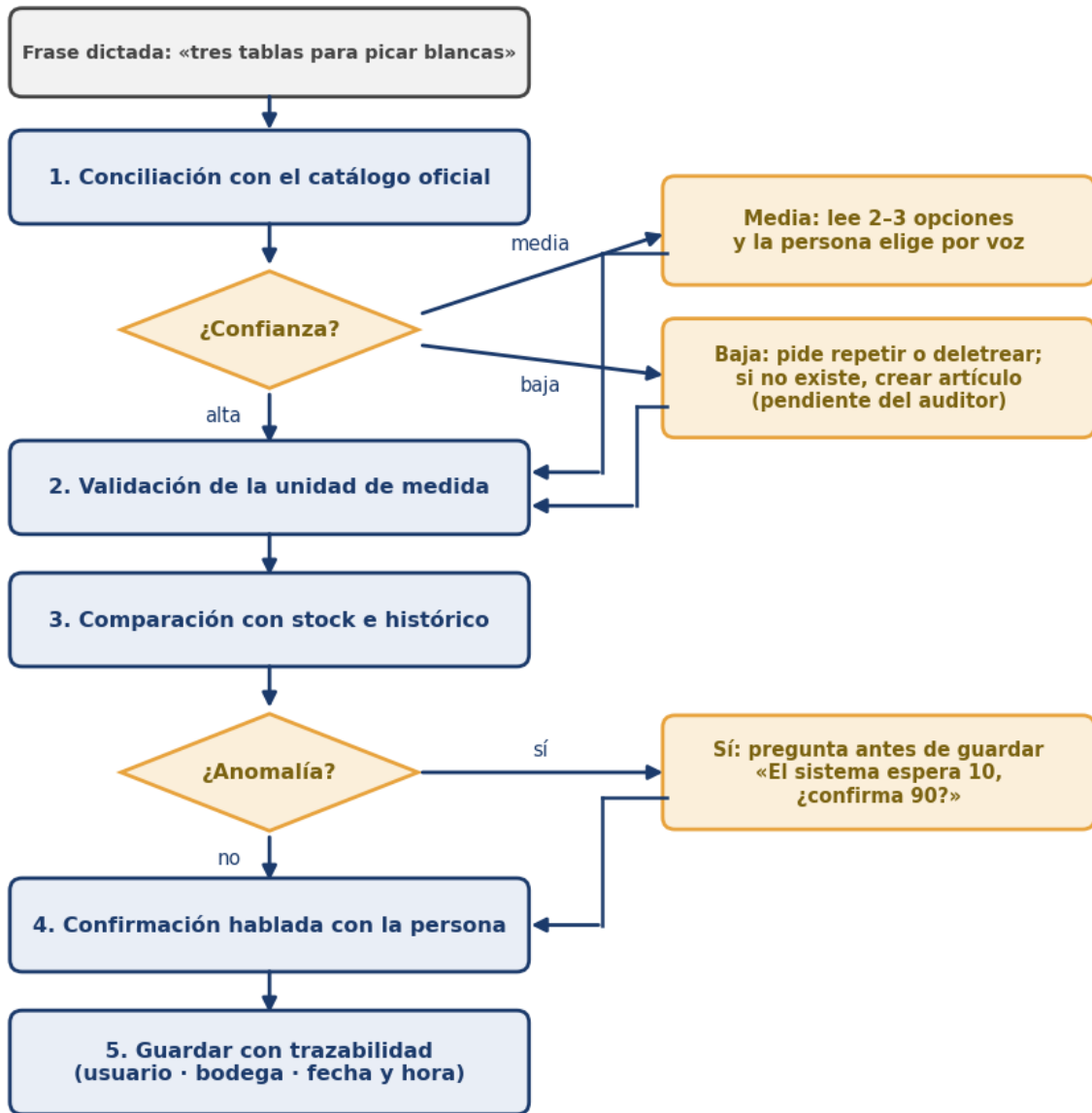
PASO A PASO DE UNA CAPTURA: LAS DECISIONES DEL AGENTE

Figura 3. Decisiones del agente en una captura; toda rama termina en una salida hablada y nada se guarda sin validación.

4. Herramientas tecnológicas e instalación desde cero: todo gratis, paso a paso

Esta sección está escrita para alguien que nunca ha instalado un entorno de desarrollo. Cada paso dice qué descargar, dónde, cuánto pesa, qué escribir y cómo verificar que quedó bien antes de seguir. Todas las herramientas son gratuitas: el costo del prototipo es cero.

4.1 Las siete herramientas y para qué sirve cada una

Herramienta	Para qué sirve en CuentaVoz	Costo	Dónde se descarga
Visual Studio Code	El editor donde se escribe y se organiza todo el código	Gratis	code.visualstudio.com
Node.js (versión LTS)	Motor que ejecuta el frontend en React durante el desarrollo	Gratis	nodejs.org
Python 3.12	Lenguaje del backend, el agente y los servicios de validación	Gratis	python.org/downloads
Docker Desktop	Levanta la base de datos y el backend con un solo comando, sin instalar PostgreSQL a mano	Gratis (uso personal y empresas pequeñas)	docker.com/products/docker-desktop
Git	Guarda el historial del código y lo sube a GitHub (entregable del reto)	Gratis	git-scm.com
Google AI Studio	Entrega la llave de Gemini, el cerebro del agente	Gratis (cuota diaria)	aistudio.google.com

Nota sobre PostgreSQL: no aparece en la lista porque no se instala. Docker lo descarga y lo ejecuta solo cuando levantamos el proyecto, y así nadie tiene que configurar una base de datos a mano.

4.2 Instalar las herramientas (Windows, unos 25 minutos)

Paso 1 — Visual Studio Code. Entre a code.visualstudio.com y pulse el botón azul de descarga para Windows. Ejecute el archivo descargado y acepte todo, pero marque la casilla «Agregar al PATH» cuando aparezca: eso permite abrir proyectos escribiendo code en la terminal.

Paso 2 — Node.js. Entre a nodejs.org y descargue la versión que diga LTS (soporte a largo plazo), nunca la «Current». Ejecute el instalador y acepte las opciones por defecto.

Paso 3 — Python. Entre a python.org/downloads y descargue Python 3.12. Al abrir el instalador, antes de pulsar «Install Now», marque abajo la casilla «Add python.exe to PATH». Si no la marca, la terminal no reconocerá el comando python y tendrá que reinstalar.

Paso 4 — Git. Entre a git-scm.com y descargue la versión para Windows. Acepte todas las opciones por defecto del instalador.

Paso 5 — Docker Desktop. Entre a docker.com/products/docker-desktop y descargue la versión para Windows. Al instalar pedirá activar WSL 2 (el subsistema de Linux de Windows): acepte. Reinicie el computador cuando lo pida y abra Docker Desktop; la primera vez tarda un par de minutos en arrancar. Debe quedar diciendo «Engine running» en la esquina inferior izquierda.

Verificación obligatoria. Abra PowerShell (tecla Windows, escriba «powershell», Enter) y escriba estos cuatro comandos, uno por uno. Cada uno debe responder con un número de versión; si alguno dice «no se reconoce», esa herramienta no quedó instalada o falta el PATH:

```
node -v          # debe responder algo como v20.11.0
python --version # debe responder Python 3.12.x
git --version    # debe responder git version 2.4x
docker --version # debe responder Docker version 27.x
```

4.3 Conseguir la llave de Gemini en Google AI Studio (gratis, 3 minutos)

1. Entre a aistudio.google.com e inicie sesión con la cuenta de Google del equipo (sirve cualquier cuenta de Gmail).
2. Acepte los términos de uso que aparecen la primera vez.
3. En el menú de la izquierda, pulse «Get API key» (Obtener clave de API).
4. Pulse «Create API key» y elija «Create API key in new project».
5. Copie la clave que aparece — es una cadena larga que empieza por Alza. Guárdela en un bloc de notas: la va a pegar en el archivo `.env` del paso 14.5.
6. No comparta esa clave ni la suba a GitHub. Por eso el archivo `.env` está en el `.gitignore` (sección 14.4).

¿Cuánto cuesta? El plan gratuito de AI Studio permite una cantidad generosa de consultas por día, más que suficiente para desarrollar y demostrar el prototipo durante el hackathon. No pide tarjeta de crédito.

4.4 Crear la carpeta del proyecto

CuentaVoz ya es un repositorio real y desplegado (no hace falta crear carpetas ni archivos vacíos a mano): se clona completo con Git. Abra una terminal en la carpeta donde quiera guardar el proyecto y escriba:

```
git clone https://github.com/lnieto044/cuentavoz.git
cd cuentavoz
code .          # abre el proyecto ya completo en Visual Studio Code
```

El clon ya trae adentro el extracto real de Colsubsidio (`data/BODEGAS_Y_STOCK.xlsx`) y los pasos verificados de arranque en `LEEME_PRIMERO.md`. Así queda la estructura completa del proyecto — cada archivo con su función y, en verde, el componente de cada vista:

ESTRUCTURA DE CARPETAS Y ARCHIVOS — CUENTAVOZ

Actualizada a la versión desplegada en Render: WebAuthn, PWA sin conexión, recetas administrables, Mensajes, hora de Bogotá y reportes en base de datos
 ● en verde, el componente de cada una de las catorce vistas del menú

cuentavoz/	carpeta raíz del proyecto (repositorio real en GitHub)
backend/	el cerebro y las reglas (Python + FastAPI)
☐ main.py	la API: ~90 endpoints REST + el WebSocket del tablero
☐ bd.py	conexión SQLAlchemy — SQLite en local, PostgreSQL en Render
☐ modelos.py	las 19 tablas del sistema (ver sección 9)
☐ horario.py	hora de Bogotá para toda la app (Render corre en UTC)
☐ seguridad.py	bcrypt, JWT con versión de sesión, permisos por perfil
☐ reportes.py	arma los .xlsx/.csv (consolidado, diferencias, trazabilidad...)
☐ requirements.txt	librerías a instalar
☐ Dockerfile	receta del contenedor del backend (uso local)
agente/	el agente conversacional
☐ cerebro.py	habla con Gemini; cae al intérprete local sin llave o si falla
☐ orquestador.py	procesar_turno(): decide qué hacer con lo que dijo la persona
servicios/	las reglas duras, sin IA
☐ conciliacion.py	«tabla para picar blanca» → código oficial (fuzzy match)
☐ validacion.py	negativos, unidades equivocadas, desviaciones (el 9 vs. 90)
☐ recetas.py	pedido por receta y comparación de la legalización
☐ analitica.py	métricas del panel gerencial y de reportes, en vivo
☐ huella.py	registro/verificación de huella del dispositivo (WebAuthn)
☐ archivos.py	guarda los reportes generados en la base — el disco de Render es efímero
☐ interprete.py	respaldo sin Gemini: números en palabras + palabras clave
tests/	12 archivos de pruebas automáticas (ver sección 12)
frontend/	lo que ve la persona (React + Vite + PWA)
☐ Dockerfile	receta del contenedor del frontend (uso local)
src/	código de la aplicación
☐ main.jsx	arranca React. No tiene lógica
☐ App.jsx	enrutador, menú lateral y sesión
☐ api.js	pedir(): único punto por donde pasa cada llamada al backend
☐ voz.js	escuchar() (Web Speech API) y hablar() (voz neuronal de Gemini)
☐ webauthn.js	huella del dispositivo, del lado del navegador
☐ interpreteLocal.js	puerto a JS de servicios/interprete.py, para el modo sin conexión
☐ confirmacionVoz.js	confirmaciones/negaciones dichas en voz alta ("sí", "cancele")
☐ correoAdmin.js	envío real de correo (EmailJS) para Ayuda y Mensajes
☐ AsistenteVoz.jsx	micrófono reutilizable para pantallas sin conversación
☐ index.css	paleta, tipografía y diseño responsivo (móvil/tableta/escritorio)
☐ Marco.jsx / BarraLateral.jsx / Dialogo.jsx / Iconos.jsx	layout compartido por todas las vistas
vistas/	un componente por cada pantalla del menú
☒ Ingreso.jsx	usuario/PIN o huella — perfil detectado solo
☒ Inicio.jsx	resumen del día según el perfil
☒ Pedido.jsx	pedido por voz a partir de una receta ("hoy preparamos 50 ajíacos")
☒ Conteo.jsx	conteo por voz + desambiguación + crear producto + modo sin conexión
☒ Legalizacion.jsx	lo pedido contra lo usado, con sobrante y merma
☒ Bodegas.jsx	tablero en vivo (WebSocket) + detalle + consulta de artículo
☒ Auditoria.jsx	recuento ciego, comparativo, doble firma, aprobaciones, alertas
☒ Reportes.jsx	consolidados exportables y análisis de consumo
☒ Panel.jsx	panel gerencial — solo perfil auditor
☒ Ajustes.jsx	catálogo, recetas, usuarios y trazabilidad
☒ Ayuda.jsx	preguntas frecuentes, estado del sistema, comandos de voz
☒ Mensajes.jsx	bandeja de soporte en vivo (nueva desde el reto original)
☒ MiPerfil.jsx	datos personales, foto, PIN, huella y preferencias de voz
☒ CerrarSesion.jsx	salida segura, con confirmación
data/	los datos reales de Colsubsidio
☐ BODEGAS_Y_STOCK.xlsx	el extracto real que entregó Colsubsidio
☐ cargar_excel.py	carga el Excel a la base (local o la de Render)
docs/capturas/	capturas para el README y este manual
☐ .env / .env.ejemplo	llaves y variables (Gemini, JWT, WebAuthn, Power BI) — NO se sube a GitHub
☐ .gitignore	lo que Git debe ignorar
☐ docker-compose.yml	levanta todo con un solo comando (uso local)
☐ render.yaml	blueprint de despliegue: base de datos + backend + sitio estático
☐ README.md	presentación del proyecto, con capturas y credenciales de prueba
☐ ARQUITECTURA.md	mapa del código, por qué cada librería, modelo de datos
☐ DESPLIEGUE.md	paso a paso para desplegar en Render
☐ LEEME_PRIMERO.md	guía rápida para arrancar el proyecto en local

CuentaVoz · Equipo StockXperts

14 vistas de menú + Mensajes · desplegado en Render (Static Site + Web Service + PostgreSQL)

Figura 4. Estructura de carpetas y archivos del proyecto (actualizada): en verde, el componente de cada una de las catorce vistas de menú del capítulo 5.

4.5 Llenar los archivos y configurar las llaves

En Visual Studio Code, abra el proyecto ya clonado. La sección 12 trae el código completo y actual de cada archivo, en el mismo orden en que se explica en este manual, así que sirve como mapa para ubicar cualquier pieza. Lo único que hay que personalizar es `.env` (cópelo desde `.env.ejemplo`), donde va la llave de Gemini que obtuvo en 4.3:

```
GOOGLE_API_KEY=Aiza...pegue_aqui_su_llave # opcional: sin ella, respaldo al interprete local
MODELO=gemini-flash-latest

DB_URL=postgresql://cuentavoz:cambieme@db:5432/cuentavoz

IDIOMA_VOZ=es-CO
UMBRAL_ANOMALIA=0.5

SECRETO_JWT=escriba_aqui_una_frase_larga_y_dificil
MINUTOS_TOKEN=480
ORIGEN_PERMITIDO=http://localhost:5173

# ingreso con huella (WebAuthn) -- RP_ID sin esquema/puerto, ORIGENES con esquema
WEBAUTHN_RP_ID=localhost
WEBAUTHN_RP_NAME=CuentaVoz
WEBAUTHN_ORIGENES=http://localhost:5173
```

Y en `.gitignore`, para que las llaves nunca salgan de su computador:

```
.env
.venv/
venv/
__pycache__/
*.pyc
node_modules/
dist/
**/reportes/*.xlsx
**/reportes/*.csv
*.db
.DS_Store
```

Extensiones recomendadas en VS Code (ícono de bloques en la barra izquierda, o Ctrl+Shift+X): Python y Pylance de Microsoft, Docker, ESLint y Prettier. Opcional pero muy útil: Thunder Client, para probar la API sin salir del editor.

4.6 Levantar el proyecto por primera vez

Con Docker Desktop abierto, en la terminal de VS Code (menú Terminal → Nueva terminal) escriba:

```
docker compose up --build
```

La primera vez tarda entre 3 y 5 minutos porque descarga PostgreSQL y las librerías. Cuando termine, verifique estas dos direcciones en el navegador:

- <http://localhost:8000/docs> — debe aparecer la documentación automática de la API con todos los endpoints. Si la ve, el backend y la base de datos están funcionando.

- <http://localhost:5173> — debe aparecer la pantalla de ingreso de CuentaVoz (vista 5.1).

Ahora cargue los datos reales de Colsubsidio. Abra una segunda terminal (el símbolo + en el panel de terminal, para no detener la primera) y escriba:

```
cd data
python cargar_excel.py
```

Debe imprimir el conteo de lo cargado: 54 bodegas, 1.040 artículos y 1.405 registros de stock. Si ese número aparece, el prototipo ya tiene el inventario real adentro.

4.7 Probar que el agente responde

Abra <http://localhost:5173> en Chrome o Edge (Firefox no soporta bien el reconocimiento de voz), ingrese con cualquier usuario y pruebe este guion:

Diga esto	Debe pasar esto	Si falla, revise
«Iniciar conteo en almacén de suministros»	El agente abre la bodega y dice cuántas referencias tiene	Que cargar_excel.py haya corrido bien
«Tres tablas para picar blancas»	Ofrece las dos opciones con su código	La llave de Gemini en el .env
«La FB»	Repite el nombre oficial y pregunta si confirma	El servicio de conciliación (12.7)
«Confirmo»	Guarda y anuncia el avance	La conexión con PostgreSQL
«Noventa cazuelas de dieciséis onzas»	Dispara la alerta de cantidad fuera de lo esperado	El umbral en el .env (0.5)

4.8 Publicar en GitHub

GitHub (es entregable del reto): el repositorio ya existe en github.com/Iniето044/cuentavoz. En VS Code, pulse el ícono de Control de código fuente en la barra izquierda (el tercero), escriba el mensaje del cambio, pulse «Confirmar» y después «Sincronizar cambios». Verifique en github.com que el archivo .env NO aparezca en la lista: si aparece, bórralo del repositorio y revise el .gitignore.

4.9 Problemas frecuentes y cómo resolverlos

Lo que ve en pantalla	Qué significa	Cómo se resuelve
«docker: command not found»	Docker Desktop no está instalado o no está abierto	Abra Docker Desktop y espere a que diga «Engine running»

Lo que ve en pantalla	Qué significa	Cómo se resuelve
«port is already allocated»	Otro programa está usando el puerto 8000 o el 5432	Cambie el puerto en docker-compose.yml (por ejemplo 8001:8000) o cierre el otro programa
«python no se reconoce»	Python quedó instalado sin agregarse al PATH	Reinstale Python marcando la casilla «Add python.exe to PATH»
El agente responde «no le entendí» siempre	La llave de Gemini está mal copiada o vencida	Genere una nueva en aistudio.google.com y péguela completa en el .env
El micrófono no hace nada	El navegador no tiene permiso o no es Chrome/Edge	Use Chrome o Edge y acepte el permiso de micrófono que pide el navegador
«relation does not exist»	Las tablas no se crearon antes de cargar el Excel	Levante primero con docker compose up y luego corra cargar_excel.py
La página queda en blanco	El frontend no encuentra la API	Revise que VITE_API_URL apunte a http://localhost:8000

Resumen del costo: cero pesos. Visual Studio Code, Node.js, Python, Git, Docker Desktop, PostgreSQL y la llave de Gemini en Google AI Studio son gratuitos para este uso. Lo único que se necesita es un computador con Windows 10 u 11, 8 GB de memoria y conexión a internet.

5. Vistas de la aplicación: cómo la ve el usuario

Catorce entradas de menú organizan las treinta vistas del prototipo, tal como las ve el personal de Colsubsidio. Cada vista se describe con sus elementos y con la conexión que usa por debajo, para que quien programe el frontend sepa exactamente qué construir y contra qué endpoint. El diseño busca tres cosas: botones grandes (se usa de pie, a veces con guantes), una sola acción principal por pantalla, y la voz siempre como camino más rápido — nunca como obligación.

Antes de entrar al menú: la puerta de la aplicación.

5.0a Cómo se conecta una vista con la siguiente

Una plataforma no es un catálogo de pantallas sueltas: es un recorrido. Este mapa muestra las treinta vistas agrupadas por la entrada de menú donde viven, y las flechas que las unen. Las azules son navegación por el menú; las doradas, transiciones que dispara el propio agente sin que nadie toque nada.

MAPA DE NAVEGACIÓN: CÓMO SE CONECTA CADA VISTA CON LA SIGUIENTE

Azul: navegación por el menú · Dorado: transiciones que dispara el propio agente

LOS TRES MOMENTOS DEL RETO

MENÚ · PEDIDOS

5.3 Pedido por voz

5.4 Pedido calculado

5.5 Receta

momento 1: pedir al almacén

MENÚ · CONTEO

5.6 Conteo por voz

5.7 Opciones

5.8 Alerta

5.9 Crear producto

5.10 Modo sin conexión

momento 2: toma física — PWA, funciona sin señal

MENÚ · LEGALIZACIÓN

5.11 Pedido vs. usado

5.12 Cierre e histórico

momento 3: cerrar servicio

CONTROL Y ANÁLISIS DE LA OPERACIÓN

MENÚ · BODEGAS

5.13 Tablero en vivo

5.14 Detalle de bodega

5.15 Consulta de artículo

MENÚ · AUDITORÍA

5.16 Comparativo

5.17 Doble firma

5.18 Aprobaciones

5.19 Bandeja de alertas

MENÚ · REPORTES

5.20 Consolidado

5.21 Análisis de consumo

MENÚ · PANEL

5.22 Resumen ejecutivo

5.23 Bodegas y alertas

ADMINISTRACIÓN Y CUENTA — accesibles desde cualquier vista

MENÚ · AJUSTES

5.24 Ajustes (recetas, catálogo)

5.25 Usuarios

5.26 Trazabilidad

MENÚ · AYUDA / MENSAJES

5.27 Ayuda

5.27a Mensajes (nueva)

MENÚ · MI PERFIL / SALIR

5.28 Mi perfil (huella, WebAuthn)

5.29 Cerrar sesión con aviso

→ vuelve a 5.1 Ingreso; lo sin confirmar queda como borrador

Figura 5. Mapa de navegación (actualizado): los tres momentos del reto arriba, el control y el análisis en el medio, y la administración —incluida Mensajes— abajo.

Las transiciones que no nacen del menú son las que hacen que la aplicación se sienta fluida. Estas son todas:

Desde	Qué la dispara	Hacia
5.2 Inicio	Los cuatro accesos rápidos del día	5.3 Pedido · 5.13 Tablero · 5.16 Auditoría · 5.20 Reportes
5.3 Pedido por voz	El agente reconoce el plato y las porciones	5.4 Pedido calculado
5.4 Pedido calculado	La persona pregunta de dónde salió la cantidad	5.5 Receta consultada
5.6 Conteo por voz	El agente encuentra dos candidatos parecidos	5.7 Elegir entre opciones
5.6 Conteo por voz	La cantidad se desvía de lo esperado	5.8 Alerta de anomalía
5.6 Conteo por voz	El artículo no existe en el catálogo	5.9 Crear producto
5.6 Conteo por voz	Se pierde la conexión o el agente no responde	5.10 Modo sin conexión
5.9 Crear producto	El registro queda pendiente de aprobación	5.18 Aprobaciones (del administrador)

Desde	Qué la dispara	Hacia
5.8 Alerta de anomalía	La alerta se confirma y queda abierta	5.19 Bandeja de alertas
5.11 Legalización	Se confirma el ajuste del servicio	5.12 Cierre e histórico
5.12 Cierre e histórico	El consumo real alimenta el modelo	5.21 Análisis · 5.22 Panel
5.13 Tablero de bodegas	Se toca la tarjeta de una bodega	5.14 Detalle de esa bodega
5.17 Doble firma	La bodega queda cerrada	5.13 Tablero (se actualiza en vivo)
Cualquier vista	El botón «Pedir ayuda» o la entrada del menú	5.27 Centro de ayuda
5.29 Cerrar sesión	Se confirma la salida	5.1 Ingreso (lo pendiente queda como borrador)

Una regla de coherencia que se cumple en las treinta vistas: la entrada del menú que aparece resaltada es siempre aquella donde vive la pantalla. Si la persona está creando un producto, el menú resalta Conteo, porque crear nace del conteo; si está cerrando la sesión, resalta Cerrar sesión. Nunca hay duda de dónde se está.

5.00 Identidad visual: marca Colsubsidio aplicada

Las pantallas cumplen el manual de marca de Colsubsidio. La aplicación usa la paleta institucional y lleva el logotipo en dos lugares: en color, arriba a la derecha de la barra superior, y en blanco al pie de la barra lateral, sobre el fondo azul profundo. CuentaVoz conserva su propio logotipo a la izquierda, en la disposición de co-marca habitual: la solución se identifica, la institución la respalda.

Color oficial	HEX	Pantone	Dónde se usa en la aplicación
Azul Colsubsidio	#0067B1	Pantone 2196 C	Color principal: barra superior, títulos, botones de acción y series de datos
Amarillo Colsubsidio	#FFD000	Pantone 109 C	Acento: línea de marca bajo la barra, alertas de atención y la serie destacada de las gráficas
Grafito	#575756	Cool Gray 11 C	Textos secundarios, etiquetas y notas al pie
Blanco	#FFFFFF	—	Fondo de tarjetas y texto sobre azul

Nota de aplicación: el amarillo institucional se reserva para llamar la atención, nunca para el fondo de bloques grandes de texto, y las alertas usan ese amarillo en lugar de rojo: el sistema pregunta, no regaña. La tipografía es Calibri en toda la aplicación y en este manual.

5.0 Las vistas organizadas por el menú

El reto plantea tres momentos manuales: pedir insumos al almacén, contar el inventario físico y legalizar el servicio. Este capítulo recorre los tres, y las vistas de administración que una plataforma empresarial necesita alrededor de ellos, en el mismo orden del menú lateral.

Entrada del menú	Momento del reto	Vistas de este capítulo
(antes de entrar)	—	5.1 Ingreso con usuario, PIN y perfil
Inicio	—	5.2 Resumen del día, accesos rápidos y actividad reciente
Pedidos	1. Pedido al almacén	5.3 Pedido por voz · 5.4 Pedido calculado por receta · 5.5 Consulta de la receta
Conteo	2. Toma física	5.6 Conteo por voz · 5.7 Elegir entre opciones · 5.8 Alerta de anomalía · 5.9 Crear producto · 5.10 Modo sin conexión
Legalización	3. Legalización	5.11 Lo pedido contra lo usado · 5.12 Cierre y actualización del histórico
Bodegas	2. Toma física	5.13 Tablero en vivo · 5.14 Detalle de una bodega · 5.15 Consulta de artículo
Auditoría	2. Toma física	5.16 Comparativo de diferencias · 5.17 Cierre con doble firma · 5.18 Aprobaciones pendientes · 5.19 Bandeja de alertas
Reportes	los tres	5.20 Consolidado para My Inventory · 5.21 Análisis de consumo (lo pedido vs. lo usado)
Panel	los tres	5.22 Resumen ejecutivo · 5.23 Bodegas y alertas
Ajustes	—	5.24 Parámetros del sistema · 5.25 Gestión de usuarios · 5.26 Registro de trazabilidad
Ayuda	—	5.27 Preguntas frecuentes, soporte, estado del sistema y comandos de voz
Mi perfil	—	5.28 Datos personales, foto, PIN y preferencias de voz
Cerrar sesión	—	5.29 Confirmación que avisa del trabajo sin guardar

5.1 Ingreso por perfil

Figura 6. Vista de ingreso, captura real de producción: usuario o código de empleado, PIN, y la opción de entrar con la huella del dispositivo (WebAuthn).

- **Qué tiene:** los dos perfiles del reto como botones grandes (auxiliar de inventarios y administrador de bodega), el campo de usuario y el ingreso con el PIN o la huella que la tableta ya trae.
- **Qué conecta:** crea la identidad que viaja en cada turno; el perfil decide qué pantallas y qué herramientas están disponibles (el auxiliar no puede cerrar bodegas; el auditor no ve el conteo 1 durante su recuento).

Lo primero que ve cualquier persona al entrar.

MENÚ · INICIO *lo primero que ve la persona al entrar*

5.2 Inicio: el punto de partida del día

The screenshot shows the 'Inicio' (Home) screen of the CuentaVoz application. The header indicates the date is Sunday, August 16, 2026, and the user is 'Diana'. A 'TOMA EN CURSO' (Taking in progress) status is shown next to the 'Colsubsidio' logo. The main content area is titled 'Buenas tardes, Diana. Esto es lo que hay para hoy.' (Good afternoon, Diana. This is what you have for today).

Below the greeting, there is a section for asking the agent a question, with a text input field containing '¿cuántas bodegas tengo?, lléveme a bodegas...' and a microphone icon. Below this, there are three summary cards:

- Bodegas asignadas a usted** (Bodegas assigned to you): 0 según su perfil (according to your profile).
- Referencias contadas hoy** (References counted today): 0 en sus sesiones de conteo (in your counting sessions).
- Alertas por revisar** (Alerts to review): 2 en toda la operación (in the entire operation).

Below these cards, there is a card for 'Su exactitud del mes' (Your accuracy this month) showing 88.2 % promedio de bodegas cerradas (average of closed warehouses).

The next section is titled '¿Qué desea hacer?' (What do you want to do?) and contains four action cards:

- Iniciar un conteo** (Start a count): dicte y CuentaVoz registra (dictate and CuentaVoz registers).
- Ver el tablero** (View the dashboard): estado de las bodegas (status of the warehouses).
- Continuar auditoría** (Continue audit): recuento ciego y aprobaciones (blind count and approvals).
- Generar reporte** (Generate report): consolidado del día (daily consolidated).

The final section is 'Actividad reciente' (Recent activity), which lists a series of actions performed by Diana:

- 14:58: Diana inició el recuento ciego en Restaurante Fuentes Ayb (auditoria)
- 14:56: Diana inició el recuento ciego en Restaurante Fuentes Ayb (auditoria)
- 14:55: Diana inició el recuento ciego en Restaurante Fuentes Ayb (auditoria)
- 14:55: Diana inició el recuento ciego en Restaurante Fuentes Ayb (auditoria)
- 14:54: Diana inició el recuento ciego en Restaurante Fuentes Ayb (auditoria)
- 14:54: Diana firmó el conteo de Restaurante Fuentes Ayb (firma)
- 22:58: Diana: Receta eliminada: sopa (receta)
- 22:40: Diana envió un pedido al almacén (pedido)

A vertical sidebar on the left contains navigation icons for Inicio, Pedidos, Conteo, Legalización, Bodegas, Auditoría, Reportes, Panel, Ajustes, Ayuda, Mensajes, Mi perfil, and Cerrar sesión. At the bottom of the sidebar is a profile picture of Diana.

Figura 7. Vista de inicio, captura real: el resumen del día y los cuatro accesos rápidos a lo que la persona hace de verdad.

- **Qué tiene:** el saludo con la fecha, cuatro indicadores personales (bodegas asignadas, referencias contadas hoy, alertas por revisar y la exactitud del mes), cuatro accesos directos con su ícono y la actividad reciente del equipo con hora y responsable.
- **Por qué existe:** el menú lateral tenía la sección Inicio pero ninguna pantalla detrás. Esta vista responde la pregunta con la que llega el auxiliar cada mañana: ¿qué me toca hoy y cómo voy?

Las cinco pantallas del auxiliar mientras cuenta en la bodega.

MENÚ · PEDIDOS *el primero de los tres momentos manuales del reto: pedir insumos al almacén*

5.3 Pedido por voz: del plato a los insumos

Pedidos · Cocina Piscilago SERVICIO ALMUERZO Colsubsidio

Domingo · almuerzo **Cocina Piscilago** **Pedido sin enviar**

Lo que dijo el chef

Aún no ha dictado nada: pulse el micrófono y diga, por ejemplo, «hoy preparamos cincuenta ajiacos».

Lo que entendió CuentaVoz

Todavía no hay ningún plato dictado en esta conversación.

Mantenga presionado y hable
También sirve: «pedir insumos para 30 sancochos»
Receta + stock = pedido

► [Ver frases exactas que entiende el agente aquí](#)

Platos con receta registrada

Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.

AJIACO SANTAFEREÑO **ARROZ CON POLLO** **FRIJOLES CON GARRA** **LENTEJAS** **SANCOCHO DE GALLINA**

CuentaVoz responde

Diga el plato y las porciones: «hoy preparamos cincuenta ajiacos».

Calcular el pedido **Corregir cantidad**

Ver la receta

Figura 8. Vista de pedidos, captura real: los platos con receta registrada, listos para dictar o elegir.

The screenshot shows the CuentaVoz interface with a sidebar on the left containing icons for Inicio, Pedidos, Conteo, Legalización, Bodegas, Auditoría, Reportes, Panel, Ajustes, Ayuda, Mensajes, Mi perfil, and Cerrar sesión. The main header displays 'Pedidos · Cocina Piscilago' and 'SERVICIO ALMUERZO' with the Colsubsidio logo. Below the header, there are tabs for 'Domingo · almuerzo', 'Cocina Piscilago', and 'Pedido sin enviar'. The main content area is divided into sections: 'Lo que dijo el chef' with the text «AJIACO SANTAFEREÑO» (elegido de la lista de recetas), 'Lo que entendió CuentaVoz' with the text 'Todavía no hay ningún plato dictado en esta conversación.', and a modal titled 'Porciones' asking '¿Para cuántas porciones prepara ajiaco santafereño?'. The modal has a numeric input field with the value '50', a microphone icon, and buttons for 'Cancelar' and 'Aceptar'. Below the modal, it says 'Receta + stock = pedido' and 'Ver frases exactas que entiende el agente aquí'. The next section is 'Platos con receta registrada' with the text 'Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.' and a list of buttons: 'AJIACO SANTAFEREÑO', 'ARROZ CON POLLO', 'FRIJOLES CON GARRA', 'LENTEJAS', and 'SANCOCHO DE GALLINA'. The bottom section is 'CuentaVoz responde' with the text 'Listo, ajiaco santafereño. ¿Para cuántas porciones?'.

Pedidos · Cocina Piscilago SERVICIO ALMUERZO Colsubsidio

Domingo · almuerzo **Cocina Piscilago** **Pedido sin enviar**

Lo que dijo el chef

«AJIACO SANTAFEREÑO» (elegido de la lista de recetas)

Lo que entendió CuentaVoz

Todavía no hay ningún plato dictado en esta conversación.

Porciones

¿Para cuántas porciones prepara ajiaco santafereño?

50

Dígalos y confírmelo de viva voz, o escríbalos y use el botón.

Cancelar **Aceptar**

Receta + stock = pedido

► Ver frases exactas que entiende el agente aquí

Platos con receta registrada

Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.

AJIACO SANTAFEREÑO **ARROZ CON POLLO** **FRIJOLES CON GARRA** **LENTEJAS**

SANCOCHO DE GALLINA

CuentaVoz responde

Listo, ajiaco santafereño. ¿Para cuántas porciones?

Confirmar porciones antes de calcular.

- **El caso que plantea el reto:** «hoy preparamos cincuenta ajiacos». La persona no dicta ingredientes ni cantidades: dicta el plato. El agente reconoce la preparación, las porciones y la receta que aplica, y lo confirma en voz antes de calcular nada.
- **Qué tiene:** lo que dijo el chef tal cual, lo que entendió CuentaVoz en cuatro campos (preparación, porciones, receta aplicada y momento), el micrófono con ejemplos de otras frases válidas, y la respuesta hablada que anuncia el siguiente paso.
- **Qué conecta:** la intención pedir del JSON de la sección 6.2 activa la herramienta `calcular_pedido`; el agente no inventa cantidades: las saca de la receta del catálogo de Colsubsidio.

5.4 El pedido calculado: solo lo que de verdad falta

Inicio

Perfiles

Compras

Impresiones

Bodega

Almacén

Reportes

Perfil

Ajustes

Foro

Mensajes

Mi perfil

Cerrar sesión

Pedidos · Cocina Piscilago

SERVICIO ALMACÉN Colsubsidio

Deming: almuerzo

Cocina Piscilago

Pedido sin enviar

Se descuenta de: ALMACEN AYB

Lo que dijo el chef

«AJIACO SANTAFERÑO» (elegido de la lista de recetas)

Lo que entendió CuentaVoz

Preparación	AJACO SANTAFERÑO
Porciones	50
Receta aplicada	estándar de Colsubsidio
Momento	pedido al almacén

Mantenga presionado y hable

También sirve: «pedir insumos para 30 sandwiches»

Receta + stock + pedido

Ver frases exactas que entiende el agente aquí

Platos con receta registrada

Entre son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno para armar el pedido sin dictarlo, o dígame tal cual suene igual.

AJACO SANTAFERÑO

ARROZ CON POLLO

FRIOLES CON GARRA

LENTIJS

SANCOCHO DE GALLINA

CuentaVoz responde

Necesita 5 insumos. Hay que pedir 2 al almacén; el resto ya está en la bodega. ¿Lo enviamos?

Calcular el pedido

Corregir cantidad

Revisar antes de enviar

Esta bodega nunca ha tenido registro de: PAPA CRIOLLA

— se asumió 0 disponible; confirme con el almacén antes de descontar existencias.

AJACO SANTAFERÑO

Rendimiento: 1 porción

5 ingredientes

ADMINISTRAR ESTA RECETA

Catálogo de Colsubsidio

Ingrediente	Unidad	Por porción	Para 50 porciones
POLLO ENTERO	Kilogram	0.25	12.5
PAPA CRIOLLA	Kilogram	0.2	10
ARROZ	Kilogram	0.08	4
SAL	Kilogram	0.005	0.25
ACEITE	Liter	0.01	0.5

Preparación paso a paso

1. Cocinar el pollo entero en agua con sal hasta que este tierno; desmenujar y reservar el caldo.
2. En el mismo caldo, agregar la papa criolla y cocinar hasta que se deshaga un poco y espese el caldo.
3. Aparte, cocinar el arroz como acompañamiento.
4. Volver a poner el pollo desmenujado en el caldo; ajustar la sal y un chorrito de aceite.
5. Servir caliente, acompañado de arroz blanco.

Como administrador puede editar cantidades, agregar o quitar ingredientes, o crear un plato nuevo desde Ajustes → Recetas.

Usar para un pedido

Editar receta

Cerrar

Insumos de la receta

5

Ingredientes estándar

Ya disponibles en bodega

3

no se piden

Hay que pedir al almacén

2

faltante calculado

Insumos calculados para 50 AJACO SANTAFERÑO

Insumo (nombre oficial)	Unidad	Necesario	Hay en bodega	Falta: pedir
POLLO ENTERO	Kilogram	12.5	1600.2	—
PAPA CRIOLLA	Kilogram	10	0	10
ARROZ	Kilogram	4	-720	724
SAL	Kilogram	0.25	158	—
ACEITE	Liter	0.5	333	—

La cantidad necesaria sale de la receta por porción; el faltante es lo necesario menos lo que ya hay en la bodega.

Enviar pedido al almacén

Figura 9. Pedido calculado, captura real: necesario según receta, lo que ya hay en bodega y el faltante real, con aviso de revisión.

Inicio

Recetas

Compras

Comentarios

Reservas

Historial

Perfil

Notificaciones

Ayuda

Mi perfil

Cerrar sesión

Inicio

Recetas

Compras

Comentarios

Reservas

Historial

Perfil

Notificaciones

Ayuda

Mi perfil

Cerrar sesión

SERVICIO ALMUERZO

Cotsubsidio

Pedidos • Cocina Píscologo

Pedido enviado

Se descuenta de: ALMACEN AYB

Lo que dijo el chef

«AJIACO SANTAFERÑO» (elegido de la lista de recetas)

Lo que entendió Cuentabot

Preparación

AJIACO SANTAFERÑO

Porciones

50

Receta aplicada

estándar de Cotsubsidio

Momento

pedido al almacén

Mantenga presionado y hable

También puede: «pedir insumos para 30 sandwiches»

Receta + stock = pedido

Ver otros ejemplos para entender el agente así!

Platos con receta registrada

Entre con la última photo que Cuentabot sabe calcular por ahora. Toma una para armar el pedido sin distractos, si dijiste tal cual suena así!

AJIACO SANTAFERÑO

ARROZ CON POLLO

FRÍJOLS CON GARBIA

LENTISAS

SANCOCHO DE GALLINA

Cuentabot responde

Pedido PED-20260817-064325 registrado y enviado a ALMACEN AYB. ¿Desea armar otro pedido?

Calcular el pedido

Corregir cantidad

Revisar antes de enviar

Esta botella nunca ha tenido registro de: PAPA CIRIOLLA
— se sumará si disponible; confirme con el almacén antes de descontar existencias.

AJIACO SANTAFERÑO

Rendimiento: 1 porción

3 ingredientes

ADMINISTRA ESTA RECETA

Catálogo de Cuentabot

Ingrediente	Unidad	Por porción	Para 50 porciones
POLLO ENTERO	Kilogram	0.25	12.5
PAPA CIRIOLLA	Kilogram	0.2	10
ARROZ	Kilogram	0.08	4
SAL	Kilogram	0.005	0.25
ACEITE	Liter	0.01	0.5

Preparación paso a paso

1. Cocinar el pollo entero en agua con sal hasta que este tierno; desmenujar y reservar el caldo.
2. En el mismo caldo, agregar la papa cirilla y cocinar hasta que se deshaga un poco y espese el caldo.
3. Agregar, cocinar el arroz como acompañamiento.
4. Verter a poner el pollo desmenuado en el caldo, ajustar la sal y un chorrito de aceite.
5. Servir caliente, acompañado de arroz blanco.

Como administradora puede editar cantidades, agregar o quitar ingredientes, o crear un plato nuevo desde Aquí → Recetas.

Usar para un pedido

Editar receta

Cerrar

Insueros de la receta

5 ingredientes estándar

Te la hipotético en la bodega

3 no hay piden

Hay que pedir al almacén

2 suficiente calculado

Insumos calculados para 50 AJIACO SANTAFERÑO

Ingrediente (nombre oficial)	Unidad	Peso porción	Hay en bodega	Falta: pedir
POLLO ENTERO	Kilogram	12.5	1600.2	—
PAPA CIRIOLLA	Kilogram	10	0	10
ARROZ	Kilogram	4	-720	724
SAL	Kilogram	0.25	158	—
ACEITE	Liter	0.5	333	—

La cantidad necesaria sale de la receta por porción, el faltante es lo necesario menos lo que ya hay en la bodega.

PEDIDO REGISTRADO

PED-20260817-064325

Recibo de pedido

Número de pedido

PED-20260817-064325

Plato

AJIACO SANTAFERÑO

500 porciones

Bodega de descuento

ALMACEN AYB

Registrado por

Orana

Hora

06:43

Insumos solicitados

2

Detalle enviado al almacén

Ingrediente	Cantidad	Unidad
PAPA CIRIOLLA	10	Kilogram
ARROZ	724	Kilogram

Este pedido quedó guardado en el sistema y aparecerá tal cual en la liquidación del servicio, comparando el pedido contra lo consumido.

Imprimir recibo

El recibo del pedido enviado al almacén, con número real de pedido.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Pedidos · Cocina Piscilago

SIN CONEXIÓN

Sin conexión
Calcular un pedido necesita el stock en vivo de la bodega, así que no se puede hacer sin señal. Lo que ya dictó no se pierde - siga cuando vuelva la conexión y retome donde quedó.

Lunes · almuerzo

Cocina Piscilago

Pedido sin enviar

Se descuenta de: ALMACEN AYB

Lo que dijo el chef

Aún no ha dictado nada: pulse el micrófono y diga, por ejemplo, «hoy preparamos cincuenta ajiacos».

Lo que entendió CuentaVoz

Todavía no hay ningún plato dictado en esta conversación.

Mantenga presionado y hable

También sirve: «pedir insumos para 30 sancochos»

Receta + stock = pedido

► Ver frases exactas que entiende el agente aquí

Platos con receta registrada

Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.

AJIACO SANTAFEREÑO

ARROZ CON POLLO

FRIJOLES CON GARRA

LENTEJAS

SANCOCHO DE GALLINA

CuentaVoz responde

Diga el plato y las porciones: «hoy preparamos cincuenta ajiacos».

Calcular el pedido

Corregir cantidad

Ver la receta

Aviso cuando se pierde la conexión: no se calcula con stock desactualizado.

26

- **La cuenta que hace el agente:** multiplica la cantidad por porción de la receta por las porciones pedidas, consulta el stock de esa bodega y resta. Lo que da cero o negativo no se pide: por eso en el ejemplo solo salen tres de los ocho ingredientes.
- **Por qué importa:** es exactamente el resultado que el reto pone como ejemplo de buena solución. Sin este cálculo, el pedido se hace «a ojo» y de ahí nacen los sobrantes que después nadie sabe explicar.
- **Qué conecta:** POST /api/pedidos/calcular devuelve la tabla; POST /api/pedidos/enviar la deja registrada para el almacén, con los nombres y códigos oficiales del catálogo.

5.5 La receta que sustenta el cálculo

Pedidos · Cocina Piscilago SERVICIO ALMUERZO Colsubsidio

Domingo · almuerzo Cocina Piscilago Pedido sin enviar

Se descuenta de: ALMACEN AYW

Lo que dijo el chef

«AJIACO SANTA FERREÑO» (elegido de la lista de recetas)

Lo que entendió CuentaVoz

Preparación	AJIACO SANTA FERREÑO
Porciones	50
Receta aplicada	estándar de Colsubsidio
Momento	pedido al almacén

Mantenga presionado y hable
También sirve: «pedir insumos para 30 sancochos»
Receta + stock = pedido

► Ver frases exactas que entiende el agente aquí

Platos con receta registrada
Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.

AJIACO SANTA FERREÑO ARROZ CON POLLO FRIJOL CON GARRA LENTEJAS SANCOCHO DE GALLINA

CuentaVoz responde

Listo, ajiaco santaferreño. ¿Para cuántas porciones?

Calcular el pedido Corregir cantidad

AJIACO SANTA FERREÑO Rendimiento: 1 porción 5 ingredientes ADMINISTRA ESTA RECETA

Catálogo de Colsubsidio

Ingrediente	Unidad	Por porción	Para 50 porciones
POLLO ENTERO	Kilogram	0.25	12.5
PAPA CRIOLLA	Kilogram	0.2	10
ARROZ	Kilogram	0.08	4
SAL	Kilogram	0.005	0.25
ACEITE	Liter	0.01	0.5

Preparación paso a paso

1. Cocinar el pollo entero en agua con sal hasta que este tierno; desmechar y reservar el caldo.
2. En el mismo caldo, agregar la papa criolla y cocinar hasta que se deshaga un poco y espese el caldo.
3. Aparte, cocinar el arroz como acompañamiento.
4. Volver a poner el pollo desmechado en el caldo, ajustar la sal y un chorrito de aceite.
5. Servir caliente, acompañado de arroz blanco.

Como administradora puede editar cantidades, agregar o quitar ingredientes, o crear un plato nuevo desde Ajustes → Recetas.

Usar para un pedido Editar receta

Cerrar

Figura 10. Vista de receta, captura real: el desglose por porción, del catálogo de Colsubsidio, del que sale todo el cálculo del pedido.

- **Qué tiene:** la receta estándar con su rendimiento, la cantidad por porción de cada ingrediente y la proyección a las porciones pedidas. Es la pantalla que responde «¿de dónde salió ese 12,5?».

- **Límite de alcance, declarado en la propia vista:** CuentaVoz consulta las recetas, no las gestiona. Crear o modificar recetas y menús está fuera del alcance del reto y lo resuelve el sistema de Colsubsidio.

MENÚ · CONTEO *el trabajo diario del auxiliar de inventarios*

5.6 Conteo por voz (pantalla principal del auxiliar)

Conteo por voz SIN BODEGA Colsubsidio

¿Qué bodega va a contar?

o diga el nombre de la bodega en voz alta

Lo que reconozca aparece abajo, se lo pregunta de vuelta y usted confirma diciendo «sí» (nombres como «Piscilago» a veces se transcriben mal) — o corrijalo a mano y use «Abrir por nombre exacto».

► Ver frases exactas que entiende el agente aquí

nombre de la bodega...

Abrir por nombre exacto

ADMINISTRACION SUMINISTROS PISCILAGO CERRADA 6 referencias	AIRE LIBRE SUMINISTROS PISCILAGO PENDIENTE 0 referencias
ALMACEN AYB CERRADA 270 referencias	ALMACEN GENERAL EN AUDITORIA 6 referencias
ALMACEN SUMINISTROS CERRADA 297 referencias	AUTOSERVICIO MIRADOR EN CONTEO 0 referencias
AUTOSERVICIOS CASCADA PENDIENTE 0 referencias	AUTOSERVICIOS LAS FUENTES PENDIENTE 0 referencias
BODEGA DE RECREACION SUMINISTROS PENDIENTE 0 referencias	CAFETERIA ACUARIO SUMINISTROS PENDIENTE 0 referencias
CAF.VELAS PENDIENTE 0 referencias	CAF. VELAS SUMINISTROS PENDIENTE 0 referencias
COCINA MIRADOR SUMINISTROS PENDIENTE 0 referencias	COCINA PRINCIPAL SUMINISTROS PENDIENTE 0 referencias
ENFERMERIA SUMINISTROS PENDIENTE 0 referencias	KIOSCO 1 PISCILAGO PENDIENTE 0 referencias
KIOSCO 2 PISCILAGO PENDIENTE 0 referencias	KIOSCO 2 SUMINISTROS PENDIENTE 0 referencias
KIOSCO 3 PISCILAGO PENDIENTE 0 referencias	KIOSCO 3 SUMINISTROS PENDIENTE 0 referencias
KIOSCO BOSQUES PENDIENTE 0 referencias	KIOSCO BOSQUE SUMINISTROS PENDIENTE 0 referencias
KIOSCO CASCADA SUMINISTROS PENDIENTE 0 referencias	KIOSCO PAQUEADERO SUMINISTROS PISCILAGO PENDIENTE 0 referencias

Mostrando 24 de 54 — escriba para filtrar.

Figura 11. Vista de conteo, captura real: selector de bodegas con su estado, punto de entrada del auxiliar al conteo por voz.

Conteo por voz · BODEGA DE RECREACION SUMINISTROS EN CONTEO

Colsubsidio

Avance: 1 de 0 Alertas: 2 Sesión bloqueada

CuentaVoz responde

Listo, bodega de recreacion suministros abierta con 0 referencias. Dícteme el primer producto.

Últimos registros confirmados

✓	AJI CASERO PISCILAGO	36 Kilogram
---	----------------------	-------------

Escribir en vez de hablar

Escriba lo que diría en voz alta:

Cancelar Aceptar

Confirmar **Corregir**

Teclado **Pedir ayuda**

La voz es el camino rápido; el teclado siempre queda como respaldo.

Modo teclado: escribir en vez de hablar, sin salir del conteo.

- **Qué tiene:** la bodega activa y el avance arriba (2 de 312), la burbuja con lo que CuentaVoz entendió y pregunta, la lista de últimos registros con su chulo de confirmación, el botón grande de micrófono («mantenga presionado y hable») y los botones de respaldo: Confirmar, Corregir, Teclado y Ayuda.
- **Qué conecta:** cada frase va por POST /api/agente/turno (sección 12.13); la respuesta vuelve como texto que la tableta lee en voz alta; el chip de avance se refresca con la respuesta de cada turno.

5.7 Desambiguación: elegir entre opciones

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Conteo por voz · BODEGA DE RECREACION SUMINISTROS

EN CONTEO

Avance: 1 de 0

Alertas: 2

Sesión bloqueada

Usted dijo: «hay noventa cazuelas»

Tengo dos parecidos: CAZUELA 16 ONZ o TAPA CAZUELA 16 ONZ. ¿Cual de los dos?

Opción 1

CAZUELA 16 ONZ

Código 95026919

Unidad

Opción 2

TAPA CAZUELA 16 ONZ

Código 95026920

Unidad

Últimos registros confirmados

AJI CASERO PISCILAGO 36 Kilogram

Toque y hable

Reconocimiento en español (Colombia)

► Ver frases exactas que entiende el agente aquí

Confirmar

Corregir

Teclado

Pedir ayuda

La voz es el camino rápido; el teclado siempre queda como respaldo.

Figura 12. Vista de opciones, captura real: dos candidatas parecidas («cazuela 16 onz» / «tapa cazuela 16 onz»), se eligen por voz o con un toque.

- **Qué tiene:** las candidatas como tarjetas grandes con el nombre oficial del catálogo, el código y lo que el sistema registra en esa bodega; y la salida «ninguna de las dos» que lleva al flujo de crear producto (pendiente del auditor).
- **Qué conecta:** las tarjetas vienen del campo opciones del JSON del turno (los candidatos de `buscar_articulo`, sección 12.7); tocar una tarjeta equivale a decir su nombre — ambas rutas llaman al mismo endpoint.

5.8 Alerta de anomalía: preguntar antes de guardar

Conteo por voz · BODEGA DE RECREACION SUMINISTROS EN CONTEO

Colsubsidio

Avance: 1 de 0 Alertas: 2 Sesión bloqueada

Unidad de medida distinta
Ese articulo se maneja en Kilogram, no en Liter. ¿Cual es la cantidad en Kilogram?

Usted dijo: «ARROZ»

Ese articulo se maneja en Kilogram, no en Liter. ¿Cual es la cantidad en Kilogram?

Últimos registros confirmados

✓ AJI CASERO PISCILAGO	36 Kilogram
------------------------	-------------

Toque y hable
Reconocimiento en español (Colombia)
► Ver frases exactas que entiende el agente aquí

Confirmar **Corregir**

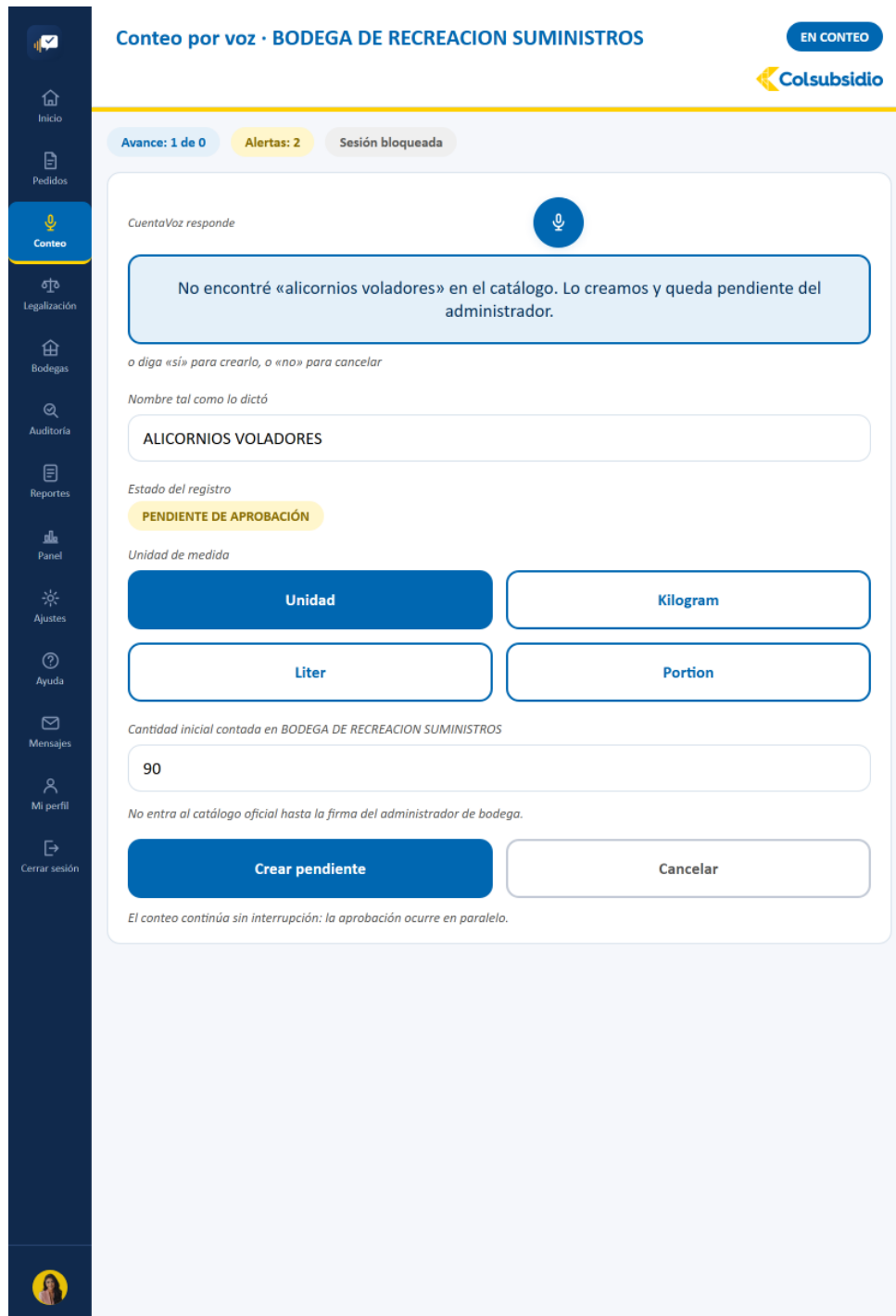
Teclado **Pedir ayuda**

La voz es el camino rápido; el teclado siempre queda como respaldo.

Figura 13. Banner de alerta, captura real (caso de unidad equivocada; el mismo componente detiene también negativos y desviaciones de cantidad como «9 vs. 90», ver backend/servicios/validacion.py).

- **Qué tiene:** el banner amarillo con la razón exacta («el sistema espera alrededor de 10 · usted dijo 90»), la pregunta hablada y dos salidas claras: RECONTAR o CONFIRMAR con marca para el auditor. Nada se guarda solo.
- **Qué conecta:** la alerta nace en validar_conteo (sección 12.8) comparando contra StockSistema — el extracto real de My Inventory —; si la persona confirma, el registro se guarda con su Alerta asociada para que la auditoría la revise.

5.9 Crear producto: pendiente de aprobación



Conteo por voz · BODEGA DE RECREACION SUMINISTROS

EN CONTEO

Colsubsidio

Avance: 1 de 0 Alertas: 2 Sesión bloqueada

CuentaVoz responde

No encontré «alicornios voladores» en el catálogo. Lo creamos y queda pendiente de administrador.

o diga «sí» para crearlo, o «no» para cancelar

Nombre tal como lo dictó

ALICORNIOS VOLADORES

Estado del registro

PENDIENTE DE APROBACIÓN

Unidad de medida

Unidad Kilogram

Liter Portion

Cantidad inicial contada en BODEGA DE RECREACION SUMINISTROS

90

No entra al catálogo oficial hasta la firma del administrador de bodega.

Crear pendiente Cancelar

El conteo continúa sin interrupción: la aprobación ocurre en paralelo.

Figura 14. Vista de creación, captura real: el formulario llega precargado con lo dictado y nace en estado pendiente de aprobación.

- **Qué tiene:** el nombre tal como se dictó, la unidad por chips grandes, la cantidad inicial y el sello visible «PENDIENTE DE APROBACIÓN»: nadie mete un artículo al catálogo oficial por su cuenta.

- **Qué conecta:** crear_articulo (12.10) guarda el registro con estado pendiente_aprobacion; aparece en la bandeja del administrador, y solo su firma lo vuelve oficial.

5.10 Modo formulario: contingencia sin voz ni internet

The screenshot shows the 'Conteo por voz' interface for 'BODEGA DE RECREACION SUMINISTROS'. A yellow banner at the top right indicates 'SIN CONEXIÓN' with the Colsubsidio logo. A sidebar on the left contains navigation icons for Inicio, Pedidos, Conteo (highlighted), Legalización, Bodegas, Auditoría, Reportes, Panel, Ajustes, Ayuda, Mensajes, Mi perfil, and Cerrar sesión.

Conteo por voz · BODEGA DE RECREACION SUMINISTROS

SIN CONEXIÓN

! Sin conexión
El reconocimiento de voz necesita señal, pero CuentaVoz sigue entendiendo lo que escriba. Lo que registre se guarda en este equipo y se sincroniza solo al volver la red.

CuentaVoz responde

Sin conexión, pero sigo entendiendo lo que escriba. Dígame el artículo y la cantidad.

hay noventa cazuelas blancas...

Enviar

Reintentar la voz Registrar campo por campo

El catálogo local sigue sugiriendo nombres oficiales sin internet.

Guardados en este equipo

Nada por sincronizar todavía.

Al volver la señal, la cola se envía sola a la API y las validaciones se aplican igual que en línea.

Figura 15. Modo sin conexión, captura real: formulario manual con cola de sincronización — el conteo no se detiene.

- **Qué tiene:** el aviso claro de «sin conexión», el buscador que sigue sugiriendo nombres oficiales del catálogo local, la cantidad con botones grandes – / + y la cola de registros «por sincronizar».
- **Qué conecta:** los mismos endpoints de siempre: los registros esperan en el equipo y se sincronizan al volver la señal — es la prueba 5 de la sección 14.6 hecha pantalla, y el respaldo si Gemini llegara a caerse en plena demo.

El estado de toda la operación, en vivo.

MENÚ · LEGALIZACIÓN *el tercer momento del reto: cerrar el servicio comparando lo pedido con lo usado*

5.11 Legalización del servicio: lo pedido contra lo usado

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Legalización · ajiaco santafereño

SERVICIO CERRADO

Colsubsidio

400 porciones servidas

1 sobrantes

1 merma por revisar

Al cerrar el servicio se compara lo pedido con lo realmente usado: cada diferencia queda explicada como sobrante o como merma.

Insumo	Pedido	Usado	Diferencia	Qué significa
PAPA CRIOLLA	26.008	23.5	-2.508	Sobrante: vuelve a bodega
SAL	2	2.3	+0.3	Merma: revisar con el chef

CuentaVoz responde

Sobró 2,51 de papa criolla: lo devuelvo a la bodega. Hubo merma de 0,3 en sal. ¿Confirmo la legalización?

o diga «sí» para confirmar, o dicte un ajuste como «el pollo fueron doce kilos»

► Ver frases exactas que entiende el agente aquí

Confirmar legalización

Explicar la merma

Ajustar por voz

Figura 16. Vista de legalización, captura real: cada diferencia entre lo pedido y lo usado, explicada como sobrante o como merma.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Legalización · ajiaco santafereño

SERVICIO CERRADO

Colsubsidio

400 porciones servidas

1 sobrantes

1 merma por revisar

Al cerrar el servicio se compara lo pedido con lo realmente usado: cada diferencia queda explicada como sobrante o como merma.

Insumo	Pedido	Usado	Diferencia	Qué significa
PAPA CRIOLLA	26.008	23.5	-2.508	Sobrante: vuelve a bodega
SAL	2	2.3	+0.3	Merma: revisar con el chef

CuentaVoz responde

Sobró 2,51 de papa criolla: lo devuelvo a la bodega. Hubo merma de 0,3 en sal. ¿Confirмо la legalización?

o diga «sí» para... *[frase de ajuste]*

► Ver frases e...

Completar

Ajustar

Ajustar por voz

Diga el insumo y cuánto se usó realmente:

el pollo fueron doce kilos

Dígalo y confírmelo de viva voz, o escríbalo y use el botón.

Cancelar

Aceptar

Ajustar un renglón por voz, sin salir de la tabla.

- **Qué resuelve:** al cerrar el servicio, la cocina devuelve lo que no usó y reporta lo que se perdió. Hoy eso se anota en papel y se digita después — el mismo problema del conteo, en otro momento del día.
- **Qué tiene:** la tabla de comparación insumo por insumo con la diferencia resaltada y su lectura en palabras: «sobrante que vuelve a bodega», «merma para revisar con el chef» o «cuadró exacto». Y la pregunta hablada que resume todo antes de confirmar.
- **La diferencia entre sobrante y merma:** el sobrante vuelve a la bodega y suma al stock; la merma no vuelve y queda registrada con responsable y hora. Confundirlas es lo que descuadra el inventario del mes siguiente.

5.12 Cierre de la legalización y actualización del histórico

Legalización · ajiaco santafereño SERVICIO CERRADO

400 porciones servidas 1 sobrantes 1 merma por revisar

Al cerrar el servicio se compara lo pedido con lo realmente usado: cada diferencia queda explicada como sobrante o como merma.

Insumo	Pedido	Usado	Diferencia	Qué significa
PAPA CRIOLLA	26.008	23.5	-2.508	Sobrante: vuelve a bodega
SAL	2	2.3	+0.3	Merma: revisar con el chef

CuentaVoz responde

Sobró 2,51 de papa criolla: lo devuelvo a la bodega. Hubo merma de 0,3 en sal. ¿Confirмо la legalización?

o diga «sí» para confirmar, o dicte un ajuste como «el pollo fueron doce kilos»
► Ver frases exactas que entiende el agente aquí

Merma de este servicio

SAL: +0,3 de más que lo pedido — revisar con el chef.

Cerrar **Entendido**

Figura 17. Explicar la merma, captura real: qué pasa con lo devuelto, con la merma y con el consumo real que alimenta el histórico.

- **Las tres salidas del cierre:** lo devuelto vuelve a bodega, la merma queda registrada con su nota, y el consumo real alimenta el histórico.

- **El histórico es la pieza que cierra la meta del reto:** si el ajiaco gasta siempre más papa criolla de la que dice la receta, el sistema lo detecta comparando servicio tras servicio. Ahí es donde la solución deja de ser un capturador de datos y se vuelve el sistema integral que valida con recetas e histórico.
- **Qué conecta:** POST /api/legalizacion/confirmar registra el ajuste, devuelve el sobrante al stock de la bodega y deja el archivo listo para cargarse a My Inventory con los nombres oficiales.

MENÚ · BODEGAS *la vista de control del administrador*

5.13 Tablero de bodegas en vivo

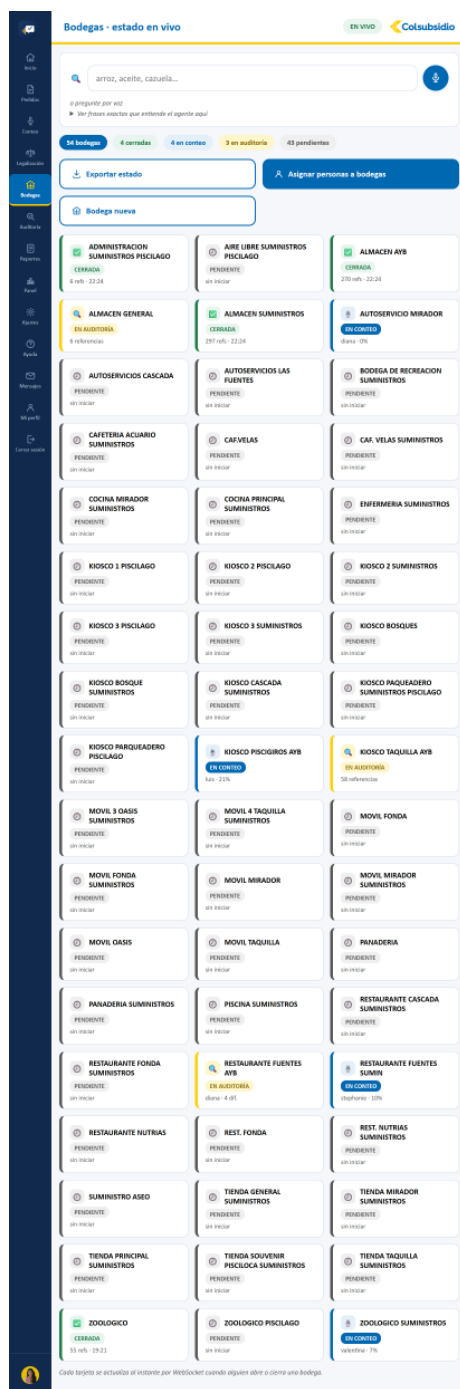


Figura 18. Vista de tablero, captura real: las 54 bodegas de un vistazo, cada una con su estado y su responsable.

Bodegas - estado en vivo EN VIVO Colsubsidio

Buscador: arroz, aceite, cazuela... 🔍

o pregunte por voz
▶ Ver frases exactas que entiende el agente aquí

54 bodegas 4 cerradas 4 en conteo 9 en auditoría 43 pendientes

Exportar estado Asignar personas a bodegas

Bodega nueva

Bodega nueva

Como administrador, la bodega entra directo al catálogo: no necesita otra aprobación.

nombre de la bodega 🗣️

Digito y confirmo de viva voz, o escribido y con el botón.

Cancelar Crear

Cada tarjeta se actualiza al instante por Websocket cuando alguien abre o cierra una bodega.

Registrar una bodega nueva, pendiente de aprobación.

- **Qué tiene:** una tarjeta por bodega con su estado por color (verde cerrada, azul en conteo, amarillo en auditoría, gris pendiente) y quién está en ella; los totales arriba; tocar una tarjeta abre el detalle.
- **Qué conecta:** el WebSocket /api/bodegas/estado (sección 12.12): cuando alguien abre o cierra una bodega, todas las tabletas se actualizan al instante. Es la vista que demuestra los conteos simultáneos sin choques.

Lo que hace el administrador para dar por buena una bodega.

5.14 Detalle de una bodega

The screenshot shows the 'Bodegas · Administracion Suministros Piscilago' interface. The top bar indicates the status is 'CERRADA' (Closed) and the Colsubsidio logo is present. The left sidebar contains navigation options: Inicio, Pedidos, Conteo, Legalización, Bodegas (selected), Auditoría, Reportes, Panel, Ajustes, Ayuda, Mensajes, Mi perfil, and Cerrar sesión.

The main content area displays the following information:

- artículo en esta bodega, o pregúntele algo al agente...** (search bar)
- o pregunte por voz - si no es un artículo de esta bodega, se lo pregunta al agente** (voice search instruction)
- Ver frases exactas que entiende el agente aquí** (link)
- 6 referencias** (status)
- Cerrada 22:24** (status)
- Diana y Valentina** (agents)

Key metrics and status:

- Exactitud de esta bodega:** 50.0 % (3 diferencias de 6)
- Duración del conteo:** 14 min (frente a ~2 min en papel (estimado))
- Unidades en stock:** 185 (sumadas todas las referencias)
- Última toma anterior:** 12 de jul (exactitud 97.1%)

A modal titled **Reabrir bodega cerrada** is displayed, requiring justification for reopening. The text states: 'Esta acción exige una justificación escrita y queda registrada en el historial. Motivo:'. A text input field contains 'revisión solicitada por el chef'. Below the input field, it says: 'Digalo y confírmelo de viva voz, o escríbalo y use el botón.' The modal has two buttons: **Cancelar** and **Reabrir**.

The background interface shows a table of items with columns 'Artículo' and 'Dif.'. The items listed are ACEITE, AGUA BOT, and ACELGA. The 'Dif.' column shows values -3, -7, and -5 respectively. Below the table, there is a 'Línea de tiempo' (Timeline) section with events such as '17:36', '19:22 3 referencias contadas en ADMINISTRACION SUMINISTROS PISCILAGO (datos de muestra)', '20:43 Diana cierra el conteo', '22:24 Diana inicia el recuento ciego', '22:24 Auditoría firmado - ADMINISTRACION SUMINISTROS PISCILAGO', and '22:24 Cierre con doble firma'.

At the bottom, there are buttons for **Reabrir la bodega**, **Ver todos los artículos (6)**, **Descargar detalle**, **Ver en el panel gerencial**, and **Cerrar detalle**.

A note at the bottom states: 'Reabrir una bodega cerrada exige justificación escrita y queda en el registro de trazabilidad.'

Figura 19. Detalle de una bodega cerrada, captura real: reabrir exige justificación y queda en trazabilidad.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Auditoría · ADMINISTRACION SUMINISTROS PISCILAGO

CERRADA

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre

Aprobaciones

Pedidos pendientes

Bandeja de alertas

ADMINISTRACION SUMINISTROS PISCILAGO

CERRADA

Exactitud del cierre

50.0 %

3 diferencias de 6

Referencias

6

6 contadas

Hora de cierre

22:24

Auditó

Diana

Esta bodega ya está cerrada con doble firma. Para volver a contarla hace falta reabrirla desde Bodegas con una justificación escrita.

Volver a la lista

El detalle de solo lectura de una bodega ya cerrada, con sus KPIs de cierre.

- **Cierra una promesa del tablero:** la vista 5.13 invita a tocar una tarjeta para ver el detalle. Esta es esa pantalla, y es la que abre el administrador cuando algo no cuadra.
- **Qué tiene:** la exactitud de esa bodega, la duración del conteo frente al método en papel, el valor del inventario y la comparación con la toma anterior; la tabla de las referencias con diferencia; y la línea de tiempo minuto a minuto, desde la apertura hasta la firma.
- **La línea de tiempo es la pieza de auditoría:** muestra que la alerta del 9 contra 90 se disparó a las 08:20 y se corrigió en el momento, no al final del mes. Reabrir una bodega cerrada exige justificación y queda en el registro de trazabilidad (vista 5.26).

5.15 Consulta de artículo: dónde está y cuánto hay

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Búsqueda GLOBAL

Colsubsidio

Bodegas · consulta de artículo

arroz

CuentaVoz responde

ARROZ: -458.984 Kilogram en 3 bodegas. Encontré artículos parecidos (ARROZ BASMATI, ARROZ FEDERAL); confirme cuál necesita si no es este.

¿Era este otro?

ARROZ BASMATI

ARROZ FEDERAL

Total en el sistema

-459 Kilogram

en 3 bodegas

Consumo del mes

3.6 Kilogram

1 servicios

Cobertura estimada

sin consumo reciente

Bodega	Cantidad	Última toma	Estado
ALMACEN AYB	-720 Kilogram	sin toma	CERRADA
ALMACEN GENERAL	51 Kilogram	15 aug	EN AUDITORÍA
RESTAURANTE FUENTES AYB	210.0156 Kilogram	hoy 14:53	EN AUDITORÍA

Generar reporte

Ver movimientos

Comparar con la receta

← Volver al tablero

Figura 20. Vista de consulta, captura real: la pregunta «¿cuánto arroz hay y en qué bodegas?» resuelta en una pantalla.

50

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Bodegas · consulta de artículo

BÚSQUEDA GLOBAL

Colsubsidio

arroz

CuentaVoz responde

ARROZ: -458.984 Kilogram en 3 bodegas. Encontré artículos parecidos (ARROZ BASMATI, ARROZ FEDERAL); confirme cuál necesita si no es este.

¿Era este otro?

ARROZ BASMATI

ARROZ FEDERAL

Total en el sistema

-459 Kilogram

en 3 bodegas

Consumo del mes

3.6 Kilogram

1 servicios

Cobertura estimada

sin consumo reciente

Bodega	Cantidad	Última toma	Estado
ALMACEN			CERRADA
ALMACEN			EN AUDITORÍA
RESTAURA			EN AUDITORÍA

Comparar con la receta

← Volver al tablero

Movimientos recientes

2026-08-16 14:53 · RESTAURANTE FUENTES AYB · stephanie: 210.0156 Kilogram

2026-08-15 19:22 · ALMACEN GENERAL · luis: 51 Kilogram

Cancelar

Cerrar

Ver movimientos: la trazabilidad de ese artículo en el tiempo.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Bodegas · consulta de artículo

BÚSQUEDA GLOBAL

arroz

CuentaVoz responde

ARROZ: -458.984 Kilogram en 3 bodegas. Encontré artículos parecidos (ARROZ BASMATI, ARROZ FEDERAL); confirme cuál necesita si no es este.

¿Era este otro?

ARROZ BASMATIARROZ FEDERAL

Total en el sistema

-459 Kilogram

en 3 bodegas

Consumo del mes

3.6 Kilogram

1 servicios

Cobertura estimada

sin consumo reciente

Bodega	Cantidad	Última toma	Estado
ALMACEN			CERRADA
ALMACEN			EN AUDITORÍA
RESTAURA			EN AUDITORÍA

Cancelar

Cerrar

Comparar con la receta

← Volver al tablero

Comparar con la receta: en qué platos entra ese insumo.

- **Qué resuelve:** la conversación 11.3 del manual documenta esta pregunta hablada, pero faltaba la pantalla que la muestra. Aquí está: se busca escribiendo o preguntando en voz alta, y el resultado sale en texto y en voz.
- **Qué tiene:** el total del artículo en el sistema, el consumo del mes, la cobertura estimada en días al ritmo actual, y la tabla de en qué bodegas está con el estado de cada una.
- **Por qué la cobertura importa:** no basta saber cuánto hay; el chef necesita saber para cuántos días alcanza. Ese dato sale de cruzar el stock con el consumo real que dejan las legalizaciones.

MENÚ · AUDITORÍA *el segundo conteo, las diferencias y las aprobaciones*

5.16 Auditoría: comparativo y doble firma

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Auditoría · recuento ciego, aprobaciones y cierre

ADMINISTRADORA

Colsubsidio

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre

Aprobaciones

Pedidos pendientes

Bandeja de alertas

Bodegas listas para recuento ciego o cierre

CuentaVoz responde

Tiene 3 bodegas listas para recuento ciego: almacén general, kiosco taquilla ayb, restaurante fuentes ayb. ¿Cuál quiere auditar?

CERRADA	ADMINISTRACION SUMINISTROS PISCILAGO	6 refs	Abrir
CERRADA	ALMACEN AYB	270 refs	Abrir
EN AUDITORÍA	ALMACEN GENERAL	6 refs	Abrir
CERRADA	ALMACEN SUMINISTROS	297 refs	Abrir
EN AUDITORÍA	KIOSCO TAQUILLA AYB	58 refs	Abrir
EN AUDITORÍA	RESTAURANTE FUENTES AYB	344 refs	Abrir
CERRADA	ZOOLOGICO	55 refs	Abrir

El recuento del administrador es a ciegas: no ve el conteo del auxiliar hasta comparar.

Figura 21. Vista del auditor, captura real: el tablero de recuento ciego y cierre, con las bodegas listas para auditar.

Inicio

Pedidos

Conteo

Legalización

Bodega

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Auditoría • RESTAURANTE FUENTES AYB

RECONTEO CIEGO

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre

Aprobaciones

Pedidos pendientes

Bandeja de alertas

298 referencias

3 con diferencia

295 coinciden

Solo se muestran las referencias con diferencia. Su recuento fue a ciegas: el conteo 1 se revela al comparar.

Artículo	Conteo 1	Su conteo	Sistema	Diferencia	Acción
ACEITE	1400	—	851.4316	+548.568	Aceptar
ACEITE DE OLIVA	24	—	28.82	-4.82	Aceptar
ACELGA	16	—	19.093	-3.093	Aceptar

CuentaVoz responde

Recuento ciego iniciado en restaurante fuentes ayb. Dícteme el primer producto.

Confirmar

Actualizar comparación

Teclado

Reconocimiento en español

Ocultar dictado

Aceptar todas

Cerrar con doble firma

Volver a la lista

- **Qué tiene:** la tabla de solo diferencias (3 de 178, no las 178), la columna Dif. resaltada, la explicación de los negativos del sistema, y las dos acciones: resolver por voz o cerrar la bodega con doble firma digital.
- **Qué conecta:** POST /api/bodegas/cerrar arma el comparativo conteo 1 vs. conteo 2 vs. sistema; al firmar, la bodega pasa a «cerrada», el tablero se actualiza por WebSocket y sus datos entran al consolidado (sección 12.11).

5.17 Cierre con doble firma digital

The screenshot displays the 'Auditoría' (Audit) interface for 'ADMINISTRACION SUMINISTROS PISCILAGO'. The status is 'CERRADA' (Closed). The interface includes a sidebar with navigation options: Inicio, Pedidos, Conteo, Legalización, Bodegas, Auditoría (highlighted), Reportes, Panel, Ajustes, Ayuda, Mensajes, Mi perfil, and Cerrar sesión.

Auditoría · ADMINISTRACION SUMINISTROS PISCILAGO CERRADA

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► *Ver frases exactas que entiende el agente aquí*

Recuento ciego y cierre | Aprobaciones | Pedidos pendientes | Bandeja de alertas

ADMINISTRACION SUMINISTROS PISCILAGO CERRADA

Métrica	Valor	Detalles
Exactitud del cierre	50.0 %	3 diferencias de 6
Referencias	6	6 contadas
Hora de cierre	22:24	

Auditó
Diana

Esta bodega ya está cerrada con doble firma. Para volver a contarla hace falta abrirla desde Bodegas con una justificación escrita.

[Volver a la lista](#)

Figura 22. Vista de cierre, captura real: el resultado de solo lectura una vez la bodega quedó cerrada con las dos firmas.

- **Qué tiene:** el resumen de la bodega (178 referencias, diferencias y alertas resueltas), la firma del conteo 1 ya estampada con hora, y la tarjeta de la administradora con «Firmar ahora» por PIN o huella.
- **Qué conecta:** POST /api/bodegas/cerrar exige el perfil de administrador (requiere_perfil, 14.2); con las dos firmas la bodega pasa a «cerrada», el tablero 5.9 se entera por WebSocket y sus datos entran al consolidado 12.11.

5.18 Aprobaciones pendientes: cerrar el ciclo de lo creado

Aprobaciones pendientes 1 PENDIENTES Colsubsidio

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre **Aprobaciones** Pedidos pendientes Bandeja de alertas

Productos y bodegas creados durante la toma que esperan su firma para entrar al catálogo oficial.
CuentaVoz responde

Tiene una aprobación pendiente: restaurante fuentes zoom. ¿Quiere aprobarla o rechazarla?

RESTAURANTE FUENTES ZOOM

Bodega nueva
sin referencias asignadas todavía
Stephanie · 21:03

Rechazar Aprobar

Al aprobar, el producto entra al catálogo con su código definitivo y el conteo asociado deja de estar marcado. Mientras tanto el conteo nunca se detiene: la aprobación ocurre en paralelo al trabajo en bodega.

Figura 23. Vista de aprobaciones, captura real: lo que se creó durante la toma, esperando la firma del administrador.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Pedidos pendientes de aprobación

0 PENDIENTES

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre

Aprobaciones

Pedidos pendientes

Bandeja de alertas

El auxiliar pide y queda registrado de una vez, pero solo sale del almacén cuando usted lo aprueba aquí - igual que con productos y bodegas nuevas.
CuentaVoz responde

No hay pedidos pendientes.

Nada pendiente de aprobación.

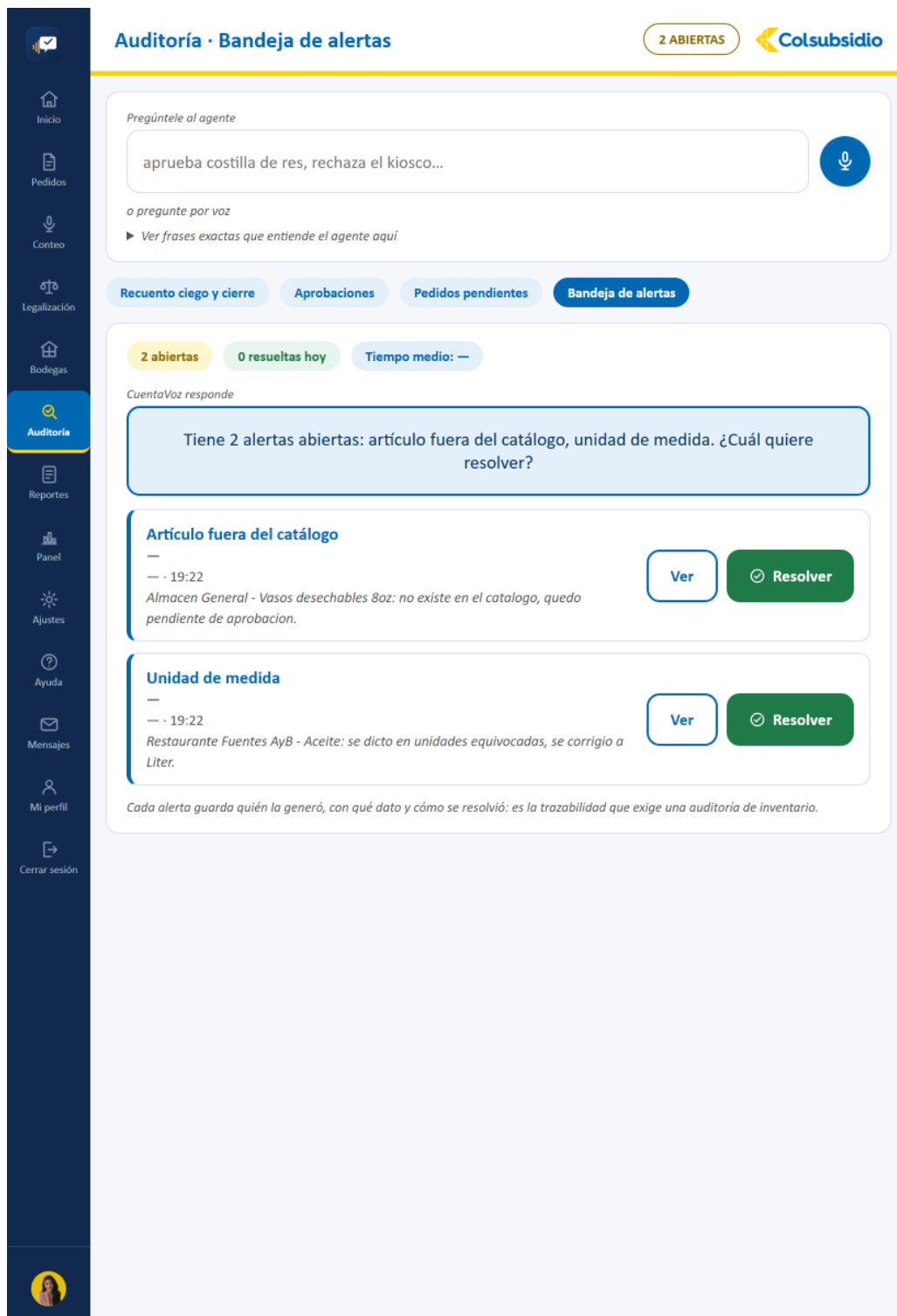
Al aprobar, el pedido queda abierto para su legalización al final del servicio. Al rechazar, no cuenta como enviado y el auxiliar debe volver a intentarlo.

Pedidos pendientes de aprobación, en la misma bandeja.

- **Qué tiene:** una tarjeta por producto o bodega creada, con quién lo creó, a qué hora, qué cantidad se contó y los botones Aprobar o Rechazar.
- **Por qué importa:** la vista 5.16 deja los productos nuevos en estado pendiente, pero faltaba la pantalla donde ese estado se resuelve. Sin ella, el ciclo quedaba abierto: el auxiliar creaba y nadie aprobaba.

Los archivos oficiales que salen de la toma.

5.19 Bandeja de alertas



Auditoría · Bandeja de alertas 2 ABIERTAS

Pregúntele al agente

aprueba costilla de res, rechaza el kiosco...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Recuento ciego y cierre Aprobaciones Pedidos pendientes **Bandeja de alertas**

2 abiertas **0 resueltas hoy** Tiempo medio: —

CuentaVoz responde

Tiene 2 alertas abiertas: artículo fuera del catálogo, unidad de medida. ¿Cuál quiere resolver?

Artículo fuera del catálogo

— · 19:22

Almacen General - Vasos desechables 8oz: no existe en el catalogo, quedo pendiente de aprobacion.

Ver Resolver

Unidad de medida

— · 19:22

Restaurante Fuentes AyB - Aceite: se dicto en unidades equivocadas, se corrigio a Liter.

Ver Resolver

Cada alerta guarda quién la generó, con qué dato y cómo se resolvió: es la trazabilidad que exige una auditoría de inventario.

Figura 24. Vista de alertas, captura real: la cola de trabajo del administrador, con el contexto de cada caso.

The screenshot displays the 'Auditoría · Bandeja de alertas' (Audit · Alert Tray) interface. The top navigation bar includes the title, a '2 ABIERTAS' (2 OPEN) status indicator, and the 'Colsubsidio' logo. A sidebar on the left contains icons for Inicio, Pedidos, Conteo, Legalización, Bodegas, Auditoría (highlighted), Reportes, Panel, Ajustes, Ayuda, Mensajes, Mi perfil, and Cerrar sesión.

The main content area features a search bar with the text 'pregúntele al agente' and a microphone icon. Below it, a section titled 'CuentaVoz responde' shows a message: 'Tiene 2 alertas abiertas: artículo fuera del catálogo, unidad de medida. ¿Cuál quiere resolver?'. The interface also displays a summary of alerts: '2 abiertas', '0 resueltas hoy', and 'Tiempo medio: —'. A list of alerts is shown, with the first one being 'Artículo fuera del catálogo' (Article out of catalog) with a timestamp of '19:22'. A modal dialog is open over this alert, displaying the title 'Artículo fuera del catálogo', the timestamp '19:22', and the message: 'Almacén General - Vasos desechables 8oz: no existe en el catálogo, quedo pendiente de aprobación.' (General Warehouse - Disposable cups 8oz: does not exist in the catalog, pending approval). The modal has 'Cancelar' (Cancel) and 'Cerrar' (Close) buttons. A 'Resolver' (Resolve) button is also visible next to the alert in the background.

Each alert saves who generated it, with what data and how it was resolved: it is the traceability that requires an inventory audit.

El detalle de una alerta puntual, con su motivo exacto.

- **Qué resuelve:** el panel gerencial informa «37 alertas gestionadas», pero faltaba el lugar donde se gestionan. Esta es la cola de trabajo del administrador durante la toma.
- **Qué tiene:** una tarjeta por alerta con su tipo, el artículo y la bodega, el detalle de por qué se disparó, quién la generó y a qué hora, y las dos acciones: ver el caso o resolverlo.
- **Los cuatro tipos que produce el sistema:** desviación de cantidad, saldo negativo del sistema, unidad de medida equivocada y artículo fuera del catálogo — los mismos que devuelve validar_conteo en la sección 12.8.

MENÚ · REPORTES *los archivos oficiales que salen de la toma*

5.20 Reportes y consolidado

ARCHIVOS

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Reportes • consolidado de la toma

Pregúntele al agente

¿cuántas filas tiene el último consolidado?, lléveme al panel...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Consolidado de la toma

Análisis de consumo

Archivos generados con los códigos oficiales de la base

Generar consolidado

Generar diferencias

Imprimir

Estado del tablero
XLSX · hoy 22:24

Exportado

Detalle de bodega
XLSX · hoy 21:35

Exportado

Análisis de consumo
XLSX · hoy 21:34 · 5 filas

Exportado

Registro de trazabilidad
XLSX · hoy 20:22 · 390 filas

Exportado

Diferencias por bodega
XLSX · hoy 18:45 · 28 filas

10 bodegas con descuadre

Consolidado para My Inventory
XLSX · hoy 18:45 · 28 filas

Listo para carga

Dele clic a cualquier tarjeta para ver su contenido aquí al lado.

Vista previa del consolidado

Genere un archivo o dele clic a uno ya generado para ver una muestra aquí.

Figura 25. Vista de reportes, captura real: los archivos oficiales listos para descargar, imprimir o enviar — también por voz.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Reportes · consolidado de la toma

ARCHIVOS

Colsubsidio

Pregúntele al agente

¿cuántas filas tiene el último consolidado?, lléveme al panel...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Consolidado de la toma

Análisis de consumo

Archivos generados con los códigos oficiales de la base

Generar consolidado

Generar diferencias

Imprimir

Vista previa de Consolidado para My Inventory, 28 filas. ¿Desea descargarlo?

Estado del tablero

XLSX · hoy 22:24

Exportado

Detalle de bodega

XLSX · hoy 21:35

Exportado

Análisis de consumo

XLSX · hoy 21:34 · 5 filas

Exportado

Registro de trazabilidad

XLSX · hoy 20:22 · 390 filas

Exportado

Diferencias por bodega

XLSX · hoy 18:45 · 28 filas

10 bodegas con descuadre

Consolidado para My Inventory

XLSX · hoy 18:45 · 28 filas

Listo para carga

Dele clic a cualquier tarjeta para ver su contenido aquí al lado.

Vista previa · Consolidado para My Inventory

Nr.Artículo	Artículo	Unidad	Bodega	Contado	SD	Diferencia
5001	ACELGA	Kilogram	ZOOLOGICO	190	220.7	-30.7
5004	AGUACATE	Kilogram	ZOOLOGICO	25	30	-5
5005	AHUYAMA	Kilogram	ZOOLOGICO	105	123	-18
PEND-0747141809	AJI CASERO PISCILAGO	Kilogram	ALMACEN SUMINISTROS	36	0	+36
7290	ACEITE	Liter	ALMACEN GENERAL	60	70	-10
29018	AGUA BOTELLA	Unidad	ADMINISTRACION SUMINISTROS PISCILAGO	40	47	-7
29018	AGUA BOTELLA	Unidad	ALMACEN GENERAL	45	52	-7
5001	ACELGA	Kilogram	ADMINISTRACION SUMINISTROS PISCILAGO	30	35	-5

Mostrando 8 de 28 filas. Los códigos SIN-#### corresponden a registros del extracto que llegaron sin número de artículo.

Descargar este archivo

Vista previa del consolidado para My Inventory.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Reportes · consolidado de la toma

ARCHIVOS

Colsubsidio

Pregúntele al agente

¿cuántas filas tiene el último consolidado?, lléveme al panel...

o pregunte por voz

Ver frases exactas que entiende el agente aquí

Consolidado de la toma

Análisis de consumo

Archivos generados con los códigos oficiales de la base

Generar consolidado

Generar diferencias

Imprimir

Vista previa de Registro de trazabilidad, 390 filas. ¿Desea descargarlo?

Estado del tablero

XLSX · hoy 22:24

Exportado

Detalle de bodega

XLSX · hoy 21:35

Exportado

Análisis de consumo

XLSX · hoy 21:34 · 5 filas

Exportado

Registro de trazabilidad

XLSX · hoy 20:22 · 390 filas

Exportado

Diferencias por bodega

XLSX · hoy 18:45 · 28 filas

10 bodegas con descuadre

Consolidado para My Inventory

XLSX · hoy 18:45 · 28 filas

Listo para carga

Dele clic a cualquier tarjeta para ver su contenido aquí al lado.

Vista previa · Registro de trazabilidad

fecha	persona	accion	detalle
2026-08-15 20:22:16	diana	REPORTE	Análisis de consumo exportado: reportes/analisis_consumo_20260815_2022.xlsx (5 filas)
2026-08-15 20:22:15	diana	INGRESO	diana inicio sesion
2026-08-15 19:59:23	diana	INGRESO	diana inicio sesion
2026-08-15 19:29:33	diana	INGRESO	diana inicio sesion
2026-08-15 19:28:56	diana	INGRESO	diana inicio sesion
2026-08-15 19:28:42	diana	INGRESO	diana inicio sesion
2026-08-15 19:22:05	diana	ALERTA	Alerta resuelta - AGUJA DESECHABLES CAL.18 (USO ZOOLO): El sistema esperaba alrededor de 160, se contaron 400.
2026-08-15 19:22:05	diana	ALERTA	Alerta 36 resuelta

Mostrando 8 de 390 filas. Los códigos SIN-#### corresponden a registros del extracto que llegaron sin número de artículo.

Descargar este archivo

Vista previa del registro de trazabilidad exportable.

67

- **Qué tiene:** la lista de archivos generados con su estado (el consolidado para My Inventory «listo para carga»), la vista previa con códigos y nombres oficiales —incluido el negativo del sistema ya diagnosticado—, y las tres salidas: descargar, imprimir o enviar al administrador.
- **Qué conecta:** POST /api/reportes ejecuta el consolidado de la sección 12.11 (SQL sobre las tablas reales) y deja el XLSX/CSV en la carpeta de reportes; por voz, «genérame el consolidado» llega aquí a través del agente.

El panel gerencial nativo, calculado en vivo.

5.21 Análisis de consumo: lo pedido contra lo usado

Reportes · análisis de consumo

ÚLTIMOS 30 DÍAS

Colsubsidio

Pregúntele al agente

¿cuántas filas tiene el último consolidado?, lléveme al panel...

o pregunte por voz

Ver frases exactas que entiende el agente aquí

Consolidado de la toma

Análisis de consumo

Pedido en el período

27,25 kg

en 1 servicios

Usado realmente

24,52 kg

90 % de lo pedido

Insumos subutilizados

5

se piden y no se usan

Ahorro potencial

2,73 kg

al mes sí se ajusta

Sobrepedido frente a la receta

POLLO ENTERO		10
PAPA CRIOLLA		10
ARROZ		10
SAL		10
ACEITE		10

% por encima de lo que pide la receta.

Insumos subutilizados: se piden y sobran

Pollo Entero	1,25 kg sin usar	1 de 1 servicios
Papa Criolla	1 kg sin usar	1 de 1 servicios
Arroz	0,4 kg sin usar	1 de 1 servicios
Sal	0,03 kg sin usar	1 de 1 servicios
Aceite	0,05 kg sin usar	1 de 1 servicios

Recomendación automática del agente

pollo entero sobra en 1 de los servicios legalizados: la receta pide 10% más de lo que en promedio se usa. Ajustar la receta liberaría 1,25 kg al mes. La decisión es del chef: el agente solo muestra el patrón.

Exportar análisis

Figura 26. Vista de análisis, captura real: dónde se está sobrepidiendo y qué insumos se piden y no se usan.

- **Responde al «suma puntos» del reto:** el reto pide reportes útiles sobre qué se pidió contra qué se usó, dónde hay insumos subutilizados y dónde se está sobrepidiendo. Esta vista es exactamente eso, construida sobre los datos que dejan las legalizaciones.
- **Qué tiene:** lo pedido y lo usado del período con su porcentaje de aprovechamiento, el conteo de insumos subutilizados y el ahorro potencial mensual; las barras de sobrepedido frente a la receta; y la tabla de insumos que se piden y sobran, con en cuántos servicios ocurre.
- **La recomendación y su límite:** el agente calcula el patrón («la receta pide 1,0 kg de alcaparras y el consumo real promedio es 0,4 kg») y sugiere el ajuste, pero la decisión sigue siendo del chef. El sistema muestra evidencia, no cambia recetas: eso está fuera del alcance del reto.

MENÚ · PANEL *el panel gerencial nativo, calculado en vivo*

5.22 Panel: resumen ejecutivo

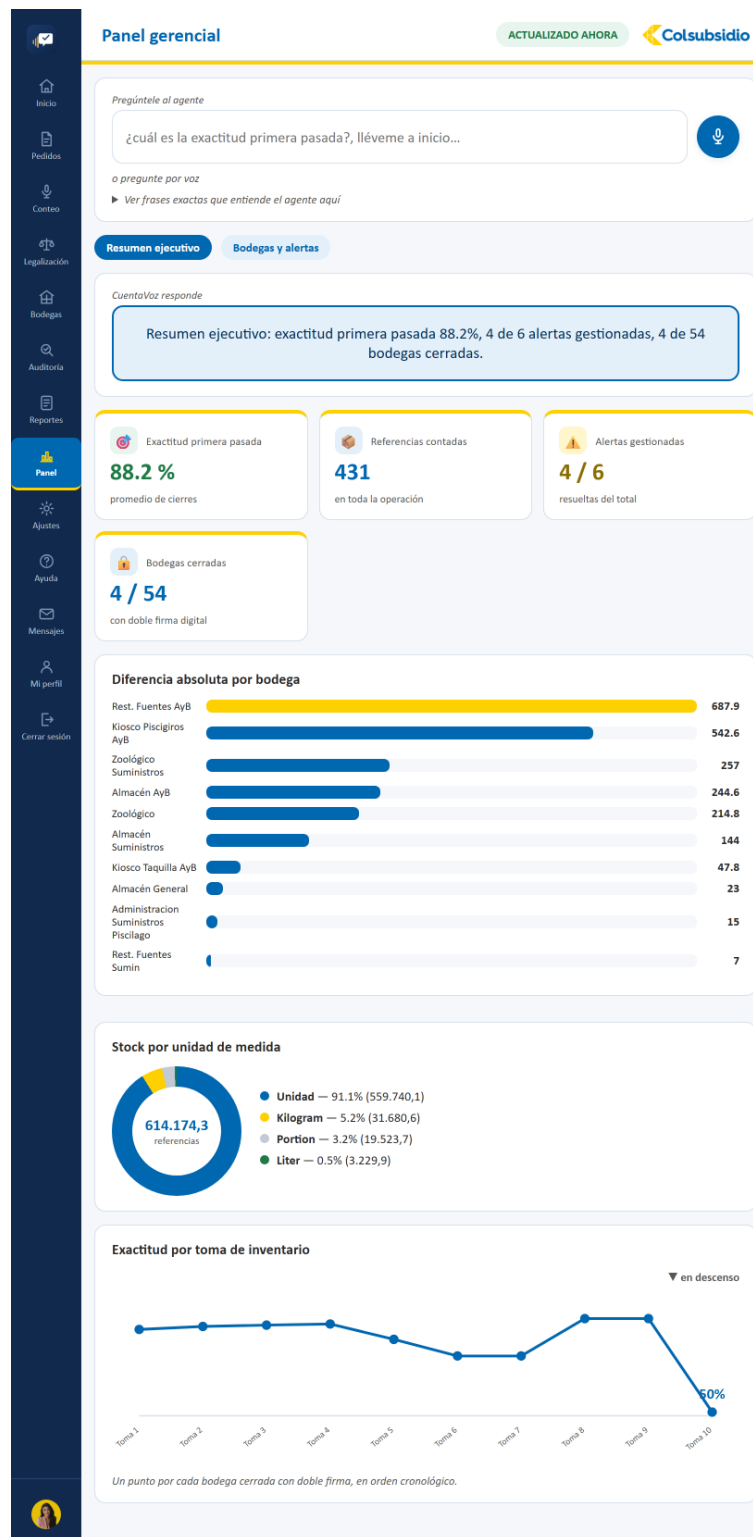
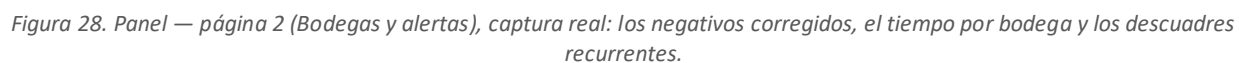


Figura 27. Panel — página 1 (Resumen ejecutivo), captura real: los cuatro indicadores del cierre, la diferencia por bodega, la composición del stock y la tendencia entre tomas.

- **Qué tiene:** cuatro tarjetas de indicador con su variación (exactitud 98,2 % con su flecha, referencias contadas, alertas gestionadas y bodegas cerradas), las diferencias por bodega con la crítica resaltada en dorado, la dona de stock por unidad de medida y la tendencia de exactitud entre tomas.
- **Qué conecta:** no es una maqueta: GET /api/panel/resumen calcula estos números en vivo sobre la base real (backend/servicios/analitica.py), y la dona y las barras se dibujan con CSS puro (conic-gradient), sin ninguna librería de gráficas ni informe externo insertado. El detalle está en la sección 13.



- **Qué tiene:** los 79 saldos negativos llevados a cero, el tiempo promedio por bodega con su mejora frente al conteo manual, los alias aprendidos por el agente, el mapa de puntos con el estado de las 54 bodegas, las alertas por tipo y la tabla de descuadres recurrentes con su acción sugerida.
- **Para qué sirve:** esta página responde la pregunta de fondo del reto: no solo cuántos descuadres hubo, sino dónde está la causa raíz para que la próxima toma tenga menos.

MENÚ · AJUSTES *los parámetros que el administrador mueve sin tocar código*

5.24 Ajustes: la aplicación se adapta a la operación

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Ajustes · configuración del sistema

ADMINISTRADORA

Colsubsidio

Pregúntele al agente

¿cuál es el umbral de anomalía?, lléveme a reportes...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Configuración

Gestión de usuarios

Recetas

Registro de trazabilidad

Usuarios activos

4

con acceso al sistema

Aprobaciones pendientes

1

por revisar en Auditoría

Modo sin conexión

Activo

refresco del tablero en tiempo real

Validación de datos

Umbral de anomalía

50 %

Bloquear cantidades negativas

Exigir confirmación en alertas

Permitir crear productos pendientes

Las reglas fijas del reto (bloquear negativos, exigir confirmación) no se pueden desactivar: son la garantía de integridad de la toma.

Conexión y sincronización

Modo sin conexión

Refresco del tablero

tiempo real

Acerca de CuentaVoz

Versión

1.0.0

Modelo del agente

gemini-flash-latest

Base de datos

SQLite

Idioma del reconocimiento

es-CO

Guardar configuración

Restaurar valores

Figura 29. Vista de ajustes, captura real: los parámetros que el administrador puede mover sin tocar el código.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Ajustes · configuración del sistema

ADMINISTRADORA

Pregúntele al agente

¿cuál es el umbral de anomalía?, lléveme a reportes...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Configuración
Gestión de usuarios
Recetas
Registro de trazabilidad

Recetas (5)

Las porciones que dicte el chef se calculan sobre estas recetas: cantidad por porción × porciones, menos lo que ya haya en la bodega. Aquí se crean, editan y borran — CuentaVoz sí gestiona el catálogo de recetas y menús.

Receta	Rendimiento	Ingredientes		
AJIACO SANTAFEREÑO	1 porción	5	Editar	Eliminar
ARROZ CON POLLO	1 porción	6	Editar	Eliminar
FRIJOLES CON GARRA	1 porción	6	Editar	Eliminar
LENTEJAS	1 porción	6	Editar	Eliminar
SANCOCHO DE GALLINA	1 porción	5	Editar	Eliminar

+ Nueva receta

Pestaña Recetas: ingredientes, rendimiento y preparación, administrables desde aquí.

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Ajustes · configuración del sistema

ADMINISTRADORA

Colsubsidio

Pregúntele al agente

¿cuál es el umbral de anomalía?, lléveme a reportes...

Editar AJIACO SANTAFEREÑO

Cada campo queda disponible de inmediato para calcular pedidos por voz.

Nombre del plato

AJIACO SANTAFEREÑO

Rendimiento (porciones)

1

Ingredientes

Cantidad por cada porción de la receta.

POLLO ENTERO (Kilogram)	0,25	X
PAPA CRIOLLA (Kilogram)	0,2	X
ARROZ (Kilogram)	0,08	X
SAL (Kilogram)	0,005	X
ACEITE (Liter)	0,01	X

+ Agregar ingrediente

Preparación paso a paso

Opcional: lo que ve el auxiliar al consultar esta receta desde Pedidos.

- Cocinar el pollo entero en agua con sal hasta que este tierno; desmechar y reservar el caldo.
- En el mismo caldo, agregar la papa criolla y cocinar hasta que se deshaga un poco y espese el caldo.
- Aparte, cocinar el arroz como acompañamiento.
- Volver a poner el pollo desmechado en el caldo, ajustar la sal y un chorrito de aceite.
- Servir caliente, acompañado de arroz blanco.

Cancelar

Guardar receta

Editar una receta existente.

77

- **Validación de datos:** el umbral de anomalía (50 % por defecto), el bloqueo de cantidades negativas, la confirmación obligatoria ante alertas y el permiso para crear productos. Son los mismos valores del archivo .env de la sección 12.2, pero editables desde la pantalla.
- **Conexión y sincronización:** modo sin conexión, refresco del tablero y reintentos automáticos.
- **Administración:** gestión de usuarios, aprobaciones pendientes, respaldo automático diario y registro de auditoría — visible solo para el perfil de administrador.
- **Acerca de:** versión de la aplicación, modelo del agente en uso, versión de la base y fecha de la última copia de seguridad. Esta tarjeta es la primera que pide soporte cuando algo falla.

Los datos, la foto y las preferencias de cada persona.

5.25 Gestión de usuarios y permisos

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Ajustes · configuración del sistema

ADMINISTRADORA

Colsubsidio

Pregúntele al agente

¿cuál es el umbral de anomalía?, lléveme a reportes...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Configuración

Gestión de usuarios

Recetas

Registro de trazabilidad

Gestión de usuarios (4)

3 auxiliares

1 administradores

0 inactivos

+ Nuevo usuario

Persona	Perfil	Bodegas asignadas	Último acceso	Estado	
S Stephanie	Auxiliar	3	08/14 02:13	Activo	Asignar bodegas Editar Desactivar
L Luis	Auxiliar	4	08/12 13:14	Activo	Asignar bodegas Editar Desactivar
D Diana	Administrador	0	08/16 16:40	Activo	Asignar bodegas Editar Desactivar
V Valentina	Auxiliar	3	08/15 03:38	Activo	Asignar bodegas Editar Desactivar

Ver asignación por bodega

Qué puede hacer cada perfil

✓

Contar por voz y corregir sus propios registros

Auxiliar y administrador

✓

Crear productos y bodegas (quedan pendientes)

Auxiliar y administrador

✓

Aprobar creaciones, auditar y cerrar bodegas

Solo administrador

✓

Gestionar usuarios y cambiar los ajustes del sistema

Solo administrador

Figura 30. Vista de usuarios, captura real: quién puede entrar, con qué perfil y a qué bodegas.

79

The screenshot shows the 'Ajustes · configuración del sistema' (Settings · system configuration) page. The left sidebar contains navigation icons for Inicio, Pedidos, Conteo, Legalización, Bodegas, Auditoría, Reportes, Panel, Ajustes (highlighted), Ayuda, Mensajes, Mi perfil, and Cerrar sesión. The top right shows 'ADMINISTRADORA' and the 'Colsubsidio' logo.

The main content area is titled 'Ajustes · configuración del sistema'. Below the header, there's a section for interacting with the agent, followed by tabs: 'Configuración', 'Gestión de usuarios' (active), 'Recetas', and 'Registro de trazabilidad'.

The 'Gestión de usuarios' section shows a summary: '3 auxiliares', '1 administradores', and '0 inactivos'. A '+ Nuevo usuario' button is present. Below this is a table of users with columns: Persona, Perfil, Bodegas, Último, and Estado. The table lists users: S. Stephan, L. Luis, D. Diana, and V. Valentina.

A modal titled 'Editar A Luis' is open, allowing editing of user 'Luis'. It contains fields for 'Correo' (Email) with the value 'patinho200391@gmail.com' and 'Perfil' (Role) with a dropdown menu set to 'Auxiliar'. At the bottom of the modal are 'Cancelar' and 'Guardar' buttons.

Below the modal, there's a button 'Ver asignación por bodega' and a section titled 'Qué puede hacer cada perfil' (What each profile can do) with a list of permissions:

Permiso	Perfil
☒ Contar por voz y corregir sus propios registros	Auxiliar y administrador
☒ Crear productos y bodegas (quedan pendientes)	Auxiliar y administrador
☒ Aprobar creaciones, auditar y cerrar bodegas	Solo administrador
☒ Gestionar usuarios y cambiar los ajustes del sistema	Solo administrador

Editar una persona existente.

Ajustes · configuración del sistema ADMINISTRADORA Colsubsidio

Pregúntele al agente

¿cuál es el umbral de anomalía?, lléveme a reportes...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Configuración **Gestión de usuarios** Recetas Registro de trazabilidad

Bodegas Para Luis

Marque las bodegas que va a poder contar esta persona.

- ☐ ADMINISTRACION SUMINISTROS PISCILAGO
- ☐ AIRE LIBRE SUMINISTROS PISCILAGO
- ☒ ALMACEN AYB
- ☒ ALMACEN GENERAL
- ☒ ALMACEN SUMINISTROS
- ☐ AUTOSERVICIO MIRADOR
- ☐ AUTOSERVICIOS CASCADA
- ☐ AUTOSERVICIOS LAS FUENTES
- ☐ BODEGA DE RECREACION SUMINISTROS
- ☐ CAFETERIA ACUARIO SUMINISTROS

4 bodegas marcadas

Cancelar **Guardar**

Qué puede hacer cada perfil

☒ Contar por voz y corregir sus propios registros	Auxiliar y administrador
☒ Crear productos y bodegas (quedan pendientes)	Auxiliar y administrador
☒ Aprobar creaciones, auditar y cerrar bodegas	Solo administrador
☒ Gestionar usuarios y cambiar los ajustes del sistema	Solo administrador

Asignar bodegas a un auxiliar.

- **Qué resuelve:** la vista de ajustes anuncia «gestión de usuarios · 12 activos» pero no había pantalla detrás. Sin ella, dar de alta a un auxiliar nuevo obligaría a tocar la base de datos.
- **Qué tiene:** la lista de personas con su perfil, las bodegas que tiene asignadas, su último acceso y su estado; el botón para crear una persona nueva; y la tabla que deja explícito qué puede hacer cada perfil.
- **La regla de permisos, escrita en la propia pantalla:** el auxiliar cuenta y crea (pendiente de aprobación); solo el administrador aprueba, audita, cierra bodegas y gestiona usuarios. Es la traducción visual de requiere_perfil de la sección 14.2.

5.26 Registro de trazabilidad

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes

Mi perfil

Cerrar sesión

Ajustes · configuración del sistema

ADMINISTRADORA

Colsubsidio

Pregúntele al agente

¿cuál es el umbral de anomalía?, lléveme a reportes...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Configuración

Gestión de usuarios

Recetas

Registro de trazabilidad

Registro de trazabilidad (43 acciones)

Hoy

Todas las personas

Todas las acciones

Exportar

Hora	Persona	Acción	Detalle
16:41:10	Diana	AUDITORIA	Recuento ciego iniciado en RESTAURANTE FUENTES AYB
16:40:49	Diana	INGRESO	diana inicio sesion
16:38:24	Diana	INGRESO	diana inicio sesion
16:10:54	Diana	INGRESO	diana inicio sesion
16:06:18	Diana	INGRESO	diana inicio sesion
16:03:10	Diana	INGRESO	diana inicio sesion
16:00:54	Diana	INGRESO	diana inicio sesion
15:51:09	Diana	INGRESO	diana inicio sesion
15:50:43	Diana	INGRESO	diana inicio sesion
15:48:33	Diana	INGRESO	diana inicio sesion
15:48:04	Diana	INGRESO	diana inicio sesion
15:46:45	Diana	INGRESO	diana inicio sesion
15:46:17	Diana	INGRESO	diana inicio sesion
15:45:10	Diana	INGRESO	diana inicio sesion
15:44:49	Diana	INGRESO	diana inicio sesion
14:58:01	Diana	AUDITORIA	Recuento ciego iniciado en RESTAURANTE FUENTES AYB
14:57:56	Diana	INGRESO	diana inicio sesion
14:56:44	Diana	AUDITORIA	Recuento ciego iniciado en RESTAURANTE FUENTES AYB
14:56:39	Diana	INGRESO	diana inicio sesion
14:55:49	Diana	AUDITORIA	Recuento ciego iniciado en RESTAURANTE FUENTES AYB
14:55:44	Diana	INGRESO	diana inicio sesion
14:55:07	Diana	AUDITORIA	Recuento ciego iniciado en RESTAURANTE FUENTES AYB
14:55:00	Diana	INGRESO	diana inicio sesion
14:54:31	Diana	AUDITORIA	Recuento ciego iniciado en RESTAURANTE FUENTES AYB
14:54:29	Diana	FIRMA	Conteo firmado - RESTAURANTE FUENTES AYB

Cumple la Ley 1581 de 2012: se registra la acción, no datos personales sensibles. El registro no se puede editar ni borrar.

Por qué el registro es inmutable

Una corrección no borra el valor anterior: crea un registro nuevo que apunta al original (campo corrige_a de la tabla Conteo). Así, si alguien pregunta «¿quién cambió este dato y por qué?», la respuesta está en el sistema y no en la memoria de nadie.

Figura 31. Vista de trazabilidad, captura real: quién hizo qué, cuándo y con qué dato — un registro que no se puede editar.

- **Qué resuelve:** los ajustes muestran «registro de auditoría: activado» y la sección 14 promete trazabilidad total, pero no existía la pantalla donde se consulta. Para una empresa que audita inventarios, esta vista no es opcional.
- **Qué tiene:** la lista cronológica de acciones con hora exacta, persona, tipo de acción y el detalle del dato; filtros por fecha, persona y tipo; y la exportación para llevar el registro a una auditoría externa.
- **Por qué es inmutable:** una corrección no borra el valor anterior: crea un registro nuevo que apunta al original mediante el campo `corrige_a` de la tabla Conteo (sección 12.5). Por eso el sistema puede responder «quién cambió este dato y por qué» sin depender de la memoria de nadie.
- **Cumplimiento:** el registro guarda la acción y el dato de inventario, no información personal sensible, en línea con la Ley 1581 de 2012 (sección 14.6).

MENÚ · AYUDA *el soporte dentro de la aplicación, para no depender de un manual impreso*

5.27 Centro de ayuda y estado del sistema

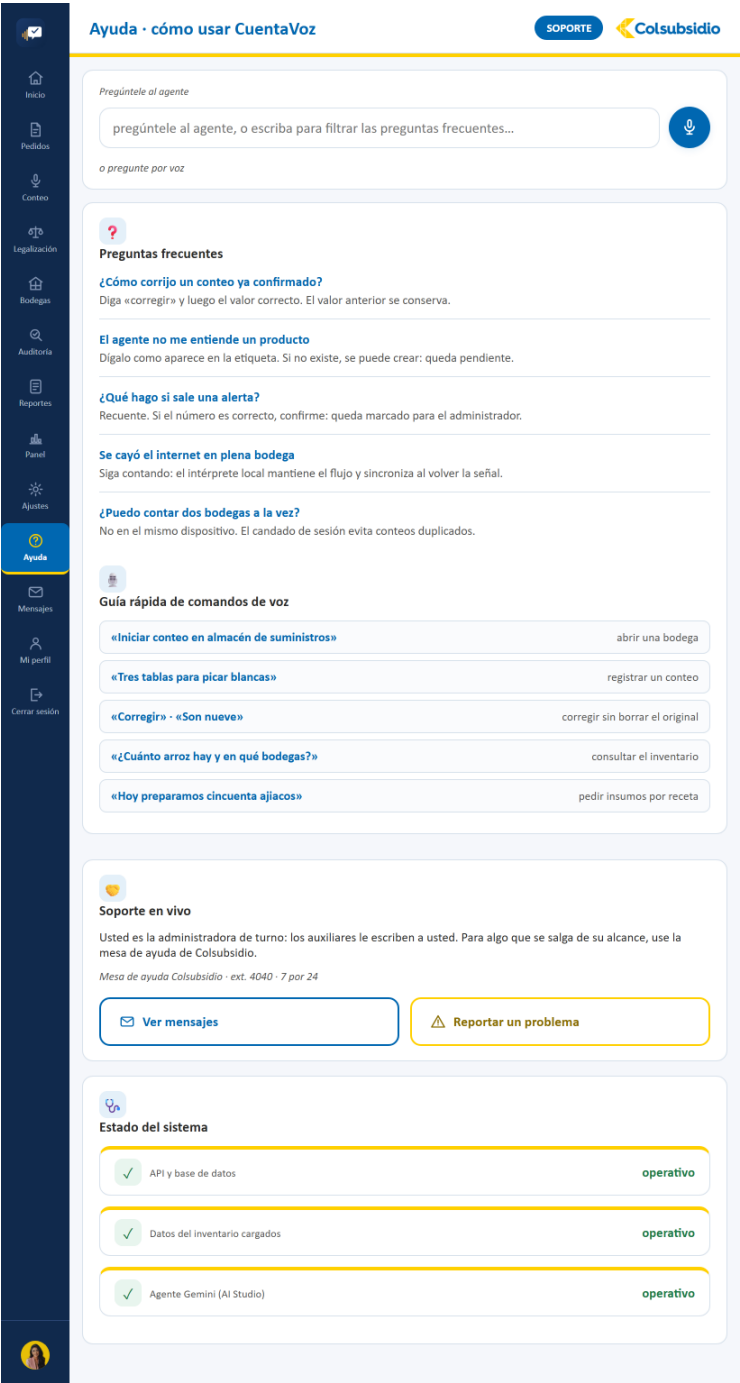


Figura 32. Vista de ayuda, captura real: preguntas frecuentes, soporte en vivo, estado del sistema y la guía de comandos de voz.

Ayuda · cómo usar CuentaVoz SOPORTE Colsubsidio

Pregúntele al agente

pregúntele al agente, o escriba para filtrar las preguntas frecuentes...

o pregunte por voz

Preguntas frecuentes

¿Cómo corrijo un conteo ya confirmado?
Diga «corregir» y luego el valor correcto. El valor anterior se conserva.

El agente no me entiende un producto
Dígalo como aparece en la etiqueta. Si no existe, se puede crear: queda pendiente.

¿Qué hago si sale una alerta?
Recuente. Si el número es correcto, confirme: queda marcado para el administrador.

Se cayó el internet en plena bodega
Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal.

¿Puedo contar dos bodegas a la vez?
No en el mismo dispositivo. El candado de sesión evita conteos duplicados.

Reportar un problema
Describe el problema que encontró:

Se cayó el micrófono al confirmar un conteo...

Dígallo y confírmelo de viva voz, o escríbalo y use el botón.

Cancelar Enviar

Guía rápida

«Iniciar co...» abrir una bodega

«Tres tabl...» registrar un conteo

«Corregir...» registrar sin borrar el original

«¿Cuánto...» consultar el inventario

«Hoy prep...» pedir insumos por receta

Soporte en vivo

Usted es la administradora de turno: los auxiliares le escriben a usted. Para algo que se salga de su alcance, use la mesa de ayuda de Colsubsidio.

Mesa de ayuda Colsubsidio · ext. 4040 · 7 por 24

Ver mensajes Reportar un problema

Estado del sistema

✓ API y base de datos	operativo
✓ Datos del inventario cargados	operativo
✓ Agente Gemini (AI Studio)	operativo

Reportar un problema, con envío real por correo.

The screenshot displays the 'Ayuda · cómo usar CuentaVoz' (Help · how to use CuentaVoz) section of the application. The interface includes a sidebar with navigation options: Inicio, Pedidos, Conteo, Legalización, Bodegas, Ayuda (selected), Mensajes, Mi perfil, and Cerrar sesión. The main content area features a search bar for asking questions, a list of frequent questions, and a 'Soporte en vivo' (Live support) section. A modal titled 'Escribirle al administrador' (Write to the administrator) is open, showing a message composition screen. The message is addressed to Diana and from Luis, with the subject 'Mensaje desde CuentaVoz'. The message content is 'Necesito que me asigne la bodega de Kiosco Taquilla...'. The modal also includes a 'Cancelar' (Cancel) button and an 'Enviar' (Send) button. Below the modal, the 'Soporte en vivo' section shows the administrator Diana is available, with buttons for 'Escribirle al administrador', 'Ver mensajes', and 'Reportar un problema'. At the bottom, the 'Estado del sistema' (System status) section shows three components: API y base de datos, Datos del inventario cargados, and Agente Gemini (AI Studio), all marked as 'operativo'.

Ayuda · cómo usar CuentaVoz

Pregúntele al agente

pregúntele al agente, o escriba para filtrar las preguntas frecuentes...

o pregunte por voz

Preguntas frecuentes

¿Cómo corrijo un conteo ya confirmado?
Diga «corregir» y luego el valor correcto. El valor anterior se conserva.

El agente no me entiende un producto
Dígallo como aparece en la etiqueta. Si no existe, se puede crear: queda pendiente.

¿Qué hago si sale una alerta?
Recuente. Si el número es correcto, confirme: queda marcado para el administrador.

Se cayó el internet en plena bodega
Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal.

¿Puedo contar dos bodegas a la vez?
No en el mismo dispositivo. El candado de sesión evita conteos duplicados.

Guía rápida

«Iniciar conteo»

«Tres tablas»

«Corregir»

«¿Cuánto?»

«Hoy prepago»

«abrir una bodega»

«registrar un conteo»

«registrar sin borrar el original»

«consultar el inventario»

«pedir insumos por receta»

Escribirle al administrador

Para: Diana
De: Luis
Asunto: Mensaje desde CuentaVoz

Escriba su mensaje:

Necesito que me asigne la bodega de Kiosco Taquilla...

Dígallo y confírmelo de viva voz, o escríbalo y use el botón.

Cancelar Enviar

Soporte en vivo

Administrador de bodega · Diana · disponible

Escribirle al administrador

Mesa de ayuda Colsubsidio · ext. 4040 · 7 por 24

Ver mensajes Reportar un problema

Estado del sistema

✓ API y base de datos	operativo
✓ Datos del inventario cargados	operativo
✓ Agente Gemini (AI Studio)	operativo

Escribirle al administrador (perfil auxiliar).

- **Qué resuelve:** la pantalla de conteo tiene un botón «Pedir ayuda» desde la primera versión, pero no llevaba a ninguna parte. Y en una bodega, con las manos ocupadas, nadie va a buscar un manual impreso.
- **Qué tiene:** las cinco preguntas que de verdad surgen en bodega (cómo corregir, qué hacer si el agente no entiende, qué significa una alerta, qué pasa sin internet, si se pueden contar dos bodegas a la vez); el soporte en vivo con el administrador disponible y la mesa de ayuda de Colsubsidio; el estado de los tres servicios; y la guía rápida de comandos de voz.
- **El estado del sistema no es decorativo:** si el agente de voz aparece caído, la persona sabe de inmediato que debe usar el modo formulario (vista 5.10) en lugar de pensar que hace algo mal.

MENÚ · MENSAJES *soporte en vivo entre auxiliares y administrador, agregado tras el reto original*

5.27a Mensajes: bandeja de soporte en vivo

- **Qué resuelve:** la vista de Ayuda (5.27) ya permitía «escribirle al administrador», pero esa conversación no tenía una bandeja propia donde verla, responderla o consultar el historial: cada mensaje se perdía en el correo. Mensajes es esa bandeja, con entrada propia en el menú lateral.
- **Qué tiene:** la lista de mensajes recibidos y enviados entre el auxiliar y el administrador de bodega, con quién escribió, cuándo, y el botón Responder que abre el hilo completo de la conversación.
- **Qué conecta:** POST /api/soporte/mensaje-administrador crea el mensaje; GET /api/soporte/mensajes lo lista; POST /api/soporte/mensajes/{id}/responder cierra el hilo. El mismo formulario vive en Ayuda.jsx («Escribirle al administrador») y en Mensajes.jsx — ambos comparten frontend/src/correoAdmin.js, que además envía un correo real por EmailJS para que la respuesta no dependa de que alguien esté mirando la pantalla en ese momento.
- **Por qué no estaba en la propuesta original:** la mesa de ayuda de Colsubsidio (vista 5.27) cubre el soporte institucional; Mensajes cubre algo más chico y más frecuente: la pregunta rápida de un auxiliar a su administrador de bodega durante la toma, sin necesitar el correo corporativo ni una llamada.

Mensajes

Mi perfil

Cerrar sesión

Inicio

Pedidos

Conteo

Legalización

Bodegas

Auditoría

Reportes

Panel

Ajustes

Ayuda

Mensajes · soporte en vivo

BANDEJA DEL ADMINISTRADOR

Colsubsidio

Mensajes recibidos

6

en total

Pendientes de respuesta

1

por responder

Respondidos

5

ya resueltos

Responder por voz

Diga o escriba la respuesta...

Un solo mensaje pendiente: lo que diga se usa directo como respuesta.

V

Valentina

respondido 2026-08-15 03:38

Necesito que me asigne una bodega.

D

Su respuesta

2026-08-15 03:40

Ya quedó asignado la bodega.

S

Stephanie

pendiente 2026-08-14 01:22

Necesito que me asigne lo Vega kiosco.

Responder

S

Stephanie

respondido 2026-08-14 00:51

necesito que me asigne una bodega por favor

D

Su respuesta

2026-08-14 01:13

Si.

S

Stephanie

respondido 2026-08-14 00:08

Confirmando correo correcto de Diana tras el fix.

D

Su respuesta

2026-08-14 00:20

gracias por el mensaje

S

Stephanie

respondido 2026-08-13 23:44

prueba

D

Su respuesta

2026-08-13 23:53

guia completa

S

Stephanie

respondido 2026-08-13 23:38

Prueba real de bandeja en produccion.

D

Su respuesta

2026-08-13 23:38

Confirmado, funciona en produccion.

Figura 34a. Vista de Mensajes, captura real: bandeja de soporte en vivo entre auxiliares y administrador.

The screenshot displays the 'Mensajes · soporte en vivo' (Messages · live support) interface. At the top, there's a header with 'BANDEJA DEL ADMINISTRADOR' and the 'Colsubsidio' logo. Below the header, three summary cards show: 'Mensajes recibidos' (6 en total), 'Pendientes de respuesta' (1 por responder), and 'Respondidos' (5 ya resueltos). The main area is titled 'Responder por voz' and contains a text input field 'Diga o escriba la respuesta...' and a microphone icon. A note below states: 'Un solo mensaje pendiente: lo que diga se usa directo como respuesta.' Below this, a list of messages is visible. A modal titled 'Responder mensaje' is open, showing details for a message from 'Stephanie' to 'Diana'. The modal contains the original message, a text input for the response, and 'Cancelar' and 'Enviar' buttons. The background messages include:

- Valentina** (respondido 2026-08-15 03:38): Necesito que me asigne una bodega.
- Stephanie** (pendiente 2026-08-14 01:22): Necesito que me asigne lo Vega kiosco.
- Stephanie** (respondido 2026-08-14 00:51): Ya quedó asignada esa bodega, revise en unos minutos...
- Stephanie** (respondido 2026-08-14 00:08): Confirmando correo correcto de Diana tras el fix.
- Stephanie** (respondido 2026-08-13 23:44): prueba
- Stephanie** (respondido 2026-08-13 23:38): Prueba real de bandeja en produccion.

Responder un mensaje, sin salir de la bandeja.

MENÚ · MI PERFIL *los datos personales de quien usa la aplicación*

5.28 Mi perfil: datos personales y foto

Mi perfil · datos personales SESIÓN ACTIVA Colsubsidio

Pregúntele al agente

cambia mi voz a puck, ¿cuándo fue mi último acceso?...

o pregunte por voz

► Ver frases exactas que entiende el agente aquí

Diana
Administrador de bodega

[Cambiar foto](#)

Datos personales

Nombre completo
diana

Correo corporativo
diana.arguello@colsubsidio.com

Teléfono de contacto

Código de empleado
CS-48200

Bodegas asignadas
Sin bodegas asignadas todavía.

Seguridad de la cuenta

Último acceso: 08/16 20:15
Su PIN vence en 68 días

PIN actual

PIN nuevo (6 dígitos)

Confirmar PIN

[Cambiar PIN](#)

[Registrar huella de este dispositivo](#) [Cerrar sesión en todos los dispositivos](#)

El PIN se guarda cifrado (encrypt); este equipo o navegador no tiene un lector de huella / Windows Hello configurado, así que el ingreso con huella no está disponible aquí.

Preferencias de voz

Voz de CuentaVoz
Kore — Femenina, firme y clara

[Probar voz](#)

Voz neuronal real (no la del navegador): suena humana y fluida. El reconocimiento de lo que usted dice sigue siempre en español de Colombia; esto solo cambia cómo suena CuentaVoz al responder.

Velocidad de la respuesta

Lenta Normal Rapida

Confirmación hablada

Activada Desactivada

[Guardar cambios](#)

Figura 33. Vista de perfil, captura real: la persona actualiza sus datos, su foto, su PIN y sus preferencias de voz.

Cerrar sesión en todos los dispositivos

Esto cierra la sesión en todas las tabletas donde haya entrado, incluida esta. ¿Continuar?

Cancelar

Cerrar todas

Cerrar sesión en todos los dispositivos: invalida todos los tokens emitidos.

- **Qué tiene:** la foto con el botón «Cambiar foto», los datos personales editables (nombre, correo corporativo, teléfono y código de empleado), las bodegas asignadas, la seguridad de la cuenta (cambiar PIN, registrar huella, cerrar sesión en todos los dispositivos) y las preferencias de voz: idioma del reconocimiento, velocidad de la respuesta y confirmación hablada.
- **Qué conecta:** PUT /api/usuarios/yo actualiza los datos; la foto se sube por POST /api/usuarios/yo/foto y se guarda como archivo, no en la base. El cambio de PIN reutiliza hash_clave de la sección 14.2, así que la contraseña nueva tampoco queda en texto plano.
- **Cómo se llega:** por la entrada «Mi perfil» del menú lateral, disponible para cualquier perfil.

MENÚ · CERRAR SESIÓN *la salida segura, sin perder trabajo*

5.29 Cerrar sesión con trabajo sin guardar

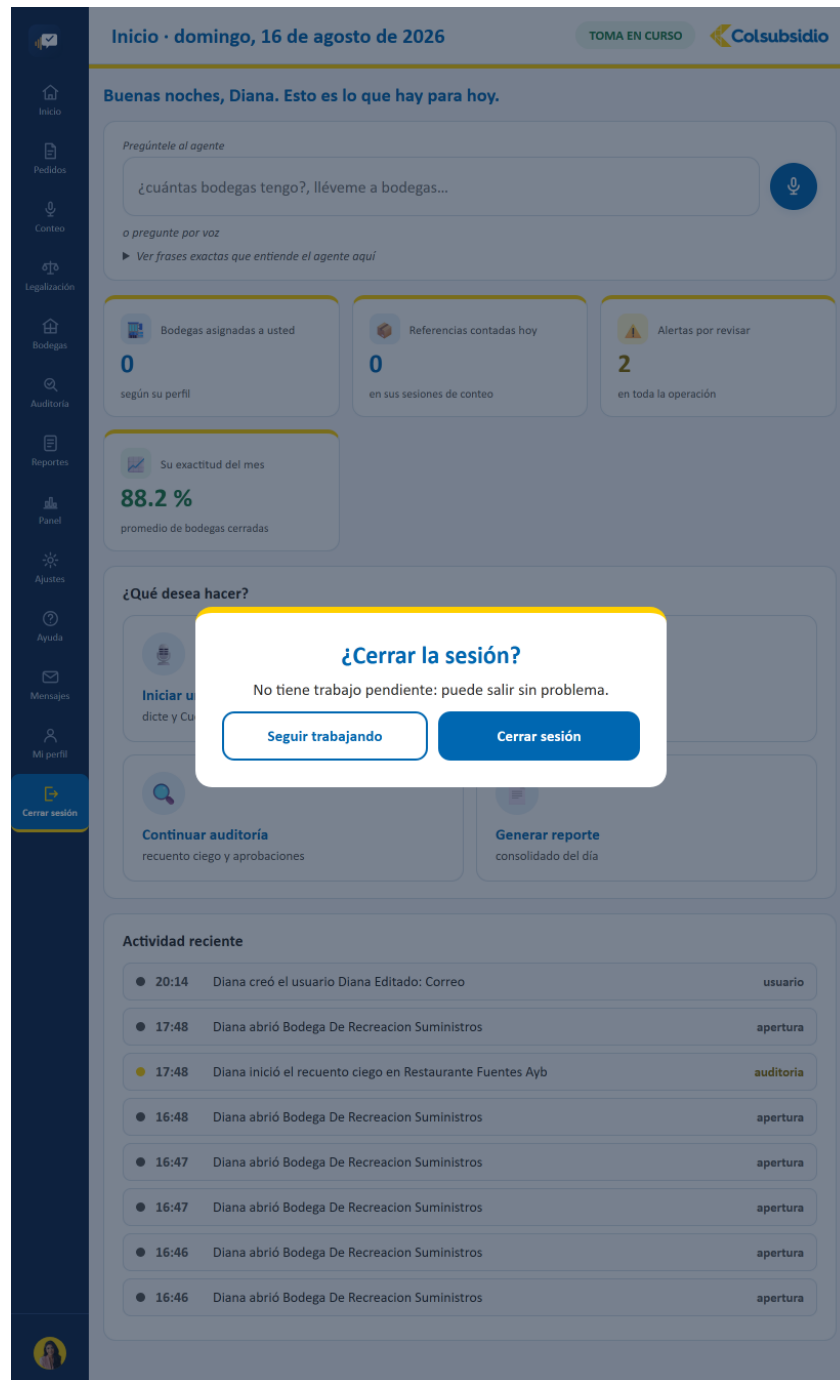


Figura 34. Ventana de confirmación al salir, captura real: avisa lo que quedaría sin confirmar y ofrece volver.

- **Qué tiene:** el aviso exacto de lo que hay abierto («1 bodega abierta con 2 referencias sin confirmar»), la promesa de que se guarda como borrador, y las dos salidas: seguir contando o cerrar sesión.
- **Por qué así:** en una bodega, salir por error con media hora de conteo encima sería el peor momento para perder datos. Por eso el cierre de sesión nunca es un botón silencioso: la sesión se cierra, pero el trabajo queda a salvo.

5b. El código del diseño adaptable

5b.0 Diseño adaptable: la misma aplicación en cualquier dispositivo

CuentaVoz corre en el navegador del dispositivo que haya disponible — tableta, celular o computador —, sin instalar nada. Todas las vistas de este capítulo son la misma aplicación web empresarial: una barra lateral de navegación, una barra superior con la sección activa y el área de trabajo. Sobre esa base, el código React reorganiza los bloques según el ancho disponible mediante cuatro puntos de quiebre (breakpoints), sin cambiar una sola línea de la lógica del agente:

Ancho de pantalla	Dispositivos típicos	Cómo se reorganiza la interfaz
Menos de 600 px	Celular en vertical	Una sola columna; la barra lateral se colapsa en un menú; el micrófono ocupa el centro y los botones se apilan en cuadrícula de dos por dos.
600 a 1023 px	Celular horizontal, tableta vertical	Dos columnas: conversación a la izquierda y micrófono a la derecha; barra lateral reducida a íconos.
1024 a 1279 px	Tableta horizontal (el equipo de bodega)	Disposición de referencia de las vistas 5.1 a 5.20.
1280 px o más	Computador de escritorio y portátil	Barra lateral completa con navegación y más datos visibles a la vez, sin desplazamiento.

La regla de uso es la misma en todos los formatos: la voz como camino rápido, el teclado como respaldo y botones grandes pensados para usarse de pie o con guantes. El esquema completo de las tres disposiciones está en la sección 5b, con su código.

Los cuatro formatos de la sección 5.21 nacen de tres herramientas en el frontend: la cuadrícula fluida de CSS Grid, cuatro reglas de @media que ajustan el layout al ancho y un único hook de React que detecta el dispositivo y oculta o muestra la barra lateral.

UN MISMO CÓDIGO, TRES DISPOSICIONES DE PANTALLA

El CSS reorganiza los bloques según el ancho disponible; la lógica del agente no cambia



Figura 35. Las tres disposiciones de pantalla: mismos bloques, distinta organización según el ancho disponible.

Nota: los fragmentos de esta sección (5b.1 a 5b.6) son un extracto simplificado con fines pedagógicos, para explicar la técnica sin el ruido del archivo completo — no una copia literal. El `index.css` y el `Conteo.jsx` reales, completos y actuales (con desambiguación, alertas, crear producto y el formulario sin conexión ya integrados) están en las secciones 12.24 y 12.28.

5b.1 Variables de diseño y tipografía

Archivo: `frontend/src/index.css` — variables de marca (parte 1)

```
/* Paleta oficial de CuentaVoz */
:root {
  --azul:      #0067B1; /* azul Colsubsidio */
  --azul-prof: #10294C;
  --azul-claro: #E4F0F9;
  --dorado:    #FFD000; /* amarillo Colsubsidio */
  --dorado-bg: #FFF6CC;
  --verde:     #1E7A46;
  --verde-bg:  #E8F5EE;
  --gris-txt:  #575756; /* grafito Colsubsidio */
  --gris-fondo: #EFEFEF;

  /* Tipografía — Calibri en Windows, Carlito en Linux */
  --fuente: "Calibri", "Carlito", "Liberation Sans", sans-serif;

  /* Espacios */
  --radio: 12px;
  --sombra: 0 2px 8px rgba(27,58,107,.12);
}

*, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: var(--fuente); font-size: 16px;
  color: var(--gris-txt); background: #fff; }
button { font-family: var(--fuente); cursor: pointer; border: none; }
```

5b.2 Cuadrícula principal y puntos de quiebre

Archivo: `frontend/src/index.css` — layout responsivo (parte 2)

```
/* — Layout base (tableta horizontal ≥ 1024 px) — */
.app-root {
  display: grid;
  grid-template-rows: 56px 4px 1fr; /* barra + faja dorada + contenido */
  height: 100dvh;
  overflow: hidden;
}

.contenido {
  display: grid;
  grid-template-columns: 1fr;
  overflow-y: auto;
  padding: 16px 18px;
  gap: 14px;
}

/* — Escritorio ≥ 1280 px: barra lateral */
@media (min-width: 1280px) {
  .app-root { grid-template-rows: 60px 4px 1fr; }
  .contenido {
    grid-template-columns: 200px 1fr;
  }
}
```

```

        padding: 20px;
    }
    .sidebar { display: flex; flex-direction: column; gap: 6px; }
}

/* — Tableta vertical 768-1023 px: sidebar colapsado */
@media (min-width: 768px) and (max-width: 1023px) {
    .sidebar { width: 56px; } /* solo íconos */
    .sidebar span { display: none; } /* oculta el texto */
}

/* — Móvil < 600 px: columna única */
@media (max-width: 599px) {
    .app-root { grid-template-rows: 48px 3px 1fr; }
    .contenido { padding: 12px 10px; gap: 10px; }
    .sidebar { display: none; } /* sin barra lateral */
    .barra-nombre { font-size: 15px; }
    .chip { font-size: 11px; padding: 3px 8px; }
}

```

5b.3 Botones responsivos

Archivo: **frontend/src/index.css — botones (parte 3)**

```

.btn {
    border-radius: var(--radio);
    font-weight: bold;
    padding: 14px 20px;
    font-size: 15px;
    transition: opacity .15s;
}
.btn:active { opacity: .82; }
.btn-primario { background: var(--azul); color: #fff; }
.btn-borde { background: #fff; color: var(--azul);
    border: 2px solid var(--azul); }
.btn-verde { background: var(--verde); color: #fff; }
.btn-dorado { background: var(--dorado); color: #fff; }

/* Móvil: botones más grandes y en cuadrícula 2x2 */
@media (max-width: 599px) {
    .btn { padding: 18px 14px; font-size: 14px; min-height: 56px; }
    .grilla-botones {
        display: grid;
        grid-template-columns: 1fr 1fr;
        gap: 10px;
    }
}

/* Tableta y escritorio: botones en fila */
@media (min-width: 600px) {
    .grilla-botones { display: flex; gap: 12px; flex-wrap: wrap; }
    .btn { padding: 13px 22px; }
}

```

5b.4 El micrófono: diferente en cada formato

Archivo: **frontend/src/index.css — micrófono (parte 4)**

```

.mic-btn {
    border-radius: 50%;
}

```

```

background: var(--azul);
color: #fff;
display: flex;
align-items: center;
justify-content: center;
box-shadow: var(--sombra);
user-select: none;
-webkit-user-select: none;
}

/* Tableta: micrófono en columna derecha, tamaño medio */
@media (min-width: 600px) and (max-width: 1279px) {
  .mic-btn { width: 96px; height: 96px; font-size: 36px; }
}

/* Móvil: micrófono central, toma el ancho disponible */
@media (max-width: 599px) {
  .mic-btn {
    width: min(72vw, 200px); height: min(72vw, 200px);
    font-size: 48px; margin: 16px auto;
  }
}

/* Escritorio: micrófono discreto en la barra superior */
@media (min-width: 1280px) {
  .mic-btn { width: 44px; height: 44px; font-size: 20px; }
}

```

5b.5 Hook de detección de dispositivo

Archivo: `frontend/src/hooks/useDispositivo.js`

```

import { useState, useEffect } from "react";

export function useDispositivo() {
  const [tipo, setTipo] = useState(detectar());

  function detectar() {
    const w = window.innerWidth;
    if (w < 600) return "movil";
    if (w < 1280) return "tableta";
    return "escritorio";
  }

  useEffect(() => {
    const obs = new ResizeObserver(() => setTipo(detectar()));
    obs.observe(document.documentElement);
    return () => obs.disconnect();
  }, []);

  return tipo;
}

```

5b.6 Cómo lo usa la pantalla de conteo

Archivo: `frontend/src/vistas/Conteo.jsx` — layout adaptable

```

import { useDispositivo } from "../../hooks/useDispositivo";

```

```

export default function Conteo({ sesion, token }) {
  const dispositivo = useDispositivo();

  return (
    <div className={`conteo conteo--${dispositivo}`} >
      { /* chips siempre en fila, se comprimen en movil */ }
      <div className="chips">
        <Chip tipo="primario">EN CONTEO</Chip>
        <Chip>Avance: {sesion.hechas} de {sesion.total}</Chip>
        <Chip tipo="verde">Alertas: {sesion.alertas}</Chip>
      </div>

      { /* burbuja siempre arriba */ }
      <Burbuja texto={respuesta} />

      { /*
        Móvil      → micrófono central, botones 2x2 abajo
        Tableta    → micrófono a la derecha en flex-row
        Escritorio → micrófono en la barra superior
      */ }
      {dispositivo === "movil" ? (
        <div className="movil-layout">
          <MicBoton grande onPress={escuchar} />
          <Registros items={registros} />
          <div className="grilla-botones">
            <button className="btn btn-verde" onClick={confirmar}>Confirmar</button>
            <button className="btn btn-borde" onClick={corregir}>Corregir</button>
            <button className="btn btn-borde" onClick={abrirTeclado}>Teclado</button>
            <button className="btn btn-dorado" onClick={pedirAyuda}>Ayuda</button>
          </div>
        </div>
      ) : (
        <div className="tableta-layout">
          <div className="izquierda">
            <Burbuja texto={respuesta} />
            <Registros items={registros} />
          </div>
          <div className="derecha">
            <MicBoton onPress={escuchar} />
          </div>
          <div className="grilla-botones">
            <button className="btn btn-verde" onClick={confirmar}>Confirmar</button>
            <button className="btn btn-borde" onClick={corregir}>Corregir</button>
            <button className="btn btn-borde" onClick={abrirTeclado}>Teclado</button>
            <button className="btn btn-dorado" onClick={pedirAyuda}>Ayuda</button>
          </div>
        </div>
      )}
    </div>
  );
}

```

Regla de oro del diseño adaptable de CuentaVoz: el CSS reorganiza; el código de negocio no cambia. La frase dictada siempre va al mismo endpoint, la validación siempre ocurre en el backend y la voz siempre responde — solo el tamaño del botón y la posición del micrófono se adaptan al dispositivo que tenga la persona en la mano.

6. Cómo piensa el agente: intención y salida estructurada

El «pensar» se vuelve construible cuando el modelo responde siempre en un formato fijo. En cada turno, Gemini clasifica la frase en una de ocho intenciones —contar, corregir, consultar, explicar, crear, navegar, ayuda, cerrar— y extrae los datos. Las instrucciones del sistema se envían como `system_instruction` y la respuesta se fuerza a JSON con `response_mime_type`, dos capacidades nativas de la API de Gemini (el código real está en la sección 12.9):

6.1 Instrucciones del sistema (prompt del agente)

```
Eres CuentaVoz, el asistente de inventarios de Colsubsidio.
Hablas español claro, en frases cortas, con respeto y calidez.

Reglas:
1. Nunca inventes artículos: usa solo los que devuelva buscar_articulo.
2. Antes de guardar, repite siempre artículo, cantidad y unidad.
3. Con varios candidatos, lee las opciones y deja que la persona elija.
4. Si validar_conteo devuelve una alerta, pregunta antes de continuar.
   Nunca guardes un dato con alerta sin confirmación explícita.
5. Si la persona no entiende algo (una tabla, un reporte, un paso),
   explícalo con palabras simples y ofrece leer solo las diferencias.
6. Si piden datos, usa consultar_stock y resume en voz; ofrece
   generar_reporte si quieren el archivo o imprimirlo.
7. Si no puedes resolver un problema, dilo con honestidad y ofrece
   llamar al administrador de bodega.

Responde SOLO con el JSON del esquema acordado.
```

6.2 Esquema de salida (el JSON que el backend procesa)

```
{
  "intencion": "contar",
  "articulo_texto": "tabla para picar blanca",
  "cantidad": 3,
  "unidad": "Unidad",
  "herramienta": {
    "nombre": "buscar_articulo",
    "argumentos": { "texto": "tabla para picar blanca" }
  },
  "respuesta_hablada": "Tengo dos blancas: la 50 por 38 FB o la
    P barra picar. ¿Cuál de las dos?"
}
```

Con este contrato, la tableta solo reproduce `respuesta_hablada`, el orquestador solo ejecuta `herramienta`, y cada integrante del equipo programa su pieza sin ambigüedades.

7. Las manos del agente: herramientas del backend

Cada herramienta es un servicio con entradas y salidas definidas; son el único camino del agente hacia los datos. La tercera columna indica dónde vive su código en la sección 12:

Herramienta	Recibe → Devuelve	Código
abrir_bodega	nombre de la bodega, usuario → sesión bloqueada + total de referencias	12.12
buscar_articulo	texto libre dictado → candidatos oficiales con código y confianza	12.7
validar_conteo	artículo, cantidad, unidad, bodega → ok, o alerta con motivo	12.8
guardar_conteo	captura confirmada → registro con trazabilidad (usuario, hora)	12.10
corregir_conteo	id o «el último», valor nuevo → registro corregido con rastro	12.10
consultar_stock	filtros (artículo, bodega, solo diferencias) → datos + resumen en voz	12.10 / 12.12
explicar_reporte	tabla en pantalla → significado de columnas y filas clave	12.10
avance_bodega	bodega activa → contadas, pendientes, alertas abiertas	12.10
crear_articulo / crear_bodega	nombre, unidad, bodega → registro «pendiente de aprobación»	12.10
generar_reporte	alcance y formato → archivo XLSX/CSV listo para descargar o imprimir	12.11
cerrar_bodega	bodega, perfil → comparativo 1 vs. 2 vs. sistema + cambio de estado	12.12

8. Arquitectura: cómo queda construido y cómo se conecta

8.1 Las cinco capas



Figura 36. Arquitectura de capas; cada capa solo conversa con la vecina. En la capa de agente, el LLM es Gemini vía Google AI Studio.

8.2 Componentes y conexiones: quién habla con quién

Este es el mapa físico de la construcción. Cada caja es un componente que se programa por separado (y tiene su archivo en la sección 12), y cada flecha es un cable concreto que hay que dejar funcionando:

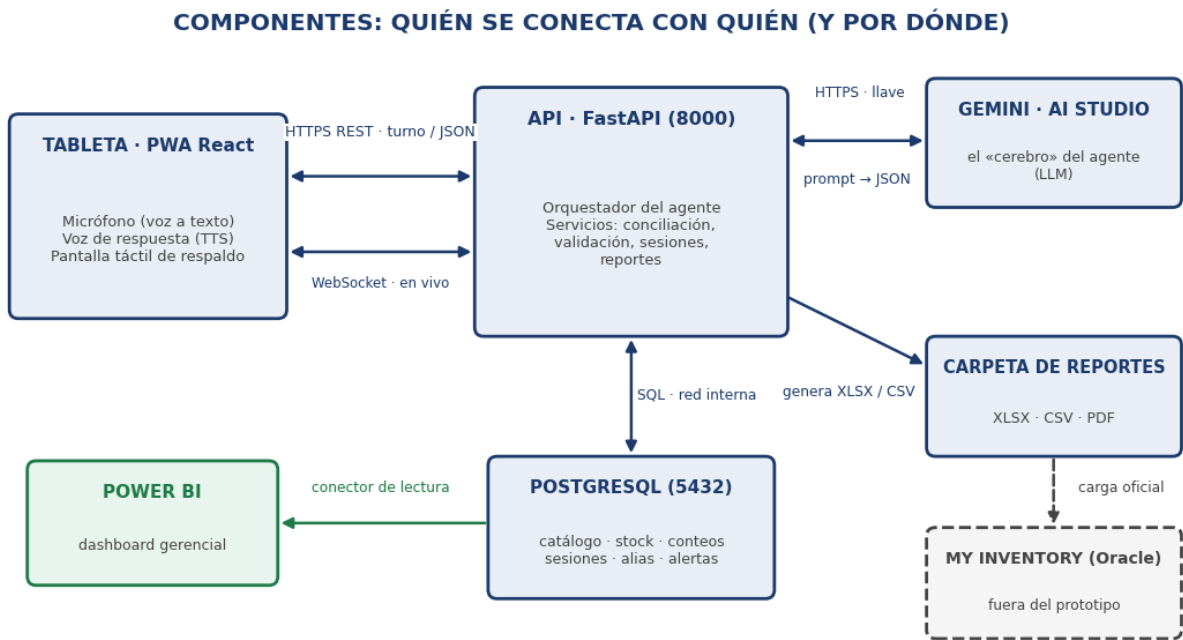


Figura 37. Componentes y canales; la línea punteada hacia My Inventory es el procedimiento oficial de carga, fuera del prototipo.

Conexión (origen → destino)	Canal	Qué viaja y dónde está el código
Tableta → API	HTTPS REST, puerto 8000	la frase transcrita y la sesión; de vuelta, el JSON del turno — código 12.13
Tableta ↔ API	WebSocket	estado en vivo de las 54 bodegas para el tablero — código 12.12 y 12.14
API → Gemini (Google AI Studio)	HTTPS (internet), llave GOOGLE_API_KEY	system_instruction + contexto + frase; de vuelta, el JSON de intención — código 12.9
API → PostgreSQL	SQL, puerto 5432 (red interna de Docker)	lecturas de catálogo/stock y escritura de conteos con trazabilidad — código 12.4 a 12.8
API → carpeta de reportes	sistema de archivos	XLSX/CSV generados para descargar o imprimir — código 12.11
Reportes → My Inventory	procedimiento oficial de carga (fuera del prototipo)	el consolidado con los nombres y códigos oficiales de la base

8.3 Secuencia de un turno completo

Y así se ve, mensaje por mensaje, el recorrido de una frase real a través de todos los componentes:

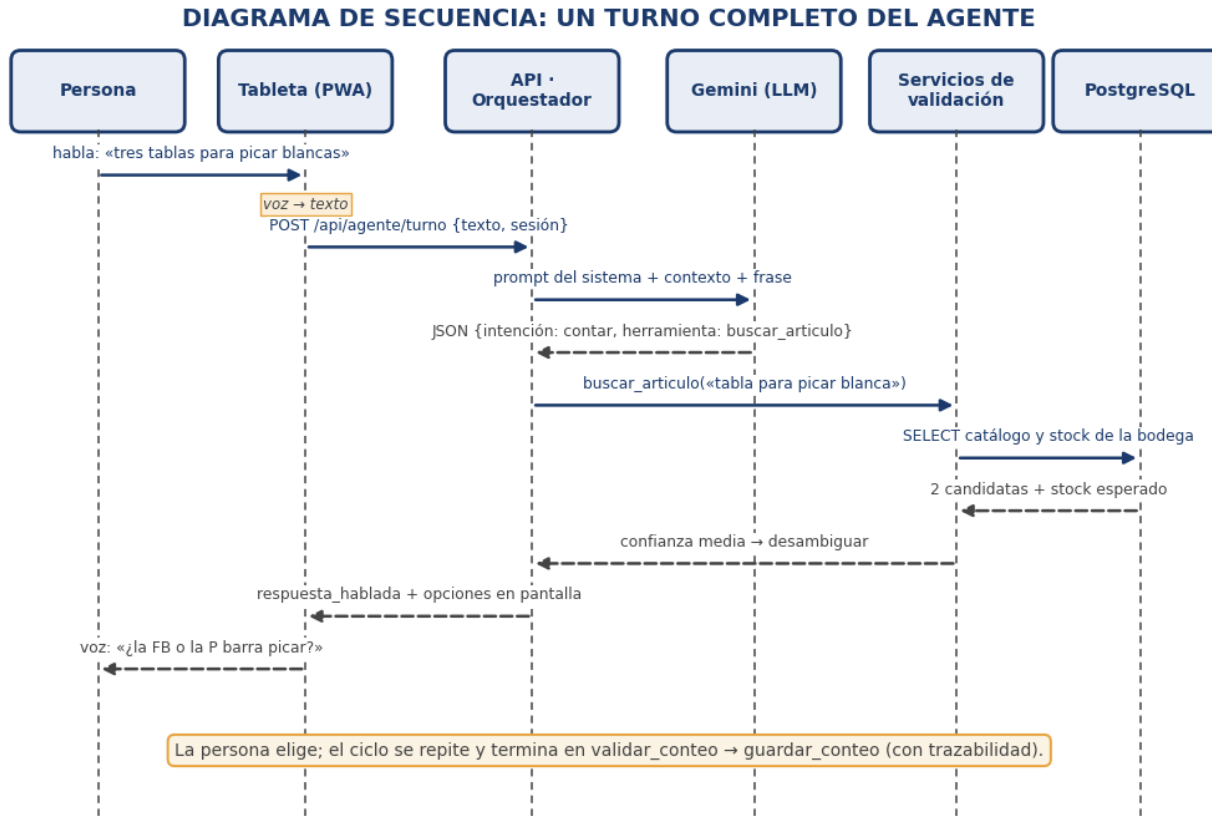
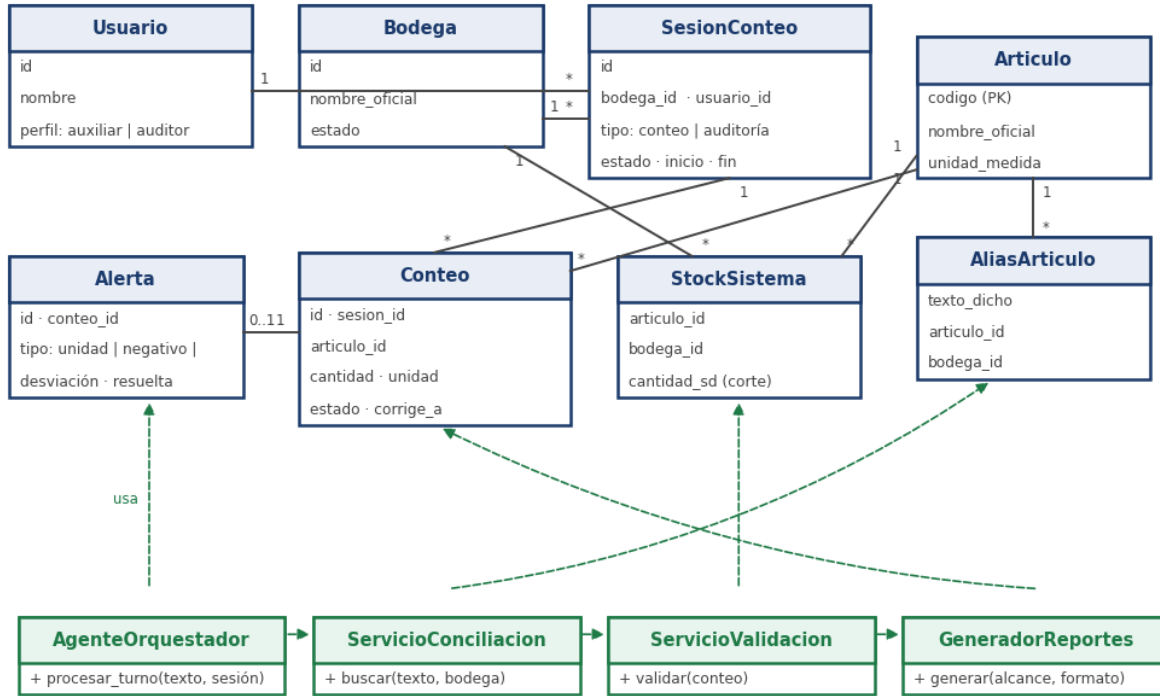


Figura 38. Secuencia de un turno: las flechas azules son llamadas y las punteadas, respuestas.

9. Diagrama de clases y modelo de datos

Las entidades del diagrama son, a la vez, las tablas de PostgreSQL: quien programe el backend crea exactamente estas clases (el código está en 12.6). Abajo, en verde, la capa de servicios que las usa:

DIAGRAMA DE CLASES Y MODELO DE DATOS (entidades = tablas de la base)

Verde: capa de servicios (usa las entidades). Las flechas punteadas indican dependencia «usa».

Figura 39. Clases y relaciones; las multiplicidades (1, *) se leen, por ejemplo: una sesión tiene muchos conteos.

- **Lectura rápida:** una Bodega tiene muchas SesionConteo (una por conteo inicial y otra por auditoría); cada SesionConteo tiene muchos Conteo; cada Conteo apunta a un Articulo del catálogo oficial y puede tener 0 o 1 Alerta; StockSistema guarda la cantidad del corte por artículo y bodega (el extracto de My Inventory); AliasArticulo acumula las formas reales de decir cada artículo que el agente va aprendiendo.
- **Campo clave para la auditoría:** Conteo.corrige_a enlaza una corrección con el registro original: nada se borra, todo queda trazable.

10. Estados de la bodega y trabajo en paralelo

ESTADOS DE UNA BODEGA (varias bodegas avanzan en paralelo, cada una con su sesión)



Figura 40. Estados de una bodega; el candado de sesión evita conteos duplicados y permite bodegas en paralelo.

El estado vive en `SesionConteo` y se difunde por el `WebSocket` al tablero (vista 5.13): cuando el auxiliar abre una bodega, todos ven «en conteo» al instante; cuando el auditor cierra, la bodega pasa a «cerrada» con la doble firma registrada.

11. Conversaciones de ejemplo (guiones implementables)

La columna del medio es lo que ocurre por dentro (Pensar–Actuar–Validar); la persona solo escucha la columna derecha. Cifras ilustrativas.

11.1 Conteo con conciliación de nombre y anomalía

La persona dice	El agente piensa y hace	El agente responde
«Iniciar conteo en almacén de suministros.»	navegar → abrir_bodega: sesión bloqueada; 312 referencias.	«Listo, almacén de suministros abierto. Dícteme el primer producto cuando quiera.»
«Tres tablas para picar blancas.»	contar → buscar_articulo: dos candidatas blancas; confianza media.	«Tengo dos blancas: la acrílica 50 por 38 FB o la P barra picar 50 por 38. ¿Cuál de las dos?»
«La FB.»	código 97503004 → validar_conteo(3): ok.	«Tabla acrílica picar blanco 50 por 38 FB: tres unidades. ¿Confirma?»
«Confirмо. Ahora noventa cazuelas de dieciséis onzas.»	guardar_conteo; validar_conteo(90): alerta, el sistema espera ~10.	«Guardado. Ojo con las cazuelas: el sistema espera alrededor de diez y usted dijo noventa. ¿Las contamos otra vez o confirma noventa?»
«Uy, no: son nueve.»	corregir → cantidad 9 → validar: ok.	«Nueve cazuelas de dieciséis onzas. ¿Confirma?»
«Confirмо. ¿Cuánto me falta?»	guardar_conteo → avance_bodega: 2 de 312.	«Guardado. Llevamos dos referencias y faltan trescientas diez. Siga cuando quiera.»

11.2 «No entiendo esta tabla»: el agente explica el reporte

La persona dice	El agente piensa y hace	El agente responde
«Oiga, no entiendo esta tabla del reporte.»	explicar → explicar_reporte(pantalla actual).	«Con gusto. Cada fila es un producto de esta bodega. “Contado” es lo que usted dictó; “Sistema” es lo que My Inventory esperaba; “Diferencia” es la resta. Si sale resaltada, hay que revisarla. ¿Le leo solo las filas con diferencia?»
«Sí, solo esas.»	consultar_stock(solo diferencias): 3 filas.	«Son tres: arroz, faltan dos kilos; cazuela de dieciséis onzas, cuadró con su corrección; y aceite, falta un litro. ¿Las revisamos una por una?»
«¿Y por qué el aceite sale en negativo abajo?»	explicar → regla del mini reto de negativos.	«Ese negativo no viene de su conteo: el sistema traía -1 desde antes, seguramente una salida registrada sin la entrada. Su conteo físico es el que vale; el reporte lo marca para el administrador.»

11.3 Consulta de datos con salida impresa

La persona dice	El agente piensa y hace	El agente responde
«¿Cuánto arroz hay en total y en qué bodegas? Y me lo imprime.»	consultar → consultar_stock(«arroz», por bodega) → generar_reporte(XLSX).	«Arroz hay en cuatro bodegas: almacén AyB ciento veinte kilos, restaurante Fuentes cuarenta y cinco, kiosco taquilla doce y Piscigiros ocho: ciento ochenta y cinco en total. Ya generé el archivo: quedó en la carpeta de reportes, listo para imprimir. ¿Algo más?»
«Mándele también el Excel al administrador.»	generar_reporte(XLSX) → sesión del auditor + notificación.	«Hecho: el Excel ya aparece en la tableta del administrador de bodega, con la fecha y la hora del corte.»

12. Código completo del prototipo

Esta sección contiene el código completo y actual de CuentaVoz —el mismo que corre hoy en <https://cuentavoz.onrender.com>—, archivo por archivo. Cada archivo trae una nota de qué hace y con qué se conecta. El prototipo del hackathon creció a una plataforma real: donde la versión anterior de este manual cubría unos 15 archivos, esta cubre los cerca de 45 que forman hoy el backend y el frontend, incluidas piezas que no existían entonces (WebAuthn, el intérprete local en JavaScript para el modo sin conexión, recetas administrables, Mensajes, la hora de Bogotá y el despliegue en Render).

12.1 Dependencias y contenedores

Archivo: `backend/requirements.txt`

```
fastapi==0.115.6
uvicorn[standard]==0.34.0
sqlalchemy==2.0.36
psycogp2-binary==2.9.10
pandas==2.2.3
openpyxl==3.1.5
rapidfuzz==3.10.1
google-generativeai==0.3.0
python-dotenv==1.0.1
pyjwt==2.10.1
bcrypt==4.2.1
python-multipart==0.0.20
slowapi==0.1.9
webauthn==3.0.0
tzdata==2026.3
```

Archivo: `backend/Dockerfile`

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Archivo: `frontend/Dockerfile`

```
FROM node:20-slim

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

RUN npm run build

RUN npm install -g serve

EXPOSE 3000

CMD ["serve", "-s", "dist", "-l", "3000"]
```

Archivo: `frontend/package.json` — React 18 + Vite 6 + vite-plugin-pwa (service worker, ver 12.24).

```
{
  "name": "cuentavoz-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "node --test src/*.test.js"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  }
}
```

```

  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^6.0.7",
    "vite-plugin-pwa": "^1.3.0"
  }
}

```

12.2 Variables de entorno

Archivo: .env.ejemplo — Plantilla real; cópiese a .env y llénense las llaves. Incluye las variables de WebAuthn (5) que no existían en la versión anterior del manual.

```

# — Copie este archivo como .env y ponga su llave —

# 1) LLAVE DE GEMINI (gratis en aistudio.google.com → Get API key)
# Sin ella la app funciona igual con el intérprete local,
# pero con ella el agente entiende muchas más variantes de frase.
GOOGLE_API_KEY=
MODELO=gemini-flash-latest

# 2) Base de datos (docker-compose la sobrescribe; para modo local déjela así)
DB_URL=sqlite:///cuentavoz.db

# 3) Reglas del negocio
IDIOMA_VOZ=es-CO
UMBRAL_ANOMALIA=0.5          # 0.5 = 50 % de desviación dispara la alerta

# 4) Seguridad
SECRETO_JWT=cambie-esto-por-una-frase-larga-y-dificil
MINUTOS_TOKEN=480
# uno o varios orígenes separados por coma (el/los dominio(s) del frontend)
ORIGEN_PERMITIDO=http://localhost:5173

# 5) Ingreso con huella (WebAuthn) - el dominio debe coincidir exacto con
# el que usa el navegador (sin esquema ni puerto en RP_ID) y el origen
# completo (con esquema) en WEBAUTHN_ORIGENES. En producción es el
# dominio del frontend en Render, ej. WEBAUTHN_RP_ID=cuentavoz-web.onrender.com
WEBAUTHN_RP_ID=localhost
WEBAUTHN_RP_NAME=CuentaVoz
WEBAUTHN_ORIGENES=http://localhost:5173

```

12.3 Orquestación con Docker (uso local)

Archivo: docker-compose.yml

```

services:
  db:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: cuentavoz
      POSTGRES_PASSWORD: cambieme
      POSTGRES_DB: cuentavoz
    volumes:
      - datos:/var/lib/postgresql/data
    ports:
      - "5432:5432"          # para conectarse con un cliente de PostgreSQL en local
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U cuentavoz"]
      interval: 10s
      timeout: 5s

```



```

    retries: 5

api:
  build: ./backend
  restart: always
  env_file: .env
  environment:
    DB_URL: postgresql://cuentavoz:cambieme@db:5432/cuentavoz
  depends_on:
    db:
      condition: service_healthy
  ports:
    - "8000:8000"
  volumes:
    - ./reportes:/app/reportes
    - ./backend/fotos:/app/fotos

web:
  build: ./frontend
  restart: always
  environment:
    VITE_API_URL: http://localhost:8000
  depends_on:
    - api
  ports:
    - "5173:5173"

volumes:
  datos: {}

```

12.4 Despliegue en producción: Render

Archivo: `render.yaml` — El Blueprint real con el que corre <https://cuentavoz.onrender.com>: PostgreSQL + Web Service (backend) + Static Site (frontend). Paso a paso completo en `DESPLIEGUE.md`.

```

# Blueprint de Render para CuentaVoz: un Web Service (API FastAPI) y un
# Static Site (frontend Vite ya compilado) desplegados por separado, mas la
# base de datos gestionada. Un "Static Site" en Render solo puede servir
# archivos; por eso el backend va como Web Service aparte y el frontend le
# habla por HTTPS via VITE_API_URL. Ver DESPLIEGUE.md para el paso a paso.

databases:
  - name: cuentavoz-db
    plan: free
    databaseName: cuentavoz
    user: cuentavoz

services:
  - type: web
    name: cuentavoz-api
    env: python
    plan: free
    rootDir: backend
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /api/salud
    envVars:
      - key: PYTHON_VERSION
        value: "3.12.7" # sin esto Render usa Python 3.13, donde rapidfuzz
                        # no tiene wheel precompilado y falla al compilar
      - key: DB_URL
        fromDatabase:
          name: cuentavoz-db

```

```

    property: connectionString
  - key: SECRETO_JWT
    generateValue: true
  - key: MINUTOS_TOKEN
    value: "480"
  - key: IDIOMA_VOZ
    value: es-CO
  - key: UMBRAL_ANOMALIA
    value: "0.5"
  - key: MODELO
    value: gemini-flash-latest
  - key: GOOGLE_API_KEY
    sync: false      # se pega a mano en el dashboard: la llave es secreta
  - key: ORIGEN_PERMITIDO
    sync: false      # URL del static site una vez exista, ver DESPLIEGUE.md
  - key: WEBAUTHN_RP_ID
    sync: false      # dominio del static site SIN esquema ni puerto, ver DESPLIEGUE.md
  - key: WEBAUTHN_RP_NAME
    value: CuentaVoz
  - key: WEBAUTHN_ORIGENES
    sync: false      # URL completa del static site (con https://), ver DESPLIEGUE.md

- type: web
  name: cuentavoz
  env: static
  rootDir: frontend
  buildCommand: npm install && npm run build
  staticPublishPath: dist
  envVars:
    - key: VITE_API_URL
      sync: false      # URL del cuentavoz-api una vez exista, ver DESPLIEGUE.md
  routes:
    - type: rewrite
      source: /*
      destination: /index.html

```

12.5 Conexión a la base de datos

Archivo: **backend/bd.py**

```

"""Conexión a la base de datos. Todo lo demás importa de aquí."""
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

class Base(DeclarativeBase):
    pass

# Ruta absoluta anclada a backend/, para que sqlite apunte siempre al mismo
# archivo sin importar desde que carpeta se ejecute (uvicorn corre desde
# backend/, pero cargar_excel.py corre desde data/).
_RUTA_SQLITE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "cuentavoz.db")
DB_URL = os.getenv("DB_URL", f"sqlite:/// {_RUTA_SQLITE}")
# Render (como antes Heroku) entrega la cadena de Postgres con el esquema
# viejo "postgres://"; SQLAlchemy 2.x ya no lo traduce solo y falla al
# arrancar si no se corrige aquí.
if DB_URL.startswith("postgres://"):
    DB_URL = DB_URL.replace("postgres://", "postgresql://", 1)

# SQLite necesita este argumento extra cuando hay varios hilos (uvicorn)

```

```

kwargs = {"connect_args": {"check_same_thread": False}} if DB_URL.startswith("sqlite") else {}
motor = create_engine(DB_URL, **kwargs)
Sesion = sessionmaker(bind=motor, expire_on_commit=False)

def iniciar_bd():
    import modelos # noqa: F401 (registra todas las tablas)
    Base.metadata.create_all(motor)
    _migrar_columnas_faltantes()

def _migrar_columnas_faltantes():
    """create_all() solo crea tablas nuevas, nunca agrega columnas a una
    tabla que ya existe. Sin esto, una columna agregada al modelo (como
    Usuario.foto) se ve localmente porque cuentavoz.db se recrea facil,
    pero nunca llega a la base de Postgres ya desplegada en Render - hay
    que agregarla a mano en la tabla que ya esta viva. Corre en cada
    arranque y no hace nada si ya esta al dia."""
    from sqlalchemy import inspect, text
    inspector = inspect(motor)
    with motor.begin() as conexion:
        for tabla in Base.metadata.sorted_tables:
            if not inspector.has_table(tabla.name):
                continue
            existentes = {c["name"] for c in inspector.get_columns(tabla.name)}
            for columna in tabla.columns:
                if columna.name in existentes:
                    continue
                tipo = columna.type.compile(dialect=motor.dialect)
                conexion.execute(text(
                    f'ALTER TABLE {tabla.name} ADD COLUMN {columna.name} {tipo}'))
                print(f"[bd] columna agregada: {tabla.name}.{columna.name}")

```

12.6 Las tablas (el diagrama de clases hecho código)

Archivo: backend/modelos.py — 19 tablas — 7 más que en la versión anterior de este manual: AsignacionBodega, Receta/RecetaIngrediente/LineaServicio, Aprobacion, ArchivoGenerado, MensajeSoporte y CredencialWebAuthn.

```

"""Las tablas del sistema: el diagrama de clases hecho código."""
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, LargeBinary
from bd import Base
from horario import ahora

class Usuario(Base):
    __tablename__ = "usuario"
    id = Column(Integer, primary_key=True)
    nombre = Column(String, nullable=False)
    perfil = Column(String, nullable=False) # auxiliar | auditor
    clave_hash = Column(String, nullable=False)
    correo = Column(String, default="")
    telefono = Column(String, default="")
    codigo = Column(String, default="")
    activo = Column(Integer, default=1)
    version_token = Column(Integer, default=0) # +1 invalida sesiones anteriores
    huella_registrada = Column(Integer, default=0) # tiene al menos una CredencialWebAuthn
    pin_actualizado = Column(DateTime, default=ahora)
    ultimo_acceso = Column(DateTime, nullable=True)
    idioma_voz = Column(String, default="es-MX")
    velocidad_voz = Column(String, default="normal") # lenta | normal | rapida
    confirmacion_hablada = Column(Integer, default=1)
    # la foto va en la misma base (no en disco): el disco del Web Service
    # en Render es efimero y se borra en cada deploy/reinicio, la base no.

```

```

foto = Column(LargeBinary, nullable=True)
foto_tipo = Column(String, nullable=True)          # ej. "image/jpeg"

class AsignacionBodega(Base):
    """Qué bodegas puede contar cada persona."""
    __tablename__ = "asignacion_bodega"
    id = Column(Integer, primary_key=True)
    usuario_id = Column(Integer, ForeignKey("usuario.id"))
    bodega_id = Column(Integer, ForeignKey("bodega.id"))

class Bodega(Base):
    __tablename__ = "bodega"
    id = Column(Integer, primary_key=True)
    nombre_oficial = Column(String, unique=True, nullable=False)
    estado = Column(String, default="pendiente")      # pendiente | en_conteo | en_auditoria | cerrada

class Articulo(Base):
    __tablename__ = "articulo"
    codigo = Column(String, primary_key=True)
    nombre_oficial = Column(String, nullable=False)
    unidad_medida = Column(String, nullable=False)

class StockSistema(Base):
    """El extracto de My Inventory: lo que el sistema cree que hay."""
    __tablename__ = "stock_sistema"
    id = Column(Integer, primary_key=True)
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    bodega_id = Column(Integer, ForeignKey("bodega.id"))
    cantidad_sd = Column(Float, default=0)

class AliasArticulo(Base):
    """Cómo habla el equipo de verdad. El agente lo va aprendiendo."""
    __tablename__ = "alias_articulo"
    id = Column(Integer, primary_key=True)
    texto_dicho = Column(String, index=True)
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    bodega_id = Column(Integer, nullable=True)

class SesionConteo(Base):
    __tablename__ = "sesion_conteo"
    id = Column(Integer, primary_key=True)
    bodega_id = Column(Integer, ForeignKey("bodega.id"))
    usuario_id = Column(Integer, ForeignKey("usuario.id"))
    tipo = Column(String, default="conteo")          # conteo | auditoria
    estado = Column(String, default="abierta")
    firmada = Column(Integer, default=0)              # firma digital de cierre
    inicio = Column(DateTime, default=ahora)
    fin = Column(DateTime, nullable=True)

class Conteo(Base):
    __tablename__ = "conteo"
    id = Column(Integer, primary_key=True)
    sesion_id = Column(Integer, ForeignKey("sesion_conteo.id"))
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))

```

```

cantidad = Column(Float, nullable=False)
unidad = Column(String)
estado = Column(String, default="confirmado")
# confirmado | corregido | pendiente_aprobacion | borrador
corrige_a = Column(Integer, ForeignKey("conteo.id"), nullable=True)
creado = Column(DateTime, default=ahora)

class Alerta(Base):
    __tablename__ = "alerta"
    id = Column(Integer, primary_key=True)
    conteo_id = Column(Integer, ForeignKey("conteo.id"), nullable=True)
    tipo = Column(String) # unidad | negativo | desviacion | inexistente
    detalle = Column(String)
    resuelta = Column(Integer, default=0)
    creado = Column(DateTime, default=ahora)
    resuelto = Column(DateTime, nullable=True)

class Traza(Base):
    """Registro inmutable: solo crece, nunca se edita ni se borra."""
    __tablename__ = "traza"
    id = Column(Integer, primary_key=True)
    usuario_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
    persona = Column(String)
    accion = Column(String) # APERTURA | CIERRE | CORRECCION | ALERTA...
    detalle = Column(String)
    tipo = Column(String, default="info")
    creado = Column(DateTime, default=ahora)

# — los tres momentos del reto: recetas y servicios —
class Receta(Base):
    __tablename__ = "receta"
    id = Column(Integer, primary_key=True)
    nombre = Column(String, nullable=False)
    rendimiento = Column(Integer, default=1)
    # paso a paso de cocina, no solo la lista de ingredientes con su cantidad.
    preparacion = Column(String, default="")

class RecetaIngrediente(Base):
    __tablename__ = "receta_ingredient"
    id = Column(Integer, primary_key=True)
    receta_id = Column(Integer, ForeignKey("receta.id"))
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    cantidad_por_porcion = Column(Float, nullable=False)

class LineaServicio(Base):
    __tablename__ = "linea_servicio"
    id = Column(Integer, primary_key=True)
    servicio_id = Column(Integer, default=1)
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"))
    nombre = Column(String)
    pedido = Column(Float, default=0)
    usado = Column(Float, default=0)
    # abierto | legalizado | pendiente_aprobacion | rechazado
    estado = Column(String, default="abierto")
    plato = Column(String, default="")
    porciones = Column(Integer, default=0)
    creado = Column(DateTime, default=ahora)
    bodega_id = Column(Integer, ForeignKey("bodega.id"), nullable=True)

```

```

    creado_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
    # agrupa las lineas de un mismo envio para poder aprobarlo/rechazarlo
    # como una sola unidad, no linea por linea.
    numero_pedido = Column(String, nullable=True)

# — aprobación en paralelo: productos y bodegas creados en plena toma —
class Aprobacion(Base):
    __tablename__ = "aprobacion"
    id = Column(Integer, primary_key=True)
    tipo = Column(String, nullable=False)           # producto | bodega
    nombre = Column(String, nullable=False)
    unidad_medida = Column(String, nullable=True)
    cantidad_inicial = Column(Float, nullable=True)
    bodega_id = Column(Integer, ForeignKey("bodega.id"), nullable=True)
    articulo_codigo = Column(String, ForeignKey("articulo.codigo"), nullable=True)
    conteo_id = Column(Integer, ForeignKey("conteo.id"), nullable=True)
    creado_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
    estado = Column(String, default="pendiente")    # pendiente | aprobado | rechazado
    creado = Column(DateTime, default=ahora)
    resuelto_por_id = Column(Integer, ForeignKey("usuario.id"), nullable=True)
    resuelto = Column(DateTime, nullable=True)

class HistorialCierre(Base):
    """Una foto de la exactitud cada vez que una bodega cierra con doble firma."""
    __tablename__ = "historial_cierre"
    id = Column(Integer, primary_key=True)
    bodega_id = Column(Integer, ForeignKey("bodega.id"))
    exactitud = Column(Float, default=100)
    referencias = Column(Integer, default=0)
    diferencias = Column(Integer, default=0)
    fecha = Column(DateTime, default=ahora)

class ConfigClave(Base):
    """Ajustes editables desde la aplicación, con el .env como valor por defecto."""
    __tablename__ = "config_clave"
    clave = Column(String, primary_key=True)
    valor = Column(String, nullable=False)

class ArchivoGenerado(Base):
    """Los XLSX/CSV que se generan (consolidado, diferencias, análisis,
    trazabilidad...) guardados en la base, no en disco - mismo motivo que
    la foto de perfil: el disco del Web Service en Render es efímero y se
    borra en cada deploy/reinicio, así que un archivo "generado hace una
    hora" debe poder seguir viéndose/descargándose después de un redeploy,
    no solo hasta el próximo `git push`. """
    __tablename__ = "archivo_generado"
    id = Column(Integer, primary_key=True)
    ruta = Column(String, nullable=False, unique=True)
    contenido = Column(LargeBinary, nullable=False)
    creado = Column(DateTime, default=ahora)

class MensajeSoporte(Base):
    """Un mensaje de "Soporte en vivo" (auxiliar -> administrador) con su
    respuesta, si ya la dio - la bandeja dentro de la app en Ayuda,
    aparte del correo real que tambien se manda por fuera (EmailJS)."""
    __tablename__ = "mensaje_soporte"
    id = Column(Integer, primary_key=True)
    remitente_id = Column(Integer, ForeignKey("usuario.id"), nullable=False)

```

```

destinatario_id = Column(Integer, ForeignKey("usuario.id"), nullable=False)
mensaje = Column(String, nullable=False)
respuesta = Column(String, nullable=True)
creado = Column(DateTime, default=ahora)
respondido = Column(DateTime, nullable=True)

class CredencialWebAuthn(Base):
    """Una llave de acceso (huella, Windows Hello, etc.) registrada en un
    dispositivo para un usuario - el ingreso real por biometria via WebAuthn,
    no la simulacion anterior."""
    __tablename__ = "credencial_webauthn"
    id = Column(Integer, primary_key=True)
    usuario_id = Column(Integer, ForeignKey("usuario.id"), nullable=False)
    credential_id = Column(String, nullable=False, unique=True) # base64url
    public_key = Column(String, nullable=False) # base64url
    sign_count = Column(Integer, default=0)
    dispositivo = Column(String, default="") # nombre amigable, ej. "Edge en este equipo"
    creado = Column(DateTime, default=ahora)

```

12.7 Hora de Bogotá para toda la aplicación

Archivo: backend/horario.py — Render corre en UTC; sin esto, cada hora guardada o mostrada queda adelantada 5 horas — bug real ya corregido en producción.

```

"""La hora de Bogotá para toda la aplicación.

El servidor (Render) corre en UTC, pero CuentaVoz es una sola operación
en Colombia: lo que se guarda en la base (creado, resuelto, inicio, fin...)
y lo que se muestra (trazabilidad, alertas, reportes) debe leerse en hora
de Bogotá, no en la del contenedor - si no, cada hora mostrada queda
adelantada 5 horas (confirmado en producción: un reporte generado a las
2:56pm quedó con el nombre de archivo en 19:56)."""
from datetime import datetime
from zoneinfo import ZoneInfo

_BOGOTA = ZoneInfo("America/Bogota")

def ahora() -> datetime:
    """La hora de Bogotá ahora mismo, sin info de zona (naive) - para
    seguir siendo comparable con las columnas datetime ya guardadas y con
    el resto del código, que nunca maneja zonas horarias explícitas."""
    return datetime.now(_BOGOTA).replace(tzinfo=None)

```

12.8 Carga del extracto real de Colsubsidio

Archivo: data/cargar_excel.py

```

"""Carga el extracto real de Colsubsidio y unas recetas de ejemplo.

Uso: cd data && python cargar_excel.py
"""
import os
import re
import sys
import unicodedata
import pandas as pd

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "backend"))

```

```

from bd import Sesion, iniciar_bd # noqa: E402
from modelos import (Bodega, Articulo, StockSistema, # noqa: E402
                    Receta, RecetaIngrediente, LineaServicio)

ARCHIVO = os.path.join(os.path.dirname(__file__), "BODEGAS_Y_STOCK.xlsx")

def _norm(t: str) -> str:
    """Compara nombres de bodega sin tildes ni variantes de A y B."""
    t = str(t).upper()
    t = "".join(c for c in unicodedata.normalize("NFD", t)
                if unicodedata.category(c) != "Mn")
    t = t.replace("A Y B", "AYB").replace("&", "Y")
    t = re.sub(r"^[A-Z0-9]+", " ", t)
    return " ".join(t.split())

def cargar_inventario():
    iniciar_bd()
    xl = pd.ExcelFile(ARCHIVO)
    s = Sesion()

    # 1) las bodegas disponibles (la hoja trae encabezado en la fila 2)
    bod = pd.read_excel(xl, "BODEGAS DISPONIBLES", header=1)
    col = "BODEGAS" if "BODEGAS" in bod.columns else bod.columns[-1]
    nuevas = 0
    for n in bod[col].dropna().astype(str).strip().unique():
        nombre = re.sub(r"\s+", " ", n.upper()).strip()
        if not nombre or nombre == "NAN":
            continue
        if not s.query(Bodega).filter_by(nombre_oficial=nombre).first():
            s.add(Bodega(nombre_oficial=nombre))
            nuevas += 1
    s.commit()

    # 2) el stock hoja por hoja (una hoja = una bodega con su corte)
    sin_codigo = 0
    solo_en_stock = []
    filas_vacias = 0
    listadas = {_norm(x.nombre_oficial): x for x in s.query(Bodega).all()}

    for hoja in [h for h in xl.sheet_names if h != "BODEGAS DISPONIBLES"]:
        nombre_b = re.sub(r"\s+", " ", hoja.replace("STOCK", "", 1).strip().upper())
        clave = _norm(nombre_b)
        b = listadas.get(clave)
        if b is None and clave:
            # coincidencia parcial: solo si es inequivoca (un unico candidato).
            # con dos candidatos ambiguos (p.ej. "ZOOLOGICO" hoja suelta vs
            # "ZOOLOGICO SUMINISTROS" y "ZOOLOGICO PISCILAGO" en el listado)
            # es mejor crear una bodega nueva que adivinar mal y mezclar stock.
            candidatos = [v for k, v in listadas.items() if clave in k or k in clave]
            if len(candidatos) == 1:
                b = candidatos[0]
        if b is None: # bodega que solo existe como hoja
            b = Bodega(nombre_oficial=nombre_b)
            s.add(b)
            s.commit()
            s.refresh(b)
            listadas[clave] = b
            solo_en_stock.append(nombre_b)

    df = pd.read_excel(xl, hoja)
    df.columns = [str(c).strip() for c in df.columns]

```



```

for _, fila in df.iterrows():
    nombre = str(fila.get("Artículo", "")).strip()
    if not nombre or nombre.lower() == "nan":
        filas_vacias += 1
        continue
    cod = fila.get("Nr.Artículo")
    if pd.isna(cod):
        sin_codigo += 1
        cod = f"SIN-{sin_codigo:04d}"      # ningun dato se pierde
    else:
        cod = str(cod).split(".")[0].strip()

    if s.get(Articulo, cod) is None:
        unidad = str(fila.get("Unidad", "Unidad")).strip()
        s.add(Articulo(codigo=cod, nombre_oficial=nombre.upper(),
                        unidad_medida=unidad if unidad != "nan" else "Unidad"))
        s.commit()

    sd = fila.get("SD", 0)
    s.add(StockSistema(articulo_codigo=cod, bodega_id=b.id,
                      cantidad_sd=float(sd) if not pd.isna(sd) else 0))
    s.commit()

tot_b = s.query(Bodega).count()
tot_a = s.query(Articulo).count()
tot_s = s.query(StockSistema).count()
neg = s.query(StockSistema).filter(StockSistema.cantidad_sd < 0).count()
print(f" Bodegas listadas en la hoja:      {nuevas}")
print(f" Bodegas solo en hojas de stock:    {len(solo_en_stock)}  {solo_en_stock}")
print(f" Bodegas en total:                   {tot_b}")
print(f" Articulos del catalogo:               {tot_a}")
print(f" Registros de stock cargados:          {tot_s}")
print(f" Filas sin nombre de articulo:         {filas_vacias} (omitidas)")
print(f" Registros sin codigo (SIN-####):      {sin_codigo}")
print(f" SALDOS NEGATIVOS DETECTADOS:           {neg}    <- el mini reto")
s.close()

def cargar_recetas():
    """Recetas de ejemplo. Reemplace por el archivo oficial cuando llegue."""
    s = Sesion()
    if s.query(Receta).count() > 0:
        print(" Recetas ya cargadas.")
        s.close()
        return

    # descarta lo que no es alimento (empaques, aseo, medicamentos)
    NO_ALIMENTO = ("BOLSA", "CAJA", "VASO", "TAPA", "SERVILLETA", "GUANTE",
                   "PAPEL", "MECHERO", "CREMA TBOX", "BETAMETASONA", "JABON",
                   "DESINFECT", "TOALLA", "CUBIERTO", "PLATO", "EMPAQUE",
                   "CONTENEDOR", "PITILLO", "TENEDOR", "CUCHARA")

    def art(*claves):
        """Busca un alimento del catalogo comparando PALABRAS completas.

        Sin esto, «PAPA» encuentra «PAPAYA» y la receta queda absurda."""
        candidatos = []
        for a in s.query(Articulo).all():
            if any(x in a.nombre_oficial for x in NO_ALIMENTO):
                continue
            palabras = re.findall(r"[A-ZÁÉÍÓÚÑ]+", a.nombre_oficial.upper())
            if all(k.upper() in palabras for k in claves):
                candidatos.append(a)
        if not candidatos:

```

```

        return None
    # el nombre mas corto suele ser el ingrediente base, no una variante
    return sorted(candidatos, key=lambda a: len(a.nombre_oficial))[0]

RECETAS = {
    "AJIACO SANTAFEREÑO": [(("POLLO",), 0.250), (("PAPA",), 0.200),
                           (("ARROZ",), 0.080), (("SAL",), 0.005),
                           (("ACEITE",), 0.010)],
    "SANCOCHO DE GALLINA": [(("POLLO",), 0.300), (("PAPA",), 0.250),
                            (("YUCA",), 0.150), (("ARROZ",), 0.100),
                            (("ACEITE",), 0.015)],
}
creadas = 0
for nombre, ings in RECETAS.items():
    encontrados = [(art(*c), q) for c, q in ings]
    encontrados = [(a, q) for a, q in encontrados if a is not None]
    if not encontrados:
        continue
    r = Receta(nombre=nombre, rendimiento=1)
    s.add(r)
    s.commit()
    s.refresh(r)
    for a, q in encontrados:
        s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=a.codigo,
                                cantidad_por_porcion=q))
    s.commit()
    creadas += 1
    print(f" Receta «{nombre}»: {len(encontrados)} ingredientes")

# un servicio de ejemplo ya legalizado, para que el analisis tenga datos
if s.query(LineaServicio).count() == 0 and creadas:
    r = s.query(Receta).first()
    for ing in s.query(RecetaIngrediente).filter_by(receta_id=r.id).all():
        a = s.get(Articulo, ing.articulo_codigo)
        pedido = round(ing.cantidad_por_porcion * 50, 3)
        usado = round(pedido * 0.9, 3) # sobro un 10 %
        s.add(LineaServicio(servicio_id=1, articulo_codigo=a.codigo,
                             nombre=a.nombre_oficial, pedido=pedido,
                             usado=usado, estado="legalizado"))
    s.commit()
    print(" Servicio de ejemplo creado (para el analisis de consumo)")
s.close()

if __name__ == "__main__":
    print("Cargando el extracto de Colsubsidio...")
    cargar_inventario()
    print("\nCargando recetas...")
    cargar_recetas()
    print("\nListo. Ya puede abrir http://localhost:5173")

```

12.9 Servicio de conciliación (la sugerencia oficial de Colsubsidio)

Archivo: `backend/servicios/conciliacion.py`

```

"""De «tabla para picar blanca» al nombre y codigo oficiales del catalogo."""
import re
import unicodedata
from rapidfuzz import process, fuzz
from bd import Sesion
from modelos import Articulo, AliasArticulo, Bodega

```

```

# palabras que no distinguen un producto de otro
VACIAS = {"PARA", "DE", "DEL", "LA", "EL", "LOS", "LAS", "CON", "SIN",
          "POR", "EN", "Y", "A", "UN", "UNA", "UNAS", "UNOS", "AL"}

def _tokens(t: str) -> list[str]:
    return [p for p in t.split() if len(p) >= 3 and p not in VACIAS]

def _cobertura(consulta: str, candidato: str) -> float:
    """Cuantas palabras significativas de la consulta aparecen en el nombre.

    Compara por raiz de 4 letras, para que BLANCA encuentre BLANCO y
    CAZUELAS encuentre CAZUELA - en los DOS sentidos exige que la raiz
    tenga esas 4 letras de verdad (no baste una palabra de 3): sin ese
    piso en el lado del candidato, una palabra corta del catalogo como
    "RES" (de "COSTILLA DE RES") volvía "raiz" a ella misma y cualquier
    palabra dicha que empezara igual - como "RESTAURANTE" - contaba como
    coincidencia; sin el mismo piso del lado de lo dicho, decir "ver"
    (de una frase suelta como "sí para ver el detalle", no una búsqueda
    de verdad) encontraba "VERDE"/"VERDURAS" en artículos reales sin
    relación alguna. Una palabra de 3 letras solo cuenta si coincide
    EXACTA con alguna del candidato."""
    tc = _tokens(consulta)
    if not tc:
        return 0.0
    tn = _tokens(candidato)
    aciertos = 0
    for p in tc:
        raiz = p[:4]
        if any(p == q or (len(p) >= 4 and q.startswith(raiz))
              or (len(q) >= 4 and p.startswith(q[:4])) for q in tn):
            aciertos += 1
    return aciertos / len(tc) * 100

def puntuar(consulta: str, candidato: str) -> float:
    """Combina la forma general con la cobertura de palabras clave.

    Sin esto, «tabla para picar blanca» devuelve la AZUL: comparte
    tabla-picar y el color se diluye."""
    forma = fuzz.token_set_ratio(consulta, candidato)
    cob = _cobertura(consulta, candidato)
    return 0.45 * forma + 0.55 * cob

def normalizar(t: str) -> str:
    t = str(t).upper().strip()
    t = "".join(c for c in unicodedata.normalize("NFD", t)
               if unicodedata.category(c) != "Mn") # sin tildes
    t = t.replace("P/", "PARA ")
    # "kiosco"/"kiosko" son la misma palabra, dicha o escrita de dos
    # formas igual de comunes en español - sin esto, decir "kiosko" no
    # encontraba "KIOSCO TAQUILLA AYB" aunque sea exactamente la misma
    # bodega. La K es rarísima en el catálogo salvo en estos préstamos
    # (kilo, kiosco...), así que unificarla con la C no choca con nada.
    t = t.replace("K", "C")
    # el reconocimiento de voz del navegador suele cerrar la frase con un
    # punto ("Zoológico.") aunque nadie lo haya dicho - sin quitarlo, una
    # bodega real dejaba de encontrarse por un simple "." de mas.
    t = re.sub(r"[.,;:!?i?j]+", " ", t)
    return re.sub(r"\s+", " ", t).strip()

```

```

def buscar_bodegas_candidatas(s, texto: str, ids_permitidos: set[int] | None = None) -> list[Bodega]:
    """Todas las bodegas razonables para lo dicho/escrito, de más a menos
    específica - vacía si no hay ninguna, un solo elemento si es
    inequívoco, varios si es ambiguo ("restaurante" a secas encaja por
    igual en "RESTAURANTE FUENTES AYB" y "RESTAURANTE FUENTES SUMIN": ahí
    quien llama debe ofrecer las opciones, no adivinar cuál de las dos).

    ids_permitidos: si se da, restringe las coincidencias PARCIALES/
    ambiguas (por dónde se filtran nombres de bodegas ajenas que un
    auxiliar ni debería ver sugeridas). Una coincidencia EXACTA nunca se
    restringe: si dijo el nombre completo de una bodega real que no es
    suya, debe decir "no está asignada a usted" - no fingir que no
    existe y ofrecerle crear una bodega duplicada."""
    clave = normalizar(texto)
    if not clave:
        return []

    # El catálogo de bodegas es chico (unas pocas decenas): comparar en
    # Python, con normalizar() de los DOS lados, es lo que permite que
    # reglas como K->C funcionen igual para lo dicho y para el nombre
    # guardado - comparar contra la columna cruda (como antes, con SQL)
    # solo servía porque el nombre YA estaba guardado en mayúsculas sin
    # tildes, por pura coincidencia; una regla nueva de normalizar() no
    # se aplicaba del lado de la base de datos.
    todas = s.query(Bodega).all()

    def _permitidas():
        if ids_permitidos is None:
            return todas
        return [b for b in todas if b.id in ids_permitidos]

    exacta = next((b for b in todas if normalizar(b.nombre_oficial) == clave), None)
    if exacta:
        return [exacta]
    # coincidencia parcial: "kiosco" a secas es substring de "KIOSCO 2
    # SUMINISTROS", "KIOSCO TAQUILLA AYB" y varias más - todas cuentan.
    contienen = [b for b in _permitidas() if clave in normalizar(b.nombre_oficial)]
    if contienen:
        return contienen
    # ultimo recurso: cuantas palabras dichas (de mas de 3 letras) tiene
    # cada bodega - "autoservicios las fuentes" perdía "fuentes" antes y
    # se quedaba con la primera bodega que solo compartiera
    # "autoservicios"; ahora esa comparte 1 palabra y "AUTOSERVICIOS LAS
    # FUENTES" comparte 2, así que gana sola. Un empate real (dos
    # candidatas con el mismo máximo) se devuelve como ambigüedad.
    palabras = [p for p in clave.split() if len(p) > 3]
    if not palabras:
        return []
    candidatas = [(sum(1 for p in palabras if p in normalizar(b.nombre_oficial)), b)
                   for b in _permitidas()]
    candidatas = [(n, b) for n, b in candidatas if n > 0]
    if not candidatas:
        return []
    maximo = max(n for n, _ in candidatas)
    return [b for n, b in candidatas if n == maximo]

def buscar_bodega(s, texto: str, ids_permitidos: set[int] | None = None) -> Bodega | None:
    """La misma bodega para el nombre dicho o escrito, en un solo lugar
    (antes /api/bodegas/abrir y el agente conversacional cada uno tenía
    su propia version, y se desincronizaban). Nunca adivina entre varias
    candidatas igual de buenas: mejor decir "no la encuentro" (o, para
    quien necesite las opciones, ver buscar_bodegas_candidatas) que abrir

```

```

la bodega equivocada."""
candidatas = buscar_bodegas_candidatas(s, texto, ids_permitidos)
if len(candidatas) == 1:
    return candidatas[0]
if ids_permitidos is not None and not candidatas:
    # nada dentro de lo permitido - pero si el nombre dicho identifica
    # sin ambigüedad una bodega REAL en todo el catálogo, hay que
    # decir que existe (y dejar que quien llama la rechace por no
    # estar asignada), no fingir que no existe y ofrecer crear un
    # duplicado de una bodega que ya está en el sistema.
    sin_restriccion = buscar_bodegas_candidatas(s, texto, None)
    if len(sin_restriccion) == 1:
        return sin_restriccion[0]
return None

def buscar_articulo(texto: str, bodega_id=None, limite: int = 3) -> list[dict]:
    consulta = normalizar(texto)
    if not consulta:
        return []
    with Sesion() as s:
        # 1) ¿ya aprendio como lo dice este equipo?
        alias = s.query(AliasArticulo).filter_by(texto_dicho=consulta).first()
        if alias:
            a = s.get(Articulo, alias.articulo_codigo)
            if a:
                return [{"codigo": a.codigo, "nombre": a.nombre_oficial,
                        "unidad": a.unidad_medida, "confianza": 100}]
        # 2) si no, compara contra todo el catalogo
        arts = s.query(Articulo).all()

    if not arts:
        return []
    puntajes = []
    for a in arts:
        p = puntuar(consulta, normalizar(a.nombre_oficial))
        if p >= 45:
            puntajes.append((p, a))
    puntajes.sort(key=lambda z: -z[0])
    return [{"codigo": a.codigo, "nombre": a.nombre_oficial,
            "unidad": a.unidad_medida, "confianza": round(p)}
            for p, a in puntajes[:limite]]

def aprender_alias(texto: str, codigo: str, bodega_id=None):
    """Cada eleccion confirmada hace al agente mas preciso."""
    consulta = normalizar(texto)
    if not consulta or not codigo:
        return
    with Sesion() as s:
        ya = s.query(AliasArticulo).filter_by(texto_dicho=consulta,
                                              articulo_codigo=codigo).first()
        if not ya:
            s.add(AliasArticulo(texto_dicho=consulta,
                                articulo_codigo=codigo, bodega_id=bodega_id))
        s.commit()

```

12.10 Servicio de validación (incluye el mini reto de los negativos)

Archivo: `backend/servicios/validacion.py`

```

"""Las reglas duras. Codigo determinista: se comporta igual siempre."""
import os
from bd import Sesion

```

```

from modelos import Artículo, StockSistema, ConfigClave

UMBRAL_DEFEECTO = float(os.getenv("UMBRAL_ANOMALIA", 0.5))

def umbral_actual() -> float:
    """El umbral puede ajustarse desde Ajustes; si no, manda el .env."""
    with Sesion() as s:
        c = s.get(ConfigClave, "umbral_anomalia")
        return float(c.valor) if c else UMBRAL_DEFEECTO

def validar conteo(codigo: str, cantidad: float, unidad: str, bodega_id) -> dict:
    UMBRAL = umbral_actual()
    with Sesion() as s:
        art = s.get(Artículo, codigo)
        if art is None:
            return {"ok": False, "tipo": "inexistente",
                    "mensaje": "Ese artículo no está en el catálogo. "
                               "¿Lo creamos? Quedará pendiente de aprobación."}

        if cantidad is None:
            return {"ok": False, "tipo": "vacío",
                    "mensaje": "No entendí la cantidad. ¿Me la repite?"}

        # mini reto de Colsubsidio: un saldo físico nunca baja de cero
        if cantidad < 0:
            return {"ok": False, "tipo": "negativo",
                    "mensaje": "Una cantidad no puede ser negativa: un saldo "
                               "físico nunca baja de cero. Dígame el valor correcto."}

        # «cinco kilos» no se confunde con «cinco gramos»
        if unidad and unidad.strip().lower() != art.unidad_medida.strip().lower():
            return {"ok": False, "tipo": "unidad",
                    "mensaje": f"Ese artículo se maneja en {art.unidad_medida}, "
                               f"no en {unidad}. ¿Cuál es la cantidad "
                               f"en {art.unidad_medida}?"}

        stock = s.query(StockSistema).filter_by(
            artículo_codigo=codigo, bodega_id=bodega_id).first()
        esperado = stock.cantidad_sd if stock else None

        # el famoso 9 contra 90
        if esperado is not None and esperado > 0:
            desvio = abs(cantidad - esperado) / esperado
            if desvio > UMBRAL:
                return {"ok": False, "tipo": "desviación", "esperado": esperado,
                        "mensaje": f"El sistema espera alrededor de {esperado:g}. "
                                   f"¿Confirma {cantidad:g}?"}

    return {"ok": True, "esperado": esperado}

```

12.11 Servicio de recetas (pedido y legalización)

Archivo: `backend/servicios/recetas.py`

```

"""Momentos 1 y 3 del reto: pedido por receta y legalización del servicio."""
import unicodedata
from datetime import timedelta
from bd import Sesion
from horario import ahora

```

```

from modelos import (Receta, RecetaIngrediente, Artículo,
                    StockSistema, LineaServicio)

def _sin_tildes(t: str) -> str:
    """«santafereno» dicho o escrito sin tilde («santafereno») no debe
    fallar la búsqueda - un LIKE normal en SQLite si distingue Ñ de N, y
    eso ya causo un "no encuentre la receta" con un plato que si existia."""
    t = str(t or "").upper().strip()
    return "".join(c for c in unicodedata.normalize("NFD", t)
                  if unicodedata.category(c) != "Mn")

def _buscar_receta(s, preparacion: str):
    objetivo = _sin_tildes(preparacion)
    if not objetivo:
        return None
    for r in s.query(Receta).all():
        if objetivo in _sin_tildes(r.nombre):
            return r
    return None

def calcular_pedido(preparacion: str, porciones: int, bodega_id: int) -> dict:
    """La regla del reto: cantidad por porcion x porciones - lo que ya hay.

    Si un ingrediente de la receta no esta en el catalogo, o esta pero esa
    bodega nunca lo ha tenido registrado, no se salta en silencio: queda
    en «faltantes_catalogo» / «sin_registro_bodega» para que la persona
    se entere y no crea que el pedido esta completo cuando no lo esta."""
    with Sesion() as s:
        receta = _buscar_receta(s, preparacion)
        if receta is None:
            return {"lineas": [], "faltantes_catalogo": [],
                  "sin_registro_bodega": [], "receta_encontrada": False}

        ings = s.query(RecetaIngrediente).filter_by(receta_id=receta.id).all()
        lineas, faltantes, sin_registro = [], [], []
        for ing in ings:
            art = s.get(Artículo, ing.articulo_codigo)
            if art is None:
                faltantes.append(ing.articulo_codigo)
                continue
            necesario = round(ing.cantidad_por_porcion * porciones, 3)
            stock = s.query(StockSistema).filter_by(
                articulo_codigo=art.codigo, bodega_id=bodega_id).first()
            if stock is None:
                sin_registro.append(art.nombre_oficial)
            hay = stock.cantidad_sd if stock else 0
            falta = max(0, round(necesario - hay, 3))
            lineas.append({"codigo": art.codigo, "nombre": art.nombre_oficial,
                          "unidad": art.unidad_medida, "necesario": necesario,
                          "stock": hay, "falta": falta,
                          "sin_registro_bodega": stock is None})
        return {"lineas": lineas, "faltantes_catalogo": faltantes,
              "sin_registro_bodega": sin_registro, "receta_encontrada": True}

def detalle_receta(preparacion: str) -> dict:
    """Solo consulta: cómo está armada la receta, sin tocar stock."""
    with Sesion() as s:
        receta = _buscar_receta(s, preparacion)
        if receta is None:

```

```

        return {}
    ings = s.query(RecetaIngrediente).filter_by(receta_id=receta.id).all()
    lineas, faltantes = [], []
    for ing in ings:
        art = s.get(Articulo, ing.articulo_codigo)
        if art is None:
            faltantes.append(ing.articulo_codigo)
            continue
        lineas.append({"codigo": art.codigo, "nombre": art.nombre_oficial,
                       "unidad": art.unidad_medida,
                       "por_porcion": ing.cantidad_por_porcion})
    return {"id": receta.id, "nombre": receta.nombre, "rendimiento": receta.rendimiento,
           "preparacion": receta.preparacion or "", "lineas": lineas,
           "faltantes_catalogo": faltantes}

def comparar_legalizacion(servicio_id: int) -> dict:
    """Lo pedido contra lo usado, con la lectura en palabras. Solo lo que
    sigue abierto: si ya se legalizo antes (el servicio de ejemplo, u otro
    pedido con el mismo numero de servicio) no debe mezclarse aqui ni
    volver a legalizarse por accidente."""
    with Sesion() as s:
        filas = s.query(LineaServicio).filter_by(
            servicio_id=servicio_id, estado="abierto").all()
        lineas = []
        for l in filas:
            dif = round((l.usado or 0) - (l.pedido or 0), 3)
            if dif < 0:
                lectura = "Sobrante: vuelve a bodega"
            elif dif > 0:
                lectura = "Merma: revisar con el chef"
            else:
                lectura = "Cuadro exacto"
            art = s.get(Articulo, l.articulo_codigo)
            lineas.append({"codigo": l.articulo_codigo, "nombre": l.nombre,
                           "unidad": art.unidad_medida if art else "",
                           "pedido": l.pedido, "usado": l.usado,
                           "diferencia": dif, "lectura": lectura})
        plato = next((f.plato for f in filas if f.plato), "")
        porciones = next((f.porciones for f in filas if f.porciones), 0)
        return {"lineas": lineas, "plato": plato, "porciones": porciones}

def analisis_consumo(dias: int = 30) -> dict:
    """El «suma puntos» del reto: subutilizados y sobrepedido, acotado a
    los ultimos N dias (por defecto 30, como en el tablero de Reportes)."""
    desde = ahora() - timedelta(days=dias)
    with Sesion() as s:
        filas = (s.query(LineaServicio)
                 .filter(LineaServicio.estado == "legalizado",
                         LineaServicio.creado >= desde).all())
        total_ped = sum(f.pedido or 0 for f in filas)
        total_uso = sum(f.usado or 0 for f in filas)
        servicios = len({f.servicio_id for f in filas})
        porart = {}
        for f in filas:
            d = porart.setdefault(f.nombre, {"pedido": 0, "usado": 0, "veces": 0})
            d["pedido"] += f.pedido or 0
            d["usado"] += f.usado or 0
            d["veces"] += 1
        subutil = []
        for nombre, d in porart.items():
            sobra = round(d["pedido"] - d["usado"], 3)
            if sobra > 0:
                pct = round(sobra / d["pedido"] * 100) if d["pedido"] else 0
                subutil.append({"nombre": nombre, "sobra": sobra,
                                "pct": pct})

```



```

        "veces": d["veces"], "sobrepedido_pct": pct})
subutil.sort(key=lambda x: -x["sobrepedido_pct"])
ahorro_potencial = round(sum(s["sobra"] for s in subutil), 2)
return {"pedido_total": round(total_ped, 2), "usado_total": round(total_uso, 2),
        "aprovechamiento": round(total_uso / total_ped * 100, 1) if total_ped else 0,
        "servicios_periodo": servicios, "ahorro_potencial": ahorro_potencial,
        "dias": dias, "subutilizados": subutil}

```

12.12 Servicio de analítica (panel gerencial y reportes)

Archivo: backend/servicios/analitica.py — No existía en la versión anterior: todas las métricas del Panel (5.22–5.23) y de Reportes se calculan en vivo aquí, sobre la misma base.

```

"""Las métricas del panel gerencial y de reportes: todo calculado en vivo
sobre la misma base que usa el resto de la aplicación – una sola fuente."""
from datetime import datetime
from bd import Sesion
from modelos import (Bodega, Articulo, StockSistema, SesionConteo, Conteo,
                     Alerta, AliasArticulo, HistorialCierre, ConfigClave)

def _config(clave: str, defecto: str) -> str:
    with Sesion() as s:
        c = s.get(ConfigClave, clave)
        return c.valor if c else defecto

def diferencias_por_bodega(limite: int = 6) -> list[dict]:
    """Suma de |contado - sistema| por bodega, sobre lo confirmado."""
    with Sesion() as s:
        bodegas = s.query(Bodega).all()
        salida = []
        for b in bodegas:
            conteos = (s.query(Conteo).join(SesionConteo)
                       .filter(SesionConteo.bodega_id == b.id,
                               Conteo.estado == "confirmado").all())
            if not conteos:
                continue
            total = 0.0
            for c in conteos:
                st = s.query(StockSistema).filter_by(
                    articulo_codigo=c.articulo_codigo, bodega_id=b.id).first()
                esperado = st.cantidad_sd if st else 0
                total += abs((c.cantidad or 0) - esperado)
            if total > 0:
                salida.append({"bodega": b.nombre_oficial, "diferencia": round(total, 1)})
        salida.sort(key=lambda x: -x["diferencia"])
        return salida[:limite]

def stock_por_unidad() -> list[dict]:
    with Sesion() as s:
        arts = {a.codigo: a.unidad_medida for a in s.query(Articulo).all()}
        acumulado = {}
        for st in s.query(StockSistema).all():
            u = arts.get(st.articulo_codigo, "Otra")
            acumulado[u] = acumulado.get(u, 0) + max(st.cantidad_sd or 0, 0)
        total = sum(acumulado.values()) or 1
        salida = [{"unidad": u, "cantidad": round(c, 1),
                  "pct": round(c / total * 100, 1)}
                  for u, c in acumulado.items()]
        salida.sort(key=lambda x: -x["cantidad"])
        return salida

```

```

def alertas_por_tipo() -> list[dict]:
    etiquetas = {"desviacion": "Desviación", "unidad": "Unidad",
                "negativo": "Saldo negativo", "inexistente": "Inexistente"}
    with Sesion() as s:
        conteo = {}
        for a in s.query(Alerta).all():
            conteo[a.tipo] = conteo.get(a.tipo, 0) + 1
        salida = [{"tipo": etiquetas.get(t, t), "cantidad": n} for t, n in conteo.items()]
        salida.sort(key=lambda x: -x["cantidad"])
        return salida

def descuadres_recurrentes(limite: int = 5) -> list[dict]:
    """Un artículo con diferencia en más de una toma: ahí está la causa raíz."""
    with Sesion() as s:
        arts = {a.codigo: a.nombre_oficial for a in s.query(Articulo).all()}
        por_articulo = {}
        conteos = (s.query(Conteo).join(SesionConteo)
                    .filter(Conteo.estado == "confirmado").all())
        for c in conteos:
            ses = s.get(SesionConteo, c.sesion_id)
            st = s.query(StockSistema).filter_by(
                articulo_codigo=c.articulo_codigo, bodega_id=ses.bodega_id).first()
            esperado = st.cantidad_sd if st else 0
            dif = (c.cantidad or 0) - esperado
            if abs(dif) < 0.01:
                continue
            d = por_articulo.setdefault(c.articulo_codigo,
                                         {"dif": 0.0, "tomas": set(), "negativo": False})
            d["dif"] += dif
            d["tomas"].add(c.sesion_id)
            if esperado < 0:
                d["negativo"] = True
        salida = []
        for cod, d in por_articulo.items():
            if len(d["tomas"]) < 2 and not d["negativo"]:
                continue
            salida.append({
                "articulo": arts.get(cod, cod),
                "tipo": "Saldo negativo" if d["negativo"] else "Desviación",
                "diferencia": round(d["dif"], 1),
                "tomas": len(d["tomas"]),
                "accion": "Revisar salidas sin entrada" if d["negativo"]
                        else "Crear en catálogo oficial" if cod.startswith("PEND")
                        else "Ajustar el saldo del sistema",
            })
        salida.sort(key=lambda x: -abs(x["diferencia"]))
        return salida[:limite]

def historial_exactitud(limite: int = 12) -> list[dict]:
    with Sesion() as s:
        filas = (s.query(HistorialCierre).order_by(HistorialCierre.fecha).all())[-limite:]
        nombres = {b.id: b.nombre_oficial for b in s.query(Bodega).all()}
        return [{"id": f.id, "fecha": f.fecha.strftime("%Y-%m-%d"), "exactitud": round(f.exactitud, 1),
                "bodega_id": f.bodega_id, "bodega": nombres.get(f.bodega_id, "")} for f in filas]

def resumen_ejecutivo() -> dict:
    with Sesion() as s:
        bodegas = s.query(Bodega).all()
        cerradas = [b for b in bodegas if b.estado == "cerrada"]

```

```

total_conteos = s.query(Conteo).filter_by(estado="confirmado").count()
alertas_total = s.query(Alerta).count()
alertas_resueltas = s.query(Alerta).filter_by(resuelta=1).count()
historial = s.query(HistorialCierre).all()
exact = (round(sum(h.exactitud for h in historial) / len(historial), 1)
         if historial else 100.0)

return {
    "exactitud_primera_pasada": exact,
    "referencias_contadas": total_conteos,
    "alertas_gestionadas": alertas_resueltas,
    "alertas_total": alertas_total,
    "bodegas_cerradas": len(cerradas),
    "bodegas_total": len(bodegas),
    "diferencia_por_bodega": diferencias_por_bodega(limite=10),
    "stock_por_unidad": stock_por_unidad(),
    "historial_exactitud": historial_exactitud(),
}

def resumen_alertas_panel() -> dict:
    with Sesion() as s:
        negativos_actuales = s.query(StockSistema).filter(
            StockSistema.cantidad_sd < 0).count()
        alias = s.query(AliasArticulo).count()
        bodegas = s.query(Bodega).all()
        cerradas_ses = (s.query(SesionConteo).filter_by(tipo="conteo", estado="cerrada")
                        .filter(SesionConteo.fin.isnot(None)).all())
        minutos = [(ses.fin - ses.inicio).total_seconds() / 60
                    for ses in cerradas_ses if ses.fin]
        negativos_iniciales = int(_config("negativos_iniciales", str(negativos_actuales or 0)))
        estado_bodegas = {}
        for b in bodegas:
            estado_bodegas[b.estado] = estado_bodegas.get(b.estado, 0) + 1
    return {
        "negativos_iniciales": negativos_iniciales,
        "negativos_actuales": negativos_actuales,
        "tiempo_promedio_min": round(sum(minutos) / len(minutos)) if minutos else 0,
        "alias_aprendidos": alias,
        "estado_bodegas": estado_bodegas,
        "alertas_por_tipo": alertas_por_tipo(),
        "descuadres_recurrentes": descuadres_recurrentes(),
    }

```

12.13 Huella del dispositivo (WebAuthn)

Archivo: `backend/servicios/huella.py` — Ingreso sin volver a teclear el PIN: Windows Hello, Touch ID o huella de Android, con el protocolo WebAuthn real (no una simulación).

```

"""Ingreso con huella del dispositivo: WebAuthn real (Windows Hello, Touch
ID, huella de Android), no la simulacion anterior de un solo campo booleano.

```

```

Los retos (challenges) viven en memoria del proceso - igual que ESTADOS del
agente en orquestador.py -, alcanza para esta app de un solo servidor y se
vencen solos a los pocos minutos."""

```

```

import json
import os
import secrets
from datetime import datetime, timedelta

```

```

import webauthn
from webauthn.helpers import base64url_to_bytes, bytes_to_base64url
from webauthn.helpers.structs import (
    AuthenticatorAttachment, AuthenticatorSelectionCriteria,
    PublicKeyCredentialDescriptor, ResidentKeyRequirement,

```

```

    UserVerificationRequirement,
)

RP_ID = os.getenv("WEBAUTHN_RP_ID", "localhost")
RP_NOMBRE = os.getenv("WEBAUTHN_RP_NAME", "CuentaVoz")
# uno o varios orígenes separados por coma - el/los dominio(s) reales del
# frontend, con esquema (http/https) y sin barra final.
ORIGENES = [o.strip() for o in
             os.getenv("WEBAUTHN_ORIGENES", "http://localhost:5173").split(",")
             if o.strip()]

_RETOS_REGISTRO: dict[int, dict] = {}      # usuario_id -> {challenge, expira}
_RETOS_INGRESO: dict[str, dict] = {}      # reto_id -> {challenge, usuario_id, expira}

def _vencido(expira: datetime) -> bool:
    return expira < datetime.now()

def opciones_registro(usuario_id: int, nombre: str, credenciales_existentes: list[str]) -> dict:
    opciones = webauthn.generate_registration_options(
        rp_id=RP_ID, rp_name=RP_NOMBRE,
        user_id=str(usuario_id).encode(), user_name=nombre, user_display_name=nombre,
        authenticator_selection=AuthenticatorSelectionCriteria(
            authenticator_attachment=AuthenticatorAttachment.PLATFORM,
            user_verification=UserVerificationRequirement.REQUIRED,
            resident_key=ResidentKeyRequirement.PREFERRED),
        exclude_credentials=[PublicKeyCredentialDescriptor(id=base64url_to_bytes(c))
                              for c in credenciales_existentes],
    )
    _RETOS_REGISTRO[usuario_id] = {"challenge": opciones.challenge,
                                    "expira": datetime.now() + timedelta(minutes=3)}
    return json.loads(webauthn.options_to_json(opciones))

def verificar_registro(usuario_id: int, credencial: dict) -> dict:
    entrada = _RETOS_REGISTRO.pop(usuario_id, None)
    if not entrada or _vencido(entrada["expira"]):
        raise ValueError("El registro de huella expiró. Intente de nuevo.")
    verificacion = webauthn.verify_registration_response(
        credential=credencial, expected_challenge=entrada["challenge"],
        expected_rp_id=RP_ID, expected_origin=ORIGENES,
        require_user_verification=True,
    )
    return {"credential_id": bytes_to_base64url(verificacion.credential_id),
            "public_key": bytes_to_base64url(verificacion.credential_public_key),
            "sign_count": verificacion.sign_count}

def opciones_ingreso(usuario_id: int, credenciales: list[str]) -> tuple[str, dict]:
    opciones = webauthn.generate_authentication_options(
        rp_id=RP_ID,
        allow_credentials=[PublicKeyCredentialDescriptor(id=base64url_to_bytes(c))
                           for c in credenciales],
        user_verification=UserVerificationRequirement.REQUIRED,
    )
    reto_id = secrets.token_urlsafe(16)
    _RETOS_INGRESO[reto_id] = {"challenge": opciones.challenge, "usuario_id": usuario_id,
                              "expira": datetime.now() + timedelta(minutes=2)}
    return reto_id, json.loads(webauthn.options_to_json(opciones))

```

```
def verificar_ingreso(reto_id: str, credencial: dict, public_key_b64: str,
                    sign_count: int) -> tuple[int, int]:
    """Devuelve (usuario_id, nuevo_sign_count) o lanza ValueError."""
    entrada = _RETOS_INGRESO.pop(reto_id, None)
    if not entrada or _vencido(entrada["expira"]):
        raise ValueError("El reto de huella expiró. Intente de nuevo.")
    verificacion = webauthn.verify_authentication_response(
        credential=credencial, expected_challenge=entrada["challenge"],
        expected_rp_id=RP_ID, expected_origin=ORIGENES,
        credential_public_key=base64url_to_bytes(public_key_b64),
        credential_current_sign_count=sign_count,
        require_user_verification=True,
    )
    return entrada["usuario_id"], verificacion.new_sign_count
```

12.14 Intérprete local (respaldo sin Gemini)

Archivo: `backend/servicios/interprete.py` — Si falta `GOOGLE_API_KEY` o Gemini no responde, `cerebro.py` cae aquí sin que el resto del sistema lo note: números en palabras + intenciones por palabra clave.

```
"""Interprete local: permite demostrar el flujo aun sin llave de Gemini.

No reemplaza al modelo. Reconoce numeros en palabras y las intenciones
mas frecuentes, para que la demo nunca se caiga por falta de internet.
"""
import re

NUMEROS = {
    "cero": 0, "un": 1, "una": 1, "uno": 1, "dos": 2, "tres": 3, "cuatro": 4,
    "cinco": 5, "seis": 6, "siete": 7, "ocho": 8, "nueve": 9, "diez": 10,
    "once": 11, "doce": 12, "trece": 13, "catorce": 14, "quince": 15,
    "dieciseis": 16, "dieciséis": 16, "diecisiete": 17, "dieciocho": 18,
    "diecinueve": 19, "veinte": 20, "veinticinco": 25, "treinta": 30,
    "cuarenta": 40, "cincuenta": 50, "sesenta": 60, "setenta": 70,
    "ochenta": 80, "noventa": 90, "cien": 100, "ciento": 100,
    "doscientos": 200, "doscientas": 200, "trescientos": 300, "trescientas": 300,
    "cuatrocientos": 400, "cuatrocientas": 400, "quinientos": 500, "quinientas": 500,
    "seiscientos": 600, "seiscientas": 600, "setecientos": 700, "setecientas": 700,
    "ochocientos": 800, "ochocientas": 800, "novecientos": 900, "novecientas": 900,
    "mil": 1000,
}

UNIDADES = {
    "kilo": "Kilogram", "kilos": "Kilogram", "kilogramo": "Kilogram",
    "kilogramos": "Kilogram", "kg": "Kilogram",
    "litro": "Liter", "litros": "Liter", "l": "Liter",
    "unidad": "Unidad", "unidades": "Unidad",
    "porcion": "Portion", "porciones": "Portion", "porción": "Portion",
}

DECENAS = (20, 30, 40, 50, 60, 70, 80, 90)

def _numero(texto: str):
    """La PRIMERA cantidad de la frase. Ignora numeros que son parte
    del nombre del producto («cazuela 16 onzas», «tabla 50x38»)."""
    t = texto.lower()
    m = re.search(r"-?\d+(?:[.]\d+)?", t)
    if m:
        val = float(m.group(0).replace(", ", "."))
        return -val if "menos" in t[:m.start()] else val
```

```

palabras = [p.strip(".,;?!") for p in t.split()]
total = None
for i, p in enumerate(palabras):
    if p not in NUMEROS:
        continue
    v = NUMEROS[p]
    if total is None:
        total = v
        # «ciento ochenta»: centena seguida de decena
        if total >= 100 and i + 1 < len(palabras) and palabras[i + 1] in NUMEROS:
            sig = NUMEROS[palabras[i + 1]]
            if sig < 100:
                total += sig
        # «treinta y cinco»: decena + y + unidad
        elif total in DECENAS and i + 2 < len(palabras) \
            and palabras[i + 1] == "y" and palabras[i + 2] in NUMEROS:
            total += NUMEROS[palabras[i + 2]]
        break # lo demas pertenece al nombre del producto
if total is not None and "menos" in t:
    total = -abs(total)
return total

def _unidad(texto: str):
    for p in texto.lower().replace("?", " ").split():
        p = p.strip(".,")
        if p in UNIDADES:
            return UNIDADES[p]
    return None

CANONICAS = {"Kilogram", "Liter", "Unidad", "Portion"}

def normalizar_unidad(dicha):
    """Lo que sea que devuelva el agente (humano o IA) para la unidad,
    lo lleva a uno de los 4 valores canonicos de la base de datos.
    Gemini a veces repite la palabra tal cual la dijo la persona
    ("kilos") en vez de traducirla ("Kilogram"); esto lo corrige aqui,
    sin depender de que el modelo obedezca el prompt al pie de la letra."""
    if not dicha:
        return dicha
    if dicha in CANONICAS:
        return dicha
    return UNIDADES.get(str(dicha).strip().lower(), dicha)

def interpretar_local(frase: str) -> dict:
    f = frase.lower().strip()
    cant = _numero(f)
    uni = _unidad(f)

    def sin_ruido(t):
        t = re.sub(r"-?\d+(?:[.]\d+)?", " ", t)
        fuera = set(NUMEROS) | set(UNIDADES) | {
            "de", "del", "la", "el", "los", "las", "hay", "en", "y", "por",
            "confirmo", "confirmar", "son", "hoy", "preparamos", "quiero",
            "pedir", "insumos", "para", "cuanto", "cuánto", "cuantos",
            "iniciar", "conteo", "abrir", "bodega", "corregir", "ultimo",
            "último", "no", "uy", "que", "qué", "me", "genera", "generame",
            "consolidado", "reporte", "ayuda", "menos", "onzas", "onza",
        }
        return " ".join(w for w in t.split() if w.strip(".,;?!") not in fuera).strip()

```

```

# — intenciones —
if any(k in f for k in ("iniciar conteo", "abrir bodega", "abrir la bodega")):
    return {"intencion": "navegar", "bodega_texto": sin_ruido(f),
            "respuesta_hablada": "Voy a abrir esa bodega."}

if any(k in f for k in ("preparamos", "vamos a preparar", "pedir insumos")):
    return {"intencion": "pedir", "preparacion": sin_ruido(f),
            "porciones": int(cant) if cant else 0,
            "respuesta_hablada": "Le calculo los insumos segun la receta."}

if f.startswith("confirmo") or f in ("confirmar", "si", "sí", "correcto", "confirmo."):
    return {"intencion": "confirmar", "respuesta_hablada": "Listo."}

if any(k in f for k in ("corregir", "no son", "no, son", "uy no", "uy, no",
                        "esta mal", "está mal", "me equivoque", "cambiar")):
    return {"intencion": "corregir", "cantidad": cant, "unidad": uni,
            "respuesta_hablada": "Corrijo ese registro."}

if any(k in f for k in ("cuanto", "cuánto", "cuantos", "donde hay", "dónde hay")):
    return {"intencion": "consultar", "articulo_texto": sin_ruido(f),
            "respuesta_hablada": "Reviso el inventario."}

if any(k in f for k in ("reporte", "consolidado", "archivo", "imprim")):
    return {"intencion": "reporte", "respuesta_hablada": "Genero el archivo."}

if "ayuda" in f or "no entiendo" in f:
    return {"intencion": "ayuda",
            "respuesta_hablada": "Con gusto. ¿Que necesita saber?"}

if cant is not None:
    return {"intencion": "contar", "articulo_texto": sin_ruido(f),
            "cantidad": cant, "unidad": uni, "respuesta_hablada": ""}

return {"intencion": "ayuda",
        "respuesta_hablada": "Perdon, no le entendi bien. ¿Me lo repite?"}

```

12.15 Reportes y almacenamiento de archivos

Archivo: `backend/reportes.py`

```

"""El endpoint del flujo: informacion limpia lista para el ERP."""
import pandas as pd
from sqlalchemy import text
from bd import motor
from horario import ahora
from servicios.archivos import guardar_df

# Los nombres de columna son a proposito los mismos que trae el extracto
# real de Colsubsidio (ver data/BODEGAS_Y_STOCK.xlsx: "Nr.Artículo",
# "Artículo", "Unidad", "SD") - Bodega/Contado/Diferencia son las unicas
# columnas nuevas. Asi el archivo que se descarga aqui se reconoce de una
# en My Inventory (Oracle) en vez de traer nombres genericos inventados.
CONSULTA = """
SELECT a.codigo AS "Nr.Artículo", a.nombre_oficial AS "Artículo",
       a.unidad_medida AS "Unidad", b.nombre_oficial AS "Bodega",
       c.cantidad AS "Contado", st.cantidad_sd AS "SD",
       c.cantidad - COALESCE(st.cantidad_sd, 0) AS "Diferencia"
FROM conteo c
JOIN articulo a ON a.codigo = c.articulo_codigo
JOIN sesion_conteo sc ON sc.id = c.sesion_id

```

```

JOIN bodega b ON b.id = sc.bodega_id
LEFT JOIN stock_sistema st ON st.articulo_codigo = a.codigo
    AND st.bodega_id = b.id
WHERE c.estado = 'confirmado'
"""

def consolidado(formato: str = "xlsx") -> tuple[str, int, list[dict]]:
    df = pd.read_sql(CONSULTA, motor)
    # "SD" sale NULL/NaN cuando el articulo no tiene fila de stock en esa
    # bodega (LEFT JOIN); NaN no es JSON valido para la vista previa, y
    # aqui significa lo mismo que en el resto de la app: no hay dato, se
    # trata como 0.
    df["SD"] = df["SD"].fillna(0)
    df["Diferencia"] = df["Diferencia"].round(2)
    marca = ahora().strftime("%Y%m%d_%H%M")
    ruta = f"reportes/consolidado_{marca}.{formato}"
    guardar_df(ruta, df, formato)
    vista_previa = df.head(8).to_dict("records")
    return ruta, len(df), vista_previa

def diferencias_archivo(formato: str = "xlsx") -> tuple[str, int, int]:
    """«Diferencias por bodega» descargable: solo las filas donde el
    conteo no cuadra con el sistema, no el consolidado completo."""
    df = pd.read_sql(CONSULTA, motor)
    df["Diferencia"] = df["Diferencia"].round(2)
    df = df[df["Diferencia"] != 0]
    marca = ahora().strftime("%Y%m%d_%H%M")
    ruta = f"reportes/diferencias_{marca}.{formato}"
    guardar_df(ruta, df, formato)
    return ruta, len(df), df["Bodega"].nunique() if len(df) else 0

def detalle_bodega(bodega_id: int, formato: str = "xlsx") -> str:
    """«Descargar detalle»: el conteo completo de una sola bodega."""
    # text() - no un str plano - porque un str plano hace que pandas use
    # exec_driver_sql() (ejecuta tal cual contra el driver, sin pasar por
    # la traduccion de parametros de SQLAlchemy): en SQLite ":bodega_id"
    # funciona porque el driver sqlite3 lo entiende nativo, pero en
    # Postgres (psycopg2, que solo entiende %(nombre)s) ese ":bodega_id"
    # llega como texto literal invalido y revienta con 500 - paso
    # solo detectable en produccion, no en la base local.
    df = pd.read_sql(text(CONSULTA + " AND b.id = :bodega_id"),
                      motor, params={"bodega_id": bodega_id})
    df["Diferencia"] = df["Diferencia"].round(2)
    marca = ahora().strftime("%Y%m%d_%H%M")
    ruta = f"reportes/bodega_{bodega_id}_{marca}.{formato}"
    guardar_df(ruta, df, formato)
    return ruta

ESTADO_BODEGAS = """
SELECT nombre_oficial AS bodega, estado
FROM bodega ORDER BY nombre_oficial
"""

def estado_bodegas(formato: str = "xlsx") -> str:
    """«Exportar estado»: la foto del tablero en vivo, para mandarla por
    correo o adjuntarla sin tener que tomar una captura de pantalla."""
    df = pd.read_sql(ESTADO_BODEGAS, motor)
    marca = ahora().strftime("%Y%m%d_%H%M")

```



```

ruta = f"reportes/estado_bodegas_{marca}.{formato}"
guardar_df(ruta, df, formato)
return ruta

```

Archivo: `backend/servicios/archivos.py` — El disco del Web Service en Render es efímero (se borra en cada despliegue), así que los .xlsx/.csv generados se guardan como bytes en PostgreSQL, no en una carpeta del servidor.

```

"""Guardar y leer los XLSX/CSV generados desde la base, no desde disco -
el disco del Web Service en Render es efimero (se borra en cada deploy o
reinicio), asi que un reporte "generado hace una hora" quedaria
inalcanzable despues del siguiente `git push`. La "ruta" (ej.
"reportes/consolidado_20260815_1636.xlsx") se sigue usando como
identificador legible en toda la app - solo cambia DONDE vive el
contenido real."""
import io

from bd import Sesion
from modelos import ArchivoGenerado

def guardar_df(ruta: str, df, formato: str) -> None:
    buf = io.BytesIO()
    if formato == "csv":
        df.to_csv(buf, index=False)
    else:
        df.to_excel(buf, index=False)
    with Sesion() as s:
        s.add(ArchivoGenerado(ruta=ruta, contenido=buf.getvalue()))
        s.commit()

def leer_bytes(ruta: str) -> bytes | None:
    with Sesion() as s:
        obj = (s.query(ArchivoGenerado).filter_by(ruta=ruta)
              .order_by(ArchivoGenerado.id.desc()).first())
        return obj.contenido if obj else None

```

12.16 El cerebro: conexión con Gemini (Google AI Studio)

Archivo: `backend/agente/cerebro.py`

```

"""El «pensar» del agente: Gemini via Google AI Studio."""
import os
import json
import io
import wave
import concurrent.futures

MODELO_VOZ = "gemini-2.5-flash-preview-tts"

# El SDK instalado (google-genai 0.3.0) no soporta timeout en HttpOptions
# todavía (es un TODO del propio paquete) - sin esto, una llamada lenta o
# con la cuota ahogada se queda esperando sin limite (confirmado: una
# prueba real tardo mas de 30 segundos), y la persona ve el microfono
# "pensando" sin saber si sigue vivo. Con este limite, si Gemini no
# contesta a tiempo, se cae al respaldo local de una vez en vez de dejar
# la voz esperando.
TIMEOUT_GEMINI = 7
_EJECUTOR = concurrent.futures.ThreadPoolExecutor(max_workers=4, thread_name_prefix="gemini")

```

```
def _generar_con_limite(cliente, limite=TIMEOUT_GEMINI, **kwargs):
    futuro = _EJECUTOR.submit(cliente.models.generate_content, **kwargs)
    return futuro.result(timeout=limite)
```

Voces neuronales de Gemini disponibles para elegir en Mi perfil. No son
acentos de país (Gemini TTS no expone eso todavía): son voces distintas,
pero todas genuinamente humanas y fluidas - muy por encima de las voces
"Desktop" clásicas del navegador, que es lo que se quería reemplazar.

```
VOCES = {
    "kore": {"nombre": "Kore", "etiqueta": "Femenina, firme y clara"},
    "aoede": {"nombre": "Aoede", "etiqueta": "Femenina, suave y cercana"},
    "puck": {"nombre": "Puck", "etiqueta": "Masculina, animada"},
    "charon": {"nombre": "Charon", "etiqueta": "Masculina, grave y pausada"},
}
```

```
VOZ_DEFECTO = "kore"
```

PROMPT_SISTEMA = ""Eres CuentaVoz, el asistente de inventarios de Colsubsidio,
usado por auxiliares de bodega en Piscilago. Hablas como una persona
colombiana cercana y profesional: frases cortas, calidas, naturales.
Usa expresiones como "listo", "claro que sí", "de una", "perfecto" en vez
de sonar como un robot que repite siempre la misma fórmula. Varía tus
respuestas: no contestes igual dos veces seguidas.

Reglas:

1. Nunca inventes artículos: usa solo los que devuelva `buscar_articulo`.
2. Antes de guardar, repite siempre artículo, cantidad y unidad.
3. Con varios candidatos, lee las opciones y deja que la persona elija.
4. Si hay una alerta, pregunta antes de continuar. Nunca guardes un dato con alerta sin confirmación explícita.
5. Si la persona no entiende algo, explícalo con palabras simples.
6. Si piden datos, resume en voz y ofrece generar el archivo.
7. Si no puedes resolver, dilo con honestidad y ofrece llamar al administrador de bodega.
8. Se flexible con la forma de hablar: la gente no dicta como robot. Frases como "hay noventa cazuelas", "me faltan cinco kilos de arroz", "anota tres tablas blancas" son todas la intención "contar". Solo responde que no entendiste como último recurso, si de verdad no hay forma razonable de interpretar la frase.
9. OJO con las negaciones: "no quiero preparar arroz", "no es arroz", "cancela el ajiaco", "ya no necesito eso" NUNCA deben dejar preparación, artículo_texto ni cantidad rellenos como si la persona hubiera pedido o confirmado ese producto - es justo lo contrario. Si niega o cancela algo, la intención es "explicar" o "corregir", y preparación/artículo_texto/cantidad quedan en null. Nunca extraigas un dato de una frase que lo está negando.

La unidad SIEMPRE debe quedar en uno de estos 4 valores exactos, nunca la palabra que dijo la persona: "Kilogram" (kilo, kilos, kg, gramos->convierte), "Liter" (litro, litros, l), "Unidad" (unidad, unidades, paquete, caja), "Portion" (porción, porciones, ración). Ejemplo: si dice "veinte kilos", unidad = "Kilogram", NUNCA "kilos".

Intenciones posibles: contar, corregir, consultar, explicar, crear, navegar, ayuda, cerrar, pedir, legalizar, reporte, ver_receta.

Cuando la intención es "pedir", extrae además preparación y porciones (ejemplo: "hoy preparamos cincuenta ajiacos" -> preparación "AJIACO", porciones 50).

10. Cuando la persona pide de una vez generar, descargar o mandar el archivo/reporte/consolidado ("generame el archivo", "descarga el reporte", "mandame el consolidado"), sin que se lo hayas ofrecido vos antes en este turno, la intención es "reporte" - no "contar" ni "consultar". Responde con algo breve confirmando que lo vas a generar

("de una, te lo genero" / "listo, ya te lo mando"), sin inventar que ya quedo hecho: el sistema es el que arma el archivo de verdad despues de tu respuesta.

11. Cuando la persona pide ver, consultar o que le muestre la receta o la preparacion de un plato ("quiero ver la receta", "muestrame la receta", "cual es la preparacion", "como se hace"), la intencion es "ver_receta" - no "consultar" (eso es para stock de un producto) ni "pedir" (eso es para calcular insumos). Si el plato no queda claro en esta misma frase, extraelo igual en "preparacion" si aparece mencionado, o dejalo vacio si de verdad no se menciona ninguno.

Responde SOLO con un JSON con estas llaves:
intencion, articulo_texto, cantidad, unidad, preparacion, porciones,
respuesta_hablada.""

```
_cliente = None
```

```
def _obtener_cliente():
    global _cliente
    if _cliente is None:
        from google import genai
        llave = os.getenv("GOOGLE_API_KEY", "").strip()
        if not llave:
            return None
        _cliente = genai.Client(api_key=llave)
    return _cliente

def pensar(contexto: str, frase: str) -> dict:
    ""Devuelve la intencion y los datos. Si Gemini falla, cae con gracia.""
    try:
        from google.genai import types
        cliente = _obtener_cliente()
        if cliente is None:
            return _respaldo(frase, "Falta la llave de Google AI Studio en el archivo .env.")

        r = _generar_con_limite(
            cliente,
            model=os.getenv("MODELO", "gemini-flash-latest"),
            contents=f"{contexto}\nLa persona dice: {frase}",
            config=types.GenerateContentConfig(
                system_instruction=PROMPT_SISTEMA,
                response_mime_type="application/json",
                temperature=0.2,
            ),
        )
        datos = json.loads(r.text)
        if not isinstance(datos, dict):
            raise ValueError("respuesta inesperada")
        if datos.get("unidad"):
            from servicios.interprete import normalizar_unidad
            datos["unidad"] = normalizar_unidad(datos["unidad"])
        _limpiar_si_niega(frase, datos)
        return datos

    except Exception as e:
        print(f"[cerebro] Gemini no respondió: {e}")
        return _respaldo(frase)
```

```
NEGACIONES = ("no quiero", "no voy a", "no es ", "no era", "cancela",
```

```

    "cancelar", "ya no ", "no necesito", "no me sirve", "quitalo",
    "quítalo", "elimina eso", "olvida eso", "olvidalo", "olvidalo")

def _limpiar_si_niega(frase: str, datos: dict):
    """Respaldo determinista: si el prompt no bastara, una frase que niega
    o cancela algo NUNCA debe quedar como si la persona hubiera pedido o
    confirmado ese dato ("no quiero preparar arroz" no es un pedido de
    arroz). No depende de que el modelo obedezca la instruccion al pie
    de la letra - igual que la normalizacion de unidades."""
    f = frase.lower()
    if not any(n in f for n in NEGACIONES):
        return
    for campo in ("preparacion", "porciones", "articulo_texto", "cantidad"):
        datos[campo] = None
    if (datos.get("intencion") or "").lower() in ("pedir", "contar"):
        datos["intencion"] = "explicar"

def _respaldo(frase: str, mensaje: str = None) -> dict:
    """Sin Gemini el conteo no se detiene: se interpreta lo basico aqui."""
    from servicios.interprete import interpretar_local
    datos = interpretar_local(frase)
    if mensaje:
        datos["respuesta_hablada"] = mensaje
    return datos

```

PROMPT_ASISTENTE = """Eres CuentaVoz, el asistente de inventarios de Colsubsidio. La persona esta en una pantalla de la aplicacion (no esta contando ni armando un pedido) y le hace una pregunta o le pide ir a otra pantalla. Hable como una persona colombiana cercana y profesional, en frases cortas.

Reglas:

1. Responda SOLO con datos que aparezcan en el contexto que se le da. Si no tiene el dato, dígallo con honestidad en vez de inventar un numero.
2. Si la persona pide ir a otra pantalla ("llevame a bodegas", "abre mi perfil", "muéstrame los reportes"), la intencion es "navegar" y "destino" debe ser EXACTAMENTE una de las claves de la lista de pantallas disponibles del contexto - nunca invente una clave que no este ahi, y si la pantalla que piden no esta en la lista (por ejemplo porque su perfil no tiene acceso), dígallo en vez de forzar un destino.
3. Si no pide navegar, la intencion es "responder": conteste con el contexto que tiene, o explique brevemente que se hace en esta pantalla.
4. Nunca prometa una accion que esta pantalla no puede hacer todavia por voz (cambiar una configuracion, generar un archivo, aprobar algo): solo responde preguntas y navega. Si se lo piden, dígallo con claridad y sugiera usar los botones de la pantalla.
5. Algunas pantallas tienen pestañas propias (se listan en el contexto como "destino > pestaña"). Si la persona pide una pestaña especifica ("llevame al analisis de consumo", "abre la gestion de usuarios"), no se queda a medio camino: pone el destino correcto Y "pestaña" con la clave exacta de esa pestaña, para llegar directo sin dar clic despues. Si solo pide la pantalla en general, sin mencionar pestaña, deje "pestaña" en null - no adivine una.
6. Si lo que dice la persona no es una pregunta clara ni una orden reconocible (una palabra suelta sin sentido para esta pantalla, algo en otro idioma, texto que parece ruido o un eco del reconocimiento de voz), dígallo con honestidad ("no le entendí eso" o similar) en vez de responder de una con la explicacion general de la pantalla como si la hubiera pedido - no finja haber entendido una pregunta que no hicieron.

Responde SOLO con un JSON con estas llaves: intencion (navegar|responder),

```

destino, pestana, respuesta_hablada."""

def pensar_asistente(contexto: str, frase: str, destinos: dict[str, str],
                    pestanas: dict[str, dict[str, str]] | None = None) -> dict:
    """Version liviana de pensar(), para las pantallas que no son de
    conteo ni de pedido (Inicio, Ajustes, Ayuda, Reportes, Panel): mismo
    modelo y mismo cliente, pero un prompt propio y mas simple - sin el
    estado de "pendiente"/"opciones" que solo tiene sentido a mitad de un
    conteo o de un pedido en curso. pestanas permite pedir de una vez una
    pestaña especifica de un destino con pestañas propias (ver main.py)."""
    pestanas = pestanas or {}
    lista = "\n".join(f"- {clave}: {etiqueta}" for clave, etiqueta in destinos.items())
    for destino, sub in pestanas.items():
        for clave_tab, etiqueta_tab in sub.items():
            lista += f"\n- {destino} > {clave_tab}: {etiqueta_tab}"
    contexto_completo = f"{contexto}\nPantallas disponibles para navegar:\n{lista}"
    try:
        from google.genai import types
        cliente = _obtener_cliente()
        if cliente is None:
            return _respaldo_asistente(frase, destinos,
                                       "Sin conexión con el agente en este momento.")

        r = _generar_con_limite(
            cliente,
            model=os.getenv("MODELO", "gemini-flash-latest"),
            contents=f"{contexto_completo}\nLa persona dice: {frase}",
            config=types.GenerateContentConfig(
                system_instruction=PROMPT_ASISTENTE,
                response_mime_type="application/json",
                temperature=0.2,
            ),
        )
        datos = json.loads(r.text)
        if not isinstance(datos, dict):
            raise ValueError("respuesta inesperada")
        return datos
    except Exception as e:
        print(f"[cerebro] Asistente sin Gemini: {e}")
        return _respaldo_asistente(frase, destinos)

def _respaldo_asistente(frase: str, destinos: dict[str, str], mensaje: str = None) -> dict:
    """Sin Gemini, al menos reconoce «lleveme a <pantalla>» por palabra
    clave - el resto queda honesto (no inventa una respuesta) en vez de
    fingir que entendio. No intenta adivinar la pestaña sin el modelo."""
    f = frase.lower()
    for clave, etiqueta in destinos.items():
        if clave in f or etiqueta.lower() in f:
            return {"intencion": "navegar", "destino": clave, "pestana": None,
                   "respuesta_hablada": f"Vamos a {etiqueta.lower()}."}
    return {"intencion": "responder", "destino": None, "pestana": None,
           "respuesta_hablada": mensaje or
           "No pude entenderlo sin conexión al agente. Puede escribir su pregunta."}

def _pcm_a_wav(pcm: bytes, tasa: int = 24000, canales: int = 1, bits: int = 16) -> bytes:
    """Gemini TTS devuelve PCM crudo (sin encabezado): el navegador
    necesita un WAV valido para poder reproducirlo con <audio>/Audio()."""
    buf = io.BytesIO()
    with wave.open(buf, "wb") as w:
        w.setnchannels(canales)
        w.setsampwidth(bits // 8)
        w.setframerate(tasa)
        w.writeframes(pcm)

```

```

return buf.getvalue()

def sintetizar_voz(texto: str, voz: str = VOZ_DEFECTO) -> bytes | None:
    """Convierte texto a un WAV con voz neuronal real. None si Gemini no
    esta disponible - quien llama decide si cae de respaldo a la voz del
    navegador (que suena distinta a la elegida en Mi perfil, asi que cada
    caida de vuelta ahi es una voz distinta hablando a mitad de sesion)."""
    cliente = _obtener_cliente()
    if cliente is None or not texto.strip():
        return None
    nombre_voz = VOCES.get(voz, VOCES[VOZ_DEFECTO])["nombre"]
    from google.genai import types
    config = types.GenerateContentConfig(
        response_modalities=["AUDIO"],
        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(voice_name=nombre_voz)
            )
        ),
    )
    # Un reintento, y solo si la primera vez se agoto el tiempo (se ha visto
    # que Gemini a veces tarda una vez bajo carga y responde rapido la
    # siguiente) - no ante cualquier otra falla (cuota agotada, por
    # ejemplo), donde insistir de una solo suma espera sin cambiar el
    # resultado y hace mas notoria la caida a la voz del navegador.
    for intento, limite in ((1, 12), (2, 6)):
        try:
            r = _generar_con_limite(cliente, limite=limite, model=MODELO_VOZ,
                                   contents=texto, config=config)
            parte = r.candidates[0].content.parts[0].inline_data
            return _pcm_a_wav(parte.data)
        except concurrent.futures.TimeoutError as e:
            print(f"[cerebro] Voz neuronal tardo demasiado (intento {intento}): {e}")
            continue
        except Exception as e:
            print(f"[cerebro] Voz neuronal no disponible: {e}")
            return None
    return None

```

12.17 El orquestador: donde el pensar se vuelve acción

Archivo: `backend/agente/orquestador.py` — Creció bastante desde la versión anterior: además de contar y corregir, ahora resuelve crear bodegas, calcular pedidos, legalizar el servicio y contestar consultas por voz.

```

"""Donde el pensar se vuelve accion. El agente propone, el backend dispone."""
from datetime import datetime
from bd import Sesión
from modelos import (Conteo, StockSistema, SesiónConteo, Bodega, Alerta,
                    Artículo, AsignaciónBodega, Usuario)
from servicios.conciliación import buscar_articulo, aprender_alias, buscar_bodega
from servicios.validación import validar_conteo
from agente.cerebro import pensar

ESTADOS: dict[int, dict] = {}          # memoria viva por sesión

ALTA = 90
MEDIA = 60

def procesar_turno(texto: str, sesión_id: int, usuario=None,
                  opciones_respaldo=None, opciones_para_respaldo=None,
                  bodega_id_respaldo=None, bodega_nombre_respaldo=None,

```

```

        preparacion_respaldo=None, porciones_respaldo=None) -> dict:
est = ESTADOS.setdefault(sesion_id, {})
# el frontend usa sesion_id negativo para la conversacion de Pedidos,
# que no tiene bodega fisica que abrir - ver Pedido.jsx.
modo_pedido = sesion_id < 0

# resiliencia: si el backend se reinicio (se pierde ESTADOS en memoria)
# justo mientras la persona tenia una pregunta de "¿cual de los dos?"
# pendiente, el frontend ya sabe cuales eran las opciones - se las
# recuerda al backend en vez de dejar que pregunte lo mismo otra vez
# sin ningun recuerdo de la conversacion.
if opciones_respaldo and not est.get("opciones"):
    est["opciones"] = opciones_respaldo
    est["opciones_para"] = opciones_para_respaldo or "consultar"

# el mismo respaldo, para la bodega abierta: sin esto, un reinicio del
# backend a mitad de un conteo deja el frontend mostrando "en conteo"
# con la bodega en el titulo, mientras el servidor ya no tiene
# bodega_id - el agente termina diciendo "no hay ninguna bodega
# activa" aunque en pantalla se vea abierta, y "contar" cae en el
# caso de "articulo no encontrado" para cualquier cosa que se dicte.
if bodega_id_respaldo and not est.get("bodega_id"):
    est["bodega_id"] = bodega_id_respaldo
    est["bodega_nombre"] = bodega_nombre_respaldo or est.get("bodega_nombre")

# el mismo respaldo, para el plato y las porciones del pedido: el chef
# casi siempre los dicta en dos frases separadas ("arroz con pollo" y
# luego "para cuatro"), asi que sin esto un reinicio de por medio deja
# al agente sin memoria del plato aunque la pantalla lo siga mostrando.
if preparacion_respaldo and not est.get("preparacion"):
    est["preparacion"] = preparacion_respaldo
if porciones_respaldo and not est.get("porciones"):
    est["porciones"] = porciones_respaldo

bodega_id = est.get("bodega_id")

if modo_pedido:
    contexto = ("Estamos en el modulo de Pedidos: aqui se dicta el plato y las "
        "porciones para calcular insumos por receta, y se puede consultar "
        "el stock de un producto. NUNCA se abre ni se cuenta una bodega "
        "fisica aqui - eso es en el modulo de Conteo, no lo mencione. ")
    # sin esto, Gemini no tiene memoria entre turnos: si el plato se dijo
    # dos frases atras y esta ahora solo llegan las porciones, vuelve a
    # preguntar "¿de que plato estamos hablando?" como si nunca se
    # hubiera dicho - la persona termina repitiendo lo mismo varias veces.
    if est.get("preparacion"):
        contexto += f"El plato ya lo dijo antes: {est['preparacion']}. "
    if est.get("porciones"):
        contexto += f"Las porciones ya las dijo antes: {est['porciones']:g}. "
    if est.get("preparacion") or est.get("porciones"):
        contexto += ("No vuelva a preguntar por el plato ni las porciones si ya "
            "aparecen arriba: siga adelante con lo que falte, o confirme "
            "que ya tiene todo. ")
    else:
        contexto = f"Bodega activa: {est.get('bodega_nombre') or 'ninguna'}. "
    if est.get("pendiente"):
        p = est["pendiente"]
        contexto += f"Pendiente de confirmar: {p['nombre']} {p['cantidad']:g}. "
    if est.get("opciones"):
        contexto += "Opciones ofrecidas: " + ", ".join(
            o["nombre"] for o in est["opciones"]) + ". "

turno = pensar(contexto, texto)
turno.setdefault("respuesta_hablada", "")
intencion = (turno.get("intencion") or "").lower()

```

```

# "cero porciones" (o un numero negativo) no es un pedido valido - sin
# esto, Gemini a veces lo aceptaba igual ("anotado cero porciones de
# ajiaco") y el frontend, al tratar 0 como "sin dato" (0 es falsy en
# JS), terminaba en un estado inconsistente: el plato quedaba puesto
# pero las porciones sin guardar, sin ningun aviso claro de que hacia
# falta corregir el numero.
if modo_pedido and turno.get("porciones") is not None and turno["porciones"] <= 0:
    # el nombre del plato SI se guarda (si vino uno nuevo) - solo el
    # numero invalido queda sin guardar, para no perder de vista de
    # que plato se esta hablando mientras se corrige la cantidad.
    if turno.get("preparacion"):
        est["preparacion"] = turno["preparacion"]
    plato_dicho = (turno.get("preparacion") or est.get("preparacion") or "ese plato").lower()
    turno["porciones"] = None
    turno["respuesta_hablada"] = ("Las porciones tienen que ser un número mayor a "
                                  f"cero. ¿Para cuántas porciones prepara {plato_dicho}?")

    return turno

if modo_pedido:
    # guarda lo nuevo que se haya dicho esta vez, y si esta vez no vino
    # nada, rellena con lo que ya se sabia - asi el frontend no pierde
    # de vista el plato solo porque este turno hablo unicamente de las
    # porciones (o al revés).
    if turno.get("preparacion"):
        if est.get("preparacion") and turno["preparacion"] != est["preparacion"]:
            # cambio de plato: las porciones del plato anterior no sirven
            # para este - hay que volver a preguntar, no asumir el mismo
            # numero (ej. si antes eran 4 porciones de ajiaco, un sancocho
            # nuevo no tiene por que ser tambien 4).
            est["porciones"] = None
            est["preparacion"] = turno["preparacion"]
        elif est.get("preparacion"):
            turno["preparacion"] = est["preparacion"]
    if turno.get("porciones"):
        est["porciones"] = turno["porciones"]
    elif est.get("porciones"):
        turno["porciones"] = est["porciones"]
if modo_pedido and intencion == "contar":
    # en Pedidos nunca se registra un conteo fisico: mencionar un
    # producto aqui solo consulta su stock, no lo cuenta.
    intencion = "consultar"
    turno["intencion"] = "consultar"

palabras = texto.lower().replace(", ", " ").replace(".", " ").split()
dice_si = (any(p in ("si", "sí", "claro", "listo", "dale", "correcto")
               for p in palabras)
            or any(k in texto.lower() for k in
                  ("confirmo", "confirmar", "asi es", "así es", "eso es")))
dice_no = ("no" in palabras
            or any(k in texto.lower() for k in
                  ("no gracias", "mejor no", "tampoco", "no por ahora")))

# — 0) declina lo que se le esta preguntando (oferta de archivo o
# "¿cual de los dos?"): sin esto, un "no" caia en cualquier intencion que
# Gemini le hubiera adivinado a esa palabra sola (a veces "corregir"), y
# el agente terminaba preguntando algo sin relacion como "¿cual es el
# valor correcto?" - la conversacion quedaba trabada en un tema que la
# persona ya habia cerrado.
if dice_no and not est.get("pendiente"):
    if est.get("bodega_pendiente"):
        est["bodega_pendiente"] = None
        turno["intencion"] = "explicar"
        turno["respuesta_hablada"] = "Listo, no la abro. ¿Cuál bodega es?"
        return turno
    if est.get("oferta_reporte") and not est.get("opciones"):

```



```

    est["oferta_reporte"] = False
    turno["intencion"] = "explicar"
    turno["respuesta_hablada"] = "Listo, no genero nada. ¿Algo más en lo que le ayude?"
    return turno
if est.get("opciones"):
    est["opciones"] = None
    est.pop("opciones_para", None)
    turno["intencion"] = "explicar"
    turno["respuesta_hablada"] = "Listo, cancelado. ¿En qué más le ayudo?"
    return turno

# — 1) confirma lo pendiente: aqui si se guarda —
if est.get("pendiente") and (dice_si or intencion == "confirmar"):
    return _guardar(est, sesion_id, usuario)

# — 1z) confirma la bodega que se ofrecio abrir: aqui si se abre.
# Sin este paso, el nombre de una bodega dicho por voz se enviaba
# directo a abrirla - si el reconocimiento de voz entendio mal un
# nombre parecido (visto en produccion: "autoservicios las fuentes"
# -> "AUTOSERVICIOS CASCADA"), la persona quedaba metida en la bodega
# equivocada sin haber podido revisar antes.
if est.get("bodega_pendiente") and (dice_si or intencion == "confirmar"):
    return _confirmar_abrir_bodega(est, usuario)

# — 1b) confirma la oferta de generar el archivo tras una consulta —
if (est.get("oferta_reporte") and not est.get("pendiente")
    and not est.get("opciones")
    and (dice_si or intencion in ("confirmar", "reporte"))):
    est["oferta_reporte"] = False
    return _generar_reporte_voz(usuario)

# — 1c) pide generar el archivo de una vez, sin que se le haya
# ofrecido antes: "generame el archivo" dicho de la nada. Sin esto el
# agente respondia con un mensaje que sonaba a que si lo iba a hacer
# ("de una, te lo genero") pero nunca llamaba a generar nada de
# verdad - la persona se quedaba con el mensaje y ningun archivo.
if intencion == "reporte" and not est.get("pendiente") and not est.get("opciones"):
    if getattr(usuario, "perfil", None) != "auditor":
        turno["respuesta_hablada"] = ("Generar el consolidado lo hace el "
                                     "administrador de bodega; avísele.")
    return turno
return _generar_reporte_voz(usuario)

# confirma sin nada pendiente: hay que decirlo, no fingir que guardo
if (dice_si or intencion == "confirmar") and not est.get("pendiente"):
    if est.get("opciones"):
        turno["respuesta_hablada"] = _leer_opciones(est["opciones"])
        turno["opciones"] = est["opciones"]
        turno["opciones_para"] = est.get("opciones_para")
    elif modo_pedido:
        turno["respuesta_hablada"] = ("No hay nada pendiente de confirmar. Dígame "
                                     "el plato y las porciones, o pregúnteme por "
                                     "un producto.")
    elif not bodega_id:
        turno["respuesta_hablada"] = ("Primero abra una bodega: diga "
                                     "«iniciar conteo en» y el nombre.")
    else:
        turno["respuesta_hablada"] = ("No hay nada pendiente de confirmar. "
                                     "Dicteme el producto y la cantidad.")
    return turno

# — 2) habia opciones y eligio una —
if est.get("opciones"):
    elegido = _elegir(texto, est["opciones"])
    if elegido:

```

```

    para = est.pop("opciones_para", "contar")
    est["opciones"] = None
    if para == "consultar":
        turno["respuesta_hablada"] = resumen_stock(elegido)
        turno["producto_consultado"] = {"codigo": elegido["codigo"],
                                         "nombre": elegido["nombre"],
                                         "unidad": elegido["unidad"]}

        est["oferta_reporte"] = True
        return turno
    return _dejar_pendiente(est, elegido, est.get("cantidad", 1), texto, turno)

# — 3) abrir bodega —
if intencion == "navegar":
    est["oferta_reporte"] = False
    if modo_pedido:
        turno["respuesta_hablada"] = ("Aquí en Pedidos no se abren bodegas, eso es "
                                       "en el módulo de Conteo. Dígame el plato y las "
                                       "porciones, o pregúnteme por un producto.")

        return turno
    return _abrir(est, turno.get("bodega_texto") or texto, turno, usuario)

# — 4) contar —
if intencion == "contar":
    est["oferta_reporte"] = False
    if not modo_pedido and not bodega_id:
        # sin bodega abierta no hay donde guardar un conteo: decirlo,
        # no ofrecer crear un producto nuevo con lo que se haya oído.
        turno["respuesta_hablada"] = ("Primero abra una bodega: diga "
                                       "«iniciar conteo en» y el nombre.")

        return turno
    cand = buscar_articulo(turno.get("articulo_texto") or texto, bodega_id)
    if not cand:
        turno["respuesta_hablada"] = ("No encuentre ese artículo en el catálogo. "
                                       "¿Lo creamos? Quedara pendiente de aprobación.")

        turno["intencion"] = "crear"
        return turno
    est["cantidad"] = turno.get("cantidad") or 1
    est["unidad_dicha"] = turno.get("unidad")
    resultado, dato = _resolver_candidato(cand)
    if resultado == "directo":
        return _dejar_pendiente(est, dato, est["cantidad"], texto, turno)
    if resultado == "ambiguo":
        est["opciones"] = dato
        est["opciones_para"] = "contar"
        turno["opciones"] = dato
        turno["opciones_para"] = "contar"
        turno["respuesta_hablada"] = _leer_opciones(dato)
        return turno
    turno["respuesta_hablada"] = ("No estoy seguro de cual es. "
                                   "¿Me lo repite o lo deletrea?")

    return turno

# — 5) corregir el ultimo —
if intencion == "corregir":
    est["oferta_reporte"] = False
    return _corregir(est, sesion_id, turno, usuario)

# — 5b) ver la receta / preparacion de un plato, por voz —
if intencion == "ver_receta" and modo_pedido:
    plato_pedido = turno.get("preparacion") or est.get("preparacion")
    if not plato_pedido:
        turno["respuesta_hablada"] = "¿La receta de qué plato quiere ver?"
        return turno
    from servicios.recetas import detalle_receta
    r = detalle_receta(plato_pedido)
    if not r:

```

```

    turno["respuesta_hablada"] = (f"No encontré una receta para «{plato_pedido}». "
                                "Pruebe con ajiao o sancocho.")

    return turno
    turno["receta"] = r
    n_ing = len(r.get("lineas", []))
    turno["respuesta_hablada"] = (f"Aquí tiene la receta de {r['nombre'].lower()}: "
                                f"{n_ing} ingredientes" +
                                (" y la preparación paso a paso." if r.get("preparacion")
                                else ".") +
                                " ¿Calculamos el pedido con esta receta?")

    return turno

# — 6) consultar —
if intencion == "consultar":
    texto_busqueda = turno.get("articulo_texto")
    if not texto_busqueda:
        # sin un producto nombrado no hay que buscar - "consultemos el
        # stock" a secas no es el nombre de ningun articulo, y antes esto
        # caia en buscar_articulo(texto) completo, que a veces por pura
        # coincidencia de letras encontraba un producto sin relacion.
        turno["respuesta_hablada"] = "¿El stock de qué producto quiere consultar?"
        return turno
    cand = buscar_articulo(texto_busqueda)
    if not cand:
        turno["respuesta_hablada"] = ("No encuentre ese articulo en el catalogo. "
                                    "¿Me dice otro producto?")

        return turno
    resultado, dato = _resolver_candidato(cand)
    if resultado == "ambiguo":
        est["opciones"] = dato
        est["opciones_para"] = "consultar"
        turno["opciones"] = dato
        turno["opciones_para"] = "consultar"
        turno["respuesta_hablada"] = _leer_opciones(dato)
        return turno
    if resultado == "no_seguro":
        turno["respuesta_hablada"] = "No estoy seguro de cual articulo es. ¿Me lo repite?"
        return turno
    turno["respuesta_hablada"] = resumen_stock(dato)
    turno["producto_consultado"] = {"codigo": dato["codigo"], "nombre": dato["nombre"],
                                   "unidad": dato["unidad"]}

    est["oferta_reporte"] = True
    return turno

# — 7) avance —
if "falta" in texto.lower() or "avance" in texto.lower():
    turno["respuesta_hablada"] = avance(sesion_id)
    return turno

if not turno["respuesta_hablada"]:
    turno["respuesta_hablada"] = "Perdon, no le entendi bien. ¿Me lo repite?"
    return turno

# ————— auxiliares —————
def _resolver_candidato(cand):
    """Decide si hay un ganador claro, si hay que preguntar, o si no hay
    certeza. Un puntaje alto NO basta para decidir solo: si el segundo
    candidato esta empatado o casi (como ARROZ contra ARROZ BASMATI, los
    dos al 100 cuando solo se dice «arroz»), ahi es donde el agente se
    confunde de producto sin que la persona se de cuenta. Por eso siempre
    se compara el primero contra el segundo, no el primero solo."""
    if not cand:
        return "vacio", None
    if len(cand) == 1:

```

```

# un solo candidato no es garantia de que sea el correcto: puede ser
# el unico que paso el filtro de buscar_articulo con una coincidencia
# debil (ej. "consultemos" comparte raiz con "consumibles"). Sin este
# piso, ese empate a una sola opcion se daba por bueno igual.
if cand[0]["confianza"] < MEDIA:
    return "no_seguro", None
return "directo", cand[0]
if cand[0]["confianza"] < MEDIA:
    return "no_seguro", None
if cand[0]["confianza"] >= ALTA and (cand[0]["confianza"] - cand[1]["confianza"]) >= 15:
    return "directo", cand[0]
return "ambiguo", cand[:2]

def _leer_opciones(cand):
    return ("Tengo dos parecidos: " + cand[0]["nombre"] + " o " +
            cand[1]["nombre"] + ". ¿Cual de los dos?")

def _dejar_pendiente(est, art, cantidad, texto, turno):
    v = validar_conteo(art["codigo"], cantidad, est.get("unidad_dicha"),
                      est.get("bodega_id"))
    est["pendiente"] = {"articulo_codigo": art["codigo"], "nombre": art["nombre"],
                        "cantidad": cantidad, "unidad": art["unidad"],
                        "texto_original": texto,
                        "alerta": None if v["ok"] else v["tipo"],
                        "detalle_alerta": None if v["ok"] else v["mensaje"]}

    if v["ok"]:
        turno["respuesta_hablada"] = (f"{art['nombre']}: {cantidad:g} "
                                     f"{art['unidad']}. ¿Confirma?")
    else:
        turno["respuesta_hablada"] = v["mensaje"]
        turno["alerta"] = v["tipo"]
        if v["tipo"] == "desviacion":
            # datos limpios para la tarjeta "Contexto de la validacion":
            # antes solo iban mezclados dentro de la frase hablada.
            esperado = v.get("esperado") or 0
            desviacion_pct = round((cantidad - esperado) / esperado * 100) if esperado else None
            turno["contexto_alerta"] = {
                "esperado": esperado, "dicho": cantidad, "unidad": art["unidad"],
                "articulo": art["nombre"], "desviacion_pct": desviacion_pct,
            }
        if v["tipo"] in ("negativo", "unidad", "inexistente"):
            est["pendiente"] = None # no se puede guardar: hay que repetir
            # el intento no se guarda como conteo, pero si queda como alerta
            # de gestion - antes se perdía sin dejar rastro para el panel
            bodega = est.get("bodega_nombre") or "sin bodega"
            with Sesion() as s:
                s.add(Alerta(conteo_id=None, tipo=v["tipo"],
                           detalle=f"{bodega} · {art['nombre']}: {v['mensaje']}"))
                s.commit()
        turno["pendiente"] = est.get("pendiente") and {
            "nombre": art["nombre"], "cantidad": cantidad, "unidad": art["unidad"]}
    return turno

def _guardar(est, sesion_id, usuario):
    from seguridad import registrar
    c = est.pop("pendiente")
    with Sesion() as s:
        reg = Conteo(sesion_id=sesion_id, articulo_codigo=c["articulo_codigo"],
                    cantidad=c["cantidad"], unidad=c["unidad"],
                    estado="confirmado")
        s.add(reg)
        s.commit()

```

```

        s.refresh(reg)
        if c.get("alerta"):
            s.add(Alerta(conteo_id=reg.id, tipo=c["alerta"],
                        detalle=c["detalle_alerta"] or ""))
            s.commit()
            est["ultimo_id"] = reg.id
        aprender_alias(c.get("texto_original", ""), c["articulo_codigo"],
                      est.get("bodega_id"))
    if usuario:
        registrar(usuario, "CONTEO",
                  f"{c['nombre']}: {c['cantidad']:g} {c['unidad']}")
    return {"intencion": "guardado",
            "respuesta_hablada": "Guardado. " + avance(sesion_id),
            "guardado": True}

def _corregir(est, sesion_id, turno, usuario):
    """Corrige lo que esta pendiente; si no hay nada pendiente, el ultimo guardado."""
    from seguridad import registrar
    nueva = turno.get("cantidad")
    if nueva is None:
        turno["respuesta_hablada"] = "¿Cual es el valor correcto?"
        return turno

    # 1) lo mas comun: la persona corrige antes de confirmar («uy, no: son nueve»)
    p = est.get("pendiente")
    if p:
        art = {"codigo": p["articulo_codigo"], "nombre": p["nombre"],
              "unidad": p["unidad"]}
        est["cantidad"] = nueva
        return _dejar_pendiente(est, art, nueva, p.get("texto_original", ""), turno)

    # 2) si ya se habia guardado, se crea un registro que apunta al original
    ultimo_id = est.get("ultimo_id")
    if not ultimo_id:
        turno["respuesta_hablada"] = "Todavia no hay un registro que corregir."
        return turno
    with Sesion() as s:
        orig = s.get(Conteo, ultimo_id)
        if orig is None:
            turno["respuesta_hablada"] = "No encuentro ese registro."
            return turno
        orig.estado = "corregido" # nada se borra
        nuevo = Conteo(sesion_id=sesion_id, articulo_codigo=orig.articulo_codigo,
                      cantidad=nueva, unidad=orig.unidad,
                      estado="confirmado", corrige_a=orig.id)
        s.add(nuevo)
        s.commit()
        s.refresh(nuevo)
        est["ultimo_id"] = nuevo.id
        art = s.get(Articulo, orig.articulo_codigo)
        nombre = art.nombre_oficial if art else orig.articulo_codigo
    if usuario:
        registrar(usuario, "CORRECCION",
                  f"{nombre}: {orig.cantidad:g} -> {nueva:g} "
                  f"(valor anterior conservado)", "alerta")
    turno["respuesta_hablada"] = f"Corregido a {nueva:g}. Quedo registrado."
    turno["corregido"] = True
    return turno

def _completar_apertura(s, b, est, usuario):
    """La apertura de verdad: crea o reutiliza la sesion, marca la bodega
    en_conteo y arma la respuesta. La llaman tanto _abrir() (cuando la
    bodega ya la tiene la misma persona, no hay nada que confirmar) como

```

```

_confirmar_abrir_bodega() (despues del "si" a la pregunta de
confirmacion)."""
from seguridad import registrar
abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
                                         estado="abierta").first()
ses = abierta or SesionConteo(bodega_id=b.id, usuario_id=getattr(usuario, "id", 1))
if not abierta:
    s.add(ses)
    b.estado = "en_conteo"
    s.commit()
    s.refresh(ses)
refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
est["bodega_id"] = b.id
est["bodega_nombre"] = b.nombre_oficial
est["sesion_bd"] = ses.id
nombre = b.nombre_oficial
if usuario:
    registrar(usuario, "APERTURA", f"{nombre} abierta - sesion bloqueada")
return {"respuesta_hablada": (f"Listo, {nombre.lower()} abierta con "
                             f"{refs} referencias. Dícteme el primer producto."),
        "bodega": {"id": b.id, "nombre": nombre, "referencias": refs}}

def _confirmar_abrir_bodega(est, usuario):
    """Ya se pregunto "¿confirma que abro X?" y la persona dijo que si:
    aqui es cuando de verdad se crea/reusa la sesion de conteo. Antes de
    esto, decir el nombre de una bodega no habia tocado la base para nada
    - si el reconocimiento de voz entendio mal, un "no" simplemente
    descarta la oferta sin haber abierto nada."""
    pendiente = est.get("bodega_pendiente")
    est["bodega_pendiente"] = None
    turno = {}
    with Sesion() as s:
        b = s.get(Bodega, pendiente["id"])
        if b is None:
            turno["respuesta_hablada"] = "Esa bodega ya no existe. ¿Cuál abro?"
            return turno
        # revalida que siga libre: pudo haberla tomado otra persona en el
        # rato entre la pregunta y el "si".
        abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
                                                  estado="abierta").first()
        if abierta and abierta.usuario_id != getattr(usuario, "id", None):
            quien = s.get(Usuario, abierta.usuario_id)
            nombre_quien = quien.nombre.capitalize() if quien else "otra persona"
            turno["respuesta_hablada"] = (f"{b.nombre_oficial} ya la tiene {nombre_quien} "
                                         "en conteo; no la abro.")
            return turno
        turno.update(_completar_apertura(s, b, est, usuario))
    return turno

def _abrir(est, texto_bodega, turno, usuario):
    with Sesion() as s:
        # buscar_bodega() (servicios/conciliacion.py) es la misma funcion
        # que usa /api/bodegas/abrir: una sola vez quita tildes y
        # puntuacion de sobra, prioriza la coincidencia exacta, y solo si
        # no hay exacta cae al respaldo por palabras. Restringida a lo
        # asignado si es auxiliar - no debe ni encontrar una bodega de
        # otra zona, que igual el chequeo de abajo le negaria despues.
        ids_permitidos = None
        if getattr(usuario, "perfil", None) == "auxiliar":
            ids_permitidos = {a.bodega_id for a in
                             s.query(AsignacionBodega).filter_by(usuario_id=usuario.id).all()}
        b = buscar_bodega(s, texto_bodega, ids_permitidos)
        if b is None:
            # igual que un articulo que no esta en el catalogo: en vez de

```

```

# dejar a la persona repitiendo el nombre sin salida, se ofrece
# pedir la bodega nueva (queda pendiente de aprobacion, ver
# /api/bodegas/crear-pendiente).
turno["intencion"] = "crear_bodega"
turno["bodega_texto"] = texto_bodega
turno["respuesta_hablada"] = (f"No encuentro «{texto_bodega}». ¿La creamos? "
                             "Quedará pendiente de aprobación del administrador.")

return turno

# la misma regla que /api/bodegas/abrir: decirlo por voz no debe
# abrir puertas que el mismo pedido por REST tiene cerradas. Sin
# esto, un auxiliar podia saltarse la asignacion de zona con solo
# dictar el nombre de una bodega ajena.
if (getattr(usuario, "perfil", None) == "auxiliar"
    and not s.query(AsignacionBodega).filter_by(
        usuario_id=usuario.id, bodega_id=b.id).first()):
    turno["respuesta_hablada"] = f"{b.nombre_oficial} no esta asignada a usted."
    return turno

abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
                                           estado="abierta").first()

if abierta and abierta.usuario_id != getattr(usuario, "id", None):
    # antes decia "por otra persona" a secas - la persona no sabia
    # si era normal (alguien de verdad ya la esta contando) o un
    # error, ni con quien hablar para resolverlo.
    quien = s.get(Usuario, abierta.usuario_id)
    nombre_quien = quien.nombre.capitalize() if quien else "otra persona"
    turno["respuesta_hablada"] = (f"{b.nombre_oficial} ya la tiene {nombre_quien} "
                                "en conteo; no la abro.")

    return turno

if abierta and abierta.usuario_id == getattr(usuario, "id", None):
    # ya es suya (la dejo a medias): seguir donde iba no necesita
    # confirmacion, no hay ambigüedad que revisar.
    turno.update(_completar_apertura(s, b, est, usuario))
    return turno

# bodega libre y accesible: antes de tocar la base, se pregunta -
# un nombre parecido mal transcrito ("autoservicios las fuentes"
# sonando a "autoservicios cascada") no debe meter a la persona en
# la bodega equivocada sin que lo haya podido revisar.
est["bodega_pendiente"] = {"id": b.id, "nombre": b.nombre_oficial}
turno["respuesta_hablada"] = f"¿Confirma que abro {b.nombre_oficial.lower()}?"
return turno

def avance(sesion_id) -> str:
    with Sesion() as s:
        hechas = s.query(Conteo).filter_by(sesion_id=sesion_id,
                                           estado="confirmado").count()

        est = ESTADOS.get(sesion_id, {})
        total = (s.query(StockSistema)
                 .filter_by(bodega_id=est.get("bodega_id")).count()
                 if est.get("bodega_id") else 0)

    if total:
        return f"Llevamos {hechas} de {total} referencias."
    return f"Llevamos {hechas} referencias."

def _generar_reporte_voz(usuario) -> dict:
    """Cuando la persona acepta la oferta «¿le genero el archivo?» tras
    una consulta: aquí de verdad se genera, no solo se dice que si."""
    import os
    import reportes
    ruta, filas, _vista_previa = reportes.consolidado("xlsx")
    if usuario:
        from seguridad import registrar
        registrar(usuario, "REPORTE", f"Consolidado generado por voz: {ruta} ({filas} filas)")
    return {"intencion": "reporte", "guardado": True, "archivo": ruta,
            "respuesta_hablada": f"Listo, ya genere el archivo {os.path.basename(ruta)}. "

```

```

        "Lo encuentra en el menu Reportes para descargarlo. "
        "¿Algo más en lo que le ayude?}"

def resumen_stock(a) -> str:
    """El articulo ya viene resuelto (sin ambigüedad): solo arma la respuesta."""
    with Sesion() as s:
        filas = s.query(StockSistema).filter_by(articulo_codigo=a["codigo"]).all()
        total = sum(f.cantidad_sd or 0 for f in filas)
        nombres = []
        for f in filas[:4]:
            b = s.get(Bodega, f.bodega_id)
            if b:
                nombres.append(f"{b.nombre_oficial.lower()} {f.cantidad_sd:g}")
        if not filas:
            return f"{a['nombre']}: no tiene registro de stock en ninguna bodega todavia."
        detalle = "; ".join(nombres)
        return (f"{a['nombre']}: {total:g} {a['unidad']} en {len(filas)} bodegas. "
                f"{detalle}. ¿Le genero el archivo?")

def _elegir(texto, opciones):
    """Empareja «la FB», «la primera», el codigo o una palabra distintiva."""
    t = " " + texto.upper().strip() + " "
    if not opciones:
        return None

    # 1) por codigo dictado
    for o in opciones:
        if o["codigo"] in t:
            return o

    # 2) por posicion
    if any(k in t for k in (" PRIMERA ", " PRIMERO ", " UNO ", " LA UNA ")):
        return opciones[0]
    if any(k in t for k in (" SEGUNDA ", " SEGUNDO ", " DOS ")) and len(opciones) > 1:
        return opciones[1]

    import re
    dichas = set(re.findall(r"[A-Z0-9/]{2,}", t)) - {"LA", "EL", "DE", "ES"}

    # 3) dijo el nombre completo de una opcion, tal cual - incluso si ese
    # nombre esta contenido dentro del de la otra (ARROZ dentro de ARROZ
    # BASMATI). Aqui NO sirve buscar una palabra distintiva porque ARROZ
    # no tiene ninguna que no este tambien en ARROZ BASMATI; lo que importa
    # es que lo dicho coincide exactamente con el conjunto de palabras de
    # esa opcion y no con el de la otra.
    for o in opciones:
        propias = set(re.findall(r"[A-Z0-9/]{2,}", o["nombre"].upper()))
        if dichas == propias:
            return o

    # 4) por palabra o sigla que aparezca en UNA sola de las opciones
    for i, o in enumerate(opciones):
        otras = " ".join(x["nombre"].upper() for j, x in enumerate(opciones) if j != i)
        propias = set(re.findall(r"[A-Z0-9/]{2,}", o["nombre"].upper()))
        ajenas = set(re.findall(r"[A-Z0-9/]{2,}", otras))
        distintivas = propias - ajenas # lo que solo esta en esta opcion
        if dichas & distintivas:
            return o
    return None

```


12.18 Identidad y seguridad

Archivo: backend/seguridad.py — El token JWT ahora incluye una versión de sesión (ver) que se compara contra Usuario.version_token: cambiar el PIN o pedir «cerrar todas las sesiones» invalida todos los tokens anteriores sin necesitar una lista de revocación aparte.

```

"""Identidad: contraseñas cifradas, token firmado y permisos por perfil."""
import os
import secrets
import datetime
import jwt
import bcrypt
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from bd import Sesion
from modelos import Usuario

def _secreto_jwt() -> str:
    """Sin SECRETO_JWT en el entorno, antes se firmaba con una frase fija
    escrita en el código fuente público - cualquiera que leyera el repo
    podía forjar un token válido para cualquier perfil, incluido auditor.
    En Render siempre está configurado (ver render.yaml, generateValue:
    true); aquí solo cubre un arranque local sin .env, y con un secreto
    aleatorio de verdad en vez de uno que cualquiera puede leer."""
    llave = os.getenv("SECRETO_JWT")
    if llave:
        return llave
    print("[seguridad] SECRETO_JWT no está configurado: se generó uno "
          "aleatorio solo para este arranque (las sesiones no sobreviven "
          "un reinicio). Define SECRETO_JWT en su .env para desarrollo "
          "estable, o en las variables de entorno en producción.")
    return secrets.token_hex(32)

SECRETO = _secreto_jwt()
MINUTOS = int(os.getenv("MINUTOS_TOKEN", 480))
esquema = OAuth2PasswordBearer(tokenUrl="/api/ingresar", auto_error=False)

def hash_clave(clave: str) -> str:
    """bcrypt trabaja sobre bytes y no admite más de 72."""
    b = clave.encode("utf-8")[:72]
    return bcrypt.hashpw(b, bcrypt.gensalt()).decode("utf-8")

def verificar_clave(clave: str, guardado: str) -> bool:
    try:
        return bcrypt.checkpw(clave.encode("utf-8")[:72],
                              guardado.encode("utf-8"))
    except Exception:
        return False

def crear_token(u: Usuario) -> str:
    datos = {
        "sub": str(u.id),
        "perfil": u.perfil,
        "ver": u.version_token or 0,
        "exp": datetime.datetime.now(datetime.timezone.utc)
              + datetime.timedelta(minutes=MINUTOS),
    }

```

```

return jwt.encode(datos, SECRETO, algorithm="HS256")

def usuario_actual(token: str = Depends(esquema)) -> Usuario:
    if not token:
        raise HTTPException(401, "Falta la sesion.")
    try:
        datos = jwt.decode(token, SECRETO, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(401, "Sesion invalida o vencida.")
    with Sesion() as s:
        u = s.get(Usuario, int(datos["sub"]))
    if u is None:
        raise HTTPException(401, "Usuario no reconocido.")
    if not u.activo:
        raise HTTPException(403, "Ese usuario esta inactivo.")
    if (datos.get("ver", 0) or 0) != (u.version_token or 0):
        raise HTTPException(401, "Sesion cerrada desde otro dispositivo. Ingrese de nuevo.")
    return u

def verificar_token(token: str):
    """Para WebSockets: el esquema OAuth2 exige el header Authorization, que
    un WebSocket del navegador no puede mandar, así que aquí se valida el
    token pasado por query string con la misma regla que usuario_actual."""
    if not token:
        return None
    try:
        datos = jwt.decode(token, SECRETO, algorithms=["HS256"])
    except jwt.PyJWTError:
        return None
    with Sesion() as s:
        u = s.get(Usuario, int(datos["sub"]))
    if u is None or (datos.get("ver", 0) or 0) != (u.version_token or 0):
        return None
    return u

def requiere_perfil(perfil: str):
    def guardia(u: Usuario = Depends(usuario_actual)) -> Usuario:
        if u.perfil != perfil:
            raise HTTPException(403, "Su perfil no permite esta accion.")
        return u
    return guardia

def registrar(usuario, accion: str, detalle: str, tipo: str = "info"):
    """Deja rastro de una accion sensible. El registro solo crece."""
    from modelos import Traza
    with Sesion() as s:
        s.add(Traza(usuario_id=getattr(usuario, "id", None),
                    persona=getattr(usuario, "nombre", "sistema"),
                    accion=accion, detalle=detalle, tipo=tipo))
    s.commit()

```

12.19 La API completa: endpoints y tablero en vivo

Archivo: `backend/main.py` — ~90 endpoints REST + el WebSocket del tablero — más del triple que en la versión anterior de este manual. Es el archivo más grande del proyecto: aquí vive literalmente cada ruta que el frontend llama.

```

"""La API de CuentaVoz. Aqui se conectan la tableta, el agente y la base."""
import json
import os
import re
import secrets
import socket
import unicodedata
import urllib.error
import urllib.request
from datetime import timedelta
from horario import ahora

from dotenv import load_dotenv
load_dotenv()

from fastapi import (FastAPI, WebSocket, WebSocketDisconnect, Depends,
                     HTTPException, UploadFile, File, Request)
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse, Response
from fastapi.security import OAuth2PasswordRequestForm
from pydantic import BaseModel
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

from bd import Sesion, iniciar_bd
from modelos import (Usuario, Bodega, Articulo, StockSistema, SesionConteo,
                     Conteo, Alerta, Traza, LineaServicio, AsignacionBodega,
                     Aprobacion, HistorialCierre, ConfigClave,
                     Receta, RecetaIngrediente, CredencialWebAuthn, MensajeSoporte)
from seguridad import (hash_clave, verificar_clave, crear_token,
                       usuario_actual, requiere_perfil, registrar)
from agente.orquestador import procesar_turno, ESTADOS, avance
from servicios.recetas import (calcular_pedido, comparar_legalizacion,
                              analisis_consumo, detalle_receta)
from servicios.validacion import umbral_actual, validar_conteo
from servicios import analitica, huella
from servicios.interprete import _numero as _numero_de_texto
import reportes

app = FastAPI(title="CuentaVoz", version="1.0.0",
              description="Asistente por voz para inventarios · Colsubsidio")

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

_origenes = {o.strip() for o in os.getenv("ORIGEN_PERMITIDO", "").split(",") if o.strip()}
# 5173 es "npm run dev"; 4173 es "npm run preview" (el build de produccion,
# el unico que arma el service worker real - hace falta para probar el
# modo sin conexion tal como queda desplegado, no solo en modo desarrollo).
_origenes |= {"http://localhost:5173", "http://127.0.0.1:5173",
              "http://localhost:4173", "http://127.0.0.1:4173"}

app.add_middleware(
    CORSMiddleware,
    allow_origins=list(_origenes),
    allow_credentials=True, allow_methods=["*"], allow_headers=["*"],
)

@app.middleware("http")
async def cabeceras(request: Request, llamar):

```

```

resp = await llamar(request)
resp.headers["X-Content-Type-Options"] = "nosniff"
resp.headers["X-Frame-Options"] = "SAMEORIGIN"
resp.headers["Referrer-Policy"] = "no-referrer"
# obliga HTTPS en cada visita futura del navegador, no solo en esta -
# sin esto, un enlace http:// (o un portal cautivo de Wi-Fi) puede
# interceptar la primera petición antes de que el servidor la redirija.
resp.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
return resp

@app.on_event("startup")
def arranque():
    iniciar_bd()
    with Sesion() as s:
        if s.query(Usuario).count() == 0:
            s.add_all([
                Usuario(nombre="luis", perfil="auxiliar",
                        clave_hash=hash_clave("StockXperts"),
                        correo="lnieto@colsubsidio.com", codigo="CS-48127"),
                Usuario(nombre="diana", perfil="auditor",
                        clave_hash=hash_clave("StockXperts"),
                        correo="diana@colsubsidio.com", codigo="CS-48200"),
                Usuario(nombre="stephanie", perfil="auxiliar",
                        clave_hash=hash_clave("StockXperts"), codigo="CS-48311"),
                Usuario(nombre="valentina", perfil="auxiliar",
                        clave_hash=hash_clave("StockXperts"), codigo="CS-48342"),
            ])
            s.commit()
            print("[arranque] usuarios de prueba creados (clave StockXperts)")

# bodegas asignadas por persona: solo la primera vez, y solo si ya
# hay bodegas cargadas (cargar_excel.py corre antes que la API)
if s.query(AsignacionBodega).count() == 0 and s.query(Bodega).count() > 0:
    # del Excel real de Colsubsidio, solo un subconjunto de bodegas
    # trae existencia de sistema (SD) cargada - las demás son
    # ubicaciones del catálogo sin stock_sistema asociado. Priorizar
    # esas para el reparto demo evita que un auxiliar de prueba abra
    # una bodega con "0 referencias" y no pueda mostrar diferencias
    # contra el sistema en una demo en vivo.
    con_stock = {bid for (bid,) in s.query(StockSistema.bodega_id).distinct()}
    todas = s.query(Bodega).order_by(Bodega.nombre_oficial).all()
    bodegas = sorted(todas, key=lambda b: (b.id not in con_stock, b.nombre_oficial))
    usuarios = {u.nombre: u for u in s.query(Usuario).all()}
    reparto = {"luis": bodegas[0:3], "stephanie": bodegas[3:6],
              "valentina": bodegas[6:9]}

    total = 0
    for nombre, asignadas in reparto.items():
        u = usuarios.get(nombre)
        if not u:
            continue
        for b in asignadas:
            s.add(AsignacionBodega(usuario_id=u.id, bodega_id=b.id))
            total += 1
    s.commit()
    if total:
        print(f"[arranque] {total} bodegas asignadas a los auxiliares de prueba")

# el conteo inicial de negativos, para que el panel muestre "N -> 0"
if s.query(ConfigClave).filter_by(clave="negativos_iniciales").first() is None:
    neg = s.query(StockSistema).filter(StockSistema.cantidad_sd < 0).count()
    s.add(ConfigClave(clave="negativos_iniciales", valor=str(neg)))
    s.commit()

# un par de cierres de ejemplo, para que el histórico de exactitud

```

```

# del panel no arranque vacío (etiquetados como semilla, no reales)
if s.query(HistorialCierre).count() == 0 and s.query(Bodega).count() > 0:
    primera = s.query(Bodega).order_by(Bodega.nombre_oficial).first()
    from datetime import timedelta
    base = ahora() - timedelta(days=120)
    for i, exact in enumerate([94.2, 95.8, 96.5, 97.1]):
        s.add(HistorialCierre(bodega_id=primera.id, exactitud=exact,
                             referencias=0, diferencias=0,
                             fecha=base + timedelta(days=30 * i)))

s.commit()
print("[arranque] histórico de exactitud sembrado (4 puntos de ejemplo)")

@app.get("/api/salud")
def salud():
    with Sesion() as s:
        return {"api": "ok",
                "bodegas": s.query(Bodega).count(),
                "articulos": s.query(Articulo).count(),
                "stock": s.query(StockSistema).count(),
                "gemini": bool(os.getenv("GOOGLE_API_KEY", "").strip())}

# ----- identidad -----
def _buscar_usuario_por_entrada(s, entrada: str):
    """Se puede ingresar con el nombre de usuario o con el código de
    empleado (ej. CS-48127) - lo que la persona tenga a la mano."""
    entrada = entrada.strip()
    return s.query(Usuario).filter(
        (Usuario.nombre == entrada.lower()) | (Usuario.codigo == entrada.upper())
    ).first()

@app.post("/api/ingresar")
@limiter.limit("5/minute")
def ingresar(request: Request, form: OAuth2PasswordRequestForm = Depends()):
    with Sesion() as s:
        u = _buscar_usuario_por_entrada(s, form.username)
        if not u or not verificar_clave(form.password, u.clave_hash):
            raise HTTPException(401, "Usuario o clave incorrectos.")
        if not u.activo:
            raise HTTPException(403, "Ese usuario está inactivo.")
        u.ultimo_acceso = ahora()
        s.commit()
        s.refresh(u)
        registrar(u, "INGRESO", f"{u.nombre} inicio sesión")
    return {"token": crear_token(u), "perfil": u.perfil,
            "usuario": {"id": u.id, "nombre": u.nombre, "perfil": u.perfil}}

@app.get("/api/usuarios/perfil")
@limiter.limit("20/minute")
def perfil_por_usuario(request: Request, usuario: str = ""):
    """Para que la pantalla de ingreso muestre «Auxiliar» o «Administrador»
    apenas se escribe el usuario, sin esperar a iniciar sesión. Solo
    devuelve el perfil y si tiene huella (nada más sensible: ni nombre, ni
    correo) y está limitado por minuto para no servir de lista para
    adivinar usuarios válidos."""
    if not usuario.strip():
        return {"perfil": None, "tiene_huella": False}
    with Sesion() as s:
        u = _buscar_usuario_por_entrada(s, usuario)
        if not u or not u.activo:
            return {"perfil": None, "tiene_huella": False}

```

```

        tiene_huella = s.query(CredencialWebAuthn).filter_by(usuario_id=u.id).first() is not None
    return {"perfil": u.perfil, "tiene_huella": tiene_huella}

@app.get("/api/auth/huella/opciones")
@limiter.limit("20/minute")
def opciones_ingreso_huella(request: Request, usuario: str = ""):
    """Paso 1 del ingreso con huella: reto para navigator.credentials.get()."""
    with Sesion() as s:
        u = _buscar_usuario_por_entrada(s, usuario)
        if not u or not u.activo:
            raise HTTPException(404, "No hay una huella registrada para ese usuario.")
        credenciales = [c.credencial_id for c in
                        s.query(CredencialWebAuthn).filter_by(usuario_id=u.id).all()]
        if not credenciales:
            raise HTTPException(404, "No hay una huella registrada para ese usuario.")
        reto_id, opciones = huella.opciones_ingreso(u.id, credenciales)
    return {"reto_id": reto_id, "opciones": opciones}

class VerificarHuellaIn(BaseModel):
    reto_id: str
    credencial: dict

@app.post("/api/auth/huella/verificar")
@limiter.limit("10/minute")
def verificar_ingreso_huella(request: Request, datos: VerificarHuellaIn):
    cred_id = datos.credencial.get("id") if isinstance(datos.credencial, dict) else None
    with Sesion() as s:
        registro = s.query(CredencialWebAuthn).filter_by(credencial_id=cred_id).first()
        if not registro:
            raise HTTPException(401, "Credencial de huella no reconocida.")
        try:
            usuario_id, nuevo_conteo = huella.verificar_ingreso(
                datos.reto_id, datos.credencial, registro.public_key, registro.sign_count)
        except Exception as e:
            raise HTTPException(401, f"No se pudo verificar la huella: {e}")
        if usuario_id != registro.usuario_id:
            raise HTTPException(401, "Credencial de huella no reconocida.")
        registro.sign_count = nuevo_conteo
        u = s.get(Usuario, usuario_id)
        if not u.activo:
            raise HTTPException(403, "Ese usuario esta inactivo.")
        u.ultimo_acceso = ahora()
        s.commit()
        s.refresh(u)
    registrar(u, "INGRESO", f"{u.nombre} inicio sesion con huella")
    return {"token": crear_token(u), "perfil": u.perfil,
            "usuario": {"id": u.id, "nombre": u.nombre, "perfil": u.perfil}}

# ----- el agente -----
class TurnoIn(BaseModel):
    texto: str
    sesion_id: int = 1
    # respaldo de resiliencia: si el backend se reinicio a mitad de una
    # pregunta "¿cual de los dos?", el frontend ya sabe cuales opciones
    # estaban pendientes y se las recuerda - ver Pedido.jsx/Conteo.jsx.
    opciones_pendientes: list[dict] | None = None
    opciones_para: str | None = None
    # el mismo respaldo mas la bodega abierta: sin esto, un reinicio del
    # backend deja el frontend mostrando "en conteo" mientras el servidor
    # ya no sabe cual bodega es - el agente termina diciendo "no hay

```

```

# ninguna activa" con la bodega bien visible en la pantalla.
bodega_id_respaldo: int | None = None
bodega_nombre_respaldo: str | None = None
# lo mismo para Pedidos: el plato y las porciones casi siempre se dictan
# en frases separadas ("arroz con pollo" y despues "para cuatro"); sin
# este respaldo, un reinicio del backend a mitad de esas dos frases deja
# al agente sin memoria del plato aunque la pantalla lo siga mostrando.
preparacion_respaldo: str | None = None
porciones_respaldo: int | None = None

@app.post("/api/agente/turno")
async def turno(t: TurnoIn, u: Usuario = Depends(usuario_actual)):
    r = procesar_turno(t.texto, t.sesion_id, u,
                      opciones_respaldo=t.opciones_pendientes,
                      opciones_para_respaldo=t.opciones_para,
                      bodega_id_respaldo=t.bodega_id_respaldo,
                      bodega_nombre_respaldo=t.bodega_nombre_respaldo,
                      preparacion_respaldo=t.preparacion_respaldo,
                      porciones_respaldo=t.porciones_respaldo)

    if r.get("bodega"):
        await difundir_estado()
    return r

class PreguntarAsistenteIn(BaseModel):
    texto: str
    vista: str

# claves = lo mismo que usa ir(destino) en App.jsx; deben calzar exacto,
# si no la navegacion por voz apunta a una vista que no existe.
_DESTINOS_ASISTENTE = {
    "inicio": "Inicio", "pedido": "Pedidos", "conteo": "Conteo",
    "legalizacion": "Legalización", "bodegas": "Bodegas",
    "auditoria": "Auditoria", "reportes": "Reportes", "panel": "Panel",
    "ajustes": "Ajustes", "ayuda": "Ayuda", "perfil": "Mi perfil",
}
_SOLO_AUDITOR_ASISTENTE = {"auditoria", "reportes", "panel", "ajustes"}

# pantallas con pestañas propias: para no obligar a la persona a llegar y
# después dar clic en la pestaña que quería, si la pide de una vez
# ("llévame al análisis de consumo") el agente navega directo a ella. La
# clave interna debe ser la misma que usa cada vista en su useState(tab).
_PESTANAS_ASISTENTE = {
    "reportes": {"consolidado": "Consolidado de la toma", "analisis": "Análisis de consumo"},
    "ajustes": {"config": "Configuración", "usuarios": "Gestión de usuarios",
               "recetas": "Recetas", "traza": "Registro de trazabilidad"},
    "panel": {"resumen": "Resumen ejecutivo", "alertas": "Bodegas y alertas"},
    "auditoria": {"recuento": "Recuento ciego y cierre", "aprobaciones": "Aprobaciones",
                 "pedidos": "Pedidos pendientes", "alertas": "Bandeja de alertas"},
}

_FAQ_ASISTENTE = [
    ("¿Cómo corrijo un conteo ya confirmado?",
     "Diga «corregir» y luego el valor correcto; el valor anterior se conserva."),
    ("El agente no entiende un producto",
     "Dígalo como aparece en la etiqueta; si no existe se puede crear, queda pendiente."),
    ("¿Qué hago si sale una alerta?",
     "Recuente; si el número es correcto, confirme: queda marcado para el administrador."),
    ("Se cayó el internet en plena bodega",
     "Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal."),
    ("¿Puedo contar dos bodegas a la vez?",
     "No en el mismo dispositivo; el candado de sesión evita conteos duplicados."),

```

```

]

def _datos_panel_narrados() -> dict:
    """Los mismos datos de /api/panel/resumen y /api/panel/alertas, ya
    convertidos a frases listas para hablar - los usan tanto el contexto
    completo que recibe Gemini como las respuestas determinísticas de
    _responder_panel_por_voz, para no calcular el texto dos veces."""
    r = analitica.resumen_ejecutivo()
    a = analitica.resumen_alertas_panel()
    dif_bod = r["diferencia_por_bodega"]
    texto_dif = ("sin diferencias registradas todavía" if not dif_bod else
                 "; ".join(f"{x['bodega'].title(): {x['diferencia']}" for x in dif_bod[:6]))
    stock = r["stock_por_unidad"]
    texto_stock = ("sin datos de stock todavía" if not stock else
                  ", ".join(f"{x['unidad']} {x['pct']}" (x['cantidad']}" for x in stock))
    hist = r["historial_exactitud"]
    if len(hist) < 2:
        texto_hist = "todavía no hay suficientes cierres para ver una tendencia"
    else:
        tendencia = "mejorando" if hist[-1]["exactitud"] >= hist[0]["exactitud"] else "bajando"
        texto_hist = (f"{len(hist)} tomas registradas, la más reciente en "
                     f"{hist[-1]['bodega'].title()} con {hist[-1]['exactitud']}%", "
                     f"tendencia {tendencia}")
    etiquetas_estado = {"cerrada": "cerradas", "en_conteo": "en conteo",
                        "en_auditoria": "en auditoría", "pendiente": "pendientes"}
    texto_estado = ("", ".join(f"{n} {etiquetas_estado.get(e, e)}"
                             for e, n in a["estado_bodegas"].items())
                   or "sin bodegas registradas")
    alertas_tipo = a["alertas_por_tipo"]
    texto_alertas_tipo = ("sin alertas registradas todavía" if not alertas_tipo else
                          ", ".join(f"{x['tipo']}: {x['cantidad']}" for x in alertas_tipo))
    descuadres = a["descuadres_recurrentes"]
    texto_descuadres = ("sin descuadres repetidos todavía" if not descuadres else
                        "; ".join(f"{x['articulo'].title()} ({x['tipo']}, diferencia "
                                   f"{x['diferencia']}" en {x['tomas']} tomas, acción sugerida: "
                                   f"{x['accion']}" for x in descuadres[:5]))
    return {"r": r, "a": a, "texto_dif": texto_dif, "texto_stock": texto_stock,
            "texto_hist": texto_hist, "texto_estado": texto_estado,
            "texto_alertas_tipo": texto_alertas_tipo, "texto_descuadres": texto_descuadres}

# Preguntas frecuentes del Panel gerencial, resueltas sin pasar por Gemini.
# Sin esto, una pregunta tan común como "¿cuántas bodegas están cerradas?"
# a veces terminaba navegando a la pantalla Bodegas en vez de contestar
# (confirmado en pruebas: al mencionar el nombre de otra pantalla, Gemini
# no siempre distingue preguntar de navegar) - las mismas frases que se
# ofrecen en el desplegable "Ver frases exactas" de Panel quedan así
# garantizadas.
_PANEL_PREGUNTAS = [
    (re.compile(r"\bprimera\s+pasada\b", re.IGNORECASE),
     lambda d: f"La exactitud primera pasada es del {d['r']['exactitud_primera_pasada']}%", "
     "promedio de cierres."),
    (re.compile(r"\breferencias?\b.*\bcontad\w*\b|bcu[aá]nt\w*\s+referencias\b", re.IGNORECASE),
     lambda d: f"Se han contado {d['r']['referencias_contadas']} referencias en toda la "
     "operación."),
    (re.compile(r"\balertas?\b.*\bgestionad\w*", re.IGNORECASE),
     lambda d: f"Van {d['r']['alertas_gestionadas']} alertas gestionadas de "
     f"{d['r']['alertas_total']} en total."),
    (re.compile(r"\bc[oó]mo\s+(est[aá]n|van)\s+las\s+bodegas\b|bestado\s+de\s+las\s+bodegas\b",
     re.IGNORECASE),
     lambda d: f"Estado de las bodegas: {d['texto_estado']}."),
    (re.compile(r"\bcu[aá]nt\w*\s+bodegas\b.*\bcerrad\w*\bbodegas\s+cerradas\b", re.IGNORECASE),
     lambda d: f"Van {d['r']['bodegas_cerradas']} bodegas cerradas de {d['r']['bodegas_total']} "
     "en total, con doble firma digital."),
    (re.compile(r"\bqu[eé]\s+bodega\b.*\bdiferencia\b|bdiferencia\s+absoluta\s+por\s+bodega\b",

```



```

        re.IGNORECASE),
        lambda d: f"Diferencia absoluta por bodega: {d['texto_dif']}."),
    (re.compile(r"\bstock\s+por\s+unidad\b|\bc[oó]mo\s+va\s+el\s+stock\b", re.IGNORECASE),
     lambda d: f"Stock por unidad de medida: {d['texto_stock']}."),
    (re.compile(r"\btendencia\s+de\s+exactitud\b|\bexactitud\s+por\s+toma\b", re.IGNORECASE),
     lambda d: f"Exactitud por toma de inventario: {d['texto_hist']}."),
    (re.compile(r"\bnegativos?\b.*\bcorregid\w*|\bcu[aá]nt\w*\s+negativos\b", re.IGNORECASE),
     lambda d: (f"Saldos negativos en el sistema: {d['a']['negativos_actuales']} artículos. "
                "La corrección se hace en My Inventory, no en CuentaVoz."
                if d['a']['negativos_iniciales'] == d['a']['negativos_actuales'] else
                f"Saldos negativos: {d['a']['negativos_iniciales']} al inicio de este período, "
                f"{d['a']['negativos_actuales']} ahora.")),
    (re.compile(r"\btipo\s+promedio\s+de\s+conteo\b", re.IGNORECASE),
     lambda d: f"El tiempo promedio de conteo por bodega es de "
                f"{d['a']['tiempo_promedio_min']} minutos."),
    (re.compile(r"\balias\s+aprendid\w*|\bcu[aá]nt\w*\s+alias\b", re.IGNORECASE),
     lambda d: f"El agente tiene {d['a']['alias_aprendidos']} alias aprendidos."),
    (re.compile(r"\balertas\s+por\s+tipo\b", re.IGNORECASE),
     lambda d: f"Alertas por tipo: {d['texto_alertas_tipo']}."),
    (re.compile(r"\bdescuadre\w*\s+(principal|recurrent\w*)\b|\bcausa\s+ra[ií]z\b",
                re.IGNORECASE),
     lambda d: f"Descuadres recurrentes: {d['texto_descuadres']}.")
]

```

```

def _responder_panel_por_voz(texto: str, u: Usuario) -> dict | None:
    if u.perfil != "auditor":
        return None
    for patron, respuesta in _PANEL_PREGUNTAS:
        if patron.search(texto):
            return {"respuesta_hablada": respuesta(_datos_panel_narrados()),
                    "accion": None, "destino": None, "pestaña": None}
    return None

```

```

def _formato_acceso_hablado(fecha_dt) -> str | None:
    """Mismo criterio que formatoAcceso() en MiPerfil.jsx (hoy HH:MM vs
    DD/MM HH:MM) - para que lo que diga el agente coincida con lo que la
    pantalla ya muestra escrito."""
    if not fecha_dt:
        return None
    if fecha_dt.date() == ahora().date():
        return f"hoY a las {fecha_dt.strftime('%H:%M')}"
    return f"el {fecha_dt.strftime('%d/%m')} a las {fecha_dt.strftime('%H:%M')}"

```

```

def _datos_perfil_narrados(u: Usuario) -> dict:
    from agente.cerebro import VOCES, VOZ_DEFECTO
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        asigs = s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()
        nombres_bodegas = [b.nombre_oficial.title() for b in
                            (s.get(Bodega, a.bodega_id) for a in asigs) if b]
        dias_pin = (ahora() - (usr.pin_actualizado or ahora())).days
        voz = usr.idioma_voz if usr.idioma_voz in VOCES else VOZ_DEFECTO
        return {
            "ultimo_acceso_hablado": _formato_acceso_hablado(usr.ultimo_acceso),
            "pin_vence_en_dias": max(90 - dias_pin, 0),
            "n_bodegas": len(nombres_bodegas),
            "texto_bodegas": "; ".join(nombres_bodegas),
            "huella_registrada": bool(usr.huella_registrada),
            "perfil": usr.perfil,
            "velocidad_voz": usr.velocidad_voz,
            "confirmacion_hablada": bool(usr.confirmacion_hablada),
            "voz_nombre": VOCES[voz]["nombre"],

```

```

        "voz_etiqueta": VOCES[voz]["etiqueta"],
    }

_PERFIL_PREGUNTAS = [
    (re.compile(r"[uú]ltimo\s+acceso|cu[aá]ndo\s+(entr[eé]|ingres[eé])", re.IGNORECASE),
     lambda d: f"Su último acceso fue {d['ultimo_acceso_hablado']}. " if d["ultimo_acceso_hablado"]
     else "Todavía no hay un acceso anterior registrado."),
    (re.compile(r"\bpin\b.*\bvence|vence\b.*\bpin\b|cu[aá]nto\b.*\s+d[ií]as.*\bpin\b", re.IGNORECASE),
     lambda d: f"Su PIN vence en {d['pin_vence_en_días']} días."),
    (re.compile(r"bodegas?\s+(tengo|asignad\w*)|cu[aá]nto\b.*\s+bodegas", re.IGNORECASE),
     lambda d: (f" Tiene {d['n_bodegas']} bodegas asignadas: {d['texto_bodegas']}."
     if d["n_bodegas"] else "Todavía no tiene bodegas asignadas.")),
    (re.compile(r"huella", re.IGNORECASE),
     lambda d: ("Sí, tiene la huella registrada en este dispositivo." if d["huella_registrada"]
     else "No tiene la huella registrada todavía; eso solo se hace desde la "
     "pantalla, por seguridad.")),
    (re.compile(r"mi\s+perfil\b|qu[eé]\s+perfil|soy\s+(auxiliar|admin\w*)", re.IGNORECASE),
     lambda d: f"Su perfil es {'administrador de bodega' if d['perfil'] == 'auditor' else 'auxiliar de inventarios'}."),
    (re.compile(r"qu[eé]\s+voz\s+(tengo|est[aá]|uso)|voz\s+actual", re.IGNORECASE),
     lambda d: f"Está usando la voz {d['voz_nombre']}: {d['voz_etiqueta'].lower()}."),
]

def _responder_perfil_por_voz(texto: str, u: Usuario) -> dict | None:
    for patron, respuesta in _PERFIL_PREGUNTAS:
        if patron.search(texto):
            return {"respuesta_hablada": respuesta(_datos_perfil_narrados(u)),
                    "accion": None, "destino": None, "pestana": None}
    return None

_VOCES_HABLADAS = ("kore", "aoede", "puck", "charon")
_VERBO_CAMBIAR_VOZ = re.compile(
    r"\b(cambia\w*|usa\w*|pon\w*|quiero|eli[jg]\w*|selecciona\w*)\b", re.IGNORECASE)

def _cambiar_voz_por_voz(texto: str, u: Usuario) -> dict | None:
    """Cambia mi voz a puck / "usa la voz aoede" - exige el nombre EXACTO
    de una de las cuatro voces (no un genero o descripción), para no
    competir con "¿qué voz tengo?" ni con "prueba la voz" (probarVoz solo
    demuestra la que ya está elegida, no cambia nada)."""
    if not _VERBO_CAMBIAR_VOZ.search(texto):
        return None
    clave = next((k for k in _VOCES_HABLADAS if re.search(rf"\b{k}\b", texto, re.IGNORECASE)), None)
    if not clave:
        return None
    with Sesión() as s:
        usr = s.get(Usuario, u.id)
        usr.idioma_voz = clave
        s.commit()
    from agente.cerebro import VOCES
    registrar(u, "PERFIL", f"{u.nombre} cambió su voz a {VOCES[clave]['nombre']} por voz", "ok")
    return {"respuesta_hablada": f"Listo, ahora hablo con la voz {VOCES[clave]['nombre']}: "
                                f"{VOCES[clave]['etiqueta'].lower()}.",
            "accion": "actualizar", "destino": None, "pestana": None, "idioma_voz": clave}

_VELOCIDAD_LENTA = re.compile(r"\blenta\b|m[aá]\s+\s+despacio|m[aá]\s+\s+lento", re.IGNORECASE)
_VELOCIDAD_RAPIDA = re.compile(r"\br[aá]pida\b|m[aá]\s+\s+r[aá]pido", re.IGNORECASE)
_VELOCIDAD_NORMAL = re.compile(r"velocidad\s+normal|voz\s+normal", re.IGNORECASE)

```

```

def _cambiar_velocidad_por_voz(texto: str, u: Usuario) -> dict | None:
    if _VELOCIDAD_LENTA.search(texto):
        nueva = "lenta"
    elif _VELOCIDAD_RAPIDA.search(texto):
        nueva = "rapida"
    elif _VELOCIDAD_NORMAL.search(texto):
        nueva = "normal"
    else:
        return None
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        usr.velocidad_voz = nueva
        s.commit()
    registrar(u, "PERFIL", f"{u.nombre} cambió la velocidad de voz a {nueva} por voz", "ok")
    return {"respuesta_hablada": f"Listo, la velocidad queda en {nueva}.",
            "accion": "actualizar", "destino": None, "pestana": None, "velocidad_voz": nueva}

_CONFIRMACION_HABLADA_FRASE = re.compile(r"confirmaci[oó]n\s+hablada", re.IGNORECASE)

def _cambiar_confirmacion_hablada_por_voz(texto: str, u: Usuario) -> dict | None:
    if not _CONFIRMACION_HABLADA_FRASE.search(texto):
        return None
    if _ACTIVAR.search(texto):
        nuevo = True
    elif _DESACTIVAR.search(texto):
        nuevo = False
    else:
        return None
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        usr.confirmacion_hablada = 1 if nuevo else 0
        s.commit()
    estado = "activada" if nuevo else "desactivada"
    registrar(u, "PERFIL", f"{u.nombre} dejó la confirmación hablada {estado} por voz", "ok")
    return {"respuesta_hablada": f"Listo, la confirmación hablada quedó {estado}.",
            "accion": "actualizar", "destino": None, "pestana": None,
            "confirmacion_hablada": nuevo}

def _contexto_asistente(vista: str, u: Usuario) -> str:
    """Un resumen honesto de lo que hay AHORA en esa pantalla, para que el
    agente conteste con datos reales y no invente nada. Reusa las mismas
    funciones que ya alimentan cada pantalla, no vuelve a calcular nada."""
    if vista == "inicio":
        with Sesion() as s:
            n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=u.id).count()
            alertas = s.query(Alerta).filter_by(resuelta=0).count()
            historial = s.query(HistorialCierre).all()
            exact = (round(sum(h.exactitud for h in historial) / len(historial), 1)
                     if historial else 100.0)
        return (f"Pantalla: Inicio, de {u.nombre} ({u.perfil}). "
                f"Bodegas asignadas a esta persona: {n_bodegas}. "
                f"Alertas por revisar en toda la operación: {alertas}. "
                f"Exactitud del mes: {exact}%.")
    if vista == "ajustes":
        a = ver_ajustes(u)
        # El permiso real de cambiar el modo sin conexión por voz ya se
        # resuelve antes de llegar aquí (_cambiar_modos_sin_conexion, que
        # exige perfil auditor) - este texto solo llega a Gemini cuando
        # esa orden exacta no aplicó (una pregunta suelta, o quien
        # pregunta no es administrador). Sin condicionarlo al perfil,

```

```

# Gemini le decía a CUALQUIERA "ya lo activé" aunque el cambio de
# verdad solo lo hace un administrador - la misma deshonestidad
# que la Regla 6 ya evita en otras respuestas.
permiso_offline = (
    "El modo sin conexión sí se puede cambiar por voz, pero solo por un "
    "administrador: decir «activa/desactiva el modo sin conexión» lo aplica de "
    "una, sin tocar el interruptor. En Gestión de usuarios, un administrador "
    "también puede decir «activa/desactiva a <nombre>» para cambiar el estado de "
    "otra persona, «crea un usuario llamado <nombre> perfil <auxiliar o "
    "auditor>» para crear una cuenta nueva (con PIN temporal), «cambia el "
    "perfil de <nombre> a auxiliar/administrador» para lo mismo que el botón "
    "«Editar» (el correo no se cambia por voz), «asígnale/quítale la bodega "
    "<bodega> a <nombre>» para lo mismo que el botón «Asignar bodegas» pero "
    "sumando o quitando solo esa bodega, no reemplazando toda la lista, y "
    "«¿quién tiene la bodega <bodega>?» o «¿cuántas bodegas sin asignar hay?» "
    "para contestar una pregunta puntual sin abrir nada, y «muéstrame/oculta "
    "la asignación por bodega» para abrir o cerrar la tabla completa - lo "
    "mismo que el botón «Ver/Ocultar asignación por bodega». En Recetas, un "
    "administrador también puede decir «crea una receta llamada "
    "<nombre>, rendimiento <N> porciones, con <cantidad> de <ingrediente>» "
    "(crea la receta con ese primer ingrediente ya resuelto contra el "
    "catálogo), «agrega <cantidad> de <ingrediente> a la receta <nombre>» "
    "(para sumarle más ingredientes después), «quita el ingrediente "
    "<ingrediente> de la receta <nombre>», «cambia el rendimiento de la "
    "receta <nombre> a <N> porciones», «agrega la preparación a la receta "
    "<nombre>: <pasos>» (reemplaza los pasos de cocina completos, tal cual "
    "se dicten, con los dos puntos como separador del nombre), y «elimina la "
    "receta <nombre>». "
    "En Registro de trazabilidad, «exporta el registro de trazabilidad» "
    "descarga el archivo con los filtros de fecha/persona/acción que ya "
    "estén puestos en la pantalla en ese momento (no los que diga la "
    "orden), «acciones por persona» resume cuántas hizo cada quien "
    "(hoy por defecto; «esta semana»/«este mes»/«en total» cambian el "
    "período), y «¿cuántas acciones tiene <nombre>?» contesta solo por esa "
    "persona. "
    "Todo esto SOLO funciona si la orden usa exactamente esas formas («crea un "
    "usuario llamado...», «cambia el perfil de... a...», «asígnale/quítale la "
    "bodega... a...», «crea una receta llamada...», rendimiento... porciones, "
    "con... de...», «agrega... a la receta...», «quita el ingrediente... de "
    "la receta...», «cambia el rendimiento de la receta... a...», «agrega la "
    "preparación a la receta...: ...», «elimina la receta...», «exporta el "
    "registro de trazabilidad», «acciones por persona», «cuántas acciones "
    "tiene...») - si "
    "el mensaje llega hasta usted es porque esa frase no se dijo así, el "
    "nombre de la persona/bodega no coincidió con ninguna existente, o el "
    "ingrediente no se encontró en el catálogo, así que no invente que ya "
    "creó, cambió, asignó, agregó o eliminó nada: pida que lo repitan con ese "
    "formato exacto y el nombre completo tal como aparece en la pantalla."
    if u.perfil == "auditor" else
    "El modo sin conexión, el estado de otras personas, crear/editar usuarios, "
    "asignar bodegas y crear/editar/eliminar recetas solo los puede hacer un "
    "administrador - esta persona no lo es, así que si lo pide, dígalos con "
    "honestidad en vez de decir que ya quedó hecho.")
return (f"Pantalla: Ajustes. Sección Validación de datos: umbral de anomalía "
    f"{a['umbral']}%; bloquear cantidades negativas activado (regla fija, no "
    f"se puede desactivar); exigir confirmación en alertas activado (regla "
    f"fija); permitir crear productos pendientes activado (regla fija). "
    f"Sección Conexión y sincronización: modo sin conexión "
    f"{a['activado'] if a['offline'] else 'desactivado'}; refresco del tablero "
    f"en tiempo real. "
    f"Sección Administración: usuarios activos {a['usuarios_activos']}; "
    f"aprobaciones pendientes {a['aprobaciones_pendientes']}. Sección Acerca "
    f"de CuentaVoz: versión {a['version']}, modelo del agente {a['modelo']}. "
    "El umbral solo lo puede CAMBIAR un administrador, y únicamente con los "
    "controles de la pantalla (número y botón «Guardar configuración») - eso "
    f"todavía no cambia por voz. {permiso_offline}")
if vista == "panel" and u.perfil == "auditor":

```

```

d = _datos_panel_narrados()
r, a = d["r"], d["a"]
return (
    "Pantalla: Panel gerencial, con dos pestañas: «Resumen ejecutivo» y «Bodegas y "
    "alertas» - decir el nombre de cualquiera de las dos (o «llévame a...») navega "
    "directo a ella. "
    f"Pestaña Resumen ejecutivo - tarjetas: exactitud primera pasada "
    f"{r['exactitud_primera_pasada']}%, referencias contadas "
    f"{r['referencias_contadas']}, alertas gestionadas {r['alertas_gestionadas']} de "
    f"{r['alertas_total']}, bodegas cerradas {r['bodegas_cerradas']} de "
    f"{r['bodegas_total']}. Gráfica «Diferencia absoluta por bodega»: {d['texto_dif']}. "
    f"Gráfica «Stock por unidad de medida»: {d['texto_stock']}. Gráfica «Exactitud por "
    f"toma de inventario»: {d['texto_hist']}. "
    f"Pestaña Bodegas y alertas - tarjetas: saldos negativos en el sistema "
    f"{a['negativos_actuales']} artículos (la corrección se hace en My Inventory, no "
    "en CuentaVoz - dentro de una sola toma este número normalmente no se mueve), "
    f"tiempo promedio de conteo por bodega {a['tiempo_promedio_min']} minutos, "
    f"alias aprendidos por el agente {a['alias_aprendidos']}. Tarjeta «Estado de las "
    f"bodegas»: {d['texto_estado']}. Tarjeta «Alertas por tipo»: {d['texto_alertas_tipo']}. "
    f"Tabla «Descuadres recurrentes»: {d['texto_descuadres']}."
)
if vista == "reportes" and u.perfil == "auditor":
    d = _datos_reportes_narrados(u)
    a = d["analisis"]
    return ("Pantalla: Reportes, con dos pestañas: «Consolidado de la toma» y «Análisis de "
            "consumo». Aquí se genera el consolidado para My "
            "Inventory, las diferencias por bodega, y el análisis de consumo "
            "de los últimos 30 días. Los tres SÍ se pueden generar por voz "
            "«genera el consolidado», «exporta las diferencias», «exporta el "
            "análisis de consumo» - queda igual que si le hubiera dado clic "
            f"al botón. Archivos generados recientemente: {d['texto_recientes']}. "
            f"Pestaña Análisis de consumo, últimos {a['días']} días: pedido total "
            f"{a['pedido_total']} kg en {a['servicios_periodo']} servicios, usado "
            f"realmente {a['usado_total']} kg ({a['aprovechamiento']}% de "
            f"aprovechamiento), ahorro potencial {a['ahorro_potencial']} kg al mes. "
            f"Insumos subutilizados: {d['texto_subutil']}. También se puede pedir "
            "«muéstreme el archivo de <consolidado o diferencias> para ver su "
            "contenido, igual que darle clic a la tarjeta.")
if vista == "auditoria" and u.perfil == "auditor":
    d = _datos_auditoria_narrados(u)
    ra = d["resumen_alertas"]
    nombres_aprob = ("; ".join(a["nombre"].title() for a in d["aprobaciones"]))
    if d["aprobaciones"] else "ninguna"
    nombres_pedidos = ("; ".join(f"{p['persona'].title()}: {p['plato']}" for p in d["pedidos"]))
    if d["pedidos"] else "ninguno"
    return (f"Pantalla: Auditoría, con cuatro pestañas: «Recuento ciego y cierre», "
            f"«Aprobaciones», «Pedidos pendientes» y «Bandeja de alertas» - decir el "
            f"nombre de cualquiera (o «llévame a...») navega directo a ella. "
            f"Pestaña Aprobaciones: {len(d['aprobaciones'])} pendientes "
            f"{nombres_aprob} - «aprueba/rechaza <nombre>» resuelve una. "
            f"Pestaña Pedidos pendientes: {len(d['pedidos'])} pendientes "
            f"{nombres_pedidos} - «aprueba/rechaza el pedido de <persona o plato>» "
            f"resuelve uno. Pestaña Bandeja de alertas: {ra['abiertas']} abiertas, "
            f"{ra['resueltas_hoy']} resueltas hoy, tiempo medio "
            f"{ra['tiempo_medio_min']} if ra['tiempo_medio_min'] is not None else '-' "
            f"minutos - «resuelve la alerta de <artículo o bodega>» resuelve una. "
            f"Pestaña Recuento ciego y cierre: «audita <bodega>» o «abre <bodega>» "
            "selecciona esa bodega lista para recuento o cierre, igual que darle clic "
            "a «Abrir» en su tarjeta - desde ahí, dictar los productos SÍ es por voz "
            "con el micrófono normal de esa pantalla (igual que en Conteo). Pero "
            "«Iniciar recuento ciego», «Ver comparación», «Aceptar todas», «Cerrar con "
            "doble firma», firmar con PIN o huella, y «Cerrar bodega definitivamente» "
            "son A PROPÓSITO solo manuales - nadie cierra una bodega con doble firma "
            "por accidente con la voz. Si el mensaje llegó hasta usted es porque "
            "«aprueba/rechaza <nombre>», «aprueba/rechaza el pedido de <...>», "
            "«resuelve la alerta de <...>» o «audita <bodega>» NO encontraron ese "
            "nombre exacto entre lo recién listado - esas acciones YA se resuelven "
            "antes de llegar aquí cuando el nombre coincide, así que usted NUNCA debe "

```

```

        "decir que ya aprobó, rechazó, resolvió o abrió algo (sería falso: no "
        "tiene esa capacidad desde aquí). Dígallo con honestidad y sugiera repetirlo "
        "con el nombre completo tal como aparece en la pantalla.")
if vista == "ayuda":
    faq = "; ".join(f"{p} -> {r}" for p, r in _FAQ_ASISTENTE)
    return f"Pantalla: Ayuda. Preguntas frecuentes conocidas: {faq}."
if vista == "perfil":
    d = _datos_perfil_narrados(u)
    return (
        f"Pantalla: Mi perfil, de {u.nombre} ({u.perfil}). Datos personales, seguridad "
        "de la cuenta y preferencias de voz. "
        f"Último acceso: {d['ultimo_acceso_hablado'] or 'sin registro'}. PIN vence en "
        f"{d['pin_vence_en_dias']} días. Bodegas asignadas: {d['n_bodegas']}"
        + (f" ({d['texto_bodegas']})" if d["texto_bodegas"] else "") + ". "
        f"Huella registrada en este dispositivo: {'sí' if d['huella_registrada'] else 'no'}. "
        f"Voz actual: {d['voz_nombre']} ({d['voz_etiqueta'].lower()}), velocidad "
        f"{d['velocidad_voz']}, confirmación hablada "
        f"{'activada' if d['confirmacion_hablada'] else 'desactivada'}. "
        "Sí se puede cambiar por voz: la voz neuronal («cambia mi voz a puck/kore/aoede/"
        "charon»), la velocidad («habla más lento/rápido», «velocidad normal») y la "
        "confirmación hablada («activa/desactiva la confirmación hablada»). Cambiar el "
        "PIN, registrar o quitar la huella, subir una foto, y editar nombre/correo/"
        "teléfono son A PROPÓSITO solo manuales, por seguridad - nunca diga que ya "
        "cambió el PIN o la huella por voz, eso sería falso.")
if vista == "bodegas":
    with Sesion() as s:
        ids_permitidos = _ids_permitidos_para_buscar(s, u)
        q = s.query(Bodega)
        if ids_permitidos is not None:
            q = q.filter(Bodega.id.in_(ids_permitidos))
        bodegas = q.all()
        n_pendientes = sum(1 for b in bodegas if b.estado == "pendiente")
        n_en_conteo = sum(1 for b in bodegas if b.estado == "en_conteo")
    return (f"Pantalla: Bodegas. Aquí se busca el stock de un artículo en todo "
            f"el catálogo, y se ve el estado/historial de cada bodega (no se "
            f"cuente desde aquí, eso es en Conteo). De las bodegas de esta "
            f"persona: {n_pendientes} pendientes, {n_en_conteo} en conteo. Si "
            f"piden abrir o seguir contando una bodega por nombre, se resuelve "
            f"aparte antes de este mensaje - esto solo se usa para explicar o "
            f"navegar a otra pantalla.")
return f"Pantalla: {vista}."

_VERBOS_ABRIR_BODEGA = re.compile(
    r"\b(abr[ae]|abrir|contar|cuenta|siga|sigue|continu[ae]|continuar)\b",
    re.IGNORECASE)

_MODO_SIN_CONEXION = re.compile(r"sin conexi[oó]n", re.IGNORECASE)
_ACTIVAR = re.compile(
    r"\b(act[ii]v\w*|enc[ie][eé]nd\w*|prend\w*)\b", re.IGNORECASE)
_DESACTIVAR = re.compile(
    r"\b(desact[ii]v\w*|ap[aá]g\w*|qu[ii]t\w*)\b", re.IGNORECASE)

def _cambiar_modos_sin_conexion(texto: str, u: Usuario) -> dict | None:
    """Único ajuste que de verdad se puede cambiar por voz en esta
    pantalla (el resto - umbral, reglas fijas - sigue exigiendo los
    controles de la pantalla): solo un administrador, y solo si la
    orden nombra "sin conexión" junto con un verbo de activar/desactivar
    inequívoco. None si no aplica, para caer al agente general."""
    if u.perfil != "auditor" or not _MODO_SIN_CONEXION.search(texto):
        return None
    if _ACTIVAR.search(texto):
        nuevo = True
    elif _DESACTIVAR.search(texto):

```

```

        nuevo = False
    else:
        return None
    with Sesion() as s:
        existente = s.get(ConfigClave, "offline")
        valor = "1" if nuevo else "0"
        if existente:
            existente.valor = valor
        else:
            s.add(ConfigClave(clave="offline", valor=valor))
        s.commit()
    estado = "activado" if nuevo else "desactivado"
    registrar(u, "AJUSTE", f"{u.nombre} {estado} el modo sin conexión por voz", "ok")
    return {"respuesta_hablada": f"Listo, el modo sin conexión quedó {estado}.",
            "accion": "actualizar", "destino": None, "pestanas": None}

def _normalizar_nombre(t: str) -> str:
    """Sin quitar la puntuación, una pausa natural al hablar ("Cebolla,
    cabeza blanca.") deja una coma que rompe la comparación por
    substring contra "CEBOLLA CABEZONA BLANCA" (sin coma) - confirmado
    con una captura real donde por eso no lograba resolver una
    ambigüedad pendiente aunque dijo el nombre correcto. Mismo arreglo
    que normalizar() ya tiene en servicios/conciliacion.py, por la
    misma razón (el reconocimiento de voz también suele cerrar la
    frase con un punto que nadie dijo)."""
    t = str(t).lower().strip()
    t = re.sub(r"[.,;:;!¿?¡]+", " ", t)
    t = re.sub(r"\s+", " ", t).strip()
    return "".join(c for c in unicodedata.normalize("NFD", t)
                   if unicodedata.category(c) != "Mn")

_COLA_RELLENO = re.compile(
    r"\s*,?\s*(?:por\s+favor|porfa\w*|gracias|si\s+puedes|si\s+es\s+posible)"
    r"\s*[.,!¿?¡]*\s*$", re.IGNORECASE)

def _quitar_relleno(texto: str) -> str:
    """Los comandos de una sola frase («crea un usuario llamado...»,
    «crea una receta llamada..., con... de...») quedan anclados al
    FINAL del texto para extraer nombre/cantidad/ingrediente con
    precisión - pero cualquier cortesía dicha al final ("...por favor",
    "...gracias", cosa que la gente realmente dice) caía fuera de ese
    ancla y el campo capturado se tragaba esas palabras de más (un
    usuario terminaba llamándose "juan perfil auxiliar por favor" en
    vez de "juan"). Se quita ANTES de intentar cualquier comando, en un
    bucle por si hay más de una coletilla seguida ("por favor, gracias")."""
    anterior = None
    t = texto
    while anterior != t:
        anterior = t
        t = _COLA_RELLENO.sub("", t)
    return t

def _cambiar_estado_usuario(texto: str, u: Usuario) -> dict | None:
    """Activa/desactiva a <nombre> desde Gestión de usuarios - mismo
    patrón determinístico que el modo sin conexión (sin pasar por
    Gemini, así no depende de su cuota ni arriesga un cambio real por
    una mala interpretación del modelo). Reusa las mismas reglas que ya
    protegen el PUT /api/usuarios/{id}: nadie se desactiva a sí mismo
    por voz, y solo un administrador puede tocar a otra persona.
    "Quita" es sinónimo de desactivar aquí, pero "quítale la BODEGA X a

```

```

Luis" no debe desactivar a Luis por accidente solo porque comparte
el verbo - de ahí la exclusión explícita cuando se menciona una
bodega, que es harina de otro costal (_asignar_bodega_por_voz)."""
if (u.perfil != "auditor" or _MODOSINCONEXION.search(texto)
    or re.search(r"\bbodega\b", texto, re.IGNORECASE)):
    return None
if _ACTIVAR.search(texto):
    nuevo = True
elif _DESACTIVAR.search(texto):
    nuevo = False
else:
    return None
t = _normalizar_nombre(texto)
with Sesion() as s:
    candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
    encontrado = next(
        (c for c in candidatos
         if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t)),
        None)
    if not encontrado:
        return None
    if bool(encontrado.activo) == nuevo:
        estado = "activo" if nuevo else "inactivo"
        return {"respuesta_hablada": f"{encontrado.nombre.capitalize()} ya estaba {estado}.",
                "accion": "actualizar", "destino": None, "pestana": None}
    encontrado.activo = int(nuevo)
    encontrado.version_token = (encontrado.version_token or 0) + 1
    nombre = encontrado.nombre
    s.commit()
    estado = "activado" if nuevo else "desactivado"
    registrar(u, "USUARIO", f"{nombre} {estado} por voz", "ok")
    return {"respuesta_hablada": f"Listo, {nombre.capitalize()} quedó {estado}.",
            "accion": "actualizar", "destino": None, "pestana": None}

# Cuando alguien encadena dos órdenes en una sola frase ("...con 2 kg
# de papa, agrega 300 g de cebolla a la receta sopa" - confirmado con
# una captura real), el último grupo capturado (.+? anclado al final
# con $) se tragaba TODO lo que seguía, incluida la segunda orden
# completa, como si fuera parte del dato ("no encontré «papa, agrega
# 300 g de cebolla a la receta sopa» en el catálogo"). Este sufijo se
# detiene apenas aparece una coma seguida de otro verbo reconocido, y
# descarta el resto - la primera orden se cumple bien, y la segunda
# queda pendiente de decirse aparte.
_CORTE_ORDEN_ENCadenada = (
    r"(?:, \s*(?:cre[ae]|agr[eé]ga|a[ñn]ad|elimin|borr|cambia|as[ií]gn|qu[ií]t|t|
    r"muestra|ense[ñn]a|oculta|act[ií]v|desact[ií]v)\w*.*)?")
_CREAR_USUARIO = re.compile(
    r"\b(?:cre[ae]\w*\s+)?(?:un\s+|una\s+)?(?:nuev[oa]\s+)?usuario\s+llamad[oa]\s+(?P<nombre>.+)?"
    r"(?:\s+(?:de\s+|con\s+)?perfil\s+(?P<perfil>auxiliar|auditor|administrador))?"
    r"\s*" + _CORTE_ORDEN_ENCadenada + r"[!]? \s*$", re.IGNORECASE)
_INTENCION_CREAR_USUARIO = re.compile(
    r"\b(cre[ae]\w*|necesito|quiero)\b.*\busuarios?\b|\busuario\s+llamad[oa]\b",
    re.IGNORECASE)

def _crear_usuario_por_voz(texto: str, u: Usuario) -> dict | None:
    """Crea un usuario llamado <nombre> perfil <auxiliar/auditor> -
    determinístico, sin pasar por Gemini. A diferencia de
    activar/desactivar (que busca un nombre YA existente), aquí no hay
    nada contra qué validar el nombre, así que se exige un formato de
    frase concreto en vez de adivinar de cualquier forma - más fácil de
    aprender a decir que arriesgar un nombre mal extraído. Sin perfil
    dicho, queda auxiliar (el caso más común y el de menor alcance).

```



```

Decir solo "crea un usuario" (sin nombre) coincidía por casualidad
con la navegación a la pestaña Gestión de usuarios (por la palabra
"usuario") - si ya estaba en esa pestaña, no pasaba nada visible y
parecía que el agente no hacía nada. Ahora, si se nota la intención
de crear pero falta el nombre, se explica el formato exacto en vez
de caer en la navegación por casualidad."""
if u.perfil != "auditor":
    return None
m = _CREAR_USUARIO.search(texto.strip())
if not m:
    if (_INTENCION_CREAR_USUARIO.search(texto)
        and not _ES_PREGUNTA_PESTANA.search(texto)):
        return {"respuesta_hablada": "Para crear un usuario diga el nombre completo así: "
            "«crea un usuario llamado» y el nombre, «perfil» y "
            "auxiliar o administrador.",
            "accion": None, "destino": None, "pestaña": None}
    return None
nombre = re.sub(r"[.,;:;!¿?;]+$", "", m.group("nombre")).strip().lower()
if not nombre:
    return None
perfil_dicho = (m.group("perfil") or "").lower()
perfil = "auditor" if perfil_dicho in ("auditor", "administrador") else "auxiliar"
with Sesión() as s:
    if s.query(Usuario).filter_by(nombre=nombre).first():
        return {"respuesta_hablada": f"Ya existe un usuario llamado {nombre.capitalize()}.",
            "accion": None, "destino": None, "pestaña": None}
    pin_generado = secrets.token_urlsafe(6)
    nuevo = Usuario(nombre=nombre, perfil=perfil,
        clave_hash=hash_clave(pin_generado), correo="")
    s.add(nuevo)
    s.commit()
    s.refresh(nuevo)
    nuevo.codigo = f"CS-{48000 + nuevo.id}"
    s.commit()
    registrar(u, "USUARIO", f"Usuario {nombre} creado ({perfil}) por voz", "ok")
    return {"respuesta_hablada": f"Listo, creé a {nombre.capitalize()} como {perfil}. "
        f"Su PIN temporal es {pin_generado} - dígaselo para que "
        "lo cambie en Mi perfil.",
        "accion": "actualizar", "destino": None, "pestaña": None}

_VERBOS_GENERAR_REPORTE = re.compile(
    r"\b(gener[ae]|generar|exporta|exportar|crea|crear|descarga|descargar|saca|sacar) \b",
    re.IGNORECASE)
_REP_DIFERENCIAS = re.compile(r"diferencia", re.IGNORECASE)
_REP_ANALISIS = re.compile(r"an[al]lisis|consumo", re.IGNORECASE)
_REP_CONSOLIDADO = re.compile(r"consolidado", re.IGNORECASE)
_REP_TABLERO = re.compile(r"tablero", re.IGNORECASE)
_REP_DETALLE_BODEGA = re.compile(r"detalle\s+d?el?\s+(la\s+)?bodega|detalle\s+de\s+almac[eé]n",
    re.IGNORECASE)
_REP_TRAZA_ARCHIVO = re.compile(r"trazabilidad", re.IGNORECASE)

def _generar_reporte_por_voz(texto: str, u: Usuario) -> dict | None:
    """Genera el consolidado" / "exporta las diferencias" / "exporta el
    análisis de consumo" desde Reportes - mismo patrón determinístico
    que Ajustes (sin pasar por Gemini): llama la MISMA función que ya
    usa el botón correspondiente, así el resultado es idéntico al de
    darle clic, solo que solo un administrador puede pedirlo por voz."""
    if u.perfil != "auditor" or not _VERBOS_GENERAR_REPORTE.search(texto):
        return None
    if _REP_DIFERENCIAS.search(texto):
        d = reporte_diferencias(formato="xlsx", u=u)
        # archivo/titulo_archivo (ademas de "actualizar") - para que el
        # panel de vista previa muestre este archivo de una, igual que ya
        # pasa al darle clic al boton "Generar diferencias" (que llama a

```

```

# previsualizar() apenas termina). Sin esto, pedirlo por voz lo
# generaba pero la vista previa se quedaba vacia hasta un clic
# manual en la tarjeta.
return {"respuesta_hablada": f"Listo, generé las diferencias: {d['filas']} filas en "
      f"{d['bodegas_con_descuadre']} bodegas con descuadre.",
      "accion": "actualizar", "destino": None, "pestanas": "consolidado",
      "archivo": d["archivo"], "titulo_archivo": "Diferencias por bodega"}

if _REP_ANALISIS.search(texto):
    # a diferencia del consolidado/diferencias (que solo generan y
    # dejan una vista previa - descargar es un clic aparte, igual por
    # voz que a mano), el botón "Exportar análisis" SÍ descarga de
    # una vez: sin "descargar_reporte" aquí, pedirlo por voz creaba
    # el archivo en el servidor pero nunca llegaba al dispositivo -
    # la persona no notaba nada distinto de antes de pedirlo.
    d = exportar_analisis(formato="xlsx", dias=30, u=u)
    return {"respuesta_hablada": "Listo, exporté el análisis de consumo de los últimos 30 días.",
          "accion": "descargar_reporte", "destino": None, "pestanas": "analisis",
          "archivo": d["archivo"]}

if _REP_CONSOLIDADO.search(texto):
    d = reporte(formato="xlsx", u=u)
    return {"respuesta_hablada": f"Listo, generé el consolidado: {d['filas']} filas.",
          "accion": "actualizar", "destino": None, "pestanas": "consolidado",
          "archivo": d["archivo"], "titulo_archivo": "Consolidado para My Inventory"}

return None

def _datos_reportes_narrados(u: Usuario) -> dict:
    """Los archivos recientes (misma fuente que la lista de Reportes) y el
    análisis de consumo de los últimos 30 días, ya en frases listas para
    hablar - los usan tanto el contexto completo que recibe Gemini como
    las respuestas determinísticas de _responder_reportes_por_voz."""
    recientes = reportes_recientes(u=u)
    analisis = api_analisis(dias=30, u=u)

    def _fila(a):
        extra = f", {a['filas']} filas" if a["filas"] is not None else ""
        return f"{a['titulo']} ({a['formato']}), hoy {a['hora']} {extra}"

    texto_recientes = ("todavía no se ha generado ningún archivo" if not recientes else
                      "; ".join(_fila(x) for x in recientes[:5]))
    ultimo_consolidado = next((x for x in recientes if x["titulo"] == "Consolidado para My
Inventory"), None)
    ultimas_diferencias = next((x for x in recientes if x["titulo"] == "Diferencias por bodega"),
None)
    subtil = analisis["subutilizados"]
    texto_subutil = ("sin insumos subutilizados en el período" if not subtil else
                    "; ".join(f"{s['nombre'].title():} sobran {s['sobra']} kg "
                              f"({s['sobrepedido_pct']}% de sobrepedido, en {s['veces']} servicios)"
                              for s in subtil[:5]))
    return {"recientes": recientes, "analisis": analisis, "texto_recientes": texto_recientes,
          "ultimo_consolidado": ultimo_consolidado, "ultimas_diferencias": ultimas_diferencias,
          "texto_subutil": texto_subutil}

_REPORTES_PREGUNTAS = [
    (re.compile(r"\bcu[aá]nt\w*\s+(filas|bodegas)\b.*\b[uú]ltimo\s+consolidado\b|"
      r"\bfilas\s+tiene\s+el\s+consolidado\b", re.IGNORECASE),
    lambda d: (f"El último consolidado tiene {d['ultimo_consolidado']['filas']} filas, "
      f"generado hoy {d['ultimo_consolidado']['hora']}."
      if d["ultimo_consolidado"] else "Todavía no se ha generado ningún consolidado.")),
    (re.compile(r"\bcu[aá]nt\w*\s+(filas|bodegas)\b.*\bdiferencias\b|"
      r"\bbodegas\s+tienen\s+descuadre\b", re.IGNORECASE),
    lambda d: (f"Las últimas diferencias exportadas tienen {d['ultimas_diferencias']['filas']} "
      f"filas, generadas hoy {d['ultimas_diferencias']['hora']}."

```

```

        if d["ultimas_diferencias"] else "Todavía no se han exportado diferencias por
bodega.")),
    (re.compile(r"\bqu[eé]\s+archivos\b.*\bgenerad\w*|\barchivos\s+(recientes|generados)\b",
    re.IGNORECASE),
    lambda d: f"Archivos recientes: {d['texto_recientes']}."),
    (re.compile(r"\bcu[aá]nto\b.*\bpidi[oó]\b|\bpedido\s+total\b|\bpedido\s+en\s+el\s+per[ií]odo\b",
    re.IGNORECASE),
    lambda d: f"En el período se pidieron {d['analisis']['pedido_total']} kilogramos, en "
    f"{d['analisis']['servicios_periodo']} servicios."),
    (re.compile(r"\bcu[aá]nto\b.*\bus[oó]\b|\baprovechamiento\b|\busado\s+realmente\b",
    re.IGNORECASE),
    lambda d: f"Se usaron realmente {d['analisis']['usado_total']} kilogramos, el "
    f"{d['analisis']['aprovechamiento']}% de lo pedido."),
    (re.compile(r"\bahorro\s+potencial\b", re.IGNORECASE),
    lambda d: f"El ahorro potencial es de {d['analisis']['ahorro_potencial']} kilogramos al "
    "mes si se ajustan las recetas."),
    (re.compile(r"\bcu[aá]nt\w*\s+insumos\s+(est[aá]n\s+)?subutilizad\w*|\binsumos\s+subutilizados\b",
    re.IGNORECASE),
    lambda d: f"Hay {len(d['analisis']['subutilizados'])} insumos subutilizados: "
    f"{d['texto_subutil']}."),
    (re.compile(r"\b(m[aá]s|mayor)\s+sobrepedido\b|\brecomendaci[oó]n\s+(del\s+)?agente\b",
    re.IGNORECASE),
    lambda d: (f"El insumo con más sobrepedido es "
    f"{d['analisis']['subutilizados'][0]['nombre'].title()}: sobra en "
    f"{d['analisis']['subutilizados'][0]['veces']} servicios, "
    f"{d['analisis']['subutilizados'][0]['sobrepedido_pct']}% más de lo que se usa."
    if d["analisis"]["subutilizados"] else
    "No hay insumos subutilizados en el período.")),
]

```

```

def _responder_reportes_por_voz(texto: str, u: Usuario) -> dict | None:
    """Preguntas frecuentes de Reportes (ambas pestañas), resueltas sin
    pasar por Gemini - mismo motivo que _responder_panel_por_voz: estas
    frases ya se ofrecen como garantizadas en el desplegable de ejemplos."""
    if u.perfil != "auditor":
        return None
    for patron, respuesta in _REPORTES_PREGUNTAS:
        if patron.search(texto):
            return {"respuesta_hablada": respuesta(_datos_reportes_narrados(u)),
                    "accion": None, "destino": None, "pestaña": None}
    return None

```

```

_VER_ARCHIVO_VERBO = re.compile(
    r"\b(mu[eé]stra\w*|ense[ñn]a\w*|[aá]bre\w*)\b", re.IGNORECASE)
_VER_ARCHIVO_PALABRA = re.compile(r"\barchivo\b", re.IGNORECASE)

```

```

def _previsualizar_reporte_por_voz(texto: str, u: Usuario) -> dict | None:
    """Muéstrame el detalle de bodega" / "ábreme el archivo de diferencias" -
    lo mismo que darle clic a una tarjeta de "Archivos generados", sin
    tocar la pantalla.

```

Diferencias, tablero, detalle de bodega y trazabilidad NO son el nombre de ninguna pestaña de esta pantalla, así que decir su nombre con un verbo de "mostrar" no compite con nada más - se abren directo, sin exigir la palabra "archivo". Consolidado y análisis SÍ son pestañas reales («muéstrame el consolidado» ya significa cambiar de pestaña, ver _navegar_pestaña_por_voz), así que para esos dos la palabra "archivo" sigue siendo obligatoria para desambiguar - sin ella, decir solo el nombre de la pestaña dejaría de poder cambiarla. Antes la palabra "archivo" era obligatoria para los seis tipos por igual, así que pedir "muéstrame el detalle de bodega" (la frase más

natural, sin decir "archivo") no abría nada y caía al agente general, que solo podía describir el archivo en palabras y mandar a dar clic a mano - no a abrirlo de verdad.

```

Cubre los cinco tipos que puede mostrar la lista (antes solo distinguía
diferencias y consolidado; el resto - tablero, detalle de bodega,
análisis, trazabilidad - siempre caía al último archivo generado, que
con frecuencia NO era el que se pedía)."""
if u.perfil != "auditor" or not _VER_ARCHIVO_VERBO.search(texto):
    return None
recientes = reportes_recientes(u=u)
if not recientes:
    return {"respuesta_hablada": "Todavía no se ha generado ningún archivo.",
            "accion": None, "destino": None, "pestaña": "consolidado"}
tiene_archivo = bool(_VER_ARCHIVO_PALABRA.search(texto))
if _REP_DIFERENCIAS.search(texto):
    elegido = next((x for x in recientes if x["titulo"] == "Diferencias por bodega"), None)
elif _REP_TABLERO.search(texto):
    elegido = next((x for x in recientes if x["titulo"] == "Estado del tablero"), None)
elif _REP_DETALLE_BODEGA.search(texto):
    elegido = next((x for x in recientes if x["titulo"] == "Detalle de bodega"), None)
elif _REP_TRAZA_ARCHIVO.search(texto):
    elegido = next((x for x in recientes if x["titulo"] == "Registro de trazabilidad"), None)
elif _REP_ANALISIS.search(texto) and tiene_archivo:
    elegido = next((x for x in recientes if x["titulo"] == "Análisis de consumo"), None)
elif _REP_CONSOLIDADO.search(texto) and tiene_archivo:
    elegido = next((x for x in recientes if x["titulo"] == "Consolidado para My Inventory"), None)
elif tiene_archivo:
    elegido = recientes[0]
else:
    return None
if not elegido:
    return {"respuesta_hablada": "No encontré ese archivo entre los generados recientemente.",
            "accion": None, "destino": None, "pestaña": "consolidado"}
detalle_filas = f", {elegido['filas']} filas" if elegido["filas"] is not None else ""
return {"respuesta_hablada": f"Aquí está: {elegido['titulo']}{detalle_filas}. "
        "¿Desea descargarlo?",
        "accion": "previsualizar_reporte", "destino": None, "pestaña": "consolidado",
        "archivo": elegido["archivo"], "titulo_archivo": elegido["titulo"]}

# Palabras que identifican cada pestaña, para navegar entre ellas sin
# pasar por Gemini - la cuota de Gemini ha fallado justo en este tipo de
# orden ("llévame a gestión de usuarios"), dejando la pestaña sin
# cambiar aunque la respuesta hablada dijera lo contrario.
_PALABRAS_PESTANA = {
    "ajustes": {
        "config": re.compile(r"configuraci[ó]n", re.IGNORECASE),
        "usuarios": re.compile(r"usuarios?", re.IGNORECASE),
        "recetas": re.compile(r"recetas?", re.IGNORECASE),
        "traza": re.compile(r"trazabilidad", re.IGNORECASE),
    },
    "reportes": {
        "consolidado": re.compile(r"consolidado", re.IGNORECASE),
        "analisis": re.compile(r"an[al]lisis|consumo", re.IGNORECASE),
    },
    "panel": {
        "resumen": re.compile(r"resumen", re.IGNORECASE),
        "alertas": re.compile(r"\balertas\b", re.IGNORECASE),
    },
    "auditoria": {
        "recuento": re.compile(r"recuento|recont[ae]o|\bcierre\b", re.IGNORECASE),
        "aprobaciones": re.compile(r"aprobaciones", re.IGNORECASE),
        "pedidos": re.compile(r"pedidos", re.IGNORECASE),
        "alertas": re.compile(r"\balertas\b", re.IGNORECASE),
    },
}

```

```

}
_VERBOS_IR_PESTANA = re.compile(
    r"\b(ir|ll[eé]va(me)?|vamos|ve|mu[eé]stra(me)?|[aá]bre(me)?|ens[eé][ñn]a(me)?)\b",
    re.IGNORECASE)
_ES_PREGUNTA_PESTANA = re.compile(r"[i?]\b(cu[aá]nt|bqu[eé]\b\b(cu[aá]l|bpor qu[eé]", re.IGNORECASE)

def _navegar_pestana_por_voz(vista: str, texto: str) -> dict | None:
    """Llévame a gestión de usuarios" (o cualquier pestaña de Ajustes,
    Reportes o Panel) - determinístico, sin pasar por Gemini, para que
    la navegación entre pestañas no dependa de su cuota. También navega
    si lo dicho es prácticamente el nombre de la pestaña solo ("Gestión
    de usuarios.", sin decir "llévame a" - la persona lo dice tal como
    lo lee en la pantalla), siempre que sea corto y no suene a pregunta
    ("¿cuántos usuarios hay?" no debe navegar, solo responderse)."""
    palabras = _PALABRAS_PESTANA.get(vista)
    if not palabras:
        return None
    tiene_verbo = bool(_VERBOS_IR_PESTANA.search(texto))
    limpio = re.sub(r"[.,;:!?i?j]+$", "", texto.strip())
    es_solo_el_nombre = (not _ES_PREGUNTA_PESTANA.search(texto)
                        and 0 < len(limpio.split()) <= 4)
    if not tiene_verbo and not es_solo_el_nombre:
        return None
    for clave, patron in palabras.items():
        if patron.search(texto):
            etiqueta = _PESTANAS_ASISTENTE[vista][clave]
            return {"respuesta_hablada": f"Listo, vamos a {etiqueta}.",
                    "accion": "navegar", "destino": vista, "pestana": clave}
    return None

# Los 4 "accesos rápidos" de Inicio ("Iniciar un conteo", "Ver el tablero",
# "Continuar auditoría"/"Hacer un pedido", "Generar reporte"/"Legalizar
# servicio") - las mismas tarjetas que ya se navegan con un clic.
_ACCESOS_INICIO = {
    "conteo": re.compile(r"\bconteo\b", re.IGNORECASE),
    "bodegas": re.compile(r"\btablero\b|bbodegas\b", re.IGNORECASE),
    "auditoria": re.compile(r"auditor[ií]a", re.IGNORECASE),
    "pedido": re.compile(r"\bpedidos?\b", re.IGNORECASE),
    "reportes": re.compile(r"reporte", re.IGNORECASE),
    "legalizacion": re.compile(r"legaliza", re.IGNORECASE),
}
_VERBOS_ACCESO_INICIO = re.compile(
    r"\b(inicia|iniciar|continua[ae]|continuar|genera|generar|legaliza|legalizar|hacer|haga)\b",
    re.IGNORECASE)

def _acceso_rapido_inicio_por_voz(texto: str, u: Usuario) -> dict | None:
    """Decir el nombre de una tarjeta de "¿Qué desea hacer?" navega directo
    a ella - determinístico, sin pasar por Gemini: se confirmó con
    pruebas que a veces decidía contestar en vez de navegar aunque la
    frase mencionara el destino tal cual ("iniciar un conteo" se quedaba
    en Inicio). Mismo filtro que _navegar_pestana_por_voz para no
    robarle la frase a las preguntas de KPI de esta pantalla («¿cuántas
    bodegas tengo?» debe contestarse, no navegar a Bodegas)."""
    tiene_verbo = bool(_VERBOS_IR_PESTANA.search(texto)) or bool(_VERBOS_ACCESO_INICIO.search(texto))
    limpio = re.sub(r"[.,;:!?i?j]+$", "", texto.strip())
    es_solo_el_nombre = (not _ES_PREGUNTA_PESTANA.search(texto)
                        and 0 < len(limpio.split()) <= 4)
    if not tiene_verbo and not es_solo_el_nombre:
        return None
    for destino, patron in _ACCESOS_INICIO.items():
        if not patron.search(texto):
            continue

```

```

        if destino in _SOLO_AUDITOR_ASISTENTE and u.perfil != "auditor":
            continue
        etiqueta = _DESTINOS_ASISTENTE[destino]
        return {"respuesta_hablada": f"Vamos a {etiqueta.lower()}.",
                "accion": "navegar", "destino": destino, "pestana": None}
    return None

_APROBAR_ITEM = re.compile(r"\bapru[eé]b\w*|\baprobar\b", re.IGNORECASE)
_RECHAZAR_ITEM = re.compile(r"\brech[aá]z\w*", re.IGNORECASE)

def _resolver_aprobacion_por_voz(texto: str, u: Usuario) -> dict | None:
    """Aprueba/rechaza <nombre> desde Auditoría (pestaña Aprobaciones) -
    mismo patrón determinístico que Ajustes/Reportes: busca el nombre
    completo entre las aprobaciones pendientes (como frase exacta, no
    palabra por palabra - "aprueba costilla de res" no debe aprobar
    cualquier cosa que contenga "res") y llama la MISMA función que ya
    usa el botón, solo administrador."""
    if u.perfil != "auditor":
        return None
    if _APROBAR_ITEM.search(texto):
        verbo = "aprobar"
    elif _RECHAZAR_ITEM.search(texto):
        verbo = "rechazar"
    else:
        return None
    t = _normalizar_nombre(texto)
    with Sesion() as s:
        pendientes = s.query(Aprobacion).filter_by(estado="pendiente").all()
        encontrada = next(
            (a for a in pendientes
             if re.search(rf"\b{re.escape(_normalizar_nombre(a.nombre))}\b", t)),
            None)
        if not encontrada:
            return None
        aid, nombre = encontrada.id, encontrada.nombre
    if verbo == "aprobar":
        aprobar(aprobacion_id=aid, u=u)
        return {"respuesta_hablada": f"Listo, aprobé {nombre.title()}. Ya entra al catálogo oficial.",
                "accion": "actualizar", "destino": None, "pestana": None}
    rechazar(aprobacion_id=aid, u=u)
    return {"respuesta_hablada": f"Listo, rechacé {nombre.title()}.",
            "accion": "actualizar", "destino": None, "pestana": None}

_APROBAR_RECHAZAR_PEDIDO = re.compile(
    r"\b(apru[eé]b\w*|rech[aá]z\w*)\b.*\bpedido\b|\bpedido\b.*\b(apru[eé]b\w*|rech[aá]z\w*)\b",
    re.IGNORECASE)

def _resolver_pedido_pendiente_por_voz(texto: str, u: Usuario) -> dict | None:
    """Aprueba/rechaza el pedido de <persona o plato> desde Auditoría
    (pestaña Pedidos pendientes) - mismo patrón que _resolver_aprobacion_por_voz,
    pero exige la palabra "pedido" para no competir con aprobar/rechazar
    un producto o bodega nueva, que es otra lista con los mismos verbos."""
    if u.perfil != "auditor" or not _APROBAR_RECHAZAR_PEDIDO.search(texto):
        return None
    if _APROBAR_ITEM.search(texto):
        aprobar_bool = True
    elif _RECHAZAR_ITEM.search(texto):
        aprobar_bool = False
    else:
        return None

```

```

t = _normalizar_nombre(texto)
pendientes = pedidos_pendientes(u=u)
if not pendientes:
    return {"respuesta_hablada": "No hay pedidos pendientes de aprobación.",
            "accion": None, "destino": None, "pestaña": "pedidos"}
encontrado = next(
    (p for p in pendientes
     if (p["persona"] and p["persona"] != "-"
         and re.search(rf"\b{re.escape(_normalizar_nombre(p['persona']))}\b", t))
     or re.search(rf"\b{re.escape(_normalizar_nombre(p['plato']))}\b", t)),
    None)
if not encontrado:
    return None
resolver_pedido(numero_pedido=encontrado["numero_pedido"],
                 datos=ResolverPedidoIn(aprobar=aprobar_bool), u=u)
verbo_pasado = "aprobé" if aprobar_bool else "rechacé"
return {"respuesta_hablada": f"Listo, {verbo_pasado} el pedido de "
                             f"{encontrado['persona'].title(): {encontrado['plato']}}.",
        "accion": "actualizar", "destino": None, "pestaña": "pedidos"}

_RESOLVER_ALERTA = re.compile(r"\bresuelve\b|\bresolver\b", re.IGNORECASE)

def _resolver_alerta_por_voz(texto: str, u: Usuario) -> dict | None:
    """Resuelve la alerta de <artículo o bodega> desde Auditoría
    (pestaña Bandeja de alertas) - mismo patrón determinístico, busca
    entre las alertas abiertas por el artículo o la bodega que mencionan."""
    if u.perfil != "auditor" or not _RESOLVER_ALERTA.search(texto):
        return None
    abiertas = ver_alertas(resueltas=0, u=u)
    if not abiertas:
        return {"respuesta_hablada": "No hay alertas abiertas.",
                "accion": None, "destino": None, "pestaña": "alertas"}
    t = _normalizar_nombre(texto)
    encontrada = next(
        (a for a in abiertas
         if (a["articulo"] and re.search(rf"\b{re.escape(_normalizar_nombre(a['articulo']))}\b", t))
         or (a["bodega"] and re.search(rf"\b{re.escape(_normalizar_nombre(a['bodega']))}\b", t))),
        None)
    if not encontrada:
        return None
    resolver_alerta(alerta_id=encontrada["id"], u=u)
    detalle = encontrada["articulo"] or encontrada["bodega"] or encontrada["titulo"]
    return {"respuesta_hablada": f"Listo, resolví la alerta de {detalle.title()}.",
            "accion": "actualizar", "destino": None, "pestaña": "alertas"}

_VERBOS_AUDITAR_BODEGA = re.compile(r"\b(abr[ae]|abrir|audit\b*)\b", re.IGNORECASE)

def _abrir_bodega_para_auditoria_por_voz(texto: str, u: Usuario) -> dict | None:
    """Audita <bodega> / "abre <bodega>" desde Auditoría (pestaña
    Recuento ciego y cierre) - selecciona esa bodega, lo mismo que darle
    clic a "Abrir" en su tarjeta. A propósito NO llega hasta firmar ni
    cerrar: eso sigue siendo manual, es la garantía de un cierre con
    doble firma que nadie dispara sin querer con la voz."""
    if u.perfil != "auditor" or not _VERBOS_AUDITAR_BODEGA.search(texto):
        return None
    from servicios.conciliacion import buscar_bodega
    with Sesion() as s:
        candidatas_ids = {b.id for b in s.query(Bodega)
                          .filter(Bodega.estado.in_([ "en_auditoria", "cerrada" ])).all()}
        if not candidatas_ids:

```

```

        return {"respuesta_hablada": "No hay ninguna bodega lista para recuento ciego o "
            "cierre en este momento.",
            "accion": None, "destino": None, "pestanas": "recuento"}
        bodega = buscar_bodega(s, texto, candidatas_ids)
    if not bodega:
        return None
    return {"respuesta_hablada": f"Vamos a auditar {bodega.nombre_oficial.title()}.",
        "accion": "navegar", "destino": "auditoria", "pestanas": "recuento",
        "bodega_auditar": bodega.nombre_oficial}

def _datos_auditoria_narrados(u: Usuario) -> dict:
    return {"resumen_alertas": resumen_alertas(u=u), "pedidos": pedidos_pendientes(u=u),
        "aprobaciones": listar_aprobaciones(estado="pendiente", u=u)}

_AUDITORIA_PREGUNTAS = [
    (re.compile(r"\bcu[aá]nt\w*\s+alertas\b.*\babiertas\b|\balertas\s+abiertas\b", re.IGNORECASE),
        lambda d: f"Hay {d['resumen_alertas']['abiertas']} alertas abiertas."),
    (re.compile(r"\bcu[aá]nt\w*\b.*\bresolvieron\s+hoy\b|\bresueltas\s+hoy\b", re.IGNORECASE),
        lambda d: f"Se resolvieron {d['resumen_alertas']['resueltas_hoy']} alertas hoy."),
    (re.compile(r"\btipo\s+medio\b", re.IGNORECASE),
        lambda d: (f"El tiempo medio de resolución es de "
            f"{d['resumen_alertas']['tiempo_medio_min']} minutos."
            if d["resumen_alertas"]["tiempo_medio_min"] is not None else
            "Todavía no se ha resuelto ninguna alerta hoy para calcular el tiempo medio.")),
    (re.compile(r"\bcu[aá]nt\w*\s+pedidos\b.*\bpendientes\b", re.IGNORECASE),
        lambda d: (f"Hay {len(d['pedidos'])} pedidos pendientes de aprobación."
            if d["pedidos"] else "No hay pedidos pendientes de aprobación.")),
    (re.compile(r"\bcu[aá]nt\w*\s+aprobaciones\b", re.IGNORECASE),
        lambda d: (f"Hay {len(d['aprobaciones'])} aprobaciones pendientes: " +
            "; ".join(a["nombre"].title() for a in d["aprobaciones"][:6]) + "."
            if d["aprobaciones"] else "No hay aprobaciones pendientes.")),
]

def _responder_auditoria_por_voz(texto: str, u: Usuario) -> dict | None:
    if u.perfil != "auditor":
        return None
    for patron, respuesta in _AUDITORIA_PREGUNTAS:
        if patron.search(texto):
            return {"respuesta_hablada": respuesta(_datos_auditoria_narrados(u)),
                "accion": None, "destino": None, "pestanas": None}
    return None

_UNIDADES_RECETA = r"kilos?|kg|litros?|lt|unidades?|und?|gramos?|gr"
# Cuando alguien encadena dos órdenes en una sola frase ("...con 2 kg
# de papa, agrega 300 g de cebolla a la receta sopa" - confirmado con
# una captura real), el último grupo capturado (.+? anclado al final
# con $) se tragaba TODO lo que seguía, incluida la segunda orden
# completa, como si fuera parte del dato ("no encontré «papa, agrega
# 300 g de cebolla a la receta sopa» en el catálogo"). Este sufijo se
# detiene apenas aparece una coma seguida de otro verbo reconocido, y
# descarta el resto - la primera orden se cumple bien, y la segunda
# queda pendiente de decirse aparte (tal como ya se explica: "puede
# agregarle más ingredientes diciendo «agrega...»").
# El verbo "crea/crear" es OPCIONAL: el reconocimiento de voz a veces
# recorta la primera palabra si la persona empieza a hablar apenas
# hace clic en el micrófono, antes de que el navegador esté listo -
# confirmado con una captura real ("Una receta llamada sopa,
# rendimiento, cuatro porciones." sin "crea" al inicio). El resto de
# la frase (receta llamada... rendimiento... con... de...) ya es lo
# bastante específico para no confundirse con una pregunta suelta.

```



```

_CREAR_RECETA = re.compile(
    r"\b(?:cre[ae]\w*\s+)?(?:una\s+)?(?:nuev[oa]\s+)?receta\s+llamad[ao]\s+(?P<nombre>.+?)"
    r"\s*[,\y]*\s*(?:con\s+)?rendimiento\s+(?:de\s+)?(?P<rend>.+?)\s*porciones?"
    r"\s*[,\y]*\s*con\s+(?P<cant>.+?)\s*"
    rf"(?:{_UNIDADES_RECETA})?\s*de\s+(?P<ing>.+?)\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
    re.IGNORECASE)
_AGREGAR_INGREDIENTE = re.compile(
    r"\b(?:agr[eé]ga\w*|a[ñn]ad\w*)\s+(?P<cant>.+?)\s*"
    rf"(?:{_UNIDADES_RECETA})?\s*de\s+(?P<ing>.+?)\s*"
    r"\s+a\s+la\s+receta\s+(?P<receta>.+?)\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
    re.IGNORECASE)
_ELIMINAR_RECETA = re.compile(
    r"\b(elimin\w*|borr\w*)\s+(?:la\s+)?receta\s+(?P<receta>.+?)\s*"
    + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
    re.IGNORECASE)

```

```

# Memoria viva de una sola pregunta pendiente por persona: "crea una
# receta... con papa" -> ambiguo -> la persona contesta solo "papa
# criolla" en el siguiente turno, y eso completa la orden original en
# vez de tener que repetirla entera. Mismo patron que ESTADOS en
# agente/orquestador.py (memoria en RAM, se pierde si el servidor se
# reinicia - aceptable aqui: en el peor caso, toca repetir la orden
# completa, que es lo que ya se pedia antes de esto). Expira sola
# despues de _VENCIMIENTO_AMBIGUEDAD segundos para no quedar pegada a
# una respuesta que en realidad era para otra cosa, dicha mucho despues.
_PENDIENTE_AMBIGUEDAD: dict[int, dict] = {}
_VENCIMIENTO_AMBIGUEDAD = 90

```

```

def _elegir_ingredientes_sin_ambiguedad(dicho: str) -> dict | None:
    """buscar_articulo() puntúa por cobertura de palabras: una consulta
    de una sola palabra genérica ("papa") saca 100% de cobertura contra
    CUALQUIER artículo que la contenga, sin importar cuánto más tenga
    ese nombre alrededor - confirmado con una captura real donde "papa"
    empataba a 100 de confianza con "EMPANADA DE PAPA Y CARNE X50 GR",
    "PAPA A LA FRANCESA" y "PAPA CABELLO DE ANGEL" por igual, y se
    quedaba con el primero de la lista sin ningún criterio real. Igual
    que buscar_bodega ya hace con bodegas ambiguas: mejor decir "sea
    más específico" que adivinar mal en silencio y crear una receta con
    el ingrediente equivocado."""
    from servicios.conciliacion import buscar_articulo, normalizar
    # limite alto: buscar_articulo por defecto solo devuelve los 3
    # mejores, y con un empate a 100 (el caso ambiguo que nos interesa
    # aquí) esos 3 son un subconjunto ARBITRARIO del catálogo real -
    # confirmado con una captura real donde salían "Empanada De Papa Y
    # Carne" y "Papa A La Francesa" en vez de las papas de verdad
    # (Criolla, Sabanera, Pastusa) que sí existen en el catálogo. Con
    # más candidatos a la vista, se puede priorizar los nombres más
    # simples (probablemente el ingrediente crudo) al armar la lista.
    candidatos = buscar_articulo(dicho, limite=20)
    if not candidatos or candidatos[0]["confianza"] < 60:
        return {"error": f"No encontré «{dicho}» en el catálogo. Dígallo como aparece "
                        "en la etiqueta."}
    tope = candidatos[0]["confianza"]
    empatados = [c for c in candidatos if c["confianza"] == tope]
    if len(empatados) > 1:
        # Un empate NO es ambiguo si uno de ellos es el nombre EXACTO
        # dicho ("papa criolla" == "PAPA CRIOLLA") - eso gana solo,
        # aunque "PAPA CRIOLLA PRECOCIDA" haya empatado en puntaje.
        clave = normalizar(dicho)
        exacto = next((c for c in empatados if normalizar(c["nombre"]) == clave), None)
        if exacto:
            return {"articulo": exacto}
        # De los empatados, primero los nombres más cortos: un
        # ingrediente crudo ("PAPA CRIOLLA") es más probable que sea lo

```

```

        # que la persona quiso decir que un producto elaborado con
        # media frase de más ("EMPANADA DE PAPA Y CARNE X50 GR").
        empatados.sort(key=lambda c: len(c["nombre"].split()))
        mostrados = empatados[:5]
        opciones = ", ".join(c["nombre"].title() for c in mostrados)
        return {"error": f"«{dicho}» es ambiguo, encontré varios parecidos: {opciones}. "
                "Dígallo más específico, como aparece completo en la etiqueta.",
                "opciones": mostrados}
    return {"articulo": candidatos[0]}

def _buscar_receta_por_nombre(s, nombre_dicho: str):
    """Una coincidencia EXACTA siempre gana primero: con dos recetas
    "Sopa" y "Sopa de la casa", decir "Sopa de la casa" no debe caer en
    "Sopa" solo porque su nombre es substring - eso pasaba antes (se
    quedaba con la PRIMERA que calzara, sin importar el orden de
    especificidad) y edita/borraba la receta equivocada en silencio."""
    t = _normalizar_nombre(nombre_dicho)
    recetas = s.query(Receta).all()
    exacta = next((r for r in recetas if _normalizar_nombre(r.nombre) == t), None)
    if exacta:
        return exacta
    return next(
        (r for r in recetas if re.search(rf"\b{re.escape(_normalizar_nombre(r.nombre))}\b", t)
         or re.search(rf"\b{re.escape(t)}\b", _normalizar_nombre(r.nombre))),
        None)

_INTENCION_CREAR_RECETA = re.compile(
    r"\b(cre[ae]\w*|necesito|quiero)\b.*\brecetas?\b|\breceta\s+llamad[ao]\b",
    re.IGNORECASE)

def _completar_crear_receta(nombre: str, rend: float, cant: float, articulo: dict,
                           u: Usuario) -> dict:
    with Sesion() as s:
        if s.query(Receta).filter(Receta.nombre.ilike(nombre)).first():
            return {"respuesta_hablada": f"Ya existe una receta llamada {nombre}.",
                    "accion": None, "destino": None, "pestana": None}
        r = Receta(nombre=nombre, rendimiento=int(rend), preparacion="")
        s.add(r)
        s.flush()
        s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=articulo["codigo"],
                                cantidad_por_porcion=float(cant)))
        s.commit()
    registrar(u, "RECETA", f"Receta creada: {nombre} (1 ingrediente) por voz", "ok")
    return {"respuesta_hablada": f"Listo, creé la receta {nombre} con {articulo['nombre'].title()}. "
                                "Puede agregarle más ingredientes diciendo «agrega... a la "
                                f"receta {nombre}», o editarla desde la pantalla.",
            "accion": "actualizar", "destino": None, "pestana": None}

def _crear_receta_por_voz(texto: str, u: Usuario) -> dict | None:
    """Crea una receta llamada <nombre>, rendimiento <N> porciones, con
    <cantidad> de <ingrediente> - determinístico, con el mismo criterio
    que crear usuario: un formato de frase concreto, en vez de adivinar
    de cualquier forma un nombre y una lista de ingredientes que no
    existen todavía contra qué validar. El ingrediente Sí se busca
    contra el catálogo real (misma función que usa Conteo/Pedido) - si
    no lo encuentra con confianza suficiente, no crea nada a medias.
    Decir solo "crea una receta" (sin los demás datos) coincidía por
    casualidad con la navegación a la pestaña Recetas - mismo arreglo
    que crear usuario."""
    if u.perfil != "auditor":

```

```

        return None
    m = _CREAR_RECETA.search(texto.strip())
    if not m:
        if (_INTENCION_CREAR_RECETA.search(texto)
            and not _ES_PREGUNTA_PESTANA.search(texto)):
            return {"respuesta_hablada": "Para crear una receta diga: «crea una receta "
                                         "llamada» el nombre, «rendimiento» el número de "
                                         "porciones, «con» la cantidad «de» el primer "
                                         "ingrediente.",
                    "accion": None, "destino": None, "pestana": None}
        return None
    nombre = re.sub(r"[.,;:;!¿?;]+$", "", m.group("nombre")).strip()
    if not nombre:
        return None
    # El rendimiento y la cantidad se dicen casi siempre en palabras
    # ("cuatro porciones", no "4 porciones" - así habla la gente de
    # verdad), así que se interpretan con el mismo parser de números
    # que ya usa Conteo/Pedido, no un \d+ que solo entiende dígitos.
    rend = _numero_de_texto(m.group("rend"))
    cant = _numero_de_texto(m.group("cant"))
    if rend is None or cant is None:
        return {"respuesta_hablada": "No entendí el rendimiento o la cantidad. Dígalos en "
                                     "número, por ejemplo «cuatro porciones». ",
                "accion": None, "destino": None, "pestana": None}
    elegido = _elegir_ingredientes_sin_ambigüedad(m.group("ing"))
    if "error" in elegido:
        if "opciones" in elegido:
            _PENDIENTE_AMBIGÜEDAD[u.id] = {
                "tipo": "crear_receta", "nombre": nombre, "rend": rend, "cant": cant,
                "opciones": elegido["opciones"], "ts": ahora()}
            return {"respuesta_hablada": elegido["error"],
                    "accion": None, "destino": None, "pestana": None}
    return _completar_crear_receta(nombre, rend, cant, elegido["articulo"], u)

_INTENCION_AGREGAR_INGREDIENTE = re.compile(
    r"\b(agr[eé]ga\w*|a[ñn]ad\w*)\b.*\breceta\b", re.IGNORECASE)

def _completar_agregar_ingredientes(cantidad_nueva: float, receta_dicha: str,
                                    articulo: dict, u: Usuario) -> dict | None:
    with Sesión() as s:
        r = _buscar_receta_por_nombre(s, receta_dicha)
        if not r:
            return None
        lineas = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
        ya_tiene = next((li for li in lineas if li.articulo_codigo == articulo["codigo"]), None)
        if ya_tiene:
            ya_tiene.cantidad_por_porcion = cantidad_nueva
        else:
            s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=articulo["codigo"],
                                    cantidad_por_porcion=cantidad_nueva))
        s.commit()
        nombre_receta = r.nombre
        registrar(u, "RECETA", f"Receta editada: {nombre_receta} (ingrediente agregado por voz)", "ok")
        return {"respuesta_hablada": f"Listo, agregué {articulo['nombre'].title()} a la receta {nombre_receta}.",
                "accion": "actualizar", "destino": None, "pestana": None}

def _agregar_ingredientes_por_voz(texto: str, u: Usuario) -> dict | None:
    """Agrégle <cantidad> de <ingrediente> a la receta <nombre>""" -
    determinístico. Reusa la misma búsqueda de artículo que crear
    receta, y conserva los ingredientes que ya tenía (PUT reemplaza
    toda la lista, así que se lee la receta primero)."""

```

```

if u.perfil != "auditor":
    return None
m = _AGREGAR_INGREDIENTE.search(texto.strip())
if not m:
    # "preparación" en la frase es de _agregar_preparacion_por_voz,
    # no de esta - sin la exclusión, "agrega la preparación a la
    # receta X: ..." caía aquí primero (comparte "agrega...receta")
    # y nunca le daba la oportunidad a la función correcta.
    if (_INTENCION_AGREGAR_INGREDIENTE.search(texto)
        and not re.search(r"preparaci[oó]n", texto, re.IGNORECASE)
        and not _ES_PREGUNTA_PESTANA.search(texto)):
        return {"respuesta_hablada": "Para agregar un ingrediente diga: «agrega» la "
                                     "cantidad «de» el ingrediente «a la receta» el "
                                     "nombre de la receta.",
                "accion": None, "destino": None, "pestana": None}
    return None
cantidad_nueva = _numero_de_texto(m.group("cant"))
if cantidad_nueva is None:
    return {"respuesta_hablada": "No entendí la cantidad. Dígala en número, por "
                                "ejemplo «trescientos gramos».",
            "accion": None, "destino": None, "pestana": None}
elegido = _elegir_ingrediente_sin_ambigüedad(m.group("ing"))
if "error" in elegido:
    if "opciones" in elegido:
        _PENDIENTE_AMBIGUEDAD[u.id] = {
            "tipo": "agregar_ingrediente", "cant": cantidad_nueva,
            "receta": m.group("receta"), "opciones": elegido["opciones"],
            "ts": ahora()}
        return {"respuesta_hablada": elegido["error"],
                "accion": None, "destino": None, "pestana": None}
    return _completar_agregar_ingrediente(cantidad_nueva, m.group("receta"),
                                           elegido["artículo"], u)

def _resolver_ambigüedad_pendiente(texto: str, u: Usuario) -> dict | None:
    """Si la última respuesta de esta persona fue "es ambiguo, encontré
    Papa Sabanera, Papa Criolla, Papa Pastusa..." y en este turno dice
    solo "papa pastusa" (sin repetir la orden completa), esto retoma la
    orden original con ese ingrediente ya resuelto. Se revisa AL FINAL
    de la cadena de comandos de Ajustes, después de que crear/agregar
    ya tuvieron su oportunidad de matchear la frase completa - así una
    orden repetida entera (en vez de solo el nombre corto) sigue
    tomando el camino normal, con sus datos frescos, no los guardados
    aquí de la vez anterior."""
    pendiente = _PENDIENTE_AMBIGUEDAD.get(u.id)
    if not pendiente:
        return None
    if (ahora() - pendiente["ts"]).total_seconds() > _VENCIMIENTO_AMBIGUEDAD:
        del _PENDIENTE_AMBIGUEDAD[u.id]
        return None
    t = _normalizar_nombre(texto)
    elegido = next(
        (c for c in pendiente["opciones"]
         if re.search(rf"\b{re.escape(_normalizar_nombre(c['nombre']))}\b", t)
         or re.search(rf"\b{re.escape(t)}\b", _normalizar_nombre(c["nombre"]))),
        None)
    if not elegido:
        return None
    del _PENDIENTE_AMBIGUEDAD[u.id]
    if pendiente["tipo"] == "crear_receta":
        return _completar_crear_receta(pendiente["nombre"], pendiente["rend"],
                                       pendiente["cant"], elegido, u)
    return _completar_agregar_ingrediente(pendiente["cant"], pendiente["receta"], elegido, u)

def _eliminar_receta_por_voz(texto: str, u: Usuario) -> dict | None:

```

```

"""Elimina la receta <nombre>" - determinístico, busca por nombre
completo entre las recetas existentes, igual que aprobaciones."""
if u.perfil != "auditor":
    return None
m = _ELIMINAR_RECETA.search(texto.strip())
if not m:
    return None
with Sesion() as s:
    r = _buscar_receta_por_nombre(s, m.group("receta"))
    if not r:
        return None
    rid, nombre = r.id, r.nombre
    s.query(RecetaIngrediente).filter_by(receta_id=rid).delete()
    s.delete(r)
    s.commit()
    registrar(u, "RECETA", f"Receta eliminada: {nombre} por voz", "ok")
    return {"respuesta_hablada": f"Listo, eliminé la receta {nombre}.",
            "accion": "actualizar", "destino": None, "pestanas": None}

_QUITAR_INGREDIENTE = re.compile(
    r"\b(?:qu[i]ta|elim|borr|elimin|borr|w*)\s+(?:el\s+)?ingrediente\s+(?P<ing>.+)"
    r"\s+de\s+la\s+receta\s+(?P<receta>.+)\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
    re.IGNORECASE)

def _quitar_ingredientes_por_voz(texto: str, u: Usuario) -> dict | None:
    """Quita/elimina el ingrediente <ingrediente> de la receta <nombre>" -
    mismo botón «Editar» de Recetas, pero solo para sacar UN ingrediente
    (no reemplaza toda la lista). Busca el ingrediente entre los que YA
    tiene esa receta, no contra todo el catálogo - así "quita la
    cebolla" no confunde una cebolla que ni siquiera está en esa
    receta. Nunca deja una receta sin ingredientes (la misma regla que
    ya exige el botón)."""
    if u.perfil != "auditor":
        return None
    m = _QUITAR_INGREDIENTE.search(texto.strip())
    if not m:
        return None
    with Sesion() as s:
        r = _buscar_receta_por_nombre(s, m.group("receta"))
        if not r:
            return None
        lineas = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
        t_ing = _normalizar_nombre(m.group("ing"))
        objetivo = None
        nombre_articulo = None
        for li in lineas:
            art = s.get(Articulo, li.articulo_codigo)
            if not art:
                continue
            nombre_art = _normalizar_nombre(art.nombre_oficial)
            if (re.search(rf"\b{re.escape(nombre_art)}\b", t_ing)
                or re.search(rf"\b{re.escape(t_ing)}\b", nombre_art)):
                objetivo, nombre_articulo = li, art.nombre_oficial
                break
        if not objetivo:
            return {"respuesta_hablada": f"{r.nombre.title()} no tiene un ingrediente llamado "
                                         f"«{m.group('ing')}».",
                    "accion": None, "destino": None, "pestanas": None}
        if len(lineas) <= 1:
            return {"respuesta_hablada": f"No puedo quitar {nombre_articulo.title()}: "
                                         f"{r.nombre.title()} se quedaría sin ingredientes.",
                    "accion": None, "destino": None, "pestanas": None}
        s.delete(objetivo)
        nombre_receta = r.nombre

```

```

        s.commit()
        registrar(u, "RECETA", f"Receta editada: {nombre_receta} ({nombre_articulo} quitado por voz)",
"ok")
        return {"respuesta_hablada": f"Listo, quité {nombre_articulo.title()} de la receta
{nombre_receta}.",
                "accion": "actualizar", "destino": None, "pestana": None}

_CAMBIAR_RENDIMIENTO = re.compile(
    r"\bcambia\w*\s+el\s+rendimient\w*\s+de\s+la\s+receta\s+(?P<receta>.+?)"
    r"\s+a\s+(?P<rend>.+?)\s*porcion\w*\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$",
    re.IGNORECASE)

def _cambiar_rendimiento_por_voz(texto: str, u: Usuario) -> dict | None:
    """Cambia el rendimiento de la receta <nombre> a <N> porciones" -
    otro campo editable del botón «Editar» de Recetas (además de los
    ingredientes, que se agregan/quitan aparte, y la preparación, que
    se dicta con _agregar_preparacion_por_voz)."""
    if u.perfil != "auditor":
        return None
    m = _CAMBIAR_RENDIMIENTO.search(texto.strip())
    if not m:
        return None
    nuevo = _numero_de_texto(m.group("rend"))
    if nuevo is None:
        return {"respuesta_hablada": "No entendí el rendimiento. Dígallo en número, por "
                                     "ejemplo «seis porciones». ",
                "accion": None, "destino": None, "pestana": None}
    nuevo = int(nuevo)
    with Sesion() as s:
        r = _buscar_receta_por_nombre(s, m.group("receta"))
        if not r:
            return None
        if r.rendimiento == nuevo:
            return {"respuesta_hablada": f"{r.nombre.title()} ya rendía {nuevo} porciones.",
                    "accion": "actualizar", "destino": None, "pestana": None}
        r.rendimiento = nuevo
        nombre = r.nombre
        s.commit()
        registrar(u, "RECETA", f"Receta editada: {nombre} (rendimiento -> {nuevo} por voz)", "ok")
        return {"respuesta_hablada": f"Listo, {nombre.title()} ahora rinde {nuevo} porciones.",
                "accion": "actualizar", "destino": None, "pestana": None}

_AGREGAR_PREPARACION = re.compile(
    r"\b(?:agr[eé]ga\w*|cambia\w*|pon\w*|dicta\w*|escrib\w*)\s+(?:la\s+)?preparaci[oó]n"
    r"\s+(?:de\s+|a\s+)?(?:la\s+receta\s+)?(?P<receta>.+?)\s*[:,\s]*(?P<pasos>.+?)\s*$",
    re.IGNORECASE)
_INTENCION_AGREGAR_PREPARACION = re.compile(
    r"\b(?:agr[eé]ga\w*|cambia\w*|pon\w*|dicta\w*|escrib\w*)\b.*\bpreparaci[oó]n\b",
    re.IGNORECASE)

def _agregar_preparacion_por_voz(texto: str, u: Usuario) -> dict | None:
    """Agrega la preparación a la receta <nombre>: <pasos>" - a
    diferencia de nombre/ingrediente/rendimiento (frases cortas), aquí
    todo lo que sigue a los dos puntos ES el dato, tal cual, sin cortar
    en la primera coma o punto (los pasos de cocina naturalmente tienen
    varias oraciones: "primero pele las papas. Luego sofía...") - por
    eso esta es la ÚNICA función de Recetas que NO usa
    _CORTE_ORDEN_ENCADENADA. Reemplaza la preparación completa (como el
    textarea del botón «Editar»), no la suma a lo que ya había."""
    if u.perfil != "auditor":

```

```

        return None
    m = _AGREGAR_PREPARACION.search(texto.strip())
    if not m:
        if (_INTENCION_AGREGAR_PREPARACION.search(texto)
            and not _ES_PREGUNTA_PESTANA.search(texto)):
            return {"respuesta_hablada": "Para dictar la preparación diga: «agrega la "
                                     "preparación a la receta» el nombre, dos puntos, y "
                                     "los pasos - por ejemplo «agrega la preparación a "
                                     "la receta Sopa: pele las papas, sofría la cebolla, "
                                     "y cocine todo junto veinte minutos». ",
                    "accion": None, "destino": None, "pestana": None}
            return None
    pasos = re.sub(r"[.,;:!?i]+$", "", m.group("pasos")).strip()
    if not pasos:
        return None
    with Sesion() as s:
        r = _buscar_receta_por_nombre(s, m.group("receta"))
        if not r:
            return None
        r.preparacion = pasos
        nombre = r.nombre
        s.commit()
    registrar(u, "RECETA", f"Receta editada: {nombre} (preparación dictada por voz)", "ok")
    return {"respuesta_hablada": f"Listo, guardé la preparación de la receta {nombre}.",
            "accion": "actualizar", "destino": None, "pestana": None}

def _cambiar_perfil_por_voz(texto: str, u: Usuario) -> dict | None:
    """Cambia el perfil de <nombre> a auxiliar/administrador - mismo
    botón «Editar» de Gestión de usuarios, solo el campo perfil (el
    correo no se toca por voz: un correo mal transcrito rompería el
    contacto sin que se note). Llama la MISMA función que usa el botón
    (editar_usuario), así que hereda su regla de no poder cambiarse el
    propio rol - excluida además desde la búsqueda del nombre."""
    if u.perfil != "auditor" or not re.search(r"\bperfil\b", texto, re.IGNORECASE):
        return None
    m = re.search(r"\b(auxiliar|auditor|administrador)\b", texto, re.IGNORECASE)
    if not m:
        if not _ES_PREGUNTA_PESTANA.search(texto):
            return {"respuesta_hablada": "Diga a qué perfil: «cambia el perfil de» el "
                                     "nombre «a» auxiliar o administrador.",
                    "accion": None, "destino": None, "pestana": None}
            return None
    perfil_nuevo = "auditor" if m.group(1).lower() in ("auditor", "administrador") else "auxiliar"
    t = _normalizar_nombre(texto)
    with Sesion() as s:
        candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
        encontrado = next(
            (c for c in candidatos
             if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t)),
            None)
        if not encontrado:
            return {"respuesta_hablada": "No encontré a esa persona. Diga el nombre completo "
                                     "tal como aparece en la lista.",
                    "accion": None, "destino": None, "pestana": None}
        etiqueta = "administrador" if perfil_nuevo == "auditor" else "auxiliar"
        if encontrado.perfil == perfil_nuevo:
            return {"respuesta_hablada": f"{encontrado.nombre.capitalize()} ya era {etiqueta}.",
                    "accion": "actualizar", "destino": None, "pestana": None}
        eid, nombre = encontrado.id, encontrado.nombre
        editar_usuario(usuario_id=eid, datos=EditarUsuarioIn(perfil=perfil_nuevo), u=u)
    return {"respuesta_hablada": f"Listo, {nombre.capitalize()} ahora es {etiqueta}.",
            "accion": "actualizar", "destino": None, "pestana": None}

_ASIGNAR_BODEGA = re.compile(

```

```

r"\bas[ií]gn\w*\s+(?:le\s+)?(?:la\s+)?bodega\s+(?P<bodega>.+?)\s+a\s+(?P<nombre>.+?)"
r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$", re.IGNORECASE)
_QUITAR_BODEGA = re.compile(
r"\bqu[ií]t\w*\s+(?:le\s+)?(?:la\s+)?bodega\s+(?P<bodega>.+?)\s+a\s+(?P<nombre>.+?)"
r"\s*" + _CORTE_ORDEN_ENCADENADA + r"[.!?]\s*$", re.IGNORECASE)

def _asignar_bodega_por_voz(texto: str, u: Usuario) -> dict | None:
    """Asigne/quítale la bodega <bodega> a <nombre>" - mismo botón
    «Asignar bodegas», pero sumando o quitando SOLO la bodega dicha en
    vez de reemplazar toda la lista (que es lo que hace el botón, que
    parte de las bodegas ya marcadas): así una orden de voz nunca borra
    por accidente asignaciones hechas por otro medio. El nombre de la
    bodega se busca con el mismo buscador que «abra <bodega>» en
    Conteo, así que entiende apodos parciales igual ("kiosco taquilla"
    encuentra "KIOSCO TAQUILLA AYB")."""
    if u.perfil != "auditor":
        return None
    m = _ASIGNAR_BODEGA.search(texto.strip())
    quitar = False
    if not m:
        m = _QUITAR_BODEGA.search(texto.strip())
        quitar = True
    if not m:
        # OJO: el verbo debe terminar justo en "a/ale/ar" (con \b después)
        # para no confundirse con el SUSTANTIVO "asignación" - sin ese
        # límite, "muéstreme la asignación por bodega" (que es la orden
        # de abrir el panel, no de asignar nada) caía aquí por error,
        # porque "asign" es raíz común de las dos palabras.
        if (re.search(r"\bbodega\b", texto, re.IGNORECASE)
            and re.search(r"\bas[ií]gn(?:a(?:le)?|ar)\b\bqu[ií]t(?:a(?:le)?|ar)\b",
                texto, re.IGNORECASE)
            and not _ES_PREGUNTA_PESTANA.search(texto)):
            return {"respuesta_hablada": "Diga: «asigne/quítale la bodega» el nombre de "
                "la bodega «a» el nombre de la persona.",
                    "accion": None, "destino": None, "pestana": None}
        return None
    from servicios.conciliacion import buscar_bodega
    t_nombre = _normalizar_nombre(m.group("nombre"))
    with Sesion() as s:
        bodega = buscar_bodega(s, m.group("bodega"), None)
        if not bodega:
            return {"respuesta_hablada": f"No encontré una bodega llamada «{m.group('bodega')}».",
                    "accion": None, "destino": None, "pestana": None}
        candidatos = s.query(Usuario).filter(Usuario.id != u.id).all()
        persona = next(
            (c for c in candidatos
             if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t_nombre)),
            None)
        if not persona:
            return {"respuesta_hablada": "No encontré a esa persona. Diga el nombre completo "
                "tal como aparece en la lista.",
                    "accion": None, "destino": None, "pestana": None}
        actuales = {a.bodega_id for a in
            s.query(AsignacionBodega).filter_by(usuario_id=persona.id).all()}
        ya_tenia = bodega.id in actuales
        if quitar and not ya_tenia:
            return {"respuesta_hablada": f"{persona.nombre.capitalize()} no tenía asignada "
                f"{bodega.nombre_oficial.title()}.",
                    "accion": "actualizar", "destino": None, "pestana": None}
        if not quitar and ya_tenia:
            return {"respuesta_hablada": f"{persona.nombre.capitalize()} ya tenía asignada "
                f"{bodega.nombre_oficial.title()}.",
                    "accion": "actualizar", "destino": None, "pestana": None}
        if quitar:
            actuales.discard(bodega.id)
        else:

```



```

        actuales.add(bodega.id)
        pid, pnombre, bnombre = persona.id, persona.nombre, bodega.nombre_oficial
        nuevos_ids = list(actuales)
        asignar_bodegas(usuario_id=pid, body={"bodega_ids": nuevos_ids}, u=u)
        verbo = "quitó" if quitar else "asigné"
        return {"respuesta_hablada": f"Listo, le {verbo} {bnombre.title()} a {pnombre.capitalize()}.",
                "accion": "actualizar", "destino": None, "pestana": None}

_QUIEN_TIENE_BODEGA = re.compile(
    r"\bqui[eé]n\s+(?:tiene|es\s+responsable\s+de|maneja)\s+(?:la\s+)?bodega\s+"
    r"?(?P<bodega>.+?)\s*[!?!]*\s*$", re.IGNORECASE)
_BODEGAS_SIN_ASIGNAR = re.compile(r"\bbodegas?\b.*\bsin\s+asignar\b", re.IGNORECASE)

def _consultar_asignacion_bodega_por_voz(texto: str, u: Usuario) -> dict | None:
    """Lo mismo que el botón «Ver asignación por bodega», pero como
    pregunta hablada en vez de abrir la tabla completa de 54 filas:
    "¿quién tiene la bodega <bodega>?" contesta solo esa fila, y
    "¿cuántas bodegas sin asignar hay?" resume las que no tiene nadie -
    de solo lectura, así que no exige ninguna confirmación extra."""
    if u.perfil != "auditor":
        return None
    if _BODEGAS_SIN_ASIGNAR.search(texto):
        with Sesion() as s:
            asignadas = {a.bodega_id for a in s.query(AsignacionBodega).all()}
            sin = [b.nombre_oficial for b in s.query(Bodega).all() if b.id not in asignadas]
            if not sin:
                return {"respuesta_hablada": "Todas las bodegas tienen al menos una persona asignada.",
                        "accion": None, "destino": None, "pestana": None}
            listado = ", ".join(n.title() for n in sin[:8])
            extra = f" y {len(sin) - 8} más" if len(sin) > 8 else ""
            return {"respuesta_hablada": f"{len(sin)} bodegas sin asignar: {listado}{extra}.",
                    "accion": None, "destino": None, "pestana": None}
    m = _QUIEN_TIENE_BODEGA.search(texto.strip())
    if not m:
        return None
    from servicios.conciliacion import buscar_bodega
    with Sesion() as s:
        bodega = buscar_bodega(s, m.group("bodega"), None)
        if not bodega:
            return {"respuesta_hablada": f"No encontré una bodega llamada «{m.group('bodega')}».",
                    "accion": None, "destino": None, "pestana": None}
        asignados = [s.get(Usuario, a.usuario_id) for a in
                     s.query(AsignacionBodega).filter_by(bodega_id=bodega.id).all()]
        nombres = [p.nombre.capitalize() for p in asignados if p]
        titulo = bodega.nombre_oficial.title()
        if not nombres:
            return {"respuesta_hablada": f"{titulo} no tiene a nadie asignado todavía.",
                    "accion": None, "destino": None, "pestana": None}
        return {"respuesta_hablada": f"{titulo} está asignada a {'', '.join(nombres)}.",
                "accion": None, "destino": None, "pestana": None}

_ABRIR_COBERTURA = re.compile(
    r"\b(mu[eé]stra[w*]ens[eé][ñn]a[w*]|[á]bre[w*]|ver)\b.*\basignaci[oó]n\b.*\bbodega",
    re.IGNORECASE)
_CERRAR_COBERTURA = re.compile(
    r"\b(oc[úú]lta[w*]|ci[eé]rra[w*]|esc[oó]nde[w*])\b.*\basignaci[oó]n\b.*\bbodega",
    re.IGNORECASE)

def _alternar_cobertura_por_voz(texto: str, u: Usuario) -> dict | None:
    """El botón «Ver/Ocultar asignación por bodega» abre o cierra la

```

```

tabla completa de 54 filas (quién tiene cada bodega) - a diferencia
de _consultar_asignacion_bodega_por_voz (que contesta una pregunta
puntual sin tocar la pantalla), esto es la orden de abrir/cerrar el
panel mismo, igual que el clic al botón.""
if u.perfil != "auditor":
    return None
if _ABRIR_COBERTURA.search(texto):
    return {"respuesta_hablada": "Listo, ahí tiene la asignación por bodega.",
            "accion": "mostrar_cobertura", "destino": None, "pestaña": None}
if _CERRAR_COBERTURA.search(texto):
    return {"respuesta_hablada": "Listo, la oculto.",
            "accion": "ocultar_cobertura", "destino": None, "pestaña": None}
return None

_EXPORTAR_TRAZA = re.compile(
    r"\b(export\w*|descarg\w*|genera\w*)\b.*\b(traza|trazabilidad)\b", re.IGNORECASE)

def _exportar_trazabilidad_por_voz(texto: str, u: Usuario) -> dict | None:
    """El botón «Exportar» de Registro de trazabilidad exporta con los
    filtros (persona/acción/rango) que YA están marcados en pantalla -
    esos viven como estado de React en el frontend, no llegan en el
    texto de esta orden, así que no se puede armar el archivo aquí en
    el backend. En cambio, se avisa con la misma acción genérica que ya
    usa mostrar/ocultar cobertura, y el frontend llama a su propia
    función exportar() con los filtros que tenga puestos - igual que si
    hubiera dado clic al botón."""
    if u.perfil != "auditor":
        return None
    if not _EXPORTAR_TRAZA.search(texto):
        return None
    return {"respuesta_hablada": "Listo, exportando el registro de trazabilidad con los "
                                "filtros que tiene puestos.",
            "accion": "exportar_trazabilidad", "destino": None, "pestaña": None}

_ETIQUETA_RANGO_TRAZA = {"hoy": "hoy", "semana": "en la última semana",
                          "mes": "en el último mes", "": "en total"}

def _rango_dicho_traza(texto: str) -> str:
    t = texto.lower()
    if re.search(r"\bsemana\b", t):
        return "semana"
    if re.search(r"\bmese\b", t):
        return "mes"
    if re.search(r"\btodo\b|\bsiempre\b", t):
        return ""
    return "hoy" # mismo valor por defecto con el que abre la pestaña

_ACCIONES_DE_PERSONA = re.compile(
    r"\bcu[áá]ntas\s+acciones\s+tiene\s+(?P<persona>.+?)\s*[\.!?]*\s*$", re.IGNORECASE)
_ACCIONES_POR_PERSONA = re.compile(
    r"\bacciones\s+por\s+persona\b|\bcu[áá]ntas\s+acciones\s+tiene\s+cada\s+persona\b|"
    r"\bresumen\s+de\s+acciones\b", re.IGNORECASE)

def _acciones_por_persona_por_voz(texto: str, u: Usuario) -> dict | None:
    """Pedido explícito del usuario: "acciones por persona" contesta un
    resumen con cuántas hizo cada quien, y "¿cuántas acciones tiene

```

```

<nombre>?" contesta solo por esa persona - los dos de solo lectura,
sin abrir ni cambiar nada en pantalla, así que no hace falta
confirmación. El periodo por defecto es "hoy" (igual que la
pestaña al abrirse); decir "esta semana"/"este mes"/"en total"
lo cambia.""
if u.perfil != "auditor":
    return None
m = _ACCIONES_DE_PERSONA.search(texto.strip())
if m:
    rango = _rango_dicho_traza(texto)
    t_nombre = _normalizar_nombre(m.group("persona"))
    with Sesion() as s:
        candidatos = s.query(Usuario).all()
        persona_obj = next(
            (c for c in candidatos
             if re.search(rf"\b{re.escape(_normalizar_nombre(c.nombre))}\b", t_nombre)),
            None)
        if not persona_obj:
            return None
        total = _filtro_traza(s, persona_obj.nombre, "", rango).count()
        nombre = persona_obj.nombre
        return {"respuesta_hablada": f"{nombre.capitalize()} tiene {total} acciones "
                                     f"{_ETIQUETA_RANGO_TRAZA[rango]}.",
                "accion": None, "destino": None, "pestaña": None}
if _ACCIONES_POR_PERSONA.search(texto):
    rango = _rango_dicho_traza(texto)
    with Sesion() as s:
        filas = _filtro_traza(s, "", "", rango).all()
    if not filas:
        return {"respuesta_hablada": f"No hay acciones registradas
_{ETIQUETA_RANGO_TRAZA[rango]}.",
                "accion": None, "destino": None, "pestaña": None}
    conteos: dict[str, int] = {}
    for f in filas:
        conteos[f.persona] = conteos.get(f.persona, 0) + 1
    ordenado = sorted(conteos.items(), key=lambda par: -par[1])
    resumen = ", ".join(f"{p.capitalize():<n}" for p, n in ordenado[:8])
    extra = f", y {len(ordenado) - 8} personas más" if len(ordenado) > 8 else ""
    return {"respuesta_hablada": f"Acciones por persona {_ETIQUETA_RANGO_TRAZA[rango]}: "
                                f"{resumen}{extra}.",
            "accion": None, "destino": None, "pestaña": None}
return None

def _resolver_asistente(vista: str, texto: str, u: Usuario) -> dict:
    """Lo que responde el agente liviano ante una pregunta suelta o una
orden de navegar - usado tanto por /api/agente/asistente (Inicio,
Ajustes, Ayuda, Reportes, Panel) como, cuando ni un artículo ni una
bodega coinciden, por el buscador de Bodegas (ver consulta_articulo):
un solo buscador que primero prueba si es un ingrediente, luego si
es una bodega, y si no es ninguna de las dos, cae aquí."""
    texto = _quitar_relleno(texto)
    if vista == "inicio":
        acceso = _acceso_rapido_inicio_por_voz(texto, u)
        if acceso:
            return acceso
    # Desde Bodegas, "abra/siga contando <nombre>" es una orden concreta y
    # frecuente - se resuelve aparte, ANTES de Gemini, con la misma
    # búsqueda restringida que usa el resto de la app (nunca ofrece una
    # bodega ajena a lo asignado). El verbo es obligatorio: sin él, decir
    # el nombre de una bodega podría ser una PREGUNTA sobre ella ("¿cómo
    # va kiosco taquilla ayb?"), no una orden de ir a contarla.
    if vista == "bodegas" and _VERBOS_ABRIR_BODEGA.search(texto):
        from servicios.conciliacion import buscar_bodega
        with Sesion() as s:
            bodega = buscar_bodega(s, texto, _ids_permitidos_para_buscar(s, u))
        if bodega:

```

```

        return {"respuesta_hablada": f"Vamos a Conteo a abrir {bodega.nombre_oficial.title()}.",
                "accion": "navegar", "destino": "conteo", "pestaña": None,
                "bodega": bodega.nombre_oficial}
if vista == "ajustes":
    # identidad (no solo presencia) de lo que había pendiente ANTES
    # de este turno - crear_receta/agregar_ingredientes pueden dejar
    # una ambigüedad NUEVA en el mismo turno (cambio ya sale
    # "truthy" con el mensaje de "es ambiguo...") y esa no se debe
    # borrar; solo se limpia la que YA estaba ahí sin que nada de
    # este turno la haya tocado.
    pendiente_antes = _PENDIENTE_AMBIGUEDAD.get(u.id)
    cambio = (_cambiar_modos_sin_conexion(texto, u) or _cambiar_estado_usuario(texto, u)
              or _crear_usuario_por_voz(texto, u) or _cambiar_perfil_por_voz(texto, u)
              or _asignar_bodega_por_voz(texto, u) or _alternar_cobertura_por_voz(texto, u)
              or _consultar_asignacion_bodega_por_voz(texto, u)
              or _crear_receta_por_voz(texto, u)
              or _agregar_ingredientes_por_voz(texto, u) or _quitar_ingredientes_por_voz(texto, u)
              or _cambiar_rendimiento_por_voz(texto, u) or _agregar_preparacion_por_voz(texto, u)
              or _eliminar_receta_por_voz(texto, u) or _exportar_trazabilidad_por_voz(texto, u)
              or _acciones_por_persona_por_voz(texto, u))
    if cambio:
        # una orden distinta que sí coincidió significa que la
        # persona siguió para otra cosa - una pregunta ambigua sin
        # contestar de hace un rato no debe "resucitar" más tarde
        # por una frase corta que por casualidad se parezca a una
        # de esas opciones viejas.
        if _PENDIENTE_AMBIGUEDAD.get(u.id) is pendiente_antes:
            _PENDIENTE_AMBIGUEDAD.pop(u.id, None)
            return cambio
        resuelta_pendiente = _resolver_ambigüedad_pendiente(texto, u)
        if resuelta_pendiente:
            return resuelta_pendiente
if vista == "reportes":
    generado = _generar_reporte_por_voz(texto, u)
    if generado:
        return generado
    previsualizado = _previsualizar_reporte_por_voz(texto, u)
    if previsualizado:
        return previsualizado
    respondida = _responder_reportes_por_voz(texto, u)
    if respondida:
        return respondida
if vista == "auditoria":
    resuelta = _resolver_aprobacion_por_voz(texto, u)
    if resuelta:
        return resuelta
    resuelto_pedido = _resolver_pedido_pendiente_por_voz(texto, u)
    if resuelto_pedido:
        return resuelto_pedido
    resuelta_alerta = _resolver_alerta_por_voz(texto, u)
    if resuelta_alerta:
        return resuelta_alerta
    auditada = _abrir_bodega_para_auditoria_por_voz(texto, u)
    if auditada:
        return auditada
    respondida = _responder_auditoria_por_voz(texto, u)
    if respondida:
        return respondida
if vista == "panel":
    respondida = _responder_panel_por_voz(texto, u)
    if respondida:
        return respondida
if vista == "perfil":
    cambio = (_cambiar_voz_por_voz(texto, u) or _cambiar_velocidad_por_voz(texto, u)
              or _cambiar_confirmacion_hablada_por_voz(texto, u))
    if cambio:
        return cambio

```

```

    respondida = _responder_perfil_por_voz(texto, u)
    if respondida:
        return respondida
    navegada = _navegar_pestana_por_voz(vista, texto)
    if navegada:
        return navegada
    destinos = dict(_DESTINOS_ASISTENTE)
    if u.perfil != "auditor":
        for k in _SOLO_AUDITOR_ASISTENTE:
            destinos.pop(k, None)
    # Sin esto, Gemini podía "sugerir" ir a la pantalla en la que la
    # persona YA esta (confirmado: ofrecio "llevarla a Gestion de
    # usuarios" estando ella ahi mismo) - una orden real de cambiar de
    # PESTAÑA dentro de la misma pantalla ya se resolvió antes, arriba
    # (_navegar_pestana_por_voz), así que si el mensaje llega hasta aquí
    # nunca es eso.
    destinos.pop(vista, None)
    contexto = _contexto_asistente(vista, u)
    from agente.cerebro import pensar_asistente
    turno = pensar_asistente(contexto, texto, destinos, _PESTANAS_ASISTENTE)
    destino = turno.get("destino")
    if (turno.get("intencion") or "").lower() == "navegar" and destino in destinos:
        pestana = turno.get("pestana")
        pestanas_validas = _PESTANAS_ASISTENTE.get(destino, {})
        if pestana not in pestanas_validas:
            pestana = None
        return {"respuesta_hablada": turno.get("respuesta_hablada", ""),
                "accion": "navegar", "destino": destino, "pestana": pestana}
    return {"respuesta_hablada": turno.get("respuesta_hablada", ""),
            "accion": None, "destino": None, "pestana": None}

@app.post("/api/agente/asistente")
def api_asistente(p: PreguntarAsistenteIn, u: Usuario = Depends(usuario_actual)):
    """Version liviana del agente para pantallas que no cuentan ni piden
    (Inicio, Ajustes, Ayuda, Reportes, Panel): responde preguntas sobre lo
    que hay en esa pantalla y navega a otra si se lo piden. Sin el estado
    multi-turno de /api/agente/turno, que es específico del conteo/pedido."""
    return _resolver_asistente(p.vista, p.texto, u)

@app.get("/api/sesiones/{sesion_id}/avance")
def ver_avance(sesion_id: int, bodega_id_respaldo: int | None = None,
               u: Usuario = Depends(usuario_actual)):
    est = ESTADOS.setdefault(sesion_id, {})
    # mismo respaldo que en /agente/turno: si el backend se reinicio, el
    # frontend todavía sabe en que bodega estaba y este número deja de
    # verse roto ("avance: 4 de 0") en vez de exigir que se reabra.
    if bodega_id_respaldo and not est.get("bodega_id"):
        est["bodega_id"] = bodega_id_respaldo
    with Sesion() as s:
        hechas = s.query(Conteo).filter_by(sesion_id=sesion_id,
                                           estado="confirmado").count()
        alertas = s.query(Alerta).filter_by(resuelta=0).count()
        total = (s.query(StockSistema).filter_by(bodega_id=est.get("bodega_id"))).count()
        if est.get("bodega_id") else 0
        ultimos = (s.query(Conteo).filter_by(sesion_id=sesion_id, estado="confirmado")
                  .order_by(Conteo.id.desc()).limit(5).all())
        detalle = []
        for c in ultimos:
            a = s.get(Articulo, c.articulo_codigo)
            detalle.append({"nombre": a.nombre_oficial if a else c.articulo_codigo,
                           "cantidad": c.cantidad, "unidad": c.unidad})
    return {"hechas": hechas, "total": total, "alertas": alertas,
            "bodega": est.get("bodega_nombre"), "ultimos": detalle}

```

```

def _limpiar_nombre_dictado(t: str) -> str:
    """El reconocimiento de voz del navegador cierra la frase con
    puntuación que nadie dijo ("Juguetería."). Ya se le quita para
    BUSCAR una bodega (servicios.conciliacion.normalizar); esto hace lo
    mismo pero para GUARDAR un nombre nuevo - sin tocar tildes ni
    mayúsculas, que sí importan en lo que queda escrito en el catálogo."""
    import re
    return re.sub(r"[.,;:!?;]+\\s*$", "", t.strip()).strip()

class CrearProductoIn(BaseModel):
    nombre: str
    unidad_medida: str
    cantidad_inicial: float
    sesion_id: int = 1

@app.post("/api/conteo/crear-producto")
def crear_producto_pendiente(p: CrearProductoIn, u: Usuario = Depends(usuario_actual)):
    """El conteo no se detiene: el producto entra pendiente y sigue contando.
    Si quien cuenta es el administrador, el producto se confirma de una -
    igual que en crear_bodega_pendiente, dejarlo pendiente significaría que
    solo el mismo administrador puede aprobarlo despues."""
    est = ESTADOS.get(p.sesion_id, {})
    bodega_id = est.get("bodega_id")
    if not bodega_id:
        raise HTTPException(409, "Abra una bodega antes de crear un producto.")
    if p.cantidad_inicial < 0:
        raise HTTPException(400, "La cantidad inicial no puede ser negativa.")
    nombre = _limpiar_nombre_dictado(p.nombre).upper()
    codigo = f"PEND-{ahora().strftime('%H%M%S%f')[:10]}"
    es_auditor = u.perfil == "auditor"
    with Sesion() as s:
        s.add(Articulo(codigo=codigo, nombre_oficial=nombre,
                       unidad_medida=p.unidad_medida))
        s.commit()
        conteo = Conteo(sesion_id=p.sesion_id, articulo_codigo=codigo,
                        cantidad=p.cantidad_inicial, unidad=p.unidad_medida,
                        estado="confirmado" if es_auditor else "pendiente_aprobacion")
        s.add(conteo)
        s.commit()
        s.refresh(conteo)
        if es_auditor:
            s.add(StockSistema(articulo_codigo=codigo, bodega_id=bodega_id, cantidad_sd=0))
        else:
            s.add(Aprobacion(tipo="producto", nombre=nombre,
                             unidad_medida=p.unidad_medida, cantidad_inicial=p.cantidad_inicial,
                             bodega_id=bodega_id, articulo_codigo=codigo,
                             conteo_id=conteo.id, creado_por_id=u.id))
            s.commit()
    if es_auditor:
        registrar(u, "CREACION", f"{nombre} creado")
        return {"ok": True, "codigo": codigo,
                "respuesta_hablada": "Creado y confirmado en el catálogo."}
    registrar(u, "CREACION", f"{nombre} creado, pendiente de aprobación")
    return {"ok": True, "codigo": codigo,
            "respuesta_hablada": "Creado. Queda pendiente de aprobación del administrador; "
                                "el conteo sigue sin interrupción."}

# ----- bodegas -----
class AbrirIn(BaseModel):

```

```

bodega: str

def _contexto_bodega(s, b, refs):
    """Segun el estado, lo que hace falta para entender de un vistazo quien
    esta en que y cuanto lleva - se usa tanto en el tablero completo como
    en «mis bodegas asignadas» de Conteo."""
    item = {}
    if b.estado == "en_conteo":
        ses = (s.query(SesionConteo)
               .filter_by(bodega_id=b.id, tipo="conteo", estado="abierta")
               .order_by(SesionConteo.id.desc()).first())
        if ses:
            persona = s.get(Usuario, ses.usuario_id)
            hechas = s.query(Conteo).filter_by(
                sesion_id=ses.id, estado="confirmado").count()
            item["persona"] = persona.nombre if persona else None
            item["avance_pct"] = round(hechas / refs * 100) if refs else 0
    elif b.estado == "en_auditoria":
        ses = (s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="auditoria")
               .order_by(SesionConteo.id.desc()).first())
        if ses:
            persona = s.get(Usuario, ses.usuario_id)
            item["persona"] = persona.nombre if persona else None
        conteo_ses = (s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo")
                      .order_by(SesionConteo.id.desc()).first())
        difs = 0
        if conteo_ses:
            for c in (s.query(Conteo).filter_by(
                sesion_id=conteo_ses.id, estado="confirmado").all()):
                st = s.query(StockSistema).filter_by(
                    articulo_codigo=c.articulo_codigo, bodega_id=b.id).first()
                if abs((c.cantidad or 0) - (st.cantidad_sd if st else 0)) > 0.001:
                    difs += 1
            item["diferencias"] = difs
    elif b.estado == "cerrada":
        hist = (s.query(HistorialCierre).filter_by(bodega_id=b.id)
                .order_by(HistorialCierre.fecha.desc()).first())
        if hist:
            item["hora_cierre"] = hist.fecha.strftime("%H:%M")
    return item

@app.get("/api/bodegas")
def listar_bodegas(propias: int = 0, u: Usuario = Depends(usuario_actual)):
    """El tablero en vivo: cada tarjeta necesita un dato distinto segun su
    estado (quien cuenta y cuanto lleva, quien audita y cuantas diferencias
    encontro, o a que hora se cerro) - no solo el total de referencias.

    ?propias=1 limita el tablero a las bodegas asignadas a quien pregunta -
    para un auditor/administrador. Si la persona no tiene ninguna bodega
    asignada, ve todas (administrador general, sin zona propia); si tiene
    algunas asignadas, ve solo esas (un supervisor de zona con varios
    administradores repartiendo el parque). El resto de pantallas
    (Pedidos elige donde descontar, Ajustes arma la lista para asignar)
    siguen pidiendo la lista completa sin este parametro - pantallas que
    solo un auditor/administrador alcanza a abrir.

    Un auxiliar, en cambio, nunca ve bodegas fuera de su asignacion: para
    ese perfil la restriccion aplica siempre, con o sin ?propias=1, para
    que ninguna pantalla (incluida Pedidos) filtre por accidente el parque
    completo a quien solo debe ver su zona."""
    with Sesion() as s:
        ids_asignadas = {a.bodega_id for a in
                         s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}

```

```

restringir = propias or u.perfil == "auxiliar"
if restringir and ids_asignadas:
    bodegas = (s.query(Bodega).filter(Bodega.id.in_(ids_asignadas))
               .order_by(Bodega.nombre_oficial).all())
else:
    bodegas = s.query(Bodega).order_by(Bodega.nombre_oficial).all()
salida = []
for b in bodegas:
    refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
    item = {"id": b.id, "bodega": b.nombre_oficial,
            "estado": b.estado, "referencias": refs}
    item.update(_contexto_bodega(s, b, refs))
    salida.append(item)
return salida

@app.post("/api/bodegas/buscar")
def buscar_bodega_endpoint(a: AbrirIn, u: Usuario = Depends(usuario_actual)):
    """Solo resuelve el nombre, no abre nada - para que el selector de
    bodega pueda preguntar "¿confirma que abro X?" con el nombre REAL que
    de verdad se va a abrir, en vez de repetir a ciegas lo que se
    reconoció (decir «kiosco» no debe confirmarse como si "KIOSCO" fuera
    una bodega real, cuando lo que existe es "KIOSCO TAQUILLA AYB").

    Si hay varias candidatas igual de razonables ("restaurante" a secas
    encaja en "RESTAURANTE FUENTES AYB" Y "RESTAURANTE FUENTES SUMIN"),
    se devuelven como opciones en vez de decir "no la encuentro" a secas
    - así la persona puede elegir la que quería, no repetir a ciegas."""
    from servicios.conciliacion import buscar_bodegas_candidatas
    with Sesion() as s:
        ids_permitidos = _ids_permitidos_para_buscar(s, u)
        candidatas = buscar_bodegas_candidatas(s, a.bodega, ids_permitidos)
        if len(candidatas) == 1:
            b = candidatas[0]
            return {"encontrada": True, "bodega": b.nombre_oficial, "bodega_id": b.id,
                    "opciones": []}
        opciones = [{"bodega": c.nombre_oficial, "bodega_id": c.id} for c in candidatas[:6]]
        return {"encontrada": False, "bodega": None, "bodega_id": None, "opciones": opciones}

@app.post("/api/bodegas/abrir")
async def abrir(a: AbrirIn, u: Usuario = Depends(usuario_actual)):
    from servicios.conciliacion import buscar_bodega
    with Sesion() as s:
        # buscar_bodega() (misma funcion que usa el agente conversacional):
        # quita tildes y puntuación de sobra (el reconocimiento de voz a
        # veces cierra la frase con un "." que nadie dijo) y prioriza la
        # coincidencia exacta - sin esto, "Zoológico" podía no encontrarse
        # por el punto final, o abrir "ZOOLOGICO SUMINISTROS" en vez de la
        # bodega "ZOOLOGICO" que de verdad se pidió. Restringida a lo
        # asignado si es auxiliar: no debe ni encontrar, ni ofrecer,
        # bodegas de otra zona que igual no podría abrir.
        b = buscar_bodega(s, a.bodega, _ids_permitidos_para_buscar(s, u))
        if b is None:
            raise HTTPException(404, "No encuentro esa bodega.")
        if u.perfil == "auxiliar" and not s.query(AsignacionBodega).filter_by(
            usuario_id=u.id, bodega_id=b.id).first():
            raise HTTPException(403, "Esa bodega no esta asignada a usted.")
        abierta = s.query(SesionConteo).filter_by(bodega_id=b.id, tipo="conteo",
            estado="abierta").first()
        if abierta and abierta.usuario_id != u.id:
            raise HTTPException(409, "Esa bodega ya esta en conteo por otra persona.")
        ses = abierta or SesionConteo(bodega_id=b.id, usuario_id=u.id)
        if not abierta:
            s.add(ses)

```



```

        b.estado = "en_conteo"
        s.commit()
        s.refresh(ses)
        refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
        sid, nombre, bid = ses.id, b.nombre_oficial, b.id
        ESTADOS.setdefault(sid, {}).update(
            {"bodega_id": bid, "bodega_nombre": nombre, "sesion_bd": sid})
        registrar(u, "APERTURA", f"{nombre} abierta - sesion bloqueada")
        await difundir_estado()
        return {"sesion_id": sid, "bodega": nombre, "referencias": refs, "bodega_id": bid}

class CrearBodegaIn(BaseModel):
    nombre: str

@app.post("/api/bodegas/crear-pendiente")
def crear_bodega_pendiente(p: CrearBodegaIn, u: Usuario = Depends(usuario_actual)):
    """La bodega no se crea de una para un auxiliar: queda pendiente de
    aprobacion del administrador (Aprobacion tipo "bodega", ya soportada
    en aprobar()/rechazar()) - asi el catalogo no crece con lo que
    cualquiera escriba sin control. Un administrador ya tiene esa
    autoridad, asi que para el la bodega se crea de inmediato - de lo
    contrario terminaria aprobando su propia solicitud, lo cual no
    verifica nada (visto en produccion: Diana pidio "ALIMENTOS" y quedo
    esperando su propia firma en su propia bandeja)."""
    nombre = _limpiar_nombre_dictado(p.nombre).upper()
    if not nombre:
        raise HTTPException(400, "Dígame el nombre de la bodega.")
    with Sesion() as s:
        if s.query(Bodega).filter_by(nombre_oficial=nombre).first():
            raise HTTPException(409, "Ya existe una bodega con ese nombre.")
        if u.perfil == "auditor":
            s.add(Bodega(nombre_oficial=nombre, estado="pendiente"))
            s.commit()
            registrar(u, "CREACION", f"Bodega {nombre} creada")
            return {"ok": True,
                    "respuesta_hablada": f"{nombre.capitalize()} creada. Ya está en el catálogo."}
        s.add(Aprobacion(tipo="bodega", nombre=nombre, creado_por_id=u.id))
        s.commit()
        registrar(u, "CREACION", f"Bodega {nombre} solicitada, pendiente de aprobacion")
        return {"ok": True,
                "respuesta_hablada": f"Solicitud de {nombre.lower()} enviada. Queda "
                                     "pendiente de aprobación del administrador."}

def _ultima_sesion(s, bodega_id: int, tipo: str):
    return (s.query(SesionConteo).filter_by(bodega_id=bodega_id, tipo=tipo)
            .order_by(SesionConteo.id.desc()).first())

def _requiere_acceso_bodega(s, u: Usuario, bodega_id: int):
    """El tablero (listar_bodegas) ya oculta las bodegas ajenas a un
    auxiliar, pero eso no basta: sin este chequeo, pedir el detalle,
    las firmas o el inventario completo por su bodega_id directamente
    (sin pasar por el tablero) dejaba ver auditoria y stock de cualquier
    zona con solo cambiar el numero en la URL."""
    if u.perfil == "auxiliar" and not s.query(AsignacionBodega).filter_by(
        usuario_id=u.id, bodega_id=bodega_id).first():
        raise HTTPException(403, "Esa bodega no esta asignada a usted.")

def _ids_permitidos_para_buscar(s, u: Usuario) -> set[int] | None:

```

```

"""Para restringir buscar_bodega()/buscar_bodegas_candidatas() a lo
que un auxiliar de verdad puede abrir - sin esto, decir "restaurante"
ofrecia como opciones bodegas de zonas ajenas que ni siquiera podia
llegar a abrir, puro ruido (y un nombre que no tenia por qué ver).
None para un auditor: puede buscar en todo el catálogo."""
if u.perfil != "auxiliar":
    return None
return {a.bodega_id for a in
        s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()}

@app.get("/api/bodegas/{bodega_id}/firmas")
def ver_firmas(bodega_id: int, u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        _requiere_acceso_bodega(s, u, bodega_id)
        b = s.get(Bodega, bodega_id)
        conteo = _ultima_sesion(s, bodega_id, "conteo")
        auditoria = _ultima_sesion(s, bodega_id, "auditoria")

    def _lado(ses):
        if ses is None:
            return None
        au = s.get(Usuario, ses.usuario_id)
        return {"sesion_id": ses.id, "persona": au.nombre if au else "?",
                "firmada": bool(ses.firmada),
                "hora": ses.fin.strftime("%H:%M") if ses.fin else None,
                "huella": bool(au and au.huella_registrada)}

    referencias = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
    alertas_resueltas = (s.query(Alerta).join(Conteo, Alerta.conteo_id == Conteo.id)
                        .join(SesionConteo, Conteo.sesion_id == SesionConteo.id)
                        .filter(SesionConteo.bodega_id == bodega_id,
                                Alerta.resuelta == 1).count())
    return {"bodega": b.nombre_oficial if b else None, "referencias": referencias,
            "alertas_resueltas": alertas_resueltas,
            "conteo": _lado(conteo), "auditoria": _lado(auditoria),
            "lista_para_cerrar": bool(conteo and conteo.firmada
                                       and auditoria and auditoria.firmada)}

@app.post("/api/sesiones/{sesion_id}/firmar")
def firmar_sesion(sesion_id: int, u: Usuario = Depends(usuario_actual)):
    """El auxiliar firma su conteo; el administrador firma su recuento ciego."""
    with Sesion() as s:
        ses = s.get(SesionConteo, sesion_id)
        if ses is None:
            raise HTTPException(404, "Sesion no encontrada.")
        if ses.usuario_id != u.id and u.perfil != "auditor":
            raise HTTPException(403, "Solo quien contó puede firmar este conteo.")
        ses.firmada = 1
        ses.fin = ahora()
        b = s.get(Bodega, ses.bodega_id)
        if ses.tipo == "conteo" and b and b.estado == "en_conteo":
            b.estado = "en_auditoria"
        s.commit()
        nombre = b.nombre_oficial if b else "?"
        tipo = ses.tipo
        etiqueta = "Conteo" if tipo == "conteo" else "Auditoria"
        registrar(u, "FIRMA", f"{etiqueta} firmado - {nombre}", "ok")
    return {"ok": True}

@app.post("/api/bodegas/{bodega_id}/auditoria/iniciar")
async def iniciar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):

```

```

with Sesion() as s:
    b = s.get(Bodega, bodega_id)
    if b is None:
        raise HTTPException(404, "Bodega no encontrada.")
    existente = s.query(SesionConteo).filter_by(
        bodega_id=bodega_id, tipo="auditoria", estado="abierta").first()
    ses = existente or SesionConteo(bodega_id=bodega_id, usuario_id=u.id, tipo="auditoria")
    if not existente:
        s.add(ses)
        s.commit()
        s.refresh(ses)
    refs = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
    sid, nombre = ses.id, b.nombre_oficial
    ESTADOS.setdefault(sid, {}).update({"bodega_id": bodega_id, "bodega_nombre": nombre})
    registrar(u, "AUDITORIA", f"Recuento ciego iniciado en {nombre}")
    await difundir_estado()
    return {"sesion_id": sid, "bodega": nombre, "referencias": refs}

@app.get("/api/bodegas/{bodega_id}/auditoria/comparar")
def comparar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        b = s.get(Bodega, bodega_id)
        ses_conteo = _ultima_sesion(s, bodega_id, "conteo")
        ses_audit = _ultima_sesion(s, bodega_id, "auditoria")

    def _mapa(ses):
        m = {}
        if not ses:
            return m
        for c in (s.query(Conteo).filter_by(sesion_id=ses.id, estado="confirmado")
                  .order_by(Conteo.id).all()):
            m[c.articulo_codigo] = c.cantidad
        return m
    m1, m2 = _mapa(ses_conteo), _mapa(ses_audit)
    codigos = set(m1) | set(m2)
    filas = []
    diagnosticos = []
    for cod in codigos:
        art = s.get(Articulo, cod)
        nombre = art.nombre_oficial if art else cod
        st = s.query(StockSistema).filter_by(articulo_codigo=cod, bodega_id=bodega_id).first()
        sistema = st.cantidad_sd if st else 0
        c1, c2 = m1.get(cod), m2.get(cod)
        autoridad = c2 if c2 is not None else c1
        dif = round((autoridad or 0) - sistema, 3)
        if abs(dif) < 0.01 and (c1 == c2 or c2 is None):
            continue
        filas.append({"codigo": cod, "articulo": nombre,
                     "conteo1": c1, "conteo2": c2, "sistema": sistema,
                     "diferencia": dif,
                     "accion": "Revisar" if (c1 is not None and c2 is not None and c1 != c2)
                               else "Aceptar"})
        # el sistema ya traia un saldo negativo antes de esta toma (una
        # salida sin su entrada correspondiente): no es un error del
        # conteo físico, es un dato roto que My Inventory debe corregir.
        if sistema < 0:
            diagnosticos.append(
                f"El {dif:+g} de {nombre.lower()} no viene de su conteo: el sistema "
                f"ya traía saldo negativo ({sistema:g}, salida registrada sin entrada). "
                "Su conteo físico es el que vale; el reporte lo marca como negativo "
                "del sistema para corrección en My Inventory.")
    filas.sort(key=lambda f: -abs(f["diferencia"]))
    return {"bodega": b.nombre_oficial if b else None,
            "filas": filas, "coinciden": len(codigos) - len(filas) if codigos else 0,
            "total": len(codigos), "diagnosticos": diagnosticos}

```

```

@app.post("/api/bodegas/{bodega_id}/auditoria/firmar")
def firmar_auditoria(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        ses = _ultima_sesion(s, bodega_id, "auditoria")
        if ses is None:
            raise HTTPException(404, "No hay un recuento de auditoria iniciado.")
        ses.firmada = 1
        ses.fin = ahora()
        s.commit()
        b = s.get(Bodega, bodega_id)
        nombre = b.nombre_oficial if b else ""
    registrar(u, "FIRMA", f"Auditoria firmada - {nombre}", "ok")
    return {"ok": True}

@app.post("/api/bodegas/{bodega_id}/cerrar")
async def cerrar(bodega_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        b = s.get(Bodega, bodega_id)
        if b is None:
            raise HTTPException(404, "Bodega no encontrada.")
        ses_conteo = _ultima_sesion(s, bodega_id, "conteo")
        ses_audit = _ultima_sesion(s, bodega_id, "auditoria")
        if not (ses_conteo and ses_conteo.firmada and ses_audit and ses_audit.firmada):
            raise HTTPException(409, "Faltan firmas: el conteo y la auditoria deben "
                                     "estar firmados antes de cerrar.")
        conteos = (s.query(Conteo).filter_by(sesion_id=ses_conteo.id, estado="confirmado").all())
        difs = 0
        for c in conteos:
            st = s.query(StockSistema).filter_by(
                articulo_codigo=c.articulo_codigo, bodega_id=bodega_id).first()
            if abs((c.cantidad or 0) - (st.cantidad_sd if st else 0)) > 0.001:
                difs += 1
        exact = round((len(conteos) - difs) / len(conteos) * 100, 1) if conteos else 100.0
        b.estado = "cerrada"
        for ses in s.query(SesionConteo).filter_by(bodega_id=bodega_id,
                                                    estado="abierta").all():
            ses.estado = "cerrada"
            if not ses.fin:
                ses.fin = ahora()
        s.add(HistorialCierre(bodega_id=bodega_id, exactitud=exact,
                             referencias=len(conteos), diferencias=difs))
        s.commit()
        nombre = b.nombre_oficial
    registrar(u, "CIERRE", f"{nombre} cerrada con doble firma", "ok")
    await difundir_estado()
    return {"ok": True, "bodega": nombre}

@app.post("/api/bodegas/{bodega_id}/reabrir")
async def reabrir_bodega(bodega_id: int, body: dict,
                        u: Usuario = Depends(requiere_perfil("auditor"))):
    motivo = (body.get("motivo") or "").strip()
    if not motivo:
        raise HTTPException(400, "Reabrir una bodega cerrada exige una justificación escrita.")
    with Sesion() as s:
        b = s.get(Bodega, bodega_id)
        if b is None:
            raise HTTPException(404, "Bodega no encontrada.")
        if b.estado != "cerrada":
            raise HTTPException(409, "Solo se reabre una bodega que esté cerrada.")
        # las sesiones firmadas quedan como historial (nada se borra), pero
        # se marcan como reemplazadas para que abrir() arme una ronda de

```

```

        # conteo realmente nueva: reabrir sin esto no devolvía nada útil al
        # auxiliar, porque "abrir" seguía encontrando su sesión ya firmada.
        for ses in s.query(SesionConteo).filter_by(bodega_id=bodega_id, estado="abierta").all():
            ses.estado = "reemplazada"
        b.estado = "en_conteo"
        s.commit()
        nombre = b.nombre_oficial
        registrar(u, "REAPERTURA", f"{nombre} reabierta - motivo: {motivo}", "alerta")
        await difundir_estado()
        return {"ok": True, "bodega": nombre}

def _narrar_hito(t):
    p = (t.persona or "alguien").title()
    if t.accion == "APERTURA":
        return f"{p} abre la bodega"
    if t.accion == "AUDITORIA":
        return f"{p} inicia el recuento ciego"
    if t.accion == "CORRECCION":
        return "Alerta: " + t.detalle.split(" (valor anterior)")[0]
    if t.accion == "FIRMA" and "Conteo firmado" in t.detalle:
        return f"{p} cierra el conteo"
    if t.accion == "FIRMA" and "Auditoria firmada" in t.detalle:
        return f"{p} firma el recuento"
    if t.accion == "CIERRE":
        return "Cierre con doble firma"
    if t.accion == "REAPERTURA":
        return f"{p}: {t.detalle}"
    return t.detalle

def _color_hito(t):
    if t.accion == "CIERRE":
        return "verde"
    if t.accion in ("CORRECCION", "REAPERTURA"):
        return "oro"
    return "azul"

@app.get("/api/bodegas/{bodega_id}/detalle")
def detalle_bodega(bodega_id: int, u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        _requiere_acceso_bodega(s, u, bodega_id)
        b = s.get(Bodega, bodega_id)
        if b is None:
            raise HTTPException(404, "Bodega no encontrada.")
        total = s.query(StockSistema).filter_by(bodega_id=bodega_id).count()
        # solo la ronda de conteo más reciente: tras un reabrir(), la sesión
        # anterior queda como historial y no debe mezclarse con el recuento
        # nuevo en la exactitud/diferencias que se muestran aquí.
        ses_conteo_actual = _ultima_sesion(s, bodega_id, "conteo")
        conteos = (s.query(Conteo).filter_by(
            sesion_id=ses_conteo_actual.id, estado="confirmado").all()
            if ses_conteo_actual else [])
        difs = []
        for c in conteos:
            st = s.query(StockSistema).filter_by(
                articulo_codigo=c.articulo_codigo, bodega_id=bodega_id).first()
            esperado = st.cantidad_sd if st else 0
            if abs((c.cantidad or 0) - esperado) > 0.001:
                a = s.get(Articulo, c.articulo_codigo)
                difs.append({"articulo": a.nombre_oficial if a else c.articulo_codigo,
                    "contado": c.cantidad, "sistema": esperado,
                    "diferencia": round(c.cantidad - esperado, 3)})

```

```

sesiones = s.query(SesionConteo).filter_by(bodega_id=bodega_id).all()
usuario_ids = {ses.usuario_id for ses in sesiones if ses.usuario_id}
personas_nombres = []
for uid in usuario_ids:
    us = s.get(Usuario, uid)
    if us and us.nombre.title() not in personas_nombres:
        personas_nombres.append(us.nombre.title())
if len(personas_nombres) <= 1:
    personas = personas_nombres[0] if personas_nombres else None
else:
    personas = ", ".join(personas_nombres[:-1]) + " y " + personas_nombres[-1]

# hitos: los que ya mencionan la bodega por nombre (apertura/cierre/etc)
# mas las correcciones de la persona en el rango de su sesion (esas
# no mencionan la bodega, solo el articulo).
hitos_t = list(s.query(Traza).filter(
    Traza.detalle.contains(b.nombre_oficial)).all())
vistos = {t.id for t in hitos_t}
for ses in sesiones:
    if not ses.usuario_id:
        continue
    fin = ses.fin or ahora()
    correcciones = (s.query(Traza)
        .filter(Traza.accion == "CORRECCION",
            Traza.usuario_id == ses.usuario_id,
            Traza.creado >= ses.inicio, Traza.creado <= fin)
        .all())
    for t in correcciones:
        if t.id not in vistos:
            hitos_t.append(t)
            vistos.add(t.id)
    hitos_t.sort(key=lambda t: t.creado)
    hitos = [{ "hora": t.creado.strftime("%H:%M"), "texto": _narrar_hito(t),
        "tipo": _color_hito(t)} for t in hitos_t]

alertas_resueltas = (s.query(Alerta).join(Conteo, Alerta.conteo_id == Conteo.id)
    .join(SesionConteo, Conteo.sesion_id == SesionConteo.id)
    .filter(SesionConteo.bodega_id == bodega_id,
        Alerta.resuelta == 1).count())

exact = round((len(conteos) - len(difs)) / len(conteos) * 100, 1) if conteos else 0
duracion_min = None
if ses_conteo_actual and ses_conteo_actual.fin:
    duracion_min = round((ses_conteo_actual.fin - ses_conteo_actual.inicio).total_seconds() /
60)

# referencia fija de un conteo manual en papel (no hay dato real de
# esto: es un supuesto declarado, no una medicion de Colsubsidio)
tiempo_papel_min = round(total * 20 / 60) if total else None

unidades_stock = sum(f.cantidad_sd or 0 for f in
    s.query(StockSistema).filter_by(bodega_id=bodega_id).all())

cierre_t = next((t for t in reversed(hitos_t) if t.accion == "CIERRE"), None)
hora_cierre = cierre_t.creado.strftime("%H:%M") if cierre_t else None

ses_auditoria = next((ses for ses in sesiones if ses.tipo == "auditoria"), None)
revisor = None
if ses_auditoria and ses_auditoria.usuario_id:
    us = s.get(Usuario, ses_auditoria.usuario_id)
    revisor = us.nombre.title() if us else None

historial = (s.query(HistorialCierre).filter_by(bodega_id=bodega_id)
    .order_by(HistorialCierre.fecha.desc()).all())

```

```

    anterior = historial[1] if len(historial) > 1 else None
    return {"bodega": b.nombre_oficial, "estado": b.estado,
            "referencias": total, "contadas": len(conteos),
            "exactitud": f"{exact} %", "diferencias": difs, "hitos": hitos,
            "duracion_min": duracion_min, "tiempo_papel_min": tiempo_papel_min,
            "unidades_stock": round(unidades_stock, 1),
            "personas": personas, "alertas_resueltas": alertas_resueltas,
            "hora_cierre": hora_cierre, "revisor": revisor,
            "ultima_toma_anterior": ({ "fecha": anterior.fecha.strftime("%Y-%m-%d"),
                                       "exactitud": anterior.exactitud}
                                     if anterior else None)}

@app.get("/api/bodegas/{bodega_id}/articulos")
def articulos_bodega(bodega_id: int, u: Usuario = Depends(usuario_actual)):
    """El extracto completo de My Inventory para esta bodega, tal cual
    quedo cargado del Excel: codigo, articulo, unidad y saldo del sistema."""
    with Sesion() as s:
        _requiere_acceso_bodega(s, u, bodega_id)
        b = s.get(Bodega, bodega_id)
        if b is None:
            raise HTTPException(404, "Bodega no encontrada.")
        filas = (s.query(StockSistema, Articulo)
                 .join(Articulo, Articulo.codigo == StockSistema.articulo_codigo)
                 .filter(StockSistema.bodega_id == bodega_id)
                 .order_by(Articulo.nombre_oficial).all())
        return [{"codigo": a.codigo, "articulo": a.nombre_oficial,
                "unidad": a.unidad_medida, "sd": st.cantidad_sd}
                for st, a in filas]

# ----- consultas y reportes -----
@app.get("/api/articulos/catalogo-ligero")
def catalogo_ligero(u: Usuario = Depends(usuario_actual)):
    """El catálogo completo pero liviano (solo codigo/nombre/unidad), para
    que el modo sin conexión pueda sugerir nombres oficiales guardandolo
    en el equipo mientras hay señal."""
    with Sesion() as s:
        return [{"codigo": a.codigo, "nombre": a.nombre_oficial, "unidad": a.unidad_medida}
                for a in s.query(Articulo).all()]

@app.get("/api/articulos/consulta")
def consulta_articulo(q: str, codigo: str = "", u: Usuario = Depends(usuario_actual)):
    from servicios.conciliacion import buscar_articulo, buscar_bodegas_candidatas
    # Una orden explícita ("abra kiosco taquilla ayb") se resuelve ANTES
    # que cualquier otra cosa: si se dejara caer primero por la búsqueda
    # de artículo/bodega, el propio "abra" sumaba como palabra de más en
    # la coincidencia por palabras contra esa misma bodega y la orden
    # explícita nunca llegaba a reconocerse como tal - terminaba en la
    # sugerencia de "vea su detalle", no en abrirla de una vez.
    if _VERBOS_ABRIR_BODEGA.search(q):
        r = _resolver_asistente("bodegas", q, u)
        return {
            "resumen": r.get("respuesta_hablada", ""),
            "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
            "accion": r.get("accion"), "destino": r.get("destino"),
            "pestana": r.get("pestana"), "bodega": r.get("bodega"),
        }
    cand_todos = buscar_articulo(q)
    # Este buscador acepta CUALQUIER frase (nombre de ingrediente, de
    # bodega, una pregunta, una orden) - no solo un nombre de artículo
    # dictado a propósito, como sí es el caso en Conteo/Pedido (donde
    # este mismo umbral global de 45 sigue igual, sin tocar). Con ese
    # umbral más flojo aquí, frases sueltas sin relación con ningún

```

```

# producto ("seguir contando", "no entendí", "espera") coincidían
# por casualidad con productos reales que comparten una raíz de 4
# letras (SEGUIr/SEGUnDos, ENTEndí/ENTERo...) - una coincidencia
# LEGÍTIMA en el sentido estructural (la misma que hace que BLANCA
# encuentre BLANCO), pero sin ninguna relación real de significado.
# Probado contra el catálogo real: una búsqueda de artículo genuina,
# hasta parcial o descuidada ("tomate", "vino", "tabla picar"),
# siempre puntuó 84 o más; el ruido de frases sueltas se quedó entre
# 45 y 80. 82 separa limpio los dos grupos.
UMBRALE_CONFianza_BODEGAS = 82
# ya eligió una alternativa específica (clic en un chip de "¿era este
# otro?") - se respeta tal cual, sin el filtro extra: ahí la persona
# ya confirmó cuál quería, así que la confianza original no importa.
cand = cand_todos if codigo else [c for c in cand_todos if c["confianza"] >=
UMBRALE_CONFianza_BODEGAS]
if not cand:
    # lo dicho no es ningún artículo del catálogo - pero si SI es el
    # nombre de una bodega (p. ej. alguien dice "restaurante" en este
    # buscador de ingredientes por error, cuando quería ir a Conteo a
    # abrirla), vale la pena decirlo en vez de un "no encuentre" a
    # secas que deja a la persona sin saber por qué. "restaurante" a
    # secas encaja en más de una bodega real - ahí no hay una sola
    # que prellenar, pero igual vale la pena decir que existen y
    # mandar a Conteo a terminar de decir cual.
    with Sesion() as s:
        ids_permitidos = _ids_permitidos_para_buscar(s, u)
        candidatas = buscar_bodegas_candidatas(s, q, ids_permitidos)
        if ids_permitidos is not None:
            # buscar_bodegas_candidatas() no restringe una coincidencia
            # EXACTA (para que /abrir pueda decir "existe pero no es
            # suya" en vez de "no existe") - aquí, en cambio, esto es
            # solo una sugerencia informativa sin ese motivo: no hay
            # por qué nombrarle a un auxiliar una bodega ajena.
            candidatas = [c for c in candidatas if c.id in ids_permitidos]
        if len(candidatas) == 1:
            b = candidatas[0]
            return {
                "resumen": (f"No encuentro ese artículo, pero {b.nombre_oficial.title()} "
                           "sí es una bodega - vea su detalle para abrirla."),
                "bodegas": [], "sugerencia_bodega": b.nombre_oficial,
                "sugerencia_bodega_id": b.id,
            }
        if candidatas:
            nombres = ", ".join(c.nombre_oficial.title() for c in candidatas[:4])
            return {
                "resumen": (f"No encuentro ese artículo. Ese nombre coincide con varias "
                           f"bodegas ({nombres}) - búsquelas en Bodegas para ver cuál es."),
                "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
            }
    # Ni artículo ni bodega: puede ser una pregunta suelta ("¿cuántas
    # bodegas están pendientes?") o una orden de navegar ("llévame a
    # reportes") - un solo buscador para todo, en vez de dos cuadros
    # de búsqueda separados (uno para ingredientes, otro para
    # cualquier otra cosa) que además terminaban dando respuestas
    # distintas para lo mismo.
    r = _resolver_asistente("bodegas", q, u)
    return {
        "resumen": r.get("respuesta_hablada", ""),
        "bodegas": [], "sugerencia_bodega": None, "sugerencia_bodega_id": None,
        "accion": r.get("accion"), "destino": r.get("destino"),
        "pestaña": r.get("pestaña"), "bodega": r.get("bodega"),
    }
# si la persona ya eligió una de las alternativas, usa esa; si no, la mejor
a = next((c for c in cand if c["codigo"] == codigo), None) or cand[0]
hoy = ahora().date()
with Sesion() as s:
    filas = s.query(StockSistema).filter_by(articulo_codigo=a["codigo"]).all()

```



```

total = sum(f.cantidad_sd or 0 for f in filas)
det = []
for f in filas:
    b = s.get(Bodega, f.bodega_id)
    ultima = (s.query(Conteo).join(SesionConteo)
               .filter(SesionConteo.bodega_id == f.bodega_id,
                       Conteo.articulo_codigo == a["codigo"],
                       Conteo.estado == "confirmado")
               .order_by(Conteo.id.desc()).first())
    if ultima:
        ultima_toma = (f"hoy {ultima.creado.strftime('%H:%M')}")
        if ultima.creado.date() == hoy
            else ultima.creado.strftime("%d %b").lower()
    elif b and b.estado == "en_conteo":
        ultima_toma = "en conteo"
    else:
        ultima_toma = "sin toma"
    det.append({"bodega": b.nombre_oficial if b else "?",
               "cantidad": f.cantidad_sd, "estado": b.estado if b else "?",
               "ultima_toma": ultima_toma})
# consumo real del ultimo mes, sobre servicios ya legalizados
hace_30 = ahora() - timedelta(days=30)
lineas_mes = (s.query(LineaServicio)
               .filter(LineaServicio.articulo_codigo == a["codigo"],
                       LineaServicio.estado == "legalizado",
                       LineaServicio.creado >= hace_30).all())
consumo_mes = sum(l.usado or 0 for l in lineas_mes)
servicios_mes = len(lineas_mes)
cobertura_dias = round(total / (consumo_mes / 30)) if consumo_mes > 0 and total > 0 else None
# otros articulos parecidos y CERCANOS en puntaje: un top score alto no
# basta si el segundo esta empatado o casi (ARROZ y ARROZ BASMATI
# empatan al 100 cuando solo se dice "arroz"); ahí es donde se confunde
# el producto sin que la persona se de cuenta.
alternativas = [c for c in cand if c["codigo"] != a["codigo"]
                 and (a["confianza"] - c["confianza"]) < 15][:2]
ambiguo = bool(alternativas)
resumen = f"{a['nombre']}: {total:g} {a['unidad']} en {len(filas)} bodegas."
if ambiguo:
    resumen += (f" Encontré artículos parecidos ({', '.join(c['nombre'] for c in alternativas)});
    "
    "confirme cuál necesita si no es este.")
return {"articulo": a["nombre"], "codigo": a["codigo"], "unidad": a["unidad"],
        "total": total, "bodegas": det, "resumen": resumen,
        "ambiguo": ambiguo, "alternativas": alternativas,
        "consumo_mes": round(consumo_mes, 2), "servicios_mes": servicios_mes,
        "cobertura_dias": cobertura_dias}

@app.get("/api/articulos/{codigo}/movimientos")
def movimientos_articulo(codigo: str, u: Usuario = Depends(usuario_actual)):
    """«Ver movimientos»: los ultimos conteos confirmados de este articulo,
    en cualquier bodega, con quien y cuando."""
    with Sesion() as s:
        conteos = (s.query(Conteo).filter_by(articulo_codigo=codigo, estado="confirmado")
                   .order_by(Conteo.id.desc()).limit(20).all())
        salida = []
        for c in conteos:
            ses = s.get(SesionConteo, c.sesion_id)
            b = s.get(Bodega, ses.bodega_id) if ses else None
            persona = s.get(Usuario, ses.usuario_id) if ses else None
            salida.append({"hora": c.creado.strftime("%Y-%m-%d %H:%M"),
                           "bodega": b.nombre_oficial if b else "?",
                           "persona": persona.nombre if persona else "?",
                           "cantidad": c.cantidad, "unidad": c.unidad})
    return salida

```

```

@app.get("/api/articulos/{codigo}/en-recetas")
def articulo_en_recetas(codigo: str, u: Usuario = Depends(usuario_actual)):
    """«Comparar con la receta»: en que recetas aparece este articulo y
    cuanto pide por porcion."""
    with Sesion() as s:
        ings = s.query(RecetaIngrediente).filter_by(articulo_codigo=codigo).all()
        salida = []
        for ing in ings:
            r = s.get(Receta, ing.receta_id)
            if r:
                salida.append({"receta": r.nombre, "por_porcion": ing.cantidad_por_porcion,
                              "rendimiento": r.rendimiento})
    return salida

@app.post("/api/reportes")
def reporte(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
    ruta, filas, vista_previa = reportes.consolidado(formato)
    registrar(u, "REPORTE", f"Consolidado generado: {ruta} ({filas} filas)")
    return {"archivo": ruta, "filas": filas, "vista_previa": vista_previa}

@app.post("/api/reportes/diferencias")
def reporte_diferencias(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
    ruta, filas, bodegas_con_descuadre = reportes.diferencias_archivo(formato)
    registrar(u, "REPORTE",
              f"Diferencias por bodega exportado: {ruta} ({filas} filas, "
              f"{bodegas_con_descuadre} bodegas)")
    return {"archivo": ruta, "filas": filas, "bodegas_con_descuadre": bodegas_con_descuadre}

@app.post("/api/bodegas/exportar-estado")
def exportar_estado_bodegas(formato: str = "xlsx", u: Usuario = Depends(requiere_perfil("auditor"))):
    ruta = reportes.estado_bodegas(formato)
    registrar(u, "REPORTE", f"Estado de bodegas exportado: {ruta}")
    return {"archivo": ruta}

def _parsear_archivo_reporte(t):
    """Convierte una traza cruda de tipo REPORTE en la tarjeta de archivo
    que muestra Reportes.jsx - sin esto, cada archivo generado se perdía
    en cuanto se salía de la pantalla (no había historial real)."""
    d = t.detalle
    if ":" not in d or "reportes/" not in d:
        return None
    _, resto = d.split(":", 1)
    archivo = resto.split("(")[0].strip()
    filas = None
    if "(" in resto and "filas" in resto:
        try:
            filas = int(resto.split("(")[1].split(" filas")[0])
        except (ValueError, IndexError):
            filas = None
    if d.startswith("Consolidado generado"):
        titulo, subtítulo = "Consolidado para My Inventory", "Listo para carga"
    elif d.startswith("Diferencias por bodega"):
        titulo = "Diferencias por bodega"
        bodegas_txt = resto.split(", ")[-1].replace(" bodegas", "").strip() if "bodegas" in resto
    else None
    subtítulo = f"{bodegas_txt} bodegas con descuadre" if bodegas_txt else "Exportado"
    if d.startswith("Estado de bodegas"):
        titulo, subtítulo = "Estado del tablero", "Exportado"

```

```

elif d.startswith("Detalle de bodega"):
    titulo, subtítulo = "Detalle de bodega", "Exportado"
elif d.startswith("Análisis de consumo"):
    titulo, subtítulo = "Análisis de consumo", "Exportado"
elif d.startswith("Registro de trazabilidad"):
    titulo, subtítulo = "Registro de trazabilidad", "Exportado"
else:
    titulo, subtítulo = "Reporte", "Exportado"
return {"titulo": titulo, "subtitulo": subtítulo, "archivo": archivo, "filas": filas,
        "formato": archivo.rsplit(".", 1)[-1].upper() if "." in archivo else "?",
        "hora": t.creado.strftime("%H:%M"), "persona": (t.persona or "").title()}

@app.get("/api/reportes/recientes")
def reportes_recientes(u: Usuario = Depends(requiere_perfil("auditor"))):
    """El historial real de archivos generados (via pantalla o por voz),
    leído de la trazabilidad - para que la lista sobreviva a salir de la
    pantalla o recargar, en vez de vivir solo en el estado del componente.
    Un solo archivo por tipo (el mas reciente): generar el mismo reporte
    varias veces en el día no debe ir apilando copias viejas - lo que
    importa es el último, los anteriores ya quedaron obsoletos apenas se
    genere uno nuevo del mismo tipo."""
    with Sesion() as s:
        trazas = (s.query(Traza).filter_by(accion="REPORTE")
                  .order_by(Traza.id.desc()).limit(40).all())
    vistos = set()
    salida = []
    for t in trazas:
        x = _parsear_archivo_reporte(t)
        if not x or x["titulo"] in vistos:
            continue
        vistos.add(x["titulo"])
        salida.append(x)
    return salida[:10]

@app.post("/api/bodegas/{bodega_id}/exportar-detalle")
def exportar_detalle_bodega(bodega_id: int, formato: str = "xlsx",
                             u: Usuario = Depends(requiere_perfil("auditor"))):
    ruta = reportes.detalle_bodega(bodega_id, formato)
    registrar(u, "REPORTE", f"Detalle de bodega {bodega_id} exportado: {ruta}")
    return {"archivo": ruta}

@app.get("/api/reportes/descargar")
def descargar(archivo: str, u: Usuario = Depends(requiere_perfil("auditor"))):
    if ".." in archivo or not archivo.startswith("reportes/"):
        raise HTTPException(400, "Ruta no permitida.")
    from servicios.archivos import leer_bytes
    contenido = leer_bytes(archivo)
    if contenido is None:
        raise HTTPException(404, "Archivo no encontrado.")
    tipo = "text/csv" if archivo.endswith(".csv") else (
        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
    return Response(content=contenido, media_type=tipo, headers={
        "Content-Disposition": f'attachment; filename="{os.path.basename(archivo)}"'})

@app.get("/api/reportes/vista-previa")
def vista_previa_reporte(archivo: str, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Para poder ver el contenido de cualquier archivo ya generado (no solo
    el que se acaba de crear) con solo dar clic en su tarjeta, sin tener
    que descargarlo primero. Lee el archivo tal cual quedo guardado (de la
    base, no de disco - ver servicios/archivos.py), así que la vista previa

```

```

de un reporte viejo muestra lo que ese reporte realmente tenia, no el
estado actual de la base, y sigue funcionando despues de un redeploy."""
if ".." in archivo or not archivo.startswith("reportes/"):
    raise HTTPException(400, "Ruta no permitida.")
import io
import pandas as pd
from servicios.archivos import leer_bytes
contenido = leer_bytes(archivo)
if contenido is None:
    raise HTTPException(404, "Archivo no encontrado.")
buf = io.BytesIO(contenido)
df = pd.read_csv(buf) if archivo.endswith(".csv") else pd.read_excel(buf)
df = df.fillna(0)
return {"filas": df.head(8).to_dict("records"), "total": len(df)}

# ----- los tres momentos -----
class PedidoIn(BaseModel):
    plato: str
    porciones: int
    bodega_id: int = 1

@app.post("/api/pedidos/calcular")
def api_calcular(p: PedidoIn, u: Usuario = Depends(usuario_actual)):
    return calcular_pedido(p.plato, p.porciones, p.bodega_id)

@app.post("/api/pedidos/enviar")
def api_enviar(body: dict, u: Usuario = Depends(usuario_actual)):
    sid = body.get("servicio_id", 1)
    plato = body.get("plato", "")
    porciones = body.get("porciones", 0)
    bodega_id = body.get("bodega_id")
    with Sesion() as s:
        # si el mismo pedido (servicio + plato + porciones) ya quedo abierto
        # O esperando aprobacion, no lo duplica: cubre un doble clic, un
        # doble disparo por voz, o un reintento tras una desconexion que si
        # alcanzo a llegar la primera vez.
        ya_existe = (s.query(LineaServicio)
                     .filter_by(servicio_id=sid, plato=plato, porciones=porciones)
                     .filter(LineaServicio.estado.in_([ "abierto", "pendiente_aprobacion" ]))
                     .first())
        if ya_existe:
            return {"ok": True, "duplicado": True}
        # un auxiliar pide, pero es el administrador quien de verdad autoriza
        # que salga del almacen - igual que ya pasa con productos y bodegas
        # creados en plena toma. El administrador autoriza su propio pedido
        # de una vez: no tiene sentido pedirse permiso a si mismo.
        estado_inicial = "abierto" if u.perfil == "auditor" else "pendiente_aprobacion"
        numero = f"PED-{ahora().strftime('%Y%m%d-%H%M%S')}"
        n = 0
        items = []
        for l in body.get("lineas", []):
            if (l.get("falta") or 0) > 0:
                s.add(LineaServicio(servicio_id=sid,
                                    articulo_codigo=l["codigo"],
                                    nombre=l["nombre"], pedido=l["falta"],
                                    plato=plato, porciones=porciones,
                                    estado=estado_inicial, bodega_id=bodega_id,
                                    creado_por_id=u.id, numero_pedido=numero))
                items.append({"nombre": l["nombre"], "cantidad": l["falta"],
                             "unidad": l.get("unidad", "")})
            n += 1
        s.commit()

```

```

        bodega_nombre = None
        if bodega_id:
            b = s.get(Bodega, bodega_id)
            bodega_nombre = b.nombre_oficial if b else None
        if estado_inicial == "pendiente_aprobacion":
            registrar(u, "PEDIDO", f"Pedido {numero} enviado, pendiente de aprobacion ({n} lineas)")
        else:
            registrar(u, "PEDIDO", f"Pedido {numero} enviado y aprobado ({n} lineas)")
        return {"ok": True, "numero_pedido": numero, "estado": estado_inicial,
                "hora": ahora().strftime("%H:%M"),
                "bodega": bodega_nombre, "persona": u.nombre,
                "items": items, "total_lineas": n}

@app.get("/api/pedidos/pendientes")
def pedidos_pendientes(u: Usuario = Depends(requiere_perfil("auditor"))):
    """Pedidos de auxiliares esperando la firma del administrador antes de
    contar como enviados de verdad al almacen - el pedir no se detiene
    (queda registrado de una vez), pero salir del almacen si depende de
    esta aprobacion, igual que con productos y bodegas nuevas."""
    with Sesion() as s:
        filas = (s.query(LineaServicio)
                 .filter_by(estado="pendiente_aprobacion")
                 .order_by(LineaServicio.creado.desc()).all())
        grupos = {}
        for f in filas:
            g = grupos.setdefault(f.numero_pedido, {
                "numero_pedido": f.numero_pedido, "plato": f.plato,
                "porciones": f.porciones, "hora": f.creado.strftime("%H:%M"),
                "bodega_id": f.bodega_id, "creado_por_id": f.creado_por_id,
                "items": [],
            })
            g["items"].append({"nombre": f.nombre, "cantidad": f.pedido})
        out = []
        for g in grupos.values():
            b = s.get(Bodega, g["bodega_id"]) if g["bodega_id"] else None
            persona = s.get(Usuario, g["creado_por_id"]) if g["creado_por_id"] else None
            out.append(**g, "bodega": b.nombre_oficial if b else None,
                       "persona": persona.nombre if persona else "-")
        out.sort(key=lambda x: x["numero_pedido"], reverse=True)
        return out

class ResolverPedidoIn(BaseModel):
    aprobar: bool

@app.post("/api/pedidos/{numero_pedido}/resolver")
def resolver_pedido(numero_pedido: str, datos: ResolverPedidoIn,
                    u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        filas = (s.query(LineaServicio)
                 .filter_by(numero_pedido=numero_pedido, estado="pendiente_aprobacion").all())
        if not filas:
            raise HTTPException(404, "Pedido no encontrado o ya resuelto.")
        nuevo_estado = "abierto" if datos.aprobar else "rechazado"
        plato = filas[0].plato
        for f in filas:
            f.estado = nuevo_estado
        s.commit()
        accion = "aprobado" if datos.aprobar else "rechazado"
        registrar(u, "APROBACION", f"Pedido {numero_pedido} ({plato}) {accion}",
                 "ok" if datos.aprobar else "alerta")
        return {"ok": True, "estado": nuevo_estado}

```

```

@app.get("/api/legalizacion/{servicio_id}")
def api_legalizacion(servicio_id: int, u: Usuario = Depends(usuario_actual)):
    return comparar_legalizacion(servicio_id)

@app.post("/api/legalizacion/confirmar")
def api_confirmar(body: dict, u: Usuario = Depends(usuario_actual)):
    sid = body.get("servicio_id", 1)
    # opcional: permite mandar lo usado junto con la confirmación en vez de
    # exigir un "/legalizacion/ajustar" por cada insumo antes de cerrar.
    usos = {item.get("codigo"): item.get("usado") for item in body.get("usos", [])}
    with Sesion() as s:
        for l in s.query(LineaServicio).filter_by(servicio_id=sid, estado="abierto").all():
            if l.articulo_codigo in usos and usos[l.articulo_codigo] is not None:
                l.usado = usos[l.articulo_codigo]
            dif = (l.usado or 0) - (l.pedido or 0)
            if dif < 0:
                # sobrante: vuelve a bodega
                st = s.query(StockSistema).filter_by(
                    articulo_codigo=l.articulo_codigo, bodega_id=l.bodega_id).first()
                if st:
                    st.cantidad_sd = (st.cantidad_sd or 0) + abs(dif)
            l.estado = "legalizado"
        s.commit()
    registrar(u, "LEGALIZACION", f"Servicio {sid} legalizado", "ok")
    return {"ok": True}

@app.post("/api/legalizacion/ajustar")
def api_ajustar_legalizacion(body: dict, u: Usuario = Depends(usuario_actual)):
    """«Ajustar por voz»: corrige lo realmente usado de un insumo antes de
    confirmar, dictando en vez de editar celda por celda."""
    from agente.cerebro import pensar
    sid = body.get("servicio_id", 1)
    texto = (body.get("texto") or "").strip()
    if not texto:
        raise HTTPException(400, "Dígame el insumo y la cantidad.")
    turno = pensar("Esta corrigiendo cuanto se uso de un insumo en un "
        "servicio de comidas ya cerrado, antes de legalizarlo.", texto)
    articulo_texto = (turno.get("articulo_texto") or texto).upper()
    cantidad = turno.get("cantidad")
    if cantidad is None:
        raise HTTPException(400, "No entendí la cantidad. ¿Me la repite?")
    palabras = [p for p in articulo_texto.split() if len(p) >= 3]
    with Sesion() as s:
        filas = s.query(LineaServicio).filter_by(servicio_id=sid, estado="abierto").all()
        objetivo = next((f for f in filas
            if any(p in f.nombre.upper() for p in palabras)), None)
        if objetivo is None:
            raise HTTPException(404, "No encontré ese insumo en este servicio.")
        objetivo.usado = cantidad
        s.commit()
        nombre = objetivo.nombre
    registrar(u, "LEGALIZACION", f"Ajuste por voz: {nombre} -> {cantidad:g}", "alerta")
    return comparar_legalizacion(sid)

@app.get("/api/analisis/consumo")
def api_analisis(dias: int = 30, u: Usuario = Depends(requiere_perfil("auditor"))):
    return analisis_consumo(dias)

```

```

@app.post("/api/analisis/exportar")
def exportar_analisis(formato: str = "xlsx", dias: int = 30,
                      u: Usuario = Depends(requiere_perfil("auditor"))):
    import pandas as pd
    from servicios.archivos import guardar_df
    datos = analisis_consumo(dias)
    df = pd.DataFrame(datos["subutilizados"])
    marca = ahora().strftime("%Y%m%d_%H%M")
    ruta = f"reportes/analisis_consumo_{marca}.{formato}"
    guardar_df(ruta, df, formato)
    registrar(u, "REPORTE", f"Análisis de consumo exportado: {ruta} ({len(df)} filas)")
    return {"archivo": ruta}

@app.get("/api/pedidos/receta")
def api_receta(plato: str, u: Usuario = Depends(usuario_actual)):
    return detalle_receta(plato)

# ----- recetas: catálogo administrado -----
# CuentaVoz si gestiona las recetas: crearlas, editarlas y borrarlas queda a
# cargo del administrador (perfil auditor), igual que la gestión de usuarios
# o de bodegas. El auxiliar solo las consulta al armar un pedido.
class IngredienteIn(BaseModel):
    articulo_codigo: str
    cantidad_por_porcion: float

class RecetaIn(BaseModel):
    nombre: str
    rendimiento: int = 1
    preparacion: str = ""
    ingredientes: list[IngredienteIn]

@app.get("/api/recetas")
def listar_recetas(u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        recetas = s.query(Receta).order_by(Receta.nombre).all()
        return [{"id": r.id, "nombre": r.nombre, "rendimiento": r.rendimiento,
                  "ingredientes": s.query(RecetaIngrediente)
                                  .filter_by(receta_id=r.id).count()}
                for r in recetas]

@app.get("/api/recetas/{receta_id}")
def obtener_receta(receta_id: int, u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        r = s.get(Receta, receta_id)
        if r is None:
            raise HTTPException(404, "Receta no encontrada.")
        ings = s.query(RecetaIngrediente).filter_by(receta_id=r.id).all()
        lineas = []
        for i in ings:
            art = s.get(Articulo, i.articulo_codigo)
            lineas.append({"articulo_codigo": i.articulo_codigo,
                           "nombre": art.nombre_oficial if art else i.articulo_codigo,
                           "unidad": art.unidad_medida if art else "",
                           "cantidad_por_porcion": i.cantidad_por_porcion})
        return {"id": r.id, "nombre": r.nombre, "rendimiento": r.rendimiento,
                "preparacion": r.preparacion or "", "lineas": lineas}

```

```

def _validar_ingredientes(s, ingredientes):
    if not ingredientes:
        raise HTTPException(400, "La receta necesita al menos un ingrediente.")
    for ing in ingredientes:
        if s.get(Articulo, ing.articulo_codigo) is None:
            raise HTTPException(400, f"El artículo {ing.articulo_codigo} no existe en el catálogo.")
        if ing.cantidad_por_porcion <= 0:
            raise HTTPException(400, "La cantidad por porción debe ser mayor que cero.")

@app.post("/api/recetas")
def crear_receta(datos: RecetaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        _validar_ingredientes(s, datos.ingredientes)
        if s.query(Receta).filter(Receta.nombre.ilike(datos.nombre.strip())).first():
            raise HTTPException(400, "Ya existe una receta con ese nombre.")
        r = Receta(nombre=datos.nombre.strip(), rendimiento=datos.rendimiento,
                  preparacion=datos.preparacion.strip())
        s.add(r)
        s.flush()
        for ing in datos.ingredientes:
            s.add(RecetaIngrediente(receta_id=r.id, articulo_codigo=ing.articulo_codigo,
                                   cantidad_por_porcion=ing.cantidad_por_porcion))
        s.commit()
        rid, nombre = r.id, r.nombre
        registrar(u, "RECETA", f"Receta creada: {nombre} ({len(datos.ingredientes)} ingredientes)", "ok")
        return {"ok": True, "id": rid}

@app.put("/api/recetas/{receta_id}")
def editar_receta(receta_id: int, datos: RecetaIn,
                  u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        r = s.get(Receta, receta_id)
        if r is None:
            raise HTTPException(404, "Receta no encontrada.")
        _validar_ingredientes(s, datos.ingredientes)
        otra = s.query(Receta).filter(Receta.nombre.ilike(datos.nombre.strip()),
                                     Receta.id != receta_id).first()
        if otra:
            raise HTTPException(400, "Ya existe otra receta con ese nombre.")
        r.nombre = datos.nombre.strip()
        r.rendimiento = datos.rendimiento
        r.preparacion = datos.preparacion.strip()
        s.query(RecetaIngrediente).filter_by(receta_id=receta_id).delete()
        for ing in datos.ingredientes:
            s.add(RecetaIngrediente(receta_id=receta_id, articulo_codigo=ing.articulo_codigo,
                                   cantidad_por_porcion=ing.cantidad_por_porcion))
        s.commit()
        nombre = r.nombre
        registrar(u, "RECETA", f"Receta editada: {nombre} ({len(datos.ingredientes)} ingredientes)", "ok")
        return {"ok": True}

@app.delete("/api/recetas/{receta_id}")
def eliminar_receta(receta_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        r = s.get(Receta, receta_id)
        if r is None:
            raise HTTPException(404, "Receta no encontrada.")
        nombre = r.nombre
        s.query(RecetaIngrediente).filter_by(receta_id=receta_id).delete()
        s.delete(r)

```



```

        s.commit()
    registrar(u, "RECETA", f"Receta eliminada: {nombre}", "ok")
    return {"ok": True}

@app.get("/api/reportes/diferencias-por-bodega")
def api_diferencias_por_bodega(u: Usuario = Depends(requiere_perfil("auditor"))):
    return analitica.diferencias_por_bodega(limite=50)

@app.get("/api/panel/resumen")
def api_panel_resumen(u: Usuario = Depends(requiere_perfil("auditor"))):
    return analitica.resumen_ejecutivo()

@app.get("/api/panel/alertas")
def api_panel_alertas(u: Usuario = Depends(requiere_perfil("auditor"))):
    return analitica.resumen_alertas_panel()

# ----- alertas, usuarios y trazas -----
_TIPO_ALERTA_TITULO = {
    "desviacion": "Desviación de cantidad", "negativo": "Cantidad negativa dictada",
    "unidad": "Unidad de medida", "inexistente": "Artículo fuera del catálogo",
}

@app.get("/api/alertas")
def ver_alertas(resueltas: int = 0, u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        salida = []
        for a in s.query(Alerta).filter_by(resuelta=resueltas).order_by(
            Alerta.id.desc()).all():
            art = bodega = persona = None
            if a.conteo_id:
                c = s.get(Conteo, a.conteo_id)
                if c:
                    ar = s.get(Articulo, c.articulo_codigo)
                    art = ar.nombre_oficial if ar else None
                    ses = s.get(SesionConteo, c.sesion_id)
                    if ses:
                        b = s.get(Bodega, ses.bodega_id)
                        bodega = b.nombre_oficial if b else None
                        us = s.get(Usuario, ses.usuario_id)
                        persona = us.nombre.title() if us else None
            salida.append({"id": a.id, "tipo": a.tipo,
                          "titulo": _TIPO_ALERTA_TITULO.get(a.tipo, a.tipo),
                          "detalle": a.detalle, "articulo": art, "bodega": bodega,
                          "persona": persona, "hora": a.creado.strftime("%H:%M")})

    return salida

@app.get("/api/alertas/resumen")
def resumen_alertas(u: Usuario = Depends(usuario_actual)):
    """Estadísticas reales para la bandeja: abiertas, resueltas hoy y el
    tiempo medio real entre que se genera la alerta y se resuelve."""
    hoy = ahora().date()
    with Sesion() as s:
        abiertas = s.query(Alerta).filter_by(resuelta=0).count()
        resueltas_hoy = (s.query(Alerta).filter(Alerta.resuelta == 1,
            Alerta.resuelto.isnot(None)).all())
        resueltas_hoy = [a for a in resueltas_hoy if a.resuelto.date() == hoy]

```

```

    tiempo_medio_min = None
    if resueltas_hoy:
        segundos = [(a.resuelto - a.creado).total_seconds() for a in resueltas_hoy]
        tiempo_medio_min = round(sum(segundos) / len(segundos) / 60, 1)
    return {"abiertas": abiertas, "resueltas_hoy": len(resueltas_hoy),
            "tiempo_medio_min": tiempo_medio_min}

@app.post("/api/alertas/{alerta_id}/resolver")
def resolver_alerta(alerta_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        a = s.get(Alerta, alerta_id)
        if a is None:
            raise HTTPException(404, "Alerta no encontrada.")
        a.resuelta = 1
        a.resuelto = ahora()
        s.commit()
        nombre = None
        if a.conteo_id:
            c = s.get(Conteo, a.conteo_id)
            if c:
                art = s.get(Articulo, c.articulo_codigo)
                nombre = art.nombre_oficial if art else None
        detalle = f"Alerta resuelta - {nombre}: {a.detalle}" if nombre else f"Alerta {alerta_id} resuelta"
        registrar(u, "ALERTA", detalle, "ok")
        return {"ok": True}

@app.get("/api/usuarios")
def listar_usuarios(u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        salida = []
        for x in s.query(Usuario).all():
            n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=x.id).count()
            salida.append({"id": x.id, "nombre": x.nombre, "perfil": x.perfil,
                           "correo": x.correo, "activo": bool(x.activo),
                           "bodegas_asignadas": n_bodegas,
                           "ultimo_acceso": (x.ultimo_acceso.strftime("%Y-%m-%d %H:%M")
                                              if x.ultimo_acceso else None)})
        return salida

class CrearUsuarioIn(BaseModel):
    nombre: str
    perfil: str
    correo: str = ""
    pin: str | None = None    # None: se genera uno temporal, ver mas abajo

@app.post("/api/usuarios")
def crear_usuario(datos: CrearUsuarioIn, u: Usuario = Depends(requiere_perfil("auditor"))):
    nombre = datos.nombre.strip().lower()
    if datos.perfil not in ("auxiliar", "auditor"):
        raise HTTPException(400, "El perfil debe ser auxiliar o auditor.")
    # sin esto, todo usuario creado desde Ajustes (la pantalla nunca pide
    # un PIN) quedaba con la misma clave de siempre - la misma que
    # aparece publicada en el README para las cuentas de demostracion.
    # Un temporal aleatorio, distinto cada vez, obliga a que quien lo
    # reciba lo cambie por uno propio desde Mi perfil.
    pin_generado = None
    if datos.pin is None:
        pin_generado = secrets.token_urlsafe(6)
        pin_para_guardar = pin_generado
    else:

```

```

        pin_para_guardar = datos.pin
    if len(pin_para_guardar) < 6:
        raise HTTPException(400, "El PIN debe tener al menos 6 dígitos.")
    with Sesion() as s:
        if s.query(Usuario).filter_by(nombre=nombre).first():
            raise HTTPException(409, "Ya existe un usuario con ese nombre.")
        nuevo = Usuario(nombre=nombre, perfil=datos.perfil,
                        clave_hash=hash_clave(pin_para_guardar), correo=datos.correo)
        s.add(nuevo)
        s.commit()
        s.refresh(nuevo)
        nid = nuevo.id
        # código de empleado: para poder ingresar con el además del nombre
        nuevo.codigo = f"CS-{48000 + nid}"
        s.commit()
    registrar(u, "USUARIO", f"Usuario {nombre} creado ({datos.perfil})", "ok")
    respuesta = {"ok": True, "id": nid}
    if pin_generado:
        respuesta["pin_temporal"] = pin_generado
    return respuesta

@app.put("/api/usuarios/yo")
def editar_perfil(datos: dict, u: Usuario = Depends(usuario_actual)):
    """Cada quien edita su propio perfil - va antes de /usuarios/{usuario_id}
    a proposito: "yo" no es un entero, y si esta ruta queda despues, esa otra
    la intercepta primero y revienta con un 422 (el mismo problema que ya
    paso con /usuarios/yo/bodegas)."""
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        for k in ("nombre", "correo", "telefono"):
            if k in datos:
                setattr(usr, k, datos[k])
        s.commit()
    registrar(u, "PERFIL", "Datos personales actualizados")
    return {"ok": True}

class EditarUsuarioIn(BaseModel):
    correo: str | None = None
    perfil: str | None = None
    activo: bool | None = None

@app.put("/api/usuarios/{usuario_id}")
def editar_usuario(usuario_id: int, datos: EditarUsuarioIn,
                  u: Usuario = Depends(requiere_perfil("auditor"))):
    """Editar correo, rol o estado de OTRA persona - no es lo mismo que
    /usuarios/yo, que cada quien usa para su propio perfil. No se borra a
    nadie (romperia la trazabilidad de sus conteos y firmas pasadas):
    desactivar es el equivalente seguro a eliminar, y ya bloquea el
    ingreso (ver usuario_actual)."""
    if datos.perfil is not None and datos.perfil not in ("auxiliar", "auditor"):
        raise HTTPException(400, "El perfil debe ser auxiliar o auditor.")
    if usuario_id == u.id and datos.activo is False:
        raise HTTPException(400, "No puede desactivar su propia cuenta.")
    if usuario_id == u.id and datos.perfil is not None and datos.perfil != u.perfil:
        raise HTTPException(400, "No puede cambiar su propio rol.")
    with Sesion() as s:
        obj = s.get(Usuario, usuario_id)
        if obj is None:
            raise HTTPException(404, "Usuario no encontrado.")
        cambios = []
        if datos.correo is not None and datos.correo != obj.correo:
            obj.correo = datos.correo

```

```

        cambios.append("correo")
    if datos.perfil is not None and datos.perfil != obj.perfil:
        obj.perfil = datos.perfil
        cambios.append(f"perfil -> {datos.perfil}")
    if datos.activo is not None and bool(datos.activo) != bool(obj.activo):
        obj.activo = int(datos.activo)
        obj.version_token = (obj.version_token or 0) + 1 # cierra sus sesiones vivas
        cambios.append("activo" if datos.activo else "inactivo")
    s.commit()
    nombre = obj.nombre
    if cambios:
        registrar(u, "USUARIO", f"{nombre} editado: {'', '.join(cambios)}", "ok")
    return {"ok": True}

@app.delete("/api/usuarios/{usuario_id}")
def eliminar_usuario(usuario_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Borra de verdad a alguien (no solo desactivar) - para limpiar cuentas
    de prueba de un ambiente antes de una demo, sin dejarlas visibles en
    Gestión de usuarios. A propósito NO es lo que hace editar_usuario()
    de arriba para el uso normal (esa sigue prefiriendo desactivar, por la
    misma razón documentada ahí: no romper la trazabilidad de conteos y
    firmas reales). Aquí sí se borra la cuenta, pero la Traza (registro
    inmutable) NUNCA se toca como fila - solo se le suelta la referencia al
    usuario (usuario_id queda en NULL; el nombre en "persona" ya es texto
    aparte) para que el historial real de esa persona, si lo hubo, siga
    intacto y legible después de borrar la cuenta."""
    if usuario_id == u.id:
        raise HTTPException(400, "No puede eliminar su propia cuenta.")
    with Sesion() as s:
        obj = s.get(Usuario, usuario_id)
        if obj is None:
            raise HTTPException(404, "Usuario no encontrado.")
        nombre = obj.nombre
        # sesiones de conteo/auditoria de esta persona: primero los Conteo
        # de cada sesion (y las Alertas que esos Conteo hayan generado),
        # despues la sesion misma - en ese orden, por las llaves foraneas.
        sesiones = s.query(SesionConteo).filter_by(usuario_id=usuario_id).all()
        for ses in sesiones:
            conteos = s.query(Conteo).filter_by(sesion_id=s.id).all()
            for c in conteos:
                s.query(Alerta).filter_by(conteo_id=c.id).delete()
                s.delete(c)
            s.delete(ses)
        s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).delete()
        s.query(CredencialWebAuthn).filter_by(usuario_id=usuario_id).delete()
        # MensajeSoporte exige remitente/destinatario (no admite NULL) - sin
        # cuenta de uno de los dos lados, el mensaje no puede seguir existiendo.
        s.query(MensajeSoporte).filter(
            (MensajeSoporte.remitente_id == usuario_id)
            | (MensajeSoporte.destinatario_id == usuario_id)).delete()
        # estas si admiten NULL: se sueltan en vez de borrarse, para no
        # perder pedidos/aprobaciones reales solo porque quien los creo
        # ya no tiene cuenta.
        for linea in s.query(LineaServicio).filter_by(creado_por_id=usuario_id).all():
            linea.creado_por_id = None
        for ap in s.query(Aprobacion).filter_by(creado_por_id=usuario_id).all():
            ap.creado_por_id = None
        for ap in s.query(Aprobacion).filter_by(resuelto_por_id=usuario_id).all():
            ap.resuelto_por_id = None
        for t in s.query(Traza).filter_by(usuario_id=usuario_id).all():
            t.usuario_id = None
        s.delete(obj)
        s.commit()
    registrar(u, "USUARIO", f"{nombre} eliminado de forma permanente", "ok")
    return {"ok": True}

```

```

class ItemSemillaIn(BaseModel):
    articulo_codigo: str
    cantidad: float
    # solo para bodegas sin ningun stock de sistema cargado todavia (el
    # extracto de My Inventory nunca llego para esa bodega): crea la fila
    # de StockSistema con este valor ANTES de contar, para tener contra
    # que comparar. Si la bodega ya tiene stock de este articulo, se
    # ignora - nunca pisa un valor real ya cargado.
    cantidad_sistema_si_falta: float | None = None
    # una cantidad que se sale del umbral normalmente se salta (ver
    # docstring de sembrar_conteo) - forzar=true la guarda igual Y crea la
    # Alerta de "desviacion" que le corresponderia, exactamente como
    # cuando una persona real dice "sí, confirmo" tras la advertencia. Es
    # para poblar Alertas/Auditoria con casos reales en un ambiente de
    # demo, no para forzar cantidades negativas o de otro articulo/unidad
    # (esas siguen sin poder guardarse: no tendria sentido real).
    forzar: bool = False

class ConteoSemillaIn(BaseModel):
    bodega_id: int
    usuario_id: int
    items: list[ItemSemillaIn]

@app.post("/api/admin/sembrar-conteo")
def sembrar_conteo(datos: ConteoSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Crea conteos CONFIRMADOS reales (misma tabla, misma trazabilidad, se
    validan igual que si la persona lo hubiera dictado) para poblar un
    ambiente con datos de muestra antes de una demo - ej. que el panel
    gerencial tenga varias bodegas con diferencia real, no solo una. No
    pasa por reconocimiento de voz/texto: articulo y cantidad van directos,
    para que sembrar muchos a la vez no dependa de que el agente entienda
    cada frase (ni de la cuota de Gemini). Los items que dispararian una
    alerta (negativo, fuera de umbral...) se saltan en vez de forzarse -
    esto es para que la demo se vea real, no para inventar anomalias."""
    with Sesion() as s:
        persona = s.get(Usuario, datos.usuario_id)
        if persona is None:
            raise HTTPException(404, "Usuario no encontrado.")
        bodega = s.get(Bodega, datos.bodega_id)
        if bodega is None:
            raise HTTPException(404, "Bodega no encontrada.")
        ses = (s.query(SesionConteo)
               .filter_by(bodega_id=datos.bodega_id, usuario_id=datos.usuario_id, estado="abierta")
               .first())
        if not ses:
            ses = SesionConteo(bodega_id=datos.bodega_id, usuario_id=datos.usuario_id,
                               tipo="conteo", estado="abierta")
            s.add(ses)
            s.commit()
            s.refresh(ses)
        sesion_id = ses.id
        if bodega.estado == "pendiente":
            bodega.estado = "en_conteo"
            s.commit()
        nombre_bodega = bodega.nombre_oficial

    guardados = 0
    saltados = []
    for item in datos.items:
        with Sesion() as s:

```

```

    art = s.get(Articulo, item.articulo_codigo)
    if art is None:
        saltados.append(item.articulo_codigo)
        continue
    if item.cantidad_sistema_si_falta is not None:
        existe = s.query(StockSistema).filter_by(
            articulo_codigo=item.articulo_codigo, bodega_id=datos.bodega_id).first()
        if not existe:
            s.add(StockSistema(articulo_codigo=item.articulo_codigo,
                                bodega_id=datos.bodega_id,
                                cantidad_sd=item.cantidad_sistema_si_falta))
            s.commit()
    v = validar_conteo(item.articulo_codigo, item.cantidad, art.unidad_medida, datos.bodega_id)
    if not v["ok"] and not (item.forzar and v["tipo"] == "desviacion"):
        saltados.append(item.articulo_codigo)
        continue
    with Sesion() as s:
        reg = Conteo(sesion_id=sesion_id, articulo_codigo=item.articulo_codigo,
                     cantidad=item.cantidad, unidad=art.unidad_medida, estado="confirmado")
        s.add(reg)
        s.commit()
        s.refresh(reg)
        if not v["ok"]:
            esperado = v.get("esperado") or 0
            s.add(Alerta(conteo_id=reg.id, tipo="desviacion",
                        detalle=f"El sistema esperaba alrededor de {esperado:g}, "
                               f"se contaron {item.cantidad:g}."))
            s.commit()
        guardados += 1
    if guardados:
        registrar(persona, "CONTEO",
                  f"{guardados} referencias contadas en {nombre_bodega} (datos de muestra)")
    return {"ok": True, "guardados": guardados, "saltados": saltados}

class DuracionSemillaIn(BaseModel):
    sesion_id: int
    minutos: float

@app.post("/api/admin/sembrar-duracion")
def sembrar_duracion(datos: DuracionSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Ajusta cuánto "duró" una sesión de conteo (su inicio, relativo al fin
    que ya tiene) - para cuando una sesión de muestra se abrió hace rato
    (en otra prueba) y se firma/cierra recién ahora: sin esto, "tiempo
    promedio de conteo" del panel sale con la diferencia real entre esos
    dos momentos, que no fue tiempo contando de verdad."""
    with Sesion() as s:
        ses = s.get(SesionConteo, datos.sesion_id)
        if ses is None:
            raise HTTPException(404, "Sesion no encontrada.")
        if not ses.fin:
            raise HTTPException(409, "Esta sesion todavia no tiene fin (no está firmada/cerrada).")
        ses.inicio = ses.fin - timedelta(minutes=datos.minutos)
        s.commit()
    return {"ok": True}

class AlertaSemillaIn(BaseModel):
    tipo: str # negativo | unidad | inexistente | desviacion
    detalle: str

@app.post("/api/admin/sembrar-alerta")

```

```

def sembrar_alerta(datos: AlertaSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Crea una Alerta suelta (sin conteo asociado) para los tipos que en
    el flujo real NUNCA llegan a guardarse como conteo (negativo, unidad
    equivocada, artículo inexistente) - la persona tiene que repetir el
    dato, así que lo único que queda de ese intento es la alerta misma.
    Para poblar Auditoría/Panel con variedad real de tipos antes de una
    demo, no solo "desviación" (que sí se puede sembrar junto con su
    conteo via /admin/sembrar-conteo con forzar=true)."""
    if datos.tipo not in ("negativo", "unidad", "inexistente", "desviacion"):
        raise HTTPException(400, "Tipo de alerta no reconocido.")
    with Sesion() as s:
        s.add(Alerta(conteo_id=None, tipo=datos.tipo, detalle=datos.detalle))
        s.commit()
    return {"ok": True}

class NegativosSemillaIn(BaseModel):
    inicial: int

@app.put("/api/admin/negativos-iniciales")
def ajustar_negativos_iniciales(datos: NegativosSemillaIn, u: Usuario =
Depends(requiere_perfil("auditor"))):
    """La tarjeta "Negativos detectados en el sistema" compara el inicio de
    ESTE período contra el conteo actual - si nunca se fijó un punto de
    partida (o se fijó igual al valor de hoy, por default), la tarjeta
    siempre muestra "X → X" y no cuenta ninguna historia. Este valor es la
    foto de hace un tiempo (de antes de que My Inventory corrigiera los
    que ya se corrigieron), no algo que la app recalcula sola: por diseño,
    corregir un negativo del sistema pasa por fuera de CuentaVoz."""
    with Sesion() as s:
        existente = s.get(ConfigClave, "negativos_iniciales")
        valor = str(datos.inicial)
        if existente:
            existente.valor = valor
        else:
            s.add(ConfigClave(clave="negativos_iniciales", valor=valor))
        s.commit()
    return {"ok": True}

@app.delete("/api/admin/historial-cierre/{historial_id}")
def borrar_historial_cierre(historial_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Para limpiar una foto de "Exactitud por toma de inventario" que
    quedó mal (ej. una bodega de muestra que se cerró antes de tener
    suficientes items contados, y después se reabrió para completarla) -
    a diferencia de la Traza, HistorialCierre no es un registro legal
    inmutable: es una serie de tiempo para la gráfica del panel, y una
    foto tomada por error de una prueba no debería quedar mezclada con el
    histórico real."""
    with Sesion() as s:
        h = s.get(HistorialCierre, historial_id)
        if h is None:
            raise HTTPException(404, "Registro no encontrado.")
        s.delete(h)
        s.commit()
    return {"ok": True}

class UsoSemillaItem(BaseModel):
    articulo_codigo: str
    usado: float

```

```

class UsoSemillaIn(BaseModel):
    servicio_id: int
    usos: list[UsoSemillaItem]

@app.put("/api/admin/legalizacion-uso")
def sembrar_uso_legalizacion(datos: UsoSemillaIn, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Registra cuánto se usó de verdad en las líneas ya "abierto" de un
    servicio, SIN legalizarlo - a diferencia de /api/legalizacion/confirmar,
    que además cierra el servicio de una vez. Para dejar una comparación
    real y completa (pedido vs. usado) lista para revisar y confirmar en
    vivo en la demo, en vez de una pantalla vacía o con todo en cero."""
    with Sesion() as s:
        actualizadas = 0
        for item in datos.usos:
            l = (s.query(LineaServicio)
                 .filter_by(servicio_id=datos.servicio_id, articulo_codigo=item.articulo_codigo,
                           estado="abierto").first())
            if l:
                l.usado = item.usado
                actualizadas += 1
        s.commit()
    return {"ok": True, "actualizadas": actualizadas}

@app.get("/api/usuarios/yo/bodegas")
def bodegas_asignadas(u: Usuario = Depends(usuario_actual)):
    """Las bodegas asignadas a la persona que tiene la sesion. Va ANTES de
    la ruta parametrizada /usuarios/{usuario_id}/bodegas a proposito: si
    quedara despues, FastAPI hace match de "yo" contra {usuario_id} primero
    y esta ruta nunca se alcanza (por eso nunca respondia para un auxiliar)."""
    with Sesion() as s:
        asigs = s.query(AsignacionBodega).filter_by(usuario_id=u.id).all()
        salida = []
        for a in asigs:
            b = s.get(Bodega, a.bodega_id)
            if b:
                refs = s.query(StockSistema).filter_by(bodega_id=b.id).count()
                item = {"id": b.id, "bodega": b.nombre_oficial,
                       "estado": b.estado, "referencias": refs}
                item.update(_contexto_bodega(s, b, refs))
                salida.append(item)
    return salida

@app.get("/api/usuarios/{usuario_id}/bodegas")
def ver_bodegas_de_usuario(usuario_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    """Para no borrar asignaciones existentes al agregar una nueva (ver
    «Asignar auditor» en el tablero de bodegas)."""
    with Sesion() as s:
        return [a.bodega_id for a in
                s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).all()]

@app.put("/api/usuarios/{usuario_id}/bodegas")
def asignar_bodegas(usuario_id: int, body: dict,
                    u: Usuario = Depends(requiere_perfil("auditor"))):
    ids = body.get("bodega_ids", [])
    with Sesion() as s:
        objetivo = s.get(Usuario, usuario_id)
        if objetivo is None:
            raise HTTPException(404, "Usuario no encontrado.")

```



```

s.query(AsignacionBodega).filter_by(usuario_id=usuario_id).delete()
for bid in ids:
    s.add(AsignacionBodega(usuario_id=usuario_id, bodega_id=bid))
s.commit()
nombre = objetivo.nombre
registrar(u, "ASIGNACION", f"Bodegas asignadas a {nombre}: {len(ids)}", "ok")
return {"ok": True}

@app.get("/api/bodegas/asignaciones")
def ver_asignaciones_completas(u: Usuario = Depends(requiere_perfil("auditor"))):
    """Vista de cobertura: por cada bodega, quien la tiene asignada (o
    nadie). Con varios auxiliares repartiendo 54 bodegas, el administrador
    necesita ver esto de un vistazo para no dejar bodegas sin nadie a cargo
    ni confundir a dos personas asignandoles las mismas por error - revisar
    ficha por ficha de cada persona no lo deja ver."""
    with Sesion() as s:
        por_bodega: dict[int, list] = {}
        for a in s.query(AsignacionBodega).all():
            por_bodega.setdefault(a.bodega_id, []).append(a.usuario_id)
        salida = []
        for b in s.query(Bodega).order_by(Bodega.nombre_oficial).all():
            personas = []
            for uid in por_bodega.get(b.id, []):
                us = s.get(Usuario, uid)
                if us:
                    personas.append({"id": us.id, "nombre": us.nombre, "perfil": us.perfil})
            salida.append({"id": b.id, "bodega": b.nombre_oficial, "asignados": personas})
        return salida

# acciones sin valor para el resumen de "Inicio": ingresos repetidos, ajustes
# de perfil y cada conteo individual (eso ya se ve en el tablero de Bodegas)
_RUIDO_ACTIVIDAD = ("INGRESO", "SEGURIDAD", "PERFIL", "CONTEO")

def _narrar_actividad(t):
    p = (t.persona or "alguien").title()
    d = t.detalle
    if t.accion == "CREACION":
        nombre = d.split(" creado,")[0]
        return f"{p} creó {nombre.title()}, pendiente de aprobación"
    if t.accion == "APERTURA":
        bodega = d.split(" abierta")[0]
        return f"{p} abrió {bodega.title()}"
    if t.accion == "AUDITORIA":
        bodega = d.replace("Recuento ciego iniciado en ", "")
        return f"{p} inició el recuento ciego en {bodega.title()}"
    if t.accion == "FIRMA":
        tipo, _, bodega = d.partition(" - ")
        verbo = "firmó el conteo de" if tipo.lower().startswith("conteo") else "firmó la auditoría de"
        return f"{p} {verbo} {bodega.title()}"
    if t.accion == "CIERRE":
        bodega = d.split(" cerrada")[0]
        return f"{bodega.title()} quedó cerrada con doble firma"
    if t.accion == "REAPERTURA":
        motivo = d.split("motivo: ")[-1]
        bodega = d.split(" reabierta")[0]
        return f"{p} reabrió {bodega.title()} - motivo: {motivo}"
    if t.accion == "REPORTE":
        if d.startswith("Consolidado generado"):
            filas = f" ({d.rsplit(' ', 1)[-1]})" if "(" in d else ""

```

```

        return f"{p} generó el consolidado para My Inventory{filas}"
    if d.startswith("Estado de bodegas exportado"):
        return f"{p} exportó el estado del tablero"
    if d.startswith("Detalle de bodega"):
        return f"{p} exportó el detalle de una bodega"
    return f"{p} generó un reporte"
if t.accion == "PEDIDO":
    return f"{p} envió un pedido al almacén"
if t.accion == "LEGALIZACION":
    if d.startswith("Ajuste por voz"):
        return f"{p} ajustó un consumo por voz"
    return f"{p} legalizó un servicio"
if t.accion == "ALERTA":
    if d.startswith("Alerta resuelta - "):
        nombre, _, msg = d[len("Alerta resuelta - "):].partition(": ")
        try:
            esperado = msg.split("alrededor de ")[1].split(".")[0]
            confirmado = msg.split("¿Confirma ")[1].rstrip("?")
            return f"Alerta resuelta: {nombre} pasó de {esperado} a {confirmado}"
        except IndexError:
            return f"{p} resolvió una alerta de {nombre}"
    return f"{p} resolvió una alerta"
if t.accion == "CORRECCION":
    cambio = d.split(" (valor anterior)")[0]
    return f"{p} corrigió un conteo: {cambio}"
if t.accion == "USUARIO":
    nombre = d.split(" creado")[0].replace("Usuario ", "")
    return f"{p} creó el usuario {nombre.title()}"
if t.accion == "ASIGNACION":
    return f"{p}: {d.lower()}"
if t.accion == "APROBACION":
    if "entra al catalogo" in d:
        nombre = d.split(" aprobado")[0]
        return f"{p} aprobó el producto {nombre.title()}"
    nombre = d.split(" rechazado")[0]
    return f"{p} rechazó el producto {nombre.title()}"
if t.accion == "AJUSTE":
    return f"{p} actualizó la configuración del sistema"
if t.accion == "SOPORTE":
    if " le escribió a " in d:
        destinatario = d.split(" le escribió a ")[1].split(":")[0]
        return f"{p} le escribió a {destinatario.title()}"
    return f"{p} reportó un problema"
return f"{p}: {d}"

@app.get("/api/trazabilidad/reciente")
def traza_reciente(u: Usuario = Depends(usuario_actual)):
    """Vista compartida para Inicio: sin datos sensibles, solo la acción y quién."""
    with Sesion() as s:
        trazas = (s.query(Traza).filter(~Traza.accion.in_(RUIDO_ACTIVIDAD))
                  .order_by(Traza.id.desc()).limit(8).all())
        return [{"hora": t.creado.strftime("%H:%M"), "persona": (t.persona or "").title(),
                  "accion": t.accion, "detalle": _narrar_actividad(t), "tipo": t.tipo}
                for t in trazas]

# ----- aprobaciones en paralelo -----
@app.get("/api/aprobaciones")
def listar_aprobaciones(estado: str = "pendiente",
                        u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        salida = []
        q = s.query(Aprobacion)
        if estado:
            q = q.filter_by(estado=estado)

```

```

for a in q.order_by(Aprobacion.id.desc()).all():
    b = s.get(Bodega, a.bodega_id) if a.bodega_id else None
    creador = s.get(Usuario, a.creado_por_id) if a.creado_por_id else None
    salida.append({"id": a.id, "tipo": a.tipo, "nombre": a.nombre,
                  "unidad_medida": a.unidad_medida,
                  "cantidad_inicial": a.cantidad_inicial,
                  "bodega": b.nombre_oficial if b else None,
                  "creado_por": creador.nombre if creador else "?",
                  "hora": a.creado.strftime("%H:%M")})

return salida

@app.post("/api/aprobaciones/{aprobacion_id}/aprobar")
def aprobar(aprobacion_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        a = s.get(Aprobacion, aprobacion_id)
        if a is None or a.estado != "pendiente":
            raise HTTPException(404, "Aprobacion no encontrada o ya resuelta.")
        a.estado = "aprobado"
        a.resuelto_por_id = u.id
        a.resuelto = ahora()
        if a.tipo == "producto" and a.articulo_codigo:
            existe = s.query(StockSistema).filter_by(
                articulo_codigo=a.articulo_codigo, bodega_id=a.bodega_id).first()
            if not existe:
                s.add(StockSistema(articulo_codigo=a.articulo_codigo,
                                   bodega_id=a.bodega_id, cantidad_sd=0))
            if a.conteo_id:
                c = s.get(Conteo, a.conteo_id)
                if c:
                    c.estado = "confirmado"
        elif a.tipo == "bodega":
            s.add(Bodega(nombre_oficial=a.nombre, estado="pendiente"))
        s.commit()
        nombre = a.nombre
        registrar(u, "APROBACION", f"{nombre} aprobado y entra al catalogo oficial", "ok")
    return {"ok": True}

@app.post("/api/aprobaciones/{aprobacion_id}/rechazar")
def rechazar(aprobacion_id: int, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        a = s.get(Aprobacion, aprobacion_id)
        if a is None or a.estado != "pendiente":
            raise HTTPException(404, "Aprobacion no encontrada o ya resuelta.")
        a.estado = "rechazado"
        a.resuelto_por_id = u.id
        a.resuelto = ahora()
        if a.conteo_id:
            c = s.get(Conteo, a.conteo_id)
            if c:
                c.estado = "rechazado"
        s.commit()
        nombre = a.nombre
        registrar(u, "APROBACION", f"{nombre} rechazado", "alerta")
    return {"ok": True}

@app.post("/api/soporte/reportar")
def reportar_problema(body: dict, u: Usuario = Depends(usuario_actual)):
    detalle = (body.get("detalle") or "").strip() or "Sin detalle."
    registrar(u, "SOPORTE", f"{u.nombre} reporto: {detalle}", "alerta")
    return {"ok": True}

```

```

def _enviar_correo_real(destinatario: str, asunto: str, cuerpo: str) -> tuple[bool, str]:
    """Envía un correo de verdad por la API de Brevo (HTTPS) si hay
    credenciales configuradas (BREVO_API_KEY). Antes se intentó SMTP
    directo a Gmail (Render bloquea el puerto 587, "Network is
    unreachable") y luego la API de Resend (Cloudflare la rechazaba con
    "error code: 1010", probablemente por la reputación de las IPs
    compartidas de Render) - ninguna de las dos depende del código, así
    que se cambió de proveedor otra vez. Sin BREVO_API_KEY configurada,
    no intenta nada: el llamador sigue funcionando igual que antes (solo
    trazabilidad), el mismo patrón de "se degrada sin romperse" que ya
    usa el agente sin GOOGLE_API_KEY. Devuelve (enviado, motivo-si-fallo)
    - el motivo es temporal mientras se termina de calibrar el envío
    real."""
    api_key = os.getenv("BREVO_API_KEY", "").strip()
    remitente = os.getenv("BREVO_FROM", os.getenv("SMTP_CORREO", "")).strip()
    if not api_key or not remitente or not destinatario:
        return False, "sin BREVO_API_KEY/BREVO_FROM configuradas"
    payload = json.dumps({
        "sender": {"name": "CuentaVoz", "email": remitente},
        "to": [{"email": destinatario}],
        "subject": asunto, "textContent": cuerpo,
    }).encode("utf-8")
    petition = urllib.request.Request(
        "https://api.brevo.com/v3/smtp/email", data=payload, method="POST",
        headers={"api-key": api_key, "Content-Type": "application/json",
                 "Accept": "application/json",
                 "User-Agent": "CuentaVoz/1.0 (+https://cuentavoz.onrender.com)"}
    )
    # El contenedor de Render no tiene ruta de salida por IPv6, pero la
    # resolución de nombres a veces sí devuelve una dirección IPv6 (y
    # Python la intenta primero) - eso da exactamente "Network is
    # unreachable" aunque el IPv4 normal funcione bien. Se fuerza IPv4
    # solo durante esta conexión puntual.
    getaddrinfo_original = socket.getaddrinfo

    def _forzar_ipv4(host, port, family=0, type=0, proto=0, flags=0):
        return getaddrinfo_original(host, port, socket.AF_INET, type, proto, flags)

    socket.getaddrinfo = _forzar_ipv4
    try:
        with urllib.request.urlopen(petition, timeout=10) as resp:
            return 200 <= resp.status < 300, ""
    except urllib.error.HTTPError as e:
        motivo = f"HTTP {e.code}: {e.read().decode('utf-8', 'ignore')[:300]}"
        print(f"[correo] no se pudo enviar a {destinatario}: {motivo}")
        return False, motivo
    except Exception as e:
        motivo = f"{type(e).__name__}: {e}"
        print(f"[correo] no se pudo enviar a {destinatario}: {motivo}")
        return False, motivo
    finally:
        socket.getaddrinfo = getaddrinfo_original

@app.post("/api/sopORTE/mensaje-administrador")
def mensaje_administrador(body: dict, u: Usuario = Depends(usuario_actual)):
    """Escribirle al administrador. El mensaje siempre queda trazado -
    se ve en Ajustes → Registro de trazabilidad - y además se intenta un
    correo real (vía Brevo) si el servidor tiene BREVO_API_KEY
    configurada. Sin esa credencial (el caso normal por ahora: no
    depende de registrar una cuenta con un tercero), el frontend abre el
    correo ya instalado en el dispositivo de quien envía, con el mensaje
    listo - así igual llega un correo real, desde la cuenta de esa
    persona, sin que CuentaVoz tenga que gestionar ningún servicio de
    correo por su cuenta."""

```

```

mensaje = (body.get("mensaje") or "").strip()
if not mensaje:
    raise HTTPException(400, "Escriba el mensaje antes de enviarlo.")
with Sesion() as s:
    admin = (s.query(Usuario)
              .filter(Usuario.perfil == "auditor", Usuario.activo == 1, Usuario.id != u.id)
              .first())
    if not admin:
        raise HTTPException(404, "No hay un administrador disponible por ahora.")
    nombre_admin, correo_admin, admin_id = admin.nombre, admin.correo, admin.id
    s.add(MensajeSoporte(remite_id=u.id, destinatario_id=admin_id, mensaje=mensaje))
    s.commit()
    registrar(u, "SOPORTE", f"{u.nombre} le escribió a {nombre_admin}: {mensaje}")
    cuerpo = f"Mensaje enviado desde CuentaVoz por {u.nombre}.\n\n{mensaje}"
    correo_enviado, _motivo = _enviar_correo_real(correo_admin, "Mensaje desde CuentaVoz", cuerpo)
    return {"ok": True, "administrador": nombre_admin, "correo_enviado": correo_enviado}

@app.get("/api/soporte/mensajes")
def mis_mensajes_soporte(u: Usuario = Depends(usuario_actual)):
    """La bandeja de "Soporte en vivo" dentro de la app: para un auxiliar,
    lo que ha escrito y si ya le respondieron; para el administrador, lo
    que le han escrito y lo que falta por responder."""
    with Sesion() as s:
        if u.perfil == "auditor":
            filas = (s.query(MensajeSoporte)
                     .filter(MensajeSoporte.destinatario_id == u.id)
                     .order_by(MensajeSoporte.creado.desc()).all())
        else:
            filas = (s.query(MensajeSoporte)
                     .filter(MensajeSoporte.remite_id == u.id)
                     .order_by(MensajeSoporte.creado.desc()).all())
        ids_personas = {m.remite_id for m in filas} | {m.destinatario_id for m in filas}
        personas = {p.id: p.nombre for p in
                    s.query(Usuario).filter(Usuario.id.in_(ids_personas)).all()}
        return [{"id": m.id,
                  "de": personas.get(m.remite_id, "?"),
                  "para": personas.get(m.destinatario_id, "?"),
                  "mensaje": m.mensaje, "respuesta": m.respuesta,
                  "creado": m.creado.strftime("%Y-%m-%d %H:%M"),
                  "respondido": m.respondido.strftime("%Y-%m-%d %H:%M") if m.respondido else None}
                for m in filas]

@app.post("/api/soporte/mensajes/{mensaje_id}/responder")
def responder_mensaje_soporte(mensaje_id: int, body: dict, u: Usuario = Depends(usuario_actual)):
    """Solo quien recibió el mensaje puede responderlo - no hace falta
    pedir perfil "auditor" aparte: si no es el destinatario, no puede."""
    respuesta = (body.get("respuesta") or "").strip()
    if not respuesta:
        raise HTTPException(400, "Escriba la respuesta antes de enviarla.")
    with Sesion() as s:
        m = s.get(MensajeSoporte, mensaje_id)
        if not m or m.destinatario_id != u.id:
            raise HTTPException(404, "Ese mensaje no existe o no es suyo para responder.")
        m.respuesta = respuesta
        m.respondido = ahora()
        s.commit()
        remitente = s.get(Usuario, m.remite_id)
        nombre_remitente = remitente.nombre if remitente else "?"
        correo_remitente = remitente.correo if remitente else None
        registrar(u, "SOPORTE", f"{u.nombre} le respondió a {nombre_remitente}: {respuesta}")
        return {"ok": True, "destinatario": nombre_remitente, "correo_destinatario": correo_remitente}

```

```

@app.get("/api/soporte/administrador")
def administrador_de_turno(u: Usuario = Depends(usuario_actual)):
    """Con quien puede contactarse cualquier persona (no solo el
    administrador) desde Ayuda - sin exponer el resto del listado de
    usuarios, que si es exclusivo del administrador. Si quien pregunta ya
    es administrador, no tiene sentido mostrarse a si mismo como contacto:
    se busca a OTRO administrador, y si no hay ninguno mas, no hay a quien
    escribirle (para eso esta la mesa de ayuda de Colsubsidio aparte)."""
    with Sesion() as s:
        admin = (s.query(Usuario)
                  .filter(Usuario.perfil == "auditor", Usuario.activo == 1,
                          Usuario.id != u.id)
                  .first())
    if not admin:
        return {"nombre": None, "correo": None, "es_usted": u.perfil == "auditor"}
    return {"nombre": admin.nombre, "correo": admin.correo, "es_usted": False}

@app.get("/api/usuarios/yo")
def ver_perfil(u: Usuario = Depends(usuario_actual)):
    from agente.cerebro import VOCES, VOZ_DEFEECTO
    dias_pin = (ahora() - (u.pin_actualizado or ahora())).days
    # idioma_voz guardaba un codigo de idioma de navegador (es-MX, es-CO...)
    # de cuando la voz salia por speechSynthesis; ahora guarda la clave de
    # la voz neuronal elegida (kore, puck...). Una cuenta con el valor
    # viejo cae a la voz por defecto en vez de romper el selector.
    voz = u.idioma_voz if u.idioma_voz in VOCES else VOZ_DEFEECTO
    return {"nombre": u.nombre, "correo": u.correo, "telefono": u.telefono,
            "codigo": u.codigo, "perfil": u.perfil,
            "ultimo_acceso": u.ultimo_acceso.strftime("%Y-%m-%d %H:%M") if u.ultimo_acceso else None,
            "pin_vence_en_dias": max(90 - dias_pin, 0),
            "huella_registrada": bool(u.huella_registrada),
            "idioma_voz": voz, "velocidad_voz": u.velocidad_voz,
            "confirmacion_hablada": bool(u.confirmacion_hablada)}

@app.get("/api/usuarios/yo/resumen")
def resumen_inicio(u: Usuario = Depends(usuario_actual)):
    hoy = ahora().date()
    with Sesion() as s:
        n_bodegas = s.query(AsignacionBodega).filter_by(usuario_id=u.id).count()
        sesiones_usr = [x.id for x in s.query(SesionConteo).filter_by(usuario_id=u.id).all()]
        ref_hoy = 0
        if sesiones_usr:
            ref_hoy = sum(1 for c in (s.query(Conteo)
                                      .filter(Conteo.sesion_id.in_(sesiones_usr),
                                              Conteo.estado == "confirmado").all())
                          if c.creado.date() == hoy)
        alertas_abiertas = s.query(Alerta).filter_by(resuelta=0).count()
        historial = s.query(HistorialCierre).all()
        exact_mes = (round(sum(h.exactitud for h in historial) / len(historial), 1)
                      if historial else 100.0)
    return {"bodegas_asignadas": n_bodegas, "referencias_hoy": ref_hoy,
            "alertas_por_revisar": alertas_abiertas, "exactitud_mes": exact_mes}

@app.put("/api/usuarios/yo/preferencias")
def guardar_preferencias(body: dict, u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        for k in ("idioma_voz", "velocidad_voz"):
            if k in body:
                setattr(usr, k, body[k])
        if "confirmacion_hablada" in body:
            usr.confirmacion_hablada = 1 if body["confirmacion_hablada"] else 0

```

```

        s.commit()
    return {"ok": True}

@app.post("/api/usuarios/yo/huella/opciones")
@limiter.limit("10/minute")
def opciones_registro_huella(request: Request, u: Usuario = Depends(usuario_actual)):
    """Paso 1 del registro: reto de WebAuthn para navigator.credentials.create()."""
    with Sesion() as s:
        existentes = [c.credential_id for c in
                      s.query(CredencialWebAuthn).filter_by(usuario_id=u.id).all()]
    return huella.opciones_registro(u.id, u.nombre, existentes)

@app.post("/api/usuarios/yo/huella/verificar")
@limiter.limit("10/minute")
def verificar_registro_huella(request: Request, credencial: dict, u: Usuario =
Depends(usuario_actual)):
    """Paso 2: verifica la respuesta del navegador y guarda la credencial."""
    try:
        verificado = huella.verificar_registro(u.id, credencial)
    except Exception as e:
        raise HTTPException(400, f"No se pudo registrar la huella: {e}")
    with Sesion() as s:
        s.add(CredencialWebAuthn(usuario_id=u.id, credential_id=verificado["credential_id"],
                                public_key=verificado["public_key"],
                                sign_count=verificado["sign_count"]))
        usr = s.get(Usuario, u.id)
        usr.huella_registrada = 1
        s.commit()
    registrar(u, "SEGURIDAD", f"{u.nombre} registro una huella en este dispositivo", "ok")
    return {"ok": True}

@app.delete("/api/usuarios/yo/huella")
def eliminar_huella(u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        s.query(CredencialWebAuthn).filter_by(usuario_id=u.id).delete()
        usr = s.get(Usuario, u.id)
        usr.huella_registrada = 0
        s.commit()
    registrar(u, "SEGURIDAD", f"{u.nombre} elimino sus huellas registradas", "ok")
    return {"ok": True}

@app.post("/api/usuarios/yo/cerrar-todas")
def cerrar_todas_sesiones(u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        usr.version_token = (usr.version_token or 0) + 1
        s.commit()
    registrar(u, "SEGURIDAD", "Sesion cerrada en todos los dispositivos")
    return {"ok": True, "token": crear_token(usr)}

@app.put("/api/usuarios/yo/pin")
@limiter.limit("5/minute")
def cambiar_pin(request: Request, body: dict, u: Usuario = Depends(usuario_actual)):
    pin = str(body.get("pin", ""))
    if len(pin) < 6:
        raise HTTPException(400, "El PIN debe tener al menos 6 digitos.")
    with Sesion() as s:
        usr = s.get(Usuario, u.id)

```

```

        # sin esto, cualquiera con el token abierto (tableta compartida sin
        # bloquear, token filtrado) podia tomarse la cuenta por completo con
        # solo mandar un PIN nuevo - ni siquiera hacia falta saber el viejo.
        if not verificar_clave(str(body.get("pin_actual", "")), usr.clave_hash):
            raise HTTPException(401, "El PIN actual no es correcto.")
        usr.clave_hash = hash_clave(pin) # nunca en texto plano
        usr.version_token = (usr.version_token or 0) + 1 # cierra sesiones con el PIN viejo
        s.commit()
        nuevo_token = crear_token(usr)
        registrar(u, "SEGURIDAD", "PIN actualizado")
        return {"ok": True, "token": nuevo_token}

def _tipo_imagen_real(contenido: bytes) -> str | None:
    """El Content-Type que manda el navegador lo elige quien sube el
    archivo, no el archivo: antes se guardaba y se devolvía tal cual al
    pedir la foto, así que subir algo con Content-Type "text/html" lo
    serviría después como HTML. Aquí se detecta el tipo real por los
    primeros bytes (firma del formato), no por lo que diga la cabecera."""
    if contenido.startswith(b"\xff\xd8\xff"):
        return "image/jpeg"
    if contenido.startswith(b"\x89PNG\r\n\x1a\n"):
        return "image/png"
    if contenido.startswith((b"GIF87a", b"GIF89a")):
        return "image/gif"
    if contenido[:4] == b"RIFF" and contenido[8:12] == b"WEBP":
        return "image/webp"
    return None

@app.post("/api/usuarios/yo/foto")
async def subir_foto(foto: UploadFile = File(...),
                    u: Usuario = Depends(usuario_actual)):
    contenido = await foto.read()
    if len(contenido) > 3 * 1024 * 1024:
        raise HTTPException(400, "La foto no puede pesar mas de 3 MB.")
    tipo_real = _tipo_imagen_real(contenido)
    if tipo_real is None:
        raise HTTPException(400, "El archivo no es una imagen valida (JPEG, PNG, GIF o WebP).")
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        usr.foto = contenido
        usr.foto_tipo = tipo_real
        s.commit()
    return {"ok": True}

@app.get("/api/usuarios/yo/foto")
def ver_foto(u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        usr = s.get(Usuario, u.id)
        if not usr.foto:
            raise HTTPException(404, "Sin foto.")
        return Response(content=usr.foto, media_type=usr.foto_tipo or "image/jpeg")

@app.get("/api/voz/voces")
def voces_disponibles(u: Usuario = Depends(usuario_actual)):
    from agente.cerebro import VOCES, VOZ_DEFECTO
    return {"voces": [{"clave": k, **v} for k, v in VOCES.items()],
            "defecto": VOZ_DEFECTO}

```



```

class HablarIn(BaseModel):
    texto: str
    voz: str = "kore"

@app.post("/api/voz/hablar")
def api_hablar(body: HablarIn, u: Usuario = Depends(usuario_actual)):
    from agente.cerebro import sintetizar_voz
    texto = (body.texto or "").strip()
    if not texto:
        raise HTTPException(400, "Falta el texto.")
    wav = sintetizar_voz(texto, body.voz)
    if wav is None:
        raise HTTPException(503, "Voz neuronal no disponible en este momento.")
    return Response(content=wav, media_type="audio/wav")

def _filtro_traza(s, persona: str, accion: str, rango: str):
    q = s.query(Traza)
    if persona:
        q = q.filter_by(persona=persona)
    if accion:
        q = q.filter_by(accion=accion)
    if rango == "hoy":
        q = q.filter(Traza.creado >= ahora().replace(hour=0, minute=0, second=0))
    elif rango == "semana":
        q = q.filter(Traza.creado >= ahora() - timedelta(days=7))
    elif rango == "mes":
        q = q.filter(Traza.creado >= ahora() - timedelta(days=30))
    return q

@app.get("/api/trazabilidad")
def ver_traza(persona: str = "", accion: str = "", rango: str = "",
              u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        q = _filtro_traza(s, persona, accion, rango)
        return [{"id": t.id, "hora": t.creado.strftime("%H:%M:%S"),
                  "persona": t.persona, "accion": t.accion,
                  "detalle": t.detalle, "tipo": t.tipo}
                for t in q.order_by(Traza.id.desc()).limit(200)]

@app.get("/api/trazabilidad/exportar")
def exportar_traza(persona: str = "", accion: str = "", rango: str = "",
                  formato: str = "xlsx",
                  u: Usuario = Depends(requiere_perfil("auditor"))):
    import pandas as pd
    from servicios.archivos import guardar_df
    with Sesion() as s:
        q = _filtro_traza(s, persona, accion, rango)
        filas = [{"fecha": t.creado.strftime("%Y-%m-%d %H:%M:%S"), "persona": t.persona,
                  "accion": t.accion, "detalle": t.detalle}
                for t in q.order_by(Traza.id.desc()).all()]
        df = pd.DataFrame(filas)
        marca = ahora().strftime("%Y%m%d_%H%M")
        ruta = f"reportes/trazabilidad_{marca}.{formato}"
        guardar_df(ruta, df, formato)
        registrar(u, "REPORTE", f"Registro de trazabilidad exportado: {ruta} ({len(df)} filas)")
    return {"archivo": ruta, "filas": len(df)}

@app.get("/api/ajustes")

```

```

def ver_ajustes(u: Usuario = Depends(usuario_actual)):
    with Sesion() as s:
        offline = s.get(ConfigClave, "offline")
        usuarios_activos = s.query(Usuario).filter_by(activo=1).count()
        aprobaciones_pend = s.query(Aprobacion).filter_by(estado="pendiente").count()
        pedidos_pend = (s.query(LineaServicio.numero_pedido)
                        .filter_by(estado="pendiente_aprobacion")
                        .distinct().count())
    return {"umbral": round(umbral_actual() * 100),
            "bloquear_negativos": True, "confirmar_alertas": True,
            "offline": (offline.valor == "1") if offline else True,
            "version": "1.0.0", "modelo": os.getenv("MODELO", "gemini-flash-latest"),
            "idioma_voz": os.getenv("IDIOMA_VOZ", "es-CO"),
            "base_datos": "SQLite",
            "usuarios_activos": usuarios_activos,
            "aprobaciones_pendientes": aprobaciones_pend + pedidos_pend}

@app.put("/api/ajustes")
def guardar_ajustes(body: dict, u: Usuario = Depends(requiere_perfil("auditor"))):
    with Sesion() as s:
        if "umbral" in body:
            valor = str(max(1, min(500, float(body["umbral"]))) / 100)
            existente = s.get(ConfigClave, "umbral_anomalia")
            if existente:
                existente.valor = valor
            else:
                s.add(ConfigClave(clave="umbral_anomalia", valor=valor))
        if "offline" in body:
            valor = "1" if body["offline"] else "0"
            existente = s.get(ConfigClave, "offline")
            if existente:
                existente.valor = valor
            else:
                s.add(ConfigClave(clave="offline", valor=valor))
        s.commit()
    registrar(u, "AJUSTE", "Configuracion del sistema actualizada", "ok")
    return {"ok": True}

# ----- tablero en vivo -----
conexiones: list[WebSocket] = []

def estado_bodegas():
    with Sesion() as s:
        return [{"bodega": b.nombre_oficial, "estado": b.estado, "id": b.id}
                for b in s.query(Bodega).order_by(Bodega.nombre_oficial).all()]

async def difundir_estado():
    for ws in list(conexiones):
        try:
            await ws.send_json(estado_bodegas())
        except Exception:
            if ws in conexiones:
                conexiones.remove(ws)

@app.websocket("/api/bodegas/estado")
async def ws_estado(ws: WebSocket):
    from seguridad import verificar_token
    if not verificar_token(ws.query_params.get("token")):

```

```

        await ws.close(code=1008)
        return
    await ws.accept()
    conexiones.append(ws)
    await ws.send_json(estado_bodegas())
    try:
        while True:
            await ws.receive_text()
    except WebSocketDisconnect:
        if ws in conexiones:
            conexiones.remove(ws)

```

12.20 Pruebas automáticas

El código base de la sección 12 no incluiría el proyecto entero si no pasara sus propias pruebas: 14 archivos, todos automáticos, que corren sin depender de Gemini ni de internet. No se reproducen aquí completos (son código de prueba, no el producto), pero cada uno indica exactamente qué garantiza:

- **backend/tests/test_interprete.py:** números en palabras, unidades, cada intención.
- **backend/tests/test_cerebro.py:** limpieza de negaciones, respaldo sin Gemini.
- **backend/tests/test_conciliacion.py:** coincidencia difusa contra el catálogo real.
- **backend/tests/test_orquestador_desambiguacion.py:** `_resolver_candidato/_elegir` (funciones puras, sin base de datos).
- **backend/tests/test_agente_conversacion.py:** conversaciones completas de extremo a extremo vía `/api/agente/turno`.
- **backend/tests/test_asistente_abre_bodega.py:** el micrófono suelto (AsistenteVoz) también puede abrir una bodega.
- **backend/tests/test_asistente_contextual.py:** el asistente entiende la pantalla en la que está la persona.
- **backend/tests/test_consulta_articulo_sugiere_bodega.py:** buscar un nombre de bodega en el buscador de artículos sugiere la bodega.
- **backend/tests/test_crear_bodega.py:** alta de bodega pendiente de aprobación, por voz y por formulario.
- **backend/tests/test_previsualizar_reporte_por_voz.py:** generar y previsualizar un reporte pidiéndolo hablado.
- **backend/tests/test_regresiones_seguridad.py:** autorización, sesión, IDOR, WebSocket.
- **backend/tests/test_regresiones_legalizacion.py:** idempotencia del cierre de servicio.
- **frontend/src/interpreteLocal.test.js:** los mismos casos que `test_interprete.py`, para confirmar que el puerto a JS se comporta igual que el original en Python.
- **frontend/src/confirmacionVoz.test.js:** reconocimiento de confirmaciones y negaciones dichas en voz alta.

Se corren con: `cd backend && .venv/Scripts/python -m pytest -q (backend) y cd frontend && npm test (frontend, node --test).`

12.21 Frontend: el puente con la API

Archivo: `frontend/src/api.js`

```
// IPv4 explícito, no "localhost": en Windows el navegador suele resolver
// "localhost" a ::1 (IPv6), y uvicorn por defecto solo escucha en 127.0.0.1.
export const BASE = import.meta.env.VITE_API_URL || "http://127.0.0.1:8000";

export function guardarToken(t) {
  localStorage.setItem("cv_token", t);
}
export function leerToken() {
  return localStorage.getItem("cv_token");
}
export function borrarToken() {
  localStorage.removeItem("cv_token");
}

// La sesión completa (token + datos del usuario), no solo el token: sin
// esto, reabrir la app (o perder la señal justo al recargar) obligaba a
// iniciar sesión otra vez contra el backend - imposible sin red, aunque
// la persona ya se hubiera identificado minutos antes en el mismo equipo.
// Con la sesión guardada, App.jsx entra directo sin depender de la red.
const CLAVE_SESION = "cv_sesion";

export function guardarSesion(s) {
  localStorage.setItem(CLAVE_SESION, JSON.stringify(s));
  if (s?.token) guardarToken(s.token); // cv_token queda siempre en sync
}
export function leerSesion() {
  try { return JSON.parse(localStorage.getItem(CLAVE_SESION) || "null"); }
  catch (_) { return null; }
}
export function borrarSesion() {
  localStorage.removeItem(CLAVE_SESION);
  borrarToken();
}
/** Cambiar el PIN o "cerrar todas las sesiones" emite un token nuevo sin
    volver a pasar por /api/ingresar: hay que refrescar también la sesión
    cacheada, no solo el token suelto, o la próxima vez que la app arranque
    sin red restauraría el token viejo ya invalidado. */
export function actualizarTokenSesion(nuevoToken) {
  guardarToken(nuevoToken);
  const s = leerSesion();
  if (s) guardarSesion({ ...s, token: nuevoToken });
}

/** fetch() rechaza con un TypeError cuando la red falla (Wi-Fi caído, DNS,
    CORS bloqueado) - la especificación lo llama "network error". Detectarlo
    así (en vez de por texto del mensaje) es lo que ya usaba Conteo.jsx para
    activar el modo sin conexión; se comparte aquí para que todo el frontend
    reconozca el mismo caso una sola vez. */
export function esFalloRed(error) {
  return error?.esFalloRed === true || error instanceof TypeError
    || /failed to fetch|networkerror|load failed/i.test(error?.message || "");
}

/** El error técnico crudo en inglés que fetch() lanza tal cual, traducido a
    algo que un auxiliar en una bodega con mal Wi-Fi pueda entender. Marca el
    error con esFalloRed=true para que quien lo reciba después de pedir()
    (que ya perdió el TypeError original) lo siga reconociendo con
```

```

    esFalloRed(), en vez de tener que repetir la detección por texto. */
function errorDeRed() {
  const error = new Error("Sin conexión con el servidor. Revise el Wi-Fi e intente de nuevo.");
  error.esFalloRed = true;
  return error;
}

/** Llama al backend adjuntando el token de la sesión. */
export async function pedir(ruta, opciones = {}, token) {
  const t = token || leerToken();
  // arma las opciones completas ANTES del try: si algo aquí lanza (un
  // "opciones.headers" mal formado, por ejemplo), es un error de
  // programación real y no debe disfrazarse de fallo de red.
  const opcionesCompletas = {
    ...opciones,
    headers: {
      "Content-Type": "application/json",
      ...(t ? { Authorization: `Bearer ${t}` } : {}),
      ...opciones.headers,
    },
  };
  let res;
  try {
    res = await fetch(`${BASE}${ruta}`, opcionesCompletas);
  } catch (error) {
    if (esFalloRed(error)) throw errorDeRed();
    throw error;
  }
  if (!res.ok) {
    // 401 de verdad (no un fallo de red: el servidor sí respondió) significa
    // que el servidor ya no reconoce este token - ni reintentar ni seguir
    // usando la sesión cacheada sirve de nada; se limpia para que la
    // próxima carga pida iniciar sesión de nuevo en vez de quedar
    // atascada reusando un token que el backend rechaza siempre.
    if (res.status === 401) borrarSesion();
    let detalle = `Error ${res.status}`;
    try {
      const j = await res.json();
      detalle = j.detail || j.detalle || detalle;
    } catch (_) {}
    throw new Error(detalle);
  }
  return res.json();
}

export async function ingresar(usuario, clave) {
  const cuerpo = new URLSearchParams({ username: usuario, password: clave });
  let res;
  try {
    res = await fetch(`${BASE}/api/ingresar`, { method: "POST", body: cuerpo });
  } catch (error) {
    if (esFalloRed(error)) throw errorDeRed();
    throw error;
  }
  if (!res.ok) throw new Error("Usuario o clave incorrectos.");
  return res.json();
}

// «respaldo» son las opciones que la pantalla ya tiene mostradas en
// pantalla ({ opciones, opcionesPara }); si el backend se reinició y
// perdió la memoria de la conversación, las recupera de ahí en vez de
// preguntar «¿cuál de los dos?» otra vez sin recordar nada.
export const enviarTurno = (texto, sesionId, token, respaldo = {}) =>
  pedir("/api/agente/turno", {
    method: "POST",
    body: JSON.stringify({

```

```

    texto, sesion_id: sesionId,
    opciones_pendientes: respaldo.opciones || null,
    opciones_para: respaldo.opcionesPara || null,
    bodega_id_respaldo: respaldo.bodegaId || null,
    bodega_nombre_respaldo: respaldo.bodegaNombre || null,
    preparacion_respaldo: respaldo.preparacion || null,
    porciones_respaldo: respaldo.porciones || null,
  }},
}, token);

// Para pantallas que no cuentan ni piden (Inicio, Ajustes, Ayuda, Reportes,
// Panel): una pregunta suelta o una orden de navegar, sin el estado de
// conversación multi-turno que sí necesita enviarTurno. Ver AsistenteVoz.jsx.
export const preguntarAsistente = (texto, vista, token) =>
  pedir("/api/agente/asistente", {
    method: "POST",
    body: JSON.stringify({ texto, vista }),
  }, token);

export const abrirBodega = (bodega, token) =>
  pedir("/api/bodegas/abrir", {
    method: "POST",
    body: JSON.stringify({ bodega }),
  }, token);

// Resuelve un nombre dicho/escrito al nombre real del catálogo, sin abrir
// nada - para poder confirmar "¿abro X?" con el nombre que de verdad se
// va a usar, no con lo que se haya transcrito tal cual (ver Conteo.jsx).
export const buscarBodega = (bodega, token) =>
  pedir("/api/bodegas/buscar", {
    method: "POST",
    body: JSON.stringify({ bodega }),
  }, token);

/** Descarga un reporte generado por el backend. Un <a href> comun no sirve
    aquí: el endpoint exige el token en la cabecera, y un enlace del
    navegador no lo puede enviar (por eso terminaba en una pagina en blanco
    pidiendo sesion). Se trae como blob autenticado y se dispara la
    descarga nativa del navegador, sin salir de la aplicacion. */
export async function descargarReporte(archivo, token) {
  const t = token || leerToken();
  let res;
  try {
    res = await fetch(
      `${BASE}/api/reportes/descargar?archivo=${encodeURIComponent(archivo)}`,
      { headers: { Authorization: `Bearer ${t}` } }
    );
  } catch (error) {
    if (esFalloRed(error)) throw errorDeRed();
    throw error;
  }
  if (!res.ok) throw new Error("No se pudo descargar el archivo.");
  const blob = await res.blob();
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = archivo.split("/").pop();
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(url), 4000);
}

```

12.22 Frontend: voz, huella e intérprete local

Archivo: `frontend/src/voz.js` — `escuchar()` (Web Speech API) y `hablar()` — ahora con voz neuronal real de Gemini, con la voz del navegador como respaldo si falla.

```
/** Escuchar y hablar. Las dos piezas de la capa de voz. */
import { BASE, leerToken } from "./api";
// quitarTildes/esAfirmacion/esNegacion viven en su propio módulo sin
// dependencias (ni de "./api" ni del navegador) para poder probarlas con
// node --test igual que interpreteLocal.js - se reexportan aquí para que
// quien ya las importaba desde "./voz" siga funcionando igual.
export { quitarTildes, esAfirmacion, esNegacion } from "./confirmacionVoz.js";

const IDIOMA_ESCUCHA = "es-CO"; // como reconoce lo que usted dice: fijo
let vozNeuronal = "kore"; // que voz neuronal usa CuentaVoz al hablar: elegible
const TASA = { lenta: 0.82, normal: 1.02, rapida: 1.3 };
let tasaActual = TASA.normal;
let confirmacionHablada = true;

// A que genero suena cada voz neuronal - para que, si la neuronal falla y
// hay que caer al respaldo del navegador, se busque una voz de navegador
// del mismo genero en vez de una elegida al azar. No es la MISMA voz (eso
// no se puede: el navegador no tiene "Kore"), pero al menos no suena como
// si de repente le estuviera hablando otra persona.
const _GENERO_VOZ = { kore: "femenina", aoede: "femenina", puck: "masculina", charon: "masculina" };

const Reconocedor =
  typeof window !== "undefined" &&
  (window.SpeechRecognition || window.webkitSpeechRecognition);

export const vozDisponible = () => Boolean(Reconocedor);

/** MiPerfil llama esto una vez cargadas (o cambiadas) las preferencias.
  "idioma" quedo con ese nombre por compatibilidad con quien ya llama
  configurarVoz(), pero ahora guarda la clave de la voz neuronal
  (kore, puck...), no un codigo de idioma de navegador. */
export function configurarVoz({ idioma, velocidad, confirmacionHablada: ch } = {}) {
  if (idioma && idioma !== vozNeuronal) {
    vozNeuronal = idioma;
    // la voz de respaldo se habia elegido (o cacheado como "no hay") con
    // la preferencia VIEJA - sin esto, cambiar la voz en Mi perfil no
    // cambiaba nada si en algun momento se habia caido al respaldo antes.
    vozNavegadorBuscada = false;
    vozNavegadorElegida = null;
  }
  if (velocidad && TASA[velocidad]) tasaActual = TASA[velocidad];
  if (ch !== undefined) confirmacionHablada = ch;
}

/* — Respaldo: voz del navegador (speechSynthesis) —
  Solo se usa si la voz neuronal no responde (sin internet, sin llave de
  Gemini configurada, o la API falla) - para que el conteo por voz nunca
  se quede completamente mudo. Suena mas robotica, pero es mejor que
  nada mientras se recupera la conexion. */
let vozNavegadorElegida = null;
let vozNavegadorBuscada = false;

// Nombres tipicos de voces de navegador/SO en espanol, para adivinar de
// que genero suena cada una - el estandar Web Speech API no expone genero,
// asi que esto es lo mas cercano a saberlo sin reproducirla.
const _NOMBRES_FEM =
  /sabina|helenah|elvira|laura|m[oó]nica|paulina|luc[ií]a|elena|isabela|marisol|conchita|pen[eé]lope|espe
  ranza|camila|valentina|female|mujer/;
const _NOMBRES_MASC = /ra[uú]l|jorge|diego|pablo|carlos|alonso|enrique|miguel|male|hombre/;
```

```

function _puntuarVozNavegador(v, generoPreferido) {
  const lang = (v.lang || "").toLowerCase();
  const nombre = (v.name || "").toLowerCase();
  let p = 0;
  if (lang.startsWith("es-mx") || lang.startsWith("es-419")) p += 20;
  else if (lang.startsWith("es-")) p += 10;
  else if (lang.startsWith("es")) p += 5;
  else return -1;
  if (/natural|online|neural|wavenet|premium/.test(nombre)) p += 30;
  if (/google/.test(nombre)) p += 10;
  if (/desktop|compact/.test(nombre)) p -= 20;
  // que suene del mismo genero que la voz neuronal elegida en Mi perfil
  // pesa mas que cualquier otra cosa: es justo lo que hace notorio el
  // cambio de voz cuando toca caer al respaldo.
  if (generoPreferido === "femenina" && _NOMBRES_FEM.test(nombre)) p += 50;
  else if (generoPreferido === "masculina" && _NOMBRES_MASC.test(nombre)) p += 50;
  else if (generoPreferido === "femenina" && _NOMBRES_MASC.test(nombre)) p -= 50;
  else if (generoPreferido === "masculina" && _NOMBRES_FEM.test(nombre)) p -= 50;
  return p;
}

function _elegirVozNavegador() {
  if (typeof window === "undefined" || !window.speechSynthesis) return null;
  const voces = window.speechSynthesis.getVoices();
  if (!voces.length) return null;
  const generoPreferido = _GENERO_VOZ[vozNeuronal];
  const candidatas = voces.map((v) => ({ v, p: _puntuarVozNavegador(v, generoPreferido) }));
  .filter((x) => x.p >= 0).sort((a, b) => b.p - a.p);
  return candidatas[0]?v || null;
}

/** Devuelve una promesa que se resuelve cuando termina de decirlo (no
    cuando empieza) - quien vaya a escuchar() justo después de hablar()
    necesita esperar a que la pregunta se termine de decir, o el
    micrófono arranca mientras el navegador todavía está sonando y se
    pierde la respuesta. */
function hablarConNavegador(texto) {
  return new Promise((resolve) => {
    if (typeof window === "undefined" || !window.speechSynthesis) { resolve(); return; }
    if (!vozNavegadorBuscada) {
      vozNavegadorElegida = _elegirVozNavegador();
      if (!vozNavegadorElegida) {
        window.speechSynthesis.onvoiceschanged = () => { vozNavegadorElegida = _elegirVozNavegador();
        };
      } else {
        vozNavegadorBuscada = true;
      }
    }
    window.speechSynthesis.cancel();
    const u = new SpeechSynthesisUtterance(texto);
    u.lang = vozNavegadorElegida?.lang || "es-MX";
    u.rate = tasaActual;
    u.pitch = 1.03;
    if (vozNavegadorElegida) u.voice = vozNavegadorElegida;
    u.onend = resolve;
    u.onerror = resolve;
    window.speechSynthesis.speak(u);
  });
}

/* — Voz neuronal real (Gemini TTS, via el backend) — */
let audioActual = null;

// La síntesis real tarda unos segundos (viaja al backend y a Gemini): si

```



```

// mientras tanto la persona ya cambió de pantalla, esa respuesta ya no
// tiene contexto y no debe hablar sola de repente sobre una pantalla que
// ya no está en uso. Cada hablar() (y detenerVoz()) sube esta generación;
// una respuesta que llega tarde, de una generación vieja, se descarta en
// vez de reproducirse.
let generacion = 0;

// Si detenerVoz() corta el audio a la mitad (pause() no dispara "ended"),
// esto es lo que despierta a quien esté esperando hablar() - si no, un
// cambio de pantalla a mitad de frase dejaría esa promesa colgada para
// siempre.
let resolverAudioActual = null;

/** App.jsx la llama en cada cambio de pantalla: corta lo que se esté
    reproduciendo y invalida cualquier hablar() todavía en camino. */
export function detenerVoz() {
  generacion++;
  if (audioActual) { audioActual.pause(); audioActual.currentTime = 0; }
  if (typeof window !== "undefined" && window.speechSynthesis) window.speechSynthesis.cancel();
  resolverAudioActual?.();
  resolverAudioActual = null;
}

async function hablarConNeuronal(texto, miGeneracion) {
  const res = await fetch(`${BASE}/api/voz/hablar`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${leerToken()}`,
    },
    body: JSON.stringify({ texto, voz: vozNeuronal }),
  });
  if (!res.ok) throw new Error("voz neuronal no disponible");
  const blob = await res.blob();
  if (miGeneracion !== generacion) return; // se cambió de pantalla mientras se generaba
  const url = URL.createObjectURL(blob);
  if (audioActual) { audioActual.pause(); audioActual.currentTime = 0; }
  const audio = new Audio(url);
  audio.playbackRate = tasaActual;
  audioActual = audio;
  // Espera a que el audio TERMINE de sonar, no solo a que arranque -
  // audio.play() por sí solo se resuelve apenas empieza a reproducir, y
  // quien llama a hablar() esperando poder escuchar() justo después
  // necesita que la pregunta ya se haya terminado de decir.
  await new Promise((resolve) => {
    resolverAudioActual = resolve;
    audio.addEventListener("ended", resolve, { once: true });
    audio.addEventListener("error", resolve, { once: true });
    audio.play().catch(resolve);
  });
  resolverAudioActual = null;
  URL.revokeObjectURL(url);
}

// Tope de seguridad: ninguna frase de esta app tarda tanto en decirse.
// Sin esto, si el navegador alguna vez no dispara "ended"/"onend" (se ha
// visto en algunos Chrome tras minimizar la pestaña, y en entornos sin
// salida de audio real), hablar() quedaría esperando para siempre y el
// micrófono que depende de "await hablar()" nunca llegaría a abrirse.
const TOPE_HABLAR_MS = 12000;

/** Habla el texto y devuelve una promesa que se resuelve cuando termina
    de decirlo (o, como mucho, a los TOPE_HABLAR_MS). Casi todo el código
    la llama "al aire" (sin await) y eso sigue funcionando igual; quien
    necesite escuchar() justo después de preguntar algo sí debe

```

```

    esperarla, o el micrófono arranca mientras todavía se está hablando y
    se pierde parte de la respuesta. */
export function hablar(texto, { forzar = false } = {}) {
  if (!texto) return Promise.resolve();
  if (!forzar && !confirmacionHablada) return Promise.resolve();
  generacion++;
  const miGeneracion = generacion;
  const intento = hablarConNeuronal(texto, miGeneracion)
    .catch(() => { if (miGeneracion === generacion) return hablarConNavegador(texto); });
  return Promise.race([
    intento,
    new Promise((resolve) => setTimeout(resolve, TOPE_HABLAR_MS)),
  ]);
}

/** Escucha una frase y la devuelve como texto. */
export function escuchar({ alTexto, alEstado, alError }) {
  if (!Reconocedor) {
    alError?.("Este navegador no reconoce voz. Use Chrome o Edge.");
    return null;
  }
  const rec = new Reconocedor();
  rec.lang = IDIOMA_ESCUCHA;
  rec.interimResults = true;
  rec.continuous = false;
  rec.maxAlternatives = 1;

  rec.onstart = () => alEstado?.("escuchando");
  rec.onresult = (e) => {
    const r = e.results[e.results.length - 1];
    const texto = r[0].transcript.trim();
    if (r.isFinal) {
      alEstado?.("procesando");
      alTexto?.(texto);
    } else {
      alEstado?.("escuchando", texto);
    }
  };
  rec.onerror = (e) => {
    alEstado?.("listo");
    if (e.error === "not-allowed")
      alError?.("Debe permitir el micrófono en el navegador.");
    else if (e.error !== "aborted" && e.error !== "no-speech")
      alError?.(`No pude escuchar (${e.error}).`);
  };
  rec.onend = () => alEstado?.("listo");
  try {
    rec.start();
  } catch (_) {}
  return rec;
}

```

Archivo: `frontend/src/webauthn.js` — El lado cliente de WebAuthn: conversión base64url ↔ buffers para hablar con el navegador.

```

// Ingreso con huella del dispositivo (WebAuthn real: Windows Hello, Touch
// ID, huella de Android) - conversión manual base64url <-> ArrayBuffer en
// vez de PublicKeyCredential.parseCreationOptionsFromJSON/toJSON() para que
// funcione en cualquier navegador con soporte WebAuthn, no solo los más
// recientes.
import { pedir } from "../api";

function base64urlABuffer(b64url) {
  const relleno = "=".repeat((4 - (b64url.length % 4)) % 4);
  const b64 = (b64url + relleno).replace(/-/g, "+").replace(/_/g, "/");

```

```

const cruda = atob(b64);
const buf = new Uint8Array(cruda.length);
for (let i = 0; i < cruda.length; i++) buf[i] = cruda.charCodeAt(i);
return buf.buffer;
}

function bufferABase64url(buf) {
  const bytes = new Uint8Array(buf);
  let str = "";
  for (const b of bytes) str += String.fromCharCode(b);
  return btoa(str).replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");
}

export function soportaHuella() {
  return typeof window !== "undefined" && !!window.PublicKeyCredential && !!navigator.credentials;
}

/** No basta con que el navegador soporte WebAuthn: hay que confirmar que
    este equipo tiene un autenticador integrado configurado (Windows Hello,
    Touch ID, huella de Android). Sin esto el botón aparecería para
    cualquiera y fallaría en silencio en equipos sin lector ni PIN de
    Windows Hello activado. */
export async function hayAutenticadorDisponible() {
  if (!soportaHuella()) return false;
  try {
    return await PublicKeyCredential.isUserVerifyingPlatformAuthenticatorAvailable();
  } catch {
    return false;
  }
}

function opcionesCreacionDesdeJSON(json) {
  return {
    ...json,
    challenge: base64urlABuffer(json.challenge),
    user: { ...json.user, id: base64urlABuffer(json.user.id) },
    excludeCredentials: (json.excludeCredentials || []).map((c) => ({
      ...c, id: base64urlABuffer(c.id),
    })),
  };
}

function opcionesPeticiónDesdeJSON(json) {
  return {
    ...json,
    challenge: base64urlABuffer(json.challenge),
    allowCredentials: (json.allowCredentials || []).map((c) => ({
      ...c, id: base64urlABuffer(c.id),
    })),
  };
}

function credencialRegistroAJSON(cred) {
  return {
    id: cred.id, rawId: bufferABase64url(cred.rawId), type: cred.type,
    response: {
      clientDataJSON: bufferABase64url(cred.response.clientDataJSON),
      attestationObject: bufferABase64url(cred.response.attestationObject),
    },
  };
}

function credencialIngresoAJSON(cred) {
  return {

```

```

    id: cred.id, rawId: bufferABase64url(cred.rawId), type: cred.type,
    response: {
      clientDataJSON: bufferABase64url(cred.response.clientDataJSON),
      authenticatorData: bufferABase64url(cred.response.authenticatorData),
      signature: bufferABase64url(cred.response.signature),
      userHandle: cred.response.userHandle ? bufferABase64url(cred.response.userHandle) : null,
    },
  };
}

/** Registra la huella de este dispositivo para el usuario ya autenticado. */
export async function registrarHuellaDispositivo(token) {
  const opciones = await pedir("/api/usuarios/yo/huella/opciones", { method: "POST" }, token);
  const credencial = await navigator.credentials.create({
    publicKey: opcionesCreacionDesdeJSON(opciones),
  });
  if (!credencial) throw new Error("No se pudo crear la credencial de huella.");
  await pedir("/api/usuarios/yo/huella/verificar", {
    method: "POST", body: JSON.stringify(credencialRegistroAJSON(credencial)),
  }, token);
}

/** Ingresa con la huella ya registrada. Devuelve {token, perfil, usuario},
    igual que ingresar() con clave. */
export async function ingresarConHuella(usuarioCodigo) {
  const { reto_id, opciones } = await pedir(
    `/api/auth/huella/opciones?usuario=${encodeURIComponent(usuarioCodigo)}`);
  const credencial = await navigator.credentials.get({
    publicKey: opcionesPeticionDesdeJSON(opciones),
  });
  if (!credencial) throw new Error("No se pudo leer la huella.");
  return pedir("/api/auth/huella/verificar", {
    method: "POST",
    body: JSON.stringify({ reto_id, credencial: credencialIngresoAJSON(credencial) }),
  });
}

```

Archivo: [frontend/src/interpreteLocal.js](#) — Puerto 1:1 a JavaScript de servicios/interprete.py — corre en el navegador para cuando ni el backend es alcanzable (modo sin conexión, vista 5.10).

```

/** Puerto a JavaScript de backend/servicios/interprete.py: el mismo
    reconocimiento de números en palabras e intenciones frecuentes, pero
    corriendo en el navegador en vez del servidor - para que el modo sin
    conexión entienda texto escrito de verdad (no solo un formulario
    numérico) incluso con la red completamente apagada, cero llamadas al
    backend. Mantener esto en sincronía con interprete.py a mano: no hay
    una fuente única compartida entre Python y JS todavía.

    No reemplaza a Gemini ni al intérprete del servidor: es el mismo
    respaldo de última instancia, ahora también disponible sin backend. */

const NUMEROS = new Map(Object.entries({
  cero: 0, un: 1, una: 1, uno: 1, dos: 2, tres: 3, cuatro: 4,
  cinco: 5, seis: 6, siete: 7, ocho: 8, nueve: 9, diez: 10,
  once: 11, doce: 12, trece: 13, catorce: 14, quince: 15,
  dieciseis: 16, "dieciséis": 16, diecisiete: 17, dieciocho: 18,
  diecinueve: 19, veinte: 20, veinticinco: 25, treinta: 30,
  cuarenta: 40, cincuenta: 50, sesenta: 60, setenta: 70,
  ochenta: 80, noventa: 90, cien: 100, ciento: 100,
  doscientos: 200, quinientos: 500, mil: 1000,
})));

const UNIDADES = new Map(Object.entries({
  kilo: "Kilogram", kilos: "Kilogram", kilogramo: "Kilogram",
  kilogramos: "Kilogram", kg: "Kilogram",

```

```

    litro: "Liter", litros: "Liter", l: "Liter",
    unidad: "Unidad", unidades: "Unidad",
    porcion: "Portion", porciones: "Portion", "porción": "Portion",
  }));

const DECENAS = [20, 30, 40, 50, 60, 70, 80, 90];
const CANONICAS = new Set(["Kilogram", "Liter", "Unidad", "Portion"]);

const RE_NUMERO_DIGITOS = /-?\d+(?:[.]\d+)?/;
const RE_PUNTUACION_BORDE = /^[.,;?¡!]+|[.,;?¡!]+$ /g;

function quitarPuntuacion(palabra) {
  return palabra.replace(RE_PUNTUACION_BORDE, "");
}

/** La PRIMERA cantidad de la frase. Ignora números que son parte del
    nombre del producto («cazuela 16 onzas», «tabla 50x38»). */
function numero(texto) {
  const t = texto.toLowerCase();
  const m = RE_NUMERO_DIGITOS.exec(t);
  if (m) {
    const val = parseFloat(m[0].replace(",", "."));
    return t.slice(0, m.index).includes("menos") ? -val : val;
  }
}

const palabras = t.trim().split(/\s+/).filter(Boolean).map(quitarPuntuacion);
let total = null;
for (let i = 0; i < palabras.length; i++) {
  const p = palabras[i];
  if (!NUMEROS.has(p)) continue;
  const v = NUMEROS.get(p);
  total = v;
  // «ciento ochenta»: centena seguida de decena
  if (total >= 100 && i + 1 < palabras.length && NUMEROS.has(palabras[i + 1])) {
    const sig = NUMEROS.get(palabras[i + 1]);
    if (sig < 100) total += sig;
    // «treinta y cinco»: decena + y + unidad
  } else if (DECENAS.includes(total) && i + 2 < palabras.length
    && palabras[i + 1] === "y" && NUMEROS.has(palabras[i + 2])) {
    total += NUMEROS.get(palabras[i + 2]);
  }
  break; // lo demas pertenece al nombre del producto
}
if (total !== null && t.includes("menos")) total = -Math.abs(total);
return total;
}

function unidad(texto) {
  for (let p of texto.toLowerCase().replace(/\?/g, " ").trim().split(/\s+/)) {
    p = p.replace(/^[.,;]+|[.,;]+$/g, "");
    if (UNIDADES.has(p)) return UNIDADES.get(p);
  }
  return null;
}

/** Lo que sea que devuelva el agente (humano o el modelo) para la unidad,
    lo lleva a uno de los 4 valores canónicos de la base de datos. */
export function normalizarUnidad(dicha) {
  if (!dicha) return dicha;
  if (CANONICAS.has(dicha)) return dicha;
  const clave = String(dicha).trim().toLowerCase();
  return UNIDADES.has(clave) ? UNIDADES.get(clave) : dicha;
}

```

```

const PALABRAS_FUERA = new Set([
  ...NUMEROS.keys(), ...UNIDADES.keys(),
  "de", "del", "la", "el", "los", "las", "hay", "en", "y", "por",
  "confirmo", "confirmar", "son", "hoy", "preparamos", "quiero",
  "pedir", "insumos", "para", "cuanto", "cuánto", "cuantos",
  "iniciar", "conteo", "abrir", "bodega", "corregir", "ultimo",
  "último", "no", "uy", "que", "qué", "me", "genera", "generame",
  "consolidado", "reporte", "ayuda", "menos", "onzas", "onza",
]);

function sinRuido(t) {
  const sinDigitos = t.replace(RE_NUMERO_DIGITOS, " ");
  return sinDigitos.trim().split(/\s+/).filter(Boolean)
    .filter(w => !PALABRAS_FUERA.has(quitarPuntuacion(w)))
    .join(" ").trim();
}

const contiene = (f, claves) => claves.some((k) => f.includes(k));

/** El mismo respaldo de última instancia que interprete.py, corriendo en
    el navegador: entiende lo básico sin backend ni internet. */
export function interpretarLocal(frase) {
  const f = frase.toLowerCase().trim();
  const cant = numero(f);
  const uni = unidad(f);

  if (contiene(f, ["iniciar conteo", "abrir bodega", "abrir la bodega"])) {
    return { intencion: "navegar", bodega_texto: sinRuido(f),
      respuesta_hablada: "Voy a abrir esa bodega." };
  }
  if (contiene(f, ["preparamos", "vamos a preparar", "pedir insumos"])) {
    return { intencion: "pedir", preparacion: sinRuido(f),
      porciones: cant ? Math.trunc(cant) : 0,
      respuesta_hablada: "Le calculo los insumos según la receta." };
  }
  if (f.startsWith("confirmo") || ["confirmar", "si", "sí", "correcto", "confirmo."].includes(f)) {
    return { intencion: "confirmar", respuesta_hablada: "Listo." };
  }
  if (contiene(f, ["corregir", "no son", "no, son", "uy no", "uy, no",
    "esta mal", "está mal", "me equivoque", "cambiar"])) {
    return { intencion: "corregir", cantidad: cant, unidad: uni,
      respuesta_hablada: "Corrijo ese registro." };
  }
  if (contiene(f, ["cuanto", "cuánto", "cuantos", "donde hay", "dónde hay"])) {
    return { intencion: "consultar", articulo_texto: sinRuido(f),
      respuesta_hablada: "Reviso el inventario." };
  }
  if (contiene(f, ["reporte", "consolidado", "archivo", "imprim"])) {
    return { intencion: "reporte", respuesta_hablada: "Genero el archivo." };
  }
  if (f.includes("ayuda") || f.includes("no entiendo")) {
    return { intencion: "ayuda", respuesta_hablada: "Con gusto. ¿Qué necesita saber?" };
  }
  if (cant !== null) {
    return { intencion: "contar", articulo_texto: sinRuido(f),
      cantidad: cant, unidad: uni, respuesta_hablada: "" };
  }
  return { intencion: "ayuda", respuesta_hablada: "Perdón, no le entendí bien. ¿Me lo repite?" };
}

```

Archivo: [frontend/src/confirmacionVoz.js](#) — Reconoce confirmaciones y negaciones dichas en voz alta («sí», «cancele»), compartido por los diálogos que preguntan antes de una acción irreversible.

```

/** Detectar un "sí"/"no" hablado o escrito - sin depender de nada más
    (ni del navegador, ni de la API), para poder probarlo con node --test

```

```

igual que interpreteLocal.js. voz.js reexporta estas mismas funciones
para quien ya las importaba desde ahí. */

/** Sin esto, "sí" (como lo transcribe la voz, con tilde) NUNCA hacía
match contra /^(si|sí|...)\b/: en JavaScript \b y \w son ASCII puros,
así que "í" no cuenta como letra y la frontera de palabra justo
después de la tilde no se reconoce - "sí" (y hasta "sí ver detalle")
fallaban en seco, y solo "si" sin tilde funcionaba. Bug real,
encontrado en producción, que llevaba viva desde el principio en
varios confirmaciones habladas de la app. */
export function quitarTildes(t) {
  return String(t).normalize("NFD").replace(/[\u0300-\u036f]/g, "");
}

const AFIRMACIONES_BASE = ["si", "confirmo", "confirmar", "claro", "dale", "correcto", "listo"];
const FRASES_AFIRMATIVAS = ["eso es", "así es"];

/** ¿La respuesta hablada/escrita es un "sí" a lo que se le preguntó?
"extra" agrega palabras válidas solo en ese contexto puntual - p. ej.
"ver" cuando la pregunta fue "¿ve el detalle?", o el texto del botón
de turno ("Enviar", "Solicitar...") para que decir esa misma palabra
(en cualquier conjugación: "envíalo", "enviar", "envíelo") confirme
igual que tocar el botón. */
export function esAfirmacion(texto, extra = []) {
  const t = quitarTildes(String(texto).toLowerCase().replace(/[,;:!?i]/g, ""));
  const palabras = t.split(/\s+/).filter(Boolean);
  if (!palabras.length || palabras.includes("no")) return false;
  const extrasNorm = extra.map((e) => quitarTildes(e.toLowerCase()));
  const afirmaciones = [...AFIRMACIONES_BASE, ...extrasNorm];
  if (palabras.some((p) => afirmaciones.includes(p))) return true;
  if (FRASES_AFIRMATIVAS.some((f) => t.includes(f))) return true;
  // conjugaciones del extra ("enviar" -> envía/envíalo/enviando...): se
  // compara por raíz, quitando la terminación de infinitivo -ar/-er/-ir,
  // en vez de enumerar cada forma verbal a mano.
  const raices = extrasNorm.map((e) => e.replace(/(ar|er|ir)$/, "")).filter((r) => r.length >= 3);
  return raices.length > 0 && palabras.some((p) => raices.some((r) => p.startsWith(r)));
}

/** ¿La respuesta es un "no" limpio? (primera palabra, no en cualquier
parte de la frase, para no confundir "no sé, tal vez" con un cierre). */
export function esNegacion(texto) {
  const t = quitarTildes(String(texto).toLowerCase().replace(/[,;:!?i]/g, ""));
  const palabras = t.split(/\s+/).filter(Boolean);
  return palabras[0] === "no";
}

```

12.23 Frontend: piezas de layout compartidas

Archivo: `frontend/src/Marco.jsx`

```

/** El armazón que comparten todas las vistas: barra superior con la
sección, la faja dorada de marca y el logo de Colsubsidio. */

import React from "react";
export default function Marco({ titulo, chip, children }) {
  return (
    <>
      <div className="barra-sup">
        <div className="barra-sup-inner">
          <h1>{titulo}</h1>
          {chip && <span className={`chip ${chip.tipo} || ""}`>{chip.texto}</span>}
          
        </div>
      </div>
      {children}
    </>
  );
}

```

```

    <div className="faja" />
    <div className="area">{children}</div>
  </>
);
}

```

Archivo: frontend/src/BarraLateral.jsx

```

import { useEffect, useState } from "react";
import { MENU } from "../App";
import Icono from "../Iconos";
import { BASE, leerToken } from "../api";

export default function BarraLateral({ activo, usuario, token, onNavegar }) {
  const esAuditor = usuario?.perfil === "auditor";
  const items = MENU.filter((m) => !m.soloAuditor || esAuditor);
  const [fotoUrl, setFotoUrl] = useState(null);

  // Igual que en Mi perfil: <img src> no puede mandar el header
  // Authorization que /foto exige, así que se trae como blob autenticado.
  // Se vuelve a pedir cuando Mi perfil avisa que hubo una foto nueva.
  useEffect(() => {
    function cargar() {
      fetch(`${BASE}/api/usuarios/yo/foto`, {
        headers: { Authorization: `Bearer ${token || leerToken()}` },
      }).then((r) => (r.ok ? r.blob() : null))
        .then((b) => setFotoUrl((prev) => {
          if (prev) URL.revokeObjectURL(prev);
          return b ? URL.createObjectURL(b) : null;
        }))
        .catch(() => setFotoUrl(null));
    }
    cargar();
    window.addEventListener("cuentavoz:foto-actualizada", cargar);
    return () => window.removeEventListener("cuentavoz:foto-actualizada", cargar);
  }, [token]);

  return (
    <nav className="sidebar">
      <div className="marca">
        
        <span>
          <b>CuentaVoz</b>
          <i>Tu voz cuenta</i>
        </span>
      </div>
      <hr />

      <ul>
        {items.map((m) => (
          <li
            key={m.id}
            className={m.id === activo ? "sel" : ""}
            onClick={() => onNavegar(m.id)}
            role="button"
            tabIndex={0}
            aria-current={m.id === activo ? "page" : undefined}
            // El menú principal solo tenía onClick - inalcanzable con
            // teclado (Tab no lo enfocaba, nada respondía a Enter/Espacio).
            // Para quien navega sin mouse (movilidad reducida, o un lector
            // de pantalla), esto es el único menú de toda la app: sin
            // esto, no había forma de cambiar de pantalla sin mouse.
            onKeyDown={(e) => {
              if (e.key === "Enter" || e.key === " ") {
                e.preventDefault();
                onNavegar(m.id);
              }
            }}
          >

```



```

    }
  }}
  >
  <span className="icono-menu">
    <Icono nombre={m.id} tam={19} />
  </span>
  <span>{m.titulo}</span>
</li>
)}}
</ul>

<footer>
  {fotoUrl ? (
    <img src={fotoUrl} alt="" className="avatar"
      style={{ objectFit: "cover" }} />
  ) : (
    <span className="avatar">
      {(usuario?.nombre || "?")[0].toUpperCase()}
    </span>
  )}
  <b style={{ fontSize: ".86rem", textTransform: "capitalize" }}>
    {usuario?.nombre}
  </b>
  <small>
    {usuario?.perfil === "auditor"
      ? "Administrador de bodega"
      : "Auxiliar de inventarios"}
  </small>
  
</footer>
</nav>
);
}

```

Archivo: frontend/src/Dialogo.jsx

```

import { useState, useRef, useEffect } from "react";
import { escuchar, hablar, esAfirmacion, esNegacion } from "../voz";
import { interpretarLocal } from "../interpreteLocal";
import Icono from "../Iconos";

/** Reemplaza a window.prompt()/window.confirm(): un modal con el estilo
    propio de la app, en vez del cuadro genérico y feo del navegador.
    Uso: <Dialogo titulo="..." mensaje="..." conCampo onAceptar={...} onCancel={...} />

    conVoz agrega el mismo micrófono que ya existe en el resto de la app,
    con el mismo patrón de confirmación hablada que usa el agente al abrir
    una bodega: lo dicho llena el campo (visible, editable) Y el agente
    pregunta en voz alta "¿confirma X?" - un "sí" acepta directo, un "no"
    descarta y deja intentar de nuevo. El botón de Aceptar sigue ahí como
    respaldo (para quien prefiera escribir, o si la voz no entendió el
    sí/no). No se usa en los diálogos "Escribir en vez de hablar": esos ya
    son el respaldo a prueba de fallos cuando la voz no funciona. */
export default function Dialogo({
  titulo, mensaje, conCampo = false, conVoz = false, multilinea = false, tipo = "text",
  valorInicial = "", placeholder = "", textoAceptar = "Aceptar",
  textoCancelar = "Cancelar", peligro = false, confirmarAlAbrir = false, onAceptar, onCancel,
  // Nombre accesible del campo para un lector de pantalla - por defecto se
  // usa "mensaje" (funciona bien cuando es una pregunta corta, como "¿Cuántas
  // porciones son en realidad?"), pero en varios diálogos "mensaje" es un
  // párrafo explicativo largo o hasta el contenido de un mensaje ajeno
  // (ver Mensajes.jsx "Responder mensaje": ahí incluye el mensaje ORIGINAL
  // completo) - leerlo entero cada vez que el campo recibe foco es
  // ruidoso e inútil como etiqueta. Estos casos pasan su propio
  // etiquetaCampo, corto y específico.
  etiquetaCampo,

```

```

})) {
  const [valor, setValor] = useState(valorInicial);
  const [escuchando, setEscuchando] = useState(false);
  const [confirmando, setConfirmando] = useState(false);
  const [errorVoz, setErrorVoz] = useState("");
  // Igual que en el selector de bodega de Conteo: cada escucha lleva su
  // propio número, y si el micrófono anterior entrega su resultado tarde
  // (por ejemplo, mientras todavía se está diciendo "¿confirma X?") se
  // descarta en vez de procesarse como si fuera un nuevo valor dictado.
  const idEscucha = useRef(0);
  const modalRef = useRef(null);

  // Accesibilidad: quien navega con teclado o con un lector de pantalla
  // necesita que Escape cierre el modal (como cualquier diálogo nativo),
  // y que el foco entre al modal apenas aparece - sin esto, un lector de
  // pantalla seguía anunciando el contenido de atrás como si el modal ni
  // existiera. Si hay un campo, ese ya se autoenfoca solo (ver más abajo);
  // si no lo hay, el modal mismo recibe el foco.
  useEffect(() => {
    function alTecla(e) { if (e.key === "Escape") onCancel(); }
    window.addEventListener("keydown", alTecla);
    return () => window.removeEventListener("keydown", alTecla);
  }, [onCancelar]);

  useEffect(() => {
    if (!conCampo) modalRef.current?.focus();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  function aceptar(valorAUsar = valor) {
    if (conCampo && !String(valorAUsar).trim()) return;
    onAceptar(valorAUsar);
  }

  async function preguntarConfirmacion(valorDictado, miId) {
    setConfirmando(true);
    // Con un campo de varias líneas (un mensaje libre, no un dato corto)
    // no se repite todo lo dictado - sería una lectura larga, y encima
    // alarga la ventana en la que la persona ya contestó "sí"/"envíalo"
    // pero el micrófono todavía no se reabrió (seguía sonando la
    // pregunta). Una confirmación corta reduce esa ventana.
    const pregunta = multilinea ? "¿Confirma el mensaje?" : `¿Confirma «${valorDictado}»?`;
    // hay que esperar a que termine de decir la pregunta antes de abrir
    // el micrófono - si arranca apenas empieza a sonar, se pierde el
    // principio de la respuesta.
    await hablar(pregunta);
    if (miId !== idEscucha.current) return;
    let obtuvoResultado = false;
    let yaReintento = false;
    escuchar({
      alTexto: (respuesta) => {
        if (miId !== idEscucha.current) return;
        obtuvoResultado = true;
        setConfirmando(false);
        // El botón de este diálogo en particular puede decir "Enviar",
        // "Solicitar", "Crear"... no solo "Aceptar" - decir esa misma
        // palabra (en cualquier conjugación) también debe confirmar,
        // igual que tocar el botón.
        if (esAfirmacion(respuesta, [textoAceptar])) {
          aceptar(valorDictado);
        } else if (esNegacion(respuesta)) {
          setValor("");
          setErrorVoz("Cancelado. Dígalo de nuevo cuando quiera, o escríbalo.");
        } else {
          setErrorVoz("No le entendí un «sí» o un «no». Use el botón para confirmar.");
        }
      }
    });
  }

```

```

    },
    alEstado: (e) => {
      setEscuchando(e === "escuchando");
      // El reconocedor del navegador se cierra solo tras unos segundos
      // de silencio, sin avisar con un error (voz.js lo ignora a
      // propósito) - sin esto, la pregunta se quedaba "colgada" en
      // pantalla diciendo que estaba esperando un sí/no, cuando en
      // realidad el micrófono ya se había apagado solo. Se vuelve a
      // preguntar y a escuchar sola, en vez de obligar a tocar el
      // micrófono de nuevo (lo que arrancaba una dictada nueva y
      // borraba el mensaje ya armado).
      if (e === "listo" && !obtuvoResultado && !yaReintento && miId === idEscucha.current) {
        yaReintento = true;
        preguntarConfirmacion(valorDictado, miId);
      }
    },
    alError: (e) => { setConfirmando(false); setErrorVoz(e); },
  });
}

function alTextoDictado(t, miId) {
  if (!t) return;
  if (tipo === "number") {
    // el reconocimiento a veces ya entrega el número tal cual ("50"),
    // y a veces la palabra ("cincuenta") - se intenta directo primero,
    // y si no es un número se reusa el mismo interprete de cantidades
    // que ya usa el conteo por voz, en vez de dejar el campo vacío.
    const directo = Number(String(t).replace(", ", "."));
    if (!Number.isNaN(directo)) {
      setValor(directo);
      preguntarConfirmacion(directo, miId);
      return;
    }
    const r = interpretarLocal(t);
    if (r.cantidad !== null) {
      setValor(r.cantidad);
      preguntarConfirmacion(r.cantidad, miId);
      return;
    }
    setErrorVoz(`No reconocí un número en «${t}». Dígallo de nuevo o escríbalo.`);
    return;
  }
  setValor(t);
  preguntarConfirmacion(t, miId);
}

function porVoz() {
  setErrorVoz("");
  const miId = ++idEscucha.current;
  escuchar({
    alTexto: (t) => {
      if (miId !== idEscucha.current) return;
      alTextoDictado(t, miId);
    },
    alEstado: (e) => setEscuchando(e === "escuchando"),
    alError: setErrorVoz,
  });
}

// Cuando el valor ya llega armado por voz desde afuera (por ejemplo,
// "respóndele a Stephanie que ya quedó asignada la bodega" en la
// bandeja de Mensajes), tocar el micrófono de este diálogo para decir
// "envíalo" NO debe funcionar como una dictada nueva - eso pisaba el
// texto ya armado con la palabra "envíalo" misma. En vez de obligar a
// usar el botón, se pregunta la confirmación de una vez al abrir, así
// "envíalo"/"sí" ya confirma lo que se armó, sin tocar nada.

```

```

useEffect(() => {
  if (confirmarAlAbrir && String(valorInicial || "").trim()) {
    preguntarConfirmacion(valorInicial, ++idEscucha.current);
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

return (
  <div className="overlay" onClick={onCancelar}>
    <div className="modal" onClick={(e) => e.stopPropagation()}
      role="dialog" aria-modal="true" aria-labelledby="dialogo-titulo"
      ref={modalRef} tabIndex=-1>
      <h2 id="dialogo-titulo">{titulo}</h2>
      {mensaje && <p style={{ whiteSpace: "pre-line" }}>{mensaje}</p>}
      {conCampo && (
        <div style={{ display: "flex", gap: 10, marginBottom: conVoz ? 6 : 16,
          alignItems: multilinea ? "flex-start" : "center" }}>
          {multilinea ? (
            <textarea autoFocus value={valor} placeholder={placeholder} rows={3}
              onChange={(e) => setValor(e.target.value)}
              aria-label={etiquetaCampo || mensaje || titulo}
              style={{ flex: 1, padding: "12px 14px",
                border: "1px solid var(--borde)", borderRadius: 12,
                fontFamily: "inherit", fontSize: "1rem", resize: "vertical" }} />
          ) : (
            <input type={tipo} autoFocus value={valor} placeholder={placeholder}
              onChange={(e) => setValor(e.target.value)}
              onKeyDown={(e) => e.key === "Enter" && aceptar()}
              aria-label={etiquetaCampo || mensaje || titulo}
              style={{ flex: 1, padding: "12px 14px",
                border: "1px solid var(--borde)", borderRadius: 12,
                textAlign: "center", fontSize: "1.05rem" }} />
          )}
          {conVoz && (
            <button type="button" className={`mic-btn ${escuchando ? "escuchando" : ""}`}
              style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
              onClick={porVoz} title="dívalo en voz alta"
              aria-label="Dívalo en voz alta"><Icono nombre="microfono" tam={22} /></button>
          )}
        </div>
      )}
      {conVoz && (
        <p className="pista" style={{ marginTop: -2, marginBottom: 14 }}>
          {confirmando ? "Diga «sí» para confirmar, o «no» para intentar de nuevo..."
            : escuchando ? "Escuchando..."
            : "Dívalo y confírmelo de viva voz, o escríbalo y use el botón."}
        </p>
      )}
      {errorVoz && <p className="error" style={{ marginBottom: 10 }}>{errorVoz}</p>}
      <div className="botones">
        <button className="btn borde" onClick={onCancelar}>{textoCancelar}</button>
        <button className={`btn ${peligro ? "oro" : ""}`} onClick={() => aceptar()}>
          {textoAceptar}
        </button>
      </div>
    </div>
  </div>
);
}

```

Archivo: **frontend/src/iconos.jsx**

```

/** Iconos del menú lateral. Uno por cada entrada, en trazo simple.
 * Heredan el color del texto (currentColor), así el CSS decide si van
 * en dorado (activo) o en gris azulado (inactivo). Sin fondo. */
const TRAZO = {
  fill: "none",

```

```

    stroke: "currentColor",
    strokeWidth: 1.8,
    strokeLinecap: "round",
    strokeLinejoin: "round",
  });

const FORMAS = {
  // Inicio: casa
  inicio: <><path d="M3 10.5 12 319 7.5" /><path d="M5.5 9.5V20h13V9.5" /><path d="M10 20v-5h4v5" /></>,
  // Pedidos: nota de pedido con esquina doblada
  pedidos: <><path d="M6 3h8l4 4v14H6z" /><path d="M14 3v4h4" /><path d="M9 12h6M9 16h4" /></>,
  // Conteo: micrófono
  conteo: <><rect x="10" y="3" width="4" height="9" rx="2" /><path d="M6.5 11a5.5 5.5 0 0 0 11 0" /><path d="M12 16.5V20" /><path d="M9 20h6" /></>,
  // Legalización: balanza (lo pedido contra lo usado)
  legalizacion: <><path d="M12 4v15" /><path d="M5 7h14" /><path d="M8.5 19h7" /><path d="M2.5 12 5 712.5 5a2.5 2.5 0 0 1-5 0z" /><path d="M16.5 12 19 712.5 5a2.5 2.5 0 0 1-5 0z" /></>,
  // Bodegas: bodega con techo a dos aguas
  bodegas: <><path d="M3 9.5 12 319 6.5" /><path d="M5 9v11h14V9" /><path d="M5 14.5h14" /><path d="M12 9v11" /></>,
  // Auditoría: lupa con visto bueno
  auditoria: <><circle cx="11" cy="10.5" r="6" /><path d="M15.5 15 20 19.5" /><path d="M8.5 10.5l2 2 3.5-4" /></>,
  // Reportes: documento con líneas
  reportes: <><rect x="5" y="3" width="14" height="18" rx="2" /><path d="M8.5 8h7M8.5 12h7M8.5 16h4" /></>,
  // Panel: barras
  panel: <><path d="M4 20h16" /><rect x="6" y="12" width="3" height="6" /><rect x="11" y="7" width="3" height="11" /><rect x="16" y="14" width="3" height="4" /></>,
  // Ajustes: engranaje
  ajustes: <><circle cx="12" cy="12" r="3.2" /><path d="M12 3v2.6M12 18.4V21M3 12h2.6M18.4 12H21M5.6 5.6l1.9 1.9M16.5 16.5l1.9 1.9M18.4 5.6l-1.9 1.9M7.5 16.5l-1.9 1.9" /></>,
  // Ayuda: interrogación en círculo
  ayuda: <><circle cx="12" cy="12" r="8.5" /><path d="M9.6 9.4a2.5 2.5 0 1 1 3.4 2.3c-.6.3-1 .9-1 1.6v.3" /><circle cx="12" cy="17" r=".9" fill="currentColor" stroke="none" /></>,
  // Mensajes: sobre de correo
  mensajes: <><rect x="3" y="5.5" width="18" height="13" rx="2" /><path d="M3.5 7 12 13l8.5-6" /></>,
  // Mi perfil: persona
  perfil: <><circle cx="12" cy="8" r="3.5" /><path d="M5 20a7 7 0 0 1 14 0" /></>,
  // Cerrar sesión: puerta con flecha de salida
  salir: <><path d="M14 4H6v16h8" /><path d="M11 12h9" /><path d="M17 8.5 20.5 12 17 15.5" /></>,
  // Candado: campo de PIN en el ingreso
  candado: <><rect x="5" y="11" width="14" height="9" rx="2" /><path d="M8 11v7.5a4 4 0 0 1 8 0v11" /></>,
  // Descargar: flecha hacia una bandeja
  descargar: <><path d="M12 3v11.5" /><path d="M7.5 10.5 12 15l4.5-4.5" /><path d="M4.5 17.5v2a2 2 0 0 2 2h11a2 2 0 0 2-2v-2" /></>,
  // Aprobar/resolver: visto bueno en círculo
  aprobar: <><circle cx="12" cy="12" r="8.5" /><path d="M8 12.3l2.6 2.6L16 9.3" /></>,
  // Rechazar: equis en círculo
  rechazar: <><circle cx="12" cy="12" r="8.5" /><path d="M9 9l6 6M15 9l-6 6" /></>,
  // Advertencia: triángulo con exclamación (reportar un problema)
  advertencia: <><path d="M12 3.5 21.5 20h-19z" /><path d="M12 9.5v5" /><circle cx="12" cy="17" r=".9" fill="currentColor" stroke="none" /></>,
  // Refrescar: dos flechas en ciclo (restaurar valores)
  refrescar: <><path d="M4 12a8 8 0 0 1 13.66-5.66L20 8.5" /><path d="M20 4v4.5h-4.5" /><path d="M20 12a8 8 0 0 1-13.66 5.66L4 15.5" /><path d="M4 20v-4.5h4.5" /></>,
  // Micrófono: mismo trazo que el icono "conteo" del menú, para que
  // TODO botón de escuchar en la app se vea igual - antes cada uno usaba
  // el emoji 🎧, que se ve distinto (y en algunos Windows, irreconocible)
  // según la fuente de emoji del sistema operativo.
  microfono: <><rect x="10" y="3" width="4" height="9" rx="2" /><path d="M6.5 11a5.5 5.5 0 0 0 11 0" /><path d="M12 16.5V20" /><path d="M9 20h6" /></>,
};

```

```
export default function Icono({ nombre, tam = 20 }) {
  const forma = FORMAS[nombre];
  if (!forma) return null;
  return (
    <svg
      width={tam}
      height={tam}
      viewBox="0 0 24 24"
      aria-hidden="true"
      style={{ flex: "none" }}
      {...TRAZO}
    >
      {forma}
    </svg>
  );
}
```

Archivo: `frontend/src/AsistenteVoz.jsx` — El micrófono reutilizable para pantallas sin una conversación en curso (Inicio, Ajustes, Ayuda, Reportes, Panel) — una pregunta suelta, no un conteo.

```
import { useState } from "react";
import { escuchar, hablar } from "../voz";
import { preguntarAsistente } from "../api";
import Icono from "../Iconos";

/** El mismo microfono que ya existe en Bodegas/Conteo/Pedido, pero para
    pantallas sin conversacion (Inicio, Ajustes, Ayuda, Reportes, Panel):
    una pregunta suelta sobre lo que hay en esta pantalla, o una orden de
    ir a otra - sin el estado de "pendiente"/"opciones" que solo tiene
    sentido en un conteo o un pedido en curso. */
export default function AsistenteVoz({ token, vista, ir, placeholder, alActualizar, ejemplos }) {
  const [texto, setTexto] = useState("");
  const [respuesta, setRespuesta] = useState("");
  const [error, setError] = useState("");
  const [escuchando, setEscuchando] = useState(false);
  const [pensando, setPensando] = useState(false);

  async function preguntar(dicho) {
    if (!dicho || !dicho.trim()) return;
    setTexto(dicho);
    setError("");
    // Algunas pantallas (ej. el filtro de Registro de trazabilidad)
    // resuelven ciertas frases del todo en el frontend, sin backend ni
    // Gemini de por medio - si un componente hijo la reconoce, llama
    // preventDefault() y avisa así que ya quedó resuelta, para no
    // preguntarle al agente algo que esta pantalla ya sabe contestar
    // sola. Sin esto, decir aquí arriba exactamente uno de los
    // ejemplos de la lista (que sí describen esas frases) no hacía
    // nada, porque solo el micrófono chiquito junto a los filtros
    // estaba conectado a esa lógica.
    const evento = new CustomEvent("cuentavoz:filtro-local", {
      detail: { texto: dicho }, cancelable: true,
    });
    window.dispatchEvent(evento);
    if (evento.defaultPrevented) {
      // El componente que lo reclamó (ej. TabTraza) es quien sabe el
      // resultado real (cuántas filas encontró) - anuncia ESO por su
      // cuenta de forma asíncrona en vez de un "listo, apliqué el
      // filtro" genérico que no dice nada útil.
      return;
    }
    setPensando(true);
    try {
      const r = await preguntarAsistente(dicho, vista, token);
      setRespuesta(r.respuesta_hablada || "");
      hablar(r.respuesta_hablada);
    }
  }
}
```

```

if (r.accion === "navegar" && r.destino && ir) {
  // ir() combina el contexto en vez de reemplazarlo: sin fijar
  // tabInicial explícitamente (aunque sea undefined), una pestaña
  // pedida en una navegacion anterior por otro camino se quedaba
  // pegada y aparecia en un destino que no la tiene. Lo mismo para
  // bodegaSugerida: sin fijarlo a undefined cuando no aplica, un
  // "abra kiosco" pedido antes se quedaba pegado y la próxima vez
  // que se entraba a Conteo por otro camino la abría sola. Lo mismo
  // para bodegaAuditar, su equivalente en Auditoría.
  ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined,
    bodegaAuditar: r.bodega_auditar || undefined });
} else if (r.archivo && r.titulo_archivo && r.pestana && ir) {
  // Cubre dos casos: "previsualizar_reporte" (pedir VER un archivo
  // ya generado) y "actualizar" con archivo (GENERAR uno nuevo por
  // voz - "genera el consolidado", "exporta las diferencias") -
  // ninguno trae "destino" (no cambian de PANTALLA: ya estamos en
  // Reportes), pero sí pueden pedirse desde OTRA pestaña de esta
  // misma pantalla. Sin este ir(), el evento de abajo llegaba a un
  // TabConsolidado que ni siquiera estaba montado - solo se dibuja
  // cuando la pestaña activa YA es "consolidado" - así que la vista
  // previa no aparecía nunca si no se estaba, por casualidad, ya en
  // esa pestaña (ni al pedir verla, ni al generarla). Va por props
  // (como bodegaSugerida) en vez del evento genérico de abajo,
  // precisamente porque necesita sobrevivir a un cambio de pestaña
  // que todavía no ocurrió cuando se dispara el evento.
  // r.titulo_archivo exigido a proposito: "exporta el análisis de
  // consumo" trae archivo+pestana ("analisis") pero SIN
  // titulo_archivo (usa su propio evento "descargar_reporte", ver
  // TabAnalisis) - sin este chequeo, ese caso también disparaba
  // este ir() con titulo undefined, y al volver a la pestaña
  // "Consolidado" a mano, TabConsolidado remontaba y previsualizaba
  // ese archivo viejo con "Vista previa · undefined" de encabezado.
  ir(vista, { tabInicial: r.pestana,
    archivoPrevisualizar: { archivo: r.archivo, titulo: r.titulo_archivo } });
}
// "actualizar": el agente cambió algo de verdad (ej. el modo sin
// conexión en Ajustes) - la pantalla que lo pidió es quien sabe
// cómo refrescar su propio dato, este componente no.
if (r.accion === "actualizar" && alActualizar) alActualizar();
// Cualquier otra acción (ej. "mostrar_cobertura"/"ocultar_cobertura"
// para abrir/cerrar un panel específico de la pantalla) se avisa
// como evento genérico - así un componente hijo que le interesa
// puede escucharlo sin que este componente necesite saber qué es
// cada pantalla ni acumular una prop nueva por cada acción. Ojo:
// "previsualizar_reporte" queda afuera a propósito (va por props,
// ver arriba) para no disparar el mismo aviso dos veces.
if (r.accion && r.accion !== "navegar" && r.accion !== "actualizar"
  && r.accion !== "previsualizar_reporte") {
  // detail: r - para acciones que traen datos propios, no solo la
  // señal de que pasó algo. Las que no lo necesitan lo ignoran.
  window.dispatchEvent(new CustomEvent(`cuentavoz:accion:${r.accion}`, { detail: r }));
}
} catch (e) {
  setError(e.message);
}
setPensando(false);
}

function porVoz() {
  escuchar({
    alTexto: preguntar,
    alEstado: (e) => setEscuchando(e === "escuchando"),
    alError: setError,
  });
}

return (

```

```

<div className="card">
  <p className="rotulo">Pregúntele al agente</p>
  <div style={{ display: "flex", gap: 10 }}>
    <input value={texto} onChange={(e) => setTexto(e.target.value)}
      onKeyDown={(e) => e.key === "Enter" && preguntar(texto)}
      placeholder={placeholder || "pregunte algo, o pida ir a otra pantalla..."}
      aria-label="Pregúntele al agente"
      style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
        border: "1px solid var(--borde)", borderRadius: 12 }} />
    <button className={mic-btn ${escuchando ? "escuchando" : ""}}
      style={{ width: 46, height: 46, fontSize: "1.2rem" }}
      onClick={porVoz} title="pregúntele por voz" aria-label="Pregúntele al agente por voz">
      <Icono nombre="microfono" tam={22} /></button>
    </div>
    <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
    {ejemplos && ejemplos.length > 0 && (
      <details style={{ marginTop: 8 }}>
        <summary className="pista" style={{ cursor: "pointer" }}>
          Ver frases exactas que entiende el agente aquí
        </summary>
        <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
          {ejemplos.map((ej, i) => (
            <li key={i} className="pista" style={{ marginBottom: 4 }}><{ej}></li>
          ))}
        </ul>
      </details>
    )}
    {pensando && <p className="cargando" style={{ marginTop: 10 }}>Pensando...</p>
    {respuesta && !pensando && (
      <>
        <p className="rotulo" style={{ marginTop: 10 }}>CuentaVoz responde</p>
        <p className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</p>
      </>
    )}
    {error && <p className="error" style={{ marginTop: 10 }}>{error}</p>}
  </div>
);
}

```

12.24 Frontend: estilo y diseño responsivo (PWA)

Archivo: [frontend/src/index.css](#) — Archivo completo real (los fragmentos de la sección 5b son un extracto pedagógico de este mismo archivo).

```

/* — Paleta oficial de Colsubsidio — */
:root {
  --azul:          #0067B1;    /* azul Colsubsidio · Pantone 2196 C */
  --azul-prof:    #10294C;
  --azul-claro:   #E4F0F9;
  --amarillo:     #FFD000;     /* amarillo Colsubsidio · Pantone 109 C */
  --amarillo-bg:  #FFF6CC;
  --amarillo-tx:  #8A6D00;
  --verde:        #1E7A46;
  --verde-bg:     #E8F5EE;
  --grafito:      #575756;     /* Cool Gray 11 C */
  --txt:          #2B2B2A;
  --borde:        #D8E2EC;
  --fondo:        #F5F7FB;
  --fuente: "Calibri", "Carlito", "Segoe UI", system-ui, sans-serif;
  --radio: 12px;
  --sombra: 0 2px 8px rgba(0, 103, 177, 0.10);

  /* paleta categórica para gráficas (validada: ΔE normal-vision >= 15) */
  --serie-1: #0067B1;
  --serie-2: #B38600;

```



```

--serie-3: #1E7A46;
--serie-4: #6B4FA0;
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
html, body, #root { height: 100%; }
body {
  font-family: var(--fuente);
  font-size: 16px;
  color: var(--txt);
  background: var(--fondo);
  -webkit-font-smoothing: antialiased;
}
button { font-family: inherit; cursor: pointer; border: none; }
input { font-family: inherit; font-size: 1rem; }

/* Accesibilidad: quien navega solo con teclado (movilidad reducida, o un
lector de pantalla que también depende del foco) necesita ver con
claridad dónde está parado - antes solo el input de ingreso tenía un
:focus explícito, todo lo demás dependía del anillo por defecto del
navegador, que sobre botones de color queda con poco contraste o casi
invisible. :focus-visible (no :focus) para no dibujar el anillo en un
clic de mouse, solo al navegar con Tab/flechas.

--azul-prof (no --amarillo) por defecto: casi toda la app vive sobre
tarjetas blancas o el fondo claro, y --amarillo ahí da 1.47:1 de
contraste (por debajo del 3:1 que exige WCAG para el aro de foco) -
el mismo error que ya se corrigió en .kpi.oro b más abajo, esta vez en
el propio indicador de foco. --azul-prof da 14.55:1 sobre blanco. La
única zona con fondo oscuro de verdad es el menú lateral, así que ahí
se vuelve a --amarillo (9.89:1 sobre su fondo azul marino). */
a:focus-visible, button:focus-visible, input:focus-visible, select:focus-visible,
textarea:focus-visible, summary:focus-visible, [tabindex]:focus-visible {
  outline: 3px solid var(--azul-prof);
  outline-offset: 2px;
}
.sidebar :focus-visible {
  outline-color: var(--amarillo);
}

/* — Estructura — */
.app-root { display: grid; grid-template-columns: 232px 1fr; height: 100dvh; }
.contenido { overflow-y: auto; overflow-x: hidden; padding: 0; display: flex; flex-direction: column; }

.barra-sup {
  background: #fff; border-bottom: 1px solid var(--borde);
  padding: 14px 22px; position: sticky; top: 0; z-index: 5;
}
.barra-sup-inner {
  max-width: 1180px; margin: 0 auto; width: 100%;
  display: flex; align-items: center; gap: 14px; flex-wrap: wrap;
}
.barra-sup h1 { font-size: 1.28rem; color: var(--azul); font-weight: 700; flex: 1 1 auto; }
.barra-sup::after { content: ""; }
.faja { height: 4px; background: var(--amarillo); }
.cs-logo { height: 22px; margin-left: auto; flex: none; }
.area { padding: 20px 22px 28px; max-width: 1180px; margin: 0 auto; width: 100%; }

/* — Barra lateral — */
.sidebar {
  background: var(--azul-prof); color: #fff;
  display: flex; flex-direction: column; padding: 16px 0;
}
.sidebar .marca { display: flex; align-items: center; gap: 10px; padding: 0 18px 14px; }

```

```

.sidebar .marca img { width: 34px; height: 34px; border-radius: 8px; }
.sidebar .marca b { font-size: 1.05rem; display: block; line-height: 1.1; }
.sidebar .marca i { font-size: .68rem; color: var(--amarillo); font-style: italic; }
.sidebar hr { border: none; border-top: 1px solid #2A4573; margin: 0 18px 10px; }
.sidebar ul { list-style: none; flex: 1; overflow-y: auto; }
.sidebar li {
  display: flex; align-items: center; gap: 10px;
  padding: 9px 18px; font-size: .88rem; color: #B9C4D6;
  border-left: 3px solid transparent; user-select: none;
}
.sidebar li:hover { background: #16305A; color: #fff; }
.sidebar li.sel {
  background: var(--azul); color: #fff; font-weight: 700;
  border-left-color: var(--amarillo);
}
.sidebar li .icono-menu {
  display: inline-flex; align-items: center; justify-content: center;
  flex: none; color: #8194B3;
}
.sidebar li:hover .icono-menu { color: #fff; }
.sidebar li.sel .icono-menu { color: var(--amarillo); }
.sidebar footer { padding: 12px 18px 0; border-top: 1px solid #2A4573; }
.sidebar footer .avatar {
  width: 32px; height: 32px; border-radius: 50%; background: var(--amarillo);
  color: var(--azul-prof); display: inline-flex; align-items: center;
  justify-content: center; font-weight: 700; margin-right: 8px;
}
.sidebar footer small { display: block; color: #9FADC4; font-size: .68rem; }
.sidebar footer img { height: 15px; margin-top: 10px; opacity: .9; }

/* — Tarjetas, chips y botones — */
.card {
  background: #fff; border: 1px solid var(--borde);
  border-radius: var(--radio); padding: 16px 18px; margin-bottom: 14px;
  /* casi toda tabla de la app vive dentro de un .card sin envoltorio propio;
  esto le da scroll horizontal contenido en vez de romper el layout de
  toda la pantalla en móviles y tablets angostos. */
  max-width: 100%; overflow-x: auto;
}
.card h3 { font-size: .98rem; margin-bottom: 12px; color: var(--txt); }
.chips { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 14px; }
.chip {
  background: var(--azul-claro); color: var(--azul); font-weight: 700;
  font-size: .8rem; padding: 6px 13px; border-radius: 20px;
}
.chip.verde { background: var(--verde-bg); color: var(--verde); }
.chip.oro { background: var(--amarillo-bg); color: var(--amarillo-tx); }
.chip.gris { background: #EFEFEF; color: var(--grafito); }
.chip.azul { background: var(--azul); color: #fff; }
.chip.borde { background: #fff; border: 1.5px solid currentColor; }
.chip.borde.azul { color: var(--azul); }

.btn {
  border-radius: var(--radio); font-weight: 700; padding: 13px 20px;
  font-size: .95rem; background: var(--azul); color: #fff;
  transition: opacity .15s; min-height: 46px;
}
.btn:active { opacity: .82; }
.btn.disabled { opacity: .45; cursor: not-allowed; }
.btn.borde { background: #fff; color: var(--azul); border: 2px solid var(--azul); }
.btn.verde { background: var(--verde); }
.btn.oro { background: #fff; color: var(--amarillo-tx); border: 2px solid var(--amarillo); }
.btn.gris { background: #fff; color: var(--grafito); border: 2px solid #C3CAD6; }
.grilla-botones { display: flex; gap: 12px; flex-wrap: wrap; margin-top: 16px; }

/* — Conversación y micrófono — */

```

```

.conteo-cols { display: grid; grid-template-columns: 1fr 300px; gap: 14px; }
.dos-columnas { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; align-items: start; }
.perfil-cols { display: grid; grid-template-columns: 260px 1fr; gap: 14px; align-items: start; }
.campos-2col { display: grid; grid-template-columns: 1fr 1fr; gap: 0 14px; }
.avatar-grande {
  width: 110px; height: 110px; border-radius: 50%; border: 3px solid var(--azul);
  display: flex; align-items: center; justify-content: center;
  font-size: 2.6rem; font-weight: 700; color: var(--azul); background: var(--azul-claro);
  margin: 0 auto;
}
.avatar-chico {
  width: 26px; height: 26px; border-radius: 50%; flex: none;
  display: inline-flex; align-items: center; justify-content: center;
  font-size: .78rem; font-weight: 700; color: var(--azul); background: var(--azul-claro);
}
.rotulo { font-size: .74rem; color: var(--grafito); font-style: italic; margin-bottom: 4px; }
.cita {
  background: var(--fondo); border: 1px solid var(--borde); border-radius: 10px;
  padding: 14px 16px; font-style: italic; color: var(--txt); font-size: .95rem;
}
.burbuja {
  background: var(--azul-claro); border: 2px solid var(--azul);
  border-radius: var(--radio); padding: 18px; text-align: center;
  color: #06315B; font-size: 1.06rem; min-height: 74px;
  display: flex; align-items: center; justify-content: center;
}
.mic-caja { display: flex; flex-direction: column; align-items: center; gap: 10px; }
.mic-btn {
  width: 116px; height: 116px; border-radius: 50%;
  background: var(--azul); color: #fff; font-size: 2.6rem;
  box-shadow: var(--sombra); display: flex; align-items: center;
  justify-content: center; user-select: none; -webkit-user-select: none;
  transition: transform .12s, background .2s;
}
.mic-btn:active { transform: scale(.94); }
.mic-btn.escuchando { background: var(--amarillo); color: var(--azul-prof); animation: latir 1.1s
infinite; }
@keyframes latir { 0%,100% { box-shadow: 0 0 0 0 rgba(255,208,0,.55); } 50% { box-shadow: 0 0 0 16px
rgba(255,208,0,0); } }
/* quien tiene un trastorno vestibular puede marearse con el pulso
continuo del micrófono escuchando - la pantalla ya avisa igual con el
texto "Escuchando..." y el cambio de color, así que el movimiento no es
la única señal y se puede apagar sin perder información. */
@media (prefers-reduced-motion: reduce) {
  .mic-btn.escuchando { animation: none; }
}
.mic-caja small { color: var(--grafito); text-align: center; font-size: .78rem; }

.registro { display: flex; align-items: center; gap: 10px; padding: 9px 12px;
  border: 1px solid var(--borde); border-radius: 8px; margin-bottom: 7px;
  background: #FAFBFD; font-size: .87rem; }
.registro.ok { width: 20px; height: 20px; border-radius: 50%; background: var(--verde);
  color: #fff; display: inline-flex; align-items: center; justify-content: center;
  font-size: .7rem; flex: none; }
.registro.cant { margin-left: auto; color: var(--grafito); font-size: .82rem; }

.etiqueta-actividad { margin-left: auto; font-weight: 700; font-size: .8rem; flex: none; }
.etiqueta-actividad.azul { color: var(--azul); }
.etiqueta-actividad.verde { color: var(--verde); }
.etiqueta-actividad.oro { color: var(--amarillo-tx); }
.etiqueta-actividad.gris { color: var(--grafito); }

.banner {
  display: flex; gap: 14px; align-items: flex-start;
  background: var(--amarillo-bg); border: 2px solid var(--amarillo);
  border-radius: var(--radio); padding: 14px 16px; margin-bottom: 14px;

```

```

}
.banner .ico { width: 34px; height: 34px; border-radius: 50%; background: var(--amarillo);
  color: #fff; font-weight: 700; display: inline-flex; align-items: center;
  justify-content: center; flex: none; font-size: 1.2rem; }
.banner b { color: var(--amarillo-tx); display: block; margin-bottom: 3px; }
.banner span { color: var(--amarillo-tx); font-size: .87rem; }
.banner.ok { background: var(--verde-bg); border-color: var(--verde); }
.banner.ok .ico { background: var(--verde); }
.banner.ok b, .banner.ok span { color: var(--verde); }

/* — Opciones — */
.opciones { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
.opcion {
  background: #fff; border: 2px solid var(--azul); border-radius: var(--radio);
  padding: 18px; text-align: center;
}
.opcion .n { background: var(--azul); color: #fff; font-size: .68rem;
  font-weight: 700; padding: 3px 9px; border-radius: 10px; }
.opcion h4 { color: var(--azul); font-size: 1.02rem; margin: 12px 0 8px; }
.opcion p { color: var(--txt); font-size: .87rem; margin: 3px 0; }

/* — Tablas y KPI — */
table { width: 100%; border-collapse: collapse; font-size: .87rem; }
thead th { background: var(--azul); color: #fff; padding: 9px 10px; text-align: left;
  font-size: .8rem; }
tbody td { padding: 9px 10px; border-bottom: 1px solid var(--borde); }
tbody tr:nth-child(even) { background: #FAFBFD; }
td.dif, td.falta { background: var(--amarillo-bg); color: var(--amarillo-tx);
  font-weight: 700; }
.kpis { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 12px; margin-bottom: 14px; }
.kpi { background: #fff; border: 1px solid var(--borde); border-radius: var(--radio);
  padding: 14px 16px; border-top: 4px solid var(--amarillo); }
.kpi small { color: var(--grafito); font-size: .76rem; display: block; }
.kpi b { font-size: 1.6rem; color: var(--azul); display: block; margin: 3px 0; }
.kpi.verde b { color: var(--verde); }
/* --amarillo (#FFD000) es el acento de marca, hecho para franjas y fondos,
  no para texto: sobre blanco da 1.47:1 de contraste, muy por debajo del
  3:1 mínimo (WCAG) hasta para texto grande en negrita. --amarillo-tx
  (#8A6D00) es la variante ya pensada para texto - Reportes.jsx ya la usa
  por esta misma razón para los chips "oro"; esta regla se había quedado
  con la de fondo por error. */
.kpi.oro b { color: var(--amarillo-tx); }
.kpi i { font-size: .72rem; color: var(--grafito); font-style: normal; }
.kpi-cabeza { display: flex; align-items: center; gap: 9px; margin-bottom: 2px; }
.kpi-cabeza small { margin: 0; }
.icono-kpi {
  width: 30px; height: 30px; border-radius: 9px; flex: none;
  background: var(--azul-claro); display: flex; align-items: center;
  justify-content: center; font-size: 1rem;
}
.kpi.verde .icono-kpi { background: var(--verde-bg); }
.kpi.oro .icono-kpi { background: var(--amarillo-bg); }

/* — Bodegas — */
.grid-bodegas { display: grid; grid-template-columns: repeat(auto-fill, minmax(230px, 1fr));
  gap: 12px; }
.tarjeta-bodega { background: #fff; border: 1px solid var(--borde);
  border-left: 5px solid var(--grafito); border-radius: 10px; padding: 12px 14px;
  text-align: left; width: 100%; }
.tarjeta-bodega b { display: flex; align-items: center; gap: 8px;
  font-size: .87rem; margin-bottom: 6px; }
.tarjeta-bodega .est { font-size: .72rem; font-weight: 700; padding: 3px 8px;
  border-radius: 9px; }
.tarjeta-bodega.cerrada { border-left-color: var(--verde); }
.tarjeta-bodega.en_conteo { border-left-color: var(--azul); }

```

```

.tarjeta-bodega.en_auditoria{ border-left-color: var(--amarillo); }
.tarjeta-bodega small { display: block; color: var(--grafito); font-size: .72rem;
  margin-top: 6px; }
.icono-tarjeta { width: 26px; height: 26px; border-radius: 8px; flex: none;
  display: flex; align-items: center; justify-content: center; font-size: .85rem;
  background: #EFEFEF; }
.tarjeta-bodega.cerrada .icono-tarjeta { background: var(--verde-bg); }
.tarjeta-bodega.en_conteo .icono-tarjeta { background: var(--azul-claro); }
.tarjeta-bodega.en_auditoria .icono-tarjeta { background: var(--amarillo-bg); }
.icono-accion { width: 42px; height: 42px; border-radius: 50%; background: var(--azul-claro);
  display: flex; align-items: center; justify-content: center; font-size: 1.3rem;
  margin-bottom: 10px; }
.accesos-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px; }
.acceso-rapido {
  background: #fff; border: 1px solid var(--borde); border-radius: var(--radio);
  padding: 16px 18px; text-align: left; width: 100%;
  transition: box-shadow .15s, transform .1s;
}
.acceso-rapido: hover { box-shadow: var(--sombra); transform: translateY(-1px); }
.acceso-rapido b { display: block; color: var(--azul); font-size: .95rem; margin-bottom: 3px; }
.acceso-rapido small { color: var(--grafito); font-size: .8rem; }

/* — Ingreso — */
.ingreso-fondo { min-height: 100dvh; display: flex; align-items: center;
  justify-content: center; background: var(--fondo); padding: 24px; }
.ingreso { display: grid; grid-template-columns: 1fr 1fr; width: 100%;
  max-width: 960px; min-height: 560px; background: #fff; border-radius: 22px;
  overflow: hidden; box-shadow: 0 24px 60px -16px rgba(11, 27, 51, .28); }
.ingreso.marca-lado { background: var(--azul-prof); color: #fff;
  display: flex; flex-direction: column; align-items: center;
  justify-content: center; gap: 8px; padding: 30px; }
.ingreso.marca-lado img.app { width: 110px; border-radius: 22px; }
.ingreso.marca-lado h1 { font-size: 2.5rem; }
.ingreso.marca-lado i { color: var(--amarillo); font-size: 1.05rem; }
.ingreso.marca-lado hr { width: 130px; border: none; border-top: 3px solid var(--amarillo);
  margin: 12px 0; }
.ingreso.marca-lado p { color: #C6D2E6; text-align: center; max-width: 300px;
  font-size: .95rem; }
.ingreso.marca-lado img.cs { height: 22px; margin-top: 22px; }
.ingreso.marca-lado .hackathon { color: #7C93BC; font-size: .78rem;
  margin-top: 6px; letter-spacing: .02em; }
.ingreso.form { display: flex; flex-direction: column; justify-content: center;
  gap: 14px; padding: 40px 12px; }
.ingreso.form h2 { color: var(--azul); font-size: 1.7rem; }
.ingreso.form label { font-size: .8rem; color: var(--grafito); }
.ingreso.campo-icono { position: relative; display: flex; align-items: center; }
.ingreso.campo-icono svg { position: absolute; left: 14px; color: #9AA7BC; }
.ingreso.form input { width: 100%; padding: 13px 14px 13px 42px;
  border: 1px solid var(--borde); border-radius: var(--radio); }
.ingreso.form input:focus { outline: 2px solid var(--azul); border-color: var(--azul); }
.ingreso.perfiles { display: flex; gap: 10px; margin-top: 4px; }
.ingreso.perfiles span { flex: 1; text-align: center; padding: 12px 8px;
  border-radius: var(--radio); font-weight: 700; font-size: .88rem;
  border: 2px solid var(--azul); color: var(--azul); background: #fff;
  transition: background .15s, color .15s; }
.ingreso.perfiles span.sel { background: var(--azul); color: #fff; }
.ingreso.form .btn { margin-top: 6px; }
.ingreso.form .pista-huella { text-align: center; margin-top: 4px; }

/* En pantallas angostas el card apilado puede quedar más alto que la
  pantalla; centrarlo verticalmente ahí recorta el logo por arriba y obliga
  a subir el scroll para verlo. Mejor anclarlo arriba y dejar que crezca
  hacia abajo - así siempre se ve completo desde el primer momento. */
@media (max-width: 860px) {
  .ingreso-fondo { align-items: flex-start; padding: 20px 14px; }
  .ingreso { grid-template-columns: 1fr; max-width: 440px; min-height: auto; }
  .ingreso.marca-lado { padding: 26px 24px 18px; gap: 6px; }

```

```

.ingreso .marca-lado img.app { width: 64px; }
.ingreso .marca-lado h1 { font-size: 1.7rem; }
.ingreso .marca-lado hr { margin: 8px 0; }
.ingreso .marca-lado p { font-size: .82rem; }
.ingreso .marca-lado img.cs { margin-top: 14px; }
.ingreso .form { padding: 24px; gap: 12px; }
}
.pista { font-size: .78rem; color: var(--grafito); font-style: italic; }
.error { color: #B3261E; font-size: .85rem; }
.msg-ok { color: var(--verde); font-size: .85rem; }

/* — Modal — */
.overlay { position: fixed; inset: 0; background: rgba(11,27,51,.55);
  display: flex; align-items: center; justify-content: center; z-index: 50; }
.modal { background: #fff; border-radius: 16px; padding: 26px; max-width: 460px;
  width: 90%; text-align: center; border-top: 6px solid var(--amarillo); }
.modal h2 { color: var(--azul); margin-bottom: 10px; }
.modal p { margin-bottom: 12px; }
.modal .aviso { background: var(--amarillo-bg); border: 1px solid var(--amarillo);
  color: var(--amarillo-tx); border-radius: 10px; padding: 10px; font-size: .87rem;
  font-weight: 700; margin-bottom: 16px; }
.modal .botones { display: flex; gap: 10px; }
.modal .botones .btn { flex: 1; }

/* — Gráficas (hechas a mano: sin librería externa) — */
.barra-fila { display: flex; align-items: center; gap: 10px; margin-bottom: 7px; }
.barra-fila .etq { width: 190px; font-size: .82rem; flex: none; }
.barra-fila .pista { flex: 1; background: var(--fondo); border-radius: 6px;
  height: 14px; overflow: hidden; font-style: normal; }
.barra-fila .rellena { height: 100%; border-radius: 6px; transition: width .3s; }
.barra-fila .val { width: 54px; text-align: right; font-size: .82rem;
  font-weight: 700; flex: none; }
.leyenda { display: flex; gap: 16px; flex-wrap: wrap; margin-top: 10px; font-size: .8rem;
  color: var(--grafito); }
.leyenda .punto { width: 10px; height: 10px; border-radius: 50%; display: inline-block;
  margin-right: 6px; }
.puntos-bodegas { display: flex; flex-wrap: wrap; gap: 6px; }
.puntos-bodegas span { width: 15px; height: 15px; border-radius: 50%; display: inline-block; }

.vacio { color: var(--grafito); font-style: italic; padding: 20px 0; }
.cargando { color: var(--grafito); padding: 24px; }

.reportes-cols { display: grid; grid-template-columns: 1.3fr 1fr; gap: 16px; }
.tabla-scroll { overflow-x: auto; }

/* == Móvil: una columna, micrófono grande == */
@media (max-width: 860px) {
  .app-root { grid-template-columns: 1fr; grid-template-rows: auto 1fr; }
  .sidebar { flex-direction: row; overflow-x: auto; padding: 8px 6px; gap: 4px; }
  .sidebar .marca, .sidebar hr, .sidebar footer { display: none; }
  .sidebar ul { display: flex; gap: 4px; }
  .sidebar li { border-left: none; border-bottom: 3px solid transparent;
    white-space: nowrap; padding: 8px 12px; font-size: .8rem; border-radius: 8px; }
  .sidebar li.sel { border-left: none; border-bottom-color: var(--amarillo); }
  .conteo-cols { grid-template-columns: 1fr; }
  .dos-columnas { grid-template-columns: 1fr; }
  .reportes-cols { grid-template-columns: 1fr; }
  .perfil-cols { grid-template-columns: 1fr; }
  .campos-2col { grid-template-columns: 1fr; }
  .mic-btn { width: min(64vw, 200px); height: min(64vw, 200px); font-size: 3.4rem;
    margin: 6px auto; }
  .opciones { grid-template-columns: 1fr; }
  .accesos-grid { grid-template-columns: repeat(2, 1fr); }
  .grilla-botones { display: grid; grid-template-columns: 1fr 1fr; }
  .grilla-botones .btn { min-height: 54px; }

```

```

.area { padding: 14px; }
/* la etiqueta fija de 190px de las gráficas de barra hechas a mano no
   cabe junto al valor en un telefono angosto - se achica en vez de
   empujar la fila a scroll horizontal. */
.barra-fila { gap: 6px; }
.barra-fila .etq { width: 96px; font-size: .74rem; }
.barra-fila .val { width: 40px; }
}

/* == Tablets angostas (retrato ~600-860px): layout de escritorio ya no
   entra completo, pero el menu horizontal de móvil desperdicia el ancho
   disponible - se mantiene el menu lateral, solo mas angosto. == */
@media (min-width: 601px) and (max-width: 860px) {
  .app-root { grid-template-columns: 76px 1fr; grid-template-rows: none; }
  .sidebar { flex-direction: column; overflow-x: visible; overflow-y: auto;
    padding: 16px 0; gap: 0; }
  .sidebar .marca { display: flex; padding: 0 0 14px; justify-content: center; }
  .sidebar .marca b, .sidebar .marca i { display: none; }
  .sidebar hr, .sidebar footer small, .sidebar footer img,
  .sidebar footer b { display: none; }
  .sidebar ul { display: block; }
  .sidebar li { flex-direction: column; gap: 3px; padding: 10px 4px;
    font-size: .62rem; text-align: center; border-left: none;
    border-bottom: 3px solid transparent; white-space: normal; }
  .sidebar li.sel { border-left: none; border-bottom-color: var(--amarillo); }
  .sidebar footer { padding: 12px 0 0; display: flex; justify-content: center; }
  .sidebar footer .avatar { margin-right: 0; }
}

@media (max-width: 600px) {
  .barra-sup { padding: 12px 14px; }
  .barra-sup h1 { font-size: 1.08rem; }
}

```

12.25 Frontend: Ingreso e Inicio (vistas 5.1 y 5.2)

Archivo: `frontend/src/vistas/Ingreso.jsx`

```

import { useEffect, useState } from "react";
import { ingresar, pedir } from "../api";
import { hayAutenticadorDisponible, ingresarConHuella } from "../webauthn";
import Icono from "../Iconos";

const ETIQUETAS_PERFIL = {
  auxiliar: "Auxiliar de inventarios",
  auditor: "Administrador de bodega",
};

export default function Ingreso({ alEntrar }) {
  const [usuario, setUsuario] = useState("");
  const [clave, setClave] = useState("");
  const [err, setErr] = useState("");
  const [cargando, setCargando] = useState(false);
  const [perfil, setPerfil] = useState(null);
  const [tieneHuella, setTieneHuella] = useState(false);
  const [entrandoConHuella, setEntrandoConHuella] = useState(false);
  const [hayLector, setHayLector] = useState(false);

  useEffect(() => { hayAutenticadorDisponible().then(setHayLector); }, []);

  // apenas escribe el usuario (o el codigo de empleado) se identifica el
  // perfil solo - no hay que elegirlo a mano. Con demora corta para no
  // disparar una llamada por cada tecla.

```

```

useEffect(() => {
  const entrada = usuario.trim();
  if (!entrada) { setPerfil(null); setTieneHuella(false); return; }
  const espera = setTimeout(() => {
    pedir(`/api/usuarios/perfil?usuario=${encodeURIComponent(entrada)}`)
      .then((r) => { setPerfil(r.perfil); setTieneHuella(r.tiene_huella); })
      .catch(() => { setPerfil(null); setTieneHuella(false); });
  }, 350);
  return () => clearTimeout(espera);
}, [usuario]);

async function entrar(e) {
  e?.preventDefault();
  setCargando(true);
  setErr("");
  try {
    const r = await ingresar(usuario.trim(), clave);
    alEntrar(r);
  } catch (e2) {
    setErr(e2.message);
  } finally {
    setCargando(false);
  }
}

async function entrarConHuella() {
  setErr("");
  setEntrandoConHuella(true);
  try {
    const r = await ingresarConHuella(usuario.trim());
    alEntrar(r);
  } catch (e2) {
    setErr(e2.name === "NotAllowedError"
      ? "No se reconoció la huella o se canceló la lectura."
      : e2.message);
  } finally {
    setEntrandoConHuella(false);
  }
}

return (
  <div className="ingreso-fondo">
    <div className="ingreso">
      <div className="marca-lado">
        
        <h1>CuentaVoz</h1>
        <i>Tu voz cuenta</i>
        <hr />
        <p>Asistente conversacional para la toma física de inventarios</p>
        
        <p className="hackathon">Hackathon Colsubsidio × 30X</p>
      </div>

      <form className="form" onSubmit={entrar}>
        <h2>Iniciar sesión</h2>
        <p className="pista">Ingrese con su usuario corporativo.</p>

        <label htmlFor="campo-usuario">Usuario o código de empleado</label>
        <div className="campo-icone">
          <Icono nombre="perfil" tam={18} />
          <input
            id="campo-usuario"
            value={usuario}
            onChange={(e) => setUsuario(e.target.value)}
            autoComplete="username"
          />
        </div>
      </form>
    </div>
  </div>
)

```



```

        autoFocus
      />
    </div>

    <label htmlFor="campo-pin">PIN</label>
    <div className="campo-icono">
      <Icono nombre="candado" tam={18} />
      <input
        id="campo-pin"
        type="password"
        value={clave}
        onChange={(e) => setClave(e.target.value)}
        autoComplete="current-password"
      />
    </div>

    <div>
      <span id="etiqueta-perfil" style={{ fontSize: ".8rem", color: "var(--grafito)" }}>
        Perfil
      </span>
      /* aria-live: el texto de arriba no cambia (siempre dice "Perfil"),
        pero esta línea sí - solo así un lector de pantalla se enteraría
        de qué perfil se detectó al escribir el usuario, sin depender
        del color que solo ve quien puede ver la pantalla. */
      <p aria-live="polite" className="pista" style={{ minHeight: "1.1em", margin: "2px 0 4px"
        >>>
        {perfil ? `Perfil detectado: ${ETIQUETAS_PERFIL[perfil]}` : ""}
      </p>
      <div className="perfiles" role="group" aria-labelledby="etiqueta-perfil">
        <span className={perfil === "auxiliar" ? "sel" : ""}>
          {ETIQUETAS_PERFIL.auxiliar}
        </span>
        <span className={perfil === "auditor" ? "sel" : ""}>
          {ETIQUETAS_PERFIL.auditor}
        </span>
      </div>
    </div>

    {err && <span className="error" role="alert">{err}</span>}

    <button className="btn" type="submit" disabled={cargando || !usuario || !clave}>
      {cargando ? "Entrando..." : "ENTRAR"}
    </button>

    {hayLector && tieneHuella && (
      <button type="button" className="btn borde" disabled={entrandoConHuella}
        onClick={entrarConHuella}>
        {entrandoConHuella ? "Leyendo huella..." : "Entrar con la huella del dispositivo"}
      </button>
    )}
  </form>
</div>
</div>
);
}

```

Archivo: **frontend/src/vistas/Inicio.jsx**

```

import { useEffect, useState } from "react";
import { pedir } from "../api";
import Marco from "../Marco";
import AsistenteVoz from "../AsistenteVoz";

// timeZone fijo en Bogotá (no el del navegador/equipo): CuentaVoz es para
// la operación de Colsubsidio en Colombia, así que la fecha y el saludo

```

```

// deben leerse en hora de Bogotá aunque el dispositivo esté mal
// configurado o alguien entre desde otro huso horario.
const HUSO = "America/Bogota";
const FECHA = new Date().toLocaleDateString("es-CO", {
  weekday: "long", day: "numeric", month: "long", year: "numeric",
  timeZone: HUSO,
});

/** "Buenos días" antes de mediodía, "Buenas tardes" hasta las 7pm, y
    "Buenas noches" después - siempre en hora de Bogotá. Sin esto el
    saludo decía "Buenos días" a cualquier hora, incluso de noche. */
function saludoSegunHora() {
  const hora = Number(
    new Intl.DateTimeFormat("en-US", { timeZone: HUSO, hour: "numeric", hourCycle: "h23" })
      .format(new Date())
  );
  if (hora < 12) return "Buenos días";
  if (hora < 19) return "Buenas tardes";
  return "Buenas noches";
}

const COLOR_ACCION = {
  FIRMA: "verde", CIERRE: "verde", APROBACION: "verde", REPORTE: "azul",
  ALERTA: "oro", CORRECCION: "oro", REAPERTURA: "oro",
  CREACION: "azul", AUDITORIA: "oro",
};
const DOT = { verde: "var(--verde)", azul: "var(--azul)",
  oro: "var(--amarillo)", gris: "var(--grafito)" };

export default function Inicio({ token, usuario, ir }) {
  const [resumen, setResumen] = useState(null);
  const [actividad, setActividad] = useState([]);
  const esAuditor = usuario?.perfil === "auditor";

  // Ejemplos de lo que ya entiende el agente aquí: preguntar por los KPI de
  // esta pantalla, o pedir directo cualquiera de los accesos rápidos de
  // abajo - decir su nombre ya navega, sin tener que tocarlo.
  const ejemplos = [
    "¿cuántas bodegas tengo?", "¿cuántas alertas hay por revisar?",
    "¿cuál es mi exactitud del mes?", "iniciar un conteo", "ver el tablero",
    ...(esAuditor ? ["continuar auditoría", "generar reporte"]
      : ["hacer un pedido", "legalizar servicio"]),
  ];

  useEffect(() => {
    pedir("/api/usuarios/yo/resumen", {}, token).then(setResumen).catch(() => {});
    pedir("/api/trazabilidad/reciente", {}, token).then(setActividad).catch(() => {});
  }, [token]);

  // "Auditoría"/"Reportes"/"Panel" ni siquiera aparecen en el menú lateral
  // de un auxiliar (soloAuditor en App.jsx:MENU) - ofrecerlos aquí como
  // acceso directo llevaba a una pantalla que el propio perfil no puede
  // usar, aunque el menu de al lado dijera lo contrario.
  const accesos = esAuditor ? [
    { t: "Iniciar un conteo", s: "dicte y CuentaVoz registra", d: "conteo", ic: "🗒️" },
    { t: "Ver el tablero", s: "estado de las bodegas", d: "bodegas", ic: "📊" },
    { t: "Continuar auditoría", s: "recuento ciego y aprobaciones", d: "auditoria", ic: "🔍" },
    { t: "Generar reporte", s: "consolidado del día", d: "reportes", ic: "📄" },
  ] : [
    { t: "Iniciar un conteo", s: "dicte y CuentaVoz registra", d: "conteo", ic: "🗒️" },
    { t: "Ver el tablero", s: "estado de las bodegas", d: "bodegas", ic: "📊" },
    { t: "Hacer un pedido", s: "receta y porciones por voz", d: "pedido", ic: "📋" },
    { t: "Legalizar servicio", s: "sobrante y merma del turno", d: "legalizacion", ic: "🏠" },
  ];
}

```

```

return (
  <Marco titulo={`Inicio · ${FECHA}`}
    chip={{ texto: "TOMA EN CURSO", tipo: "verde" }}>
    <h2 style={{ fontSize: "1.15rem", color: "var(--azul)", marginBottom: 14 }}>
      {saludoSegunHora()}, <span style={{ textTransform: "capitalize" }}>{usuario?.nombre}</span>.
      Esto es lo que hay para hoy.
    </h2>

    <AsistenteVoz token={token} vista="inicio" ir={ir} ejemplos={ejemplos}
      placeholder="¿cuántas bodegas tengo?, lléveme a bodegas..." />

    <div className="kpis">
      <div className="kpi">
        <div className="kpi-cabeza">
          <span className="icono-kpi">📦</span>
          <small>Bodegas asignadas a usted</small>
        </div>
        <b>{resumen?.bodegas_asignadas ?? "-"}</b>
        <i>según su perfil</i>
      </div>
      <div className="kpi">
        <div className="kpi-cabeza">
          <span className="icono-kpi">📦</span>
          <small>Referencias contadas hoy</small>
        </div>
        <b>{resumen?.referencias_hoy ?? "-"}</b>
        <i>en sus sesiones de conteo</i>
      </div>
      <div className={`kpi ${resumen?.alertas_por_revisar ? "oro" : "verde"}`}>
        <div className="kpi-cabeza">
          <span className="icono-kpi">{resumen?.alertas_por_revisar ? "⚠️" : "✅"}</span>
          <small>Alertas por revisar</small>
        </div>
        <b>{resumen?.alertas_por_revisar ?? "-"}</b>
        <i>en toda la operación</i>
      </div>
      <div className="kpi verde">
        <div className="kpi-cabeza">
          <span className="icono-kpi">📊</span>
          <small>Su exactitud del mes</small>
        </div>
        <b>{resumen ? `${resumen.exactitud_mes} %` : "-"}</b>
        <i>promedio de bodegas cerradas</i>
      </div>
    </div>

    <div className="card">
      <h3>¿Qué desea hacer?</h3>
      <div className="accesos-grid">
        {accesos.map((a) => (
          <button key={a.d} className="acceso-rapido"
            onClick={() => ir(a.d)}>
            <span className="icono-accion">{a.ic}</span>
            <b>{a.t}</b>
            <small>{a.s}</small>
          </button>
        ))}
      </div>
    </div>

    <div className="card">
      <h3>Actividad reciente</h3>
      {actividad.length === 0 ? (
        <p className="vacio">Todavía no hay actividad registrada hoy.</p>

```

```

    ) : (
      actividad.map((a, i) => {
        const color = COLOR_ACCION[a.accion] || "gris";
        return (
          <div className="registro" key={i}>
            <span style={{ width: 9, height: 9, borderRadius: "50%", flex: "none",
              background: DOT[color] }} />
            <b style={{ color: "var(--grafito)", minWidth: 46 }}>{a.hora}</b>
            <span>{a.detalle}</span>
            <span className={`etiqueta-actividad ${color}`}>
              {a.accion.toLowerCase()}
            </span>
          </div>
        );
      })
    )}
  </div>
</Marco>
);
}

```

12.26 Frontend: Pedido (vistas 5.3 a 5.5)

Archivo: `frontend/src/vistas/Pedido.jsx`

```

import { useState, useEffect } from "react";
import { pedir, enviarTurno, descargarReporte } from "../api";
import { escuchar, hablar } from "../voz";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import Icono from "../Iconos";

const DIA = new Date().toLocaleDateString("es-CO", { weekday: "long" });
// Palabras sueltas que valen como "sí" a lo que se le esté preguntando
// (confirmar un guardado, o enviar el pedido calculado).
const AFIRMACIONES = ["confirmando", "confirmar", "si", "sí", "claro", "listo", "dale",
  "correcto", "hazlo", "adelante", "procede"];
// Frases de varias palabras que también confirman, tal como las dice una
// persona real ("así es", "va, mándalo") y no calzarían con el chequeo por
// palabra exacta de arriba.
const FRASES_AFIRMATIVAS = ["así es", "asi es", "eso es", "va pues", "de una"];

function quitarTildes(t) {
  return t.normalize("NFD").replace(/[\u0300-\u036f]/g, "");
}

function esConfirmacion(texto) {
  const t = quitarTildes(texto.toLowerCase()).replace(/[.,!¿?]/g, "");
  const palabras = t.split(/\s+/);
  if (palabras.includes("no")) return false; // "no lo envíes" no es un sí
  if (palabras.some((p) => AFIRMACIONES.includes(quitarTildes(p)))) return true;
  if (FRASES_AFIRMATIVAS.some((f) => t.includes(quitarTildes(f)))) return true;
  // cualquier conjugación de "enviar"/"mandar" cuenta como un sí a la
  // pregunta de mandar el pedido ("envíalo", "envíelo", "envíemelo",
  // "mándalo", "mándemelo"... ) - enumerar cada forma verbal a mano es
  // interminable y siempre se queda corto; sin esto, decir la forma
  // "usted" en vez de la forma "tú" hacía que la pantalla repitiera el
  // mismo mensaje en vez de entender que la persona ya dijo que sí.
  return /envi|mand/.test(t);
}

/** Un "no" limpio a lo que se le está preguntando (¿prepara otro plato?,
  ¿lo enviamos?), sin que además mencione un plato nuevo - si mencionara
  uno, no es un cierre sino un cambio de tema, y eso lo debe interpretar

```

```

    Gemini como corresponde, no cerrarse en seco aquí. */
function esNegacionSimple(texto, platosConocidos) {
  const t = quitarTildes(texto.toLowerCase()).replace(/[.,!¿?]/g, "");
  const palabras = t.split(/\s+/);
  if (!palabras.includes("no")) return false;
  const mencionaOtroPlato = platosConocidos.some(
    (nombre) => t.includes(quitarTildes(nombre.toLowerCase()))
  );
  return !mencionaOtroPlato;
}

// Pedidos se sale y se vuelve a entrar todo el tiempo (a mirar otra pantalla
// a mitad de un pedido, por ejemplo): sin guardar esto, cada regreso mostraba
// la pantalla en blanco como si la conversación nunca hubiera pasado, aunque
// el chef ya llevara todo un plato armado. Se guarda en sessionStorage (no
// sobrevive a cerrar la pestaña) y por persona, para que dos sesiones en el
// mismo navegador no se mezclen.
function claveEstadoPedido(sesionId) {
  return `cv_pedido_estado_${sesionId}`;
}
function cargarEstadoPedido(sesionId) {
  try { return JSON.parse(sessionStorage.getItem(claveEstadoPedido(sesionId)) || "{}"); }
  catch { return {}; }
}

export default function Pedido({ token, usuario, ir }) {
  const esAuditor = usuario?.perfil === "auditor";
  // Pedidos tiene su propia conversación con el agente, aislada de la del
  // conteo físico: usa un numero de sesion negativo (unico por persona) para
  // que nunca choque con una sesion real de bodega (esas siempre son >= 1).
  const sesionId = -1000 - (usuario?.id || 0);
  const guardado = cargarEstadoPedido(sesionId);
  // Vacío hasta que de verdad se dicte algo: mostrar «ajiacos/50» desde el
  // arranque hacía parecer que ya se había entendido un pedido que nadie
  // dictó todavía, y esos datos no cambiaban con nada más que se dijera.
  const [plato, setPlato] = useState(guardado.plato || "");
  const [porciones, setPorciones] = useState(guardado.porciones ?? null);
  const [dicho, setDicho] = useState(guardado.dicho || "");
  const [lineas, setLineas] = useState(guardado.lineas ?? null);
  const [receta, setReceta] = useState(null);
  const [enviado, setEnviado] = useState(guardado.enviado || false);
  const [recibo, setRecibo] = useState(guardado.recibo ?? null);
  const [respuesta, setRespuesta] = useState(
    guardado.respuesta || "Diga el plato y las porciones: «hoy preparamos cincuenta ajiacos»."
  );
  const [estado, setEstado] = useState("listo");
  const [err, setErr] = useState("");
  const [archivo, setArchivo] = useState(guardado.archivo ?? null);
  const [pedirPorciones, setPedirPorciones] = useState(false);
  const [avisos, setAvisos] = useState(guardado.avisos ?? null);
  const [opciones, setOpciones] = useState(guardado.opciones ?? null);
  const [opcionesPara, setOpcionesPara] = useState(guardado.opcionesPara ?? null);
  const [productoConsultado, setProductoConsultado] = useState(guardado.productoConsultado ?? null);
  const [bodegas, setBodegas] = useState([]);
  const [bodegaId, setBodegaId] = useState(guardado.bodegaId ?? null);
  const [platos, setPlatos] = useState([]);
  const [offline, setOffline] = useState(!navigator.onLine);

  // a diferencia de Conteo (que solo guarda "conté X de Y" y sincroniza
  // después), calcular un pedido necesita el stock EN VIVO de la bodega
  // para saber cuánto falta pedir - no hay forma honesta de simularlo sin
  // conexión sin arriesgar un cálculo con datos viejos. Por eso aquí no
  // hay cola offline: solo se avisa con claridad que hay que esperar a
  // tener señal, y lo que el chef ya dictó no se pierde (sigue guardado
  // en sessionStorage, ver claveEstadoPedido arriba).
  useEffect(() => {

```

```

const alVolver = () => setOffline(false);
const alPerder = () => setOffline(true);
window.addEventListener("online", alVolver);
window.addEventListener("offline", alPerder);
return () => {
  window.removeEventListener("online", alVolver);
  window.removeEventListener("offline", alPerder);
};
}, []);

// Guarda una foto de la conversación en cada cambio relevante, para que
// salir y volver a esta pantalla la deje tal cual como quedó.
useEffect(() => {
  const foto = { plato, porciones, dicho, lineas, enviado, recibo, respuesta,
    archivo, avisos, opciones, opcionesPara, productoConsultado, bodegaId };
  sessionStorage.setItem(claveEstadoPedido(sesionId), JSON.stringify(foto));
}, [plato, porciones, dicho, lineas, enviado, recibo, respuesta,
  archivo, avisos, opciones, opcionesPara, productoConsultado, bodegaId, sesionId]);

// para que el chef sepa que puede pedir sin tener que adivinar el nombre
// exacto de una receta que quizás no existe todavía.
useEffect(() => {
  pedir("/api/recetas", {}, token).then(setPlatos).catch(() => {});
}, [token]);

function elegirPlato(nombre) {
  setErr("");
  setDicho(`«${nombre}» (elegido de la lista de recetas)`);
  setPlato(nombre);
  setLineas(null);
  setReceta(null);
  setEnviado(false);
  setRecibo(null);
  setProductoConsultado(null);
  const msg = `Listo, ${nombre.toLowerCase()}. ¿Para cuántas porciones?`;
  setRespuesta(msg);
  hablar(msg);
  setPedirPorciones(true);
}

// el agente saluda primero en vez de esperar a que el chef adivine que
// hay que hablar: sin esto el mensaje de bienvenida solo se veía en la
// pantalla, nunca se oía, y no parecía un asistente por voz de verdad.
// Ahora que la conversación se guarda (ver arriba), esto ya no necesita
// adivinar por tiempo: si ya hay un plato en curso (porque se restauró de
// una visita anterior), es un pedido que sigue abierto y no se interrumpe;
// si no hay nada, es un pedido nuevo de verdad y sí saluda.
useEffect(() => {
  if (!plato) {
    hablar("Dígame por favor qué va a preparar y para cuántas porciones.");
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

// Con qué bodega se descuenta el pedido: por defecto el almacén de
// Alimentos y Bebidas (donde de verdad están los ingredientes de cocina),
// no una bodega al azar - antes quedaba fijo en la primera de la lista
// (una oficina administrativa sin ningún stock) y todo salía en cero.
// Si ya había una bodega elegida de una visita anterior, se respeta esa.
useEffect(() => {
  pedir("/api/bodegas", {}, token).then((bs) => {
    setBodegas(bs);
    if (bodegaId) return;
    const conStock = bs.find((b) => /ALMACEN\s*AYB/i.test(b.bodega))
    || bs.find((b) => b.referencias > 0) || bs[0];
  });

```

```

    if (conStock) setBodegaId(conStock.id);
  }).catch(() => {});
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [token]);

async function verReceta(platoForzado) {
  setErr("");
  const pl = platoForzado ?? plato;
  if (!pl) {
    const msg = "Primero dígame qué va a preparar.";
    setRespuesta(msg);
    hablar(msg);
    return;
  }
  try {
    const r = await pedir(`/api/pedidos/receta?plato=${encodeURIComponent(pl)}`, {}, token);
    if (!r.lineas) {
      setErr(`No encontré una receta para «${pl}». Pruebe con ajiaco o sancocho.`);
      return;
    }
    setReceta(r);
    if (r.faltantes_catalogo?.length) {
      setAvisos({ faltantes: r.faltantes_catalogo, sinRegistro: [] });
    }
  } catch (e) {
    setErr(e.message);
  }
}

/** La frase del chef la interpreta el agente. «mostrar» es lo que se ve
    como «lo que dijo el chef»; «texto» es lo que de verdad se manda al
    backend. Al tocar una tarjeta de opción se manda el código exacto,
    nunca el nombre: si un nombre está contenido en el otro (ARROZ
    dentro de ARROZ BASMATI), el agente no tiene con qué distinguirlos. */
async function dictar(texto, mostrar = texto) {
  setErr("");
  setDicho(`${mostrar}`);
  setEstado("listo");

  // «confirmo» aquí no es del conteo físico: es seguir con el pedido.
  if (esConfirmacion(texto)) {
    // Si lo último que dijo fue "no" a enviarlo, un "sí" suelto un rato
    // después no debe mandarlo de golpe - puede ser un "sí" para otra
    // cosa (confirmando que entendió, por ejemplo), no una segunda
    // vuelta pidiendo enviarlo. Un "envíalo"/"mándalo" explícito sí
    // pasa de una: no hay ambigüedad posible en eso.
    const esExplicitoEnviar = /envi|mand/.test(quitarTildes(texto.toLowerCase()));
    if (!esExplicitoEnviar && respuesta.includes("Para empezar uno nuevo")
        && lineas && !enviado) {
      const msg = "Para enviarlo dígallo de nuevo con «envíalo», o dígame otro plato para uno
nuevo.";
      setRespuesta(msg);
      hablar(msg);
      return;
    }
    if (lineas && !enviado && porPedir === 0) {
      // igual que el botón, que se oculta en este caso: no hay nada que
      // enviar, así que "envíalo" no debe crear un pedido vacío.
      const msg = "No hay nada que enviar: ya tiene todo en la bodega. ¿Prepara otro plato?";
      setRespuesta(msg);
      hablar(msg);
      return;
    }
    if (lineas && !enviado) return enviar();
    if (!lineas) return calcular();
    const msg = "Este pedido ya se envió al almacén. ¿Desea armar otro pedido?";

```

```

    setRespuesta(msg);
    hablar(msg);
    return;
}

// un "no" limpio (sin mencionar otro plato) cierra la conversación en
// vez de caer en Gemini, que sin una intención propia para "declinar"
// terminaba repitiendo el mismo mensaje una y otra vez.
if (esNegacionSimple(texto, platos.map((p) => p.nombre))) {
    // si ya le habíamos contestado con la misma frase de cierre (porque
    // ya dijo "no" antes, a "¿lo enviamos?" o a esta misma), un segundo
    // "no" no debe repetirla - hay que dar el cierre final de una vez,
    // no seguir insistiendo con la misma pregunta.
    const yaSeDespidio = respuesta.includes("Para empezar uno nuevo")
    || respuesta.includes("estaré pendiente");
    const msg = yaSeDespidio
        ? "Listo, gracias. Quedo pendiente si requiere consultar otro pedido. Que tenga un buen día."
        : (lineas && !enviado && porPedir > 0)
        // A propósito no termina en "¿sí o no?": un "sí" aquí se
        // confundiría con el "sí" que envía el pedido (esConfirmacion
        // más abajo lo manda directo con lineas && !enviado), justo lo
        // opuesto de lo que la persona acaba de decir. En cambio se
        // describe dónde queda el pedido y cómo empezar uno nuevo -
        // nombrar otro plato ya lo hace, sin ambigüedad.
        ? "Está bien, no lo envío por ahora; el pedido queda calculado aquí, "
        + "pendiente de enviar cuando usted quiera. Para empezar uno nuevo, "
        + "dígame otro plato."
        : "Listo, gracias. Que tenga un buen día – aquí estaré pendiente si desea consultar otro
pedido.";
    setRespuesta(msg);
    hablar(msg);
    return;
}

try {
    const t = await enviarTurno(texto, sesionId, token,
        { opciones, opcionesPara, preparacion: plato, porciones });
    // un plato nuevo es un tema distinto: la última consulta de producto
    // ya no aplica y solo estorbaría junto al pedido que se está armando.
    if (t.preparacion && t.porciones) setProductoConsultado(null);
    if (t.preparacion && t.preparacion !== plato) {
        // cambio de plato: las porciones y lo ya calculado eran del plato
        // anterior, no de este - si no vienen porciones nuevas, se limpian
        // en vez de dejar el numero viejo puesto como si ya se hubiera dicho.
        setPlato(t.preparacion);
        setPorciones(t.porciones || null);
        setLineas(null);
        setReceta(null);
        setEnviado(false);
        setRecibo(null);
    } else if (t.porciones) {
        setPorciones(t.porciones);
    }
    setRespuesta(t.respuesta_hablada || "");
    setArchivo(t.archivo || null);
    setOpciones(t.opciones || null);
    setOpcionesPara(t.opciones_para || null);
    if (t.producto_consultado) setProductoConsultado(t.producto_consultado);
    if (t.receta) setReceta(t.receta);
    // el chef ya dijo plato Y porciones en la misma frase: el agente dice
    // "voy a consultar la receta y los insumos" en su respuesta, así que
    // hay que calcular de una - si no, la pantalla se queda sin nada
    // mientras el mensaje hablado ya prometió que se iba a hacer. Ese
    // cálculo ya trae su propio mensaje hablado (calcular() más abajo);
    // hablar también esta respuesta intermedia disparaba dos audios casi
    // simultáneos que competían por la misma voz neuronal, y si uno de
    // los dos fallaba caía en la voz de respaldo del navegador, sonaba

```



```

    // como si hablaran dos personas distintas.
    if (t.preparacion && t.porciones) {
        calcular(t.porciones, t.preparacion);
    } else {
        hablar(t.respuesta_hablada);
    }
} catch (e) {
    setErr(e.message);
}
}

function corregirCantidad() {
    setPedirPorciones(true);
}

function confirmarPorciones(v) {
    setPedirPorciones(false);
    if (v && !isNaN(Number(v))) {
        const p = Number(v);
        // cero o negativo no es una cantidad real de porciones - sin este
        // chequeo, el campo lo dejaba pasar igual (aquí "0" es un string no
        // vacío, así que sí entra al if de arriba) y el cálculo salía con
        // "ya tiene todo, no hace falta pedir nada" para cualquier receta.
        if (p <= 0) {
            const msg = "Las porciones tienen que ser un número mayor a cero. ¿Cuántas son en realidad?";
            setRespuesta(msg);
            hablar(msg);
            return;
        }
        setPorciones(p);
        setLineas(null);
        // sin esto, tras elegir un plato de la lista y poner las porciones no
        // pasaba nada visible hasta darle aparte a "Calcular el pedido" - se
        // sentía como si el numero no hubiera servido de nada.
        calcular(p);
    }
}

/** El backend explota la receta y descuenta el stock.
    porcionesForzadas/platoForzado: para poder calcular con el valor recién
    confirmado sin esperar al próximo render (si no, calcular() vería el
    plato o las porciones viejas por el cierre de la función). */
async function calcular(porcionesForzadas, platoForzado) {
    setErr("");
    const p = porcionesForzadas ?? porciones;
    const pl = platoForzado ?? plato;
    if (!pl || !p) {
        const msg = "Primero dígame qué va a preparar y para cuántas porciones.";
        setRespuesta(msg);
        hablar(msg);
        return;
    }
    setEnviado(false);
    setAvisos(null);
    setRecibo(null);
    try {
        const r = await pedir("/api/pedidos/calcular", {
            method: "POST",
            body: JSON.stringify({ plato: pl, porciones: Number(p), bodega_id: bodegaId }),
            token;
        });
        if (!r.receta_encontrada) {
            // no se queda solo con el texto rojo en silencio: cierra la
            // conversación con una salida clara en vez de dejar la pantalla
            // varada con un plato que no tiene receta y ningún paso siguiente.
            const msg = `No encontré una receta para «${pl}». Elija uno de los platos de la lista o dígame
            otro nombre.`;

```

```

        setErr(msg);
        setRespuesta(msg);
        hablar(msg);
        setPlato("");
        setPorciones(null);
        return;
    }
    setLineas(r.lineas);
    if (r.faltantes_catalogo?.length || r.sin_registro_bodega?.length) {
        setAvisos({ faltantes: r.faltantes_catalogo, sinRegistro: r.sin_registro_bodega });
    }
    const faltan = r.lineas.filter((l) => l.falta > 0).length;
    const msg = faltan === 0
        ? `Ya tiene los ${r.lineas.length} insumos que necesita en la bodega; no hace falta pedir nada al almacén. ¿Prepara otro plato?`
        : `Necesita ${r.lineas.length} insumos. Hay que pedir ${faltan} al almacén; el resto ya está en la bodega. ¿Lo enviamos?`;
    setRespuesta(msg);
    hablar(msg);
    // ya se calculó el pedido con esta receta: de una se muestra también,
    // sin que la persona tenga que pedirla aparte con "Ver la receta".
    verReceta(pl);
} catch (e) {
    setErr(e.message);
}
}

async function enviar() {
    try {
        const r = await pedir("/api/pedidos/enviar", {
            method: "POST",
            body: JSON.stringify({ lineas, servicio_id: 1, plato, porciones, bodega_id: bodegaId }),
            token;
        });
        if (r.duplicado) {
            const msg = "Este pedido ya se había enviado antes; no lo mandé dos veces. ¿Desea armar otro pedido?";
            setRespuesta(msg);
            hablar(msg);
            setEnviado(true);
            return;
        }
        const msg = r.estado === "pendiente_aprobacion"
            ? `Pedido ${r.numero_pedido} registrado. Queda pendiente de que el administrador lo apruebe antes de salir del almacén. ¿Desea armar otro pedido?`
            : `Pedido ${r.numero_pedido} registrado y enviado a ${r.bodega || "el almacén"}. ¿Desea armar otro pedido?`;
        setRespuesta(msg);
        hablar(msg);
        setEnviado(true);
        setRecibo(r);
    } catch (e) {
        setErr(e.message);
    }
}

const porPedir = lineas ? lineas.filter((l) => l.falta > 0).length : 0;

return (
    <Marco titulo="Pedidos · Cocina Piscilago"
        chip={offline ? { texto: "SIN CONEXIÓN", tipo: "oro" }
            : { texto: "SERVICIO ALMUERZO", tipo: "" }}>
        {offline && (
            <div className="banner">
                <span className="ico">!</span>
                <span>
                    <b>Sin conexión</b>
                </span>
            </div>
        )}
    </Marco>

```

```

        <span>Calcular un pedido necesita el stock en vivo de la bodega, así que
        no se puede hacer sin señal. Lo que ya dictó no se pierde - siga
        cuando vuelva la conexión y retome donde quedó.</span>
    </span>
</div>
})
<div className="chips">
    <span className="chip">{DIA[0].toUpperCase() + DIA.slice(1)} · almuerzo</span>
    <span className="chip">Cocina Piscilago</span>
    <span className={`chip ${!enviado ? "oro" : recibo?.estado === "pendiente_aprobacion" ? "oro"
: "verde"}`}>
        {!enviado ? "Pedido sin enviar"
        : recibo?.estado === "pendiente_aprobacion" ? "Pendiente de aprobación"
        : "Pedido enviado"}
    </span>
    {bodegas.length > 0 && (
        <select value={bodegaId || ""}
            onChange={(e) => { setBodegaId(Number(e.target.value)); setLineas(null); }}
            aria-label="Bodega de la que se descuenta el pedido"
            style={{ marginLeft: "auto", padding: "6px 12px", border: "1px solid var(--borde)",
            borderRadius: 20, fontSize: ".8rem", fontWeight: 700, color: "var(--azul)"
        }}>
            {bodegas.map((b) => (
                <option key={b.id} value={b.id}>Se descuenta de: {b.bodega}</option>
            ))}
        </select>
    )}
</div>

<div className="conteo-cols">
    <div className="card">
        <h3>Lo que dijo el chef</h3>
        <p className="cita">
            {dicho || "Aún no ha dictado nada: pulse el micrófono y diga, por "
            + "ejemplo, «hoy preparamos cincuenta ajiacos.»"}
        </p>

        <h3 style={{ marginTop: 18 }}>Lo que entendió CuentaVoz</h3>
        {(productoConsultado || (plato && porciones)) ? (
            <table>
                <tbody>
                    {productoConsultado && (
                        <>
                            <tr><td>Último producto consultado</td><td><b style={{ color: "var(--azul)" }}>
                                {productoConsultado.nombre}</b></td></tr>
                            <tr><td>Código · unidad</td><td><b style={{ color: "var(--azul)" }}>
                                {productoConsultado.codigo} · {productoConsultado.unidad}</b></td></tr>
                        </>
                    )}
                    {plato && porciones && (
                        <>
                            <tr><td>Preparación</td><td><b style={{ color: "var(--azul)" }}>
                                {plato.toUpperCase()}</b></td></tr>
                            <tr><td>Porciones</td><td><b style={{ color: "var(--azul)" }}>
                                {porciones}</b></td></tr>
                            <tr><td>Receta aplicada</td><td><b style={{ color: "var(--azul)" }}>
                                estándar de Colsubsidio</b></td></tr>
                            <tr><td>Momento</td><td><b style={{ color: "var(--azul)" }}>
                                pedido al almacén</b></td></tr>
                        </>
                    )}
                </tbody>
            </table>
        ) : (
            <p className="vacio">Todavía no hay ningún plato dictado en esta conversación.</p>
        )}
    </div>

```

```

<div className="card mic-caja">
  <button className={mic-btn} ${estado === "escuchando" ? "escuchando" : ""}`
    onClick={() => escuchar({
      alTexto: dictar,
      alEstado: setEstado,
      alError: setErr,
    })}
    aria-label="Hable para decir el plato y las porciones">
    <Icono nombre="microfono" tam={52} />
  </button>
  <b>{estado === "escuchando" ? "Escuchando..." : "Mantenga presionado y hable"}</b>
  <small>También sirve: «pedir insumos para 30 sancochos»</small>
  <small style={{ color: "var(--verde)", fontWeight: 700 }}>
    Receta + stock = pedido
  </small>
  <details style={{ marginTop: 4, textAlign: "left", width: "100%" }}>
    <summary className="pista" style={{ cursor: "pointer" }}>
      Ver frases exactas que entiende el agente aquí
    </summary>
    <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
      [{"hoy preparamos cincuenta ajiacos", "pedir insumos para 30 sancochos",
        "cambia a treinta porciones", "sí", "envíalo", "no",
        "corregir cantidad"].map((ej, i) => (
        <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
      ))}
    </ul>
  </details>
</div>
</div>

{platos.length > 0 && (
  <div className="card">
    <h3>🍽️ Platos con receta registrada</h3>
    <p className="pista" style={{ marginBottom: 10 }}>
      Estos son los únicos platos que CuentaVoz sabe calcular por ahora. Toque uno
      para armar el pedido sin dictarlo, o dígalos tal cual suena aquí.
    </p>
    <div style={{ display: "flex", flexWrap: "wrap", gap: 8 }}>
      {platos.map((r) => (
        <button key={r.id}
          className={`chip ${plato.toUpperCase() === r.nombre.toUpperCase() ? "azul" :
            ""}`}
          onClick={() => elegirPlato(r.nombre)}>
          {r.nombre.toUpperCase()}
        </button>
      ))}
    </div>
  </div>
)}

<div className="card">
  <p className="rotulo">CuentaVoz responde</p>
  <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
  {opciones && (
    <div className="opciones" style={{ marginTop: 14 }}>
      {opciones.map((o, i) => (
        <button key={o.codigo} className="opcion" onClick={() => dictar(o.codigo, o.nombre)}>
          <span className="n">Opción {i + 1}</span>
          <h4>{o.nombre}</h4>
          <p>Código {o.codigo}</p>
          <p>{o.unidad}</p>
        </button>
      ))}
    </div>
  )}
}

```

```

{archivo && (
  <div className="grilla-botones" style={{ marginTop: 10 }}>
    <button className="btn verde"
      onClick={() => descargarReporte(archivo, token).catch((e) => setErr(e.message))}
      style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
      <Icono nombre="descargar" tam={16} />
      Descargar archivo generado
    </button>
  </div>
)}
{err && <p className="error" style={{ marginTop: 10 }}>{err}</p>}
<div className="grilla-botones">
  <button className="btn" onClick={() => calcular()} disabled={offline}
    style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
    <Icono nombre="pedidos" tam={16} />
    Calcular el pedido
  </button>
  <button className="btn borde" onClick={corregirCantidad}
    style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
    <Icono nombre="refrescar" tam={16} />
    Corregir cantidad
  </button>
  {!receta && (
    <button className="btn oro" onClick={() => verReceta()}
      style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
      <Icono nombre="reportes" tam={16} />
      Ver la receta
    </button>
  )}
</div>
</div>

{avisos && (avisos.faltantes.length > 0 || avisos.sinRegistro.length > 0) && (
  <div className="banner">
    <span className="ico">!</span>
    <span>
      <b>Revise antes de enviar</b>
      {avisos.faltantes.length > 0 && (
        <span>
          No encontré en el catálogo: <b>{avisos.faltantes.join(", ")}</b> – esos
          ingredientes de la receta no se pudieron calcular. Avise al administrador.
        </span>
      )}
      {avisos.sinRegistro.length > 0 && (
        <span>
          Esta bodega nunca ha tenido registro de: <b>{avisos.sinRegistro.join(", ")}</b> –
          se asumió 0 disponible; confirme con el almacén antes de descontar existencias.
        </span>
      )}
    </span>
  </div>
)}

{receta && (
  <div className="card">
    <div className="chips">
      <span className="chip">{receta.nombre}</span>
      <span className="chip">Rendimiento: {receta.rendimiento} porción</span>
      <span className="chip">{receta.lineas.length} ingredientes</span>
      <span className={`chip ${esAuditor ? "azul" : "gris"}`}>
        {esAuditor ? "ADMINISTRA ESTA RECETA" : "SOLO CONSULTA"}
      </span>
    </div>
    <h3>Catálogo de Colsubsidio</h3>
    <table>
      <thead>

```

```

      <tr><th>Ingrediente</th><th>Unidad</th><th>Por porción</th>
      <th>Para {porciones} porciones</th></tr>
    </thead>
    <tbody>
      {receta.lineas.map((l) => (
        <tr key={l.codigo}>
          <td>{l.nombre}</td><td>{l.unidad}</td><td>{l.por_porcion}</td>
          <td className="dif">{Math.round(l.por_porcion * porciones * 1000) / 1000}</td>
        </tr>
      ))}
    </tbody>
  </table>
  {receta.preparacion ? (
    <>
      <h3 style={{ marginTop: 18 }}>Preparación paso a paso</h3>
      <p style={{ whiteSpace: "pre-line", fontSize: ".92rem" }}>{receta.preparacion}</p>
    </>
  ) : (
    <p className="pista" style={{ marginTop: 18 }}>
      Esta receta todavía no tiene preparación paso a paso cargada.
      {esAuditor && " Agréguela desde Ajustes → Recetas."}
    </p>
  )}
  <p className="pista" style={{ marginTop: 10 }}>
    {esAuditor
      ? "Como administradora puede editar cantidades, agregar o quitar ingredientes, o crear
        un plato nuevo desde Ajustes → Recetas."
      : "Esta receta la mantiene el administrador; si algo está desactualizado, avísele desde
        Reportes."}
  </p>
  <div className="grilla-botones">
    <button className="btn" onClick={() => { setReceta(null); calcular(); }}
      style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
      <Icono nombre="pedidos" tam={16} />
      Usar para un pedido
    </button>
    {esAuditor && (
      <button className="btn borde"
        onClick={() => ir && ir("ajustes", { tabInicial: "recetas", recetaId: receta.id
          style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
          <Icono nombre="ajustes" tam={16} />
          Editar receta
        </button>
      )}
      <button className="btn gris" onClick={() => setReceta(null)}
        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="salir" tam={16} />
        Cerrar
      </button>
    </div>
  </div>
)}

{lineas && (
  <>
    <div className="kpi">
      <div className="kpi">
        <div className="kpi-cabeza">
          <span className="icono-kpi"> 📋 </span>
          <small>Insumos de la receta</small>
        </div>
        <b>{lineas.length}</b>
        <i>ingredientes estándar</i>
      </div>
      <div className="kpi verde">
        <div className="kpi-cabeza">

```

```

        <span className="icono-kpi">☑</span>
        <small>Ya disponibles en bodega</small>
      </div>
      <b>{lineas.length - porPedir}</b><i>no se piden</i>
    </div>
    <div className="kpi oro">
      <div className="kpi-cabeza">
        <span className="icono-kpi">☹</span>
        <small>Hay que pedir al almacén</small>
      </div>
      <b>{porPedir}</b>
      <i>faltante calculado</i>
    </div>
  </div>

<div className="card">
  <h3>Insumos calculados para {porciones} {plato}</h3>
  <table>
    <thead>
      <tr>
        <th>Insumo (nombre oficial)</th><th>Unidad</th>
        <th>Necesario</th><th>Hay en bodega</th><th>Falta: pedir</th>
      </tr>
    </thead>
    <tbody>
      {lineas.map((l) => (
        <tr key={l.codigo}>
          <td>{l.nombre}</td>
          <td>{l.unidad}</td>
          <td>{l.necesario}</td>
          <td>{l.stock}</td>
          <td className={l.falta > 0 ? "falta" : ""}>
            {l.falta > 0 ? l.falta : "-"}
          </td>
        </tr>
      ))}
    </tbody>
  </table>
  <p className="pista" style={{ marginTop: 10 }}>
    La cantidad necesaria sale de la receta por porción; el faltante es
    lo necesario menos lo que ya hay en la bodega.
  </p>
  {porPedir === 0 ? (
    <p className="msg-ok" style={{ marginTop: 10 }}>
      ☑ Ya tiene todo en la bodega: no hace falta enviar nada al almacén.
    </p>
  ) : !enviado && (
    <div className="grilla-botones">
      <button className="btn verde" onClick={enviar} disabled={offline}
        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="bodegas" tam={16} />
        Enviar pedido al almacén
      </button>
    </div>
  )}
</div>
</>
}}

{recibo && (
  <div className="card" style={{ border: `1px solid var(--${recibo.estado ===
"pendiente_aprobacion" ? "amarillo" : "verde"})` }}>
    <div className="chips">
      {recibo.estado === "pendiente_aprobacion" ? (
        <span className="chip oro">⌚ PENDIENTE DE APROBACIÓN</span>
      ) : (

```

```

        <span className="chip verde">☑ PEDIDO REGISTRADO</span>
      )}
      <span className="chip">{recibo.numero_pedido}</span>
    </div>
    <h3 style={{ marginTop: 12 }}>Recibo de pedido</h3>
    <table>
      <tbody>
        <tr><td>Número de pedido</td><td><b style={{ color: "var(--verde)" }}>
          {recibo.numero_pedido}</b></td></tr>
        <tr><td>Plato</td><td><b>{plato.toUpperCase()}</b> · {porciones} porciones</td></tr>
        <tr><td>Bodega de descuento</td><td><b>{recibo.bodega || "-"}</b></td></tr>
        <tr><td>Registrado por</td><td style={{ textTransform: "capitalize"
      }}>{recibo.persona}</td></tr>
        <tr><td>Hora</td><td>{recibo.hora}</td></tr>
        <tr><td>Insumos solicitados</td><td>{recibo.total_lineas}</td></tr>
      </tbody>
    </table>
    {recibo.items?.length > 0 && (
      <>
        <h3 style={{ marginTop: 16 }}>Detalle enviado al almacén</h3>
        <table>
          <thead><tr><th>Insumo</th><th>Cantidad</th><th>Unidad</th></tr></thead>
          <tbody>
            {recibo.items.map((it, i) => (
              <tr key={i}><td>{it.nombre}</td><td>{it.cantidad}</td><td>{it.unidad}</td></tr>
            ))}
          </tbody>
        </table>
      </>
    )}
    <p className="pista" style={{ marginTop: 10 }}>
      {recibo.estado === "pendiente_aprobacion"
        ? "El administrador de bodega debe aprobar este pedido en Auditoría antes de que cuente
        como enviado de verdad al almacén."
        : "Este pedido queda guardado en el sistema y aparecerá tal cual en la legalización del
        servicio, comparando lo pedido contra lo consumido."}
    </p>
    <div className="grilla-botones">
      <button className="btn gris" onClick={() => window.print()}>Imprimir recibo</button>
    </div>
  </div>
)}

{pedirPorciones && (
  <Dialogo titulo={porciones ? "Corregir cantidad" : "Porciones"}
    mensaje={porciones ? "¿Cuántas porciones son en realidad?"
      : `¿Para cuántas porciones prepara ${plato.toLowerCase()}?`}
    conCampo conVoz tipo="number" valorInicial={porciones ?? ""}
    onAceptar={confirmarPorciones}
    onCancel={(() => setPedirPorciones(false))} />
)}
</Marco>
);
}

```

12.27 Frontend: Legalización (vistas 5.11 y 5.12)

Archivo: `frontend/src/vistas/Legalizacion.jsx`

```

import { useEffect, useState } from "react";
import { pedir } from "../api";
import { hablar, escuchar, esAfirmacion, esNegacion } from "../voz";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import Icono from "../Iconos";

```



```

const fmt = (n) => Number(n).toLocaleString("es-CO", { maximumFractionDigits: 2 });

/** Arma en palabras lo que ya se ve en la tabla, igual a como lo diría
    CuentaVoz - sobre datos reales, no un guion fijo. */
function narrar(sobranste, merma) {
  const partes = [];
  if (sobranste.length) {
    const detalle = sobranste
      .map((l) => `${fmt(Math.abs(l.diferencia))} de ${l.nombre.toLowerCase()}`)
      .join(" y ");
    partes.push(`Sobró ${detalle}: lo devuelvo a la bodega.`);
  }
  if (merma.length) {
    const detalle = merma
      .map((l) => `${fmt(l.diferencia)} en ${l.nombre.toLowerCase()}`)
      .join(", ");
    partes.push(`Hubo merma de ${detalle}.`);
  }
  if (!partes.length) partes.push("Todo cuadró exacto entre lo pedido y lo usado.");
  partes.push("¿Confirmo la legalización?");
  return partes.join(" ");
}

export default function Legalizacion({ token, servicioId = 1, ir, usuario }) {
  const esAuditor = usuario?.perfil === "auditor";
  const [comp, setComp] = useState(null);
  const [listo, setListo] = useState(false);
  const [err, setErr] = useState("");
  const [respuesta, setRespuesta] = useState("");
  const [pedirAjuste, setPedirAjuste] = useState(false);
  const [verMerma, setVerMerma] = useState(false);
  const [estado, setEstado] = useState("listo");

  function cargar() {
    pedir(`/api/legalizacion/${servicioId}`, {}, token)
      .then((c) => {
        setComp(c);
        const s = c.lineas.filter((l) => l.diferencia < 0);
        const m = c.lineas.filter((l) => l.diferencia > 0);
        const texto = narrar(s, m);
        setRespuesta(texto);
        // sin esto, la pregunta "¿Confirmo la legalización?" solo se veía
        // escrita - nunca se oía al llegar, a diferencia de cómo ya saluda
        // el resto de la app (Panel, Auditoría) al entrar a una pantalla.
        if (c.lineas.length > 0) hablar(texto);
      })
      .catch((e) => setErr(e.message));
  }
  useEffect(cargar, [servicioId, token]);

  // La pregunta hablada ("¿Confirmo la legalización?") no tenía con qué
  // contestar por voz - solo el botón. "sí/confirmo" confirma; un "no"
  // limpio (sin nada más dicho) solo lo reconoce; cualquier otra cosa se
  // trata como el mismo tipo de corrección que ya hace "Ajustar por voz"
  // (texto libre: "el pollo fueron doce kilos"), así decir la corrección
  // de una vez - sin abrir el diálogo aparte - también funciona.
  function alMicrofono() {
    escuchar({
      alTexto: (texto) => {
        setEstado("listo");
        if (esAfirmacion(texto, ["confirma", "legaliza"])) { confirmar(); return; }
        if (esNegacion(texto) && texto.trim().split(/\s+/).length <= 2) {
          const msg = "Entendido, no confirmo todavía. Dígame qué insumo ajustar, "
            + "por ejemplo «el pollo fueron doce kilos».";
        }
      }
    });
  }
}

```

```

        setRespuesta(msg);
        hablar(msg);
        return;
    }
    ajustarPorVoz(texto);
  },
  alEstado: setEstado,
  alError: setErr,
});
}

async function confirmar() {
  try {
    await pedir("/api/legalizacion/confirmar", {
      method: "POST",
      body: JSON.stringify({ servicio_id: servicioId }),
    }, token);
    setListo(true);
    hablar("Legalización confirmada. El sobrante volvió a la bodega y la merma quedó registrada.");
  } catch (e) {
    setErr(e.message);
  }
}

async function ajustarPorVoz(texto) {
  setPedirAjuste(false);
  if (!texto) return;
  setErr("");
  try {
    const c = await pedir("/api/legalizacion/ajustar", {
      method: "POST",
      body: JSON.stringify({ servicio_id: servicioId, texto }),
    }, token);
    setComp(c);
    const s = c.lineas.filter((l) => l.diferencia < 0);
    const m = c.lineas.filter((l) => l.diferencia > 0);
    const msg = "Listo, ajustado. " + narrar(s, m);
    setRespuesta(msg);
    hablar(msg);
  } catch (e) {
    setErr(e.message);
  }
}

if (err) return <Marco titulo="Legalización"><p className="error">{err}</p></Marco>;
if (!comp) return <Marco titulo="Legalización"><p className="cargando">Cargando...</p></Marco>;

const sobrante = comp.lineas.filter((l) => l.diferencia < 0);
const merma = comp.lineas.filter((l) => l.diferencia > 0);

if (listo) {
  const fecha = new Date().toLocaleDateString("es-CO", { day: "numeric", month: "long" });
  return (
    <Marco titulo="Legalización · Ajuste confirmado" chip={{ texto: "LEGALIZADO", tipo: "verde"
  }}}>
    <div className="banner ok">
      <span className="ico">></span>
      <span>
        <b>Legalización confirmada – servicio de almuerzo del {fecha}</b>
        <span>El consumo real quedó registrado y el inventario de la cocina ya refleja los
ajustes.</span>
      </span>
    </div>
    <div className="kpis">
      <div className="card" style={{ borderTop: "4px solid var(--verde)", marginBottom: 0 }}>

```

```

<h3 style={{ color: "var(--verde)" }}>Devuelto a bodega</h3>
{sobranter.length ? sobranter.map((l, i) => (
  <p key={i} style={{ fontSize: ".9rem" }}>
    {fmt(Math.abs(l.diferencia))} {l.unidad} de {l.nombre.toLowerCase()}
  </p>
)) : <p className="vacio">Nada por devolver.</p>}
</div>
<div className="card" style={{ borderTop: "4px solid var(--amarillo)", marginBottom: 0 }}>
  <h3 style={{ color: "var(--amarillo-tx)" }}>Merma registrada</h3>
  {merma.length ? merma.map((l, i) => (
    <p key={i} style={{ fontSize: ".9rem" }}>
      {fmt(l.diferencia)} {l.unidad} de {l.nombre.toLowerCase()} – con nota del chef
    </p>
  )) : <p className="vacio">Sin merma en este servicio.</p>}
</div>
<div className="card" style={{ borderTop: "4px solid var(--azul)", marginBottom: 0 }}>
  <h3 style={{ color: "var(--azul)" }}>Histórico actualizado</h3>
  <p style={{ fontSize: ".9rem" }}>
    Consumo real por porción, para afinar la próxima receta.
  </p>
</div>
</div>
<div className="card">
  <h3>Qué pasa ahora con esta información</h3>
  <div className="registro"><span className="ok">✓</span>
    <span>El archivo de ajuste queda listo para cargarse a My Inventory con los nombres y
    códigos oficiales.</span></div>
  <div className="registro"><span className="ok">✓</span>
    <span>El consumo real alimenta el histórico: si el plato gasta siempre más de un insumo
    que lo que dice la receta, el sistema lo detecta.</span></div>
  <div className="registro"><span className="ok">✓</span>
    <span>La merma queda trazada con responsable y hora: la revisión ya no depende de la
    memoria de nadie.</span></div>
  {esAuditor && (
    <div className="grilla-botones">
      <button className="btn"
        onClick={() => ir && ir("reportes", { tabInicial: "analisis" })}>
        Ver el reporte del servicio
      </button>
    </div>
  )}
</div>
</Marco>
);
}

return (
  <Marco titulo={`Legalización · ${comp.plato ? comp.plato.toLowerCase() : "servicio de
almuerzo"}`}>
    chip={{ texto: "SERVICIO CERRADO", tipo: "" }}
    <div className="chips">
      <span className="chip">
        {comp.porciones ? `${comp.porciones} porciones servidas` : `${comp.lineas.length} insumos`}
      </span>
      <span className="chip verde">{sobranter.length} sobrantes</span>
      <span className="chip oro">{merma.length} merma{merma.length === 1 ? "" : "s"} por
      revisar</span>
    </div>

    <p className="pista" style={{ marginBottom: 12 }}>
      Al cerrar el servicio se compara lo pedido con lo realmente usado: cada
      diferencia queda explicada como sobrante o como merma.
    </p>

    <div className="card">
      {comp.lineas.length === 0 ? (

```

```

    <p className="vacio">
      No hay líneas de servicio todavía. Genere un pedido en el menú Pedidos.
    </p>
  ) : (
    <table>
      <thead>
        <tr>
          <th>Insumo</th><th>Pedido</th><th>Usado</th>
          <th>Diferencia</th><th>Qué significa</th>
        </tr>
      </thead>
      <tbody>
        {comp.lineas.map((l) => (
          <tr key={l.codigo}>
            <td>{l.nombre}</td>
            <td>{l.pedido}</td>
            <td>{l.usado}</td>
            <td className="dif">
              {l.diferencia > 0 ? `+${l.diferencia}` : l.diferencia}
            </td>
            <td>{l.lectura}</td>
          </tr>
        ))}
      </tbody>
    </table>
  )}
</div>

{comp.lineas.length > 0 && (
  <div className="card">
    <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", gap:
10 }}>
      <p className="rotulo" style={{ margin: 0 }}>CuentaVoz responde</p>
      <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
        style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
        onClick={alMicrofono} title="responda «sí» para confirmar, o dicte un ajuste"
        aria-label="Responder por voz: diga sí para confirmar, o dicte un ajuste">
        <Icono nombre="microfono" tam={20} />
      </button>
    </div>
    <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
    <p className="pista" style={{ marginTop: 8 }}>
      o diga «sí» para confirmar, o dicte un ajuste como «el pollo fueron doce kilos»
    </p>
    <details style={{ marginTop: 4 }}>
      <summary className="pista" style={{ cursor: "pointer" }}>
        Ver frases exactas que entiende el agente aquí
      </summary>
      <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
        {[ "sí", "confirmo", "legaliza",
          "no (le pregunta qué insumo ajustar)",
          "el pollo fueron doce kilos", "el aceite fue un litro"].map((ej, i) => (
            <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
          ))}
      </ul>
    </details>
    {err && <p className="error" style={{ marginTop: 10 }}>{err}</p>}
    <div className="grilla-botones">
      <button className="btn verde" onClick={confirmar}
        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="aprobar" tam={16} />
        Confirmar legalización
      </button>
      <button className="btn oro" onClick={() => setVerMerma(true)} disabled={!merma.length}
        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="advertencia" tam={16} />
        Explicar la merma
    </div>
  </div>
)

```

```

        </button>
        <button className="btn borde" onClick={() => setPedirAjuste(true)}
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="microfono" tam={16} />
            Ajustar por voz
        </button>
    </div>
</div>
))

{pedirAjuste && (
    <Dialogo titulo="Ajustar por voz"
        mensaje="Diga el insumo y cuánto se usó realmente:"
        conCampo conVoz placeholder="el pollo fueron doce kilos"
        onAceptar={ajustarPorVoz}
        onCancel={() => setPedirAjuste(false)} />
    )}

{verMerma && (
    <Dialogo titulo="Merma de este servicio"
        mensaje={merma.length
            ? merma.map((l) => `${l.nombre}: +${fmt(l.diferencia)} de más que lo pedido –
revisar con el chef.`).join("\n")
            : "No hay merma en este servicio."}
        textoAceptar="Entendido" textoCancelar="Cerrar"
        onAceptar={() => setVerMerma(false)}
        onCancel={() => setVerMerma(false)} />
    )}
</Marco>
);
}

```

12.28 Frontend: Conteo — la vista principal del auxiliar (vistas 5.6 a 5.10)

Archivo: frontend/src/vistas/Conteo.jsx — El archivo más grande del frontend: conteo por voz, desambiguación, alerta de anomalía, crear producto y el formulario sin conexión viven todos aquí, no en componentes separados.

```

import { useState, useEffect, useRef } from "react";
import { enviarTurno, abrirBodega, buscarBodega, pedir, descargarReporte, esFalloRed } from "../api";
import { escuchar, hablar, vozDisponible, esAfirmacion, esNegacion } from "../voz";
import { interpretarLocal } from "../interpreteLocal";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import Icono from "../Iconos";

const UNIDADES = ["Unidad", "Kilogram", "Liter", "Portion"];
const CLAVE_COLA = (sesionId) => `cv_offline_${sesionId}`;
const CLAVE_CATALOGO = "cv_catalogo_ligero";

function leerCola(sesionId) {
    try { return JSON.parse(localStorage.getItem(CLAVE_COLA(sesionId))) || "[]"; }
    catch (_) { return []; }
}

function guardarCola(sesionId, cola) {
    localStorage.setItem(CLAVE_COLA(sesionId), JSON.stringify(cola));
}

function leerCatalogo() {
    try { return JSON.parse(localStorage.getItem(CLAVE_CATALOGO)) || "[]"; }
    catch (_) { return []; }
}

function guardarCatalogo(lista) {
    try { localStorage.setItem(CLAVE_CATALOGO, JSON.stringify(lista)); } catch (_) {}
}

```

```

}
function normalizarLocal(t) {
  return (t || "").toUpperCase().normalize("NFD").replace(/[-]/g, "").trim();
}
/** Sugerencia sin internet: coincidencia de palabras contra el catálogo
    que ya se guardó en el equipo mientras había señal. */
function sugerirDeCatalogo(texto, catalogo) {
  const q = normalizarLocal(texto);
  const palabras = q.split(/\s+/).filter((p) => p.length >= 3);
  if (!palabras.length || !catalogo.length) return null;
  let mejor = null, mejorPuntaje = 0;
  for (const a of catalogo) {
    const nombre = normalizarLocal(a.nombre);
    const aciertos = palabras.filter((p) => nombre.includes(p)).length;
    const puntaje = aciertos / palabras.length;
    if (puntaje > mejorPuntaje) { mejorPuntaje = puntaje; mejor = a; }
  }
  return mejorPuntaje >= 0.5 ? mejor : null;
}

const ETIQUETA_BODEGA = {
  pendiente: ["PENDIENTE", "gris"], en_conteo: ["EN CONTEO", "azul"],
  en_auditoria: ["EN AUDITORÍA", "oro"], cerrada: ["CERRADA", "verde"],
};

export default function Conteo({ token, sessionId = 1, ir, usuario, bodegaSugerida }) {
  const [bodega, setBodega] = useState(null);
  const [asignadas, setAsignadas] = useState(null);
  const [todas, setTodas] = useState(null);
  // Llega desde Bodegas cuando alguien buscó por voz/texto el nombre de
  // una bodega en el buscador de INGREDIENTES (no encontró artículo, pero
  // el nombre sí es una bodega real) - deja el nombre ya listo para
  // confirmar con "Abrir por nombre exacto" en vez de tener que dictarlo
  // otra vez desde cero.
  const [textoBodega, setTextoBodega] = useState(bodegaSugerida || "");
  const [buscarOtra, setBuscarOtra] = useState(false);
  const [bodegaNoEncontrada, setBodegaNoEncontrada] = useState(null);
  const [opcionesBodega, setOpcionesBodega] = useState(null);
  const [msgBodega, setMsgBodega] = useState("");
  const [dicho, setDicho] = useState("");
  const [respuesta, setRespuesta] = useState(
    "Abra una bodega para empezar a contar."
  );
  const [opciones, setOpciones] = useState(null);
  const [opcionesPara, setOpcionesPara] = useState(null);
  const [alerta, setAlerta] = useState(null);
  const [alertaEsperado, setAlertaEsperado] = useState(null);
  const [contextoAlerta, setContextoAlerta] = useState(null);
  const [pendiente, setPendiente] = useState(null);
  const [avance, setAvance] = useState({ hechas: 0, total: 0, alertas: 0, ultimos: [] });
  const [estado, setEstado] = useState("listo");
  const [err, setErr] = useState("");
  const [crear, setCrear] = useState(null); // {nombre, unidad_medida, cantidad_inicial}
  const [archivo, setArchivo] = useState(null);
  const [mostrarTeclado, setMostrarTeclado] = useState(false);
  const [offline, setOffline] = useState(!navigator.onLine);
  const [cola, setCola] = useState(() => leerCola(sessionId));
  const rec = useRef(null);
  // Cada vez que se abre un micrófono para el flujo de bodega (nombre
  // dictado, "¿cuál de todas?", "¿confirma?") sube en 1: el reconocedor
  // anterior a veces tarda en apagarse del todo y llega a entregar un
  // resultado extra y tardío (p. ej. recoger también el "confirmando" de la
  // pregunta siguiente). Cada escucha guarda SU número; si para cuando
  // responde ya no es el número vigente, se descarta sin hacer nada -
  // así lo tardío nunca se confunde con una bodega nueva ni con una
  // respuesta a la pregunta equivocada.
  const idEscuchaBodega = useRef(0);

```

```

// guarda el catalogo liviano en el equipo mientras hay señal, para que
// el modo sin conexión pueda seguir sugiriendo nombres oficiales.
useEffect(() => {
  if (navigator.onLine && token) {
    pedir("/api/articulos/catalogo-ligero", {}, token).then(guardarCatalogo).catch(() => {});
  }
}, [token]);

async function refrescar() {
  try {
    const bid = bodega?.bodega_id ? `?bodega_id_respaldo=${bodega.bodega_id}` : "";
    setAvance(await pedir(`/api/sesiones/${sesionId}/avance${bid}`, {}, token));
  } catch (_) {}
}
useEffect(() => {
  refrescar();
  // si quedaron registros sin sincronizar de una sesión anterior (se
  // cerró la pestaña, o se recargó ya con señal) y ahora sí hay
  // conexión, se reintenta al entrar en vez de esperar a que el
  // navegador dispare otro evento "online" que quizás no vuelva a pasar.
  if (navigator.onLine) sincronizar();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [bodega]);

useEffect(() => {
  pedir("/api/usuarios/yo/bodegas", {}, token).then((r) => {
    setAsignadas(r);
    // sin bodegas propias (el caso típico del administrador, que audita
    // en vez de contar rutinariamente) se necesita el listado completo
    // de una vez - si no, la única opción es escribir el nombre a mano.
    if (r.length === 0) cargarTodas();
  }).catch(() => setAsignadas([]));
}, [token]);

function cargarTodas() {
  pedir("/api/bodegas", {}, token).then(setTodas).catch(() => setTodas([]));
}

useEffect(() => {
  const alVolver = () => { setOffline(false); sincronizar(); };
  const alPerder = () => setOffline(true);
  window.addEventListener("online", alVolver);
  window.addEventListener("offline", alPerder);
  return () => {
    window.removeEventListener("online", alVolver);
    window.removeEventListener("offline", alPerder);
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [sesionId, bodega]);

async function sincronizar() {
  const restantes = leerCola(sesionId);
  if (!restantes.length || !bodega) return;
  // solo se quita de la cola lo que de verdad se confirmó: si un item
  // falla (o vuelve a caerse la red a mitad de la sincronización), el
  // resto queda guardado para el próximo intento en vez de perderse -
  // antes, cualquier falla borraba TODA la cola sin distinguir qué
  // alcanzó a guardarse.
  while (restantes.length) {
    const item = restantes[0];
    try {
      await enviarTurno(`${item.articulo} ${item.cantidad} ${item.unidad}`, sesionId, token);
      await enviarTurno("confirmo", sesionId, token);
    } catch (_) {}
  }
}

```

```

        break;
    }
    restantes.shift();
    guardarCola(sesionId, restantes);
    setCola([...restantes]);
  }
  refrescar();
}

async function abrir(nombre = textoBodega) {
  setErr("");
  setBodegaNoEncontrada(null);
  setOpcionesBodega(null);
  try {
    const r = await abrirBodega(nombre, token);
    setBodega(r);
    const saludo = `Listo, ${r.bodega.toLowerCase()} abierta con ${r.referencias} referencias.
Dícteme el primer producto.`;
    setRespuesta(saludo);
    hablar(saludo);
    refrescar();
  } catch (e) {
    setErr(e.message);
    // igual que un articulo que no esta en el catalogo (ver
    // FormularioCrearProducto): en vez de dejar a la persona sin salida,
    // se ofrece pedir la bodega nueva - queda pendiente de aprobacion.
    if (e.message === "No encuentro esa bodega." && nombre.trim()) {
      setBodegaNoEncontrada(nombre.trim());
    }
  }
}

// Al llegar desde otra pantalla con una bodega ya resuelta (el "Ir a
// Conteo"/"Seguir contando" de Bodegas), se abre de una vez en lugar de
// solo dejarla escrita esperando un clic más: a diferencia de lo
// dictado por voz, este nombre ya salió de una búsqueda exacta contra
// el catálogo, no hay nada que confirmar.
useEffect(() => {
  if (bodegaSugerida) abrir(bodegaSugerida);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

/** El botón "Abrir por nombre exacto" (y Enter en el campo) escribía
    directo a abrir(), que con un nombre ambiguo ("restaurante" a
    secas, cuando hay cinco "RESTAURANTE ...") solo sabía decir "no la
    encuentro" y ofrecer crear una bodega duplicada - nunca mostraba
    las que sí existían. Esto resuelve primero, igual que la voz. */
async function abrirPorBoton() {
  if (!textoBodega.trim()) return;
  setErr("");
  setBodegaNoEncontrada(null);
  setOpcionesBodega(null);
  let resuelto;
  try {
    resuelto = await buscarBodega(textoBodega, token);
  } catch (e) {
    setErr(e.message);
    return;
  }
  if (resuelto.encontrada) {
    abrir(resuelto.bodega);
    return;
  }
  if (resuelto.opciones?.length) {
    setOpcionesBodega(resuelto.opciones);
    return;
  }

```



```

    }
    setBodegaNoEncontrada(textoBodega.trim());
  }

  async function solicitarBodegaNueva() {
    if (!bodegaNoEncontrada) return;
    setErr("");
    try {
      const r = await pedir("/api/bodegas/crear-pendiente", {
        method: "POST", body: JSON.stringify({ nombre: bodegaNoEncontrada }),
      }, token);
      setBodegaNoEncontrada(null);
      setMsgBodega(r.respuesta_hablada || "Solicitud enviada.");
      hablar(r.respuesta_hablada);
    } catch (e) { setErr(e.message); }
  }

  /** Un turno de conversación: el ciclo completo del agente.
  «mostrar» es lo que se ve como «Usted dijo»; «texto» es lo que de
  verdad se le manda al backend. Al tocar una tarjeta de opción se
  manda el código exacto (nunca falla), no el nombre: si un nombre
  está contenido dentro del otro (ARROZ dentro de ARROZ BASMATI), el
  agente no tiene con qué distinguirlos por palabra. */
  async function procesar(texto, mostrar = texto) {
    if (!texto) return;
    setDicho(mostrar);
    setErr("");
    try {
      const t = await enviarTurno(texto, sesionId, token, {
        opciones, opcionesPara,
        bodegaId: bodega?.bodega_id, bodegaNombre: bodega?.bodega,
      });
      setRespuesta(t.respuesta_hablada || "");
      setOpciones(t.opciones || null);
      setOpcionesPara(t.opciones_para || null);
      setAlerta(t.alerta || null);
      setAlertaEsperado(t.alerta === "desviacion" ? (t.pendiente?.cantidad ?? null) : null);
      setContextoAlerta(t.contexto_alerta || null);
      setPendiente(t.pendiente || null);
      setArchivo(t.archivo || null);
      hablar(t.respuesta_hablada);
      if (t.guardado || t.corregido || t.bodega) refrescar();
      if (t.bodega) {
        setBodega({ bodega: t.bodega.nombre, referencias: t.bodega.referencias, bodega_id: t.bodega.id
      });
    }
    if (t.intencion === "crear") {
      setCrear({ nombre: t.articulo_texto || texto, unidad_medida: "Unidad",
        cantidad_inicial: t.cantidad || 0 });
    } else {
      setCrear(null);
    }
  } catch (e) {
    if (esFalloRed(e)) {
      setOffline(true);
      setErr("");
    } else {
      setErr(e.message);
    }
  } finally {
    setEstado("listo");
  }
}

function recontar() {
  setAlerta(null);

```

```

    setAlertaEsperado(null);
    setContextoAlerta(null);
    setPendiente(null);
    setDicho("");
    const msg = "¿La contamos otra vez? Dígame el nuevo valor.";
    setRespuesta(msg);
    hablar(msg);
  }

  async function crearPendiente() {
    setErr("");
    try {
      const r = await pedir("/api/conteo/crear-producto", {
        method: "POST",
        body: JSON.stringify({ ...crear, sesion_id: sesionId }),
      }, token);
      setRespuesta(r.respuesta_hablada);
      hablar(r.respuesta_hablada);
      setCrear(null);
      setAlerta(null);
      refrescar();
    } catch (e) {
      setErr(e.message);
    }
  }

  function guardarEnCola(item) {
    const nueva = [...leerCola(sesionId), item];
    guardarCola(sesionId, nueva);
    setCola(nueva);
  }

  function alMicrofono() {
    if (estado === "escuchando") {
      rec.current?.stop();
      return;
    }
    rec.current = escuchar({
      alTexto: procesar,
      alEstado: (e, parcial) => {
        setEstado(e);
        if (parcial) setDicho(parcial);
      },
      alError: setErr,
    });
  }

  /** Para elegir bodega (a diferencia del conteo, que confirma hablado
  antes de guardar): esto llena el buscador con lo que se reconoció Y
  pregunta en voz alta "¿confirma X?" - un "sí" abre directo, un "no"
  descarta sin tocar nada. Nombres poco comunes ("Piscilago") son
  justo donde más se equivoca el reconocimiento de voz del navegador;
  "Abrir por nombre exacto" sigue ahí como respaldo si prefiere
  escribir o corregir a mano en vez de confirmar hablado. */
  function alMicrofonoBodega() {
    if (estado === "escuchando") {
      rec.current?.stop();
      return;
    }
    setErr("");
    setMsgBodega("");
    setOpcionesBodega(null);
    const miId = ++idEscuchaBodega.current;
    rec.current = escuchar({
      alTexto: (t) => {
        if (miId !== idEscuchaBodega.current) return;

```

```

    setTextoBodega(t);
    preguntarConfirmacionBodega(t, miId);
  },
  alEstado: (e, parcial) => {
    setEstado(e);
    if (parcial) setTextoBodega(parcial);
  },
  alError: setErr,
});
}

/** Antes esto preguntaba "¿confirma X?" con lo que se acababa de
    transcribir tal cual - decir "kiosco" confirmaba "kiosco" como si
    fuera una bodega real, cuando lo que existe es "KIOSCO TAQUILLA
    AYB". Ahora primero RESUELVE el nombre contra el catálogo
    (/api/bodegas/buscar, sin abrir nada todavía) y confirma con el
    nombre real que de verdad se va a abrir. Si lo dicho es ambiguo
    ("restaurante" a secas encaja en dos bodegas reales por igual), se
    ofrecen las opciones en vez de fallar en seco; si de verdad no
    encuentra nada, ofrece crearla.

    "miId" identifica ESTA escucha en particular (viene de
    idEscuchaBodega, ver el comentario junto a esa ref): antes de tocar
    cualquier estado se revisa que siga siendo la vigente, porque el
    micrófono anterior a veces entrega su resultado tarde - sin este
    control, ese resultado tardío se procesaba como una bodega nueva
    dictada (por ejemplo, el "confirmo" de la pregunta de abajo llegaba
    aquí como si alguien hubiera dicho "confirmo" como nombre de
    bodega). */
async function preguntarConfirmacionBodega(nombreDictado, miId) {
  if (!nombreDictado || !nombreDictado.trim()) return;
  if (miId === undefined) miId = ++idEscuchaBodega.current;
  setOpcionesBodega(null);
  let resuelto;
  try {
    resuelto = await buscarBodega(nombreDictado, token);
  } catch (e) {
    if (miId !== idEscuchaBodega.current) return;
    setErr(e.message);
    return;
  }
  if (miId !== idEscuchaBodega.current) return;
  if (!resuelto.encontrada) {
    if (resuelto.opciones?.length) {
      const nombres = resuelto.opciones.map((o) => o.bodega.toLowerCase());
      setOpcionesBodega(resuelto.opciones);
      // reservar el id ANTES de hablar corta de una vez cualquier
      // escucha anterior que responda tarde mientras se dice esto.
      const miIdOpciones = ++idEscuchaBodega.current;
      // hay que esperar a que la pregunta se termine de decir - si el
      // micrófono arranca apenas empieza a sonar, se pierde el
      // principio de la respuesta (o, si la voz tarda, el micrófono
      // hasta se cansa de esperar antes de que la persona alcance a
      // contestar).
      await hablar(`Hay varias: ${nombres.join(", ")}. ¿Cuál de todas?`);
      if (miIdOpciones !== idEscuchaBodega.current) return;
      // se puede elegir tocando una tarjeta, o siguiendo la conversación
      // y diciendo el nombre completo - que vuelve a pasar por aquí,
      // ahora sí resolviendo a una sola.
      rec.current = escuchar({
        alTexto: (t) => {
          if (miIdOpciones !== idEscuchaBodega.current) return;
          setTextoBodega(t);
          preguntarConfirmacionBodega(t, miIdOpciones);
        },
        alEstado: (e) => setEstado(e),
        alError: setErr,
      });
    }
  }
}

```

```

    });
    return;
  }
  hablar(`No encuentro ${nombreDictado.toLowerCase()}. Puede solicitarla como bodega nueva.`);
  setBodegaNoEncontrada(nombreDictado.trim());
  return;
}
const nombreReal = resuelto.bodega;
setTextoBodega(nombreReal);
const miIdConfirmar = ++idEscuchaBodega.current;
await hablar(`¿Confirma que abro ${nombreReal.toLowerCase()}?`);
if (miIdConfirmar !== idEscuchaBodega.current) return;
rec.current = escuchar({
  alTexto: (respuesta) => {
    if (miIdConfirmar !== idEscuchaBodega.current) return;
    if (esAfirmacion(respuesta)) {
      abrir(nombreReal);
    } else if (esNegacion(respuesta)) {
      setTextoBodega("");
      const msg = "Gracias, queda pendiente. Cuando quiera abrir una bodega, dígame el nombre.";
      setMsgBodega(msg);
      hablar(msg);
    } else {
      setErr("No le entendí un «sí» o un «no». Use «Abrir por nombre exacto».");
    }
  },
  alEstado: (e) => setEstado(e),
  alError: setErr,
});
}

return (
  <Marco
    titulo={`Conteo por voz${bodega ? " " + bodega.bodega : ""}`}
    chip={offline ? { texto: "SIN CONEXIÓN", tipo: "oro" }
      : bodega ? { texto: "EN CONTEO", tipo: "azul" }
      : { texto: "SIN BODEGA", tipo: "gris" }}
  >
    {!bodega ? (
      <div className="card">
        <h3>¿Qué bodega va a contar?</h3>
        <div style={{ display: "flex", gap: 10, alignItems: "center", marginBottom: 6 }}>
          <button
            className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
            style={{ width: 46, height: 46, fontSize: "1.2rem" }}
            onClick={alMicrofonoBodega} title="diga el nombre de la bodega"
            aria-label="Decir el nombre de la bodega por voz">
            <Icono nombre="microfono" tam={22} />
          </button>
          <small style={{ color: "var(--grafito)" }}>
            {estado === "escuchando" ? "Escuchando..."
              : "o diga el nombre de la bodega en voz alta"}
          </small>
        </div>
        <p className="pista" style={{ marginBottom: 8 }}>
          Lo que reconozca aparece abajo, se lo pregunta de vuelta y usted
          confirma diciendo «sí» (nombres como «Piscilago» a veces se
          transcriben mal) – o corrija a mano y use «Abrir por nombre
          exacto».
        </p>
        <details style={{ marginBottom: 8 }}>
          <summary className="pista" style={{ cursor: "pointer" }}>
            Ver frases exactas que entiende el agente aquí
          </summary>
          <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
            {["almacén general", "kiosco taquilla ayb", "sí (confirma la bodega sugerida)",
              "no (descarta la sugerencia)"].map((ej, i) => (

```

```

        <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
      </ul>
    </details>
    <div style={{ display: "flex", gap: 10, flexWrap: "wrap", marginBottom: 14 }}>
      <input
        style={{ flex: 1, minWidth: 240, padding: "12px 14px",
          border: "1px solid var(--borde)", borderRadius: 12 }}
        value={textoBodega}
        onChange={(e) => setTextoBodega(e.target.value)}
        onKeyDown={(e) => e.key === "Enter" && abrirPorBoton()}
        placeholder="nombre de la bodega..."
        aria-label="Nombre de la bodega"
      />
      <button className="btn" onClick={abrirPorBoton}
        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="aprobar" tam={18} />
        Abrir por nombre exacto
      </button>
    </div>
    <div>
      {err && <p className="error" style={{ marginBottom: 10 }}>{err}</p>}
      {msgBodega && <p className="msg-ok" style={{ marginBottom: 10 }}>{msgBodega}</p>}
      {bodegaNoEncontrada && (
        <div className="banner" style={{ marginBottom: 10 }}>
          <span className="ico">!</span>
          <span>
            <b>¿Crear «{bodegaNoEncontrada}»?</b>
            <span>No existe en el catálogo. Queda pendiente de aprobación del
              administrador; el conteo no se detiene por esto.</span>
          </span>
          <button className="btn" style={{ flex: "none" }}
            onClick={solicitarBodegaNueva}>
            Solicitar bodega nueva
          </button>
        </div>
      )}
      {opcionesBodega && (
        <div style={{ marginBottom: 14 }}>
          <p className="pista" style={{ marginBottom: 8 }}>
            Hay varias que encajan con lo dicho – toque una, o diga el
            nombre completo:
          </p>
          <div className="opciones">
            {opcionesBodega.map((o, i) => (
              <button key={o.bodega_id} className="opcion"
                onClick={() => { setOpcionesBodega(null); abrir(o.bodega); }}>
                <span className="n">Opción {i + 1}</span>
                <h4>{o.bodega}</h4>
              </button>
            ))}
          </div>
        </div>
      )}
      {asignadas === null ? (
        <p className="cargando">Cargando sus bodegas asignadas...</p>
      ) : asignadas.length > 0 && !buscarOtra ? (
        <>
          <div className="grid-bodegas">
            {asignadas.map((b) => {
              const [txt, cls] = ETIQUETA_BODEGA[b.estado] || ["?", "gris"];
              const esOtraPersona = b.estado === "en_conteo" && b.persona
                && b.persona.toLowerCase() !== (usuario?.nombre || "").toLowerCase();
              let detalleLinea;
              if (b.estado === "en_conteo" && b.persona) {
                detalleLinea = esOtraPersona
                  ? ` ${b.persona} ya la está contando · ${b.avance_pct ?? 0}%`
                  : `usted la dejó a medias · ${b.avance_pct ?? 0}%`;
              }
            })}
          </div>
        </>
      )}
    </div>
  </div>

```

```

    } else if (b.estado === "en_auditoria" && b.persona) {
      detalleLinea = `${b.persona} auditando · ${b.diferencias ?? 0} dif.`;
    } else if (b.estado === "cerrada" && b.hora_cierre) {
      detalleLinea = `cerrada ${b.hora_cierre}`;
    } else if (b.estado === "pendiente") {
      detalleLinea = "sin iniciar · disponible";
    } else {
      detalleLinea = `${b.referencias} referencias`;
    }
  }
  return (
    <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
      onClick={() => abrir(b.bodega)}
      title={esOtraPersona
        ? "Ya la está contando otra persona; puede intentar de todas formas."
        : ""}>
      <b>{b.bodega}</b>
      <span className={`chip ${cls} est`} >{txt}</span>
      <small>{detalleLinea}</small>
    </button>
  );
}
}}
</div>
<div className="grilla-botones" style={{ marginTop: 14 }}>
  <button className="btn borde"
    onClick={() => { setBuscarOtra(true); if (!todas) cargarTodas(); }}>
    Buscar otra bodega
  </button>
</div>
</>
) : (
  <>
    {asignadas.length > 0 && (
      <p className="pista" style={{ marginBottom: 10 }}>
        Buscando fuera de sus bodegas asignadas.
      </p>
    )}
    {todas === null ? (
      <p className="cargando" style={{ marginTop: 10 }}>Cargando bodegas...</p>
    ) : (
      <div className="grid-bodegas" style={{ marginTop: 14 }}>
        {(textoBodega.trim()
          ? todas.filter((b) => b.bodega.toLowerCase()
            .includes(textoBodega.trim().toLowerCase()))
          : todas
        ).slice(0, 24).map((b) => {
          const [txt, cls] = ETIQUETA_BODEGA[b.estado] || ["?", "gris"];
          return (
            <button key={b.id} className={`tarjeta-bodega ${b.estado}`}
              onClick={() => abrir(b.bodega)}>
              <b>{b.bodega}</b>
              <span className={`chip ${cls} est`} >{txt}</span>
              <small>{b.referencias} referencias</small>
            </button>
          );
        })}
      </div>
    )}
    {todas && !textoBodega.trim() && todas.length > 24 && (
      <p className="pista" style={{ marginTop: 8 }}>
        Mostrando 24 de {todas.length} – escriba para filtrar.
      </p>
    )}
  </>
)

<div className="grilla-botones" style={{ marginTop: 10 }}>
  {asignadas.length > 0 && (
    <button className="btn gris" onClick={() => setBuscarOtra(false)}>
      Volver a sus bodegas asignadas
  )}

```

```

        </button>
      })
    </div>
  </>
})
</div>
) : offline ? (
  <FormularioOffline
    cola={cola}
    onGuardar={guardarEnCola}
    onReintentar={() => { setOffline(!navigator.onLine); if (navigator.onLine) sincronizar(); }}
  />
) : (
  <>
    <div className="chips">
      <span className="chip">Avance: {avance.hechas} de {avance.total}</span>
      <span className={`chip ${avance.alertas ? "oro" : "verde"}`}>
        Alertas: {avance.alertas}
      </span>
      <span className="chip gris">Sesión bloqueada</span>
      {cola.length > 0 && (
        <span className="chip oro">{cola.length} por sincronizar</span>
      )}
    </div>

    {alerta && alerta !== "desviacion" && !crear && (
      <div className="banner">
        <span className="ico">!</span>
        <span style={{ flex: 1 }}>
          <b>
            {alerta === "negativo" ? "Cantidad imposible"
              : alerta === "unidad" ? "Unidad de medida distinta"
              : "Artículo fuera del catálogo"}
          </b>
          <span>{respuesta}</span>
        </span>
      </div>
    )}

    {alerta === "desviacion" && !crear && (
      <AlertaDesviacion
        contexto={contextoAlerta} respuesta={respuesta}
        onRecontar={recontar}
        onConfirmar={() => procesar("confirmo")}
      />
    )}

    {crear && (
      <FormularioCrearProducto
        crear={crear} setCrear={setCrear}
        onCrear={crearPendiente} onCancelar={() => setCrear(null)}
        bodega={bodega.bodega} err={err}
      />
    )}

    {!crear && alerta !== "desviacion" && (
      <div className="conteo-cols">
        <div className="card">
          <p className="rotulo">
            {dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}
          </p>
          <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
          {archivo && (
            <div className="grilla-botones" style={{ marginTop: 10 }}>
              <button className="btn verde"

```

```

        onClick={() => descargarReporte(archivo, token).catch((e) =>
setErr(e.message))})>
        Descargar archivo generado
      </button>
    </div>
  )}

  {opciones && (
    <div className="opciones" style={{ marginTop: 14 }}>
      {opciones.map((o, i) => (
        <button key={o.codigo} className="opcion"
          onClick={() => procesar(o.codigo, o.nombre)}>
          <span className="n">Opción {i + 1}</span>
          <h4>{o.nombre}</h4>
          <p>Código {o.codigo}</p>
          <p>{o.unidad}</p>
        </button>
      ))}
    </div>
  )}

  {pendiente && !opciones && (
    <p className="pista" style={{ marginTop: 12 }}>
      Por confirmar: <b>{pendiente.nombre}</b> - {pendiente.cantidad}{ " " }
      {pendiente.unidad}
    </p>
  )}

  <h3 style={{ marginTop: 18 }}>Últimos registros confirmados</h3>
  {avance.ultimos?.length ? (
    avance.ultimos.map((u, i) => (
      <div className="registro" key={i}>
        <span className="ok">✓</span>
        <span>{u.nombre}</span>
        <span className="cant">{u.cantidad} {u.unidad}</span>
      </div>
    ))
  ) : (
    <p className="vacio">Todavía no hay registros en esta sesión.</p>
  )}
</div>

<div className="card mic-caja">
  <button
    className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
    onClick={alMicrofono}
    aria-label="Toque y hable para dictar el artículo y la cantidad contada"
  >
    <Icono nombre="microfono" tam={52} />
  </button>
  <b>
    {estado === "escuchando" ? "Escuchando..."
      : estado === "procesando" ? "Procesando..."
      : "Toque y hable"}
  </b>
  <small>
    {vozDisponible()
      ? "Reconocimiento en español (Colombia)"
      : "Este navegador no reconoce voz: use el teclado"}
  </small>
  <details style={{ marginTop: 8, textAlign: "left" }}>
    <summary className="pista" style={{ cursor: "pointer" }}>
      Ver frases exactas que entiende el agente aquí
    </summary>
    <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
      {["tres tablas para picar blancas", "hay noventa cazuelas",

```



```

        "me faltan cinco kilos de arroz", "corregir", "son nueve",
        "confirmo", "¿cuánto arroz hay contado?"].map((ej, i) => (
        <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
        )))
    </ul>
</details>
{err && <p className="error">{err}</p>}
</div>
</div>
)}

{!crear && alerta !== "desviacion" && (
  <>
    <div className="grilla-botones">
      <button className="btn verde" onClick={() => procesar("confirmo")}>
        Confirmar
      </button>
      <button className="btn borde" onClick={() => procesar("corregir")}>
        Corregir
      </button>
      <button className="btn gris" onClick={() => setMostrarTeclado(true)}>
        Teclado
      </button>
      <button className="btn oro" onClick={() => ir("ayuda")}>
        Pedir ayuda
      </button>
    </div>
    <p className="pista" style={{ marginTop: 10 }}>
      La voz es el camino rápido; el teclado siempre queda como respaldo.
    </p>
  </>
)}
</>
)}
}

{mostrarTeclado && (
  <Dialogo titulo="Escribir en vez de hablar"
    mensaje="Escriba lo que diría en voz alta:"
    conCampo placeholder="tres tablas para picar blancas"
    onAceptar={(t) => { setMostrarTeclado(false); if (t) procesar(t); }}
    onCancel={() => setMostrarTeclado(false)} />
)}
</Marco>
);
}

const fmt = (n) => Number(n).toLocaleString("es-CO", { maximumFractionDigits: 2 });

function AlertaDesviacion({ contexto, respuesta, onRecontar, onConfirmar }) {
  return (
    <>
      <div className="banner">
        <span className="ico">!</span>
        <span>
          <b>Cantidad fuera de lo esperado</b>
          <span>
            {contexto
              ? `El sistema espera alrededor de ${fmt(contexto.esperado)} · usted dijo `
              + `${fmt(contexto.dicho)} (${contexto.articulo)}`
              : respuesta}
            </span>
          </span>
        </div>

      <div className="card">

```

```

    <p className="rotulo">CuentaVoz responde</p>
    <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
  </div>

  <div className="conteo-cols">
    {contexto ? (
      <div className="card">
        <h3>Contexto de la validación</h3>
        <table>
          <tbody>
            <tr><td>Stock del sistema</td><td><b style={{ color: "var(--azul)" }}>
              {fmt(contexto.esperado)} {contexto.unidad}</b></td></tr>
            <tr><td>Su conteo</td><td><b style={{ color: "var(--azul)" }}>
              {fmt(contexto.dicho)} {contexto.unidad}</b></td></tr>
            <tr><td>Desviación</td><td className="dif">
              {contexto.desviacion_pct != null
                ? `${contexto.desviacion_pct > 0 ? "+" : ""}${contexto.desviacion_pct}%`
                : "-"}</td></tr>
          </tbody>
        </table>
      </div>
    ) : <div />}
    <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
      <button className="btn" onClick={onRecontar}>RECONTAR</button>
      <button className="btn oro" onClick={onConfirmar}>
        CONFIRMAR {contexto ? fmt(contexto.dicho) : ""}
      </button>
    </div>
  </div>

  <p className="pista">
    Si confirma, el registro se guarda marcado para revisión del administrador de bodega.
  </p>
  <p className="pista" style={{ color: "var(--azul)" }}>
    Ninguna cantidad con alerta se guarda sin una confirmación explícita de la persona.
  </p>
</>
);
}

function FormularioCrearProducto({ crear, setCrear, onCreatear, onCancel, bodega, err }) {
  const [estado, setEstado] = useState("listo");
  // El micrófono principal de Conteo se oculta mientras se decide si
  // crear este producto (para no mezclar esa respuesta con un conteo
  // nuevo) - sin esto, la pregunta hablada "¿lo creamos?" solo se podía
  // contestar con clic, aunque el resto de la pantalla es por voz.
  function alMicrofono() {
    escuchar({
      alTexto: (texto) => {
        setEstado("listo");
        if (esAfirmacion(texto, ["crea", "crear"])) { onCreatear(); return; }
        if (esNegacion(texto)) { onCancel(); return; }
      },
      alEstado: setEstado,
      alError: () => {},
    });
  }
  return (
    <div className="card">
      <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", gap: 10 }}>
        <p className="rotulo" style={{ margin: 0 }}>CuentaVoz responde</p>
        <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
          style={{ width: 46, height: 46, fontSize: "1.2rem", flex: "none" }}
          onClick={alMicrofono} title="diga «sí» para crear, o «no» para cancelar"
          aria-label="Responder por voz: diga sí para crear, o no para cancelar">

```

```

        <Icono nombre="microfono" tam={20} />
      </button>
    </div>
    <div className="burbuja" aria-live="polite" aria-atomic="true">
      No encontré «{crear.nombre}» en el catálogo. Lo creamos y queda pendiente
      del administrador.
    </div>
    <p className="pista" style={{ marginTop: 8 }}>o diga «sí» para crearlo, o «no» para cancelar</p>

    <div className="conteo-cols" style={{ marginTop: 16 }}>
      <div>
        <label className="pista" htmlFor="campo-nombre-articulo">Nombre tal como lo dictó</label>
        <input id="campo-nombre-articulo" value={crear.nombre}
          onChange={(e) => setCrear({ ...crear, nombre: e.target.value })}
          style={{ width: "100%", padding: "11px 13px", marginTop: 6,
            border: "1px solid var(--borde)", borderRadius: 12,
            textTransform: "uppercase" }} />
      </div>
      <div>
        <label className="pista">Estado del registro</label>
        <div style={{ marginTop: 6 }}>
          <span className="chip oro">PENDIENTE DE APROBACIÓN</span>
        </div>
      </div>
    </div>

    <div className="dos-columnas" style={{ marginTop: 16 }}>
      <div>
        <label className="pista" id="etiqueta-unidad-medida">Unidad de medida</label>
        <div className="grilla-botones" style={{ marginTop: 6 }} role="group" aria-
labelledby="etiqueta-unidad-medida">
          {UNIDADES.map((u) => (
            <button key={u}
              className={`btn ${crear.unidad_medida === u ? "" : "borde"} `}
              onClick={() => setCrear({ ...crear, unidad_medida: u })}
              aria-pressed={crear.unidad_medida === u}>
              {u}
            </button>
          ))}
        </div>
      </div>
      <div>
        <label className="pista" htmlFor="campo-cantidad-inicial">Cantidad inicial contada en
{bodega}</label>
        <input id="campo-cantidad-inicial" type="number" min="0" value={crear.cantidad_inicial}
          onChange={(e) => setCrear({ ...crear, cantidad_inicial: Number(e.target.value) })}
          style={{ width: "100%", padding: "11px 13px", marginTop: 6,
            border: "1px solid var(--borde)", borderRadius: 12 }} />
      </div>
    </div>

    <p className="pista" style={{ marginTop: 12 }}>
      No entra al catálogo oficial hasta la firma del administrador de bodega.
    </p>
    {err && <p className="error" style={{ marginTop: 6 }}>{err}</p>}
    <div className="grilla-botones">
      <button className="btn" onClick={onCrear}>Crear pendiente</button>
      <button className="btn gris" onClick={onCancelar}>Cancelar</button>
    </div>
    <p className="pista" style={{ marginTop: 10 }}>
      El conteo continúa sin interrupción: la aprobación ocurre en paralelo.
    </p>
  </div>
);
}

```

```

/** Mismo ciclo de confirmación del agente real (dice el artículo y la
    cantidad, CuentaVoz repite y pregunta, la persona confirma) pero
    corriendo enteramente en el navegador con interpreteLocal.js - sin
    tocar el backend ni el internet para nada. El reconocimiento de voz
    del navegador sí necesita señal (no es algo que esta app controle),
    así que aquí se escribe en vez de dictar; el resto de la conversación
    - entender la frase, sugerir el nombre oficial, pedir confirmación -
    sigue funcionando igual. */
function FormularioOffline({ cola, onGuardar, onReintentar }) {
  const [catalogo] = useState(() => leerCatalogo());
  const [texto, setTexto] = useState("");
  const [dicho, setDicho] = useState("");
  const [respuesta, setRespuesta] = useState(
    "Sin conexión, pero sigo entendiendo lo que escriba. Dígame el artículo y la cantidad."
  );
  const [pendiente, setPendiente] = useState(null); // {articulo, cantidad, unidad, codigo}
  const [mostrarManual, setMostrarManual] = useState(false);

  // el formulario campo por campo de siempre, como respaldo del respaldo
  // si una frase no se deja interpretar bien.
  const [articulo, setArticulo] = useState("");
  const [cantidad, setCantidad] = useState(1);
  const [unidad, setUnidad] = useState("Unidad");
  const sugerenciaManual = sugerirDeCatalogo(articulo, catalogo);

  function procesarTexto(t) {
    if (!t.trim()) return;
    setDicho(t);
    setTexto("");
    const r = interpretarLocal(t);
    const simple = t.trim().toLowerCase();
    const esSi = /^(sí|sí|confirmo|correcto|dale|listo)\b/.test(simple);
    const esNo = /^no\b/.test(simple);

    if (pendiente && (esSi || r.intencion === "confirmar")) {
      onGuardar({ articulo: pendiente.articulo, cantidad: pendiente.cantidad,
        unidad: pendiente.unidad });
      setPendiente(null);
      setRespuesta("Guardado en este equipo. Se sincroniza solo cuando vuelva la señal.");
      return;
    }
    if (pendiente && esNo) {
      setPendiente(null);
      setRespuesta("Listo, no lo guardo. Dígame el artículo y la cantidad correctos.");
      return;
    }
    if (pendiente && r.intencion === "corregir" && r.cantidad !== null) {
      setPendiente({ ...pendiente, cantidad: r.cantidad });
      setRespuesta(` ${pendiente.articulo}: ${r.cantidad} ${pendiente.unidad}. ¿Confirma?`);
      return;
    }
    if (r.intencion === "contar" && r.cantidad !== null) {
      const sug = sugerirDeCatalogo(r.articulo_texto, catalogo);
      const nombreFinal = sug?.nombre || r.articulo_texto || "ese artículo";
      const unidadFinal = sug?.unidad || r.unidad || "Unidad";
      setPendiente({ articulo: nombreFinal, cantidad: r.cantidad,
        unidad: unidadFinal, codigo: sug?.codigo });
      setRespuesta(` ${nombreFinal}: ${r.cantidad} ${unidadFinal}. ¿Confirma? (escriba «sí») `);
      return;
    }
    setRespuesta(r.respuesta_hablada
      || "No le entendí bien. Dígame el artículo y la cantidad, por ejemplo «hay noventa cazuelas.»");
  }

  function usarSugerenciaManual() {
    setArticulo(sugerenciaManual.nombre);
  }

```

```

    setUnidad(sugerenciaManual.unidad);
  }
  function ciclarUnidad() {
    setUnidad(UNIDADES[(UNIDADES.indexOf(unidad) + 1) % UNIDADES.length]);
  }
  function guardarManual() {
    if (!articulo.trim()) return;
    onGuardar({ articulo: articulo.trim(), cantidad: Number(cantidad), unidad });
    setArticulo("");
    setCantidad(1);
  }
}

return (
  <div className="conteo-cols">
    <div className="card">
      <div className="banner">
        <span className="ico">!</span>
        <span>
          <b>Sin conexión</b>
          <span>El reconocimiento de voz necesita señal, pero CuentaVoz sigue
            entendiendo lo que escriba. Lo que registre se guarda en este
            equipo y se sincroniza solo al volver la red.</span>
        </span>
      </div>
    </div>

    <p className="rotulo">{dicho ? `Usted escribió: «${dicho}»` : "CuentaVoz responde"}</p>
    <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>

    <div style={{ display: "flex", gap: 10, marginTop: 12, flexWrap: "wrap" }}>
      <input value={texto} onChange={(e) => setTexto(e.target.value)}
        onKeyDown={(e) => e.key === "Enter" && procesarTexto(texto)}
        placeholder="hay noventa cazuelas blancas..."
        aria-label="Escriba el artículo y la cantidad contada"
        style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
          border: "1px solid var(--borde)", borderRadius: 12 }} />
      <button className="btn" onClick={() => procesarTexto(texto)}>Enviar</button>
    </div>

    <div className="grilla-botones" style={{ marginTop: 10 }}>
      <button className="btn borde" onClick={onReintentar}>Reintentar la voz</button>
      <button className="btn gris" onClick={() => setMostrarManual((m) => !m)}>
        {mostrarManual ? "Ocultar formulario manual" : "Registrar campo por campo"}
      </button>
    </div>

    {mostrarManual && (
      <div style={{ marginTop: 14, padding: 14, borderTop: "1px solid var(--borde)" }}>
        <label className="pista" htmlFor="campo-articulo-manual">Artículo</label>
        <input id="campo-articulo-manual" value={articulo} onChange={(e) =>
          setArticulo(e.target.value)}
          placeholder="tabla acril..."
          style={{ width: "100%", padding: "11px 13px", marginBottom: 8,
            border: "1px solid var(--borde)", borderRadius: 12 }} />
        {sugerenciaManual && (
          <button className="chip azul" onClick={usarSugerenciaManual}
            style={{ display: "block", width: "100%", textAlign: "left",
              marginBottom: 6 }}>
            {sugerenciaManual.nombre} {sugerenciaManual.codigo}
          </button>
        )}
        <label className="pista">Cantidad</label>
        <div style={{ display: "flex", gap: 10, marginTop: 6, marginBottom: 10,
          alignItems: "center" }}>
          <button className="btn borde" style={{ minHeight: 0, padding: "8px 16px" }}
            onClick={() => setCantidad((c) => Math.max(0, Number(c) - 1))}>

```

```

        aria-label="Restar uno a la cantidad">-</button>
      <input type="number" value={cantidad}
        onChange={(e) => setCantidad(e.target.value)}
        aria-label="Cantidad contada"
        style={{ width: 90, padding: "11px 13px", textAlign: "center",
          border: "1px solid var(--borde)", borderRadius: 12 }} />
      <button className="btn borde" style={{ minHeight: 0, padding: "8px 16px" }}
        onClick={() => setCantidad((c) => Number(c) + 1)}
        aria-label="Sumar uno a la cantidad">+</button>
      <button className="chip azul" onClick={ciclarUnidad}
        aria-label={`Unidad: ${unidad}. Toque para cambiarla`}>{unidad}</button>
    </div>
    <button className="btn" onClick={guardarManual}>Guardar</button>
  </div>
)}

<p className="pista" style={{ marginTop: 12 }}>
  El catálogo local sigue sugiriendo nombres oficiales sin internet.
</p>
</div>

<div className="card">
  <h3>Guardados en este equipo</h3>
  {cola.length === 0 ? (
    <p className="vacio">Nada por sincronizar todavía.</p>
  ) : (
    cola.map((c, i) => (
      <div className="registro" key={i}>
        <span>{c.articulo}</span>
        <span className="cant">{c.cantidad} {c.unidad}</span>
      </div>
    ))
  )}
  {cola.length > 0 && (
    <span className="chip oro">{cola.length} registros por sincronizar</span>
  )}
  <p className="pista" style={{ marginTop: 10 }}>
    Al volver la señal, la cola se envía sola a la API y las validaciones
    se aplican igual que en línea.
  </p>
</div>
</div>
);
}

```

12.29 Frontend: Bodegas — tablero, detalle y consulta (vistas 5.13 a 5.15)

Archivo: `frontend/src/vistas/Bodegas.jsx`

```

import { useEffect, useState, useRef } from "react";
import { pedir, BASE, leerToken, descargarReporte, preguntarAsistente } from "../api";
import { escuchar, hablar, esAfirmacion, esNegacion, quitarTildes } from "../voz";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import Icono from "../Iconos";

const ETIQUETA = {
  pendiente: ["PENDIENTE", "gris"], en_conteo: ["EN CONTEO", "azul"],
  en_auditoria: ["EN AUDITORÍA", "oro"], cerrada: ["CERRADA", "verde"],
};
const ICONO_ESTADO = {
  pendiente: "🕒", en_conteo: "🕒", en_auditoria: "🔍", cerrada: "☑️",
};

```

```

// Frases que este buscador ya entiende sin pasar por el agente general -
// se resuelven de una en el propio frontend (_accionLocalDeLista /
// _accionLocalDeResultado más abajo). Documentarlas aquí es lo mismo que
// el desplegable "Ver frases exactas" de las demás pantallas.
function _ejemplosLista(esAuditor) {
  return [
    "arroz, aceite, tres tablas para picar...",
    "bodega nueva",
    ...(esAuditor ? ["exportar estado", "asignar personas a bodegas"] : []),
    "ver movimientos", "comparar con la receta",
    ...(esAuditor ? ["generar reporte"] : []),
    "volver al tablero",
  ];
}

function _ejemplosDetalle(esAuditor, estado) {
  return [
    "un artículo de esta bodega",
    ...(estado === "pendiente" || estado === "en_conteo" ? ["seguir contando"] : []),
    ...(esAuditor && estado === "cerrada" ? ["reabrir"] : []),
    "todos los artículos",
    ...(esAuditor ? ["descargar", "ver en el panel gerencial"] : []),
    "cerrar detalle",
  ];
}

export default function Bodegas({ token, usuario, ir }) {
  const [lista, setLista] = useState(null);
  const [filtro, setFiltro] = useState("todas");
  const [detalle, setDetalle] = useState(null);
  const [detalleId, setDetalleId] = useState(null);
  const [cargandoDetalle, setCargandoDetalle] = useState(false);
  const [busca, setBusca] = useState("");
  const [consulta, setConsulta] = useState(null);
  const [escuchando, setEscuchando] = useState(false);
  const [movimientos, setMovimientos] = useState(null);
  const [enRecetas, setEnRecetas] = useState(null);
  const [articulosBodega, setArticulosBodega] = useState(null);
  const [buscaArticuloBodega, setBuscaArticuloBodega] = useState("");
  const [escuchandoBodega, setEscuchandoBodega] = useState(false);
  // Cuando lo buscado en el detalle no es ningún artículo de ESTA bodega,
  // la respuesta del agente general (pregunta suelta u orden de navegar)
  // - mismo patrón que "consulta" en la lista principal.
  const [respuestaAgenteDetalle, setRespuestaAgenteDetalle] = useState(null);
  const [msg, setMsg] = useState("");
  const [pedirMotivo, setPedirMotivo] = useState(false);
  const [pidiendoBodegaNueva, setPidiendoBodegaNueva] = useState(false);
  const esAuditor = usuario?.perfil === "auditor";
  const rec = useRef(null);
  // Igual que en el selector de bodega de Conteo: cada escucha de esta
  // búsqueda lleva su propio número, y si el micrófono anterior entrega
  // su resultado tarde (mientras todavía se está diciendo la sugerencia)
  // se descarta en vez de procesarse como una búsqueda nueva.
  const idEscuchaBusqueda = useRef(0);

  useEffect(() => {
    // si falla, resuelve a [] en vez de dejar lista en null para siempre:
    // sin esto, un fallo de red aquí dejaba el tablero mostrando
    // "Cargando..." sin fin, sin manera de saber que algo salió mal.
    pedir("/api/bodegas?propias=1", {}, token).then(setLista)
      .catch((e) => { setLista([]); setMsg(e.message); });
    /* tablero en vivo: el WebSocket avisa a todas las tabletas */
    const ws = new WebSocket(
      BASE.replace("http", "ws") + "/api/bodegas/estado?token=" + (token || leerToken())
    );
    ws.onmessage = (e) => {
      const estados = JSON.parse(e.data);
      setLista((previa) => {

```

```

const prev = previa || [];
const hayNuevas = estados.some((x) => !prev.some((b) => b.id === x.id));
if (hayNuevas) {
  // bodega creada despues de cargar el tablero: se necesita el
  // listado completo (referencias, persona, avance) que el
  // WebSocket no manda, no solo el id/estado.
  pedir("/api/bodegas?propias=1", {}, token).then(setLista).catch(() => {});
  return prev;
}
return prev.map((b) => {
  const n = estados.find((x) => x.id === b.id);
  return n ? { ...b, estado: n.estado } : b;
});
});
};
return () => ws.close();
}, [token]);

const vistas = (lista || []).filter((b) => filtro === "todas" || b.estado === filtro);
const cuenta = (e) => (lista || []).filter((b) => b.estado === e).length;

async function verDetalle(id) {
  // Si se llega desde la sugerencia de bodega del buscador de
  // ingredientes, "consulta" sigue puesto - sin limpiarlo, la sección
  // de detalle no se muestra (su condición exige "!consulta").
  setConsulta(null);
  setDetalleId(id);
  setArticulosBodega(null);
  setBuscaArticuloBodega("");
  setRespuestaAgenteDetalle(null);
  setCargandoDetalle(true);
  try {
    setDetalle(await pedir(`/api/bodegas/${id}/detalle`, {}, token));
  } finally {
    setCargandoDetalle(false);
  }
}

async function verTodosLosArticulos() {
  if (articulosBodega) { setArticulosBodega(null); return; }
  setArticulosBodega(await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token));
}

async function buscarEnBodega(texto) {
  setBuscaArticuloBodega(texto);
  setRespuestaAgenteDetalle(null);
  if (texto && !articulosBodega) {
    setArticulosBodega(await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token));
  }
}

function _filtrarArticulosBodega(texto, lista) {
  const palabras = texto.toLowerCase().replace(/[,;./g, " ").split(/\s+/).filter(Boolean);
  if (!palabras.length) return lista || [];
  return (lista || []).filter((a) => {
    const nombre = a.articulo.toLowerCase();
    return palabras.every((p) => nombre.includes(p));
  });
}

// Al confirmar (Enter, o al terminar de dictar): si lo escrito/dicho no
// encuentra ningún artículo DENTRO de esta bodega, cae al mismo agente
// general que ya usa la lista principal - un solo cuadro para todo,
// también aquí, en vez de un segundo "Pregúntele al agente" aparte.
// Igual que _accionLocalDeResultado, pero para los botones del DETALLE
// de una bodega (Cerrar detalle, Seguir contando...) - se revisan antes
// que la búsqueda de artículo/el agente general, porque esos botones
// no tienen equivalente en ningún otro lado: decir "cerrar detalle" sin
// esto caía en el agente general, que ni sabe qué botones hay en esta
// pantalla en particular ("Esa opción no la puedo hacer por voz").
function _accionLocalDeDetalle(texto) {

```



```

const t = quitarTildes(texto.toLowerCase()).replace(/[.,;:!?¿?]/g, "").trim();
if (/\\b(cerrar|cierra|salir)\\b/.test(t)) return "cerrar";
if (/\\b(seguir|sigue|continú|w*|contar|cuenta)\\b/.test(t)
  && (detalle?.estado === "pendiente" || detalle?.estado === "en_conteo")) return "contar";
if (/\\b(reabrir|reabre)\\b/.test(t) && esAuditor && detalle?.estado === "cerrada") return
"reabrir";
if (/\\b(artículos?)\\b/.test(t)) return "articulos";
if (/\\b(descargar)\\b/.test(t) && esAuditor) return "descargar";
if (/\\b(panel)\\b/.test(t) && esAuditor) return "panel";
return null;
}
async function confirmarBusquedaEnBodega(texto, miId) {
  if (!texto || !texto.trim()) return;
  if (miId === undefined) miId = ++idEscuchaBusqueda.current;
  const accion = _accionLocalDeDetalle(texto);
  if (accion === "cerrar") { setDetalle(null); setDetalleId(null); return; }
  if (accion === "contar") { ir && ir("conteo", { bodegaSugerida: detalle.bodega }); return; }
  if (accion === "reabrir") { setPedirMotivo(true); return; }
  if (accion === "articulos") { verTodosLosArticulos(); return; }
  if (accion === "descargar") { exportarDetalle(); return; }
  if (accion === "panel") { ir && ir("panel"); return; }
  setBuscaArticuloBodega(texto);
  let lista = articulosBodega;
  if (!lista) {
    lista = await pedir(`/api/bodegas/${detalleId}/articulos`, {}, token);
    if (miId !== idEscuchaBusqueda.current) return;
    setArticulosBodega(lista);
  }
  if (_filtrarArticulosBodega(texto, lista).length > 0) {
    setRespuestaAgenteDetalle(null);
    return;
  }
  const r = await preguntarAsistente(texto, "bodegas", token);
  if (miId !== idEscuchaBusqueda.current) return;
  setRespuestaAgenteDetalle(r);
  hablar(r.respuesta_hablada);
  if (r.accion === "navegar" && r.destino && ir) {
    ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined });
  }
}
function buscarEnBodegaPorVoz() {
  setMsg("");
  const miId = ++idEscuchaBusqueda.current;
  rec.current = escuchar({
    alTexto: (t) => {
      if (miId !== idEscuchaBusqueda.current) return;
      confirmarBusquedaEnBodega(t, miId);
    },
    alEstado: (e) => setEscuchandoBodega(e === "escuchando"),
    alError: setMsg,
  });
}
// Frases que activan un botón YA VISIBLE en el resultado de una
// búsqueda anterior ("volver al tablero", "ver movimientos..."), en
// vez de tratarse como una búsqueda nueva. Van ANTES que cualquier
// otra cosa: "tablero" es también parte de un producto real
// ("BORRADOR PARA TABLERO FELPA"), así que sin este chequeo "volver al
// tablero" perdía contra ese artículo en la búsqueda por palabras y
// nunca se reconocía como la orden que era.
function _accionLocalDeResultado(texto) {
  const t = quitarTildes(texto.toLowerCase()).replace(/[.,;:!?¿?]/g, "").trim();
  if (/\\b(volver|tablero|atras|regresar)\\b/.test(t)) return "volver";
  if (/\\bmovimientos?\\b/.test(t)) return "movimientos";
  if (/\\brejetas?\\b/.test(t)) return "receta";
  if (/\\breporte\\b/.test(t)) return "reporte";
  return null;
}

```

```

// "miId" solo llega cuando la búsqueda vino de buscarPorVoz() (ver ese
// comentario) - una búsqueda escrita y con clic en "Buscar" no debe
// quedarse esperando una respuesta hablada que nadie va a dar.
// Órdenes sobre los botones de la lista principal ("Bodega nueva",
// "Exportar estado", "Asignar personas a bodegas") - mismo patrón que
// _accionLocalDeResultado/_accionLocalDeDetalle: se revisan antes que
// cualquier búsqueda, porque el agente general no sabe qué botones hay
// en esta pantalla en particular.
function _accionLocalDeLista(texto) {
  const t = quitarTildes(texto.toLowerCase()).replace(/[.,;:!?;]/g, "").trim();
  if (/bodega\b/.test(t) && /\b(nueva|crear|solicitar)\b/.test(t)) return "nueva";
  if (/exportar\b/.test(t) && esAuditor) return "exportar";
  if (/asignar\b/.test(t) && esAuditor) return "asignar";
  return null;
}
async function buscarArticulo(codigo = "", texto = busca, miId) {
  if (!texto.trim()) return;
  if (!consulta && !detalle) {
    const accionLista = _accionLocalDeLista(texto);
    if (accionLista === "nueva") { setPidiendoBodegaNueva(true); return; }
    if (accionLista === "exportar") { exportarEstado(); return; }
    if (accionLista === "asignar") { ir && ir("ajustes", { tabInicial: "usuarios" }); return; }
  }
  // Si queda una sugerencia de bodega pendiente en pantalla, un "sí"/
  // "no" nuevo la responde - sin importar si llegó como continuación
  // de la misma escucha o como una búsqueda nueva escrita/hablada
  // aparte (tocando "Buscar" de nuevo, por ejemplo). Antes, un "sí"
  // que no llegara EXACTAMENTE como respuesta de esa misma escucha se
  // trataba como una búsqueda nueva de "sí" a secas, y el agente
  // general (que no tiene ni idea de la sugerencia pendiente)
  // respondía cualquier cosa en vez de abrir el detalle.
  if (consulta?.sugerencia_bodega_id) {
    if (esAfirmacion(texto, ["ver"])) { verDetalle(consulta.sugerencia_bodega_id); return; }
    if (esNegacion(texto)) {
      setConsulta(null);
      setBusca("");
      const msg = "Listo, queda ahí. Puede seguir buscando cuando quiera.";
      setMsg(msg);
      hablar(msg);
      return;
    }
  }
  if (consulta?.bodegas?.length > 0) {
    const accion = _accionLocalDeResultado(texto);
    if (accion === "volver") { volverAlTablero(); return; }
    if (accion === "movimientos") { verMovimientos(); return; }
    if (accion === "receta") { verEnRecetas(); return; }
    if (accion === "reporte" && esAuditor) { generarReporteArticulo(); return; }
  }
  setBusca(texto);
  setMovimientos(null);
  setEnRecetas(null);
  const r = await pedir(
    `/api/articulos/consulta?q=${encodeURIComponent(texto)}&codigo=${codigo}`, {}, token);
  if (miId !== undefined && miId !== idEscuchaBusqueda.current) return;
  setConsulta(r);
  // Un solo buscador para todo: si ni fue un artículo ni una bodega,
  // el backend ya lo intentó como pregunta suelta o como orden de
  // navegar ("llévame a reportes", "abra kiosco taquilla ayb") - si
  // trae una acción, se seguía igual que hace AsistenteVoz.
  if (r.accion === "navegar" && r.destino && ir) {
    hablar(r.resumen);
    ir(r.destino, { tabInicial: r.pestana || undefined, bodegaSugerida: r.bodega || undefined });
    return;
  }
  // Si la búsqueda fue por voz y salió una bodega sugerida, no basta
  // con decir la sugerencia y dejarla ahí esperando un clic: se

```

```

// pregunta y se escucha la respuesta, igual que ya hace Conteo al
// confirmar una bodega - "tocar el botón" no debería ser la única
// forma de seguir cuando se llegó hasta aquí hablando.
if (miId !== undefined && r.sugerencia_bodega_id) {
  await hablar(`${r.resumen} Diga «sí» para ver el detalle.`);
  if (miId !== idEscuchaBusqueda.current) return;
  rec.current = escuchar({
    alTexto: (respuesta) => {
      if (miId !== idEscuchaBusqueda.current) return;
      if (esAfirmacion(respuesta, ["ver"])) {
        verDetalle(r.sugerencia_bodega_id);
      } else if (esNegacion(respuesta)) {
        const msg = "Listo, queda ahí. Puede seguir buscando cuando quiera.";
        setMsg(msg);
        hablar(msg);
      } else {
        setMsg("No le entendí un «sí» o un «no». Use el botón «Ver esta bodega.»);
      }
    },
    alEstado: (e) => setEscuchando(e === "escuchando"),
    alError: setMsg,
  });
  return;
}
hablar(r.resumen);
}
function buscarPorVoz() {
  setMsg("");
  const miId = ++idEscuchaBusqueda.current;
  rec.current = escuchar({
    alTexto: (t) => {
      if (miId !== idEscuchaBusqueda.current) return;
      buscarArticulo("", t, miId);
    },
    alEstado: (e) => setEscuchando(e === "escuchando"),
    alError: setMsg,
  });
}
function volverAlTablero() {
  setConsulta(null);
  setBusca("");
}
async function verMovimientos() {
  setMovimientos(await pedir(`/api/articulos/${consulta.codigo}/movimientos`, {}, token));
}
async function verEnRecetas() {
  setEnRecetas(await pedir(`/api/articulos/${consulta.codigo}/en-recetas`, {}, token));
}
async function generarReporteArticulo() {
  setMsg("");
  try {
    const r = await pedir("/api/reportes?formato=xlsx", { method: "POST" }, token);
    await descargarReporte(r.archivo, token);
    setMsg("Reporte generado y descargado.");
  } catch (e) { setMsg(e.message); }
}
async function reabrir(motivo) {
  setPedirMotivo(false);
  if (!motivo || !motivo.trim()) return;
  try {
    await pedir(`/api/bodegas/${detalleId}/reabrir`, {
      method: "POST", body: JSON.stringify({ motivo }),
    }, token);
    setMsg("Bodega reabierta: queda registrada la justificación.");
    verDetalle(detalleId);
    pedir("/api/bodegas?propias=1", {}, token).then(setLista).catch(() => {});
  } catch (e) { setMsg(e.message); }
}

```

```

}

async function solicitarBodegaNueva(nombre) {
  setPidiendoBodegaNueva(false);
  if (!nombre || !nombre.trim()) return;
  try {
    const r = await pedir("/api/bodegas/crear-pendiente", {
      method: "POST", body: JSON.stringify({ nombre }),
    }, token);
    setMsg(r.respuesta_hablada || "Solicitud enviada.");
  } catch (e) { setMsg(e.message); }
}

async function exportarEstado() {
  setMsg("");
  try {
    const r = await pedir("/api/bodegas/exportar-estado?formato=xlsx", { method: "POST" }, token);
    await descargarReporte(r.archivo, token);
    setMsg("Estado del tablero exportado y descargado.");
  } catch (e) { setMsg(e.message); }
}

async function exportarDetalle() {
  setMsg("");
  try {
    const r = await pedir(`/api/bodegas/${detalleId}/exportar-detalle?formato=xlsx`,
      { method: "POST" }, token);
    await descargarReporte(r.archivo, token);
    setMsg("Detalle de la bodega exportado y descargado.");
  } catch (e) { setMsg(e.message); }
}

const tituloDetalle = detalle ? `Bodegas · ${detalle.bodega.toLowerCase().replace(/\b\w/g, (c) =>
c.toUpperCase())}` : "";
const chipDetalle = detalle ? (ETIQUETA[detalle.estado] || [detalle.estado?.toUpperCase(), ""]) :
null;
// solo bloquea la pantalla con "Cargando..." en la primera carga de un
// detalle: si ya había uno (ej. reabrir() refrescando el mismo), se deja
// ver el dato viejo mientras llega el nuevo, en vez de taparlo con un
// aviso de carga que compite con el detalle todavía visible.
const mostrarCargandoDetalle = cargandoDetalle && !detalle;

return (
  <Marco titulo={consulta ? "Bodegas · consulta de artículo" : detalle ? tituloDetalle : "Bodegas
· estado en vivo"}
    chip={consulta ? { texto: "BÚSQUEDA GLOBAL", tipo: "borde azul" }
      : detalle ? { texto: chipDetalle[0], tipo: `borde ${chipDetalle[1]}` }
      : { texto: "EN VIVO", tipo: "verde" }}>

    {mostrarCargandoDetalle && <p className="cargando">Cargando...</p>}

    {!detalle && !mostrarCargandoDetalle && (
      <div className="card">
        <div style={{ display: "flex", gap: 10, alignItems: "center", flexWrap: "wrap" }}>
          <span style={{ color: "var(--grafito)" }}>🔍</span>
          <input value={busca} onChange={(e) => setBusca(e.target.value)}
            onKeyDown={(e) => e.key === "Enter" && buscarArticulo()}
            placeholder="arroz, aceite, cazuela..."
            aria-label="Buscar artículo, o pregúntele algo al agente"
            style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
              border: "1px solid var(--borde)", borderRadius: 12 }} />
          <button className={mic-btn ${escuchando ? "escuchando" : ""}}`
            style={{ width: 46, height: 46, fontSize: "1.2rem" }}
            onClick={buscarPorVoz} title="o pregunte por voz"

```

```

        aria-label="Buscar por voz, o pregúntele al agente"><Icono nombre="microfono"
tam={22} /></button>
</div>
{!consulta && (
  <>
    <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
    <details style={{ marginTop: 4 }}>
      <summary className="pista" style={{ cursor: "pointer" }}>
        Ver frases exactas que entiende el agente aquí
      </summary>
      <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
        {ejemplosLista(esAuditor).map((ej, i) => (
          <li key={i} className="pista" style={{ marginBottom: 4 }}>«{ej}»</li>
        ))}
      </ul>
    </details>
  </>
)}

{consulta && (
  <>
    <p className="rotulo" style={{ marginTop: 14 }}>CuentaVoz responde</p>
    <p className="burbuja" aria-live="polite" aria-atomic="true">{consulta.resumen}</p>
    {consulta.sugerencia_bodega && (
      <div className="banner" style={{ marginTop: 10 }}>
        <span className="ico">🔍</span>
        <span>
          <b>{consulta.sugerencia_bodega}</b> es el nombre de una bodega, no de un artículo.
          <span> Este buscador es para ingredientes – vea su detalle para
            revisarla antes de abrirla.</span>
        </span>
        <button className="btn" style={{ flex: "none" }}
          onClick={() => verDetalle(consulta.sugerencia_bodega_id)}>
          Ver esta bodega
        </button>
      </div>
    )}
    {consulta.ambiguo && consulta.alternativas?.length > 0 && (
      <div className="chips" style={{ marginTop: 10 }}>
        <span className="pista" style={{ width: "100%" }}>¿Era este otro?</span>
        {consulta.alternativas.map((a) => (
          <button key={a.codigo} className="chip oro"
            onClick={() => buscarArticulo(a.codigo)}>
            {a.nombre}
          </button>
        ))}
      </div>
    )}
  </>
)}

{consulta.bodegas?.length > 0 && (
  <>
    <div className="kpis" style={{ marginTop: 14 }}>
      <div className="kpi">
        <div className="kpi-cabeza">
          <span className="icono-kpi">📦</span>
          <small>Total en el sistema</small>
        </div>
        <b>{consulta.total.toLocaleString("es-CO", { maximumFractionDigits: 1 })}
{consulta.unidad}</b>
        <i>en {consulta.bodegas.length} bodegas</i>
      </div>
      <div className="kpi">
        <div className="kpi-cabeza">
          <span className="icono-kpi">🍷</span>
          <small>Consumo del mes</small>
        </div>

```

```

        <b>{consulta.consumo_mes} {consulta.unidad}</b>
        <i>{consulta.servicios_mes} servicios</i>
    </div>
    <div className="kpi verde">
        <div className="kpi-cabeza">
            <span className="icono-kpi">📊</span>
            <small>Cobertura estimada</small>
        </div>
        <b>{consulta.cobertura_dias != null ? `${consulta.cobertura_dias} días` : "-"}</b>
        <i>{consulta.cobertura_dias != null ? "al ritmo actual" : "sin consumo
reciente"}</i>
    </div>
</div>

<table style={{ marginTop: 6 }}>
    <thead><tr><th>Bodega</th><th>Cantidad</th><th>Última
toma</th><th>Estado</th></tr></thead>
    <tbody>
        {consulta.bodegas.map((b, i) => (
            <tr key={i}>
                <td>{b.bodega}</td><td>{b.cantidad} {consulta.unidad}</td>
                <td>{b.ultima_toma}</td><td>{ETIQUETA[b.estado]?.[0] || b.estado}</td>
            </tr>
        ))}
    </tbody>
</table>

<div className="grilla-botones">
    {esAuditor && (
        <button className="btn" onClick={generarReporteArticulo}>Generar reporte</button>
    )}
    <button className="btn borde" onClick={verMovimientos}>Ver movimientos</button>
    <button className="btn borde" onClick={verEnRecetas}>Comparar con la receta</button>
    <button className="btn gris" onClick={volverAlTablero}>◀ Volver al tablero</button>
</div>
</>
    )}
</div>
    )}
</div>
    )}

{msg && <p className="msg-ok">{msg}</p>}

{!consulta && detalle && (
    <div className="card">
        <div style={{ display: "flex", gap: 10, alignItems: "center", flexWrap: "wrap" }}>
            <span style={{ color: "var(--grafito)" }}>🔍</span>
            <input value={buscaArticuloBodega}
                onChange={(e) => buscarEnBodega(e.target.value)}
                onKeyDown={(e) => e.key === "Enter" &&
confirmarBusquedaEnBodega(buscaArticuloBodega)}
                placeholder="artículo en esta bodega, o pregúntele algo al agente..."
                aria-label="Buscar artículo en esta bodega, o pregúntele algo al agente"
                style={{ flex: 1, minWidth: 200, padding: "13px 14px", fontSize: "1rem",
                    border: "1px solid var(--borde)", borderRadius: 12 }} />
            <button className={`mic-btn ${escuchandoBodega ? "escuchando" : ""}`}
                style={{ width: 46, height: 46, fontSize: "1.2rem" }}
                onClick={buscarEnBodegaPorVoz} title="o pregunte por voz"
                aria-label="Buscar por voz en esta bodega, o pregúntele al agente"><Icono
nombre="microfono" tam={22} /></button>
        </div>
        <p className="pista" style={{ marginTop: 8, marginBottom: 4 }}>
            o pregunte por voz - si no es un artículo de esta bodega, se lo pregunta al agente
        </p>
    </div>
    )}

```

```

<details style={{ marginBottom: 14 }}>
  <summary className="pista" style={{ cursor: "pointer" }}>
    Ver frases exactas que entiende el agente aquí
  </summary>
  <ul style={{ margin: "8px 0 0", paddingLeft: 18 }}>
    {_ejemplosDetalle(esAuditor, detalle.estado).map((ej, i) => (
      <li key={i} className="pista" style={{ marginBottom: 4 }}><{ej}></li>
    ))}
  </ul>
</details>
{respuestaAgenteDetalle && (
  <>
    <p className="rotulo">CuentaVoz responde</p>
    <p className="burbuja" style={{ marginBottom: 14 }}
      aria-live="polite" aria-atomic="true">{respuestaAgenteDetalle.respuesta_hablada}</p>
    </>
  </>
)}

<div className="chips">
  <span className="chip">{detalle.referencias} referencias</span>
  {detalle.estado === "cerrada" && detalle.hora_cierre && (
    <span className="chip verde">Cerrada {detalle.hora_cierre}</span>
  )}
  {detalle.personas && <span className="chip azul">{detalle.personas}</span>}
  {detalle.alertas_resueltas > 0 && (
    <span className="chip oro">
      {detalle.alertas_resueltas} alerta{detalle.alertas_resueltas === 1 ? "" : "s"}
    </span>
  )}
</div>

<div className="kpi">
  <div className="kpi verde">
    <div className="kpi-cabeza">
      <span className="icono-kpi">☑</span>
      <small>Exactitud de esta bodega</small>
    </div>
    <b>{detalle.exactitud}</b>
    <i>{detalle.diferencias.length} diferencias de {detalle.referencias}</i>
  </div>
  <div className="kpi">
    <div className="kpi-cabeza">
      <span className="icono-kpi">⌚</span>
      <small>Duración del conteo</small>
    </div>
    <b>{detalle.duracion_min != null ? `${detalle.duracion_min} min` : "-"}</b>
    <i>{detalle.tiempo_papel_min}
      ? `frente a ~${detalle.tiempo_papel_min} min en papel (estimado)`
      : "conteo aún en curso"</i>
  </div>
  <div className="kpi">
    <div className="kpi-cabeza">
      <span className="icono-kpi">📦</span>
      <small>Unidades en stock</small>
    </div>
    <b>{detalle.unidades_stock.toLocaleString("es-CO")}</b>
    <i>sumadas todas las referencias</i>
  </div>
  <div className="kpi oro">
    <div className="kpi-cabeza">
      <span className="icono-kpi">🕒</span>
      <small>Última toma anterior</small>
    </div>
    <b>{detalle.ultima_toma_anterior}

```

```

        ? new Date(detalle.ultima_toma_anterior.fecha).toLocaleDateString("es-CO", { day:
"numeric", month: "short" })
        : "-"</b>
        <i>{detalle.ultima_toma_anterior ? `exactitud
${detalle.ultima_toma_anterior.exactitud}%` : "sin cierres previos"}</i>
    </div>
</div>

<div className="dos-columnas" style={{ gap: 16, marginTop: 4 }}>
    <div>
        <h3>Referencias con diferencia</h3>
        {detalle.diferencias.length > 0 ? (
            <table>

<thead><tr><th>Artículo</th><th>Contado</th><th>Sistema</th><th>Dif.</th></tr></thead>
                <tbody>
                    {detalle.diferencias.map((d, i) => (
                        <tr key={i}><td>{d.articulo}</td><td>{d.contado}</td>
                        <td>{d.sistema}</td><td className="dif">{d.diferencia}</td></tr>
                    ))}
                </tbody>
            </table>
        ) : <p className="vacio">Todas las referencias cuadraron exactas.</p>
        {detalle.diferencias.length > 0 && detalle.diferencias.length < detalle.referencias && (
            <p className="pista" style={{ marginTop: 8 }}>
                Las otras {detalle.referencias - detalle.diferencias.length} referencias cuadraron
exactas.
            </p>
        )}
        {detalle.diferencias.length > 0 && detalle.revisor && (
            <p className="pista">
                Las {detalle.diferencias.length} diferencias fueron revisadas por {detalle.revisor}
al cierre.
            </p>
        )}
    </div>

    <div>
        <h3>Línea de tiempo de la toma</h3>
        {detalle.hitos.length > 0 ? detalle.hitos.map((h, i) => (
            <div className="registro" key={i}>
                <span style={{
                    width: 10, height: 10, borderRadius: "50%", flex: "none",
                    background: h.tipo === "verde" ? "var(--verde)"
                    : h.tipo === "oro" ? "var(--amarillo)" : "var(--azul)",
                }} />
                <b style={{ color: "var(--grafito)" }}>{h.hora}</b>
                <span>{h.texto}</span>
            </div>
        )) : <p className="vacio">Sin movimientos registrados todavía.</p>
    </div>
</div>

{articulosBodega && !respuestaAgenteDetalle && (
    <div style={{ marginTop: 16 }}>
        <h3 style={{ margin: 0 }}>
            {buscaArticuloBodega
                ? `Artículos que coinciden con "${buscaArticuloBodega}"`
                : `Todos los artículos (${articulosBodega.length})`}
        </h3>
        <div style={{ maxHeight: 420, overflowY: "auto", marginTop: 10 }}>
            <table>
                <thead><tr><th>Código</th><th>Artículo</th><th>Unidad</th><th>SD</th></tr></thead>
                <tbody>
                    {_filtrarArticulosBodega(buscaArticuloBodega, articulosBodega).map((a) => (
                        <tr key={a.codigo}>

```



```

        <td>{a.codigo}</td><td>{a.articulo}</td>
        <td>{a.unidad}</td><td>{a.sd.toLocaleString("es-CO")}</td>
    </tr>
    )}
</tbody>
</table>
</div>
</div>
)}

<div className="grilla-botones">
    {(detalle.estado === "pendiente" || detalle.estado === "en_conteo") && (
        // Este detalle es solo de lectura (estado/historial) - para
        // de verdad contar hay que ir a Conteo. Sin este atajo,
        // alguien que llega aquí buscando "abrir la bodega" se
        // queda sin poder hacerlo y tiene que volver a decir/buscar
        // el nombre desde cero en otra pantalla.
        <button className="btn" onClick={() => ir && ir("conteo", { bodegaSugerida:
detalle.bodega })}>
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="conteo" tam={18} />
            {detalle.estado === "en_conteo" ? "Seguir contando" : "Contar esta bodega"}
        </button>
    )}
    {esAuditor && detalle.estado === "cerrada" && (
        <button className="btn borde" onClick={() => setPedirMotivo(true)}>
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="candado" tam={18} />
            Reabrir la bodega
        </button>
    )}
    <button className="btn borde" onClick={verTodosLosArticulos}>
        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="reportes" tam={18} />
        {articulosBodega ? "Ocultar artículos" : `Ver todos los artículos
(${detalle.referencias})`}
    </button>
    {esAuditor && (
        <button className="btn borde" onClick={exportarDetalle}>
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="descargar" tam={18} />
            Descargar detalle
        </button>
    )}
    {esAuditor && (
        <button className="btn" onClick={() => ir && ir("panel")}>
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="panel" tam={18} />
            Ver en el panel gerencial
        </button>
    )}
    <button className="btn gris" onClick={() => { setDetalle(null); setDetalleId(null); }}>
        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="salir" tam={18} />
        Cerrar detalle
    </button>
</div>
{esAuditor && detalle.estado === "cerrada" && (
    <p className="pista" style={{ marginTop: 10 }}>
        Reabrir una bodega cerrada exige justificación escrita y queda en el registro de
trazabilidad.
    </p>
)}
</div>
)}

{!consulta && !detalle && !mostrarCargandoDetalle && lista === null && (

```

```

    <p className="cargando">Cargando...</p>
  })

  {!consulta && !detalle && !mostrarCargandoDetalle && lista !== null && (
    <>
      <div className="chips">
        {[["todas", `${lista.length} bodegas`, ""],
          ["cerrada", `${cuenta("cerrada")} cerradas`, "verde"],
          ["en_conteo", `${cuenta("en_conteo")} en conteo`, ""],
          ["en_auditoria", `${cuenta("en_auditoria")} en auditoría`, "oro"],
          ["pendiente", `${cuenta("pendiente")} pendientes`, "gris"]].map(([k, t, c]) => (
            <button key={k} className={`chip ${filtro === k ? "azul" : c}`}
              onClick={() => setFiltro(k)}>{t}</button>
          ))}
      </div>

      /* Arriba, junto a los filtros, y no al final de las 54 tarjetas -
       son acciones de la pantalla completa, no de una bodega en
       particular, así que no tiene sentido obligar a bajar todo el
       tablero para encontrarlas. */
      <div className="grilla-botones" style={{ marginBottom: 16 }}>
        {esAuditor && (
          <button className="btn borde" onClick={exportarEstado}
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="descargar" tam={18} />
            Exportar estado
          </button>
        )}
        {esAuditor && (
          <button className="btn"
            onClick={() => ir && ir("ajustes", { tabInicial: "usuarios" })}
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="perfil" tam={18} />
            Asignar personas a bodegas
          </button>
        )}
        /* Sin esAuditor a proposito: quien suele topar con una bodega
         que no esta en el catalogo es el auxiliar en el terreno, no
         el administrador desde su escritorio. */
        <button className="btn borde" onClick={() => setPidiendoBodegaNueva(true)}
          style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
          <Icono nombre="bodegas" tam={18} />
          Bodega nueva
        </button>
      </div>

      {lista.length === 0 ? (
        <p className="vacio">No hay bodegas para mostrar.</p>
      ) : (
        <div className="grid-bodegas">
          {vistas.map((b) => {
            const [txt, cls] = ETIQUETA[b.estado] || ["?", "gris"];
            let detalleLinea;
            if (b.estado === "en_conteo" && b.persona) {
              detalleLinea = `${b.persona} · ${b.avance_pct ?? 0}%`;
            } else if (b.estado === "en_auditoria" && b.persona) {
              detalleLinea = `${b.persona} · ${b.diferencias ?? 0} dif.`;
            } else if (b.estado === "cerrada" && b.hora_cierre) {
              detalleLinea = `${b.referencias} refs · ${b.hora_cierre}`;
            } else if (b.estado === "pendiente") {
              detalleLinea = "sin iniciar";
            } else {
              detalleLinea = `${b.referencias} referencias`;
            }
            return (
              <button key={b.id} className={`tarjeta-bodega ${b.estado}`}

```

```

        onClick={() => verDetalle(b.id)}>
        <b><span className="icono-tarjeta">{ICONO_ESTADO[b.estado]} ||
" 🍷 "</span>{b.bodega}</b>
        <span className={`chip ${cls} est`} >{txt}</span>
        <small>{detalleLinea}</small>
      </button>
    );
  }}}
</div>
)}
<p className="pista" style={{ marginTop: 12 }}>
  Cada tarjeta se actualiza al instante por WebSocket cuando alguien abre o cierra una
bodega.
</p>
</>
)}

{pidiendoBodegaNueva && (
  <Dialogo titulo={esAuditor ? "Bodega nueva" : "Solicitar bodega nueva"}
    mensaje={esAuditor
      ? "Como administrador, la bodega entra directo al catálogo: no necesita otra
aprobación."
      : "Queda pendiente de aprobación: no entra al catálogo hasta que un administrador
la confirme desde Auditoría → Aprobaciones."}
    conCampo conVoz placeholder="nombre de la bodega"
    etiquetaCampo="Nombre de la bodega"
    textoAceptar={esAuditor ? "Crear" : "Solicitar"}
    onAceptar={solicitarBodegaNueva}
    onCancel={() => setPidiendoBodegaNueva(false)} />
)}

{pedirMotivo && (
  <Dialogo titulo="Reabrir bodega cerrada"
    mensaje="Esta acción exige una justificación escrita y queda registrada en el
historial. Motivo:"
    conCampo conVoz multilinea placeholder="revisión solicitada por el chef"
    etiquetaCampo="Motivo de la reapertura"
    textoAceptar="Reabrir" peligro
    onAceptar={reabrir}
    onCancel={() => setPedirMotivo(false)} />
)}

{movimientos && (
  <Dialogo titulo="Movimientos recientes"
    mensaje={movimientos.length
      ? movimientos.map((m) => `${m.hora} · ${m.bodega} · ${m.persona}: ${m.cantidad}
${m.unidad}`) .join("\n")
      : "Sin conteos registrados todavía para este artículo."}
    textoAceptar="Cerrar" onAceptar={() => setMovimientos(null)}
    onCancel={() => setMovimientos(null)} />
)}

{enRecetas && (
  <Dialogo titulo="En qué recetas aparece"
    mensaje={enRecetas.length
      ? enRecetas.map((r) => `${r.receta}: ${r.por_porcion} por porción (rinde
${r.rendimiento})` ) .join("\n")
      : "Este artículo no aparece en ninguna receta cargada."}
    textoAceptar="Cerrar" onAceptar={() => setEnRecetas(null)}
    onCancel={() => setEnRecetas(null)} />
)}
</Marco>
);
}

```

12.30 Frontend: Auditoría — comparativo, doble firma, aprobaciones y alertas (vistas 5.16 a 5.19)

Archivo: `frontend/src/vistas/Auditoria.jsx`

```
import { useEffect, useRef, useState } from "react";
import { pedir, enviarTurno } from "../api";
import { escuchar, hablar, vozDisponible } from "../voz";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import AsistenteVoz from "../AsistenteVoz";
import Icono from "../Iconos";

const TABS = [
  { id: "recuento", t: "Recuento ciego y cierre" },
  { id: "aprobaciones", t: "Aprobaciones" },
  { id: "pedidos", t: "Pedidos pendientes" },
  { id: "alertas", t: "Bandeja de alertas" },
];

// Ejemplos de lo que ya entiende el agente en esta pantalla: navegar entre
// las cuatro pestañas, resolver aprobaciones/pedidos/alertas por nombre,
// seleccionar una bodega para auditar, y preguntar por los pendientes.
const _EJEMPLOS_AUDITORIA = [
  "aprueba costilla de res", "rechaza el kiosco nuevo", "aprueba el pedido de Luis",
  "rechaza el pedido de sopa de arroz", "resuelve la alerta de papa criolla",
  "audita el almacén general", "recuento ciego y cierre", "aprobaciones",
  "pedidos pendientes", "bandeja de alertas", "¿cuántas alertas están abiertas?",
  "¿cuántas se resolvieron hoy?", "¿cuál es el tiempo medio?",
  "¿cuántos pedidos están pendientes?", "¿cuántas aprobaciones hay?",
];

export default function Auditoria({ token, usuario, ir, tabInicial, bodegaAuditar, navSeq }) {
  const [tab, setTab] = useState(tabInicial || "recuento");
  // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
  // componente - sin esto, el useState de arriba no vuelve a leer
  // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
  // cada ir()) va en la dependencia también: sin él, pedir la MISMA
  // pestaña dos veces seguidas (con un clic manual a otra en el medio) no
  // hacía nada, porque tabInicial no cambiaba de valor - confirmado con
  // Playwright: "pedidos pendientes" -> clic a Aprobaciones -> "pedidos
  // pendientes" otra vez se quedaba en Aprobaciones.
  useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
  const [enfoco, setEnFoco] = useState(null); // { titulo, chip } para la vista activa
  const esAuditor = usuario?.perfil === "auditor";

  return (
    <Marco titulo={enfoco ? enfoco.titulo
      : "Auditoría · recuento ciego, aprobaciones y cierre"}
      chip={enfoco ? enfoco.chip
        : { texto: esAuditor ? "ADMINISTRADORA" : "SOLO LECTURA",
          tipo: esAuditor ? "azul" : "gris" }}>
      {!esAuditor && (
        <div className="banner">
          <span className="ico">!</span>
          <span>
            <b>Su perfil es auxiliar</b>
            <span>Auditar, aprobar y cerrar bodegas requiere perfil de administrador.
              Ingrese como «diana» para probarlo.</span>
          </span>
        </div>
      )}
      {esAuditor && (
        <AsistenteVoz token={token} vista="auditoria" ir={ir} ejemplos={_EJEMPLOS_AUDITORIA}
          placeholder="aprueba costilla de res, rechaza el kiosco..."
        </AsistenteVoz>
      )}
    </Marco>
  );
}
```

```

        alActualizar={() => window.dispatchEvent(new Event("cuentavoz:auditoria-
actualizada"))} />
    ))
    <div className="chips">
        {TABS.map((tt) => (
            <button key={tt.id} className={`chip ${tab === tt.id ? "azul" : ""}`}
                onClick={() => { setTab(tt.id); setEnFoco(null); }}>{tt.t}</button>
        ))}
    </div>

    {tab === "recuento" &&
        <TabRecuento token={token} esAuditor={esAuditor} onEnFoco={setEnFoco}
            bodegaAuditar={bodegaAuditar} navSeq={navSeq} />
        {tab === "aprobaciones" && <TabAprobaciones token={token} esAuditor={esAuditor}
onEnFoco={setEnFoco} />
        {tab === "pedidos" && <TabPedidosPendientes token={token} esAuditor={esAuditor}
onEnFoco={setEnFoco} />
        {tab === "alertas" && <TabAlertas token={token} esAuditor={esAuditor} onEnFoco={setEnFoco} />
    </Marco>
    );
}

/* — Recuento ciego + cierre con doble firma — */
function TabRecuento({ token, esAuditor, onEnFoco, bodegaAuditar, navSeq }) {
    const [bodegas, setBodegas] = useState(null);
    const [bodegaId, setBodegaId] = useState(null);
    const [sesion, setSesion] = useState(null);
    const [dicho, setDicho] = useState("");
    const [respuesta, setRespuesta] = useState("");
    const [comparar, setComparar] = useState(null);
    const [firmas, setFirmas] = useState(null);
    const [detalle, setDetalle] = useState(null);
    const [modoCierre, setModoCierre] = useState(false);
    const [msg, setMsg] = useState("");
    const [estado, setEstado] = useState("listo");
    const [mostrarTeclado, setMostrarTeclado] = useState(false);
    const [mostrarDictado, setMostrarDictado] = useState(false);
    const [opciones, setOpciones] = useState(null);
    const [opcionesPara, setOpcionesPara] = useState(null);

    function cargarBodegas() {
        // ?propias=1: si este administrador tiene bodegas asignadas (supervisor
        // de una zona), audita solo esas; sin asignaciones, ve el parque
        // completo (administrador general) - ver /api/bodegas en el backend.
        // si falla, resuelve a [] en vez de dejar bodegas en null para
        // siempre: sin esto un fallo de red dejaba "Cargando..." sin fin.
        pedir("/api/bodegas?propias=1", {}, token).then(setBodegas)
            .catch((e) => { setBodegas([]); setMsg(e.message); });
    }
    useEffect(cargarBodegas, [token]);

    const candidatas = (bodegas || []).filter((b) => b.estado === "en_auditoria" || b.estado ===
"cerrada");
    // "cerrada" ya significa cerrada de verdad (doble firma), no "lista
    // para cerrar" - se queda en la lista solo para poder revisarla
    // (ver el resumen de su cierre), no como algo pendiente de hacer.
    // Sin esta distinción, el saludo decía "lista para cerrar" o la
    // contaba entre las "listas para auditar" aunque ya estuviera cerrada.
    const pendientes = candidatas.filter((b) => b.estado === "en_auditoria");
    const cerradas = candidatas.filter((b) => b.estado === "cerrada");
    const [saludoLista, setSaludoLista] = useState("");
    const yaSaludoLista = useRef(false);

    // Al llegar a esta pestaña sin pedir una bodega puntual, el agente dice
    // de una vez si hay algo listo para recuento o cierre y qué es. Si ya

```

```

// llegó con bodegaAuditar (pidió una bodega específica por voz), esto
// no se repite: la respuesta de esa orden ya lo dijo.
useEffect(() => {
  if (bodegas === null || yaSaludoLista.current || bodegaAuditar) return;
  yaSaludoLista.current = true;
  let texto;
  if (pendientes.length === 0) {
    texto = cerradas.length > 0
      ? `No hay ninguna bodega pendiente de recuento ciego. ${cerradas.length} `
      + `ya ${cerradas.length === 1 ? "está cerrada" : "están cerradas"}.`
      : "No hay ninguna bodega lista para recuento ciego o cierre en este momento.";
  } else if (pendientes.length === 1) {
    texto = `Tiene una bodega lista para recuento ciego: ${pendientes[0].bodega.toLowerCase()}. `
      + "¿Quiere auditarla?";
  } else {
    const nombres = pendientes.slice(0, 5).map((b) => b.bodega.toLowerCase()).join(", ");
    texto = `Tiene ${pendientes.length} bodegas listas para recuento ciego: ${nombres}. `
      + "¿Cuál quiere auditar?";
  }
  setSaludoLista(texto);
  hablar(texto);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [bodegas, bodegaAuditar]);

// "Audita <bodega>" por voz llega hasta aquí como bodegaAuditar - lo
// mismo que darle clic a "Abrir" en su tarjeta, sin tocar firmar/cerrar.
// Depende solo de bodegaAuditar (no de candidatas/bodegaId) para no
// reabrir la sola si la persona ya le dio "Volver a la lista" a mano.
useEffect(() => {
  if (!bodegaAuditar) return;
  const encontrada = candidatas.find((b) => b.bodega === bodegaAuditar);
  if (encontrada) {
    setBodegaId(encontrada.id); verFirmas(encontrada.id);
    if (encontrada.estado === "cerrada") verDetalle(encontrada.id);
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [bodegaAuditar, navSeq]);

useEffect(() => {
  if (!bodegaId) { onEnFoco(null); return; }
  const bActual = (bodegas || []).find((b) => b.id === bodegaId);
  const nombre = comparar?.bodega || firmas?.bodega || detalle?.bodega || bActual?.bodega;
  if (!nombre) return;
  if (modoCierre) {
    onEnFoco({ titulo: `Cierre de bodega · ${nombre}`,
      chip: { texto: "LISTA PARA CERRAR", tipo: "borde verde" } });
  } else if (!sesion && bActual?.estado === "cerrada") {
    onEnFoco({ titulo: `Auditoría · ${nombre}`,
      chip: { texto: "CERRADA", tipo: "verde" } });
  } else {
    onEnFoco({ titulo: `Auditoría · ${nombre}`,
      chip: { texto: "RECONTEO CIEGO", tipo: "borde oro" } });
  }
}, [bodegaId, comparar, firmas, detalle, bodegas, modoCierre, sesion]);

async function iniciar(id) {
  setMsg(""); setComparar(null);
  try {
    const r = await pedir(`/api/bodegas/${id}/auditoria/iniciar`, { method: "POST" }, token);
    setBodegaId(id);
    setSesion(r);
    setMostrarDictado(true);
    const saludo = `Recuento ciego iniciado en ${r.bodega.toLowerCase()}. Dícteme el primer
producto.`;
    setRespuesta(saludo);
    hablar(saludo);
  }

```

```

        verFirmas(id);
    } catch (e) { setMsg(e.message); }
}

async function verFirmas(id) {
    try { setFirmas(await pedir(`/api/bodegas/${id}/firmas`, {}, token)); }
    catch (_) {}
}

// Solo para una bodega ya CERRADA: sin esto, abrirla aquí mostraba
// igual "Iniciar recuento ciego" (sesion sigue null hasta llamar
// iniciar()) como si le faltara auditoria, cuando ya quedó cerrada con
// doble firma - una pantalla casi vacía y encima engañosa.
async function verDetalle(id) {
    try { setDetalle(await pedir(`/api/bodegas/${id}/detalle`, {}, token)); }
    catch (_) {}
}

async function procesar(texto, mostrar = texto) {
    if (!texto || !sesion) return;
    setDicho(mostrar);
    try {
        const t = await enviarTurno(texto, sesion.sesion_id, token, {
            opciones, opcionesPara,
            bodegaId, bodegaNombre: sesion?.bodega,
        });
        setRespuesta(t.respuesta_hablada || "");
        setOpciones(t.opciones || null);
        setOpcionesPara(t.opciones_para || null);
        hablar(t.respuesta_hablada);
    } catch (e) { setMsg(e.message); }
}

async function verComparar() {
    setMsg("");
    try { setComparar(await pedir(`/api/bodegas/${bodegaId}/auditoria/comparar`, {}, token)); }
    catch (e) { setMsg(e.message); }
}

async function firmarAuditoria() {
    try {
        await pedir(`/api/bodegas/${bodegaId}/auditoria/firmar`, { method: "POST" }, token);
        setMsg("Auditoría firmada.");
        verFirmas(bodegaId);
    } catch (e) { setMsg(e.message); }
}

async function cerrarDefinitivo() {
    try {
        const r = await pedir(`/api/bodegas/${bodegaId}/cerrar`, { method: "POST" }, token);
        setMsg(`${r.bodega} cerrada definitivamente con doble firma.`);
        setBodegaId(null); setSesion(null); setComparar(null); setFirmas(null);
        setDetalle(null); setModoCierre(false);
        cargarBodegas();
    } catch (e) { setMsg(e.message); }
}

function volverAlaLista() {
    setBodegaId(null); setSesion(null); setComparar(null); setFirmas(null);
    setDetalle(null); setModoCierre(false); setMostrarDictado(false);
}

if (!sesAuditor) return null;

```



```

    </div>
    <b>{detalle.referencias}</b>
    <i>{detalle.contadas} contadas</i>
  </div>
  <div className="kpi">
    <div className="kpi-cabeza">
      <span className="icono-kpi">⌚</span>
      <small>Hora de cierre</small>
    </div>
    <b>{detalle.hora_cierre || "-"}</b>
  </div>
  <div className="kpi">
    <div className="kpi-cabeza">
      <span className="icono-kpi">👂</span>
      <small>Auditó</small>
    </div>
    <b>{detalle.revisor || "-"}</b>
  </div>
</div>
<p className="pista" style={{ marginTop: 12 }}>
  Esta bodega ya está cerrada con doble firma. Para volver a contarla hace
  falta reabrirla desde Bodegas con una justificación escrita.
</p>
</>
)}
<div className="grilla-botones" style={{ marginTop: 12 }}>
  <button className="btn gris" onClick={volverAlaLista}>Volver a la lista</button>
</div>
</div>
): !sesion ? (
  <div className="card">
    <h3>{(bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h3>
    <button className="btn" onClick={() => iniciar(bodegaId)}
      style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
      <Icono nombre="auditoria" tam={18} />
      Iniciar recuento ciego
    </button>
    <div className="grilla-botones" style={{ marginTop: 12 }}>
      <button className="btn gris" onClick={volverAlaLista}>Volver a la lista</button>
    </div>
  </div>
): !comparar ? (
  <div className="card">
    <h3>{(bodegas || []).find((b) => b.id === bodegaId)?.bodega}</h3>
    <div className="conteo-cols">
      <div>
        <p className="rotulo">{dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}</p>
        <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
        {opciones && (
          <div className="opciones" style={{ marginTop: 14 }}>
            {opciones.map((o, i) => (
              <button key={o.codigo} className="opcion"
                onClick={() => procesar(o.codigo, o.nombre)}>
                <span className="n">Opción {i + 1}</span>
              <h4>{o.nombre}</h4>
              <p>Código {o.codigo}</p>
              <p>{o.unidad}</p>
            </button>
          )}}
        </div>
      </div>
    </div>
  </div>
  <div className="grilla-botones" style={{ marginTop: 12 }}>
    <button className="btn verde" onClick={() => procesar("confirmando")}>Confirmar</button>
    <button className="btn gris" onClick={() => setMostrarTeclado(true)}>
      Teclado
    </button>
  </div>
</div>

```

```

    </div>
    <div className="mic-caja">
      <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
        onClick={() => escuchar({ alTexto: procesar, alEstado: setEstado })}
        aria-label="Hable para dictar el artículo y la cantidad del recuento ciego">
        <Icono nombre="microfono" tam={52} />
      </button>
      <small>{vozDisponible() ? "Reconocimiento en español" : "Use el teclado"}</small>
    </div>
  </div>
  <div className="grilla-botones" style={{ marginTop: 12 }}>
    <button className="btn borde" onClick={verComparar}>Ver comparación</button>
    <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
  </div>
</div>
) : !modoCierre ? (
  <div className="card">
    <div className="chips">
      <span className="chip azul">{comparar.total} referencias</span>
      <span className="chip oro">{comparar.filas.length} con diferencia</span>
      <span className="chip verde">{comparar.coinciden} coinciden</span>
    </div>
    <p className="pista">
      Solo se muestran las referencias con diferencia. Su recuento fue a ciegas:
      el conteo 1 se revela al comparar.
    </p>
    {comparar.filas.length === 0 ? (
      <p className="vacio">Sin diferencias entre el recuento ciego y el sistema.</p>
    ) : (
      <table>
        <thead>
          <tr>
            <th>Artículo</th>
            <th>Conteo 1</th>
            <th>Su conteo</th>
            <th>Sistema</th>
            <th>Diferencia</th>
            <th>Acción</th>
          </tr>
        </thead>
        <tbody>
          {comparar.filas.map((f) => (
            <tr key={f.codigo}>
              <td>{f.articulo}</td>
              <td>{f.conteo1 ?? "-"}</td>
              <td>{f.conteo2 ?? "-"}</td>
              <td>{f.sistema}</td>
              <td className="dif">{f.diferencia > 0 ? `+${f.diferencia}` : f.diferencia}</td>
              <td>{f.accion}</td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
  </div>
  {comparar.diagnosticos?.length > 0 && (
    <div style={{ background: "#FAFBFD", border: "1px solid var(--borde)",
      borderRadius: "var(--radio)", padding: 14, marginTop: 14 }}>
      <h3 style={{ marginTop: 0 }}>Diagnóstico automático</h3>
      {comparar.diagnosticos.map((d, i) => (
        <p key={i} style={{ fontSize: ".88rem", marginTop: i ? 8 : 0 }}>{d}</p>
      ))}
    </div>
  )}
  {mostrarDictado && (
    <div className="conteo-cols" style={{ marginTop: 16 }}>
      <div>
        <p className="rotulo">{dicho ? `Usted dijo: «${dicho}»` : "CuentaVoz responde"}</p>
        <div className="burbuja" aria-live="polite" aria-atomic="true">{respuesta}</div>
        {opciones && (
          <div className="opciones" style={{ marginTop: 14 }}>
            {opciones.map((o, i) => (

```

```

        <button key={o.codigo} className="opcion"
            onClick={() => procesar(o.codigo, o.nombre)}>
            <span className="n">Opción {i + 1}</span>
            <h4>{o.nombre}</h4>
            <p>Código {o.codigo}</p>
            <p>{o.unidad}</p>
        </button>
    )}
</div>
)}
<div className="grilla-botones" style={{ marginTop: 12 }}>
    <button className="btn verde" onClick={() =>
procesar("confirmando")}>Confirmar</button>
    <button className="btn borde" onClick={verComparar}>Actualizar comparación</button>
    <button className="btn gris" onClick={() =>
setMostrarTeclado(true)}>Teclado</button>
</div>
</div>
<div className="mic-caja">
    <button className={`mic-btn ${estado === "escuchando" ? "escuchando" : ""}`}
        onClick={() => escuchar({ alTexto: procesar, alEstado: setEstado })}
        aria-label="Hable para dictar el artículo y la cantidad del recuento ciego">
        <Icono nombre="microfono" tam={52} />
    </button>
    <small>{vozDisponible() ? "Reconocimiento en español" : "Use el teclado"}</small>
</div>
</div>
)}

<div className="grilla-botones" style={{ marginTop: 14 }}>
    <button className="btn" onClick={() => setMostrarDictado((v) => !v)}>
        {mostrarDictado ? "Ocultar dictado" : "Resolver por voz"}
    </button>
    <button className="btn borde" disabled={firmas?.auditoria?.firmada}
        onClick={firmarAuditoria}>
        Aceptar todas
    </button>
    <button className="btn verde" onClick={() => setModoCierre(true)}>
        Cerrar con doble firma
    </button>
</div>
<div className="grilla-botones" style={{ marginTop: 8 }}>
    <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
</div>
</div>
) : (
    <div className="card">
        <div className="chips">
            <span className="chip">{firmas?.referencias ?? comparar.total} referencias</span>
            <span className="chip verde">{comparar.filas.length} diferencias resueltas</span>
            <span className="chip verde">{firmas?.alertas_resueltas ?? 0} alertas cerradas</span>
        </div>

        <div className="conteo-cols" style={{ marginTop: 14 }}>
            <div className="opcion" style={{
                borderColor: firmas?.conteo?.firmada ? "var(--verde)" : "var(--borde)",
                background: firmas?.conteo?.firmada ? "var(--verde-bg)" : "#fff",
            }}>
                <span className="n">CONTEO 1 · AUXILIAR</span>
                <div style={{ display: "flex", alignItems: "center", gap: 10, marginTop: 6 }}>
                    <span style={{ width: 34, height: 34, borderRadius: "50%", background: "var(--verde)",
                        color: "#fff", display: "flex", alignItems: "center", justifyContent:
"center",
                        fontWeight: 700 }}>
                        {(firmas?.conteo?.persona || "?")[0]?.toUpperCase()}
                    </span>

```

```

"></h4>
    <h4 style={{ textTransform: "capitalize", margin: 0 }}>{firmas?.conteo?.persona || "-
    </div>
    {firmas?.conteo?.firmada ? (
      <p style={{ color: "var(--verde)", fontWeight: 700, marginTop: 10 }}>
        ✓ Firmado · {firmas?.conteo.hora}
      </p>
    ) : <p style={{ marginTop: 10 }}>Pendiente de su firma</p>
    <p style={{ fontSize: ".82rem", color: "var(--grafito)" }}>
      {firmas?.conteo?.huella ? "Huella digital registrada · " : ""}
      {firmas?.referencias ?? comparar.total} referencias
    </p>
    <p style={{ fontSize: ".82rem", fontStyle: "italic", color: "var(--grafito)" }}>
      Sin acceso al recuento de auditoría
    </p>
  </div>
  <div className="opcion" style={{
    borderColor: firmas?.auditoria?.firmada ? "var(--verde)" : "var(--azul)",
  }}>
    <span className="n">AUDITORÍA · ADMINISTRADORA</span>
    <div style={{ display: "flex", alignItems: "center", gap: 10, marginTop: 6 }}>
      <span style={{ width: 34, height: 34, borderRadius: "50%", background: "var(--azul)",
        color: "#fff", display: "flex", alignItems: "center", justifyContent:
"center",
        fontWeight: 700 }}>
        {(firmas?.auditoria?.persona || "?")[0]?.toUpperCase()}
      </span>
      <h4 style={{ textTransform: "capitalize", margin: 0 }}>{firmas?.auditoria?.persona ||
"-"}</h4>
    </div>
    {firmas?.auditoria?.firmada ? (
      <p style={{ color: "var(--verde)", fontWeight: 700, marginTop: 10 }}>
        ✓ Firmado · {firmas?.auditoria.hora}
      </p>
    ) : (
      <p style={{ color: "var(--azul)", marginTop: 10 }}>Pendiente de su firma</p>
      <button className="btn" style={{ marginTop: 8 }} onClick={firmarAuditoria}>
        Firmar con PIN o huella
      </button>
    </>
    </div>
  </div>

  <div className="grilla-botones" style={{ marginTop: 16 }}>
    <button className="btn verde" disabled={!firmas?.lista_para_cerrar}
      onClick={cerrarDefinitivo} style={{ flex: 1, fontSize: "1.05rem" }}>
      Cerrar bodega definitivamente
    </button>
  </div>
  <p className="pista" style={{ marginTop: 10, textAlign: "center" }}>
    Con las dos firmas la bodega pasa a CERRADA en el tablero al instante y sus datos entran
    al consolidado.
  </p>
  <p className="pista" style={{ textAlign: "center", color: "var(--azul)" }}>
    Ninguna bodega se cierra con una sola firma: es la garantía de trazabilidad del
    inventario.
  </p>
  <div className="grilla-botones" style={{ marginTop: 8 }}>
    <button className="btn borde" onClick={() => setModoCierre(false)}>
      Volver a la comparación
    </button>
    <button className="btn gris" onClick={volverALaLista}>Volver a la lista</button>
  </div>
</div>
}}

```

```

    {mostrarTeclado && (
      <Dialogo titulo="Escribir en vez de hablar"
        mensaje="Escriba el producto y la cantidad:"
        conCampo placeholder="tres tablas para picar blancas"
        onAceptar={(t) => { setMostrarTeclado(false); if (t) procesar(t); }}
        onCancel={() => setMostrarTeclado(false)} />
    )}
  </>
);
}

/* — Aprobaciones pendientes — */
function TabAprobaciones({ token, esAuditor, onEnFoco }) {
  const [lista, setLista] = useState(null);
  const [msg, setMsg] = useState("");
  const [saludo, setSaludo] = useState("");
  const yaSaludo = useRef(false);

  function cargar() {
    pedir("/api/aprobaciones?estado=pendiente", {}, token).then(setLista)
      .catch((e) => { setLista([]); setMsg(e.message); });
  }
  useEffect(cargar, [token]);
  // El agente ("Pregúntele al agente", arriba de las pestañas) puede
  // aprobar/rechazar por voz - cuando lo hace, avisa con este evento
  // para que esta lista refleje el cambio sin recargar.
  useEffect(() => {
    window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
    return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
  }, [token]);

  useEffect(() => {
    onEnFoco({ titulo: "Aprobaciones pendientes",
      chip: { texto: lista === null ? "..." : `${lista.length} PENDIENTES`, tipo: "borde oro" }
    });
  }, [lista]);

  // Al llegar a esta pestaña (por clic o por voz), el agente dice de una
  // vez si hay algo pendiente y qué es, en vez de esperar a que se lo
  // pregunten - yaSaludo evita que se repita en cada refresco de la
  // lista (ej. justo después de aprobar algo por voz).
  useEffect(() => {
    if (lista === null || yaSaludo.current) return;
    yaSaludo.current = true;
    let texto;
    if (lista.length === 0) {
      texto = "No hay aprobaciones pendientes.";
    } else if (lista.length === 1) {
      texto = `Tiene una aprobación pendiente: ${lista[0].nombre.toLowerCase()}. ¿Quiere aprobarla o rechazarla?`;
    } else {
      const nombres = lista.slice(0, 5).map((a) => a.nombre.toLowerCase()).join(", ");
      texto = `Tiene ${lista.length} aprobaciones pendientes: ${nombres}. ¿Cuál quiere aprobar o rechazar?`;
    }
    setSaludo(texto);
    hablar(texto);
  }, [lista]);

  async function resolver(id, accion) {
    try {
      await pedir(`/api/aprobaciones/${id}/${accion}`, { method: "POST" }, token);
      setMsg(accion === "aprobar" ? "Aprobado y agregado al catálogo oficial." : "Rechazado.");
      cargar();
    }
  }
}

```

```

    } catch (e) { setMsg(e.message); }
  }

  if (!esAuditor) return null;

  return (
    <div className="card">
      <p className="pista" style={{ marginTop: 0 }}>
        Productos y bodegas creados durante la toma que esperan su firma para entrar al catálogo
        oficial.
      </p>
      {saludo && (
        <>
          <p className="rotulo">CuentaVoz responde</p>
          <p className="burbuja" style={{ marginBottom: 12 }}
            aria-live="polite" aria-atomic="true">{saludo}</p>
        </>
      )}
      {msg && <p className="msg-ok">{msg}</p>}
      {lista === null ? (
        <p className="cargando">Cargando...</p>
      ) : lista.length === 0 ? (
        <p className="vacio">Nada pendiente de aprobación.</p>
      ) : (
        lista.map((a) => (
          <div key={a.id} style={{ display: "flex", alignItems: "flex-start", gap: 14,
            flexWrap: "wrap",
            border: "1px solid var(--borde)", borderRadius: 10,
            padding: "12px 14px", marginBottom: 10 }}>
            <span style={{ flex: 1, minWidth: 200 }}>
              <b style={{ color: "var(--azul)" }}>{a.nombre}</b>
              <br />
              <span style={{ fontSize: ".85rem" }}>
                {a.tipo === "producto"
                  ? `Producto nuevo · ${a.unidad_medida} || "`
                  : "Bodega nueva"}
              </span>
              <br />
              <small style={{ color: "var(--grafito)", fontStyle: "italic" }}>
                {a.tipo === "producto"
                  ? `${a.cantidad_inicial} ${a.unidad_medida} contados en ${a.bodega}`
                  : "sin referencias asignadas todavía"}
              </small>
              <br />
              <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
                {a.creado_por} · {a.hora}
              </small>
            </span>
            <span style={{ display: "flex", gap: 8 }}>
              <button className="btn gris" onClick={() => resolver(a.id, "rechazar")}
                style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
                <Icono nombre="rechazar" tam={16} />
                Rechazar
              </button>
              <button className="btn verde" onClick={() => resolver(a.id, "aprobar")}
                style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
                <Icono nombre="aprobar" tam={16} />
                Aprobar
              </button>
            </span>
          </div>
        ))
      )}
      <p className="pista" style={{ marginTop: 10 }}>
        Al aprobar, el producto entra al catálogo con su código definitivo y el
        conteo asociado deja de estar marcado. Mientras tanto el conteo nunca se
        detiene: la aprobación ocurre en paralelo al trabajo en bodega.
      </p>
    </div>
  )

```

```

    </p>
  </div>
);
}

/* — Pedidos de auxiliares esperando aprobación antes de salir del almacén — */
function TabPedidosPendientes({ token, esAuditor, onEnFoco }) {
  const [lista, setLista] = useState(null);
  const [msg, setMsg] = useState("");
  const [saludo, setSaludo] = useState("");
  const yaSaludo = useRef(false);

  function cargar() {
    pedir("/api/pedidos/pendientes", {}, token).then(setLista)
      .catch((e) => { setLista([]); setMsg(e.message); });
  }
  useEffect(cargar, [token]);
  useEffect(() => {
    window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
    return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
  }, [token]);

  useEffect(() => {
    onEnFoco({ titulo: "Pedidos pendientes de aprobación",
      chip: { texto: lista === null ? "..." : `${lista.length} PENDIENTES`, tipo: "borde oro" }
    });
  }, [lista]);

  useEffect(() => {
    if (lista === null || yaSaludo.current) return;
    yaSaludo.current = true;
    let texto;
    if (lista.length === 0) {
      texto = "No hay pedidos pendientes.";
    } else if (lista.length === 1) {
      texto = `Tiene un pedido pendiente: ${lista[0].plato.toLowerCase()}, de `
        + `${lista[0].persona}. ¿Quiere aprobarlo o rechazarlo?`;
    } else {
      const nombres = lista.slice(0, 5)
        .map((p) => `${p.plato.toLowerCase()} de ${p.persona}`).join(", ");
      texto = `Tiene ${lista.length} pedidos pendientes: ${nombres}. ¿Cuál quiere aprobar o
rechazar?`;
    }
    setSaludo(texto);
    hablar(texto);
  }, [lista]);

  async function resolver(numero, aprobar) {
    try {
      await pedir(`/api/pedidos/${numero}/resolver`, {
        method: "POST", body: JSON.stringify({ aprobar }),
      }, token);
      setMsg(aprobar ? "Pedido aprobado: ya puede salir del almacén."
        : "Pedido rechazado.");
      cargar();
    } catch (e) { setMsg(e.message); }
  }

  if (!esAuditor) return null;

  return (
    <div className="card">
      <p className="pista" style={{ marginTop: 0 }}>
        El auxiliar pide y queda registrado de una vez, pero solo sale del almacén
      </p>
    </div>
  );
}

```

```

    cuando usted lo aprueba aquí - igual que con productos y bodegas nuevas.
  </p>
  {saludo && (
    <>
      <p className="rotulo">CuentaVoz responde</p>
      <p className="burbuja" style={{ marginBottom: 12 }}
        aria-live="polite" aria-atomic="true">{saludo}</p>
    </>
  )}
  {msg && <p className="msg-ok">{msg}</p>}
  {lista === null ? (
    <p className="cargando">Cargando...</p>
  ) : lista.length === 0 ? (
    <p className="vacio">Nada pendiente de aprobación.</p>
  ) : (
    lista.map((p) => (
      <div key={p.numero_pedido} style={{ display: "flex", alignItems: "flex-start", gap: 14,
        flexWrap: "wrap",
        border: "1px solid var(--borde)", borderRadius: 10,
        padding: "12px 14px", marginBottom: 10 }}>
        <span style={{ flex: 1, minWidth: 200 }}>
          <b style={{ color: "var(--azul)" }}>{p.plato}</b> · {p.porciones} porciones
          <span className="chip" style={{ marginLeft: 8 }}>{p.numero_pedido}</span>
          <br />
          <small style={{ color: "var(--grafito)" }}>
            Se descontaría de: {p.bodega || "-"}
          </small>
          <br />
          <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
            Pedido por {p.persona} · {p.hora}
          </small>
          <ul style={{ margin: "8px 0 0", paddingLeft: 18, fontSize: ".85rem" }}>
            {p.items.map((it, i) => (
              <li key={i}>{it.nombre}: {it.cantidad}</li>
            ))}
          </ul>
        </span>
        <span style={{ display: "flex", gap: 8, flex: "none" }}>
          <button className="btn gris" onClick={() => resolver(p.numero_pedido, false)}
            style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
            <Icono nombre="rechazar" tam={16} />
            Rechazar
          </button>
          <button className="btn verde" onClick={() => resolver(p.numero_pedido, true)}
            style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
            <Icono nombre="aprobar" tam={16} />
            Aprobar
          </button>
        </span>
      </div>
    ))
  )}
  <p className="pista" style={{ marginTop: 10 }}>
    Al aprobar, el pedido queda abierto para su legalización al final del servicio.
    Al rechazar, no cuenta como enviado y el auxiliar debe volver a intentarlo.
  </p>
</div>
);
}

/* — Bandeja de alertas — */
const COLOR_ALERTA = { desviacion: "oro", negativo: "oro", unidad: "azul", inexistente: "azul" };

function TabAlertas({ token, esAuditor, onEnFoco }) {
  const [alertas, setAlertas] = useState(null);
  const [resumen, setResumen] = useState(null);
  const [verDetalle, setVerDetalle] = useState(null);

```



```

const [msg, setMsg] = useState("");
const [saludo, setSaludo] = useState("");
const yaSaludo = useRef(false);

function cargar() {
  pedir("/api/alertas?resueltas=0", {}, token).then(setAlertas)
    .catch((e) => { setAlertas([]); setMsg(e.message); });
  pedir("/api/alertas/resumen", {}, token).then(setResumen).catch(() => {});
}
useEffect(cargar, [token]);
useEffect(() => {
  window.addEventListener("cuentavoz:auditoria-actualizada", cargar);
  return () => window.removeEventListener("cuentavoz:auditoria-actualizada", cargar);
}, [token]);

useEffect(() => {
  onEnFoco({ titulo: "Auditoría · Bandeja de alertas",
    chip: { texto: alertas === null ? "..." : `${alertas.length} ABIERTAS`, tipo: "borde oro"
  }
});
}, [alertas]);

useEffect(() => {
  if (alertas === null || yaSaludo.current) return;
  yaSaludo.current = true;
  const etiqueta = (a) => a.articulo || a.bodega || a.titulo.toLowerCase();
  let texto;
  if (alertas.length === 0) {
    texto = "No hay alertas abiertas.";
  } else if (alertas.length === 1) {
    texto = `Tiene una alerta abierta: ${etiqueta(alertas[0]).toLowerCase()}.`
      + (esAuditor ? " ¿Quiere resolverla?" : "");
  } else {
    const nombres = alertas.slice(0, 5).map((a) => etiqueta(a).toLowerCase()).join(", ");
    texto = `Tiene ${alertas.length} alertas abiertas: ${nombres}.`
      + (esAuditor ? " ¿Cuál quiere resolver?" : "");
  }
  setSaludo(texto);
  hablar(texto);
}, [alertas, esAuditor]);

async function resolver(id) {
  try {
    await pedir(`/api/alertas/${id}/resolver`, { method: "POST" }, token);
    setMsg("Alerta resuelta.");
    cargar();
  } catch (e) { setMsg(e.message); }
}

return (
  <div className="card">
    <div className="chips">
      <span className="chip oro">{resumen?.abiertas ?? (alertas || []).length} abiertas</span>
      <span className="chip verde">{resumen?.resueltas_hoy ?? 0} resueltas hoy</span>
      <span className="chip">
        Tiempo medio: {resumen?.tiempo_medio_min != null ? `${resumen.tiempo_medio_min} min` : "-"}
      </span>
    </div>
    {saludo && (
      <>
        <p className="rotulo">CuentaVoz responde</p>
        <p className="burbuja" style={{ marginBottom: 12 }}
          aria-live="polite" aria-atomic="true">{saludo}</p>
        </>
      </>
    )}
    {msg && <p className="msg-ok">{msg}</p>}
  </div>
)

```

```

{alertas === null ? (
  <p className="cargando">Cargando...</p>
) : alertas.length === 0 ? (
  <p className="vacio">No hay alertas abiertas.</p>
) : (
  alertas.map((a) => (
    <div key={a.id} style={{
      display: "flex", alignItems: "center", gap: 14, marginTop: 10, flexWrap: "wrap",
      border: "1px solid var(--borde)",
      borderLeft: `4px solid ${COLOR_ALERTA[a.tipo] === "azul" ? "var(--azul)" : "var(--
amarillo)"}`,
      borderRadius: 10, padding: "12px 14px",
    }}>
      <span style={{ flex: 1, minWidth: 200 }}>
        <b style={{ color: COLOR_ALERTA[a.tipo] === "azul" ? "var(--azul)" : "var(--amarillo-
tx)" }}>
          {a.titulo}
        </b>
        <br />
        <span style={{ fontSize: ".88rem" }}>
          {a.articulo || "-"}{a.bodega ? ` · ${a.bodega}` : ""}
        </span>
        <br />
        <small style={{ color: "var(--grafito)", textTransform: "capitalize" }}>
          {a.persona || "-"} · {a.hora}
        </small>
        <br />
        <small style={{ color: "var(--grafito)", fontStyle: "italic" }}>{a.detalle}</small>
      </span>
      <span style={{ display: "flex", gap: 8, flex: "none" }}>
        <button className="btn borde" onClick={() => setVerDetalle(a)}>Ver</button>
        {esAuditor && (
          <button className="btn verde" onClick={() => resolver(a.id)}
            style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
            <Icono nombre="aprobar" tam={16} />
            Resolver
          </button>
        )}
      </span>
    </div>
  ))
)}
<p className="pista" style={{ marginTop: 12, fontStyle: "italic" }}>
  Cada alerta guarda quién la generó, con qué dato y cómo se resolvió: es la trazabilidad
  que exige una auditoría de inventario.
</p>

{verDetalle && (
  <Dialogo titulo={verDetalle.titulo}
    mensaje={` ${verDetalle.articulo || ""} ${verDetalle.bodega ? ` · ${verDetalle.bodega}`
: ""}\n` +
      ` ${verDetalle.persona || "-"} ·
    ${verDetalle.hora}\n\n ${verDetalle.detalle}` }
    textoAceptar="Cerrar" onAceptar={() => setVerDetalle(null)}
    onCancel={() => setVerDetalle(null)} />
  )}
</div>
);
}

```

12.31 Frontend: Reportes y Panel (vistas 5.20 a 5.23)

Archivo: frontend/src/vistas/Reportes.jsx

```
import { useEffect, useState } from "react";
```

```

import { pedir, descargarReporte } from "../api";
import { hablar, esAfirmacion, esNegacion } from "../voz";
import Marco from "../Marco";
import AsistenteVoz from "../AsistenteVoz";
import Icono from "../Iconos";

const fmt = (n) => Number(n ?? 0).toLocaleString("es-CO", { maximumFractionDigits: 2 });

// "oro" es el modificador de clase (.chip.oro); la variable CSS real es
// --amarillo, no --oro - de ahí los dos mapas separados.
const CLASE_TITULO = {
  "Consolidado para My Inventory": "verde",
  "Diferencias por bodega": "oro",
  "Detalle de bodega": "azul",
  "Estado del tablero": "azul",
  "Análisis de consumo": "azul",
  "Registro de trazabilidad": "azul",
};
const VAR_TITULO = {
  "Consolidado para My Inventory": "verde",
  "Diferencias por bodega": "amarillo",
  "Detalle de bodega": "azul",
  "Estado del tablero": "azul",
  "Análisis de consumo": "azul",
  "Registro de trazabilidad": "azul",
};
const ICONO_TITULO = {
  "Consolidado para My Inventory": "🏠",
  "Diferencias por bodega": "📊",
  "Detalle de bodega": "📦",
  "Estado del tablero": "📅",
  "Análisis de consumo": "📈",
  "Registro de trazabilidad": "🕒",
};

// Ejemplos de lo que ya entiende el agente en esta pantalla: generar los
// tres tipos de archivo, ver el contenido de uno ya generado, y preguntar
// por lo que muestra el análisis de consumo.
const EJEMPLOS_REPORTES = [
  "genera el consolidado", "exporta las diferencias", "exporta el análisis de consumo",
  "muéstrame las diferencias", "muéstrame el estado del tablero", "muéstrame la trazabilidad",
  "muéstrame el detalle de bodega", "muéstrame el archivo del consolidado",
  "consolidado de la toma", "análisis de consumo",
  "¿cuántas filas tiene el último consolidado?", "¿cuántas bodegas tienen descuadre?",
  "¿qué archivos se han generado?", "¿cuánto se pidió en el período?",
  "¿cuánto se usó realmente?", "¿cuál es el ahorro potencial?",
  "¿cuántos insumos están subutilizados?", "¿cuál insumo tiene más sobrepedido?",
];

export default function Reportes({ token, usuario, ir, tabInicial, navSeq, archivoPrevisualizar }) {
  const [tab, setTab] = useState(tabInicial || "consolidado");
  // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
  // componente - sin esto, el useState de arriba no vuelve a leer
  // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
  // cada ir()) va en la dependencia también: sin él, pedir la MISMA
  // pestaña dos veces seguidas (con un clic manual a otra en el medio)
  // no hacía nada, porque tabInicial no cambiaba de valor.
  useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);

  return (
    <Marco titulo={tab === "consolidado" ? "Reportes · consolidado de la toma"
      : "Reportes · análisis de consumo"}
      chip={tab === "consolidado" ? { texto: "ARCHIVOS", tipo: "borde azul" }
      : { texto: "ÚLTIMOS 30 DÍAS", tipo: "borde azul" }}>

```

```

<AsistenteVoz token={token} vista="reportes" ir={ir} ejemplos={_EJEMPLOS_REPORTES}
  placeholder="¿cuántas filas tiene el último consolidado?, lléveme al panel..."
  alActualizar={() => window.dispatchEvent(new Event("cuentavoz:reportes -
actualizados"))} />
<div className="chips">
  <button className={`chip ${tab === "consolidado" ? "azul" : ""}`}
    onClick={() => setTab("consolidado")}>Consolidado de la toma</button>
  <button className={`chip ${tab === "analisis" ? "azul" : ""}`}
    onClick={() => setTab("analisis")}>Análisis de consumo</button>
</div>

{tab === "consolidado"
  ? <TabConsolidado token={token} usuario={usuario}
    navSeq={navSeq} archivoPrevisualizar={archivoPrevisualizar} />
  : <TabAnalisis token={token} usuario={usuario} />}
</Marco>
);
}

/** Tabla genérica: cada tipo de reporte (consolidado, diferencias, estado,
detalle de bodega) trae sus propias columnas, así que en vez de mapear
columnas fijas se dibujan las que de verdad trae el archivo. */
function TablaVistaPrevia({ filas }) {
  if (!filas || filas.length === 0) return <p className="vacío">Este archivo no tiene filas.</p>;
  const columnas = Object.keys(filas[0]);
  return (
    <div className="tabla-scroll">
      <table>
        <thead><tr>{columnas.map((c) => <th key={c}>{c}</th>)}</tr></thead>
        <tbody>
          {filas.map((f, i) => (
            <tr key={i}>
              {columnas.map((c) => (
                <td key={c} className={c === "Diferencia" ? "dif" : ""}>
                  {c === "Diferencia" && f[c] > 0 ? `+${f[c]}` : String(f[c])}
                </td>
              ))}
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

/* — Consolidado de la toma — */
function TabConsolidado({ token, navSeq, archivoPrevisualizar }) {
  const [recientes, setRecientes] = useState(null);
  const [archivoActivo, setArchivoActivo] = useState(null); // {archivo, titulo}
  const [filasPrevia, setFilasPrevia] = useState(null);
  const [totalFilas, setTotalFilas] = useState(null);
  const [err, setErr] = useState("");
  const [msg, setMsg] = useState("");
  // Se activa con cualquier vista previa (clic manual o por voz) y con
  // cada archivo recién generado - así un "sí"/"no" hablado después
  // sirve para descargarlo sin importar cómo se llegó hasta ahí. Ver
  // previsualizar().
  const [pidiendoDescarga, setPidiendoDescarga] = useState(false);

  function cargar() {
    pedir("/api/reportes/recientes", {}, token).then(setRecientes).catch(() => setRecientes([]));
  }
  useEffect(cargar, [token]);
  // Generar un archivo por voz pasa por el backend directo, no por
  // generarConsolidado/generarDiferencias de aquí abajo - sin escuchar
  // este evento la lista de "Archivos generados" se quedaba vieja hasta

```

```

// que se recargaba la página a mano.
useEffect(() => {
  window.addEventListener("cuentavoz:reportes-actualizados", cargar);
  return () => window.removeEventListener("cuentavoz:reportes-actualizados", cargar);
}, [token]);

// anunciar: true en un clic manual a una tarjeta - sin voz de por medio,
// alguien con problemas de vista no tiene otra forma de saber qué
// encontró la vista previa (antes solo se narraba cuando el pedido
// llegaba por voz, porque ese camino ya trae su propia respuesta
// hablada del backend - un clic de mouse no). false cuando el pedido
// YA llegó por voz (ver el useEffect de abajo), para no narrar dos
// veces lo mismo que el agente acaba de decir.
async function previsualizar(archivo, titulo, { anunciar = false } = {}) {
  setErr("");
  try {
    const d = await pedir(`/api/reportes/vista-previa?archivo=${encodeURIComponent(archivo)}`, {},
token);
    setArchivoActivo({ archivo, titulo });
    setFilasPrevias(d.filas);
    setTotalFilas(d.total);
    setPidiendoDescarga(true);
    if (anunciar) {
      const resumen = `Vista previa de ${titulo}, ${d.total} `
        + `${d.total === 1 ? "fila" : "filas"}. ¿Desea descargarlo?`;
      setMsg(resumen);
      hablar(resumen);
    }
  } catch (e) { setErr(e.message); }
}

// "Muéstrame el archivo de diferencias" - lo mismo que darle clic a la
// tarjeta, pero pedido por voz; el backend ya resolvió CUÁL archivo es.
// Va por prop (como bodegaSugerida en Conteo), no por evento: si el
// pedido llegó desde OTRA pestaña de Reportes, este componente todavía
// no estaba montado cuando AsistenteVoz disparaba el evento viejo, así
// que la vista previa nunca aparecía. navSeq en las dependencias por la
// misma razón que en el resto de la app: pedir el MISMO archivo dos
// veces seguidas (con un clic manual a otro en el medio) no cambia el
// valor de archivoPrevisualizar por sí solo.
useEffect(() => {
  if (archivoPrevisualizar?.archivo) {
    previsualizar(archivoPrevisualizar.archivo, archivoPrevisualizar.titulo);
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [archivoPrevisualizar, navSeq]);

// "Sí"/"no" después de que el agente preguntó "¿desea descargarlo?" -
// intercepta ANTES de que AsistenteVoz le pregunte al agente general
// (que no tiene ni idea de esa pregunta pendiente). Mismo patrón que el
// filtro local de Registro de trazabilidad en Ajustes.
useEffect(() => {
  function interceptar(e) {
    if (!pidiendoDescarga) return;
    const texto = e.detail.texto;
    if (esAfirmacion(texto)) {
      e.preventDefault();
      setPidiendoDescarga(false);
      descargarReporte(archivoActivo.archivo, token).catch((err2) => setErr(err2.message));
      const msg2 = "Descargando el archivo.";
      setMsg(msg2);
      hablar(msg2);
    } else if (esNegacion(texto)) {
      e.preventDefault();
      setPidiendoDescarga(false);
      const msg2 = "Listo, queda pendiente. Dígame si desea ver otro archivo.";
      setMsg(msg2);
      hablar(msg2);
    }
  }
  window.addEventListener("cuentavoz:reportes-actualizados", interceptar);
  return () => window.removeEventListener("cuentavoz:reportes-actualizados", interceptar);
}, [archivoActivo, token]);

```

```

    }
  }
  window.addEventListener("cuentavoz:filtro-local", interceptar);
  return () => window.removeEventListener("cuentavoz:filtro-local", interceptar);
}, [pidiendoDescarga, archivoActivo, token]);

async function generarConsolidado(formato) {
  setErr(""); setMsg("");
  try {
    const d = await pedir(`/api/reportes?formato=${formato}`, { method: "POST" }, token);
    setArchivoActivo({ archivo: d.archivo, titulo: "Consolidado para My Inventory" });
    setFilasPrevia(d.vista_previa || null);
    setTotalFilas(d.filas);
    setPidiendoDescarga(true);
    const resumen = `Consolidado generado: ${d.filas} filas. ¿Desea descargarlo?`;
    setMsg(resumen);
    hablar(resumen);
    cargar();
  } catch (e) { setErr(e.message); }
}

async function generarDiferencias(formato) {
  setErr(""); setMsg("");
  try {
    const d = await pedir(`/api/reportes/diferencias?formato=${formato}`, { method: "POST" }, token);
    const resumen = `Diferencias exportadas: ${d.filas} filas en ${d.bodegas_con_descuadre} `
      + "bodegas. ¿Desea descargarlo?";
    setMsg(resumen);
    hablar(resumen);
    cargar();
    await previsualizar(d.archivo, "Diferencias por bodega");
  } catch (e) { setErr(e.message); }
}

return (
  <div className="reportes-cols">
    <div className="card">
      <h3>Archivos generados con los códigos oficiales de la base</h3>
      {/* Arriba, junto al título, y no al final de la lista - con muchos
        archivos generados en el día, quedaban fuera de la vista sin
        bajar todo el scroll, y nada indicaba que seguían ahí. */}
      <div className="grilla-botones" style={{ marginTop: 0, marginBottom: 14 }}>
        <button className="btn" onClick={() => generarConsolidado("xlsx")}
          style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
          <Icono nombre="reportes" tam={16} />
          Generar consolidado
        </button>
        <button className="btn" onClick={() => generarDiferencias("xlsx")}
          style={{ display: "inline-flex", alignItems: "center", gap: 6 }}>
          <Icono nombre="reportes" tam={16} />
          Generar diferencias
        </button>
        <button className="btn gris" onClick={() => window.print()}>Imprimir</button>
      </div>
      {msg && <p className="msg-ok">{msg}</p>}
      {err && <p className="error">{err}</p>}
      {recientes === null ? (
        <p className="cargando">Cargando...</p>
      ) : recientes.length === 0 ? (
        <p className="vacio">Todavía no se ha generado ningún archivo.</p>
      ) : (
        recientes.map((a, i) => {
          const clase = CLASE_TITULO[a.titulo] || "azul";
          const colorVar = clase === "oro" ? "var(--amarillo-tx)" : `var(--${VAR_TITULO[a.titulo] ||
"azul"})`;
          const activo = archivoActivo?.archivo === a.archivo;
          return (

```

```

<button key={i} onClick={() => previsualizar(a.archivo, a.titulo, { anunciar: true })}
  style={{
    display: "flex", justifyContent: "space-between", alignItems: "center",
    width: "100%", textAlign: "left", font: "inherit",
    borderTop: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
    borderRight: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
    borderBottom: activo ? "1px solid var(--azul)" : "1px solid var(--borde)",
    borderLeft: `4px solid var(--${VAR_TITULO[a.titulo]} || "azul")`,
    borderRadius: 10, padding: "12px 14px", marginBottom: 10,
    cursor: "pointer", background: activo ? "var(--azul-claro)" : "transparent",
  }}>
  <span style={{ display: "flex", alignItems: "center", gap: 12 }}>
    <span className="icono-kpi" style={{ fontSize: "1.1rem" }}>
      {ICONO_TITULO[a.titulo] || "📄"}
    </span>
    <span>
      <b style={{ color: colorVar }}>{a.titulo}</b>
      <br />
      <small style={{ color: "var(--grafito)" }}>
        {a.formato} · hoy {a.hora}{a.filas != null ? ` · ${a.filas} filas` : ""}
      </small>
    </span>
  </span>
  <span className={`chip ${clase}`}>{a.subtitulo}</span>
</button>
);
})
)}
<p className="pista" style={{ marginTop: 10 }}>
  Dele clic a cualquier tarjeta para ver su contenido aquí al lado.
</p>
</div>

<div className="card">
  <h3>{archivoActivo ? `Vista previa · ${archivoActivo.titulo}` : "Vista previa del
consolidado"}</h3>
  {filasPrevia === null ? (
    <p className="vacio">Genere un archivo o dele clic a uno ya generado para ver una muestra
aquí.</p>
  ) : (
    <>
      <TablaVistaPrevia filas={filasPrevia} />
      <p className="pista" style={{ marginTop: 10 }}>
        {totalFilas != null && `Mostrando ${Math.min(8, totalFilas)} de ${totalFilas} filas.`}
        Los códigos SIN-#### corresponden a registros del extracto que llegaron sin número de
artículo.
      </p>
      <div className="grilla-botones">
        <button className="btn verde"
          onClick={() => descargarReporte(archivoActivo.archivo, token).catch((e) =>
setErr(e.message))}
          style={{ display: "inline-flex", alignItems: "center", gap: 8, width: "100%",
            justifyContent: "center" }}>
          <Icono nombre="descargar" tam={18} />
          Descargar este archivo
        </button>
      </div>
    </>
  )}
</div>
</div>
);
}

/* — Análisis de consumo — */
function TabAnalisis({ token }) {

```

```

const [ analisis, setAnalisis ] = useState(null);
const [ err, setErr ] = useState("");
const [ msg, setMsg ] = useState("");

function cargar() {
  pedir("/api/analisis/consumo?dias=30", {}, token).then(setAnalisis).catch((e) =>
    setErr(e.message));
}
useEffect(cargar, [token]);
useEffect(() => {
  window.addEventListener("cuentavoz:reportes-actualizados", cargar);
  return () => window.removeEventListener("cuentavoz:reportes-actualizados", cargar);
}, [token]);

async function exportar() {
  setErr(""); setMsg("");
  try {
    const d = await pedir("/api/analisis/exportar?formato=xlsx&dias=30", { method: "POST" }, token);
    await descargarReporte(d.archivo, token);
    setMsg("Análisis exportado y descargado.");
  } catch (e) { setErr(e.message); }
}
// "Exporta el análisis de consumo" por voz ya creó el archivo en el
// servidor (el backend respondió con accion:"descargar_reporte" y el
// nombre del archivo) - falta el mismo paso que hace el botón: bajarlo
// de verdad al dispositivo.
useEffect(() => {
  async function alDescargarPorVoz(e) {
    if (!e.detail?.archivo) return;
    setErr(""); setMsg("");
    try {
      await descargarReporte(e.detail.archivo, token);
      setMsg("Análisis exportado y descargado.");
    } catch (err) { setErr(err.message); }
  }
  window.addEventListener("cuentavoz:accion:descargar_reporte", alDescargarPorVoz);
  return () => window.removeEventListener("cuentavoz:accion:descargar_reporte", alDescargarPorVoz);
}, [token]);

if (err) return <p className="error">{err}</p>;
if (!analisis) return <p className="cargando">Cargando...</p>;

const recomendacion = analisis.subutilizados[0];
const maxSobrepedido = Math.max(1, ...analisis.subutilizados.map((s) => s.sobrepedido_pct));

return (
  <>
    {msg && <p className="msg-ok">{msg}</p>}
    <div className="kpis">
      <div className="kpi">
        <div className="kpi-cabeza"><span className="icono-kpi">📦</span><small>Pedido en el
período</small></div>
        <b>{fmt(analisis.pedido_total)} kg</b>
        <i>en {analisis.servicios_periodo} servicios</i>
      </div>
      <div className="kpi">
        <div className="kpi-cabeza"><span className="icono-kpi">📦</span><small>Usado
realmente</small></div>
        <b>{fmt(analisis.usado_total)} kg</b>
        <i>{analisis.aprovechamiento} % de lo pedido</i>
      </div>
      <div className="kpi oro">
        <div className="kpi-cabeza"><span className="icono-kpi">📦</span><small>Insumos
subutilizados</small></div>
        <b>{analisis.subutilizados.length}</b>
      </div>
    </div>
  </>
);

```



```

        <i>se piden y no se usan</i>
      </div>
      <div className="kpi verde">
        <div className="kpi-cabeza"><span className="icono-kpi">📊</span><small>Ahorro
potencial</small></div>
        <b>{fmt( analisis.ahorro_potencial )} kg</b>
        <i>al mes si se ajusta</i>
      </div>
    </div>

    { analisis.subutilizados.length === 0 ? (
      <div className="card">
        <p className="vacio">
          Sin servicios legalizados en los últimos { analisis.dias } días: legalice uno para ver el
análisis.
        </p>
      </div>
    ) : (
      <div className="reportes-cols">
        <div className="card">
          <h3>📋 Sobrepedido frente a la receta</h3>
          <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
            { analisis.subutilizados.slice(0, 6).map((s) => (
              <div key={s.nombre} style={{ display: "flex", alignItems: "center", gap: 10 }}>
                <span style={{ width: 130, fontSize: ".83rem" }}>{s.nombre}</span>
                <div style={{ flex: 1, background: "var(--fondo)", borderRadius: 6, height: 16 }}>
                  <div style={{ width: `${s.sobrepedido_pct / maxSobrepedido * 100}%`, height: 16,
background: s.sobrepedido_pct >= 30 ? "var(--amarillo)" : "var(--
azul)",
borderRadius: 6 }} />
                </div>
                <b style={{ width: 34, textAlign: "right", fontSize: ".83rem"
}}>{s.sobrepedido_pct}</b>
              </div>
            ))}
          </div>
          <p className="pista" style={{ marginTop: 10 }}>% por encima de lo que pide la receta.</p>
        </div>

        <div className="card">
          <h3>🥄 Insumos subutilizados: se piden y sobran</h3>
          { analisis.subutilizados.slice(0, 6).map((s) => (
            <div key={s.nombre} className="registro">
              <span style={{ textTransform: "capitalize" }}>{s.nombre.toLowerCase()}</span>
              <span className="cant">{fmt(s.sobra)} kg sin usar</span>
              <span className="chip oro">{s.veces} de { analisis.servicios_periodo } servicios</span>
            </div>
          ))}
        </div>
      </div>
    )}

    { recomendacion && (
      <div className="card">
        <h3>🤖 Recomendación automática del agente</h3>
        <p className="burbuja">
          { recomendacion.nombre.toLowerCase() } sobra en { recomendacion.veces } de los servicios
          legalizados: la receta pide { recomendacion.sobrepedido_pct }% más de lo que en promedio
          se usa. Ajustar la receta liberaría {fmt(recomendacion.sobra)} kg al mes. La decisión
          es del chef: el agente solo muestra el patrón.
        </p>
        <div className="grilla-botones">
          <button className="btn verde" onClick={exportar}
            style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
            <Icono nombre="descargar" tam={18} />
            Exportar análisis
          </button>
        </div>
      </div>
    )}

```

```

        </button>
      </div>
    </div>
  )}
</>
);
}

```

Archivo: `frontend/src/vistas/Panel.jsx`

```

import { useEffect, useRef, useState } from "react";
import { pedir } from "../api";
import { hablar } from "../voz";
import Marco from "../Marco";
import AsistenteVoz from "../AsistenteVoz";

const SERIE = ["var(--serie-1)", "var(--serie-2)", "var(--serie-3)", "var(--serie-4)"];
const COLOR_UNIDAD = { Unidad: "var(--azul)", Kilogram: "var(--amarillo)",
  Liter: "var(--verde)", Portion: "#C3CAD6" };
const ESTADO_COLOR = { cerrada: "var(--verde)", en_conteo: "var(--azul)",
  en_auditoria: "var(--amarillo)", pendiente: "#C3CAD6" };
const ESTADO_ETQ = { cerrada: "Cerrada", en_conteo: "Conteo",
  en_auditoria: "En auditoría", pendiente: "Pendiente" };

// Nombre oficial de bodega (como en el catálogo) -> version corta para
// que quepa en el eje del gráfico. Solo cambia como se muestra, el
// nombre real que se manda al backend sigue siendo el oficial.
const PREFIJOS_CORTOS = { RESTAURANTE: "Rest.", ALMACEN: "Almacén",
  ZOOLOGICO: "Zoológico", KIOSCO: "Kiosco" };

function nombreCorto(nombre) {
  return (nombre || "").split(" ").map((p, i) => {
    if (i === 0 && PREFIJOS_CORTOS[p]) return PREFIJOS_CORTOS[p];
    if (p === "AYB") return "AyB";
    return p.charAt(0) + p.slice(1).toLowerCase();
  }).join(" ");
}

const MESES = ["ene", "feb", "mar", "abr", "may", "jun", "jul", "ago", "sep", "oct", "nov", "dic"];
const mesCorto = (fechaISO) => MESES[Number(fechaISO.slice(5, 7)) - 1];

// Ejemplos de lo que ya entiende el agente en esta pantalla: navegar entre
// las dos pestañas y preguntar por lo que muestra cada tarjeta o gráfica.
const EJEMPLOS_PANEL = [
  "resumen ejecutivo", "bodegas y alertas", "llévame a bodegas y alertas",
  "¿cuál es la exactitud primera pasada?", "¿cuántas referencias se han contado?",
  "¿cuántas alertas se han gestionado?", "¿cuántas bodegas están cerradas?",
  "¿qué bodega tiene más diferencia?", "¿cómo va el stock por unidad de medida?",
  "¿cómo va la tendencia de exactitud?", "¿cuántos negativos se han corregido?",
  "¿cuál es el tiempo promedio de conteo?", "¿cuántos alias aprendió el agente?",
  "¿cómo están las bodegas?", "¿cuáles son las alertas por tipo?",
  "¿cuál es el descuadre principal?",
];

export default function Panel({ token, ir, tabInicial, navSeq }) {
  const [tab, setTab] = useState(tabInicial || "resumen");
  // Pedir una pestaña de esta MISMA pantalla por voz no remonta el
  // componente - sin esto, el useState de arriba no vuelve a leer
  // tabInicial y la pestaña se queda en la que ya estaba. navSeq (sube en
  // cada ir()) tiene que ir en la dependencia también: sin él, pedir la
  // MISMA pestaña dos veces seguidas (con un clic manual a otra en el
  // medio) no hacía nada, porque tabInicial ya traía ese mismo valor de
  // la vez anterior y el efecto no vuelve a correr si su dependencia no
  // cambió de valor.
  useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);
  const [resumen, setResumen] = useState(null);

```

```

const [alertas, setAlertas] = useState(null);

useEffect(() => {
  pedir("/api/panel/resumen", {}, token).then(setResumen).catch(() => {});
  pedir("/api/panel/alertas", {}, token).then(setAlertas).catch(() => {});
}, [token]);

return (
  <Marco titulo="Panel gerencial" chip={{ texto: "ACTUALIZADO AHORA", tipo: "verde" }}>
    <AsistenteVoz token={token} vista="panel" ir={ir} ejemplos={EJEMPLOS_PANEL}
      placeholder="¿cuál es la exactitud primera pasada?, lléveme a inicio..." />
    <div className="chips">
      <button className={`chip ${tab === "resumen" ? "azul" : ""}`}
        onClick={() => setTab("resumen")}>Resumen ejecutivo</button>
      <button className={`chip ${tab === "alertas" ? "azul" : ""}`}
        onClick={() => setTab("alertas")}>Bodegas y alertas</button>
    </div>

    {tab === "resumen" && <TabResumen r={resumen} />}
    {tab === "alertas" && <TabAlertas a={alertas} />}
  </Marco>
);
}

function TabResumen({ r }) {
  const [saludo, setSaludo] = useState("");
  const yaSaludo = useRef(false);

  // Al llegar a esta pestaña, un titular breve en vez de esperar a que
  // pregunten - el detalle completo de cada tarjeta/gráfica sigue
  // disponible por voz (o en el desplegable de frases), esto es solo el
  // resumen de resumen para no leer las cuatro tarjetas de una sentada.
  useEffect(() => {
    if (!r || yaSaludo.current) return;
    yaSaludo.current = true;
    const texto = `Resumen ejecutivo: exactitud primera pasada ${r.exactitud_primera_pasada}%, `
      + `${r.alertas_gestionadas} de ${r.alertas_total} alertas gestionadas, `
      + `${r.bodegas_cerradas} de ${r.bodegas_total} bodegas cerradas.`;
    setSaludo(texto);
    hablar(texto);
  }, [r]);

  if (!r) return <p className="cargando">Cargando...</p>;
  const maxDif = Math.max(1, ...r.diferencia_por_bodega.map((d) => d.diferencia));

  return (
    <>
      {saludo && (
        <div className="card">
          <p className="rotulo" style={{ marginTop: 0 }}>CuentaVoz responde</p>
          <p className="burbuja" aria-live="polite" aria-atomic="true">{saludo}</p>
        </div>
      )}
      <div className="kpis">
        <div className="kpi verde">
          <div className="kpi-cabeza">
            <span className="icono-kpi">📊</span>
            <small>Exactitud primera pasada</small>
          </div>
          <b>{r.exactitud_primera_pasada} %</b><i>promedio de cierres</i></div>
        <div className="kpi">
          <div className="kpi-cabeza">
            <span className="icono-kpi">📦</span>
            <small>Referencias contadas</small>
          </div>

```

```

    </div>
    <b>{r.referencias_contadas}</b><i>en toda la operación</i></div>
  <div className="kpi oro">
    <div className="kpi-cabeza">
      <span className="icono-kpi">⚠</span>
      <small>Alertas gestionadas</small>
    </div>
    <b>{r.alertas_gestionadas} / {r.alertas_total}</b><i>resueltas del total</i></div>
  <div className="kpi">
    <div className="kpi-cabeza">
      <span className="icono-kpi">🔒</span>
      <small>Bodegas cerradas</small>
    </div>
    <b>{r.bodegas_cerradas} / {r.bodegas_total}</b><i>con doble firma digital</i></div>
</div>

<div className="conteo-cols">
  <div className="card">
    <h3>Diferencia absoluta por bodega</h3>
    {r.diferencia_por_bodega.length === 0 ? (
      <p className="vacio">Sin diferencias registradas todavía.</p>
    ) : (
      r.diferencia_por_bodega.map((d, i) => (
        <div className="barra-fila" key={d.bodega}>
          <span className="etq">{nombreCorto(d.bodega)}</span>
          <div className="pista">
            <div className="rellena" style={{ width: `${d.diferencia / maxDif * 100}%`,
              background: i === 0 ? "var(--amarillo)" : "var(--
serie-1)" }} />
          </div>
          <span className="val">{d.diferencia}</span>
        </div>
      ))
    )}
  </div>
  <div className="card">
    <h3>Stock por unidad de medida</h3>
    <Dona datos={r.stock_por_unidad.map((u, i) => ({
      etq: u.unidad, valor: u.cantidad, pct: u.pct,
      color: COLOR_UNIDAD[u.unidad] || SERIE[i % SERIE.length],
    })))} />
  </div>
</div>

<div className="card">
  <h3>Exactitud por toma de inventario</h3>
  {r.historial_exactitud.length < 2 ? (
    <p className="vacio">Se necesitan al menos dos cierres para ver la tendencia.</p>
  ) : (
    <Linea puntos={r.historial_exactitud} />
  )}
  <p className="pista" style={{ marginTop: 8 }}>
    Un punto por cada bodega cerrada con doble firma, en orden cronológico.
  </p>
</div>
</>
);
}

function TabAlertas({ a }) {
  const [saludo, setSaludo] = useState("");
  const yaSaludo = useRef(false);

  useEffect(() => {
    if (!a || yaSaludo.current) return;

```

```

yaSaludo.current = true;
const texto = a.negativos_iniciales !== a.negativos_actuales
  ? `Bodegas y alertas: saldos negativos, ${a.negativos_iniciales} al inicio del período, `
  + `${a.negativos_actuales} ahora. Tiempo promedio de conteo ${a.tiempo_promedio_min} minutos.`
  : `Bodegas y alertas: ${a.negativos_actuales} saldos negativos en el sistema, se corrigen `
  + `en My Inventory. Tiempo promedio de conteo ${a.tiempo_promedio_min} minutos.`;
setSaludo(texto);
hablar(texto);
}, [a]);

if (!a) return <p className="cargando">Cargando...</p>;
const estados = Object.entries(a.estado_bodegas);
const maxAlerta = Math.max(1, ...a.alertas_por_tipo.map((x) => x.cantidad));

return (
  <>
    {saludo && (
      <div className="card">
        <p className="rotulo" style={{ marginTop: 0 }}>CuentaVoz responde</p>
        <p className="burbuja" aria-live="polite" aria-atomic="true">{saludo}</p>
      </div>
    )}
    <div className="kpi">
      <div className="kpi oro">
        <div className="kpi-cabeza">
          <span className="icono-kpi">📊</span>
          <small>Saldos negativos en el sistema</small>
        </div>
        /* La corrección de un saldo negativo pasa por fuera de
          CuentaVoz (My Inventory, no esta app) - dentro de UNA sola
          toma este número no se mueve solo, así que la flecha
          "antes → ahora" solo se muestra si de verdad hay una
          diferencia real que contar; mostrarla igualada a sí misma
          ("79 → 79") no cuenta nada y parece un dato inventado. */
        {a.negativos_iniciales !== a.negativos_actuales ? (
          <>
            <b>{a.negativos_iniciales} → {a.negativos_actuales}</b>
            <i>corregidos desde el inicio de este período</i>
          </>
        ) : (
          <>
            <b>{a.negativos_actuales}</b>
            <i>artículos - la corrección se hace en My Inventory</i>
          </>
        )}
      </div>
      <div className="kpi">
        <div className="kpi-cabeza">
          <span className="icono-kpi">🕒</span>
          <small>Tiempo promedio de conteo por bodega</small>
        </div>
        <b>{a.tiempo_promedio_min ?? "-"} min</b><i>conteos ya cerrados</i></div>
      <div className="kpi verde">
        <div className="kpi-cabeza">
          <span className="icono-kpi">🗣️</span>
          <small>Alias aprendidos por el agente</small>
        </div>
        <b>{a.alias_aprendidos}</b><i>el agente habla como el equipo</i></div>
    </div>

    <div className="dos-columnas">
      <div className="card">
        <h3>Estado de las bodegas</h3>
        <div className="puntos-bodegas">
          {estados.flatMap(([estado, n]) =>
            Array.from({ length: n }, (_, i) => (

```

```

        <span key={` ${estado}-${i}`} style={{ background: ESTADO_COLOR[estado] }}
          title={ESTADO_ETQ[estado]} />
      ))
    })
  </div>
  <div className="leyenda">
    {estados.map(([estado, n]) => (
      <span key={estado}>
        <span className="punto" style={{ background: ESTADO_COLOR[estado] }} />
        {ESTADO_ETQ[estado]} · {n}
      </span>
    ))}
  </div>
</div>

<div className="card">
  <h3>Alertas por tipo</h3>
  {a.alertas_por_tipo.length === 0 ? (
    <p className="vacio">Sin alertas registradas todavía.</p>
  ) : (
    a.alertas_por_tipo.map((x, i) => (
      <div className="barra-fila" key={x.tipo}>
        <span className="etq">{x.tipo}</span>
        <div className="pista">
          <div className="rellena" style={{ width: `${x.cantidad / maxAlerta * 100}%`,
            background: i === 0 ? "var(--amarillo)" : "var(--
serie-1)" }} />
        </div>
        <span className="val">{x.cantidad}</span>
      </div>
    ))
  )}
</div>
</div>

<div className="card">
  <h3>Descuadres recurrentes: dónde está la causa raíz</h3>
  {a.descuadres_recurrentes.length === 0 ? (
    <p className="vacio">Sin descuadres repetidos todavía.</p>
  ) : (
    <table>
      <thead>
        <tr><th>Artículo</th><th>Tipo de alerta</th><th>Dif. acumulada</th>
          <th>Tomas</th><th>Acción sugerida</th></tr>
      </thead>
      <tbody>
        {a.descuadres_recurrentes.map((d, i) => (
          <tr key={i}>
            <td>{d.articulo}</td><td>{d.tipo}</td>
            <td className="dif">{d.diferencia}</td><td>{d.tomas}</td>
            <td>{d.accion}</td>
          </tr>
        ))}
      </tbody>
    </table>
  )}
</div>
</>
);
}

/** Dona hecha con conic-gradient: sin librerías, con leyenda etiquetada. */
function Dona({ datos }) {
  if (!datos.length) return <p className="vacio">Sin datos de stock todavía.</p>;
  let acumulado = 0;
  const segmentos = datos.map((d) => {

```

```

    const inicio = acumulado;
    acumulado += d.pct;
    return `${d.color} ${inicio}% ${acumulado}%`;
  });
  const total = datos.reduce((s, d) => s + d.valor, 0);
  return (
    <div style={{ display: "flex", gap: 20, alignItems: "center", flexWrap: "wrap" }}>
      <div style={{ position: "relative", width: 140, height: 140, flex: "none" }}>
        <div style={{
          width: 140, height: 140, borderRadius: "50%",
          background: `conic-gradient(${segmentos.join(",")}` ,
        }} />
        <div style={{
          position: "absolute", inset: 22, borderRadius: "50%", background: "#fff",
          display: "flex", flexDirection: "column",
          alignItems: "center", justifyContent: "center",
          boxShadow: "0 0 0 1px rgba(11,27,51,.04)",
        }}>
          <b style={{ fontSize: "1.05rem", color: "var(--azul)" }}>
            {total.toLocaleString("es-CO")}
          </b>
          <span style={{ fontSize: ".68rem", color: "var(--grafito)" }}>referencias</span>
        </div>
      </div>
      <div>
        {datos.map((d) => (
          <div key={d.etq} style={{ marginBottom: 6, fontSize: ".85rem" }}>
            <span className="punto" style={{ background: d.color, display: "inline-block",
              width: 10, height: 10, borderRadius: "50%", marginRight: 8 }} />
            <b>{d.etq}</b> - {d.pct}% ({d.valor.toLocaleString("es-CO")})
          </div>
        ))}
      </div>
    </div>
  );
}

/** Línea de tendencia con SVG puro: una sola serie, sin leyenda. Como
    todos los cierres pueden caer el mismo día, el eje muestra la bodega
    de cada punto (ya es información real y distinta punto a punto) en
    vez de repetir el mismo mes una y otra vez. */
function Línea({ puntos }) {
  const w = 460, h = 128, pad = 20, padInferior = 34;
  const valores = puntos.map((p) => p.exactitud);
  const min = Math.min(...valores) - 2, max = Math.max(...valores) + 2;
  const paso = (w - pad * 2) / (puntos.length - 1);
  const y = (v) => h - padInferior - ((v - min) / (max - min || 1)) * (h - pad - padInferior);
  const coords = puntos.map((p, i) => [pad + i * paso, y(p.exactitud)]);
  const linea = coords.map(([x, yy], i) => `${i === 0 ? "M" : "L"}${x},${yy}`).join(" ");

  const mejora = valores[valores.length - 1] >= valores[0];
  const color = "var(--serie-1)";

  return (
    <>
      {puntos.length > 1 && (
        <p style={{ textAlign: "right", fontSize: ".78rem", fontWeight: 700,
          color: mejora ? "var(--verde)" : "var(--grafito)", marginBottom: 4 }}>
          {mejora ? "▲ mejora sostenida" : "▼ en descenso"}
        </p>
      )}
      <svg width="100%" viewBox={`0 0 ${w} ${h}`} role="img" aria-label="Tendencia de exactitud">
        <line x1={pad} y1={h - padInferior} x2={w - pad} y2={h - padInferior} stroke="var(--borde)"
        strokeWidth="1" />
        <path d={linea} fill="none" stroke={color} strokeWidth="2" />
        {coords.map(([x, yy], i) => (

```

```

        <circle key={i} cx={x} cy={yy} r="3.5" fill={color} />
      )))
      {puntos.map((p, i) => (
        <text key={i} x={coords[i][0]} y={h - padInferior + 11} fontSize="6.5" textAnchor="end"
          fill="var(--grafito)" transform={`rotate(-40 ${coords[i][0]} ${h - padInferior +
11})`} >
          <title>{nombreCorto(p.bodega)}</title>
          {`Toma ${i + 1}`}
        </text>
      )))
      <text x={coords[coords.length - 1][0]} y={coords[coords.length - 1][1] - 10}
        fontSize="11" fontWeight="700" textAnchor="middle" fill={color}>
        {valores[valores.length - 1]}%
      </text>
    </svg>
  </>
);
}

```

12.32 Frontend: Ajustes — catálogo, recetas, usuarios y trazabilidad (vistas 5.24 a 5.26)

Archivo: `frontend/src/vistas/Ajustes.jsx`

```

import { useEffect, useState } from "react";
import { pedir, descargarReporte } from "../api";
import { escuchar, hablar, quitarTildes } from "../voz";
import Marco from "../Marco";
import AsistenteVoz from "../AsistenteVoz";
import Icono from "../Iconos";

const _EJEMPLOS_POR_PESTANA = {
  config: [
    "activa el modo sin conexión",
    "desactiva el modo sin conexión",
  ],
  usuarios: [
    "crea un usuario llamado Juan perfil auxiliar",
    "activa a Juan", "desactiva a Juan",
    "cambia el perfil de Juan a administrador",
    "asígnale la bodega Almacén General a Juan",
    "quítale la bodega Almacén General a Juan",
    "¿quién tiene la bodega Almacén General?",
    "¿cuántas bodegas sin asignar hay?",
    "muéstrame la asignación por bodega",
    "oculta la asignación por bodega",
  ],
  recetas: [
    "crea una receta llamada Sopa, rendimiento 4 porciones, con 2 kilos de papa",
    "agrega 300 gramos de cebolla a la receta Sopa",
    "quita el ingrediente cebolla de la receta Sopa",
    "cambia el rendimiento de la receta Sopa a 6 porciones",
    "agrega la preparación a la receta Sopa: pele las papas, sofría la cebolla, y cocine todo junto veinte minutos",
    "elimina la receta Sopa",
  ],
  traza: [
    "acciones de Luis hoy",
    "aprobaciones de esta semana",
    "exporta el registro de trazabilidad",
    "acciones por persona",
    "¿cuántas acciones tiene Luis?",
  ],
};

export default function Ajustes({ token, usuario, ir, tabInicial, recetaId, navSeq }) {

```



```

const [tab, setTab] = useState(tabInicial || "config");
const esAuditor = usuario?.perfil === "auditor";
// "llévame a gestión de usuarios" dicho desde esta MISMA pantalla no
// remonta el componente (sigue siendo "ajustes"), así que el useState
// de arriba no vuelve a leer tabInicial - sin esto, el agente decía
// "vamos a Gestión de usuarios" y la pestaña se quedaba en la que ya
// estaba. navSeq (sube en cada ir()) también va en la dependencia: sin
// él, pedir la MISMA pestaña dos veces seguidas (con un clic manual a
// otra en el medio) no hacía nada, porque tabInicial no cambiaba.
useEffect(() => { if (tabInicial) setTab(tabInicial); }, [tabInicial, navSeq]);

return (
  <Marco titulo="Ajustes · configuración del sistema"
    chip={{ texto: esAuditor ? "ADMINISTRADORA" : "SOLO LECTURA",
      tipo: esAuditor ? "azul" : "gris" }}>
    <AsistenteVoz token={token} vista="ajustes" ir={ir}
      placeholder="¿cuál es el umbral de anomalía?, lléveme a reportes..."
      ejemplos={_EJEMPLOS_POR_PESTANA[tab]}
      alActualizar={() => window.dispatchEvent(new Event("cuentavoz:ajustes -
actualizados"))} />
    <div className="chips">
      <button className={`chip ${tab === "config" ? "azul" : ""}`}
        onClick={() => setTab("config")}>Configuración</button>
      {esAuditor && (
        <>
          <button className={`chip ${tab === "usuarios" ? "azul" : ""}`}
            onClick={() => setTab("usuarios")}>Gestión de usuarios</button>
          <button className={`chip ${tab === "recetas" ? "azul" : ""}`}
            onClick={() => setTab("recetas")}>Recetas</button>
          <button className={`chip ${tab === "traza" ? "azul" : ""}`}
            onClick={() => setTab("traza")}>Registro de trazabilidad</button>
        </>
      )}
    </div>

    {tab === "config" && <TabConfig token={token} esAuditor={esAuditor} />}
    {tab === "usuarios" && esAuditor && <TabUsuarios token={token} usuario={usuario} />}
    {tab === "recetas" && esAuditor && <TabRecetas token={token} recetaInicial={recetaId} />}
    {tab === "traza" && esAuditor && <TabTraza token={token} />}
  </Marco>
);
}

/* — Interruptor simple, sin librería — */
function Interruptor({ activo, onChange, disabled, etiqueta }) {
  return (
    <button
      onClick={() => !disabled && onChange(!activo)}
      disabled={disabled}
      role="switch"
      aria-checked={activo}
      aria-label={etiqueta}
      style={{
        width: 44, height: 24, borderRadius: 14, padding: 2,
        background: activo ? "var(--verde)" : "#C3CAD6",
        display: "flex", justifyContent: activo ? "flex-end" : "flex-start",
        opacity: disabled ? 0.6 : 1, cursor: disabled ? "not-allowed" : "pointer",
      }}
    >
      <span style={{ width: 20, height: 20, borderRadius: "50%", background: "#fff" }} />
    </button>
  );
}

function TabConfig({ token, esAuditor }) {
  const [cfg, setCfg] = useState(null);

```

```

const [umbral, setUmbral] = useState(50);
const [offline, setOffline] = useState(true);
const [msg, setMsg] = useState("");

function cargar() {
  pedir("/api/ajustes", {}, token).then((c) => {
    setCfg(c); setUmbral(c.umbral); setOffline(c.offline);
  }).catch(() => {});
}
useEffect(cargar, [token]);
// El agente de "Pregúntele al agente" (arriba, fuera de esta pestaña)
// puede activar/desactivar el modo sin conexión por voz - cuando lo
// hace, avisa con este evento para que esta pestaña vuelva a leer el
// valor real en vez de quedarse mostrando el de antes.
useEffect(() => {
  window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
  return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
}, [token]);

async function guardar() {
  try {
    await pedir("/api/ajustes", {
      method: "PUT", body: JSON.stringify({ umbral, offline }),
    }, token);
    setMsg("Configuración guardada.");
    cargar();
  } catch (e) { setMsg(e.message); }
}

if (!cfg) return <p className="cargando">Cargando...</p>;

return (
  <>
    <div className="kpi">
      <div className="kpi">
        <div className="kpi-cabeza">
          <span className="icono-kpi">👤</span>
          <small>Usuarios activos</small>
        </div>
        <b>{cfg.usuarios_activos}</b><i>con acceso al sistema</i>
      </div>
      <div className={cfg.aprobaciones_pendientes > 0 ? "kpi oro" : "kpi verde"}>
        <div className="kpi-cabeza">
          <span className="icono-kpi">☑</span>
          <small>Aprobaciones pendientes</small>
        </div>
        <b>{cfg.aprobaciones_pendientes}</b>
        <i>{cfg.aprobaciones_pendientes > 0 ? "por revisar en Auditoría" : "todo al día"}</i>
      </div>
      <div className={offline ? "kpi verde" : "kpi"}>
        <div className="kpi-cabeza">
          <span className="icono-kpi">🌐</span>
          <small>Modo sin conexión</small>
        </div>
        <b>{offline ? "Activo" : "Inactivo"}</b><i>refresco del tablero en tiempo real</i>
      </div>
    </div>
    /* La versión/modelo ya se ve completa en la tarjeta "Acerca de
    CuentaVoz" de abajo (con base de datos e idioma también) - un KPI
    arriba con solo dos de esos cuatro datos era la misma información
    dos veces, no una tarjeta nueva. */

    <div className="dos-columnas" style={{ gap: 16 }}>
      <div className="card">
        <h3><span className="icono-kpi" style={{ marginRight: 8 }}>🔒</span>Validación de datos</h3>

```

```

<table>
  <tbody>
    <tr>
      <td>Umbral de anomalía</td>
      <td style={{ width: 140 }}>
        {esAuditor ? (
          <input type="number" min="1" max="500" value={umbral}
            onChange={(e) => setUmbral(Number(e.target.value))}
            aria-label="Umbral de anomalía, en porcentaje"
            style={{ width: 80, padding: "6px 10px",
              border: "1px solid var(--borde)", borderRadius: 8 }} />
        ) : <b>{cfg.umbral} %</b>}
        {esAuditor && " %"}
      </td>
    </tr>
    <tr><td>Bloquear cantidades negativas</td>
      <td><Interruptor activo disabled onChange={() => {}}
        etiqueta="Bloquear cantidades negativas" /></td></tr>
    <tr><td>Exigir confirmación en alertas</td>
      <td><Interruptor activo disabled onChange={() => {}}
        etiqueta="Exigir confirmación en alertas" /></td></tr>
    <tr><td>Permitir crear productos pendientes</td>
      <td><Interruptor activo disabled onChange={() => {}}
        etiqueta="Permitir crear productos pendientes" /></td></tr>
    </tbody>
  </table>
<p className="pista" style={{ marginTop: 8 }}>
  Las reglas fijas del reto (bloquear negativos, exigir confirmación) no se
  pueden desactivar: son la garantía de integridad de la toma.
</p>
</div>

<div className="card">
  <h3><span className="icono-kpi" style={{ marginRight: 8 }}>🌐</span>Conexión y
  sincronización</h3>
  <table>
    <tbody>
      <tr><td>Modo sin conexión</td>
        <td><Interruptor activo={offline} onChange={setOffline} disabled={!esAuditor}
          etiqueta="Modo sin conexión" /></td></tr>
      <tr><td>Refresco del tablero</td><td><b>tiempo real</b></td></tr>
    </tbody>
  </table>

  <h3 style={{ marginTop: 20 }}>
    <span className="icono-kpi" style={{ marginRight: 8 }}>📄</span>Acerca de CuentaVoz
  </h3>
  <table>
    <tbody>
      <tr><td>Versión</td><td><b>{cfg.version}</b></td></tr>
      <tr><td>Modelo del agente</td><td><b>{cfg.modelo}</b></td></tr>
      <tr><td>Base de datos</td><td><b>{cfg.base_datos}</b></td></tr>
      <tr><td>Idioma del reconocimiento</td><td><b>{cfg.idioma_voz}</b></td></tr>
    </tbody>
  </table>
</div>
</div>

{esAuditor && (
  <div className="grilla-botones" style={{ marginTop: 4 }}>
    <button className="btn" onClick={guardar}
      style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
      <Icono nombre="aprobar" tam={18} />
      Guardar configuración
    </button>
    <button className="btn borde" onClick={cargar}

```

```

        style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
        <Icono nombre="refrescar" tam={18} />
        Restaurar valores
      </button>
    </div>
  )}
  {msg && <p className="msg-ok">{msg}</p>}
</>
);
}

function TabUsuarios({ token, usuario }) {
  const [usuarios, setUsuarios] = useState([]);
  const [bodegas, setBodegas] = useState([]);
  const [msg, setMsg] = useState("");
  const [nuevo, setNuevo] = useState(null);
  const [asignando, setAsignando] = useState(null); // usuario al que se le estan marcando bodegas
  const [marcadas, setMarcadas] = useState(new Set());
  const [cobertura, setCobertura] = useState(null);
  const [verCobertura, setVerCobertura] = useState(false);
  const [editando, setEditando] = useState(null); // usuario que se esta editando (correo/rol)
  const [filtro, setFiltro] = useState("todos");

  function cargar() {
    pedir("/api/usuarios", {}, token).then(setUsuarios).catch(() => {});
    pedir("/api/bodegas", {}, token).then(setBodegas).catch(() => {});
    if (verCobertura) cargarCobertura();
  }
  useEffect(cargar, [token]);
  // El agente ("Pregúntele al agente", arriba de las pestañas) puede
  // activar/desactivar a alguien por voz - cuando lo hace, avisa con
  // este evento para que esta lista refleje el cambio sin recargar.
  useEffect(() => {
    window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
    return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
  }, [token]);

  // Estos dos modales estan armados a mano (overlay + modal), no con el
  // componente Dialogo compartido - por eso no traian ya el mismo Escape
  // para cerrar que ese componente sí tiene.
  useEffect(() => {
    function alTecla(e) {
      if (e.key !== "Escape") return;
      if (editando) setEditando(null);
      else if (asignando) setAsignando(null);
    }
    window.addEventListener("keydown", alTecla);
    return () => window.removeEventListener("keydown", alTecla);
  }, [editando, asignando]);

  function cargarCobertura() {
    pedir("/api/bodegas/asignaciones", {}, token).then(setCobertura).catch(() => {});
  }

  // "muéstrame/oculta la asignación por bodega" por voz - mismo botón
  // «Ver/Ocultar asignación por bodega», solo que disparado por el
  // agente en vez del clic.
  useEffect(() => {
    const mostrar = () => { setVerCobertura(true); cargarCobertura(); };
    const ocultar = () => setVerCobertura(false);
    window.addEventListener("cuentavoz:accion:mostrar_cobertura", mostrar);
    window.addEventListener("cuentavoz:accion:ocultar_cobertura", ocultar);
    return () => {
      window.removeEventListener("cuentavoz:accion:mostrar_cobertura", mostrar);
      window.removeEventListener("cuentavoz:accion:ocultar_cobertura", ocultar);
    };
  });
}

```

```

    };
  }, [token]);

  async function crear() {
    if (!nuevo?.nombre?.trim()) return;
    try {
      // sin "pin" en el body: el backend genera uno temporal distinto
      // para cada persona en vez de reusar siempre la misma clave.
      const r = await pedir("/api/usuarios", {
        method: "POST",
        body: JSON.stringify({ nombre: nuevo.nombre, perfil: nuevo.perfil || "auxiliar",
                              correo: nuevo.correo || "" }),
      }, token);
      setMsg(`Usuario ${nuevo.nombre} creado. PIN temporal: ${r.pin_temporal} `
        + "(cambíelo al primer ingreso, en Mi perfil).");
      setNuevo(null);
      cargar();
    } catch (e) { setMsg(e.message); }
  }

  function alternarBodega(id) {
    setMarcadas((prev) => {
      const nueva = new Set(prev);
      nueva.has(id) ? nueva.delete(id) : nueva.add(id);
      return nueva;
    });
  }

  async function guardarAsignacion() {
    const u = asignando;
    const ids = [...marcadas];
    setAsignando(null);
    try {
      await pedir(`/api/usuarios/${u.id}/bodegas`, {
        method: "PUT", body: JSON.stringify({ bodega_ids: ids }),
      }, token);
      setMsg(`${ids.length} bodegas asignadas a ${u.nombre}.`);
      cargar();
    } catch (e) { setMsg(e.message); }
  }

  async function guardarEdicion() {
    const u = editando;
    setEditando(null);
    try {
      await pedir(`/api/usuarios/${u.id}`, {
        method: "PUT", body: JSON.stringify({ correo: u.correo, perfil: u.perfil }),
      }, token);
      setMsg(`${u.nombre} actualizado.`);
      cargar();
    } catch (e) { setMsg(e.message); }
  }

  async function alternarActivo(persona) {
    try {
      await pedir(`/api/usuarios/${persona.id}`, {
        method: "PUT", body: JSON.stringify({ activo: !persona.activo }),
      }, token);
      setMsg(`${persona.nombre} ${persona.activo ? "desactivado" : "activado"}.`);
      cargar();
    } catch (e) { setMsg(e.message); }
  }

  const nAux = usuarios.filter((u) => u.perfil === "auxiliar").length;
  const nAdmin = usuarios.filter((u) => u.perfil === "auditor").length;

```

```

const nInactivos = usuarios.filter((u) => !u.activo).length;
const usuariosFiltrados = usuarios.filter((u) => {
  if (filtro === "auxiliar") return u.perfil === "auxiliar";
  if (filtro === "auditor") return u.perfil === "auditor";
  if (filtro === "inactivo") return !u.activo;
  return true;
});

return (
  <div className="card">
    <h3><span className="icono-kpi" style={{ marginRight: 8 }}>  </span>
      Gestión de usuarios ({usuarios.length})</h3>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center",
      marginBottom: 12, flexWrap: "wrap", gap: 10 }}>
      <div className="chips" style={{ marginBottom: 0 }}>
        <button className={`chip ${filtro === "auxiliar" ? "azul" : ""}`}
          onClick={() => setFiltro(filtro === "auxiliar" ? "todos" : "auxiliar")}>
          {nAux} auxiliares
        </button>
        <button className={`chip ${filtro === "auditor" ? "azul" : ""}`}
          onClick={() => setFiltro(filtro === "auditor" ? "todos" : "auditor")}>
          {nAdmin} administradores
        </button>
        <button className={`chip ${filtro === "inactivo" ? "azul" : ""}`}
          onClick={() => setFiltro(filtro === "inactivo" ? "todos" : "inactivo")}>
          {nInactivos} inactivos
        </button>
      </div>
      {!nuevo && (
        <button className="btn" onClick={() => setNuevo({})}>+ Nuevo usuario</button>
      )}
    </div>
    {msg && <p className="msg-ok">{msg}</p>}
    <table>
      <thead><tr><th>Persona</th><th>Perfil</th><th>Bodegas asignadas</th>
        <th>Último acceso</th><th>Estado</th><th></th></tr></thead>
      <tbody>
        {usuariosFiltrados.map((u) => (
          <tr key={u.id}>
            <td style={{ textTransform: "capitalize", display: "flex", alignItems: "center", gap: 8 }}>
              <span className="avatar-chico">{u.nombre[0]?.toUpperCase()}</span>
              {u.nombre}
            </td>
            <td>{u.perfil === "auditor" ? "Administrador" : "Auxiliar"}</td>
            <td>{u.bodegas_asignadas}</td>
            <td>{u.ultimo_acceso ? u.ultimo_acceso.slice(5).replace("-", "/") : "nunca"}</td>
            <td style={{ color: u.activo ? "var(--verde)" : "var(--grafito)", fontWeight: 700 }}>
              {u.activo ? "Activo" : "Inactivo"}
            </td>
            <td style={{ display: "flex", gap: 6, flexWrap: "wrap" }}>
              <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
                onClick={async () => {
                  setAsignando(u);
                  const propias = await pedir(`/api/usuarios/${u.id}/bodegas`, {},
token).catch(() => []);
                  setMarcadas(new Set(propias));
                }}>
                Asignar bodegas
              </button>
              <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
                onClick={() => setEditando({ id: u.id, nombre: u.nombre,
                  correo: u.correo, perfil: u.perfil })}>
                Editar
              </button>
              <button className={`btn ${u.activo ? "gris" : "verde"}`}
                style={{ padding: "4px 10px", minHeight: 0 }}

```

```

        disabled={u.id === usuario?.id}
        title={u.id === usuario?.id ? "No puede desactivar su propia cuenta" : ""}
        onClick={() => alternarActivo(u)}>
        {u.activo ? "Desactivar" : "Activar"}
      </button>
    </td>
  </tr>
</tbody>
</table>

{nuevo ? (
  <div style={{ marginTop: 16, display: "flex", gap: 10, flexWrap: "wrap" }}>
    <input placeholder="nombre de usuario" value={nuevo.nombre || ""}
      onChange={(e) => setNuevo({ ...nuevo, nombre: e.target.value })}
      aria-label="Nombre de usuario"
      style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}
    />

    <input placeholder="correo (opcional)" value={nuevo.correo || ""}
      onChange={(e) => setNuevo({ ...nuevo, correo: e.target.value })}
      aria-label="Correo, opcional"
      style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}
    />

    <select value={nuevo.perfil || "auxiliar"}
      onChange={(e) => setNuevo({ ...nuevo, perfil: e.target.value })}
      aria-label="Perfil del nuevo usuario"
      style={{ padding: "10px 12px", border: "1px solid var(--borde)", borderRadius: 10
    }}>

      <option value="auxiliar">Auxiliar</option>
      <option value="auditor">Administrador</option>
    </select>
    <button className="btn" onClick={crear}>Crear (PIN StockXperts)</button>
    <button className="btn gris" onClick={() => setNuevo(null)}>Cancelar</button>
  </div>
) : null}

<div className="grilla-botones" style={{ marginTop: 14 }}>
  <button className="btn borde"
    onClick={() => {
      const abrir = !verCobertura;
      setVerCobertura(abrir);
      if (abrir && !cobertura) cargarCobertura();
    }}>
    {verCobertura ? "Ocultar asignación por bodega" : "Ver asignación por bodega"}
  </button>
</div>

{verCobertura && (
  <div style={{ marginTop: 14 }}>
    <h3>Quién tiene cada bodega</h3>
    <p className="pista" style={{ marginBottom: 10 }}>
      Para repartir 54 bodegas entre varios auxiliares sin dejar ninguna sin dueño
      ni asignarla dos veces por error.
    </p>
    {cobertura === null ? (
      <p className="cargando">Cargando...</p>
    ) : (
      <div style={{ maxHeight: 420, overflowY: "auto" }}>
        <table>
          <thead><tr><th>Bodega</th><th>Asignada a</th></tr></thead>
          <tbody>
            {cobertura.map((b) => (
              <tr key={b.id}>
                <td>{b.bodega}</td>
                <td>
                  {b.asignados.length === 0 ? (

```

```

        <span className="chip oro">Sin asignar</span>
      ) : (
        b.asignados.map((p) => (
          <span key={p.id} className={`chip ${p.perfil === "auditor" ? "azul" :
"gris"}}`>
            style={{ marginRight: 6, textTransform: "capitalize" }}>
              {p.nombre}
            </span>
          ))
        </td>
      </tr>
    </tbody>
  </table>
</div>
)}
</div>
)}

<div className="card" style={{ background: "var(--fondo)", marginTop: 14 }}>
  <h3>Qué puede hacer cada perfil</h3>
  <table>
    <tbody>
      <tr>
        <td><input checked="" type="checkbox"> Contar por voz y corregir sus propios registros</td>
        <td style={{ textAlign: "right", color: "var(--verde)", fontWeight: 700 }}>
          Auxiliar y administrador
        </td>
      </tr>
      <tr>
        <td><input checked="" type="checkbox"> Crear productos y bodegas (quedan pendientes)</td>
        <td style={{ textAlign: "right", color: "var(--verde)", fontWeight: 700 }}>
          Auxiliar y administrador
        </td>
      </tr>
      <tr>
        <td><input checked="" type="checkbox"> Aprobar creaciones, auditar y cerrar bodegas</td>
        <td style={{ textAlign: "right", color: "var(--azul)", fontWeight: 700 }}>
          Solo administrador
        </td>
      </tr>
      <tr>
        <td><input checked="" type="checkbox"> Gestionar usuarios y cambiar los ajustes del sistema</td>
        <td style={{ textAlign: "right", color: "var(--azul)", fontWeight: 700 }}>
          Solo administrador
        </td>
      </tr>
    </tbody>
  </table>
</div>

{editando && (
  <div className="overlay" onClick={() => setEditando(null)}>
    <div className="modal" style={{ maxWidth: 420, textAlign: "left" }}
      onClick={(e) => e.stopPropagation()}
      role="dialog" aria-modal="true" aria-labelledby="titulo-editar-usuario">
      <h2 id="titulo-editar-usuario" style={{ textTransform: "capitalize" }}>
        Editar a {editando.nombre}
      </h2>
      <label className="pista" htmlFor="campo-correo-editar">Correo</label>
      <input id="campo-correo-editar" value={editando.correo || ""}
        onChange={(e) => setEditando({ ...editando, correo: e.target.value })}
        style={{ width: "100%", padding: "10px 12px", marginTop: 6, marginBottom: 14,
          border: "1px solid var(--borde)", borderRadius: 10 }} />
      <label className="pista" htmlFor="campo-perfil-editar">Perfil</label>
    </div>
  </div>
)

```



```

    <select id="campo-perfil-editar" value={editando.perfil}
      onChange={(e) => setEditando({ ...editando, perfil: e.target.value })}
      style={{ width: "100%", padding: "10px 12px", marginTop: 6,
        border: "1px solid var(--borde)", borderRadius: 10 }}
      disabled={editando.id === usuario?.id}>
      <option value="auxiliar">Auxiliar</option>
      <option value="auditor">Administrador</option>
    </select>
    {editando.id === usuario?.id && (
      <p className="pista" style={{ marginTop: 6 }}>
        No puede cambiar su propio rol desde aquí.
      </p>
    )}
    <div className="botones" style={{ marginTop: 18 }}>
      <button className="btn borde" onClick={() => setEditando(null)}>Cancelar</button>
      <button className="btn" onClick={guardarEdicion}>Guardar</button>
    </div>
  </div>
</div>
)}

{asignando && (
  <div className="overlay" onClick={() => setAsignando(null)}>
    <div className="modal" style={{ maxWidth: 520, textAlign: "left" }}
      onClick={(e) => e.stopPropagation()}
      role="dialog" aria-modal="true" aria-labelledby="titulo-asignar-bodegas">
      <h2 id="titulo-asignar-bodegas" style={{ textTransform: "capitalize" }}>
        Bodegas para {asignando.nombre}
      </h2>
      <p className="pista">
        {asignando.perfil === "auditor"
          ? "Marque las bodegas de las que esta persona es responsable como administradora
(recuento ciego, cierre)."
          : "Marque las bodegas que va a poder contar esta persona."}
      </p>
      <div style={{ maxHeight: 320, overflowY: "auto", margin: "12px 0",
        border: "1px solid var(--borde)", borderRadius: 10, padding: 8 }}>
        {bodegas.map((b) => (
          <label key={b.id} style={{ display: "flex", alignItems: "center", gap: 10,
            padding: "7px 6px", fontSize: ".9rem" }}>
            <input type="checkbox" checked={marcadas.has(b.id)}
              onChange={() => alternarBodega(b.id)} />
            {b.bodega}
          </label>
        ))}
      </div>
      <p className="pista">{marcadas.size} bodegas marcadas</p>
      <div className="botones">
        <button className="btn borde" onClick={() => setAsignando(null)}>Cancelar</button>
        <button className="btn" onClick={guardarAsignacion}>Guardar</button>
      </div>
    </div>
  </div>
)}
</div>
);
}

function TabRecetas({ token, recetaInicial }) {
  const [recetas, setRecetas] = useState([]);
  const [catalogo, setCatalogo] = useState([]);
  const [msg, setMsg] = useState("");
  const [editando, setEditando] = useState(null);
  // { id: null|number, nombre, rendimiento, lineas: [{articulo_codigo, cantidad_por_porcion}] }
  const abrioInicial = useState(false);

```

```

function cargar() {
  pedir("/api/recetas", {}, token).then(setRecetas).catch(() => {});
  pedir("/api/articulos/catalogo-ligero", {}, token).then(setCatalogo).catch(() => {});
}
useEffect(cargar, [token]);

useEffect(() => {
  window.addEventListener("cuentavoz:ajustes-actualizados", cargar);
  return () => window.removeEventListener("cuentavoz:ajustes-actualizados", cargar);
}, [token]);

useEffect(() => {
  if (recetaInicial && recetas.length && !abrioInicial[0]) {
    abrioInicial[1](true);
    abrir(recetaInicial);
  }
}, [recetaInicial, recetas]);

// Mismo caso que en TabUsuarios: este modal esta armado a mano, no con
// el componente Dialogo compartido, asi que necesita su propio Escape.
useEffect(() => {
  function alTecla(e) { if (e.key === "Escape" && editando) setEditando(null); }
  window.addEventListener("keydown", alTecla);
  return () => window.removeEventListener("keydown", alTecla);
}, [editando]);

async function abrir(id) {
  try {
    const r = await pedir(`/api/recetas/${id}`, {}, token);
    setEditando({
      id: r.id, nombre: r.nombre, rendimiento: r.rendimiento,
      preparacion: r.preparacion || "",
      lineas: r.lineas.map((l) => ({ articulo_codigo: l.articulo_codigo,
                                cantidad_por_porcion: l.cantidad_por_porcion })),
    });
  } catch (e) { setMsg(e.message); }
}

function nueva() {
  setEditando({ id: null, nombre: "", rendimiento: 1, preparacion: "",
    lineas: [{ articulo_codigo: "", cantidad_por_porcion: "" }] });
}

function actualizarLinea(i, campo, valor) {
  setEditando((e) => {
    const lineas = e.lineas.map((l, idx) => idx === i ? { ...l, [campo]: valor } : l);
    return { ...e, lineas };
  });
}

function agregarLinea() {
  setEditando((e) => ({ ...e, lineas: [...e.lineas, { articulo_codigo: "", cantidad_por_porcion: ""
}] }));
}

function quitarLinea(i) {
  setEditando((e) => ({ ...e, lineas: e.lineas.filter((_, idx) => idx !== i) }));
}

async function guardar() {
  const cuerpo = {
    nombre: editando.nombre.trim(),
    rendimiento: Number(editando.rendimiento) || 1,
  }

```

```

    preparacion: (editando.preparacion || "").trim(),
    ingredientes: editando.lineas
      .filter((l) => l.articulo_codigo && l.cantidad_por_porcion)
      .map((l) => ({ articulo_codigo: l.articulo_codigo,
        cantidad_por_porcion: Number(l.cantidad_por_porcion) })),
  });
  if (!cuerpo.nombre) { setMsg("Póngale un nombre a la receta."); return; }
  try {
    if (editando.id) {
      await pedir(`/api/recetas/${editando.id}`, { method: "PUT", body: JSON.stringify(cuerpo) },
token);
      setMsg(`Receta «${cuerpo.nombre}» actualizada.`);
    } else {
      await pedir("/api/recetas", { method: "POST", body: JSON.stringify(cuerpo) }, token);
      setMsg(`Receta «${cuerpo.nombre}» creada.`);
    }
    setEditando(null);
    cargar();
  } catch (e) { setMsg(e.message); }
}

async function eliminar(r) {
  if (!window.confirm(`¿Eliminar la receta «${r.nombre}»? Esta acción no se puede deshacer.`))
return;
  try {
    await pedir(`/api/recetas/${r.id}`, { method: "DELETE" }, token);
    setMsg(`Receta «${r.nombre}» eliminada.`);
    cargar();
  } catch (e) { setMsg(e.message); }
}

return (
  <div className="card">
    <h3>
      <span className="icono-kpi" style={{ marginRight: 8 }}>👨🍳</span>
      Recetas ({recetas.length})
    </h3>
    <p className="pista">
      Las porciones que dicte el chef se calculan sobre estas recetas: cantidad por
      porción × porciones, menos lo que ya haya en la bodega. Aquí se crean, editan
      y borran – CuentaVoz sí gestiona el catálogo de recetas y menús.
    </p>
    {msg && <p className="msg-ok">{msg}</p>}
    {recetas.length === 0 ? (
      <p className="vacio">Todavía no hay recetas creadas.</p>
    ) : (
      <table>
        <thead><tr><th>Receta</th><th>Rendimiento</th><th>Ingredientes</th><th></th></tr></thead>
        <tbody>
          {recetas.map((r) => (
            <tr key={r.id}>
              <td style={{ textTransform: "capitalize" }}>{r.nombre}</td>
              <td>{r.rendimiento} porción</td>
              <td>{r.ingredientes}</td>
              <td style={{ display: "flex", gap: 6 }}>
                <button className="btn borde" style={{ padding: "4px 10px", minHeight: 0 }}
                  onClick={() => abrir(r.id)}>Editar</button>
                <button className="btn gris" style={{ padding: "4px 10px", minHeight: 0 }}
                  onClick={() => eliminar(r)}>Eliminar</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
  </div>
  <div className="grilla-botones" style={{ marginTop: 14 }}>

```

```

    <button className="btn" onClick={nueva}>+ Nueva receta</button>
  </div>

  {editando && (
    <div className="overlay" onClick={() => setEditando(null)}>
      <div className="modal" style={{ maxWidth: 660, textAlign: "left", maxHeight: "88vh",
        overflowY: "auto" }}
        onClick={(e) => e.stopPropagation()}
        role="dialog" aria-modal="true" aria-labelledby="titulo-editar-receta">
        <h2 id="titulo-editar-receta">
          {editando.id ? `Editar ${editando.nombre}` : "Nueva receta"}
        </h2>
        <p className="pista" style={{ marginTop: -6, marginBottom: 18 }}>
          Cada campo queda disponible de inmediato para calcular pedidos por voz.
        </p>

        <div style={{ display: "grid", gridTemplateColumns: "2fr 1fr", gap: 14, marginBottom: 20
        }}>
          <div>
            <label className="pista" htmlFor="campo-nombre-plato" style={{ display: "block",
marginBottom: 6 }}>
              Nombre del plato
            </label>
            <input id="campo-nombre-plato" value={editando.nombre}
              onChange={(e) => setEditando({ ...editando, nombre: e.target.value })}
              placeholder="ej. ajiaco"
              style={{ width: "100%", padding: "10px 12px",
                border: "1px solid var(--borde)", borderRadius: 10 }} />
          </div>
          <div>
            <label className="pista" htmlFor="campo-rendimiento" style={{ display: "block",
marginBottom: 6 }}>
              Rendimiento (porciones)
            </label>
            <input id="campo-rendimiento" type="number" min="1" value={editando.rendimiento}
              onChange={(e) => setEditando({ ...editando, rendimiento: e.target.value })}
              style={{ width: "100%", padding: "10px 12px",
                border: "1px solid var(--borde)", borderRadius: 10 }} />
          </div>
        </div>

        <div style={{ background: "var(--fondo)", borderRadius: 12, padding: 16, marginBottom: 18
        }}>
          <h3 style={{ marginTop: 0, marginBottom: 4, fontSize: ".95rem", color: "var(--azul)" }}>
            🍽️ Ingredientes
          </h3>
          <p className="pista" style={{ marginBottom: 12 }}>Cantidad por cada porción de la
receta.</p>
          <div style={{ maxHeight: 280, overflowY: "auto" }}>
            {editando.lineas.map((l, i) => (
              <div key={i} style={{ display: "flex", gap: 8, alignItems: "center", marginBottom: 8
              }}>
                <select value={l.articulo_codigo}
                  onChange={(e) => actualizarLinea(i, "articulo_codigo", e.target.value)}
                  aria-label={`Artículo del ingrediente ${i + 1}`}
                  style={{ flex: 1, padding: "8px 10px", border: "1px solid var(--borde)",
                    borderRadius: 8, background: "#fff" }}>
                  <option value="">- elija un artículo del catálogo -</option>
                  {catalogo.map((a) => (
                    <option key={a.codigo} value={a.codigo}>{a.nombre} {a.unidad}</option>
                  ))}
                </select>
                <input type="number" step="0.001" min="0" placeholder="cantidad"
                  value={l.cantidad_por_porcion}
                  onChange={(e) => actualizarLinea(i, "cantidad_por_porcion",
e.target.value)}

```

```

        aria-label={`Cantidad por porción del ingrediente ${i + 1}`}
        style={{ width: 100, padding: "8px 10px", border: "1px solid var(--borde)",
            borderRadius: 8, background: "#fff" }} />
        <button className="btn gris" style={{ padding: "6px 10px", minHeight: 0, flex:
"none" }}
            onClick={() => quitarLinea(i)} aria-label={`Quitar ingrediente ${i +
1}`}>X</button>
        </div>
    )}
</div>
<button className="btn borde" style={{ marginTop: 4 }} onClick={agregarLinea}>
    + Agregar ingrediente
</button>
</div>

<div style={{ background: "var(--fondo)", borderRadius: 12, padding: 16, marginBottom: 4
}}>
    <h3 style={{ marginTop: 0, marginBottom: 4, fontSize: ".95rem", color: "var(--azul)" }}>
        📋 Preparación paso a paso
    </h3>
    <p className="pista" style={{ marginBottom: 12 }}>
        Opcional: lo que ve el auxiliar al consultar esta receta desde Pedidos.
    </p>
    <textarea value={editando.preparacion}
        onChange={(e) => setEditando({ ...editando, preparacion: e.target.value })}
        placeholder="1. Pele y pique las papas...&#10;2. Sofría la cebolla...&#10;3.
...
        aria-label="Preparación paso a paso, opcional"
        rows={5}
        style={{ width: "100%", padding: "10px 12px",
            border: "1px solid var(--borde)", borderRadius: 10,
            fontFamily: "inherit", resize: "vertical", background: "#fff" }} />
    </div>

    <div className="botones" style={{ marginTop: 20 }}>
        <button className="btn borde" onClick={() => setEditando(null)}>Cancelar</button>
        <button className="btn" onClick={guardar}>Guardar receta</button>
    </div>
</div>
</div>
    )}
</div>
);
}

function TabTraza({ token }) {
    const [traza, setTraza] = useState([]);
    const [persona, setPersona] = useState("");
    const [accion, setAccion] = useState("");
    const [rango, setRango] = useState("hoy");
    const [msg, setMsg] = useState("");
    const [todasPersonas, setTodasPersonas] = useState([]);
    const [escuchandoFiltro, setEscuchandoFiltro] = useState(false);

    function cargar() {
        const q = new URLSearchParams();
        if (persona) q.set("persona", persona);
        if (accion) q.set("accion", accion);
        if (rango) q.set("rango", rango);
        pedir(`/api/trazabilidad?${q}`, {}, token).then(setTraza).catch(() => {});
    }
    useEffect(cargar, [token, persona, accion, rango]);
    // Lista COMPLETA de personas (no solo las que aparecen en el filtro
    // actual) para que el filtro por voz reconozca un nombre aunque el
    // rango/persona ya aplicado lo haya dejado fuera de "traza".

```

```

useEffect(() => {
  pedir("/api/usuarios", {}, token).then((us) => setTodasPersonas(us.map((x) => x.nombre)))
    .catch(() => {});
}, [token]);

const acciones = ["", "INGRESO", "APERTURA", "CONTEO", "CORRECCION", "CREACION",
  "FIRMA", "AUDITORIA", "CIERRE", "REAPERTURA", "APROBACION", "ALERTA",
  "REPORTE", "PEDIDO", "RECETA", "LEGALIZACION", "AJUSTE", "USUARIO",
  "ASIGNACION", "SEGURIDAD", "PERFIL", "SOPORTE"];
const personas = [...new Set(traza.map((t) => t.persona))].sort();

// "Muéstreme las acciones de Luis hoy" - determinístico y local, sin
// pasar por el agente general: solo reconoce palabras contra las
// listas que YA existen en esta pantalla (rango, tipos de acción,
// personas), así que no hace falta ninguna llamada a Gemini para
// filtrar. Combina lo dicho con lo que YA estaba marcado (decir solo
// "esta semana" no borra la persona/acción que ya tenía puesta) -
// por eso lee rango/persona/accion actuales como punto de partida en
// vez de siempre resetear a "todos".
function _resolverFiltroVoz(texto) {
  const t = quitarTildes(texto.toLowerCase());
  let coincidio = false;
  let nuevoRango = rango, nuevaAccion = accion, nuevaPersona = persona;
  if (/^btodo\b/.test(t)) { nuevoRango = ""; coincidio = true; }
  else if (/^bhoy\b/.test(t)) { nuevoRango = "hoy"; coincidio = true; }
  else if (/^semana\b/.test(t)) { nuevoRango = "semana"; coincidio = true; }
  else if (/^bmes\b/.test(t)) { nuevoRango = "mes"; coincidio = true; }

  if (/todas las acciones/.test(t)) { nuevaAccion = ""; coincidio = true; }
  else {
    const accionEncontrada = acciones.find((a) => a && t.includes(a.toLowerCase()));
    if (accionEncontrada) { nuevaAccion = accionEncontrada; coincidio = true; }
  }

  if (/todas las personas/.test(t)) { nuevaPersona = ""; coincidio = true; }
  else {
    const personaEncontrada = todasPersonas.find(
      (p) => p && t.includes(quitarTildes(p.toLowerCase())));
    if (personaEncontrada) { nuevaPersona = personaEncontrada; coincidio = true; }
  }
  return coincidio ? { rango: nuevoRango, persona: nuevaPersona, accion: nuevaAccion } : null;
}

// Aplica el filtro resuelto Y dice el resultado real (cuántas filas
// encontró), no un "listo, apliqué el filtro" que no cuenta nada -
// pedido explícito del usuario. Trae los datos de una (en vez de
// esperar a que el useEffect de cargar() reaccione al cambio de
// estado) para poder anunciar la cifra exacta apenas llega.
async function _aplicarFiltroYAnunciar(resuelto) {
  const { rango: r, persona: p, accion: a } = resuelto;
  setRango(r); setPersona(p); setAccion(a);
  setMsg("");
  try {
    const q = new URLSearchParams();
    if (p) q.set("persona", p);
    if (a) q.set("accion", a);
    if (r) q.set("rango", r);
    const filas = await pedir(`/api/trazabilidad?${q}`, {}, token);
    setTraza(filas);
    const partes = [];
    if (a) partes.push(`de ${a.toLowerCase()}`);
    if (p) partes.push(`de ${p}`);
    const etiquetaRango = r === "hoy" ? "hoy" : r === "semana" ? "en la última semana"
      : r === "mes" ? "en el último mes" : "en total";
  }

```

```

    const texto = `Encontré ${filas.length} acciones${partes.length ? " " + partes.join(" ") : ""}
    ${etiquetaRango}.`;
    setMsg(texto);
    hablar(texto);
  } catch (e) { setMsg(e.message); }
}

// El cuadro "Pregúntele al agente" (arriba de las pestañas) es el
// MISMO cuadro que muestra los ejemplos «acciones de Luis hoy» - una
// persona que escribe/dice esa frase ahí espera que funcione, no
// solo desde el micrófono chiquito junto a los filtros. Como el
// filtro ya vive aquí (sin pasar por el backend), se intercepta la
// frase ANTES de que AsistenteVoz llegue a preguntarle al agente -
// event.preventDefault() debe llamarse de una, ANTES de cualquier
// await, para que AsistenteVoz vea la señal en el mismo ciclo
// síncrono del dispatchEvent (el fetch+anuncio siguen aparte, ya de
// forma asíncrona, sin que eso afecte esa señal).
useEffect(() => {
  const interceptar = (e) => {
    const resuelto = _resolverFiltroVoz(e.detail.texto);
    if (!resuelto) return;
    e.preventDefault();
    _aplicarFiltroYAnunciar(resuelto);
  };
  window.addEventListener("cuentavoz:filtro-local", interceptar);
  return () => window.removeEventListener("cuentavoz:filtro-local", interceptar);
}, [todasPersonas, acciones, token, rango, persona, accion]);

function escucharFiltro() {
  setMsg("");
  escuchar({
    alTexto: (texto) => {
      const resuelto = _resolverFiltroVoz(texto);
      if (resuelto) _aplicarFiltroYAnunciar(resuelto);
      else setMsg("No reconocí ningún filtro en lo que dijo.");
    },
    alEstado: (e) => setEscuchandoFiltro(e === "escuchando"),
    alError: setMsg,
  });
}

async function exportar() {
  setMsg("");
  try {
    const q = new URLSearchParams();
    if (persona) q.set("persona", persona);
    if (accion) q.set("accion", accion);
    if (rango) q.set("rango", rango);
    const d = await pedir(`/api/trazabilidad/exportar?${q}`, { method: "GET" }, token);
    await descargarReporte(d.archivo, token);
    setMsg(`Exportado: ${d.filas} filas.`);
  } catch (e) { setMsg(e.message); }
}

// "exporta el registro de trazabilidad" por voz - mismo botón
// «Exportar», con los filtros que ya estén puestos en pantalla (el
// agente no los conoce, viven aquí como estado de React).
useEffect(() => {
  window.addEventListener("cuentavoz:accion:exportar_trazabilidad", exportar);
  return () => window.removeEventListener("cuentavoz:accion:exportar_trazabilidad", exportar);
}, [token, persona, accion, rango]);

return (
  <div className="card">
    <h3><span className="icono-kpi" style={{ marginRight: 8 }}>⊕</span>

```

```

    Registro de trazabilidad ({traza.length} acciones)</h3>
    <div style={{ display: "flex", gap: 10, marginBottom: 12, flexWrap: "wrap", alignItems: "center"
  >>>
      <select value={rango} onChange={(e) => setRango(e.target.value)}
        aria-label="Rango de fechas del registro de trazabilidad"
        style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
        <option value="hoy">Hoy</option>
        <option value="semana">Última semana</option>
        <option value="mes">Último mes</option>
        <option value="">Todo</option>
      </select>
      <select value={persona} onChange={(e) => setPersona(e.target.value)}
        aria-label="Filtrar por persona"
        style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
        <option value="">Todas las personas</option>
        {personas.map((p) => <option key={p} value={p} style={{ textTransform: "capitalize"
  >>>{p}</option>}}
      </select>
      <select value={accion} onChange={(e) => setAccion(e.target.value)}
        aria-label="Filtrar por tipo de acción"
        style={{ padding: "9px 12px", border: "1px solid var(--borde)", borderRadius: 10 }}>
        {acciones.map((a) => <option key={a} value={a}>{a} || "Todas las acciones"</option>)}
      </select>
      <button className={`mic-btn ${escuchandoFiltro ? "escuchando" : ""}`}
        style={{ width: 46, height: 46, fontSize: "1.2rem" }}
        onClick={escucharFiltro} title="filtrar por voz: «acciones de Luis hoy»"
        aria-label="Filtrar el registro de trazabilidad por voz">
        <Icono nombre="microfono" tam={20} /></button>
      <button className="btn verde"
        style={{ marginLeft: "auto", display: "inline-flex", alignItems: "center", gap: 8 }}
        onClick={exportar}>
        <Icono nombre="descargar" tam={18} />
        Exportar
      </button>
    </div>
    {msg && (
      <div className={`banner ${msg.startsWith("Encontré") || msg.startsWith("Exportado")} ? "ok" :
  ""}`}
        style={{ marginBottom: 12 }}>
        <span className="ico">
          {msg.startsWith("Encontré") || msg.startsWith("Exportado")} ? "✓" : "!"
        </span>
        <span>{msg}</span>
      </div>
    )}
    {traza.length === 0 ? (
      <p className="vacio">Sin acciones registradas todavía.</p>
    ) : (
      <table>
        <thead><tr><th>Hora</th><th>Persona</th><th>Acción</th><th>Detalle</th></tr></thead>
        <tbody>
          {traza.slice(0, 25).map((t) => (
            <tr key={t.id}>
              <td>{t.hora}</td>
              <td style={{ textTransform: "capitalize" }}>{t.persona}</td>
              <td><span className={`chip ${t.tipo === "alerta" ? "oro"
                : t.tipo === "ok" ? "verde" : ""}`}>{t.accion}</span></td>
              <td>{t.detalle}</td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
    <p className="pista" style={{ marginTop: 10 }}>
      Cumple la Ley 1581 de 2012: se registra la acción, no datos personales sensibles.
      El registro no se puede editar ni borrar.
    </p>

```



```

<div className="card" style={{ marginTop: 14, background: "var(--azul-claro)" }}>
  <h3>Por qué el registro es inmutable</h3>
  <p style={{ fontSize: ".87rem" }}>
    Una corrección no borra el valor anterior: crea un registro nuevo que apunta
    al original (campo <code>corrige_a</code> de la tabla Conteo). Así, si alguien
    pregunta «¿quién cambió este dato y por qué?», la respuesta está en el
    sistema y no en la memoria de nadie.
  </p>
</div>
</div>
);
}

```

12.33 Frontend: Ayuda y Mensajes (vistas 5.27 y 5.27a)

Archivo: frontend/src/vistas/Ayuda.jsx

```

import { useEffect, useState, useRef } from "react";
import { pedir, preguntarAsistente } from "../api";
import { escuchar, hablar, quitarTildes } from "../voz";
import { enviarPorEmailJS, capitalizar, rolDe } from "../correoAdmin";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import Icono from "../Iconos";

const FAQ = [
  ["¿Cómo corrijo un conteo ya confirmado?",
    "Diga «corregir» y luego el valor correcto. El valor anterior se conserva."],
  ["El agente no me entiende un producto",
    "Dígalo como aparece en la etiqueta. Si no existe, se puede crear: queda pendiente."],
  ["¿Qué hago si sale una alerta?",
    "Recuente. Si el número es correcto, confirme: queda marcado para el administrador."],
  ["Se cayó el internet en plena bodega",
    "Siga contando: el intérprete local mantiene el flujo y sincroniza al volver la señal."],
  ["¿Puedo contar dos bodegas a la vez?",
    "No en el mismo dispositivo. El candado de sesión evita conteos duplicados."],
];

const COMANDOS = [
  ["«Iniciar conteo en almacén de suministros»", "abrir una bodega"],
  ["«Tres tablas para picar blancas»", "registrar un conteo"],
  ["«Corregir» · «Son nueve»", "corregir sin borrar el original"],
  ["«¿Cuánto arroz hay y en qué bodegas?»", "consultar el inventario"],
  ["«Hoy preparamos cincuenta ajiacos»", "pedir insumos por receta"],
];

export default function Ayuda({ token, usuario, ir }) {
  const [salud, setSalud] = useState(null);
  const [busca, setBusca] = useState("");
  const [reportando, setReportando] = useState(false);
  const [msg, setMsg] = useState("");
  const [pedirDetalle, setPedirDetalle] = useState(false);
  const [escribiendoAdmin, setEscribiendoAdmin] = useState(false);
  const [enviandoAdmin, setEnviandoAdmin] = useState(false);
  const [admin, setAdmin] = useState(null);
  const [miPerfil, setMiPerfil] = useState(null);
  // Si lo escrito/dicho no encuentra nada en las preguntas frecuentes ni
  // en la guía de comandos, la respuesta del agente general - mismo
  // patrón que el buscador de Bodegas: un solo cuadro para todo, en vez
  // de dos cuadros separados (uno para el agente, otro para filtrar)
  // que además se veían mal juntos, uno encima del otro.
  const [respuestaAgente, setRespuestaAgente] = useState(null);
  const [escuchando, setEscuchando] = useState(false);
  const [errorBusca, setErrorBusca] = useState("");
  const rec = useRef(null);

```

```

const idEscucha = useRef(0);

useEffect(() => {
  pedir("/api/salud", {}, token).then(setSalud).catch(() => setSalud({ api: "caido" }));
  pedir("/api/soporte/administrador", {}, token).then(setAdmin).catch(() => {});
  // Para la firma del correo a el administrador (nombre, rol y correo
  // de quien escribe) - "usuario" (la sesión) solo trae id/nombre/perfil.
  pedir("/api/usuarios/yo", {}, token).then(setMiPerfil).catch(() => {});
}, [token]);

async function reportarProblema(detalle) {
  setPedirDetalle(false);
  if (!detalle || !detalle.trim()) return;
  setReportando(true);
  try {
    await pedir("/api/soporte/reportar", {
      method: "POST", body: JSON.stringify({ detalle })
    }, token);
    setMsg("Reportado a la mesa de ayuda. Quedó en el registro de trazabilidad.");
  } catch (e) { setMsg(e.message); }
  setReportando(false);
}

// Reemplaza el enlace mailto: de antes - dependía de que el equipo
// tuviera un cliente de correo instalado, y sin uno mostraba el cuadro
// genérico del sistema operativo ("seleccionar una aplicación para
// abrir este vínculo") en vez de algo propio de CuentaVoz. Este envío
// queda en el registro de trazabilidad, con el mismo aviso honesto que
// ya usa "Reportar un problema": no promete un correo real que la app
// no puede garantizar.
async function enviarMensajeAdministrador(mensaje) {
  setEscribiendoAdmin(false);
  if (!mensaje || !mensaje.trim()) return;
  setEnviandoAdmin(true);
  try {
    const r = await pedir("/api/soporte/mensaje-administrador", {
      method: "POST", body: JSON.stringify({ mensaje })
    }, token);
    const nombre = capitalizar(admin?.nombre) || "el administrador";
    const rol = rolDe(usuario);
    let listo;
    if (r?.correo_enviado) {
      listo = `Correo enviado a ${nombre}.`;
    } else if (admin?.correo &&
      await enviarPorEmailJS(admin.correo, capitalizar(usuario?.nombre) || "Usuario",
        mensaje, rol, miPerfil?.correo)) {
      listo = `Correo enviado a ${nombre}.`;
    } else if (admin?.correo) {
      // Último respaldo si EmailJS tampoco responde: abre el correo ya
      // instalado en el dispositivo con todo listo.
      const asunto = encodeURIComponent("Mensaje desde CuentaVoz");
      const cuerpo = encodeURIComponent(`De: ${usuario?.nombre} || "usted"\n\n${mensaje}`);
      window.location.href = `mailto:${admin.correo}?subject=${asunto}&body=${cuerpo}`;
      listo = `Mensaje registrado. Se abrió su correo para enviarlo a ${nombre}.`;
    } else {
      listo = "Mensaje registrado.";
    }
    setMsg(listo);
    hablar(listo);
  } catch (e) { setMsg(e.message); }
  setEnviandoAdmin(false);
}

function _hayCoincidenciaLocal(texto) {
  const t = texto.trim().toLowerCase();
  return FAQ.some(([p, r]) => p.toLowerCase().includes(t) || r.toLowerCase().includes(t))
}

```

```

    || COMANDOS.some(([c, d]) => c.toLowerCase().includes(t) || d.toLowerCase().includes(t));
  }

function alEscribir(texto) {
  setBusca(texto);
  setRespuestaAgente(null);
}

// Los botones de "Soporte en vivo" ("Escribirle al administrador",
// "Reportar un problema") también son órdenes reconocibles - se
// revisan antes que cualquier otra cosa, porque el agente general no
// sabe qué botones hay en esta pantalla en particular (por eso decía
// "no puedo enviarle mensajes... esa función no está disponible", aun
// cuando el botón para hacerlo está ahí mismo, a la vista).
function _accionLocalDeAyuda(texto) {
  const t = quitarTildes(texto.toLowerCase()).replace(/[.,;:!?;?i]/g, "").trim();
  // "\bdescrib" (sin \b de cierre) para que agarre CUALQUIER conjugación
  // - "escribir", "escribale", y sobre todo "escribirle" (como dice el
  // botón mismo: "Escribirle al administrador") - un \b\escribir\b no
  // hacía match dentro de "escribirle" porque ahí "escribir" no es una
  // palabra completa, es solo el principio de una más larga.
  if (/^\badministrador\b/.test(t) && /\b(escrib\w*|contactar|mensaje)\b/.test(t)) return "escribir";
  if (/^\bproblema\b|\berror\b/.test(t) && /\breport\w*\/.test(t)) return "reportar";
  return null;
}

// Al confirmar (Enter, o al terminar de dictar): primero las órdenes
// sobre los botones de esta pantalla; si no aplica ninguna y lo dicho
// no encuentra nada en las preguntas frecuentes ni en la guía de
// comandos, cae al agente general - un solo cuadro para todo.
async function confirmarBusqueda(texto, miId) {
  if (!texto || !texto.trim()) return;
  if (miId === undefined) miId = ++idEscucha.current;
  setBusca(texto);
  const accion = _accionLocalDeAyuda(texto);
  if (accion === "escribir" && admin) { setEscribiendoAdmin(true); return; }
  if (accion === "reportar") { setPedirDetalle(true); return; }
  if (_hayCoincidenciaLocal(texto)) { setRespuestaAgente(null); return; }
  setErrorBusca("");
  const r = await preguntarAsistente(texto, "ayuda", token);
  if (miId !== idEscucha.current) return;
  setRespuestaAgente(r);
  hablar(r.respuesta_hablada);
  if (r.accion === "navegar" && r.destino && ir) {
    ir(r.destino, { tabInicial: r.pestana || undefined });
  }
}

function buscarPorVoz() {
  setErrorBusca("");
  const miId = ++idEscucha.current;
  rec.current = escuchar({
    alTexto: (t) => {
      if (miId !== idEscucha.current) return;
      confirmarBusqueda(t, miId);
    },
    alEstado: (e) => setEscuchando(e === "escuchando"),
    alError: setErrorBusca,
  });
}

const q = busca.trim().toLowerCase();
const faqFiltrado = q ? FAQ.filter(([p, r]) =>
  p.toLowerCase().includes(q) || r.toLowerCase().includes(q)) : FAQ;
const comandosFiltrados = q ? COMANDOS.filter(([c, d]) =>

```

```

c.toLowerCase().includes(q) || d.toLowerCase().includes(q)) : COMANDOS;

const servicios = [
  ["API y base de datos", salud?.api === "ok"],
  ["Datos del inventario cargados", (salud?.stock || 0) > 0],
  ["Agente Gemini (AI Studio)", Boolean(salud?.gemini)],
];

return (
  <Marco titulo="Ayuda · cómo usar CuentaVoz" chip={{ texto: "SOPORTE", tipo: "azul" }}>
    <div className="card">
      <p className="rotulo">Pregúntele al agente</p>
      <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
        <input value={busca} onChange={(e) => alEscribir(e.target.value)}
          onKeyDown={(e) => e.key === "Enter" && confirmarBusqueda(busca)}
          placeholder="pregúntele al agente, o escriba para filtrar las preguntas frecuentes..."
          aria-label="Pregúntele al agente, o escriba para filtrar las preguntas frecuentes"
          style={{ flex: 1, padding: "12px 14px",
            border: "1px solid var(--borde)", borderRadius: 12 }} />
        <button className={mic-btn ${escuchando ? "escuchando" : ""}}
          style={{ width: 46, height: 46, fontSize: "1.2rem" }}
          onClick={buscarPorVoz} title="o pregunte por voz"
          aria-label="Pregúntele al agente por voz"><Icono nombre="microfono" tam={22}
        /></button>
      </div>
      <p className="pista" style={{ marginTop: 8 }}>o pregunte por voz</p>
      {respuestaAgente && (
        <>
          <p className="rotulo" style={{ marginTop: 10 }}>CuentaVoz responde</p>
          <p className="burbuja" aria-live="polite" aria-atomic="true">{respuestaAgente.respuesta_hablada}</p>
        </>
      )}
      {errorBusca && <p className="error" style={{ marginTop: 8 }}>{errorBusca}</p>}
    </div>
    <div className="conteo-cols">
      <div className="card">
        <h3><span className="icono-kpi" style={{ marginRight: 8 }}> ? </span> Preguntas
        frecuentes</h3>
        {faqFiltrado.length === 0 && <p className="vacio">Sin resultados para «{busca}».</p>}
        {faqFiltrado.map(([p, r], i) => (
          <div key={i} style={{ marginBottom: 12, paddingBottom: 12,
            borderBottom: i < faqFiltrado.length - 1 ? "1px solid var(--borde)" : "none" }}>
            <b style={{ color: "var(--azul)", fontSize: ".92rem" }}>{p}</b>
            <p style={{ fontSize: ".86rem", color: "var(--grafito)", marginTop: 3 }}>{r}</p>
          </div>
        ))}

        <h3 style={{ marginTop: 18 }}>
          <span className="icono-kpi" style={{ marginRight: 8 }}> 🗨️ </span>
          Guía rápida de comandos de voz
        </h3>
        {comandosFiltrados.length === 0 && <p className="vacio">Sin resultados para «{busca}».</p>}
        {comandosFiltrados.map(([c, d], i) => (
          <div className="registro" key={i}>
            <span style={{ color: "var(--azul)", fontWeight: 700 }}>{c}</span>
            <span className="cant">{d}</span>
          </div>
        ))}
      </div>

      <div>
        <div className="card">
          <h3><span className="icono-kpi" style={{ marginRight: 8 }}> 🗨️ </span> Soporte en vivo</h3>
          {admin?.nombre ? (

```

```

<>
  <p style={{ fontSize: ".88rem" }}>
    Administrador de bodega · <span style={{ textTransform: "capitalize"
  }}>{admin.nombre}</span>
    <span style={{ color: "var(--verde)", fontWeight: 700 }}> · disponible</span>
  </p>
  <div className="grilla-botones">
    <button className="btn" disabled={enviandoAdmin}
      onClick={() => setEscribiendoAdmin(true)}
      style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
      <Icono nombre="mensajes" tam={18} />
      {enviandoAdmin ? "Enviando..." : "Escribirle al administrador"}
    </button>
  </div>
</>
) : admin?.es_usted ? (
  <p style={{ fontSize: ".88rem" }}>
    Usted es la administradora de turno: los auxiliares le escriben a
    usted. Para algo que se salga de su alcance, use la mesa de ayuda
    de Colsubsidio.
  </p>
) : (
  <p style={{ fontSize: ".88rem" }}>
    No hay otro administrador registrado por ahora. Use la mesa de
    ayuda de Colsubsidio.
  </p>
)
{msg && <p className="msg-ok">{msg}</p>}
<p className="pista" style={{ margin: "10px 0" }}>
  Mesa de ayuda Colsubsidio · ext. 4040 · 7 por 24
</p>
<div className="grilla-botones">
  {(admin?.nombre || admin?.es_usted) && (
    <button className="btn borde" onClick={() => ir("mensajes")}
      style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
      <Icono nombre="mensajes" tam={18} />
      Ver mensajes
    </button>
  )}
  <button className="btn oro" disabled={reportando} onClick={() => setPedirDetalle(true)}
    style={{ display: "inline-flex", alignItems: "center", gap: 8 }}>
    <Icono nombre="advertencia" tam={18} />
    {reportando ? "Enviando..." : "Reportar un problema"}
  </button>
</div>
</div>

<div className="card" style={{ marginTop: 16 }}>
  <h3><span className="icono-kpi" style={{ marginRight: 8 }}>📌</span>Estado del
sistema</h3>
  <div className="kpi" style={{ gridTemplateColumns: "1fr" }}>
    {servicios.map([t, ok], i) => (
      <div className={ok ? "kpi verde" : "kpi"} key={i}
        style={{ display: "flex", alignItems: "center", justifyContent: "space-between",
          padding: "10px 16px" }}>
        <div className="kpi-cabeza" style={{ marginBottom: 0 }}>
          <span className="icono-kpi" style={{
            background: ok ? "var(--verde-bg)" : "#FBE8E6",
            color: ok ? "var(--verde)" : "#B3261E" }}>
            {ok ? "✓" : "!"}
          </span>
          <small>{t}</small>
        </div>
        <b style={{ fontSize: ".92rem", color: ok ? "var(--verde)" : "#B3261E" }}>
          {ok ? "operativo" : "revisar"}
        </b>
      </div>
    )}
  </div>

```

```

    ))}
  </div>
  {!salud?.gemini && (
    <p className="pista" style={{ marginTop: 10 }}>
      Sin la llave de Gemini el agente usa el intérprete local: el flujo
      funciona igual, pero entiende menos variantes de frase. Ponga
      GOOGLE_API_KEY en el archivo .env para activarlo.
    </p>
  )}
</div>
</div>
</div>

{pedirDetalle && (
  <Dialogo titulo="Reportar un problema"
    mensaje="Describe el problema que encontró:"
    conCampo conVoz multilinea placeholder="Se cayó el micrófono al confirmar un conteo.."
    textoAceptar="Enviar"
    onAceptar={reportarProblema}
    onCancel={(() => setPedirDetalle(false)) />
  )}

{escribiendoAdmin && (
  <Dialogo titulo="Escribirle al administrador"
    mensaje={`Para: ${capitalizar(admin?.nombre) || "Administrador"}\nDe:
    ${capitalizar(usuario?.nombre) || "Usted"}\nAsunto: Mensaje desde CuentaVoz\n\nEscriba su mensaje:`}
    conCampo conVoz multilinea placeholder="Necesito que me asigne la bodega de Kiosco
    Taquilla..."
    etiquetaCampo={`Su mensaje para ${capitalizar(admin?.nombre) || "el administrador"}`}
    textoAceptar="Enviar"
    onAceptar={enviarMensajeAdministrador}
    onCancel={(() => setEscribiendoAdmin(false)) />
  )}
</Marco>
);
}

```

Archivo: `frontend/src/vistas/Mensajes.jsx`

```

import { useEffect, useRef, useState } from "react";
import { pedir } from "../api";
import { escuchar, hablar, quitarTildes } from "../voz";
import { enviarPorEmailJS, capitalizar, rolDe } from "../correoAdmin";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import Icono from "../Iconos";

/** La bandeja completa de "Soporte en vivo": para un auxiliar, lo que ha
    escrito y si ya le respondieron; para el administrador, lo que le
    han escrito y lo que falta por responder. Antes esta lista vivía
    dentro de la tarjeta de Ayuda, pero con varios auxiliares escribiendo
    (Luis, Valentina...) esa tarjeta se llenaba y quedaba desordenada -
    aquí tiene su propio espacio, como Auditoría o Reportes. */
export default function Mensajes({ token, usuario }) {
  const [mensajes, setMensajes] = useState(null);
  const [miPerfil, setMiPerfil] = useState(null);
  const [respondiendoId, setRespondiendoId] = useState(null);
  const [respuestaPrellenada, setRespuestaPrellenada] = useState("");
  const [enviandoRespuesta, setEnviandoRespuesta] = useState(false);
  const [msg, setMsg] = useState("");
  const [escuchandoOrden, setEscuchandoOrden] = useState(false);
  const [ordenVoz, setOrdenVoz] = useState("");
  const idEscuchaOrden = useRef(0);
  const esAdmin = usuario?.perfil === "auditor";

  useEffect(() => {

```

```

    cargar();
    pedir("/api/usuarios/yo", {}, token).then(setMiPerfil).catch(() => {});
  }, [token]);

function cargar() {
  pedir("/api/soporte/mensajes", {}, token).then(setMensajes).catch(() => setMensajes([]));
}

async function responderMensaje(respuesta) {
  const id = respondiendId;
  setRespondiendId(null);
  setRespuestaPrellenada("");
  if (!respuesta || !respuesta.trim() || !id) return;
  setEnviandoRespuesta(true);
  try {
    const r = await pedir(`/api/soporte/mensajes/${id}/responder`, {
      method: "POST", body: JSON.stringify({ respuesta }),
    }, token);
    if (r?.correo_destinatario) {
      await enviarPorEmailJS(r.correo_destinatario, capitalizar(usuario?.nombre) || "Usuario",
        respuesta, rolDe(usuario), miPerfil?.correo);
    }
    const listo = `Respuesta enviada a ${capitalizar(r?.destinatario) || "la persona"}`;
    setMsg(listo);
    hablar(listo);
    cargar();
  } catch (e) { setMsg(e.message); }
  setEnviandoRespuesta(false);
}

// "Respóndele a Stephanie que ya quedó asignada la bodega" - con un
// solo mensaje pendiente no hace falta nombrar a nadie, lo dicho es
// directamente la respuesta. Con varios, busca el nombre de quien
// escribió dentro de la frase (mismo patrón de emparejar por nombre
// que ya usa el resto de la app) y separa el resto como el texto de
// la respuesta.
function _extraerRespuestaPorVoz(texto, listaPendientes) {
  if (listaPendientes.length === 1) {
    return { mensaje: listaPendientes[0], texto: texto.trim() };
  }
  const t = quitarTildes(texto.toLowerCase());
  const encontrado = listaPendientes.find(
    (m) => m.de && t.includes(quitarTildes(m.de.toLowerCase()));
  );
  if (!encontrado) return { mensaje: null, texto: null };
  const patron = new RegExp(`^.*?\b${encontrado.de}\\b\\s*(que|:)?\\s*`, "i");
  const textoFinal = texto.replace(patron, "").trim() || texto.trim();
  return { mensaje: encontrado, texto: textoFinal };
}

function alTextoOrdenVoz(texto, miId) {
  if (miId !== idEscuchaOrden.current || !texto) return;
  setOrdenVoz(texto);
  const pendientesLista = (mensajes || []).filter((m) => !m.respuesta);
  if (pendientesLista.length === 0) return;
  const { mensaje, texto: respuestaTexto } = _extraerRespuestaPorVoz(texto, pendientesLista);
  if (!mensaje) {
    const nombres = pendientesLista.map((m) => capitalizar(m.de)).join(", ");
    const aviso = `Hay varios mensajes pendientes: ${nombres}. Diga el nombre de quién le
escribió.`;
    setMsg(aviso);
    hablar(aviso);
    return;
  }
  setRespuestaPrellenada(respuestaTexto);
  setRespondiendId(mensaje.id);
  setOrdenVoz("");
}

```

```

}

function escucharOrdenVoz() {
  setMsg("");
  const miId = ++idEscuchaOrden.current;
  escuchar({
    alTexto: (t) => alTextoOrdenVoz(t, miId),
    alEstado: (e) => setEscuchandoOrden(e === "escuchando"),
    alError: setMsg,
  });
}

const original = mensajes?.find((m) => m.id === respondiendoid);
const pendientes = mensajes?.filter((m) => !m.respuesta.length ?? 0);
const respondidos = mensajes?.filter((m) => m.respuesta.length ?? 0);

return (
  <Marco titulo="Mensajes · soporte en vivo"
    chip={{ texto: esAdmin ? "BANDEJA DEL ADMINISTRADOR" : "MIS MENSAJES", tipo: "azul" }}>
    {mensajes !== null && mensajes.length > 0 && (
      <div className="kpi" style={{ marginBottom: 16 }}>
        <div className="kpi">
          <div className="kpi-cabeza">
            <span className="icono-kpi">✉</span>
            <small>{esAdmin ? "Mensajes recibidos" : "Mensajes enviados"}</small>
          </div>
          <b>{mensajes.length}</b>
          <i>en total</i>
        </div>
        <div className={`kpi ${pendientes ? "oro" : "verde"}`>
          <div className="kpi-cabeza">
            <span className="icono-kpi">{pendientes ? "⌚" : "☑"}</span>
            <small>Pendientes de respuesta</small>
          </div>
          <b>{pendientes}</b>
          <i>{esAdmin ? "por responder" : "esperando al administrador"}</i>
        </div>
        <div className="kpi verde">
          <div className="kpi-cabeza">
            <span className="icono-kpi">🗨</span>
            <small>Respondidos</small>
          </div>
          <b>{respondidos}</b>
          <i>ya resueltos</i>
        </div>
      </div>
    )}

    {esAdmin && pendientes > 0 && (
      <div className="card" style={{ marginBottom: 16 }}>
        <h3>Responder por voz</h3>
        <div style={{ display: "flex", gap: 10, alignItems: "center" }}>
          <input value={ordenVoz} onChange={(e) => setOrdenVoz(e.target.value)}
            onKeyDown={(e) => e.key === "Enter" && alTextoOrdenVoz(ordenVoz,
idEscuchaOrden.current)}
            placeholder={pendientes === 1
              ? "Diga o escriba la respuesta..."
              : "«Respóndele a Stephanie que ya quedó asignada la bodega...»"}
            aria-label="Responder por voz o escribiendo"
            style={{ flex: 1, padding: "12px 14px",
              border: "1px solid var(--borde)", borderRadius: 12 }} />
          <button className={`mic-btn ${escuchandoOrden ? "escuchando" : ""}`}
            style={{ width: 46, height: 46, fontSize: "1.2rem" }}
            onClick={escucharOrdenVoz} title="responder por voz"
            aria-label="Responder por voz"><Icono nombre="microfono" tam={22} /></button>
        </div>
      </div>
    )}
  </Marco>
)

```



```

    </div>
    <p className="pista" style={{ marginTop: 8 }}>
      {pendientes === 1
        ? "Un solo mensaje pendiente: lo que diga se usa directo como respuesta."
        : "Varios mensajes pendientes: diga primero el nombre de quién le escribió."}
    </p>
  </div>
)}

{msg && <p className="msg-ok" style={{ marginBottom: 10 }}>{msg}</p>}

{mensajes === null ? (
  <div className="card"><p className="pista">Cargando...</p></div>
) : mensajes.length === 0 ? (
  <div className="card">
    <p className="vacio">
      {esAdmin ? "Nadie le ha escrito todavía." : "No ha escrito ningún mensaje todavía."}
    </p>
  </div>
) : (
  mensajes.map((m) => (
    <div className="card" key={m.id} style={{ marginBottom: 12 }}>
      <div style={{ display: "flex", alignItems: "flex-start", gap: 12 }}>
        <span className="avatar-chico" style={{ marginTop: 2 }}>
          {(esAdmin ? m.de : m.para)?.[0]?.toUpperCase() || "?"}
        </span>
        <div style={{ flex: 1, minWidth: 0 }}>
          <div style={{ display: "flex", justifyContent: "space-between",
            alignItems: "center", gap: 10, flexWrap: "wrap" }}>
            <strong style={{ textTransform: "capitalize", color: "var(--azul-prof)" }}>
              {esAdmin ? capitalizar(m.de) : `Para ${capitalizar(m.para)}`}
            </strong>
            <span style={{ display: "flex", alignItems: "center", gap: 10 }}>
              <span className={`etiqueta-actividad ${m.respuesta ? "verde" : "oro"}`}
                style={{ marginLeft: 0 }}>
                {m.respuesta ? "respondido" : "pendiente"}
              </span>
              <span className="pista">{m.creado}</span>
            </span>
          </div>
          <p style={{ fontSize: ".92rem", marginTop: 6, lineHeight: 1.5 }}>{m.mensaje}</p>

          {m.respuesta ? (
            <div style={{ display: "flex", alignItems: "flex-start", gap: 10,
              marginTop: 12, padding: 12, borderTop: "1px solid var(--borde)" }}>
              <span className="avatar-chico" style={{ background: "var(--verde-bg)", color:
                "var(--verde)" }}>
                {(esAdmin ? usuario?.nombre : m.para)?.[0]?.toUpperCase() || "?"}
              </span>
              <div style={{ flex: 1 }}>
                <div style={{ display: "flex", justifyContent: "space-between", alignItems:
                  "baseline" }}>
                  <strong style={{ color: "var(--verde)", fontSize: ".85rem" }}>
                    {esAdmin ? "Su respuesta" : `Respuesta de ${capitalizar(m.para)}`}
                  </strong>
                  <span className="pista">{m.respondido}</span>
                </div>
                <p style={{ fontSize: ".9rem", marginTop: 3, lineHeight: 1.5
                  }}>{m.respuesta}</p>
              </div>
            </div>
          ) : esAdmin ? (
            <button className="btn borde" disabled={enviandoRespuesta}
              style={{ marginTop: 12, fontSize: ".85rem", padding: "6px 16px" }}
              onClick={() => { setRespuestaPrellenada(""); setRespondiendoId(m.id); }}>
              Responder
            </button>
          ) : null}
        </div>
      </div>
    )
  )
}

```

```

        </button>
      ) : null}
    </div>
  </div>
</div>
))
}}

{respondiendoId && (
  <Dialogo titulo="Responder mensaje"
    mensaje={Para: ${capitalizar(original?.de) || "la persona"}\nDe:
    ${capitalizar(usuario?.nombre) || "Usted"}\n\nMensaje original:\n${original?.mensaje || ""}\n\nEscriba
    su respuesta:`}
    conCampo conVoz multilinea placeholder="Ya quedó asignada esa bodega, revise en unos
    minutos..."
    etiquetaCampo={`Su respuesta a ${capitalizar(original?.de) || "la persona"}`}
    valorInicial={respuestaPrellenada}
    confirmarAlAbrir={Boolean(respuestaPrellenada)}
    textoAceptar="Enviar"
    onAceptar={responderMensaje}
    onCancel={() => { setRespondiendoId(null); setRespuestaPrellenada(""); }} />
  )}
</Marco>
);
}

```

Archivo: `frontend/src/correoAdmin.js` — Envío real de correo (EmailJS) compartido por las dos vistas anteriores.

```

/** Envío real de correo para "Soporte en vivo" (mensajes y respuestas
entre auxiliares y administrador), compartido entre Ayuda.jsx y
Mensajes.jsx para no duplicar credenciales ni lógica.

La Public Key de EmailJS está hecha para ir en el código del
frontend (a diferencia de una clave de API común, no es secreta -
así lo documenta EmailJS). El correo sale desde el navegador de
quien escribe, usando la cuenta de Gmail conectada en el panel de
EmailJS - por eso no pasa por la red de Render, que es lo que
bloqueaba el SMTP directo a Gmail y la API de Resend. */
const EMAILJS_SERVICE_ID = "service_9dgu33i";
const EMAILJS_TEMPLATE_ID = "template_9ddkqpa";
const EMAILJS_PUBLIC_KEY = "raCRzjMA9m2ymYuwk";

export async function enviarPorEmailJS(destinatario, nombreRemitente, mensaje, rol, correoRemitente) {
  try {
    const res = await fetch("https://api.emailjs.com/api/v1.0/email/send", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        service_id: EMAILJS_SERVICE_ID,
        template_id: EMAILJS_TEMPLATE_ID,
        user_id: EMAILJS_PUBLIC_KEY,
        template_params: {
          to_email: destinatario, name: nombreRemitente, message: mensaje,
          rol: rol || "", correo_remitente: correoRemitente || "",
          fecha: new Date().toLocaleString("es-CO", {
            day: "numeric", month: "long", year: "numeric", hour: "numeric", minute: "2-digit",
          }),
        },
      }),
    });
    return res.ok;
  } catch {
    return false;
  }
}

```

```
// admin.nombre y usuario.nombre llegan en minúscula (el resto de la app
// los capitaliza con CSS) - para texto plano (firma del correo, mensajes
// de la bandeja) hay que capitalizarlos en JS.
export function capitalizar(nombre) {
  if (!nombre) return "";
  return nombre.split(" ").map((p) => p.charAt(0).toUpperCase() + p.slice(1)).join(" ");
}

// Misma etiqueta que usa el resto de la app (BarraLateral, MiPerfil,
// Ingreso) para el rol - así la firma del correo y la bandeja dicen lo
// mismo que ve la persona dentro de CuentaVoz.
export function rolDe(usuario) {
  return usuario?.perfil === "auditor" ? "Administrador de bodega" : "Auxiliar de inventarios";
}
```

12.34 Frontend: Mi perfil y Cerrar sesión (vistas 5.28 y 5.29)

Archivo: `frontend/src/vistas/MiPerfil.jsx`

```
import { useEffect, useState } from "react";
import { pedir, BASE, leerToken, actualizarTokenSesion, esFalloRed } from "../api";
import { configurarVoz, hablar } from "../voz";
import { hayAutenticadorDisponible, registrarHuellaDispositivo } from "../webauthn";
import Marco from "../Marco";
import Dialogo from "../Dialogo";
import AsistenteVoz from "../AsistenteVoz";

function formatoAcceso(fechaStr) {
  if (!fechaStr) return "-";
  const [fecha, hora] = fechaStr.split(" ");
  const hoy = new Date().toISOString().slice(0, 10);
  return fecha === hoy ? `hoy ${hora}` : `${fecha.slice(5).replace("-", "/")} ${hora}`;
}

// Lo que YA se puede cambiar por voz aquí (la voz, la velocidad, la
// confirmación hablada, y preguntar por los datos de la cuenta). El PIN,
// la huella y la foto son a propósito solo manuales, por seguridad - el
// agente lo explica si se le pide, en vez de fingir que puede hacerlo.
const EJEMPLOS_PERFIL = [
  "cambia mi voz a puck", "usa la voz aoede", "habla más lento", "velocidad normal",
  "activa la confirmación hablada", "desactiva la confirmación hablada",
  "¿cuándo fue mi último acceso?", "¿cuántos días le quedan a mi PIN?",
  "¿qué bodegas tengo asignadas?", "¿tengo la huella registrada?",
];

export default function MiPerfil({ token, ir }) {
  const [datos, setDatos] = useState(null);
  const [bodegas, setBodegas] = useState([]);
  const [msg, setMsg] = useState("");
  const [pinActual, setPinActual] = useState("");
  const [pin, setPin] = useState("");
  const [pin2, setPin2] = useState("");
  const [err, setErr] = useState("");
  const [fotoUrl, setFotoUrl] = useState(null);
  const [confirmarCierre, setConfirmarCierre] = useState(false);
  const [hayLector, setHayLector] = useState(false);
  const [voces, setVoces] = useState([]);

  // <img src> no puede mandar el header Authorization, y /foto exige
  // sesion - por eso se trae como blob autenticado (mismo patron que
  // descargarReporte) en vez de apuntar la imagen directo al endpoint.
  function cargarFoto() {
    fetch(`${BASE}/api/usuarios/yo/foto`, {
      headers: { Authorization: `Bearer ${token || leerToken()}` },
    });
  }
```

```

    }).then((r) => (r.ok ? r.blob() : null))
    .then((b) => setFotoUrl((prev) => {
      if (prev) URL.revokeObjectURL(prev);
      return b ? URL.createObjectURL(b) : null;
    }))
    .catch(() => setFotoUrl(null));
  }

  useEffect(() => {
    pedir("/api/usuarios/yo", {}, token).then(setDatos).catch(() => {});
    pedir("/api/usuarios/yo/bodegas", {}, token).then(setBodegas).catch(() => {});
    pedir("/api/voz/voces", {}, token).then((r) => setVoces(r.voces)).catch(() => {});
    hayAutenticadorDisponible().then(setHayLector);
    cargarFoto();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [token]);

  async function guardar() {
    setErr(""); setMsg("");
    try {
      await pedir("/api/usuarios/yo", { method: "PUT", body: JSON.stringify(datos) }, token);
      await pedir("/api/usuarios/yo/preferencias", {
        method: "PUT", body: JSON.stringify({
          idioma_voz: datos.idioma_voz, velocidad_voz: datos.velocidad_voz,
          confirmacion_hablada: datos.confirmacion_hablada,
        }),
      }, token);
      setMsg("Cambios guardados correctamente.");
    } catch (e) { setErr(e.message); }
  }

  async function subirFoto(e) {
    const f = e.target.files?.[0];
    if (!f) return;
    setErr(""); setMsg("");
    const form = new FormData();
    form.append("foto", f);
    try {
      const r = await fetch(`${BASE}/api/usuarios/yo/foto`, {
        method: "POST",
        headers: { Authorization: `Bearer ${token || leerToken()}` },
        body: form,
      });
      if (!r.ok) throw new Error("No se pudo subir la foto.");
      cargarFoto();
      window.dispatchEvent(new Event("cuentavoz:foto-actualizada"));
      setMsg("Foto actualizada.");
    } catch (e2) {
      setErr(esFalloRed(e2)
        ? "Sin conexión con el servidor. Revise el Wi-Fi e intente de nuevo."
        : e2.message);
    }
  }

  async function cambiarPin() {
    setErr("");
    if (!pinActual) return setErr("Digite su PIN actual.");
    if (pin.length < 6) return setErr("El PIN debe tener al menos 6 dígitos.");
    if (pin !== pin2) return setErr("Los dos PIN no coinciden.");
    try {
      const r = await pedir("/api/usuarios/yo/pin", {
        method: "PUT", body: JSON.stringify({ pin_actual: pinActual, pin }),
      }, token);
      // el PIN nuevo invalida el token con el que se pidió el cambio (igual
      // que "cerrar todas las sesiones"): sin recargar, el resto de llamadas
      // de esta pantalla seguirían usando el token viejo ya sin validez.
      actualizarTokenSesion(r.token);
      window.location.reload();
    } catch (e) { setErr(e.message); }
  }

```

```

}
async function registrarHuella() {
  setErr(""); setMsg("");
  try {
    await registrarHuellaDispositivo(token);
    setDatos({ ...datos, huella_registrada: true });
    setMsg("Huella registrada en este dispositivo.");
  } catch (e) {
    setErr(e.name === "NotAllowedError"
      ? "Se canceló el registro de la huella."
      : e.message);
  }
}
async function eliminarHuella() {
  setErr(""); setMsg("");
  try {
    await pedir("/api/usuarios/yo/huella", { method: "DELETE" }, token);
    setDatos({ ...datos, huella_registrada: false });
    setMsg("Huella eliminada de este dispositivo.");
  } catch (e) { setErr(e.message); }
}
async function cerrarTodas() {
  try {
    const r = await pedir("/api/usuarios/yo/cerrar-todas", { method: "POST" }, token);
    actualizarTokenSesion(r.token);
    window.location.reload();
  } catch (e) { setErr(e.message); }
}
function probarVoz() {
  hablar("Hola, soy CuentaVoz. Así voy a sonar de ahora en adelante.", { forzar: true });
}

// Solo actualiza en memoria (para que "Probar voz" suene distinto de una
// vez): el guardado real de estas preferencias queda para "Guardar
// cambios", junto con los datos personales - un solo botón, un solo
// guardado, como el resto de la pantalla.
function elegirPreferencia(cambios) {
  const nuevos = { ...datos, ...cambios };
  setDatos(nuevos);
  configurarVoz({ idioma: nuevos.idioma_voz, velocidad: nuevos.velocidad_voz,
    confirmacionHablada: nuevos.confirmacion_hablada });
}

// El agente ("Pregúntele al agente", arriba) SÍ puede cambiar la voz, la
// velocidad y la confirmación hablada directo en la base de datos - a
// diferencia de elegirPreferencia() (que solo actualiza en memoria hasta
// "Guardar cambios"), aquí ya quedó guardado en el servidor, así que se
// trae de nuevo (en vez de adivinar qué cambió) y se aplica de una con
// configurarVoz(), para que la SIGUIENTE frase que diga el agente ya
// suene con la preferencia nueva sin esperar a recargar la página.
function recargarPreferencias() {
  pedir("/api/usuarios/yo", {}, token).then((d) => {
    setDatos(d);
    configurarVoz({ idioma: d.idioma_voz, velocidad: d.velocidad_voz,
      confirmacionHablada: d.confirmacion_hablada });
  }).catch(() => {});
}

if (!datos) return <Marco titulo="Mi perfil"><p className="cargando">Cargando...</p></Marco>;

return (
  <Marco titulo="Mi perfil · datos personales"
    chip={{ texto: "SESIÓN ACTIVA", tipo: "verde" }}>
    <AsistenteVoz token={token} vista="perfil" ir={ir} ejemplos={_EJEMPLOS_PERFIL}
      placeholder="cambia mi voz a puck, ¿cuándo fue mi último acceso?..."
      alActualizar={recargarPreferencias} />
  </Marco>
)

```

```

<div className="perfil-cols">
  <div className="card mic-caja">
    {fotoUrl ? (
      <img src={fotoUrl} alt=""
        style={{ width: 110, height: 110, borderRadius: "50%",
          objectFit: "cover", border: "3px solid var(--azul)" }} />
    ) : (
      <span className="avatar-grande">{{datos.nombre || "?"}}[0].toUpperCase()</span>
    )}
    <b style={{ textTransform: "capitalize" }}>{{datos.nombre}}</b>
    <small>{{datos.perfil === "auditor" ? "Administrador de bodega"
      : "Auxiliar de inventarios"}}</small>
    <label className="btn borde" style={{ textAlign: "center" }}>
      Cambiar foto
      <input type="file" accept="image/*" hidden onChange={subirFoto} />
    </label>
  </div>

  <div className="card">
    <h3>Datos personales</h3>
    <div className="campos-2col">
      {[["Nombre completo", "nombre"], ["Correo corporativo", "correo"],
        ["Teléfono de contacto", "telefono"], ["Código de empleado", "codigo"]].map(([, k]) =>
        (
          <div key={k} style={{ marginBottom: 10 }}>
            <label className="pista" htmlFor={`campo-perfil-${k}`}>{{1}}</label>
            <input id={`campo-perfil-${k}`} value={datos[k] || ""}
              onChange={(e) => setDatos({ ...datos, [k]: e.target.value })}
              style={{ width: "100%", padding: "11px 13px",
                border: "1px solid var(--borde)", borderRadius: 12 }} />
          </div>
        ))}
    </div>
    <label className="pista" id="etiqueta-bodegas-asignadas">Bodegas asignadas</label>
    <div className="chips" style={{ marginTop: 6 }} aria-labelledby="etiqueta-bodegas-
asignadas">
      {bodegas.length === 0 ? (
        <span className="pista">Sin bodegas asignadas todavía.</span>
      ) : bodegas.map((b) => <span key={b.id} className="chip">{{b.bodega}}</span>)}
    </div>
  </div>

  <div className="dos-columnas">
    <div className="card">
      <h3>Seguridad de la cuenta</h3>
      <p style={{ fontSize: ".87rem", marginBottom: 4 }}>
        Último acceso: <b>{{formatoAcceso(datos.ultimo_acceso)}}</b>
      </p>
      <p style={{ fontSize: ".87rem", marginBottom: 14 }}>
        Su PIN vence en <b>{{datos.pin_vence_en_dias}} días</b>
      </p>

      <input type="password" placeholder="PIN actual" value={pinActual}
        autoComplete="current-password"
        onChange={(e) => setPinActual(e.target.value)}
        aria-label="PIN actual"
        style={{ width: "100%", padding: "11px 13px", marginBottom: 8,
          border: "1px solid var(--borde)", borderRadius: 12 }} />
      <input type="password" placeholder="PIN nuevo (6 dígitos)" value={pin}
        autoComplete="new-password"
        onChange={(e) => setPin(e.target.value)}
        aria-label="PIN nuevo, 6 dígitos"
        style={{ width: "100%", padding: "11px 13px", marginBottom: 8,
          border: "1px solid var(--borde)", borderRadius: 12 }} />
      <input type="password" placeholder="Confirmar PIN" value={pin2}

```

```

        autoComplete="new-password"
        onChange={(e) => setPin2(e.target.value)}
        aria-label="Confirmar PIN nuevo"
        style={{ width: "100%", padding: "11px 13px",
            border: "1px solid var(--borde)", borderRadius: 12 }} />
<div className="grilla-botones">
    <button className="btn borde" onClick={cambiarPin}>Cambiar PIN</button>
</div>

<div className="grilla-botones" style={{ marginTop: 10 }}>
    {datos.huella_registrada ? (
        <button className="btn gris" onClick={eliminarHuella}>
            ✓ Huella registrada – quitar de este dispositivo
        </button>
    ) : (
        <button className="btn borde" onClick={registrarHuella} disabled={!hayLector}>
            Registrar huella de este dispositivo
        </button>
    )}
    <button className="btn gris" onClick={() => setConfirmarCierre(true)}>
        Cerrar sesión en todos los dispositivos
    </button>
</div>
<p className="pista" style={{ marginTop: 8 }}>
    {hayLector
        ? "El PIN se guarda cifrado (bcrypt); la huella usa el lector o " +
        "Windows Hello de este equipo (WebAuthn) y solo sirve para " +
        "ingresar desde este mismo dispositivo."
        : "El PIN se guarda cifrado (bcrypt); este equipo o navegador no " +
        "tiene un lector de huella / Windows Hello configurado, así " +
        "que el ingreso con huella no está disponible aquí."}
    </p>
</div>

<div className="card">
    <h3>Preferencias de voz</h3>
    <label className="pista" htmlFor="campo-voz">Voz de CuentaVoz</label>
    <select id="campo-voz" value={datos.idioma_voz}
        onChange={(e) => elegirPreferencia({ idioma_voz: e.target.value })}
        style={{ width: "100%", marginTop: 6, marginBottom: 8, padding: "10px 12px",
            border: "1px solid var(--borde)", borderRadius: 10 }}>
        {voces.map((v) => (
            <option key={v.clave} value={v.clave}>{v.nombre} – {v.etiqueta}</option>
        ))}
    </select>
    <div className="grilla-botones" style={{ marginBottom: 6 }}>
        <button className="btn borde" onClick={probarVoz}>🔊 Probar voz</button>
    </div>
    <p className="pista" style={{ marginBottom: 12 }}>
        Voz neuronal real (no la del navegador): suena humana y fluida.
        El reconocimiento de lo que usted dice sigue siempre en español
        de Colombia; esto solo cambia cómo suena CuentaVoz al responder.
    </p>

    <label className="pista" id="etiqueta-velocidad">Velocidad de la respuesta</label>
    <div className="grilla-botones" style={{ marginTop: 6, marginBottom: 12 }}
        role="group" aria-labelledby="etiqueta-velocidad">
        {[ "lenta", "normal", "rapida" ].map((v2) => (
            <button key={v2}
                className={`btn ${datos.velocidad_voz === v2 ? "" : "borde"}`}
                onClick={() => elegirPreferencia({ velocidad_voz: v2 })}
                aria-pressed={datos.velocidad_voz === v2}>
                {v2[0].toUpperCase() + v2.slice(1)}
            </button>
        ))}
    </div>
</div>

```

```

<label className="pista" id="etiqueta-confirmacion">Confirmación hablada</label>
<div className="grilla-botones" style={{ marginTop: 6, marginBottom: 18 }}
  role="group" aria-labelledby="etiqueta-confirmacion">
  <button className={`btn ${datos.confirmacion_hablada ? "" : "borde"}`}
    onClick={() => elegirPreferencia({ confirmacion_hablada: true })}
    aria-pressed={!datos.confirmacion_hablada}>
    Activada
  </button>
  <button className={`btn ${!datos.confirmacion_hablada ? "" : "borde"}`}
    onClick={() => elegirPreferencia({ confirmacion_hablada: false })}
    aria-pressed={!datos.confirmacion_hablada}>
    Desactivada
  </button>
</div>

{err && <p className="error" role="alert">{err}</p>}
{msg && <p className="msg-ok" aria-live="polite">{msg}</p>}
<button className="btn" style={{ width: "100%" }} onClick={guardar}>
  Guardar cambios
</button>
</div>
</div>

{confirmarCierre && (
  <Dialogo titulo="Cerrar sesión en todos los dispositivos"
    mensaje="Esto cierra la sesión en todas las tabletas donde haya entrado, incluida
esta. ¿Continuar?"
    textoAceptar="Cerrar todas" peligro
    onAceptar={() => { setConfirmarCierre(false); cerrarTodas(); }}
    onCancel={() => setConfirmarCierre(false)} />
)}
</Marco>
);
}

```

Archivo: `frontend/src/vistas/CerrarSesion.jsx`

```

import { useEffect, useState } from "react";
import { pedir } from "../api";

export default function CerrarSesion({ token, sesionId, onCancel, onSalir }) {
  const [pend, setPend] = useState(null);

  useEffect(() => {
    pedir(`/api/sesiones/${sesionId}/avance`, {}, token)
      .then((a) => setPend({ bodega: a.bodega, hechas: a.hechas }))
      .catch(() => setPend({ bodega: null, hechas: 0 }));
  }, [sesionId, token]);

  if (!pend) return null;
  const hayTrabajo = Boolean(pend.bodega);

  return (
    <div className="overlay">
      <div className="modal">
        <h2>¿Cerrar la sesión?</h2>
        {hayTrabajo ? (
          <>
            <p>
              Tiene <b>{pend.bodega}</b> abierta con {pend.hechas} referencias contadas.
            </p>
            <div className="aviso">
              Lo registrado queda guardado y podrá continuar al volver.
            </div>
          </>
        ) : null}
      </div>
    </div>
  );
}

```



```

        </div>
      </>
    ) : (
      <p>No tiene trabajo pendiente: puede salir sin problema.</p>
    )}
    <div className="botones">
      <button className="btn borde" onClick={onCancelar}>Seguir trabajando</button>
      <button className="btn" onClick={onSalir}>Cerrar sesión</button>
    </div>
  </div>
</div>
);
}

```

12.35 Frontend: App.jsx, el enrutador que une las treinta vistas

Archivo: frontend/src/App.jsx

```

import { useRef, useState } from "react";
import { borrarSesion, guardarSesion, leerSesion, pedir } from "../api";
import { configurarVoz, detenerVoz } from "../voz";
import BarraLateral from "../BarraLateral";
import Ingreso from "../vistas/Ingreso";
import Inicio from "../vistas/Inicio";
import Conteo from "../vistas/Conteo";
import Pedido from "../vistas/Pedido";
import Legalizacion from "../vistas/Legalizacion";
import Bodegas from "../vistas/Bodegas";
import Auditoria from "../vistas/Auditoria";
import Reportes from "../vistas/Reportes";
import Panel from "../vistas/Panel";
import Ajustes from "../vistas/Ajustes";
import Ayuda from "../vistas/Ayuda";
import Mensajes from "../vistas/Mensajes";
import MiPerfil from "../vistas/MiPerfil";
import CerrarSesion from "../vistas/CerrarSesion";

/* El menú lateral. Cada entrada declara las vistas que contiene:
este objeto es el mapa de navegación hecho código. */
export const MENU = [
  { id: "inicio",          titulo: "Inicio",          vistas: ["inicio"] },
  { id: "pedidos",         titulo: "Pedidos",         vistas: ["pedido"] },
  { id: "conteo",          titulo: "Conteo",          vistas: ["conteo"] },
  { id: "legalizacion",    titulo: "Legalización",   vistas: ["legalizacion"] },
  { id: "bodegas",         titulo: "Bodegas",        vistas: ["bodegas"] },
  { id: "auditoria",       titulo: "Auditoría",      vistas: ["auditoria"], soloAuditor: true },
  { id: "reportes",        titulo: "Reportes",       vistas: ["reportes"], soloAuditor: true },
  { id: "panel",           titulo: "Panel",          vistas: ["panel"], soloAuditor: true },
  // El auxiliar no puede cambiar nada aquí (el umbral y el modo sin
  // conexión ya estaban bloqueados por el backend con requiere_perfil):
  // mostrarle la pantalla igual, solo en modo lectura, dejaba ver ajustes
  // administrativos que no le sirven de nada y no puede tocar - mejor
  // ocultarla del todo, igual que ya se hace con Auditoría/Reportes/Panel.
  { id: "ajustes",         titulo: "Ajustes",        vistas: ["ajustes"], soloAuditor: true },
  { id: "ayuda",           titulo: "Ayuda",          vistas: ["ayuda"] },
  { id: "mensajes",        titulo: "Mensajes",       vistas: ["mensajes"] },
  { id: "perfil",          titulo: "Mi perfil",      vistas: ["perfil"] },
  { id: "salir",           titulo: "Cerrar sesión",  vistas: ["salir"] },
];

const VISTAS = {
  inicio: Inicio, pedido: Pedido, conteo: Conteo,
  legalizacion: Legalizacion, bodegas: Bodegas, auditoria: Auditoria,
  reportes: Reportes, panel: Panel, ajustes: Ajustes, ayuda: Ayuda,
  mensajes: Mensajes, perfil: MiPerfil,

```

```

};

/** Qué entrada del menú resaltar para una vista. Garantiza que el menú
    señale siempre dónde vive la pantalla que se está viendo. */
export function menuDe(vista) {
  const g = MENU.find((m) => m.vistas.includes(vista));
  return g ? g.id : null;
}

export default function App() {
  // arranca directo con la sesión ya guardada (si hay) en vez de forzar
  // login otra vez: sin esto, abrir la app sin señal (o reiniciarla en
  // medio de una caída de Wi-Fi) dejaba a cualquiera sin poder entrar,
  // aunque ya se hubiera identificado antes en este mismo equipo.
  const [sesion, setSesion] = useState(() => leerSesion());
  const [vista, setVista] = useState("inicio");
  const [ctx, setCtx] = useState({ sessionId: 1 });
  const [salir, setSalir] = useState(false);
  // Contador que sube en cada ir(): las pantallas con pestañas usan
  // [tabInicial, navSeq] como dependencia de su useEffect en vez de solo
  // [tabInicial] - sin navSeq, pedir por voz la MISMA pestaña dos veces
  // seguidas (con un clic manual a otra pestaña en el medio) no cambiaba
  // nada, porque tabInicial ya traía ese mismo valor de la vez anterior y
  // React no vuelve a correr un efecto cuya dependencia no cambió de
  // valor, aunque la persona sí haya navegado a otro lado desde entonces.
  const navSeq = useRef(0);

  /** Navega y pasa contexto: ir("bodegas", { bodegaId: 3 }) */
  function ir(destino, contexto = {}) {
    // sin esto, una respuesta hablada que la pantalla anterior todavía
    // estaba esperando (la síntesis real tarda unos segundos) podía
    // reproducirse sola varios segundos después, ya con otra pantalla en
    // uso - sonaba como si el agente hablara de la nada, fuera de tema.
    detenerVoz();
    navSeq.current += 1;
    setCtx((c) => ({ ...c, ...contexto, navSeq: navSeq.current }));
    setVista(destino);
  }

  if (!sesion)
    return (
      <Ingreso
        alEntrar={(s) => {
          guardarSesion(s);
          setSesion(s);
          setVista("inicio");
          pedir("/api/usuarios/yo", {}, s.token).then((p) =>
            configurarVoz({ idioma: p.idioma_voz, velocidad: p.velocidad_voz,
              confirmacionHablada: p.confirmacion_hablada })
          ).catch(() => {});
        }}
      />
    );

  const Vista = VISTAS[vista] || Inicio;

  return (
    <div className="app-root">
      <BarraLateral
        activo={salir ? "salir" : menuDe(vista)}
        usuario={sesion.usuario}
        token={sesion.token}
        onNavegar={(id) => {
          detenerVoz();
          if (id === "salir") {

```

```

        setSalir(true);
        return;
    }
    const g = MENU.find((m) => m.id === id);
    if (g) {
        // el menu lateral es una navegacion "limpia": sin esto, un
        // contexto de una vista anterior (tabInicial="recetas" de
        // Pedido, bodegaId de un detalle, etc.) se quedaba pegado para
        // siempre - la proxima vez que se entraba a Ajustes por el
        // menu, por ejemplo, seguia abriendo la pestana equivocada.
        setCtx({ sessionId: ctx.sessionId });
        setVista(g.vistas[0]);
    }
  }}
/>

<div className="contenido">
  <Vista
    token={sesion.token}
    usuario={sesion.usuario}
    {...ctx}
    ir={ir}
  />
</div>

{salir && (
  <CerrarSesion
    token={sesion.token}
    sessionId={ctx.sessionId}
    onCancel={() => setSalir(false)}
    onSalir={() => {
      borrarSesion();
      setSesion(null);
      setSalir(false);
      setVista("inicio");
    }}
  />
)}
</div>
);
}

```

Archivo: frontend/src/main.jsx

```

import React from "react";
import { createRoot } from "react-dom/client";
import App from "../App";
import "../index.css";

createRoot(document.getElementById("root")).render(<App />);

```

13. Panel gerencial: el dashboard nativo (sin Power BI)

La propuesta original planteaba un tablero en Power BI insertado en la aplicación. Eso nunca se construyó así, y el código ya no conserva ni el gancho: el panel gerencial real (vistas 5.22 y 5.23) es nativo — React puro, calculado en vivo sobre PostgreSQL, sin Power BI Desktop, sin modelo semántico, sin medidas DAX y sin ninguna variable de entorno de Power BI en el código. Esta sección explica cómo funciona lo que sí existe.

DE DÓNDE SALE CADA NÚMERO DEL PANEL GERENCIAL

Sin Power BI: dashboard nativo en React, calculado en vivo sobre la base real

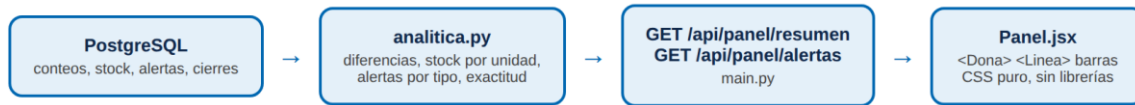


Figura 41. De dónde sale cada número del panel: sin Power BI, calculado en vivo.

13.1 Las métricas: backend/servicios/analitica.py

Un único servicio, sin dependencias de análisis externas, calcula todo lo que el panel muestra directamente sobre las mismas tablas que usa el resto de la aplicación — una sola fuente de verdad, nunca datos duplicados o desincronizados:

- **diferencias_por_bodega:** suma de $|\text{contado} - \text{sistema}|$ por bodega, sobre lo confirmado; alimenta la barra de la vista 5.22.
- **stock_por_unidad:** cuánto stock hay por cada unidad de medida (Kilogram, Liter, Unidad, Portion); alimenta la dona.
- **alertas_por_tipo:** cuenta las alertas abiertas agrupadas por su tipo (desviación, negativo, unidad, inexistente); vista 5.23.
- **descuadres_recurrentes:** qué artículos fallan una y otra vez entre bodegas, con su diferencia acumulada.
- **historial_exactitud:** un punto por cada bodega cerrada con doble firma, en orden cronológico — la línea de tendencia.
- **resumen_ejecutivo / resumen_alertas_panel:** empaquetan lo anterior en los dos objetos JSON que consumen los endpoints de 13.2. El código completo está en la sección 12.12.

13.2 Los dos endpoints del panel

Panel.jsx pide sus datos a dos rutas, ambas restringidas al perfil de administrador (requiere_perfil("auditor"), sección 14.2):

- **GET /api/panel/resumen:** los cuatro KPIs, la diferencia por bodega, el stock por unidad y el historial de exactitud — pestaña Resumen ejecutivo (5.22).
- **GET /api/panel/alertas:** los negativos corregidos, el tiempo promedio por bodega, las alertas por tipo y los descuadres recurrentes — pestaña Bodegas y alertas (5.23).

13.3 Gráficas sin librerías: donas y barras con CSS puro

Coherente con la decisión de arquitectura de no traer una librería de gráficas al bundle (ver ARQUITECTURA.md): la dona de stock por unidad es un `<Dona>` que arma un conic-gradient de CSS a partir de los porcentajes que ya calculó `analitica.py`; las barras de diferencia por bodega son `<div>` cuyo ancho en porcentaje se fija en línea (`width: {valor/máximo × 100}%`); la tendencia de exactitud es un

<Linea> con los puntos del historial. Cero dependencias nuevas, mismo resultado visual que un dashboard de BI.

14. Ciberseguridad: el blindaje de CuentaVoz

El sistema custodia el inventario de Colsubsidio, así que se blindo con la misma seriedad con que cuenta: que nadie entre sin permiso y que nadie lo tumbe. La estrategia es defensa en profundidad: capas independientes donde cada una asume que la anterior puede fallar. Esta sección trae el diagrama, el código de cada escudo, la configuración segura y el mapa amenaza por amenaza.

14.1 Defensa en profundidad



Figura 42. Las capas de defensa: un ataque debe atravesarlas todas; la franja dorada es la disponibilidad.

14.2 Identidad: contraseñas cifradas, token firmado y perfiles

Un archivo nuevo concentra la identidad. Las contraseñas jamás se guardan legibles (hash bcrypt); el ingreso entrega un token JWT firmado que vence solo a las 8 horas; y los permisos se exigen por perfil: cerrar bodegas o aprobar creaciones es solo del administrador.

El archivo completo y actual — con WebAuthn, versión de token para invalidar sesiones y permisos por asignación de bodega, que no existían en la versión anterior de este manual — está en la sección 12.18.

14.3 Blindaje de la API: límites, CORS cerrado y cabeceras

Estos son los cambios de seguridad sobre el main.py de la sección 12.12: límite de intentos por minuto y por IP (frena fuerza bruta y ataques de saturación), CORS restringido a la tableta autorizada, cabeceras de protección del navegador, el endpoint de ingreso y la exigencia de identidad en los endpoints sensibles.

El código real y completo de estos límites, cabeceras y validaciones de identidad está integrado en el main.py de la sección 12.19 (no en un archivo aparte).

14.4 Configuración segura: secretos, contenedores, HTTPS y respaldos

Archivo: .env — variables nuevas

```
SECRETO_JWT=genere_una_llave_larga # openssl rand -hex 32
MINUTOS_TOKEN=480 # la sesion vence sola (8 h)
ORIGEN_PERMITIDO=http://localhost:5173
```

Archivo: backend/requirements.txt — dependencias nuevas

```
pyjwt
passlib[bcrypt]
slowapi
```

Archivo: docker-compose.yml — resiliencia (que nadie lo tumbe)

```
api:
  restart: always # si algo lo tumba, vuelve solo
  depends_on:
    db: { condition: service_healthy }
db:
  restart: always
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U cuentavoz"]
    interval: 10s
# la base NO publica ports: existe solo en la red interna
```

Archivo: Producción: HTTPS automático y respaldo diario

```
proxy: # docker-compose (solo produccion)
  image: caddy:2
  restart: always
  ports: [ "80:80", "443:443" ]
  volumes: [ "./Caddyfile:/etc/caddy/Caddyfile" ]

# Caddyfile — Caddy pone el certificado TLS automatico:
cuentavoz.ejemplo.colsubsidio.com {
  reverse_proxy api:8000
}

# respaldo diario 2 a.m. (crontab -e):
0 2 * * * docker exec db pg_dump -U cuentavoz cuentavoz > respaldos/cv_$(date +%F).sql
```

14.5 Mapa de amenazas: cada ataque con su escudo

Amenaza	Escudo de CuentaVoz	Dónde
Robo de contraseñas	Hash bcrypt: ni la base ni los respaldos guardan claves legibles	14.2
Suplantación de identidad	Token JWT firmado con vencimiento; sin token válido, la API responde 401	14.2 · 14.3
Abuso de permisos	Perfiles: cerrar bodega o aprobar creaciones exige requiere_perfil("auditor")	14.2

Amenaza	Escudo de CuentaVoz	Dónde
Fuerza bruta de claves	Máximo cinco ingresos por minuto por dirección IP	14.3
Tumbar el servicio (DoS)	Límites por endpoint + reinicio automático + WAF/CDN delante en producción	14.3 · 14.4
Inyección SQL	Todo pasa por el ORM con parámetros; cero SQL concatenado a mano	12.5–12.12
Datos manipulados desde la tableta	Validación Pydantic + reglas duras del backend («el backend dispone»)	12.8 · 12.12
Robo de la llave de Gemini	Secretos solo en .env, fuera del repositorio (.gitignore) y rotatables	12.2 · 14.4
Espionaje del tráfico	HTTPS/TLS automático con Caddy; el tablero en vivo viaja por WSS	14.4
Pérdida de datos	Respaldo diario de PostgreSQL y base aislada sin puerto público	14.4

14.6 Datos personales y pruebas de seguridad

Cumplimiento colombiano: alineado con la Ley 1581 de 2012 (habeas data), el sistema guarda el mínimo necesario — de las personas, solo nombre, perfil y su rastro de acciones para la auditoría; nada de datos sensibles. Y antes de congelar el código, el sábado se corren estas cinco pruebas de ataque controlado:

- Llamar /api/agente/turno sin token → la API responde 401 y no procesa nada.
- Seis contraseñas seguidas equivocadas → 429: bloqueado un minuto (la fuerza bruta se vuelve inviable).
- Dictar una frase con SQL malicioso («borrar tabla conteo») → es solo texto: el ORM la neutraliza y el agente responde con naturalidad.
- Apagar a la fuerza el contenedor de la API → docker lo revive solo en segundos y el tablero se reconecta.
- Desconectar internet (Gemini inalcanzable) → el agente cae con gracia a «¿me lo repite?» y el formulario guiado (vista 5.20); el conteo no se detiene.

15. Plan de construcción paso a paso (para que todo funcione)

El manual se ejecuta en ocho fases y cada fase usa archivos concretos de la sección 11. La regla contra los errores es una sola: ninguna fase empieza sin que la anterior haya pasado su prueba.

PLAN DE CONSTRUCCIÓN: 8 FASES, CADA UNA SE CONSTRUYE, SE CONECTA Y SE PRUEBA

Figura 43. Hoja de ruta; cada pieza nueva se conecta sobre algo ya probado.

Fase 0 · Entorno y repositorio (miércoles p. m.)

- **Construye:** el repositorio con la estructura del manual, el .env (12.2) con la llave de Google AI Studio, y los contenedores (12.1 y 12.3).
- **Conecta:** Docker crea la red interna api-db; todavía no hay más cables.
- **Prueba:** docker compose up deja la base respondiendo y /docs de FastAPI abre en el navegador.

Fase 1 · Datos reales (miércoles noche)

- **Construye:** las tablas (12.4 y 12.5) y el cargador del extracto (12.6).
- **Conecta:** script → PostgreSQL.
- **Prueba:** el cargador imprime 54 bodegas y 1.405 registros de stock, y «TABLA ACRILICA PICAR BLANCO 50X38CM FB» existe con el código 97503004.

Fase 2 · Servicios y reglas (jueves a. m.)

- **Construye:** conciliación (12.7) y validación con el mini reto de negativos (12.8).
- **Conecta:** los servicios sobre la base ya cargada.
- **Prueba:** «tabla para picar blanca» devuelve las dos blancas; -2 dispara la alerta de negativo; 90 cazuelas dispara la de desviación.

Fase 3 · Agente con Gemini (jueves p. m.)

- **Construye:** el cerebro (12.9) y el orquestador (12.10) detrás de /api/agente/turno (12.12).
- **Conecta:** API → Gemini por HTTPS con la llave de AI Studio, y orquestador → servicios de la fase 2.
- **Prueba:** enviar por texto «tres tablas para picar blancas» y recibir la respuesta ofreciendo las dos opciones.

Fase 4 · Voz en la tableta (viernes a. m.)

- **Construye:** la PWA (12.13 y 12.14): micrófono, voz de respuesta y pantalla de conteo (vistas 5.7 a 5.13).
- **Conecta:** navegador → API por HTTPS.

- **Prueba:** el ciclo hablado completo —de la voz a la confirmación en voz— en menos de dos segundos.

Fase 5 · Perfiles y auditoría (viernes p. m.)

- **Construye:** login por perfil (vista 5.1), candado de sesión, recuento ciego, comparativo y cierre con doble firma (vistas 5.17 y 5.10), y el ingreso seguro con contraseña cifrada y token firmado (14.2 y 14.3).
- **Conecta:** los estados de la sección 10 sobre los conteos ya funcionando.
- **Prueba:** dos usuarios en la misma bodega → el segundo queda bloqueado; al cerrar solo aparecen las referencias con diferencia.

Fase 6 · Multibodega, consultas y creación (sábado a. m.)

- **Construye:** el tablero en vivo (vista 5.13, WebSocket de 12.12), consultar datos e imprimir, explicar reportes, y las altas «pendientes de aprobación».
- **Conecta:** el WebSocket a todas las tabletas y el generador de archivos (12.11) a la carpeta de reportes.
- **Prueba:** dos bodegas contadas en paralelo; «¿cuánto arroz hay y dónde?» responde en voz y produce el archivo.

Fase 7 · Dashboard y pruebas finales (sábado p. m.)

- **Construye:** el consolidado con formato de carga (12.15), el panel gerencial nativo (sección 13) y el manejo de errores (sin red → cola local; Gemini caído → formulario guiado, vista 5.20).
- **Conecta:** analitica.py → los endpoints del panel (13.2) y el consolidado → carpeta de carga para My Inventory.
- **Prueba:** recorrer completa la matriz de la sección 16 sin fallos y congelar el código; el domingo solo se graba el pitch.

16. Matriz de cumplimiento: ninguna funcionalidad se queda en el papel

La tabla amarra propuesta, vistas, código y pruebas. Es la lista de chequeo del sábado en la noche:

Funcionalidad de la propuesta	Vista / Código	Prueba que lo demuestra
Abrir bodega y contar por voz	5.7 · 12.12–12.14	Bodega abierta y primer ítem guardado solo hablando.
Conciliación con nombres oficiales	5.8 · 12.7	«tabla para picar blanca» queda como TABLA ACRILICA PICAR BLANCO 50X38CM FB (97503004).
Desambiguación por voz o toque	5.8 · 12.10	Con dos candidatas, el agente lee opciones y registra la elegida.
Anomalía de magnitud (9 vs. 90)	5.13 · 12.8	Dictar 90 cazuelas dispara «el sistema espera 10, ¿confirma?».

Funcionalidad de la propuesta	Vista / Código	Prueba que lo demuestra
Validación de negativos (mini reto)	5.13 · 12.8	«Menos dos kilos» se rechaza con explicación hablada.
Corrección con rastro	11.1 · 12.10	«Uy, no: son nueve» corrige y conserva el valor anterior.
Doble conteo ciego del auditor	5.17 · 5.10 · 12.12	El auditor no ve el conteo 1; el cierre muestra solo diferencias.
Conteos simultáneos multibodega	5.13 · 12.12	Dos bodegas avanzan a la vez; el candado impide duplicados.
Crear producto/bodega pendiente	5.9 · 12.10	La creación queda «pendiente» hasta la aprobación del auditor.
Consultar datos e imprimir	5.20 · 11.3 · 12.11	«¿Cuánto arroz hay?» responde en voz y genera el archivo.
Explicar tablas y reportes	11.2 · 12.10	«No entiendo esta tabla» produce la explicación simple.
Archivo de carga para My Inventory	5.28 · 12.11	Consolidado con nombres y códigos idénticos a la base.
Panel gerencial con datos reales	5.22 · 5.23 · 13	El resumen ejecutivo y las alertas se calculan en vivo sobre PostgreSQL; nada es una maqueta ni un informe externo.
Ayuda y escalamiento al auditor	12.9 (regla 7)	«Llame al administrador» registra el caso y notifica.
Ciberseguridad: token, perfiles y límites	14.2–14.4	Sin token → 401; seis claves seguidas → bloqueo; la base no expone puertos.
Contingencia sin voz ni internet	5.10 · 14.6	Con la red o Gemini caídos, el formulario guiado sigue contando y sincroniza al volver.
Pedido al almacén por voz (momento 1)	5.3 · 5.4 · 12.16	Decir «cincuenta ajiacos» produce el pedido con solo los insumos que faltan.
Cálculo por receta estándar	5.5 · 12.16	La cantidad por porción por las porciones, menos el stock de esa bodega.
Legalización del servicio (momento 3)	5.11 · 5.12 · 12.16	Lo pedido contra lo usado, con sobrante devuelto y merma registrada.
Histórico de consumo real	5.12 · 12.16	Cada legalización alimenta el histórico que compara servicio contra servicio.
Reportes de subutilización y sobrepedido	5.21 · 12.16	Muestra qué insumos se piden y sobran, y en cuánto se sobrepide frente a la receta.

Funcionalidad de la propuesta	Vista / Código	Prueba que lo demuestra
Navegación coherente entre las treinta vistas	5.0a · 12.20	El menú resalta siempre donde vive la vista; las transiciones del agente están en el enrutador.
Detalle y línea de tiempo por bodega	5.14 · 12.17	Cada bodega muestra minuto a minuto qué pasó, de la apertura a la doble firma.
Consulta de inventario por artículo	5.15 · 12.17	«¿Cuánto arroz hay?» responde total, ubicaciones y cobertura en días.
Bandeja de alertas del administrador	5.19 · 12.17	Las alertas abiertas se resuelven desde una sola cola de trabajo.
Gestión de usuarios y permisos	5.25 · 12.17	Alta, baja y perfil de cada persona sin tocar la base de datos.
Registro de trazabilidad inmutable	5.26 · 12.17	Toda acción sensible queda con persona, hora y dato; nada se borra.
Ayuda y estado del sistema en la app	5.27 · 12.17	El botón «Pedir ayuda» lleva a FAQ, soporte y estado de los servicios.
Diseño adaptable a cualquier dispositivo	5.21 · 5b	La misma pantalla en 375, 768 y 1440 px: el micrófono y los botones se reorganizan solos.
Ingreso con huella del dispositivo (WebAuthn)	5.1 · 5.28 · 12.13	Registrar la huella en Mi perfil y volver a entrar sin teclear el PIN, con reto criptográfico real.
Mensajes: soporte en vivo auxiliar-administrador	5.27a · 12.33	Un mensaje escrito llega a la bandeja del administrador y por correo real (EmailJS).
Modo sin conexión real (PWA)	5.10 · 12.24	La app carga sin red tras la primera visita; lo contado sin señal se sincroniza solo al volver.
Accesibilidad (teclado, lector de pantalla, WCAG AA)	5 · 12.23–12.35	Toda la app se navega sin mouse; los modales atrapan el foco y se cierran con Escape.
Recetas administrables desde Ajustes	5.24 · 12.32	Ingredientes, rendimiento y preparación se editan sin tocar la base de datos.
Reportes que sobreviven un despliegue	12.15	El .xlsx generado sigue disponible después de un `git push` — se guarda en PostgreSQL, no en el disco efímero de Render.
Despliegue real en producción (Render)	DESPLIEGUE.md · 12.4	https://cuentavoz.onrender.com responde con datos reales: Static Site + Web Service + PostgreSQL.

Si el sábado en la noche las treinta y siete filas pasan, el agente cumple todo lo prometido en la propuesta: se construyó por fases probadas, cada conexión tiene su código y cada funcionalidad tiene su pantalla. Lo que queda para el domingo es solo contarlo en dos minutos.